c54x_exe

/opt/GSM/c54x_exe

Banc GSM émulé — banc-max

2026-10-28

Table des matières

1 Synthèse **2**

2 Resultats des tests **3**

2.1 /opt/GSM/c54x_exe/LAUNCH.md . . . 3
2.1.1 Launching the mobile side, process by process . . . 3
2.2 /opt/GSM/c54x_exe/MAILBOX.md . . . 5
2.2.1 ARM <-> DSP mailbox (Calypso API RAM) . . . 5
2.3 /opt/GSM/c54x_exe/README.md . . . 7
2.3.1 c54x_exe — the Calypso DSP, run as a native process . . . 7
2.4 /opt/GSM/c54x_exe/proof_toa_23.md . . . 10
2.4.1 OsmocomBB's "magic 23", proven from the ROM, the layer1 code and hardware logs . . . 10
2.5 /opt/GSM/c54x_exe/docs/CLAIMS-2026-10-04.md . . . 23
2.5.1 Emulating the baseband — claims and evidence (2026-10-04) . . . 23
2.6 /opt/GSM/c54x_exe/docs/carte-memoire.md . . . 27
2.6.1 Calypso DSP memory map - API RAM, NDB, PARAM, internal memory, ports . . . 27
2.7 /opt/GSM/c54x_exe/docs/emulating-the-baseband.md . . . 37
2.7.1 Emulating the baseband . . . 37
2.8 /opt/GSM/c54x_exe/tools/isa_tests.txt . . . 39
2.9 /opt/GSM/c54x_exe/tools/cch_ref/cch_deinterleave_inverse_ref.csv . . . 72
2.10 /opt/GSM/c54x_exe/tools/cch_ref/cch_interleave_ref.csv . . . 74
2.11 /opt/GSM/c54x_exe/tools/cch_ref/cch_testvectors.txt . . . 76

3 Fichiers **77**

3.1 /opt/GSM/c54x_exe/.gitignore . . . 77
3.2 /opt/GSM/c54x_exe/Makefile . . . 77
3.3 /opt/GSM/c54x_exe/balayage.sh . . . 79
3.4 /opt/GSM/c54x_exe/harvard.sh . . . 80
3.5 /opt/GSM/c54x_exe/install.sh . . . 95
3.6 /opt/GSM/c54x_exe/mobile_pont.cfg . . . 103
3.7 /opt/GSM/c54x_exe/repro_toa.sh . . . 106
3.8 /opt/GSM/c54x_exe/run.sh . . . 107
3.9 /opt/GSM/c54x_exe/run_real.sh . . . 123
3.10 /opt/GSM/c54x_exe/run_si.sh . . . 124
3.11 /opt/GSM/c54x_exe/sign.sh . . . 125
3.12 /opt/GSM/c54x_exe/src/carte_tx.h . . . 126
3.13 /opt/GSM/c54x_exe/src/cellule.c . . . 127
3.14 /opt/GSM/c54x_exe/src/cellule.h . . . 137
3.15 /opt/GSM/c54x_exe/src/main.c . . . 138
3.16 /opt/GSM/c54x_exe/src/montant.c . . . 142
3.17 /opt/GSM/c54x_exe/src/montant.h . . . 166
3.18 /opt/GSM/c54x_exe/src/pcb-minimal.c . . . 166
3.19 /opt/GSM/c54x_exe/src/pont.c . . . 167
3.20 /opt/GSM/c54x_exe/src/pont.h . . . 214
3.21 /opt/GSM/c54x_exe/src/rejouer.c . . . 214

3.22 /opt/GSM/c54x_exe/src/rejouer.h . . . 254
3.23 /opt/GSM/c54x_exe/src/tsp_tx.c . . . 254
3.24 /opt/GSM/c54x_exe/src/tsp_tx.h . . . 255
3.25 /opt/GSM/c54x_exe/src/verbosite.c . . . 256
3.26 /opt/GSM/c54x_exe/src/verbosite.h . . . 258
3.27 /opt/GSM/c54x_exe/tools/analyse_sb.py . . . 259
3.28 /opt/GSM/c54x_exe/tools/cmp_codeur.py . . . 259
3.29 /opt/GSM/c54x_exe/tools/comparer_parole.py . . . 260
3.30 /opt/GSM/c54x_exe/tools/decoder_add.py . . . 263
3.31 /opt/GSM/c54x_exe/tools/dsp_banc_commun.c . . . 264
3.32 /opt/GSM/c54x_exe/tools/dsp_banc_commun.h . . . 277
3.33 /opt/GSM/c54x_exe/tools/dsp_tester.c . . . 280
3.34 /opt/GSM/c54x_exe/tools/dsp_txt2bin.py . . . 293
3.35 /opt/GSM/c54x_exe/tools/injecteur_bruit.py . . . 295
3.36 /opt/GSM/c54x_exe/tools/isa_examples.py . . . 306
3.37 /opt/GSM/c54x_exe/tools/isa_test.c . . . 313
3.38 /opt/GSM/c54x_exe/tools/layer1_tester.c . . . 317
3.39 /opt/GSM/c54x_exe/tools/layer1_tester.py . . . 326
3.40 /opt/GSM/c54x_exe/tools/rejeu_banc.c . . . 333
3.41 /opt/GSM/c54x_exe/tools/sacch_tf_decode.c . . . 345
3.42 /opt/GSM/c54x_exe/tools/cch_ref/cch_interleave_ref.py . . . 347

4 Logs **349**
4.1 /opt/GSM/c54x_exe/toa.log . . . 349

1 Synthèse

run	élément	valeur
(pas de verdict.txt)		

dossier de run	contenu
Verdict et couverture	échelle des barreaux et tableau de couverture couche 1
Résultats des tests	captures VTY, diagnostics, rapports pytest, grafcets (.txt .md .mmd)
Fichiers	sources, scripts, configs et docs du banc (tout fichier texte)
Logs	journaux (.log), en dernier

i Méthode

Dossiers : /opt/GSM/c54x_exe. Extensions : tous les fichiers texte (hors .git, caches, binaires, sauvegardes, LICENSE et sorties précédentes). Entrée 1080 Ko, sortie 1079 Ko.
Compactage sans perte : fichiers identiques cités une fois ; lignes consécutives identiques « ×N » ; lignes ne différant que par des nombres : gabarit avec ⟨k⟩ puis la liste ordonnée des valeurs (k-uplets), chaque ligne se reconstruit en remettant les valeurs dans l'ordre. Seuls les codes couleur, la négociation telnet des captures VTY et les caractères de commande sont retirés ; les lignes de plus de 1000 caractères sont coupées en morceaux terminés par ↩. Les .md sont inclus tels quels (titres rétrogradés) et les .mmd rendus en diagrammes Mermaid, sans compactage.

2 Resultats des tests

2.1 /opt/GSM/c54x_exe/LAUNCH.md

9099 octets, 173 lignes

2.1.1 Launching the mobile side, process by process

Two set-ups, at most five processes, all on the **mobile** side. The network (osmo-bts-trx, BSC, MSC, HLR…) is never started here; the bridge waits for it if it exists.

Set-up	Command	What runs
dsp	`qosmo-dsp`	`c54x_exe --arm` -> `qosmo` (QEMU, `CALYPSO_DSP_EXTERN=1`) -> `osmocon` -> `mobile` -> `pont_dsp.py --dsp-port 6702`
grgsm	`qosmo-grgsm`	`qosmo` (QEMU, built-in gr-gsm layer 1) -> `osmocon` -> `mobile` -> `pont.py`

Without arguments, `qosmo-dsp` and `qosmo-grgsm` start everything in order, each step waiting for an observable condition of the previous one. Other options: `--status`, `--logs`, `--stop`, `--step N`. Logs and pids go to `/tmp/c54x-pont/`. Calls carrying QEMU options (`--qemu`, `-k`, `--bin`, `--cpu`, `--gdb`, `--rundir`, `--monitor`, `-dsp`) are forwarded to the C launchers `qosmo-dsp-launch` / `qosmo-grgsm-launch`.

Known limitation: the `/usr/local/bin/qosmo-dsp` and `qosmo-grgsm` wrappers are not in any repository; a reinstall overwrites them.

Each process can also be started alone, in this order.

2.1.1.1 1. `c54x_exe` - the DSP (dsp set-up only)

```
c54x_exe                      # = /opt/GSM/c54x_exe/c54x_exe --arm -v
c54x_exe --arm --iq cell      # + a synthetic GMSK cell (FCCH/SCH/dummy)
c54x_exe --trames 50          # standalone, no ARM: the mask ROM alone
```

- **Role**: the TI TMS320C54x mask ROM, run outside QEMU (`/opt/GSM/qosmo/hw/arm/calypso/l1-dsp/`).
- **Publishes**: `/dev/shm/calypso_api_ram` (the API window, alias of the C54x `data[0x0800..]`) and `/tmp/calypso_dsp.sock` (frame lockstep, protocol `calypso_dsp_pont.h`).
- **Listens**: UDP **6702**, downlink bursts for the BSP (`calypso_bsp.c`).
- **Expected**: `pont : en attente de l'ARM sur /tmp/calypso_dsp.sock`.
- **Check**: `ls -la /dev/shm/calypso_api_ram /tmp/calypso_dsp.sock`.
- **Options**: `--insns N` budget per frame (200000 by default with `--arm`, 80000 through `run.sh`), `--iq fcch|cell|tone:x|noise`, `--amp N`, `-v` to `-vvvvvv` for core traces (pure probes are off by default; `CALYPSO_SONDES=1` or `-vvvv` turn them on).
- **Environment**: `CALYPSO_BSP_ATTENTE_MS=40` (set by `run.sh`), `PONT_TCH_DEPOT_IDLE=0` (deposit the TCH burst before IDLE), `PONT_DONE_TOT=0` (return DONE at the end of the frame). Every 1000 frames `dsp.log` prints `[chrono] ... qemu | A | go | B | apres DONE`. Probes: `/dev/shm/calypso_bsp_dedie` (BSP dedicated store: played/missed/lost); on a dedicated channel, `/dev/shm/calypso_rejeu_tch.bin` is recorded for `tools/rejeu_banc` (`CALYPSO_REJEU_ENREG=0` turns it off).

- **Uplink**: publishes `/dev/shm/calypso_rach`, `calypso_sdcch_ul`, `calypso_tch_facch_ul`, `calypso_tch_sacch_ul`, `calypso_tch_ul` by scanning the API RAM (`src/montant.c`); this is how the mobile's RACH reaches pont.py and the BTS. `MONTANT=0` disables it, `MONTANT_DEBUG=N` sets the trace level, `MONTANT_CONSOMME_RACH=1` makes triggering exact, `MONTANT_KC=0` (Kc in `calypso_kc_l1`), `MONTANT_PAROLE_TI=0` (raw uplink speech, no TI -> FR conversion), `MONTANT_TCH_TACHE=0` (switch to TCH on the bridge's announcement instead of the firmware task), `MONTANT_AFD=0` / `MONTANT_ADD=0` (`[a_fd]` / `[a_dd]` probes). **Check**: `./c54x_exe` prints `[montant] RACH ra=0x.. bsic=..` and `pont.log` goes from `rach=0` to `rach=N`. During a call: `[montant] TCH : le firmware poste la tache 13 a fn=..., BSP bascule sur TS2`, then `le firmware est revenu sur le SDCCH (tache ALLC)` on release.

2.1.1.2 2. `qosmo` - the ARM and the osmocom-bb layer 1

```
# dsp set-up
CALYPSO_DSP_EXTERN=1 /opt/GSM/qosmo/build/qemu-system-arm -M calypso -cpu arm946 \
  -display none -parallel none -serial pty -serial pty \
  -monitor unix:/tmp/qemu-monitor-pont.sock,server,nowait \
  -kernel /opt/GSM/firmware/board/compal_e88/layer1.highram.elf
# grgsm set-up: the same line without CALYPSO_DSP_EXTERN
```

- **Role**: the Calypso (ARM946) running `layer1.highram.elf`. `-kernel` is required: without it the CPU starts at 0 and crashes at `0x840000`, and osmocon's romload loads nothing.
- **With `CALYPSO_DSP_EXTERN=1`**: the `0xFFD00000` API window is the shared segment, the gr-gsm L1 is disabled before it opens its ports, the `CNTL_RST` register relays `RESET_DSP` to the DSP, and a boot timer clocks the DSP until the TPU starts.
- **Without**: the gr-gsm L1 listens on UDP **4730** (GSMTAP) and **4731** (SCH), fed by the bridge.
- **Expected** (stderr):

  ```
  char device redirected to /dev/pts/N (label serial0)       <- modem pty
  [trx] pont DSP : API RAM partagee ... + socket /tmp/calypso_dsp.sock      (dsp)
  [trx] pont DSP : RESET_DSP relache par le firmware -> PONT_RESET          (dsp)
  [trx] pont DSP : le TDMA du firmware prend le relais du timer de boot     (dsp)
  [l1] backend gr-gsm : GSMTAP udp/4730, SCH udp/4731                       (grgsm)
  ```

  and on the `c54x_exe` side: `RESET #1 ... pc=0xff80` then `DSP boote (premier IDLE)`.
- **Check**: `printf 'xp /96bx 0x008305f0\n' | nc -U /tmp/qemu-monitor-pont.sock` shows the firmware's last printf (`DSP API Version: 0x4e2a 0x491a`). `run.sh` writes the pty to `/tmp/c54x-pont/modem.pty`.
- **Added by `run.sh`**: `-gdb tcp:127.0.0.1:1234` and the `telnet 0 44444` console (`tools/gdb-telnet.py` of qosmo; Ctrl-C stops the ARM, `continue &` resumes it; `GDB=0` disables it); `CALYPSO_PONT_RETRY_DIV=64` and, with `LOCKSTEP=1` (default), `CALYPSO_PONT_LOCKSTEP=1`; `ASSEMBLY_LOGS=1` writes `qemu-asm.log`. `qemu.log` goes to `/run/user/0/osmo-nitb/logs/qemu.log`, linked from `/tmp/c54x-pont/`.

2.1.1.3 3. `osmocon` - loader and L1CTL relay

```
osmocon                    # = osmocon -m romload -i 100 -p $(cat /tmp/c54x-pont/modem.pty) \
                           #      -s /tmp/osmocom_l2 layer1.highram.bin
```

- **Role**: speaks the romload protocol with QEMU's UART stub, then relays L1CTL between the firmware (sercomm on the pty) and the `/tmp/osmocom_l2` socket. Shows the firmware console (`FB0 (fn:att): TOA=... Power=... Angle=...`).
- **Expected**: `Received ident ack`, `Progress: 100%`, **`Received branch ack,  your code is running now!`**.
- **Pitfall**: killing osmocon during the download leaves the UART romload stub mid-block; restart QEMU (step 2), then osmocon.

2.1.1.4 4. `mobile` - layers 2/3

```
mobile -c /opt/GSM/c54x_exe/mobile_pont.cfg     # layer2-socket /tmp/osmocom_l2, VTY 127.0.0.1:4247
```

- **Config**: `layer2-socket /tmp/osmocom_l2`, `sap-socket /tmp/osmocom_sap`; `tch-voice` gapk `io-tch-format ti`, ALSA `gsm_out/gsm_in`; `tch-data` unix-sock `/tmp/ms_data` (CSD 9600). `run.sh` sets `L23_SYNC_RETRIES_SELECTION=8` with `MODE=dsp` (default 1 elsewhere).
- **Expected** within 5 s: on osmocon `L1CTL_PM_REQ`, `L1CTL_RESET_REQ: FULL!`, `L1CTL_FBSB_REQ (arfcn=514 ...)`; on the DSP the step from 428 to ~570 insn/frame and `a_sch=0100 ...` (the ARM L1 set `d_task_md=5`, FB search).
- **Without bursts** (no bridge or no BTS): `FBSB RESP: result=255` in a loop, expected.
- **Check**: `telnet 127.0.0.1 4247`, then `show ms`.

2.1.1.5 5. `grgsm_exe` - the TRX bridge (gr-gsm)

```
grgsm_exe                    # = python3 pont/pont_dsp.py --no-record --dsp-port 6702
PONT_DSP_PORT=0 grgsm_exe    # grgsm set-up: pont/pont.py, towards QEMU's L1 only
```

- **Role**: receives the BTS bursts as TRXD (UDP **5700-5702** from osmo-bts-trx), decodes them with gr-gsm and feeds the mobile's layer 1; sends the uplink back to the BTS.
- **Towards the gr-gsm L1** (grgsm set-up): L2 blocks as GSMTAP on 4730, sync FN on 4731.
- **Towards the DSP** (dsp set-up, `--dsp-port 6702`): each DL burst is repacked in the BSP format, 8 bytes `[tn, fn BE32, att, 0, 0]` + 148 bits 0/1; the BSP turns them into I/Q in DARAM `0x2a00`, where the DSP reads them.
- **Through `run.sh`** (`PONT=1`, default 0): the bridge runs **without** `--no-record` (`PONT_AIRREC=1`, for the panel FFT).
- **Expected**: `pont TRX : ports 5700/5701/5702, ARFCN 514, BSIC 7`, then `STATS fn=... | DL bursts=N` lines. `DL bursts=0` while no BTS transmits.

2.1.1.6 Stop order and cleanup

`qosmo-dsp --stop` (or `run.sh --stop`) stops in reverse order (bridge, mobile, osmocon, gdb, QEMU, DSP) and removes `/dev/shm/calypso_api_ram`, `/tmp/calypso_dsp.sock`, `/tmp/osmocom_l2`, `/tmp/qemu-monitor-pont.sock`, `/tmp/osmocom_sap`, `/tmp/ms_data`, `/dev/shm/calypso_horloge`, the uplink side-bands (`calypso_rach`, `calypso_sdcch_ul`, `calypso_tch_*_ul`) and, in the dsp set-up, `calypso_tch_cfg`. The session is archived in `/tmp/c54x-pont/archives/<date>/` (`JOURNAUX_GARDES=10`).

2.1.1.7 Known issues

- B_BFI is set on every downlink speech frame although speech is intelligible.
- Occasional LOS during a TCH call (SACCH/TF FIRE check fails on every block).
- SDCCH/8: part of the downlink frames (mostly SACCH) are dropped.
- SB synchronisation succeeds only once every three to five attempts.
- Real-time margin in TCH: 4.3 to 4.6 ms of DSP work per 4.62 ms frame (`[chrono]`).

2.2 /opt/GSM/c54x_exe/MAILBOX.md

4177 octets, 100 lignes

2.2.1 ARM <-> DSP mailbox (Calypso API RAM)

Two views of the SAME memory: the ARM addresses it in bytes from 0xFFD00000, the DSP in words from 0x0800. Conversion: `dsp_word = 0x0800 + arm_byte/2`.

Sources: `qosmo/include/hw/arm/calypso/calypso_api.h` and `osmocom-bb/src/target/firmware/include/calypso/dsp_api.h` (they agree: `#define DSP 36`, a_serv_demod/a_pm/a_sch branch).

2.2.1.1 Overall map

```
area          ARM (bytes)    DSP (words)  size
W page 0      0xFFD00000     0x0800       20 words  ARM -> DSP
W page 1      0xFFD00028     0x0814       20 words
R page 0      0xFFD00050     0x0828       20 words  DSP -> ARM
R page 1      0xFFD00078     0x083C       20 words
NDB           0xFFD001A8     0x08D4      268 words  shared, not paged
```

2.2.1.2 W page (ARM -> DSP), word offsets

```
+0  d_task_d
+1  d_burst_d
+2  d_task_u
+3  d_burst_u
+4  d_task_md     0x0804 / 0x0818   task: 5=FB 6=SB 8/9=TCH
+8  d_fn          0x0808 / 0x081C
+15 d_afc         0x080F / 0x0823   AFC DAC, relayed to the TWL3025
+16 d_ctrl_system 0x0810 / 0x0824
```

2.2.1.3 R page (DSP -> ARM), word offsets

```
+0   d_task_d
+8   a_serv_demod[4]  0x0830 / 0x0844   TOA, PM, ANGLE, SNR
+12  a_pm[3]          0x0834 / 0x0848
+15  a_sch[5]         0x0837 / 0x084B   SB header + payload
     a_sch[0] status: bit15 B_BLUD (block present), bit8 B_SCH_CRC (1 = ERROR)
     a_sch[3..4]: SB word, sb = a_sch[3] | a_sch[4]<<16
                  BSIC = (sb>>2) & 0x3f
```

2.2.1.4 NDB (shared)

```
+0    d_dsp_page      0x08D4   0=reset, 2=armed page 0, 3=armed page 1
+14   d_dsp_state     0x08E2
+36   d_fb_det        0x08F8   non-zero = FCCH found
+37   d_fb_mode       0x08F9   0=wide search, 1=narrow
+38   a_sync_demod[4] 0x08FA   0x08FA TOA, 0x08FB PM, 0x08FC ANGLE, 0x08FD SNR
```

2.2.1.5 Who writes what - ARM side (osmocom-bb source)

```
dsp_end_scenario()   calypso/dsp.c
    d_dsp_page = B_GSM_TASK | w_page; then w_page ^= 1
l1s_reset_hw()       layer1/sync.c
    d_dsp_page = 0; r_page = 0; db_r -> R page 0
    called from prim_fbsb.c on every failed FBSB cycle
l1s_dsp_post()       layer1/sync.c
    memset of the R page, then a_sch[0] = (1<<B_SCH_CRC) = 0x0100,
    then r_page ^= 1
-> the ARM only ever writes a_sch[0] with 0x0100.
```

2.2.1.6 Who writes what - DSP side (mask ROM, exact PCs)

```
0xb446   a_sch[0..4] <- 0 on BOTH pages             initialisation
0xaba2   a_sch[0] <- 0x1111                         marker
         preceded by 0xaba0: LD #0x1111, A
         encoding 80e2 000f = store *(AR2 + 0x0f), AR2 = R page base
0xb214   a_sch[0..4] <- status + payload            RESULT
0xb2cc   d_fb_det <- 0        FB task entry: reset
0xb2cf   a_sync[TOA] <- 0
0xb2d2   a_sync[PM] <- 0
0xb2d5   a_sync[ANGLE] <- 0
0xb2d8   a_sync[SNR] <- 0
0xb2c4   *(0x3fb5) <- 0x0cce   FB correlator input buffer pointer
0xb2c9   *(0x3fb5) <- 0x0d2e   same, second buffer
0x795a   a_sync[TOA] <- value
0x798b   a_sync[ANGLE] <- value
0x798d   a_sync[SNR] <- 0x4000
0x79d4   a_sync[SNR] <- value
0x79de   a_sync[PM] <- value
```

2.2.1.7 SB task sequence

```
ARM: d_task_md = 6 on the current W page
ARM: d_dsp_page = 2|w_page       (dsp_end_scenario)
DSP: 0xb446 clears a_sch
DSP: 0xaba2 writes the 0x1111 marker
DSP: 0xb214 writes the result into a_sch
ARM: reads db_r->a_sch; B_SCH_CRC=1 means "SB not found"
ARM: memset R page, a_sch[0] = 0x0100, r_page ^= 1
ARM: after 2 attempts, L1CTL_RESET_REQ -> l1s_reset_hw -> d_dsp_page = 0
```

2.2.1.8 Pitfalls when probing

- The emulator has two write paths, `data_write()` and `data_write_locked()` (`c54x_mem.c`); a probe on only one misses half the writes (a_sch goes through the second, a_sync_demod through the first).
- The ARM and the DSP are TWO PROCESSES sharing this mapping. A probe comparing memory before/after a DSP instruction can blame the DSP for an ARM write: record the address passed to the write function, never a delta.
- d_dsp_page announces the task's WRITE page. The READ page is `dsp_api.r_page`, a separate ARM-side counter; the two are not linked.

2.3 /opt/GSM/c54x_exe/README.md

9737 octets, 174 lignes

2.3.1 c54x_exe — the Calypso DSP, run as a native process

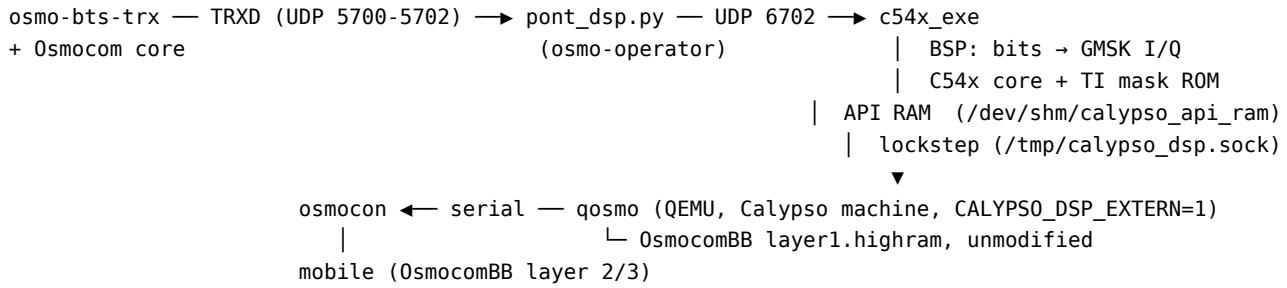
`c54x_exe` runs the **original TI mask ROM of the TMS320C54x DSP** found in Calypso GSM basebands (the chips behind OsmocomBB's Motorola C1xx targets), on a C54x core emulator, as a plain Linux process. It can run alone, replaying recorded bursts in milliseconds, or serve as the DSP of a full phone: the unmodified OsmocomBB layer 1 firmware runs under QEMU and talks to this process through the real shared API RAM, frame by frame, exactly as the ARM talks to the DSP on silicon. No radio, no SDR, no license anywhere in the chain.

What works: cell selection, location update, SMS in both directions, mobile-originated and mobile-terminated voice calls with A5/1, speech audible both ways. All downlink channel decoding is done by the ROM, and the uplink SDCCH/SACCH bursts sent to the BTS are the ROM's own coded bursts (bit-exact with GSM 05.03). The bench's coverage step prints, per signal-processing step, whether the run did it in the ROM or on the host (`/opt/GSM/tests/modules/99-couverture.sh`).

2.3.1.1 Known issues

- The ROM flags every downlink speech frame as bad (BFI) although speech is intelligible.
- SACCH/8 downlink often fails its FIRE check.
- The SB window is armed rarely enough that synchronisation takes a few tries.
- Real-time margin in TCH is thin (about 4.2 to 4.6 ms of DSP work per 4.62 ms frame).

2.3.1.2 Architecture



- **Downlink**: bursts leave the BTS as bits, the bridge forwards them, the BSP turns them into GMSK samples, and the ROM does what it does on a phone: FCCH/SCH detection, BCCH, TCH/F decoding, A5.
- **ARM ↔ DSP**: the ROM's API RAM window is memory-mapped and shared with QEMU. Both sides advance one TDMA frame at a time over a Unix socket; the per-frame sequence mirrors `calypso_tdma_tick()` of the in-QEMU DSP, so any divergence is localised (`src/pont.c`).
- **Uplink**: the ARM posts its blocks in the API RAM; `src/montant.c` scans it once per frame and publishes RACH, SDCCH, SACCH, FACCH and speech (TI format → FR) to `/dev/shm`. **SDCCH and SACCH-on-SDCCH are channel-coded by the ROM**: `src/pont.c` (`tx_rom_publier`) picks up each burst the ROM emits in `data[0x3f8a]` and publishes the 4 bursts of a block to `/dev/shm/calypso_xcch_ul_rom`; the bridge sends them (after checking they decode to the block, undoing the ROM's A5 XOR and re-ciphering at the air frame number). RACH, FACCH, SACCH on TCH, uplink speech and uplink A5 are still done by the bridge. `MONTANT_ROM_UL=0` (here) or `PONT_UL_ROM=0` (bridge) go back to host coding.
- Every deliberate departure from silicon behaviour is an environment variable, listed at startup (`hacks_actifs()` in `src/pont.c`).

2.3.1.3 Building

The C54x core, the Calypso peripherals and the DSP glue live in qosmo (`hw/arm/calypso/l1-dsp/`) and are compiled from there — nothing is copied into this tree, on purpose.

```
# needs: qosmo checked out (default /opt/GSM/qosmo), libosmocore + libosmocoding, gcc
make                      # QOSMO=/path/to/qosmo make
./c54x_exe --trames 200   # the ROM alone, no ARM: does it boot, what does it write?
./c54x_exe --help
```

The ROM images (`rom/calypso_dsp.*.bin`) are dumps of the TI mask ROM at their silicon addresses (`src/main.c`, `ROMS[]`).

2.3.1.4 Running a phone

```
./run.sh              # DSP + QEMU + osmocon + mobile, in order; logs in /tmp/c54x-pont/
PONT=1 ./run.sh       # + the TRX bridge, against a running osmo-bts-trx / core network
./run.sh --status | --logs | --stop
```

Process-by-process launch, expected log lines and checks: `LAUNCH.md`. The bridge and the network side are in osmo-operator.

2.3.1.5 Benches and tools

what	where
Replay the ROM alone on a recorded FB/SB acquisition, deterministic, no QEMU	`./c54x_exe --rejouer`, `src/rejouer.c`
Replay a recorded dedicated-channel session (TCH) through the ROM	`tools/rejeu_banc.c`
C54x instruction-level tests (the bugs they caught are listed in the source)	`tools/isa_test.c`, `tools/isa_tests.txt`
Bit-by-bit comparison: what the ROM decodes vs. libosmocoding on the same bursts	`tools/comparer_parole.py`
CCH interleaving reference vectors	`tools/cch_ref/`
Trace levels of the core, `-v` … `-vvvvvv`	`src/verbosite.c`

2.3.1.6 Repository layout

```
src/        main.c (ROM loading, CLI), pont.c (ARM/DSP bridge, per-frame sequence),
            montant.c (uplink from API RAM), cellule.c (synthetic FCCH/SCH/BCCH),
            rejouer.c (replay), verbosite.c (trace levels)
rom/        TI mask ROM dumps
tools/      benches, test vectors, analysis scripts
docs/       carte-memoire.md (DSP memory map), emulating-the-baseband.md,
            CLAIMS-2026-10-04.md (signed claims snapshot)
LAUNCH.md   process-by-process launch guide
MAILBOX.md  ARM <-> DSP API RAM map and who writes what
```

2.3.1.7 License

GPL-2.0-or-later, see `LICENSE`.

2.3.1.8 ROM and uplink probes

The DSP ROM is **not** in this repository. `rom/fetch-rom.sh` downloads the FreeCalypso dump (version 3606), converts it with `tools/dsp_txt2bin.py` into `calypso_dsp.{DROM,PDROM,PROM0..3}.bin` and checks `rom/SHA256SUMS.3606` (`--dest DIR` repeatable, `--version 3311` for the D-Sample dump). `calypso_dsp.Registers.bin` is a register snapshot, not ROM: it stays in the repository.

Two inert-by-default probes in `src/pont.c` (environment variables, `--arm` mode):

- `PONT_TX_SONDE=N` — for N frames where the ARM has posted `d_task_u`/`d_task_ra`, log which DSP data words the ROM wrote and dump the changed ranges to `/tmp/tx-sonde/`.
- `PONT_TX_INJECT=FILE` — after `montant.c` has published a real uplink block (so the link is untouched), overwrite `a_cu` with the next 23-byte block of FILE (one hex line each); the ROM output for each block lands in `/tmp/tx-sonde/inj_<n>.txt`. `PONT_TX_TRACE_BLOC=N` single-steps the ROM on injected block N and logs every write to the TX buffers with its PC (`/tmp/c54x-pont/trace-tx.txt`).
- `PONT_TX_OMBRE=1` — compares each real SDCCH block coded by the ROM with `gsm0503_xcch_encode` (`/tmp/tx-sonde/ombre.txt`, `reel.txt`).

Uplink coding (`/opt/GSM/tests/rom-tx/README.md`): the ROM writes the interleaved block in `0x4280` (4 bursts × 8 words, circular, BK=32); FIRE parity is computed at PROM0 `0x9013`; at each transmit frame `0x890e-0x891b` copies the current burst to `data[0x3f8a..0x3f91]` (114 bits + hl/hu, MSB first), advances the pointer `data[0x3d9b]` and clears word 7 of that burst in `0x4280`; in ciphered mode the A5 keystream (`0x3f9b..`, `calypso_a5.c`) is XORed onto `0x3f8a` itself. The output matches GSM 05.03 on all 456 bits. Core options that restore earlier behaviour for comparison: `CALYPSO_LOGIC40`, `CALYPSO_LD16_SEXT`, `CALYPSO_F0BX_SBIT`.

`C54X_BIN=/path/to/c54x_exe ./run.sh` runs another build of the executable.

2.3.1.9 Noise injection (downlink)

Off by default. Two independent ways to degrade what the DSP receives, both driven from the environment of `run.sh` (and therefore of `osmo-operator/start-native.sh --dsp`):

- `BRUIT_SNR_DB=<dB>` alone: AWGN added by the C54x core itself to every downlink I/Q sample (`c54x_bsp_load()` in qosmo `calypso_c54x.c`); `run.sh` passes it to `c54x_exe` as `CALYPSO_BSP_SNR_DB` (and `BRUIT_GRAINE` as `CALYPSO_BSP_BRUIT_GRAINE`).
- `BRUIT_MODE=ber|souple|iq|relais`: `run.sh` inserts the UDP proxy `/opt/GSM/qosmo/tools/injecteur_bruit.py` (fallback: `tools/injecteur_bruit.py`, a link to it) as step **b**: `c54x_exe` listens on `BRUIT_PORT_DSP` (16702, via `CALYPSO_BSP_PORT`), the proxy takes `127.0.0.1:6702`, degrades the bridge's 8-byte-header + 148 hard-bit bursts and forwards them; anything the BSP sends back is relayed unchanged to the bridge. `ber` flips bits (`BRUIT_BER`, bursty with `BRUIT_RAFALES=<mean length in bits>`), `souple` adds AWGN at `BRUIT_SNR_DB` then hard-decides (the stream carries no soft values), `iq` (experimental, numpy, refused under `CALYPSO_BSP_STREAM=1`) sends GMSK I/Q + AWGN through the BSP's IQ_PASSTHROUGH path, `relais` changes nothing. `BRUIT_PERTE` erases bursts, `BRUIT_TN` restricts timeslots, `BRUIT_GRAINE` makes errors reproducible per (fn, tn). In `MODE=grgsm` the proxy sits between `pont.py` (`PONT_GSMTAP_PORT=14730`) and QEMU's GSMTAP port 4730 (`ber`/`relais` only).

```
BRUIT_MODE=ber BRUIT_BER=0.01 PONT=1 ./run.sh       # 1 % bit errors on every downlink burst
BRUIT_SNR_DB=12 PONT=1 ./run.sh                     # core AWGN at 12 dB, no proxy
./run.sh --step b                                   # the proxy alone; --status shows its last STATS line
```

Log: `$RUNDIR/bruit.log` (periodic `STATS`, final `BILAN`); `./run.sh --stop` stops it. A rejected configuration stops `run.sh` before anything is launched.

2.4 /opt/GSM/c54x_exe/proof_toa_23.md

40376 octets, 717 lignes

2.4.1 OsmocomBB's "magic 23", proven from the ROM, the layer1 code and hardware logs

The FIXME: `prim_fbsb.c:333` and `:469`, `/* FIXME: where did this magic 23 come from? */`, followed by `last_fb->toa -= 23;`.

Claim. The DSP returns a position inside a 191-sample window hard-wired by TI; 23 is the only margin that, in the `tpu_window.c` formula, produces that window; `toa - 23` turns the position into an offset from an SB sitting at its nominal place, and the `+ 75` qbits of `synchronize_tdma` then move the burst to the centre of the NB window, where the silicon does return the 16-qbit residual that `toa.c` expects.

2.4.1.1 Rules of evidence and sources

The evidence rests on three sources:

source	what it is	where
ROM	TI Calypso DSP mask-ROM, version 3606, disassembled (binutils `tic54x`)	public dump: `ftp://ftp.freecalypso.org/pub/GSM/Calypso/dsp` (Pirelli DP-L10, Calypso PD751992AZHH, OsmocomBB dump tool). The PROM0 (0x7000-0xdfff, 28,672 words) of that dump is **identical word for word** to the PROM0 used here (0 differences out of 28,672).
layer1	OsmocomBB firmware sources, `src/target/firmware/layer1/{prim_fbsb.c,tpu_window.c,sync.c,toa.c}` and their git history	`osmocom-bb` repository

source	what it is	where
hardware logs	firmware output from real Calypso phones, posted to the baseband-devel list	[1] Aegean Chou, 2011-09-26, *Re: About sniff multi bursts in a frame, CCCH_CONF* (DSP API 0x3606, firmware without TX): https://lists.osmocom.org/pipermail/baseband-devel/2011-September/002526.html — [2] nish079858, 2024-10-02, *Osmocom bb mobile station cannot register to YateBTS basestation* (Motorola C115, `layer1.highram.bin` e88): https://www.mail-archive.com/baseband-devel@lists.osmocom.org/msg01046.html

The emulation bench (`c54x_exe`, `dsp_tester`) is **not** a source of evidence: it served to find out where to look (§7), and every time it is cited this is stated.

Levels: **PROVEN** (static reading of the ROM or of the code, or a hardware log landing on a numerical prediction), **VERY LIKELY**, **HYPOTHESIS**.

2.4.1.1.1 The result on one page

1. The DSP does not return a timing error. Its TOA is the **index of the lag it retained** in the correlation with the training sequence, the leading edge of its channel estimate (ROM, §2). A burst whose first sample is at position k of the window yields TOA = k + 3 (correlation starting at sample 39, training sequence at bit 42), or less if an earlier lag of the 7-lag window reaches the threshold (§2.4).
2. The receive windows are **immediate constants in the ROM**: 191 samples for the SB, 151 for normal bursts (§1). `tpu_window.c` computes its own as `burst + 2·margin − 3` with `L1_SB_MARGIN_Q = 23×4` and `L1_NB_MARGIN_Q = 3×4`: $\mathbf{148 + 2 \cdot 23 - 3 = 191}$ and $\mathbf{148 + 2 \cdot 3 - 3 = 151}$. Conversely (191 − 148 + 3)/2 = 23: 23 is the only margin compatible with TI's window (§4.1).
3. SB path: `TOA − 23`, then `+ 75` qbits, a net shift of `TOA_sb − 4.25` bits: the burst is placed at a TOA of **4.25**, the centre of the NB window (3 samples of play, centre 1.5, plus the ROM's offset of 3), which `toa.c` then holds at 16 qbits (§4.3). FB path: 23 subtracted **twice** since `cb71b972` (2010-05-20), net shift `TOA_fb − 27.25`, which makes the SB arrive at a TOA of 27.25 (§4.4).
4. The hardware logs land on these numbers with no free parameter (§5): printed qbits = ((TOA − 46) mod 1250)×4 (12 lines out of 12: 6 after an FB1, 6 after an SB); every first SB after an FB synchronisation, in both logs (n = 6): 29, 29, 27, 27, 24, 26 for a prediction of 27.25 (mean deviation −0.25, within the FB's granularity of 4); after SB synchronisation, every NB TOA deviation that `toa.c` prints is within 4 qbits of 16; FB TOA mode 1 3 (mod 4) (10/10) and mode 0 0 (mod 48) (7/7), as the ROM's immediates say.
5. **23 is the margin of the SB window**, the firmware's and TI's. The FIXME persists because 23 (`prim_fbsb.c`), 75 (`sync.c`) and 16 qbits (`toa.c`) only make sense together, and because the 191-sample window is written nowhere in the firmware.

2.4.1.2 1. ROM: the receive windows are immediate constants — PROVEN

Common DMA programming routine (PROM0, binutils disassembly):

```
a5cd  ld     #134,dp
a5e3  ldm    ar2,a               ; A = API address of the page (0x0cce)
a5e4  sub    #2048,a
a5e6  stl    a,1,DP+0x56         ; (A - 0x800) << 1
a5e8  portw  DP+0x56,pa64548     ; port 0xfc24: address (DMA2_AAD)
a5ea  ldm    ar3,a               ; A = AR3 = length in samples
a5ed  stl    a,2,DP+0x56         ; A << 2 = length in bytes (4 per I/Q sample)
a5ef  portw  DP+0x56,pa64550     ; port 0xfc26: length (DMA2_ALGTH)
```

```
a5f1  st     #1197,DP+0x56
a5f3  xc     2,beq
a5f4  st     #1193,DP+0x56
a5f6  portw  DP+0x56,pa64552      ; port 0xfc28: control (DMA2_CTRL)
```

The three task routines load AR3 with an immediate and then call this code through the ROM's dispatcher (`ld #41,a ; call 0xa9ea`: code 41 designates routine 0xa5cd; in emulation, AR3 does hold 191 on entry to 0xa5cd during an SB task):

task	PC	instruction	window
SB	0xb21d	`stm #191,ar3`	191 samples (764 bytes)
NB	0xb2b1	`stm #151,ar3`	151 samples (604 bytes)
FB	0xb32d	`stm #48,ar3`	48 samples per page (192 bytes), continuous stream

No word written by the ARM is involved: the window lengths are fixed by TI in the ROM. (The identification of ports 0xfc24/26/28 with the AAD/ALGTH/CTRL registers of the RHEA controller comes from the Calypso register map; the next point depends only on the immediates 191 and 151.)

2.4.1.3 2. ROM: the SB TOA is the index of the retained lag (leading edge of the channel estimate, §2.4) — PROVEN

Complete chain, static reading of PROM0.

2.1 The DMA page is copied without offset. `0x7c3d-0x7c47`: `stm #3278,ar2` (API page 0x0cce), `stm #10752,ar3` (0x2a00), `stm #10944,ar4` (0x2ac0), `stm #3660,ar5` (0x0e4c: I/Q DC component computed at `0x81a5-0x81c7` over the 191 samples), `calld 0x81c8` with `stm #47,brc` in the delay slot (48 iterations of 4 samples = 192 191). The loop `0x81cc-0x81db` reads the page in order (`*ar2+`), subtracts the DC component and stores sample n at **index n** of both buffers, applying the MSK derotation $(-j)^{\wedge}(n+1)$ through the signs and the two destinations: buffer 0x2a00 ← Q0, −I1, −Q2, I3, …; buffer 0x2ac0 ← −I0, −Q1, I2, Q3, …

2.2 The correlator looks for the extended training sequence. `0x7c49-0x7c58`: 64 words read from program memory starting at `0x7a1c` (`ld #31260,a ; rpt #25 ; reada *ar2+` ×2, `rpt #11 ; reada`) into 0x2cea. These 64 words are ±512 and their sign is, bit for bit, the **SB extended training sequence** (GSM 05.02 §5.2.5, 64 bits), with 1 → −512 and 0 → +512:

```
ETS            1011100101100010000001000000111100101101010001010111011000011011
sign(coeff)    0100011010011101111110111111100001101001010111010100010011110010   (= bitwise complement)
```

2.3 The correlation starts at sample 39. `0x84a1-0x84c6`: `stm #10791,ar3` (0x2a27 = 0x2a00 + **39**), `stm #10983,ar4` (0x2ae7 = 0x2ac0 + 39), `stm #49,brc` (50 lags), `stm #-63,ar0`; for each lag i, 64 MACs of coefficients × buffer[39 + i + j] on each component (`sth a,*ar1+` to 0x2c56 + i, `sth a,*ar5+` to 0x2c88 + i), then `add *ar3+,*ar4+,a` which advances by one sample. `0x84ca-0x84d8`: energy $I^2 + Q^2$ of lag i at 0x2be4 + 2i (`dst a,*ar4+`).

2.4 Choice of lag: leading edge of the channel estimate. Two blocks, static reading.

7-lag window. `0x84dd-0x84ef`: A = 0, B = 0, AR0 = AR1 = 0; for i = 0..42: `dadd *ar3+,a` (A += E[i+7]), `dsub *ar2+,a` (A −= E[i]), `max b`, `mar *ar0+`, `xc 1,nc ; mvmm ar0,ar1`. After iteration i, A = W[i+1] − W[0] where W[s] = E[s] + … + E[s+6]; AR1 receives i+1 whenever A exceeds the current maximum. Hence **s = argmax W[s]** (s = 0 by default if W[0] dominates), stored at 0x2f06.

Threshold. `0x7c5f-0x7c80`: energy of the 64 samples starting at 0x2a2a + s (`squra` ×64, `sfta a,-6`: average per sample), then × 1638/65536 (`stm #1638,t ; mpyu`) → DP+0x00 (0x2f00): **2.5 % of the mean received energy per sample**.

First significant lag. `0x7c8f-0x7c95`: the 7 + 7 channel coefficients (0x2cce..) are zeroed. `0x7ca6`: `rsbx tc`. `0x7ca7-0x7cae`: AR6 = AR7 = 0x2be4 + 2s. `0x7ca9-0x7cbb`, 7 iterations:

```
7caf  dld    DP+0x00,a          ; A = threshold
7cb0  dsub   *ar6,a             ; A = threshold − E[lag]
7cb1  bc     0x7cb7,agt         ; E < threshold: lag ignored
7cb3  ssbx   st0,tc             ; E ≥ threshold: TC = 1 (and what follows)
7cb4  dadd   *ar6,b
7cb5  mvdd   *ar2,*ar4          ; channel coefficient I ← correlation I of this lag
7cb6  mvdd   *ar3,*ar5          ;                      Q
7cb7  mvdd   *ar4+,*ar2+       ; copy of the coefficient (zero if ignored) into the correlation buffer
7cb8  mvdd   *ar5+,*ar3+
7cb9  dadd   *ar6+,a            ; A = threshold; next lag
7cba  xc     1,ntc              ; as long as no lag has reached the threshold:
7cbb  mvmm   ar6,ar7            ;     AR7 = pointer to the next lag
```

TC is never cleared inside the loop: AR7 stops on the **first lag of the window whose energy reaches the threshold**, and the lags before it are erased from the channel estimate. The TOA is therefore the leading edge of the estimated channel: equal to the peak k + 3 for a clean pulse, **k + 3 or less** otherwise (k + 2 as soon as the energy at k + 2 exceeds 2.5 % of the mean energy, widened pulse; k + 1 with a strong precursor, the rule taking the first lag threshold from s onwards). Then:

```
7cbc  ldm    ar7,a
7cbd  sub    #11236,a           ; 11236 = 0x2be4
7cbf  stl    a,-1,DP+0x1f       ; TOA = (AR7 − 0x2be4) / 2  →  0x2f1f
7e8a  mvdk   DP+0x1f,0x3fa4
b1e7  ld     *(0x3fa4),a
b1e9  stl    a,*ar0(0x8)        ; word 8 of the DB page = a_serv_demod[0] = D_TOA (dsp_api.h, DSP 36)
```

No constant is added: the TOA is the index of the retained lag. (Checked in emulation on three energy profiles read from table 0x2be4: the rule yields 23, 24 and 23 where the ROM published 23, 24 and 23, §7.)

2.5 Geometry. In the SB, the extended training sequence occupies bits 42 to 105 (3 tail bits, 39 data, 64 training, 39, 3). If the first sample of the burst is at index k of the DMA window, hence of the buffer (§2.1), the sequence starts at index k + 42, and the lag i that aligns it with the coefficients satisfies 39 + i = k + 42:

```
TOA_SB = k + 3   (peak), or less if an earlier lag reaches the threshold (leading edge, §2.4)
```

2.6 A false 23 not to be mistaken for the real one. If the energy of the retained channel coefficients is zero (`0x7ccd`, `bcd 0x7ced,aneq` not taken), the fallback `0x7ce5 mvmd ar0,0x2f1f` publishes the coarse index AR0 instead. In that case there is no channel estimate and the SB does not decode (bad CRC). Every TOA cited here comes with a correct CRC decode, hence from the main path (audit in §7).

2.4.1.4 3. ROM: the FB TOA — PROVEN for the form, partial for the detail

`0x7931-0x795a` (mode 1):

```
7931  rptb   0x793d              ; scan of the 32-bit energy buffer (*ar2, dld), tracking the maximum
793e  ld     *ar4+,b            ; B = c, index of the maximum
793f  sfta   b,2                ; B = 4·c
7940  ld     *(0x3fb4),16,a     ; A = N (page counter, 0x3fb4)
7942  sub    #3,16,a
7944  add    *ar4,16,a          ; + p (word following the index)
7945  sub    #2,16,a
7947  mpya   a                  ; × T = 48 (one page)
7948  add    a,b
7949  sub    #5,b               ; TOA = 48·(N + p − 5) + 4·c − 5
794b  sub    *(0xc3d),b,a ; rc aleq       ; lower and upper bounds …
795a  stl    b,*(0x8fa)         ; NDB a_sync_demod[D_TOA]
```

Immediates: 48 (DMA page), −5, step of 4 on c. In mode 1, **TOA 3 (mod 4)**, granularity 4 samples; 23 is reachable ($48 \cdot m + 23$: the logs give $10007 = 48 \cdot 208 + 23$). In mode 0 the logs give exact multiples of 48 (§5.2): the position is only known to the page, and 23 is not reachable there. The origin of N, p and the −5 has not been worked out.

2.4.1.5 4. layer1: what the firmware does with this TOA — PROVEN (reading of the code and of git)

2.4.1.5.1 4.1 The firmware's margins are the ROM's

`tpu_window.c:38-44`:

```
#define L1_NB_MARGIN_Q      (3 * 4)
#define L1_SB_MARGIN_Q      (23 * 4)
#define L1_TAIL_DURATION_Q  (3 * 4)
#define L1_NB_DURATION_Q    (L1_BURST_LENGTH_Q + 2 * L1_NB_MARGIN_Q - L1_TAIL_DURATION_Q)
#define L1_SB_DURATION_Q    (L1_BURST_LENGTH_Q + 2 * L1_SB_MARGIN_Q - L1_TAIL_DURATION_Q)
```

Same formula, with a 148-bit burst, as the immediates of §1:

```
SB: 148 + 2·23 − 3 = 191        NB: 148 + 2·3 − 3 = 151        and  (191 − 148 + 3) / 2 = 23
```

Two windows, two margins, one formula: the 151 constrains the formula as much as the 191 does; this is not a decomposition chosen after the fact. With `L1_BURST_LENGTH_Q = 625` qbits (one slot of 156.25 bits), the firmware opens its TPU windows 8.25 bits longer than the DSP's DMA, on both sides: the guard period.

2.4.1.5.2 4.2 The windows open at the same instant

On the ARM side, `l1s_rx_win_ctrl` (`tpu_window.c:100-110`): `start = DSP_SETUP_TIME` for every window type, only the duration changes. On the DSP side (§1), the SB and NB tasks arm the DMA through the same routine 0xa5cd, with the same page address (`stm #3278,ar2` at 0xb21b and 0xb2af), the same control word 0x04ad (ONE_SHOT; the FB gets 0x04a9, without ONE_SHOT), only the length differs: the ROM programs no task-specific offset. The start is therefore the same for both windows (the hardware trigger of the DMA has not been traced; §5.4 confirms it on silicon), and a burst has the same position k in the SB window (191) and in the NB window (151); only the room left around it changes.

2.4.1.5.3 4.3 SB path: TOA − 23, then + 75 qbits — target 4.25

`prim_fbsb.c:207-231` (`l1s_sbdet_resp`):

```
last_fb->toa -= 23;
qbits = last_fb->toa * 4;
…
cinfo->time_alignment = qbits;
synchronize_tdma(&l1s.serving_cell);
```

`sync.c:113-121` (`synchronize_tdma`):

```
uint32_t tpu_shift = cinfo->time_alignment;
/* NB detection only works if the TOA of the SB
 * is within 0...8. We have to add 75 to get an SB TOA of 4. */
tpu_shift += 75;
```

Time-base shift: (TOA_sb − 23)×4 + 75 qbits = **TOA_sb − 4.25 bits**. After this shift the burst sits at a TOA of 4.25 in any window opened at the same instant (§4.2), hence in the NB window. And `toa.c` (2011, `937023be`) then holds the average TOA of normal bursts at **16 qbits = 4 bits**, accepting only 0 TOA 31 qbits. In the 151-sample NB window a 148-sample burst is centred at k = 1.5, i.e. TOA = 4.5 (peak, §2.5) or 3.5 (leading edge): 4.25 is the centre of the NB window to within 0.25, which is also what the comment says ("within 0…8", "an SB TOA of 4"). And 75 qbits = 23 − 4.25 bits: the +75 is the difference between the firmware's nominal SB position and the centre of the NB window.

2.4.1.5.4 4.4 FB path: 23 is subtracted twice — net shift TOA_fb − 27.25

`prim_fbsb.c:465-493` (`l1s_fbdet_resp`, mode 1):

```
last_fb->toa -= 23;                     /* 1st subtraction, for ntdma and delay */
…
fbinfo2cellinfo(&l1s.serving_cell, last_fb);   /* 2nd subtraction, prim_fbsb.c:333 */
synchronize_tdma(&l1s.serving_cell);           /* + 75 */
tdma_schedule_set(delay, sb_sched_set, 0);
```

`time_alignment` is ((TOA_fb − 46) mod 1250)×4, and the net shift is **TOA_fb − 46 + 18.75 = TOA_fb − 27.25 bits**. The SCH follows the FCCH by exactly one frame, at the same position within the frame: in the SB window opened on the new time base it sits at 27.25 bits, plus any difference between the ROM's FB and SB measurement conventions, written Δ:

```
TOA_sb (first SB after FB synchronisation) = 27.25 + Δ
```

Git: the double subtraction dates from `cb71b972` (Welte, 2010-05-20, "Make new L1CTL_FBSB_REQ work reliably"), which introduces `fbinfo2cellinfo()` without removing the `toa -= 23` of the mode-1 block. In the initial import (`fbe7b94c`, 2010-02-18) the FB path subtracted 23 only once. `L1_SB_MARGIN_Q`, `L1_NB_MARGIN_Q`, both `toa -= 23` and `tpu_shift += 75` with its comment are all in the initial import.

2.4.1.6 5. Hardware logs: the predictions land — PROVEN

Relevant lines, verbatim, from log [1] (2011, four complete synchronisations) and log [2] (2024, one synchronisation):

```
[1] FB0 (11:6): TOA= 7200          FB1 (21:8): TOA= 9651         =>FB @ FNR 20 fn_offset=20 qbits=3420
    SB1 (46:1): TOA=   29          => SB 0x00c12184: BSIC=33 fn=89118(67/16/21) qbits=24
    =>FB @ FNR 45 fn_offset=89118 qbits=4932
    TOA AVG is not 16 qbits, correcting (got 20)
   FB0 (91861:3): TOA= 2544      FB1 (91871:8): TOA= 8751     =>FB @ FNR 91869 fn_offset=91869 qbits=4820
    SB1 (183745:1): TOA=   29      => SB 0x01e12284: BSIC=33 fn=91882(69/24/31) qbits=24
    =>FB @ FNR 183744 fn_offset=91882 qbits=4932
   FB0 (92514:8): TOA= 8784      FB1 (92524:8): TOA= 8755     =>FB @ FNR 92522 fn_offset=92522 qbits=4836
    SB1 (185051:1): TOA=   27      => SB 0x00852284: BSIC=33 fn=92535(69/ 1/21) qbits=16
    =>FB @ FNR 185050 fn_offset=92535 qbits=4924
    TOA AVG is not 16 qbits, correcting (got 19)
   FB0 (93574:9): TOA=10032      FB1 (93585:9): TOA=10003     =>FB @ FNR 93583 fn_offset=93583 qbits=4828
    SB1 (187172:1): TOA=   27      => SB 0x01582384: BSIC=33 fn=93596(70/22/11) qbits=16
    =>FB @ FNR 187171 fn_offset=93596 qbits=4924
    FB0 (94186:1): TOA=   48       FB1 (94197:9): TOA=10007
    FB1 (94217:11): TOA=12507      FB1 (94237:11): TOA=12507     FB1 (94248:2): TOA= 1259
[2] FB0 (192454:2): TOA= 1296      FB0 (192464:8): TOA= 8784     FB1 (192475:9): TOA=10007
    =>FB @ FNR 192473 fn_offset=192473 qbits=4844
    SB1 (384952:1): TOA=   24      => SB 0x019c4808: BSIC=2 fn=192485(145/ 7/11) qbits=4
    =>FB @ FNR 384951 fn_offset=192485 qbits=4912
    TOA AVG is not 16 qbits, correcting (got 17)
    FB0 (194158:8): TOA= 8784      FB0 (194168:8): TOA= 8784     FB1 (194178:8): TOA= 8755
    =>FB @ FNR 194176 fn_offset=194176 qbits=4836
    SB1 (388359:1): TOA=   26      => SB 0x01514908: BSIC=2 fn=194188(146/20/31) qbits=12
    =>FB @ FNR 388358 fn_offset=194188 qbits=4920
    (the "got 13" quoted earlier in this log precedes these lines)
```

5.1 The double subtraction (§4.4) is in the binary that runs. The qbits printed by `fbinfo2cellinfo` after the FB1 are ((TOA − 46) mod 1250)×4, never ((TOA − 23) mod 1250)×4:

FB1 TOA	printed qbits	((TOA − 46) mod 1250)×4	((TOA − 23) mod 1250)×4
9651	3420	**3420**	3512
8751	4820	**4820**	4912
8755	4836	**4836**	4928
10003	4828	**4828**	4920
10007 [2]	4844	**4844**	4936
8755 [2]	4836	**4836**	4928

And the `=>FB @ …` line printed after each SB comes from the task-completion handler `l1a_fb_compl` (`prim_fbsb.c:523-535`, registered at `:572`, triggered by `l1s_compl_sched(L1_COMPL_FB)` at `:266`), which calls `fbinfo2cellinfo` again (`:532`) on `last_fb`, where the SB result has already been reduced by 23 by `l1s_sbdet_resp` (`:207`): it reads ((TOA_sb − 46) mod 1250)×4, i.e. 29 → 4932 (×2), 27 → 4924 (×2), 24 → 4912, 26 → 4920. In total **12 lines out of 12** (6 after an FB1, 6 after an SB) follow the rule "−23 on every pass through `fbinfo2cellinfo`".

5.2 The ROM's immediates (§3). FB mode 1: 9651, 8751, 8755, 10003, 10007, 12507, 12507, 1259, 10007, 8755 are **all 3 (mod 4)** (10/10; 10007 = 48 · 208 + 23). FB mode 0: 7200, 2544, 8784, 10032, 48, 1296, 8784 are **all multiples of 48** (7/7: pages 150, 53, 183, 209, 1, 27, 183).

5.3 The hand-over from FB to SB (§4.4). Criterion: *every* first SB following an FB synchronisation, in both logs, with no exclusion (n = 6; synchronisations 2 and later start from an already aligned time base, which does not change the prediction, which depends only on TOA_fb):

log	FB1 TOA	shift applied (bits, mod 1250)	predicted SB position	measured SB TOA	Δ
[1]	9651	873.75	27.25	29	+1.75
[1]	8751	−26.25	27.25	29	+1.75
[1]	8755	−22.25	27.25	27	−0.25
[1]	10003	−24.25	27.25	27	−0.25
[2]	10007	−20.25	27.25	24	−3.25
[2]	8755	−22.25	27.25	26	−1.25

Mean Δ: **−0.25 bit**; spread ± 2.5, which is what the FB's granularity of 4 (§3) and the SB's integer rounding impose. The ROM's FB and SB measurement conventions coincide on silicon (Δ = 0 ± 2), and the SB arrives where the code puts it: 27 ± 2, **never 23**.

5.4 The 4.25 target (§4.3). After SB synchronisation, `toa.c` prints the average NB TOA over 250 frames **only when it differs from 16**: the lines "got 20", "got 19" ([1]), "got 13", "got 17" ([2]) are therefore the deviations by construction, and the periods at 16 are invisible. What can be said: every printed deviation is within 4 qbits (1 bit) of 16, for a prediction of 17 qbits = 4.25 bits (+ Δ', the NB/SB difference); none is at 23 or 27. And the FB1s of the subsequent synchronisations, taken from an already aligned time base, have TOA mod 1250 = 1, 5, 3, 7, 7, 9 (mean 5.3): the FCCH shows up at 4 ± 4 where the NB loop holds the bursts at 4. The three DSP tasks measure the TOA in the same convention, up to granularity.

2.4.1.7 6. What the 23 is — summary

statement	level	support
The DSP's TOA is a position within the window (index of the retained lag, leading edge of the channel estimate, §2.4), not an offset from nominal time	PROVEN	§2
The SB (191) and NB (151) windows are hard-wired in the ROM and equal 148 + 2 · 23 − 3 and 148 + 2 · 3 − 3	PROVEN	§1, §4.1

statement	level	support
The 23 of `prim_fbsb.c` is `L1_SB_MARGIN_Q`: same originating commit, same value, and it is the only margin that reproduces TI's window	PROVEN for the equality and the common origin; the author's intent is not documented	§4.1, §4.4
"TOA = 23" is, in the firmware's model, an SB at its nominal position, 23 bits after the window opens; in the ROM an SB centred in its 191 samples yields TOA 24.5 (21.5 + 3)	PROVEN (numbers); the 1-to-2-bit gap is absorbed by the +75 and by `toa.c`	§2.5, §4.3
The +75 qbits (= 23 − 4.25 bits) move the burst from the nominal SB position to the centre of the NB window, 40 samples shorter; `toa.c` then holds 16 qbits	PROVEN (code) and verified (logs: 13 to 20 qbits)	§4.3, §5.4
FB path: −23 twice, +75; the SB arrives at 27.25	PROVEN (code, git) and verified (logs: 24 to 29, mean 27.2)	§4.4, §5.1, §5.3
The second subtraction of the FB path was not removed when `fbinfo2cellinfo` was introduced (`cb71b972`); it places the SB near the centre of its window (27 instead of 4); intended or not: a question for the author	FACT (git); intent HYPOTHESIS	§4.4
The number 23 was chosen from the DSP's 191-sample window (TI documentation or experiment) rather than the other way round	VERY LIKELY (the ROM is earlier and frozen; the NB confirms the formula); not documented	§1, §4.1

Phrased for the list: *the DSP returns the position of the burst within its window (the leading edge of its channel estimate), not an offset; the DSP's SB window is 148 + 2 · 23 − 3 = 191 samples (a ROM immediate), and 23 is the margin with which `tpu_window.c` opens its own; `toa - 23` turns the position into an offset from an SB at its nominal place, and the `+ 75` of `synchronize_tdma` (= 23 − 4.25 bits) then shifts to the centre of the NB window, which `toa.c` holds at 16 qbits; on silicon, the SB arrives at 27 ± 2 after FB synchronisation (n = 6) and the NB TOA deviations that `toa.c` reports stay within 4 qbits of 16, as the code predicts.*

2.4.1.8 7. What the bench showed (orientation, not evidence)

- SB TOA = k + 2 (widened waveform) or k + 3 (clean waveform, at sampling instant d = 0), slope 1 with no spread over k = 0 to 40; NB TOA = k + 2 over k = −2 to 6. The "36/40 at k + 3" of an earlier version was measured at d = 0.5, a rounding switch point (TOA = round(k + 3 − d)); at d = 0 it is 40/40. Energy profiles read from the ROM: peak at k + 3 in all three waveforms, with 4 sliding 7-lag sums within 1 % of one another. With the rule of §2.4 (first lag of the window above threshold): bench waveform, E[k+2]/E[k+3] = 41 % → 23; bridge without widening, 1.4 % → 24; widened bridge, 45 % → 23; the three values the ROM published. This is what led to the static readings of §2. Whether the spread of the silicon SBs (24 to 29) comes from the same mechanism (a precursor path, or the receive filter putting energy at lag k + 2) remains a hypothesis.
- SB window: SCH decoded for k from −3 to 46 (centre 21.4); NB window: valid TOAs [0; 8], centre 4: this is what led to §4.3.
- **Fallback audit (§2.6)**: on a complete SB test (20 SCHs targeted, 40 attempts), the `A == 0` test at 0x7ccd was taken 20 times out of 40, always on the 2nd attempt, the one following the SCH frame (firmware and bench post two SB tasks on two consecutive frames, "as it is done by the TSM30"; the 2nd receives the next burst, which is not an SCH, and its channel coefficients are zero). All 20 retained TOAs (correct CRC) come from the main path. The 21/09 note in `MAILBOX.md` (TOA 23, 8/8 SBs decoded) meets the same criterion. The 23 published by the fallback is 21 + 2, the bench's default margin plus the ROM's offset: a coincidence of the bench, unrelated to the firmware's 23.

- **Withdrawn**: the "FCCH in the 1st search frame" test (TOA k − 7 to k − 11 in steps of 4, non-monotonic detection from k 34). Unexplained, hence outside the evidence; the logs (§5.2, §5.3) are sufficient for the FB.
- The `BANC_*` variables added to `tools/dsp_banc_commun.c` and the `fbpos` test remain in the repository for anyone who wants to redo these measurements.

2.4.1.9 8. What remains open, and whose it is

1. **The author's intent.** Can only be closed by Harald Welte or Dieter Spaar. Three questions for the list: was 23 chosen from the DSP's 191-sample window (TI documentation, experiment, TSM30); is the second subtraction of 23 in the FB path (`cb71b972`) intended; did the comment "an SB TOA of 4" refer to the residual after −23.
2. **Δ and Δ' to a tenth of a bit.** No bearing on the claim ($\Delta = 0 \pm 2$ is enough); out of scope without hardware. Verifiable by anyone with a C1xx: first SB after FB synchronisation at $\mathbf{27 \pm 2}$ (firmware later than 2010-05-20), $\mathbf{4 \pm 2}$ on earlier firmware. The list archives ("SB1 (", "TOA AVG is not") may still enlarge n.
3. **The ROM's choice between lags**: closed by static reading (§2.4, leading edge above 2.5 % of the mean energy). The behaviour on a real RF chain (how much energy at lag k + 2) is out of scope.
4. **N, p and the −5 of the FB formula** (§3): not worked out, no bearing on the conclusion.
5. **FreeCalypso / TI**: the TI sources (TCS211) may contain the SB window constant and its comment; not consulted here, status to be assessed.

2.4.1.10 Reproducing

```
#!/bin/bash
# Reproduces the "magic 23" evidence and writes everything to toa.log.
# Idempotent: skips downloads and builds already present.
# Needs: curl, python3, gcc, make; osmocom-bb is cloned if absent.
set -u
LOG=toa.log
: > "$LOG"
say() { printf '%s\n' "$*" | tee -a "$LOG"; }
run() { say "\$ $*"; "$@" 2>&1 | tee -a "$LOG"; say ""; }

say "# magic 23 reproduction — $(date -u +%Y-%m-%dT%H:%MZ) — $(uname -srm)"
say ""

# ------------------------------------------------------------------ 1. ROM
say "## 1. ROM dump -> PROM0.bin"
[ -f dsp-rom-3606-dump.txt ] || run curl -sO ftp://ftp.freecalypso.org/pub/GSM/Calypso/dsp-rom-3606-dump.txt
run sha256sum dsp-rom-3606-dump.txt
python3 - <<'E' 2>&1 | tee -a "$LOG"
import re,struct
sec=None;w={}
for l in open("dsp-rom-3606-dump.txt",errors="replace"):
    m=re.match(r"DSP dump: (\w+) \[",l)
    if m: sec=m.group(1); continue
    m=re.match(r"([0-9a-f]{5}) : ((?:[0-9a-f]{4} ?)+)",l)
    if m and sec=="PROM0":
        a=int(m.group(1),16)
        for i,x in enumerate(m.group(2).split()): w[a+i]=int(x,16)
open("PROM0.bin","wb").write(b"".join(struct.pack("<H",w[a]) for a in range(0x7000,0xe000)))
print("PROM0.bin: %d words (0x7000-0xdfff)" % len(range(0x7000,0xe000)))
for a in (0x7cbd,0x84a4,0xb21d,0xb2b1,0xb32d):
    print("  %04x: %04x %04x" % (a,w[a],w[a+1]))
E
run sha256sum PROM0.bin
say ""
```

```
# ------------------------------------------------------------ 2. disassembler
say "## 2. objdump tic54x (binutils 2.21.1)"
OBJDUMP=build-tic54x/binutils/objdump
if [ ! -x "$OBJDUMP" ]; then
   [ -f binutils-2.21.1.tar.bz2 ] || run curl -sO https://ftp.gnu.org/gnu/binutils/binutils-2.21.1.tar.bz2
    [ -d binutils-2.21.1 ] || tar xjf binutils-2.21.1.tar.bz2
    mkdir -p build-tic54x && ( cd build-tic54x && \
      ../binutils-2.21.1/configure --target=tic54x-coff --disable-werror --disable-nls --disable-gdb \
        --disable-gas --disable-ld --disable-gold --disable-gprof --disable-sim MAKEINFO=true \
        CFLAGS='-O1 -w -std=gnu89' >/dev/null && make -j8 all-binutils MAKEINFO=true >/dev/null ) \
      && say "built $OBJDUMP" || say "BUILD FAILED"
fi
run "$OBJDUMP" --version
"$OBJDUMP" -D -b binary -m tms320c54x --adjust-vma=0x7000 PROM0.bin > PROM0.dis 2>&1
say "full disassembly: PROM0.dis ($(wc -l < PROM0.dis) lines)"
say ""
say "### expected: 7cbd sub #11236,a | 84a4 stm #10791,ar3 | b21d stm #191,ar3 | b2b1 stm #151,ar3 | b32d stm #48,ar3"
grep -E '^ *(7cbd|84a4|b21d|b2b1|b32d):' PROM0.dis | tee -a "$LOG"
say ""
say "### SB TOA chain"
for a in 7c3d 81c8 7c49 7a1c 84a1 84ca 84dd 7c5f 7ca6 7cbc 7e8a b1e7; do
    grep -E "^ *$a:" PROM0.dis | tee -a "$LOG"
done
say ""
say "### FB TOA: 0x793e-0x795a"
awk '/^ *793e:/,/^ *795a:/' PROM0.dis | tee -a "$LOG"
say ""
say "### DMA programming routine: 0xa5cd-0xa5f6"
awk '/^ *a5cd:/,/^ *a5f6:/' PROM0.dis | tee -a "$LOG"
say ""

# ------------------------------------------------------------------ 3. git
say "## 3. layer1 git history"
[ -d osmocom-bb ] || run git clone -q https://gitea.osmocom.org/phone-side/osmocom-bb
run git -C osmocom-bb log --format='%h %ad %an %s' --date=short -S"toa -= 23" -- src/target/firmware/layer1
run git -C osmocom-bb log --format='%h %ad %an %s' --date=short -S"magic 23" -- src/target/firmware/layer1
say "### cb71b972: toa -= 23 / fbinfo2cellinfo lines (expect added fbinfo2cellinfo, no removed toa -= 23)"
git -C osmocom-bb show cb71b972 -- src/target/firmware/layer1/prim_fbsb.c | grep -nE '^[-+].*(toa -= 23|fbinfo2cellinf
say ""
say "### current constants"
grep -nE 'L1_(SB|NB)_MARGIN_Q|L1_TAIL_DURATION_Q|L1_(SB|NB)_DURATION_Q' osmocom-bb/src/target/firmware/layer1/tpu_w
grep -nE 'toa -= 23|magic 23' osmocom-bb/src/target/firmware/layer1/prim_fbsb.c | tee -a "$LOG"
grep -nE 'tpu_shift \+= 75|SB TOA of 4' osmocom-bb/src/target/firmware/layer1/sync.c | tee -a "$LOG"
say ""

# ------------------------------------------------------------------ 4. logs
say "## 4. hardware logs: qbits check ((TOA-46) mod 1250)*4"
say "sources: http://lists.osmocom.org/pipermail/baseband-devel/2011-September/002526.html"
say "         https://www.mail-archive.com/baseband-devel@lists.osmocom.org/msg01046.html"
python3 - <<'E' 2>&1 | tee -a "$LOG"
fb1 = [(9651,3420),(8751,4820),(8755,4836),(10003,4828),(10007,4844),(8755,4836)]
sb  = [(29,4932),(29,4932),(27,4924),(27,4924),(24,4912),(26,4920)]
ok = 0
for toa,q in fb1+sb:
    p46 = ((toa-46)%1250)*4; p23 = ((toa-23)%1250)*4
    ok += (p46==q)
    print("  TOA=%5d printed=%4d  -46:%4d %s  -23:%4d" % (toa,q,p46,"OK " if p46==q else "NO ",p23))
print("  %d/%d lines match the -46 rule" % (ok,len(fb1)+len(sb)))
fb0=[7200,2544,8784,10032,48,1296,8784]; m1=[9651,8751,8755,10003,10007,12507,12507,1259,10007,8755]
print("  FB mode 0 multiples of 48: %d/%d" % (sum(t%48==0 for t in fb0),len(fb0)))
print("  FB mode 1 == 3 mod 4:      %d/%d" % (sum(t%4==3 for t in m1),len(m1)))
```

```
d=[t-27.25 for t,_ in sb]
print("  first SB after FB sync: %s  mean delta %.2f" % ([t for t,_ in sb], sum(d)/len(d)))
print("  148 + 2*23 - 3 = %d ; 148 + 2*3 - 3 = %d ; (191-148+3)/2 = %g" % (148+2*23-3,148+2*3-3,(191-148+3)/2))
E
say ""
say "# done — $(wc -l < "$LOG") lines in $LOG"

# magic 23 reproduction — 2026-10-04T16:50Z — Linux 6.8.0-146-generic x86_64

## 1. ROM dump -> PROM0.bin
$ sha256sum dsp-rom-3606-dump.txt
e35798f50fd0a29aa61cd53e417c010d7f172582c7bc055211daab8af1e05470  dsp-rom-3606-dump.txt

PROM0.bin: 28672 words (0x7000-0xdfff)
  7cbd: f010 2be4
  84a4: 7713 2a27
  b21d: 7713 00bf
  b2b1: 7713 0097
  b32d: 7713 0030
$ sha256sum PROM0.bin
75ce5375a07eac85ee42ba00af7ca82089d15b5df517710a756e6c26497021d7  PROM0.bin


## 2. objdump tic54x (binutils 2.21.1)
$ build-tic54x/binutils/objdump --version
GNU objdump (GNU Binutils) 2.21.1
Copyright 2011 Free Software Foundation, Inc.
This program is free software; you may redistribute it under the terms of
the GNU General Public License version 3 or (at your option) any later version.
This program has absolutely no warranty.

full disassembly: PROM0.dis (28665 lines)

### expected: 7cbd sub #11236,a | 84a4 stm #10791,ar3 | b21d stm #191,ar3 | b2b1 stm #151,ar3 | b32d stm #48,ar3
    7cbd:   f010    sub    #11236,a
    84a4:   7713    stm    #10791,ar3
    b21d:   7713    stm    #191,ar3
    b2b1:   7713    stm    #151,ar3
    b32d:   7713    stm    #48,ar3

### SB TOA chain
    7c3d:   7712    stm    #3278,ar2
    81c8:   f272    rptbd  0x81db
    7c49:   7712    stm    #11498,ar2
    7a1c:   fe00    retd
    84a1:   f7b6    ssbx   st1,frct
    84ca:   f6b6    rsbx   st1,frct
    84dd:   e900    ld     #0,b
    7c5f:   7210    mvdm   0x2f06,ar0
    7ca6:   f4bc    rsbx   st0,tc
    7cbc:   4817    ldm    ar7,a
    7e8a:   711f    mvdk   DP+0x1f,0x3fa4
    b1e7:   10f8    ld     *(0x3fa4),a

### FB TOA: 0x793e-0x795a
    793e:   1194    ld     *ar4+,b
    793f:   f762    sfta   b,2
    7940:   44f8    ld     *(0x3fb4),16,a
    7941:   3fb4
    7942:   f061    sub    #3,16,a
    7943:   0003
```

```
    7944:   3c84    add    *ar4,16,a
    7945:   f061    sub    #2,16,a
    7946:   0002
    7947:   f48c    mpya   a
    7948:   f500    add    a,b
    7949:   f310    sub    #5,b
    794a:   0005
    794b:   6ff8    sub    *(0xc3d),b,a
    794c:   0c3d
    794d:   0e20
    794e:   fc47    rc     aleq
    794f:   44f8    ld     *(0x3fb3),16,a
    7950:   3fb3
    7951:   f061    sub    #3,16,a
    7952:   0003
    7953:   770e    stm    #48,t
    7954:   0030
    7955:   f48c    mpya   a
    7956:   08f8    sub    *(0xc3e),a
    7957:   0c3e
    7958:   f620    sub    b,a
    7959:   fc47    rc     aleq
    795a:   81f8    stl    b,*(0x8fa)

### DMA programming routine: 0xa5cd-0xa5f6
    a5cd:   ea86    ld     #134,dp
    a5ce:   76f8    st     #-3980,*(0x158)
    a5cf:   0158
    a5d0:   f074
    a5d1:   76f8    st     #29250,*(0x159)
    a5d2:   0159
    a5d3:   7242
    a5d4:   76f8    st     #-3981,*(0x15a)
    a5d5:   015a
    a5d6:   f073
    a5d7:   4811    ldm    ar1,a
    a5d8:   80f8    stl    a,*(0x15b)
    a5d9:   015b
    a5da:   69f8    orm    #4,*(0x3fdc)
    a5db:   3fdc
    a5dc:   0004
    a5dd:   75f8    portw  *(0x3fdc),pa63744
    a5de:   3fdc
    a5df:   f900
    a5e0:   47f8    rpt    *(0xc32)
    a5e1:   0c32
    a5e2:   f495    nop
    a5e3:   4812    ldm    ar2,a
    a5e4:   f010    sub    #2048,a
    a5e5:   0800
    a5e6:   6f56    stl    a,1,DP+0x56
    a5e7:   0c81
    a5e8:   7556    portw  DP+0x56,pa64548
    a5e9:   fc24
    a5ea:   4813    ldm    ar3,a
    a5eb:   f110    sub    #48,a,b
    a5ec:   0030
    a5ed:   6f56    stl    a,2,DP+0x56
    a5ee:   0c82
    a5ef:   7556    portw  DP+0x56,pa64550
    a5f0:   fc26
```

```
    a5f1:   7656    st      #1197,DP+0x56
    a5f2:   04ad
    a5f3:   ff4d    xc      2,beq
    a5f4:   7656    st      #1193,DP+0x56
    a5f5:   04a9
    a5f6:   7556    portw   DP+0x56,pa64552

## 3. layer1 git history
$ git clone -q https://gitea.osmocom.org/phone-side/osmocom-bb

$ git -C osmocom-bb log --format=%h %ad %an %s --date=short -Stoa -= 23 -- src/target/firmware/layer1
8978ec57 2010-05-19 Harald Welte layer1/l1ctl: Split L1CTL_NEW_CCCH_REQ in FBSB_REQ nad SYNC_REQ
5f751d3d 2010-04-09 Harald Welte layer1: move FB and SB detection into prim_fbsb.c
fbe7b94c 2010-02-18 Harald Welte Initial import of OsmocomBB into git repository

$ git -C osmocom-bb log --format=%h %ad %an %s --date=short -Smagic 23 -- src/target/firmware/layer1
8978ec57 2010-05-19 Harald Welte layer1/l1ctl: Split L1CTL_NEW_CCCH_REQ in FBSB_REQ nad SYNC_REQ
5f751d3d 2010-04-09 Harald Welte layer1: move FB and SB detection into prim_fbsb.c
48dfd41e 2010-04-07 Harald Welte layer1: add 'struct mon_state' argument to read_{fb,sb}_result()

### cb71b972: toa -= 23 / fbinfo2cellinfo lines (expect added fbinfo2cellinfo, no removed toa -= 23)
20:@@ -87,7 +87,7 @@ static void dump_mon_state(struct mon_state *fb)
29:@@ -98,14 +98,16 @@ static int l1ctl_fbsb_resp(uint8_t res)
48:@@ -179,25 +181,24 @@ static int l1s_sbdet_resp(__unused uint8_t p1, uint8_t attempt,
80:@@ -225,25 +226,14 @@ static int l1s_sbdet_resp(__unused uint8_t p1, uint8_t attempt,
108:@@ -259,18 +249,13 @@ static int l1s_sbdet_resp(__unused uint8_t p1, uint8_t attempt,
134:@@ -279,6 +264,9 @@ static int l1s_sbdet_cmd(__unused uint8_t p1, __unused uint8_t p2,
144:@@ -330,6 +318,41 @@ static int read_fb_result(struct mon_state *st, int attempt)
148:+static void fbinfo2cellinfo(struct l1_cell_info *cinfo,
154:+   last_fb->toa -= 23;
186:@@ -354,6 +377,8 @@ static int l1s_fbdet_cmd(__unused uint8_t p1, __unused uint8_t p2,
195:@@ -363,16 +388,7 @@ static int l1s_fbdet_resp(__unused uint8_t p1, uint8_t attempt,
213:@@ -385,8 +401,22 @@ static int l1s_fbdet_resp(__unused uint8_t p1, uint8_t attempt,
236:@@ -398,7 +428,8 @@ static int l1s_fbdet_resp(__unused uint8_t p1, uint8_t attempt,
246:@@ -408,6 +439,7 @@ static int l1s_fbdet_resp(__unused uint8_t p1, uint8_t attempt,
254:@@ -423,8 +455,15 @@ static int l1s_fbdet_resp(__unused uint8_t p1, uint8_t attempt,
267:+               fbinfo2cellinfo(&l1s.serving_cell, last_fb);
271:@@ -458,8 +497,6 @@ static const struct tdma_sched_item fb_sched_set[] = {
280:@@ -467,35 +504,10 @@ static void l1a_fb_compl(__unused enum l1_compl c)
285:-   last_fb->toa -= 23;
313:+   fbinfo2cellinfo(&l1s.serving_cell, last_fb);
319:@@ -513,6 +525,9 @@ void l1s_fbsb_req(uint8_t base_fn, struct l1ctl_fbsb_req *req)

### current constants
38:#define L1_NB_MARGIN_Q       (3 * 4)
39:#define L1_SB_MARGIN_Q       (23 * 4)
40:#define L1_TAIL_DURATION_Q   (3 * 4)
43:#define L1_NB_DURATION_Q (L1_BURST_LENGTH_Q + 2 * L1_NB_MARGIN_Q - L1_TAIL_DURATION_Q)
44:#define L1_SB_DURATION_Q (L1_BURST_LENGTH_Q + 2 * L1_SB_MARGIN_Q - L1_TAIL_DURATION_Q)
54: [L1_RXWIN_SB]   = L1_SB_DURATION_Q,
55: [L1_RXWIN_NB]   = L1_NB_DURATION_Q,
206:    last_fb->toa -= 23;
333:    /* FIXME: where did this magic 23 come from? */
334:    last_fb->toa -= 23;
469:    /* FIXME: where did this magic 23 come from? */
470:    last_fb->toa -= 23;
119:     * is within 0...8. We have to add 75 to get an SB TOA of 4. */
120:    tpu_shift += 75;

## 4. hardware logs: qbits check ((TOA-46) mod 1250)*4
sources: http://lists.osmocom.org/pipermail/baseband-devel/2011-September/002526.html
```

```
        https://www.mail-archive.com/baseband-devel@lists.osmocom.org/msg01046.html
  TOA= 9651 printed=3420  -46:3420 OK   -23:3512
  TOA= 8751 printed=4820  -46:4820 OK   -23:4912
  TOA= 8755 printed=4836  -46:4836 OK   -23:4928
  TOA=10003 printed=4828  -46:4828 OK   -23:4920
  TOA=10007 printed=4844  -46:4844 OK   -23:4936
  TOA= 8755 printed=4836  -46:4836 OK   -23:4928
  TOA=   29 printed=4932  -46:4932 OK   -23:  24
  TOA=   29 printed=4932  -46:4932 OK   -23:  24
  TOA=   27 printed=4924  -46:4924 OK   -23:  16
  TOA=   27 printed=4924  -46:4924 OK   -23:  16
  TOA=   24 printed=4912  -46:4912 OK   -23:   4
  TOA=   26 printed=4920  -46:4920 OK   -23:  12
  12/12 lines match the -46 rule
  FB mode 0 multiples of 48: 7/7
  FB mode 1 == 3 mod 4:      10/10
  first SB after FB sync: [29, 29, 27, 27, 24, 26]  mean delta -0.25
  148 + 2*23 - 3 = 191 ; 148 + 2*3 - 3 = 151 ; (191-148+3)/2 = 23

# done — 195 lines in toa.log
```

2.5 /opt/GSM/c54x_exe/docs/CLAIMS-2026-10-04.md

19354 octets, 272 lignes

2.5.1 Emulating the baseband — claims and evidence (2026-10-04)

This is the evidence file for the article, not the article. Each quoted sentence is followed by what the code, the ROM and the bench show. Read « VERIFIED » as « verified in the full system (the phone registers, calls, speaks) » unless the bench says otherwise, in which case both verdicts are given.

The text below is the article as drafted (quoted blocks), followed, claim by claim, by what the repositories, the ROM and the bench actually show on 2026-10-04. Each check is marked **VERIFIED** (reproducible from code, ROM disassembly or a run), **CORRECTED** (the claim as written is not what the code does; the corrected statement follows), or **OPEN** (not checked). Sources: `c54x_exe` (this repository), `/opt/GSM/qosmo` (QEMU fork + C54x core, `hw/arm/calypso/`), `/opt/GSM/osmocom-bb` (layer 1), `/opt/GSM/osmo-operator/pont` (bridge), `/opt/GSM/tests` (bench), the public ROM dump `ftp://ftp.freecalypso.org/pub/GSM/Calypso/dsp-rom-3606-dump.txt` (PROM0 identical, word for word, to the one used here), and the binutils `tic54x` disassembler built from the 2.21.1 sources.

> A complete GSM call on an ordinary PC, where the phone's real layer 1 firmware and real baseband DSP code take part, down to the lowest layers of 2G. I forked QEMU to emulate the TI Calypso platform so that OsmocomBB's layer 1 runs on it, and I wrote an emulator for the TI C54x DSP that layer 1 talks to. Nothing goes on air: no SDR, no radio hardware, no spectrum licence.

VERIFIED (call, SMS, A5, speech both ways) — `MAILBOX.md`, entry « 2026-09-23 20:40 », run of 20:22: location update, MO call to the Asterisk echo (ASSIGNMENT COMPLETE 20:22:54, ACTIVE, DISCONNECT), SMS MT and MO with CP-ACK/RP-ACK, MT call via osmo-sip-connector, « chiffrement descendant confirmé par la BTS » five times, speech audible both ways. Nothing on air by construction: the bridge speaks TRXD over UDP to `osmo-bts-trx` (`README.md`, « Architecture »).

VERIFIED (QEMU fork, Calypso machine) — `/opt/GSM/qosmo/hw/arm/calypso/`, machine `-M calypso -cpu arm946` (`run.sh`, QEMU command line); `calypso_l1_dispatch.c`, `l1-dsp/`, `l1-grgsm/`.

VERIFIED (C54x emulator) — `/opt/GSM/qosmo/hw/arm/calypso/l1-dsp/{c54x_exec.c,c54x_decode.c,c54x_mem.c,c54x_irq.c,calypso_c54x.c}`, 1.1 MB of C; `c54x_exe` links them (`Makefile`, `SRC`).

> On the phone side, the QEMU Calypso machine loads the OsmocomBB layer 1 firmware as the real ARM would, and a native C54x emulator runs the DSP with its ROM downloaded from FreeCalypso rather than redistributed.

VERIFIED since 2026-10-04 (was CORRECTED) — until this audit, `run.sh` (be10de1) defaulted to `layer1.highram_patched.elf`, a build that passed downlink speech frames flagged `B_BFI` at zero gain (« trames B_BFI, vols FACCH compris, passées à charge nulle, masquées par l'ECU de GAPK »). On 2026-10-04 the default was put back to the unmodified `layer1.highram.elf` and the patched `.elf/.bin/.map` were removed from `/opt/GSM/firmware/board/compal_e88/` (the `layer1.highram_patched.patch` file stays as a record of what was tried). The consequence to watch: whatever the ROM flags `B_BFI` is now handled by the firmware as on a phone.

CORRECTED (ROM download) — `install.sh` contains no FreeCalypso URL (`grep -n freecalypso install.sh` is empty); the ROM images are read from `rom/calypso_dsp.*.bin` (`src/main.c`, `ROMS[]`), and `rom/calypso_dsp.Registers.bin` is tracked in git. The PROM0 used is identical to the public FreeCalypso dump (28,672 words, 0 differences), so the sentence is true of the content, not of the mechanism. Either make `install.sh` fetch the dump, or write « identical to the public FreeCalypso dump ».

Layer 1 drives the DSP through its shared API, as on real hardware.

VERIFIED — `README.md` (« the ROM's API RAM window is memory-mapped and shared with QEMU », `/dev/shm/calypso_api_ram`); frame-by-frame lockstep over `/tmp/calypso_dsp.sock` (`CALYPSO_PONT_LOCKSTEP=1`, `run.sh:57-63`).

In the downlink, the DSP ROM itself does the work: frequency burst detection, synchronisation, control channel and speech decoding, A5 deciphering.

VERIFIED in the full system, FAIL on the bench for FB and NB — `/opt/GSM/tests/modules/99-couverture.sh`, section « CHEMIN ROM / HOTE », mode `dsp`, lines 128-134: FB (`d_fb_det`), SB/SCH (`a_sch`), BCCH/CCCH/SDCCH/SACCH (`a_cd`), FACCH (`a_fd`), TCH/F speech (`a_dd`, read as is by the firmware), A5 downlink (`[a5]`, modelled coprocessor `calypso_a5.c`). Run of 2026-10-03 17:11 (`MAILBOX.md`, last entry): « 6 ROM ». Known limits in the same table: SACCH/8 downlink « souvent FIRE KO » (the 2026-10-03 note in `MAILBOX.md` says « ~80 % », without a count), B_BFI frames masked by the patched firmware. On the bench (`./dsp_tester --all`, 2026-10-04): `sb` PASS (20/20), but `fb0`/`fb1` FAIL (5/6 detections, one FCCH frame mis-deduced), `fbsb` FAIL, `nb` FAIL (8/8 blocks with FIRE OK but a content different from what the bench expects — the bench's expectation has not been checked). The system works end to end; the bench says that on FB and NB either the ROM, the core or the bench computes something wrong, and that is not settled.

In the uplink, SDCCH is now produced by the ROM too, after four bugs in the emulated core were fixed;

VERIFIED — `src/pont.c`, `tx_rom_publier()` (bursts the ROM serialises into `data[0x3f8a]`, published to `/dev/shm/calypso_xcch_ul_rom`); bridge `pont/dsp/uplink.py`, `_bursts_xcch()` (`PONT_UL_ROM`); run 17:11: « xCCH ul rom=134 hote=1 ». The four core fixes are in the qosmo history: `4771f91d` (0xA1xx = ADD Xmem,Ymem,B; circular `*+ARx(lk)%`; SFTA carry; AND/OR/XOR src,SHIFT,dst), `1e47cd72` (F0Bx/F1Bx are OR A,-16..-1,dst, not RSBX/SSBX), `e8f477ad` (SQURS, POLY, MAC*, SUBC carry), `a19dde59` (MVKD/MVDK under RPT increment DAR). Bench: `./dsp_tester tx-sdcch` → PASS, 24/24 bursts identical to `gsm0503_xcch_encode` (after aligning the bench's hl/hu capture with `pont.c`, 2026-10-04).

RACH, FACCH/SACCH on traffic channels, uplink speech and uplink A5 are still encoded on the host side, and moving them into the DSP is the current work.

VERIFIED (state on 2026-10-03) — `99-couverture.sh:135-141`: RACH (`montant.c` reads `d_rach`, the bridge codes), A5 uplink (the bridge undoes the ROM's XOR and re-ciphers at the air fn), FACCH/F (`a_cu` → bridge `TchScheduler`), SACCH on TCH (bridge), TCH/F speech (`montant.c` TI→FR → bridge `tch_fr_encode`), vocoder (GAPK). `src/montant.c:1236-1245`: `d_task_ra` is erased from the W page by default (`MONTANT_EFFACE_TASK_RA=1`), so the ROM never executes the RACH task in the full system.

NEW on 2026-10-04 — RACH is bit-exact in the ROM *on the bench*; in the full system the host still codes it (`d_task_ra` erased, bridge `gsm.rach_burst`) — see « What the RACH trace shows » below. The diagram and the article must say « RACH: ROM on the bench, host in the full system ».

A bridge connects the phone to osmo-bts-trx, reusing code from Vadim Yanitskiy (fixeria),

OPEN (attribution not found) — `grep -rin "fixeria\|yanitskiy" /opt/GSM/osmo-operator/pont/` finds nothing in the bridge sources or its README. If code was reused, the attribution should be in the files; if the sentence refers to libosmocoding/trxcon more generally, say so.

and an Osmocom core network handles the call on the other side. An end-to-end test bench checks the whole chain, from boot to attach, SMS, USSD, a voice call and a test tone recovered through it.

VERIFIED — `/opt/GSM/tests/modules/`: `00-demarrage`, `10-coeur`, `20-bts`, `30-services`, `60-camp`, `70-attache`, `80-sms-mo`, `85-sms-mt`, `87-ussd`, `90-appel`, `92-voix`, `99-couverture`.

> An experimental mode already runs layer 1 without any TI ROM, a first step toward a fully free baseband stack.

CORRECTED (« fully free baseband stack ») — `MODE=grgsm ./run.sh` (`run.sh:16,42,95-98`: « montage grgsm : pas de c54x_exe, la couche 1 est dans QEMU »), `/opt/GSM/qosmo/hw/arm/calypso/l1-grgsm/`. In this mode the DSP functions are reimplemented on the host (gr-gsm, libosmocoding, the bridge for A5): « layer 1 without the TI ROM, with host-side signal processing », not a free baseband.

2.5.1.1 Three stages, as they appear in the code

1. Layer 1 in QEMU. The QEMU fork runs an unmodified `layer1.highram` on the Calypso machine.

VERIFIED since 2026-10-04 — see above: the patched default is gone.

2. The gr-gsm shunt. […] The DSP was bypassed; the host rebuilt the signal processing.

VERIFIED — `99-couverture.sh:117-124`, mode `grgsm`: « FB/SB : hôte : gr-gsm dans QEMU », « descendant : hôte : pont (libosmocoding) », « A5 : hôte : pont », « montant : hôte ».

3. The DSP that works. […] State on 2026-10-03: the whole downlink is ROM; uplink SDCCH is ROM; still on the host: RACH, FACCH/SACCH on TCH, uplink speech, uplink A5, the vocoder, plus a small patch of the IMMEDIATE ASSIGNMENT.

VERIFIED — `99-couverture.sh:131`: « IMMEDIATE ASSIGNMENT : ROM + retouche hôte de la référence (qosmo calypso_trx.c) ; l'hôte réécrit la référence de requête lue dans a_cd ». The line has no ROM pattern, so the table's verdict for it is HOTE by construction (it is one of the 7 host steps of the 17:11 run); the single MIXTE line is the SDCCH uplink (`:136`, rom=134 hote=1). The retouch exists because the RACH is sent by the bridge at a frame number the firmware did not choose; a ROM RACH sent at the ROM's frame removes the need for it (not yet done in the full system).

2.5.1.2 What "processor fidelity" means now

> Capturing at `0x3f8a` (what `tx_rom_publier` does) takes the coded bits *before* the modulator. The host still has to add the training sequence, synchronisation and tail bits, and redo A5 at the right frame number. That is ROM at about 80 %.

VERIFIED (mechanism), CORRECTED (the number) — `src/pont.c:555-600`: 114 data bits + hl/hu read from `data[0x3f8a..0x3f91]`; the A5 stream the ROM XORed is read back from `data[0x3f9b..]` and published so that the bridge can undo it and re-cipher (`pont/dsp/uplink.py:24-36`). « About 80 % » is not a measurement; the only count available is the coverage table (per treatment: 6 ROM, 7 host, 1 mixed on 2026-10-03). Replace the percentage by that count or by a bit count.

> Capturing the TSP script (`0x3cbb..`, marker `0x1c0a` = TOGBR2, then 16 BULDATA words at 10 bits) takes the stream *the ABB would receive on silicon*, with training sequence, tails, I/Q inversion and A5 already applied. The host then only converts ABB words into 148 bits and places them at the right fn/tn. This capture point is valid for every burst type (NB, RACH, TCH), and it is the one that matches the goal: a single pipe, the ROM decides everything.

VERIFIED in the ROM, and now reproduced in emulation (2026-10-04) — static reading of PROM0:

- `0xb5b8 stm #15547,ar2` (0x3cbb): the ROM builds a TSP script for the ABB there each frame (register writes requested through NDB words 0x8ed..0x8f7, `0xb5ba-0xb5e5`).
- `0xb6fd` (RACH) / `0xb70f` (NB): `st #7178,*ar2+` → 0x1c0a = TOGBR2 (page 0, reg 5) with data 0x70, then the burst through dispatcher code 39 (`data[0x43ae]` = 0xb77f → `fcall 0x859f` → emitter `0x85a2`) or code 40 (NB emitter `0x85c1`), then the power word `*(0x3fc9)`, then `st #18` (AUXAPC).
- Emitter `0x8605-0x868f`: a segment table driven by DP+0x49 (loads from the sync/TSC copy at 0x3dca, the data at 0x3f8a, the tail/guard words at 0x3dcd), a bit accumulator in B, and `sth a,*ar2+` at `0x8632` with the register bits added (`add #6,16,a` = BULDATA1, reg 3; TSP encoding `(data & 0x3ff) << 6 | reg << 1`, `abb/twl3025.c:156`). The emitted word is therefore `(10 data bits) << 6 | 0x06`.

- `0x85c1` (NB): `xor *ar4+` over the 8 words 0x3f8a..0x3f91 with 0x3f9b.. (A5 stream), then the 26-bit TSC copied from DROM 0xa0c9 + 2 · tsc (the 8 TSCs are at DROM 0xa0c9..0xa0d8). `0x85a2` (RACH): the 41-bit sync copied from DROM 0xa0d9..0xa0db (`4b7f 99aa 3c00` = 01001011011111111001100110101010001111000). Tail/guard words DROM 0xa0dc..0xa0dd (`1fff ffff`).
- `0xb773`: `xorm #0xAA80` over the 16 words when bit 14 of `0x3f92` is set — the I/Q inversion flips every other data bit (mask 1010101010 in the 10 data bits).

Bench, `./dsp_tester rach` (new test, 2026-10-04): for five random RA values with BSIC 7, the 16 words after 0x1c0a decode to 160 bits = `11111111` (ramp fill) + `00111010` (extended tail, 05.02 §5.2.7) + the 41-bit synchronisation sequence + the 36 coded bits + `000` + guard ones; the 36 coded bits are identical to `gsm0503_rach_ext_encode(ra, bsic)` (5/5), and the ROM's own copy of them in `data[0x3f8a..0x3f8c]` as well (5/5). Example, ra=0xd4: `1111111100111010 01001011011111111001100110101010001111000 000011011110000101011001111010110 00 1111…`.

CORRECTED (« reachable without touching the core ») — it was not. In emulation the emitter never produced the 16 words before 2026-10-04, for two reasons in the C54x core (`qosmo`, `l1-dsp/c54x_exec.c`):

1. `CMPR CC, ARx` (binutils `0xF4A8` mask `0xFCF8`, 78 sites in PROM0) was not implemented: `cmpr eq` fell into `unimpl` (a no-op, TC unchanged) and `cmpr lt/gt/neq` into `RPT #k`, the F6xx no-op and the F7xx block. The emitter's loop `0x8608 cmpr eq,ar2 ; 0x8609 rc tc` (AR0 = end of the 16 words) returned at once on a stale TC = 1 (probe: ST0 = 0x107b before and after the CMPR). Fixed; `CALYPSO_CMPR_ANCIEN=1` restores the old behaviour.
2. `LD src, ASM [, dst]` (`0xF482` mask `0xFCFF`) ignored the ASM shift (ST1[4:0], two's complement): the accumulator was never shifted and all 16 words came out as `0xff06` (probe: B unchanged across `0x8625 ld b,asm` with ASM = 10). Fixed; `CALYPSO_LDASM_ANCIEN=1` restores the old behaviour.

So the TSP script is the right capture point, but it only exists in emulation since these two fixes. With them, the bench's `tx-sdcch` (normal bursts, ROM-coded SDCCH) still passes 24/24.

> What the RACH trace shows points the same way: the ROM does execute the RACH task (`0xb6fd`, then `0x85a2` on each attempt), writes its 36 coded bits to `0x3f8a`, and the transmitter fills in the script. The only obstacle is on the system side: `montant.c` clears `d_task_ra` by default (`MONTANT_EFFACE_TASK_RA=1`), so the ROM has never been allowed to do a real RACH. The bit differences seen against the host-side encoder settle nothing until that 05.03 implementation has been checked against `gsm0503_rach_ext_encode` on the bench, which is the next measurement.

VERIFIED, and the measurement is done — probe on the bench: `0xb6fd` and `0x85a2` on each of the 5 attempts; `data[0x3f8a..0x3f8c]` = the 36 coded bits (`0de1 59eb 3000` for ra=0xd4). The « bit differences » were an error in a hand-written 05.03 encoder (the CRC-6 remainder is inverted, `remainder=0x3f` in `gsm0503_rach_crc6`); against libosmocoding the ROM is exact, 5/5. The system-side obstacle is as stated (`montant.c:1236-1245`); the second obstacle, the two core bugs above, is now removed on the bench. What remains for the full system: publish the TSP burst from `pont.c`, let the bridge send it at the ROM's frame, stop erasing `d_task_ra`.

2.5.1.3 The hierarchy for what comes next

> Maximum fidelity, reachable without touching the core: make the TSP script the single source of all uplink bursts (RACH, SDCCH, TCH, SACCH, FACCH), and transmit at the ROM's frame number so that its A5 is the right one.

CORRECTED (core) — see above. **OPEN (A5 at the ROM's frame)** — the bridge's comment says the ROM « chiffre à l'heure du DSP, pas au fn émis » (`pont/dsp/uplink.py:24`, `99-couverture.sh:137`); which frame number the ROM uses for the uplink stream (`a_a5fn`, `d_fn`, or its own counter) has not been read in the ROM, and whether « the ROM's frame » equals the air frame once the bridge sends immediately has not been measured. To check before building on it.

> Fidelity that requires work on the core: the cases where the ROM computes wrongly or not at all (downlink SACCH/8 at 80 % FIRE failures, `B_BFI`, the bench's `nb` and `fb0/fb1` tests).

VERIFIED (the list), and one number the article must state — ISA conformance of the emulated core (`./isa_test tools/isa_tests.txt`, the worked examples of SPRU172C): **of 230 examples, 150 pass, 58 fail and 22 cannot be assembled by the harness** (208 `T` records and 22 `S` records in `tools/isa_tests.txt`; unchanged by the two fixes above; CMPR is among the « not assembled »). A quarter of the manufacturer's examples fail. Every « the ROM does X » is conditional on the core executing the instructions of that path correctly, and the two bugs of 2026-10-04 were found exactly that way. The list of failing examples

(`./isa_test tools/isa_tests.txt | grep ^FAIL`) is the most profitable work queue of the project before any new pipe. Bench on 2026-10-04: `fb0`, `fb1`, `fbsb`, `nb` FAIL; `boot`, `idle`, `checksum`, `fb-afc`, `sb`, `tx-sdcch`, `rach` PASS. `99-couverture.sh:130,133`: SACCH/8 FIRE KO, B_BFI.

> The most distant fidelity: the vocoder in the ROM, which means modelling the audio path from the DSP to the ABB.

OPEN — plausible (the ROM delivers TI-format speech frames to `a_dd`, GAPK decodes them on the host, `99-couverture.sh:141`); nothing measured.

> In other words, the DSP has stopped being an obstacle to work around and has become the reference: whenever the host still does something, it is either because it has not yet been given the ROM's pipe, or because the core still has a bug. Both are measurable, and that is what is being measured.

VERIFIED as a description of the method — the RACH case of 2026-10-04 is exactly that: a ROM pipe not yet used (`d_task_ra` erased), and two core bugs found by following the ROM's own code to the point where it writes the burst.

2.5.1.4 Reproducing the new measurements

```
# ROM : disassembly of the emitter and of the TSP script builder
# the tic54x target is gone from current binutils (2.42): build objdump from the GNU 2.21.1 sources
# (tested 2026-10-04, gcc 13 -- the old C needs -std=gnu89)
curl -O https://ftp.gnu.org/gnu/binutils/binutils-2.21.1.tar.bz2
tar xjf binutils-2.21.1.tar.bz2 && mkdir build-tic54x && cd build-tic54x
../binutils-2.21.1/configure --target=tic54x-coff --disable-werror --disable-nls --disable-gdb --disable-gas \
    --disable-ld --disable-gold --disable-gprof --disable-sim MAKEINFO=true CFLAGS='-O1 -w -std=gnu89'
make -j8 all-binutils MAKEINFO=true && cd ..
build-tic54x/binutils/objdump -D -b binary -m tms320c54x --adjust-vma=0x7000 PROM0.bin | less    # 0xb5b8, 0xb6fd, 0xb7
                                                          # 0x859f/0x85a2, 0x859c/0x85c1, 0x8605-0x868f
# DROM tables : 0xa0c9..0xa0d8 (8 TSC), 0xa0d9..0xa0db (RACH sync), 0xa0dc..0xa0dd (tail/guard)
# bench
make dsp_tester && ./dsp_tester rach tx-sdcch -v
CALYPSO_CMPR_ANCIEN=1 ./dsp_tester rach          # before the CMPR fix : no burst in the script
CALYPSO_LDASM_ANCIEN=1 ./dsp_tester rach         # before the LD src,ASM fix : 16 x 0xff06
```

2.6 /opt/GSM/c54x_exe/docs/carte-memoire.md

30700 octets, 418 lignes

2.6.1 Calypso DSP memory map - API RAM, NDB, PARAM, internal memory, ports

Section 2 is generated (DWARF of the layer1 ELF + replay measurement); the other sections are written from measurement and the cited sources. `harvard.sh` reads the section 2 table (`CARTE=`).

2.6.1.1 How this file is produced

- Field offsets: `gdb-multiarch -batch -ex 'ptype /o T_NDB_MCU_DSP' /opt/GSM/firmware/board/compal_e88/layer1.highram` (and `T_DB_MCU_TO_DSP`, `T_DB_DSP_TO_MCU`, `T_PARAM_MCU_DSP`): the **compiled** layout (DSP=36, CHIPSET=12, ANLG_FAM=2, `include/calypso/l1_environment.h`). Bases: `print/x dsp_api` in the same ELF (ndb=0xffd001a8, db_r=0xffd00050, db_w=0xffd00000, param=0xffd00862).
- Measurement: `make layer1_tester && tools/layer1_tester -o mesure.tsv -j journal.tsv -p pcs.tsv rejeu.bin`, a replay of a dedicated-channel recording (copy of `/dev/shm/calypso_rejeu_tch.bin`).
- Conversion: DSP word = 0x0800 + (ARM address - 0xFFD00000)/2; "API byte" = ARM address - 0xFFD00000.
- "ROM reads / writes" = accesses made by a ROM instruction (`calypso_mbx_evt` hook); "ARM changes" = words changed on the QEMU side between two frames (`A` records), i.e. changes, not every write; values = up to the 4 most frequent (value:count). First PC in hex (`x:pppp` when the XPC page is not 0). Untouched `*hole*` fields are omitted.

2.6.1.2 1. Data space map (DSP view)

range	content	source
0x0000..0x001f	C54x MMRs (IMR, IFR, ST0/1, AR0..7, SP, BK, BRC, PMST, XPC…)	qosmo `l1-dsp/calypso_c54x.h`
0x0020..0x005f	peripherals (timer 0x24..0x26, DMA 0x54..0x57)	`calypso_c54x.h`, `calypso_dma.h`
0x0800..0x0813	W0 page (`T_DB_MCU_TO_DSP`, ARM 0xFFD00000)	ELF
0x0814..0x0827	W1 page (ARM 0xFFD00028)	ELF
0x0828..0x083b	R0 page (`T_DB_DSP_TO_MCU`, ARM 0xFFD00050)	ELF
0x083c..0x084f	R1 page (ARM 0xFFD00078)	ELF
0x08d4..0x153d	NDB (`T_NDB_MCU_DSP`, ARM 0xFFD001A8, 6356 bytes)	ELF
0x0c31..0x0cc1	PARAM (`T_PARAM_MCU_DSP`, ARM 0xFFD00862), **inside** NDB.a_sr_holes1 (290 bytes = the 145 useful PARAM words)	ELF
0x0cce..0x0df5	DMA buffer of received I/Q (AAD 0x99c), inside NDB.a_cport_holes	`c54x_mem.c` (FEED-DST)
0x0ffc..0x0fff	loader words BL_ADDR_HI / BL_SIZE / BL_ADDR_LO / BL_CMD_STATUS (ARM 0xFFD00FF8..FFE), inside NDB.a_cport_holes	`firmware/calypso/dsp.c`
0x153e..0x27ff	API RAM outside the layer1 structures	-
>= 0x2800	DSP internal memory (section 4)	-

2.6.1.3 2. API RAM - all fields (generated)

zone	field	API byte	ARM	DSP word	ROM reads (n, 1st PC)	ROM writes (n, 1st PC)	ARM changes (n, values)
W0	d_task_d	0x000	0xFFD00000	0x0800	25920, b001	0	1296, 000d:445,000e:361,0000:2
W0	d_burst_d	0x002	0xFFD00002	0x0801	14950, b005	361, ba17	777, 0000:475,0002:102,0001:1
W0	d_task_u	0x004	0xFFD00004	0x0802	13229, b009	0	1298, 000d:445,000e:361,0000:2
W0	d_burst_u	0x006	0xFFD00006	0x0803	8345, b00d	361, ba19	780, 0000:475,0002:103,0001:1
W0	d_task_md	0x008	0xFFD00008	0x0804	19843, b011	0	300, 0001:150,0000:150
W0	d_background	0x00a	0xFFD0000A	0x0805	6614, b0db	0	0
W0	d_debug	0x00c	0xFFD0000C	0x0806	13229, b015	0	0
W0	d_task_ra	0x00e	0xFFD0000E	0x0807	6614, b097	0	0
W0	d_fn	0x010	0xFFD00010	0x0808	13613, b548	0	5760, ,+
W0	d_ctrl_tch	0x012	0xFFD00012	0x0809	52823, b54b	0	518, 0000:255,0030:169,0011:8
W0	hole	0x014	0xFFD00014	0x080a	0	0	0
W0	d_ctrl_abb	0x016	0xFFD00016	0x080b	6615, b5a5	0	576, 0019:288,0010:288

zone	field	API byte	ARM	DSP word	ROM reads (n, 1st PC)	ROM writes (n, 1st PC)	ARM changes (n, values)
W0	a_a5fn[2]	0x018	0xFFD00018	0x080c..0x080d	13228, b044	0	7247, ,+
W0	d_power_ctl	0x01c	0xFFD0001C	0x080e	6615, b5a9	0	587, 0000:276,2e92:111,3212:7
W0	d_afc	0x01e	0xFFD0001E	0x080f	6615, b5ad	0	1, fd44:1
W0	d_ctrl_system	0x020	0xFFD00020	0x0810	19856, a53c	13, a549	522, 0007:255,0000:254,8000:1
W1	d_task_d	0x028	0xFFD00028	0x0814	25607, b001	0	1293, 0000:567,000d:443,0018:2
W1	d_burst_d	0x02a	0xFFD0002A	0x0815	14297, b005	81, ba17	571, 0000:267,0001:102,0003:1
W1	d_task_u	0x02c	0xFFD0002C	0x0816	13206, b009	0	1294, 0000:566,000d:443,000c:2
W1	d_burst_u	0x02e	0xFFD0002E	0x0817	7703, b00d	81, ba19	571, 0000:264,0003:103,0001:1
W1	d_task_md	0x030	0xFFD00030	0x0818	19845, b011	0	866, 0001:432,0000:427,0005:7
W1	d_background	0x032	0xFFD00032	0x0819	6603, b0db	0	0
W1	d_debug	0x034	0xFFD00034	0x081a	13206, b015	0	0
W1	d_task_ra	0x036	0xFFD00036	0x081b	6603, b097	0	0
W1	d_fn	0x038	0xFFD00038	0x081c	13325, b548	0	5756, 0000:1,+
W1	d_ctrl_tch	0x03a	0xFFD0003A	0x081d	51969, b54b	0	1061, 0000:531,0011:363,0030:1
W1	hole	0x03c	0xFFD0003C	0x081e	0	0	0
W1	d_ctrl_abb	0x03e	0xFFD0003E	0x081f	6603, b5a5	0	1132, 0019:566,0010:566
W1	a_a5fn[2]	0x040	0xFFD00040	0x0820..0x0821	13206, b044	0	7791, 0095:1,0000:1,+
W1	d_power_ctl	0x044	0xFFD00044	0x0822	6603, b5a9	0	1141, 0000:501,2392:255,2e92:4
W1	d_afc	0x046	0xFFD00046	0x0823	6603, b5ad	0	1, fd44:1
W1	d_ctrl_system	0x048	0xFFD00048	0x0824	19809, a53c	0	1057, 0000:531,0007:526
R0	d_task_d	0x050	0xFFD00050	0x0828	0	6615, b003	6076, 0000:6076
R0	d_burst_d	0x052	0xFFD00052	0x0829	0	6615, b007	302, 0000:302
R0	d_task_u	0x054	0xFFD00054	0x082a	0	11838, b00b	6080, 0000:6080
R0	d_burst_u	0x056	0xFFD00056	0x082b	0	6615, b00f	305, 0000:305
R0	d_task_md	0x058	0xFFD00058	0x082c	0	6615, b013	150, 0000:150
R0	d_background	0x05a	0xFFD0005A	0x082d	0	0	0
R0	d_debug	0x05c	0xFFD0005C	0x082e	0	6615, b017	0
R0	d_task_ra	0x05e	0xFFD0005E	0x082f	0	0	0
R0	a_serv_demod[4]	0x060	0xFFD00060	0x0830..0x0833	0	24288, b267	24225, 0000:6076
R0	a_pm[3]	0x068	0xFFD00068	0x0834..0x0836	0	150, b34b	150, 0000:150
R0	a_sch[5]	0x06e	0xFFD0006E	0x0837..0x083b	0	0	0
R1	d_task_d	0x078	0xFFD00078	0x083c	0	6603, b003	5796, 0000:5796
R1	d_burst_d	0x07a	0xFFD0007A	0x083d	0	6603, b007	304, 0000:304

zone	field	API byte	ARM	DSP word	ROM reads (n, 1st PC)	ROM writes (n, 1st PC)	ARM changes (n, values)
R1	d_task_u	0x07c	0xFFD0007C	0x083e	0	11823, b00b	5800, 0000:5800
R1	d_burst_u	0x07e	0xFFD0007E	0x083f	0	6603, b00f	307, 0000:307
R1	d_task_md	0x080	0xFFD00080	0x0840	0	6603, b013	427, 0000:427
R1	d_background	0x082	0xFFD00082	0x0841	0	0	0
R1	d_debug	0x084	0xFFD00084	0x0842	0	6603, b017	0
R1	d_task_ra	0x086	0xFFD00086	0x0843	0	0	0
R1	a_serv_demod[4]	0x088	0xFFD00088	0x0844..0x0847	0	23212, b267	23120, 0000:5796
R1	a_pm[3]	0x090	0xFFD00090	0x0848..0x084a	0	432, b34b	427, 0000:427
R1	a_sch[5]	0x096	0xFFD00096	0x084b..0x084f	0	0	0
NDB	d_dsp_page	0x1a8	0xFFD001A8	0x08d4	13218, a51c	0	13234, 0002:6614,0003:6610,000
NDB	d_error_status	0x1aa	0xFFD001AA	0x08d5	0	13218, b10a	0
NDB	d_spcx_rif	0x1ac	0xFFD001AC	0x08d6	13218, b56c	0	0
NDB	d_tch_mode	0x1ae	0xFFD001AE	0x08d7	14711, 85e4	0	13, ca00:7,ca08:6
NDB	d_debug1	0x1b0	0xFFD001B0	0x08d8	11838, b585	0	0
NDB	d_dsp_test	0x1b2	0xFFD001B2	0x08d9	10616, a90a	0	0
NDB	d_version_number1	0x1b4	0xFFD001B4	0x08da	0	0	0
NDB	d_version_number2	0x1b6	0xFFD001B6	0x08db	0	0	0
NDB	d_debug_ptr	0x1b8	0xFFD001B8	0x08dc	81581, b524	81581, b530	0
NDB	d_debug_bk	0x1ba	0xFFD001BA	0x08dd	81581, b528	0	0
NDB	d_pll_config	0x1bc	0xFFD001BC	0x08de	34471, a687	0	0
NDB	p_debug_buffer	0x1be	0xFFD001BE	0x08df	0	0	0
NDB	d_debug_buffer_size	0x1c0	0xFFD001C0	0x08e0	0	0	0
NDB	d_debug_trace_type	0x1c2	0xFFD001C2	0x08e1	0	0	0
NDB	d_dsp_state	0x1c4	0xFFD001C4	0x08e2	0	0	0
NDB	d_mcsi_select	0x1ce	0xFFD001CE	0x08e7	0	0	0
NDB	d_apcdel1_bis	0x1d0	0xFFD001D0	0x08e8	0	0	0
NDB	d_apcdel2_bis	0x1d2	0xFFD001D2	0x08e9	0	0	0
NDB	d_apcdel2	0x1d4	0xFFD001D4	0x08ea	0	0	0
NDB	d_vbctrl2	0x1d6	0xFFD001D6	0x08eb	0	0	0
NDB	d_bulgcal	0x1d8	0xFFD001D8	0x08ec	0	0	0
NDB	d_afcctladd	0x1da	0xFFD001DA	0x08ed	13218, b5ba	0	0
NDB	d_vbuctrl	0x1dc	0xFFD001DC	0x08ee	13226, b5c4	0	0
NDB	d_vbdctrl	0x1de	0xFFD001DE	0x08ef	13218, b5c9	0	0
NDB	d_apcdel1	0x1e0	0xFFD001E0	0x08f0	11885, b660	0	0
NDB	d_apcoff	0x1e2	0xFFD001E2	0x08f1	36990, b5bf	11886, b67e	11886, 1817:11886
NDB	d_bulioff	0x1e4	0xFFD001E4	0x08f2	13218, b5d3	0	0

zone	field	API byte	ARM	DSP word	ROM reads (n, 1st PC)	ROM writes (n, 1st PC)	ARM changes (n, values)
NDB	d_bulqoff	0x1e6	0xFFD001E6	0x08f3	13218, b5d8	0	0
NDB	d_dai_onoff	0x1e8	0xFFD001E8	0x08f4	13218, b5ed	0	0
NDB	d_auxdac	0x1ea	0xFFD001EA	0x08f5	13218, b5dd	0	0
NDB	d_vbctrl1	0x1ec	0xFFD001EC	0x08f6	13230, b5e2	4, b6a8	0
NDB	d_bbctrl	0x1ee	0xFFD001EE	0x08f7	13218, b5ce	0	0
NDB	d_fb_det	0x1f0	0xFFD001F0	0x08f8	6397, 778a	6397, b2cc	0
NDB	d_fb_mode	0x1f2	0xFFD001F2	0x08f9	18874, 781f	0	0
NDB	a_sync_demod[4]	0x1f4	0xFFD001F4	0x08fa..0x08fd	0	25468, b2cf	0
NDB	a_sch26[5]	0x1fc	0xFFD001FC	0x08fe..0x0902	0	0	0
NDB	d_audio_gain_x2	0x206	0xFFD00206	0x0903	0	0	0
NDB	d_audio_gain_x1	0x208	0xFFD00208	0x0904	0	0	0
NDB	d_audio_compressor_ctrl	0x20a	0xFFD0020A	0x0905	0	0	0
NDB	d_audio_init	0x20c	0xFFD0020C	0x0906	13218, c1fe	0	0
NDB	d_audio_status	0x20e	0xFFD0020E	0x0907	13218, c1ff	0	0
NDB	d_toneskb_int	0x210	0xFFD00210	0x0908	5306, 7382	0	0
NDB	d_toneskb_start	0x212	0xFFD00212	0x0909	23882, 7365	5308, 73af	0
NDB	d_k_x1_t0	0x214	0xFFD00214	0x090a	0	0	0
NDB	d_k_x1_t1	0x216	0xFFD00216	0x090b	0	0	0
NDB	d_k_x1_t2	0x218	0xFFD00218	0x090c	0	0	0
NDB	d_pe_rep	0x21a	0xFFD0021A	0x090d	0	0	0
NDB	d_pe_off	0x21c	0xFFD0021C	0x090e	0	0	0
NDB	d_se_off	0x21e	0xFFD0021E	0x090f	0	0	0
NDB	d_bu_off	0x220	0xFFD00220	0x0910	0	0	0
NDB	d_t0_on	0x222	0xFFD00222	0x0911	0	0	0
NDB	d_t0_off	0x224	0xFFD00224	0x0912	0	0	0
NDB	d_t1_on	0x226	0xFFD00226	0x0913	0	0	0
NDB	d_t1_off	0x228	0xFFD00228	0x0914	0	0	0
NDB	d_t2_on	0x22a	0xFFD0022A	0x0915	0	0	0
NDB	d_t2_off	0x22c	0xFFD0022C	0x0916	0	0	0
NDB	d_k_x1_kt0	0x22e	0xFFD0022E	0x0917	0	0	0
NDB	d_k_x1_kt1	0x230	0xFFD00230	0x0918	0	0	0
NDB	d_dur_kb	0x232	0xFFD00232	0x0919	0	0	0
NDB	d_shiftdl	0x234	0xFFD00234	0x091a	0	0	0
NDB	d_shiftul	0x236	0xFFD00236	0x091b	0	0	0
NDB	d_aec_ctrl	0x238	0xFFD00238	0x091c	10625, bf28	0	0
NDB	d_es_level_api	0x23a	0xFFD0023A	0x091d	0	0	0
NDB	d_mu_api	0x23c	0xFFD0023C	0x091e	0	0	0
NDB	d_melo_osc_ctrl	0x23e	0xFFD0023E	0x091f	0	0	0
NDB	d_melo_osc_addr	0x240	0xFFD00240	0x0920	0	0	0
NDB	a_melo_note0[4]	0x242	0xFFD00242	0x0921..0x0924	0	0	0
NDB	a_melo_note1[4]	0x24a	0xFFD0024A	0x0925..0x0928	0	0	0
NDB	a_melo_note2[4]	0x252	0xFFD00252	0x0929..0x092c	0	0	0
NDB	a_melo_note3[4]	0x25a	0xFFD0025A	0x092d..0x0930	0	0	0
NDB	a_melo_note4[4]	0x262	0xFFD00262	0x0931..0x0934	0	0	0
NDB	a_melo_note5[4]	0x26a	0xFFD0026A	0x0935..0x0938	0	0	0
NDB	a_melo_note6[4]	0x272	0xFFD00272	0x0939..0x093c	0	0	0
NDB	a_melo_note7[4]	0x27a	0xFFD0027A	0x093d..0x0940	0	0	0
NDB	d_melody_select	0x282	0xFFD00282	0x0941	0	0	0
NDB	d_sr_status	0x28a	0xFFD0028A	0x0945	0	0	0
NDB	d_sr_param	0x28c	0xFFD0028C	0x0946	0	0	0
NDB	d_sr_bit_exact_test	0x28e	0xFFD0028E	0x0947	0	0	0

zone	field	API byte	ARM	DSP word	ROM reads (n, 1st PC)	ROM writes (n, 1st PC)	ARM changes (n, values)
NDB	d_sr_nb_words	0x290	0xFFD00290	0x0948	0	0	0
NDB	d_sr_db_level	0x292	0xFFD00292	0x0949	0	0	0
NDB	d_sr_db_noise	0x294	0xFFD00294	0x094a	0	0	0
NDB	d_sr_mod_size	0x296	0xFFD00296	0x094b	0	0	0
NDB	a_n_best_words[8]	0x298	0xFFD00298	0x094c..0x0941	0	0	0
NDB	a_n_best_score[8]	0x2a0	0xFFD002A0	0x0950..0x0957	0	0	0
NDB	a_dd_1[22]	0x2b0	0xFFD002B0	0x0958..0x0960	0	0	0
NDB	a_du_1[22]	0x2dc	0xFFD002DC	0x096e..0x0983	32416, a925	2656, a925	3815, 8000:2488
NDB	d_v42b_negot	0x308	0xFFD00308	0x0984	0	0	0
NDB	d_v42b_negot	0x30a	0xFFD0030A	0x0985	0	0	0
NDB	d_v42b_contro	0x30c	0xFFD0030C	0x0986	0	0	0
NDB	d_v42b_ratio_i	0x30e	0xFFD0030E	0x0987	0	0	0
NDB	d_mcu_contro	0x310	0xFFD00310	0x0988	0	0	0
NDB	d_mcu_contro	0x312	0xFFD00312	0x0989	0	0	0
NDB	d_background_enable	0x314	0xFFD00314	0x098a	94035, dde8	0	0
NDB	d_background_abort	0x316	0xFFD00316	0x098b	0	0	0
NDB	d_background_state	0x318	0xFFD00318	0x098c	0	59564, dde8	0
NDB	d_max_background	0x31a	0xFFD0031A	0x098d	0	0	0
NDB	a_background_tasks[16]	0x31c	0xFFD0031C	0x098e..0x099d	0	0	0
NDB	a_back_task_to[16]	0x33c	0xFFD0033C	0x099e..0x09ad	0	0	0
NDB	d_gea_mode_ovly	0x35c	0xFFD0035C	0x09ae	0	0	0
NDB	a_gea_kc_ovly[4]	0x35e	0xFFD0035E	0x09af..0x09b2	0	0	0
NDB	d_thr_usf_dem	0x374	0xFFD00374	0x09ba	0	0	0
NDB	d_a5mode	0x376	0xFFD00376	0x09bb	37225, b138	0	15, 0001:8,0000:7
NDB	d_sched_mode_ovly	0x378	0xFFD00378	0x09bc	13, a541	0	0
NDB	a_ramp[16]	0x384	0xFFD00384	0x09c2..0x09d1	190160, b642	0	166, 4fd4:6,8fd4:1,+
NDB	a_cd[15]	0x3a4	0xFFD003A4	0x09d2..0x09e0	4056, 9718	8736, 96dd	220, 0040:110
NDB	a_fd[15]	0x3c2	0xFFD003C2	0x09e1..0x09ef	11180, b10d	3634, a72c	70, 0040:35
NDB	a_dd_0[22]	0x3e0	0xFFD003E0	0x09f0..0x0a05	97541, a38e	65580, 928a	5308, 0000:2654
NDB	a_cu[15]	0x40c	0xFFD0040C	0x0a06..0x0a14	4095, aa5a	315, aa5a	2239, 8000:315
NDB	a_fu[15]	0x42a	0xFFD0042A	0x0a15..0x0a23	8292, a8bd	2656, a928	109, 8000:26
NDB	a_du_0[22]	0x448	0xFFD00448	0x0a24..0x0a39	65781, ad0d	105565, ad90	0
NDB	d_rach	0x474	0xFFD00474	0x0a3a	0	0	0
NDB	a_kc[4]	0x476	0xFFD00476	0x0a3b..0x0a3e	147204, b145	0	8, 8c00:1,f000:1
NDB	d_ra_conf	0x47e	0xFFD0047E	0x0a3f	2629, b124	0	0
NDB	d_ra_act	0x480	0xFFD00480	0x0a40	0	0	0
NDB	d_ra_test	0x482	0xFFD00482	0x0a41	0	0	0
NDB	d_ra_statu	0x484	0xFFD00484	0x0a42	0	0	0
NDB	d_ra_statd	0x486	0xFFD00486	0x0a43	0	0	0
NDB	d_fax	0x488	0xFFD00488	0x0a44	0	0	0
NDB	a_data_buf_ul[28]	0x48a	0xFFD0048A	0x0a45..0x0a59	0	0	0
NDB	a_data_buf_dl[37]	0x4b4	0xFFD004B4	0x0a5a..0x0a7e	0	0	0
NDB	a_sr_holes1[105]	0x862	0xFFD00862	0x0c31..0x0cc1	133372, a5e0	0	0
NDB	d_cport_init	0x984	0xFFD00984	0x0cc2	0	0	0

zone	field	API byte	ARM	DSP word	ROM reads (n, 1st PC)	ROM writes (n, 1st PC)	ARM changes (n, values)
NDB	d_cport_ctrl	0x986	0xFFD00986	0x0cc3	0	0	0
NDB	a_cport_cfr[2]	0x988	0xFFD00988	0x0cc4..0x0cc5	0	0	0
NDB	d_cport_tcl	0x98c	0xFFD0098C	0x0cc6	0	0	0
NDB	d_cport_tdat	0x98e	0xFFD0098E	0x0cc7	0	0	0
NDB	d_cport_tvs	0x990	0xFFD00990	0x0cc8	0	0	0
NDB	d_cport_status	0x992	0xFFD00992	0x0cc9	0	0	0
NDB	d_cport_reg_value	0x994	0xFFD00994	0x0cca	0	0	0
NDB	a_cport_holes[196]	0x996	0xFFD00996	0x0ccb..0x10bd	14510066, 81ad	1096940, 819b	0
NDB	a_model[1041]	0x117c	0xFFD0117C	0x10be..0x14ce	0	0	0
NDB	a_amr_config[4]	0x19ca	0xFFD019CA	0x14e5..0x14e8	0	0	0
NDB	a_ratscch_ul[6]	0x19d2	0xFFD019D2	0x14e9..0x14ee	0	0	0
NDB	a_ratscch_dl[6]	0x19de	0xFFD019DE	0x14ef..0x14f4	0	0	0
NDB	d_amr_snr_est	0x19ea	0xFFD019EA	0x14f5	0	0	0
NDB	d_thr_onset_afs	0x19ee	0xFFD019EE	0x14f7	0	0	0
NDB	d_thr_sid_first_afs	0x19f0	0xFFD019F0	0x14f8	0	0	0
NDB	d_thr_ratscch_afs	0x19f2	0xFFD019F2	0x14f9	0	0	0
NDB	d_thr_update_afs	0x19f4	0xFFD019F4	0x14fa	0	0	0
NDB	d_thr_onset_ahs	0x19f6	0xFFD019F6	0x14fb	0	0	0
NDB	d_thr_sid_ahs	0x19f8	0xFFD019F8	0x14fc	0	0	0
NDB	d_thr_ratscch_ahs	0x19fa	0xFFD019FA	0x14fd	0	0	0
NDB	d_thr_sp_dgf	0x19fc	0xFFD019FC	0x14fe	0	0	0
NDB	d_thr_soft_bits	0x19fe	0xFFD019FE	0x14ff	0	0	0
PARAM	d_transfer_rate	0x862	0xFFD00862	0x0c31	0	0	0
PARAM	d_lat_mcu_bridge	0x864	0xFFD00864	0x0c32	43808, a5e0	0	0
PARAM	d_lat_mcu_hscsd	0x866	0xFFD00866	0x0c33	0	0	0
PARAM	d_lat_mcu_bcch_fast_access	0x868	0xFFD00868	0x0c34	0	0	0
PARAM	d_lat_dsp_afs	0x86a	0xFFD0086A	0x0c35	0	0	0
PARAM	d_gprs_install_address	0x86c	0xFFD0086C	0x0c36	0	0	0
PARAM	d_misc_config	0x86e	0xFFD0086E	0x0c37	34471, a684	0	0
PARAM	d_cn_sw_workaround	0x870	0xFFD00870	0x0c38	0	0	0
PARAM	d_hole2_param[4]	0x872	0xFFD00872	0x0c39..0x0c3c	0	0	0
PARAM	d_fb_margin_beta	0x87a	0xFFD0087A	0x0c3d	29, 794b	0	0
PARAM	d_fb_margin_ahs	0x87c	0xFFD0087C	0x0c3e	29, 7956	0	0
PARAM	d_nsubb_idle	0x87e	0xFFD0087E	0x0c3f	36, b316	0	0
PARAM	d_nsubb_dedic	0x880	0xFFD00880	0x0c40	0	0	0
PARAM	d_fb_thr_det_afs	0x882	0xFFD00882	0x0c41	6224, 78ef	0	0
PARAM	d_fb_thr_det_ahs	0x884	0xFFD00884	0x0c42	0	0	0
PARAM	d_dc_off_thres	0x886	0xFFD00886	0x0c43	11875, 7ea1	0	0
PARAM	d_dummy_thres	0x888	0xFFD00888	0x0c44	0	0	0
PARAM	d_dem_pond_gesa	0x88a	0xFFD0088A	0x0c45	11875, 8154	0	0
PARAM	d_dem_pond_ges	0x88c	0xFFD0088C	0x0c46	11875, 815a	0	0
PARAM	d_maccthresh	0x88e	0xFFD0088E	0x0c47	0	0	0
PARAM	d_mldt	0x890	0xFFD00890	0x0c48	4, e59f	0	0
PARAM	d_maccthresh	0x892	0xFFD00892	0x0c49	4, e5ad	0	0
PARAM	d_gu	0x894	0xFFD00894	0x0c4a	4, e5b8	0	0
PARAM	d_go	0x896	0xFFD00896	0x0c4b	4, e5bc	0	0
PARAM	d_attmax	0x898	0xFFD00898	0x0c4c	0	0	0
PARAM	d_sm	0x89a	0xFFD0089A	0x0c4d	4, e5c4	0	0
PARAM	d_b	0x89c	0xFFD0089C	0x0c4e	0	0	0
PARAM	d_v42b_switch_hyst	0x89e	0xFFD0089E	0x0c4f	0	0	0
PARAM	d_v42b_switch_n	0x8a0	0xFFD008A0	0x0c50	0	0	0

zone	field	API byte	ARM	DSP word	ROM reads (n, 1st PC)	ROM writes (n, 1st PC)	ARM changes (n, values)
PARAM	d_v42b_switch_max	0x8a2	0xFFD008A2	0x0c51	0	0	0
PARAM	d_v42b_reset_delay	0x8a4	0xFFD008A4	0x0c52	0	0	0
PARAM	d_ldT_hr	0x8a6	0xFFD008A6	0x0c53	0	0	0
PARAM	d_maccthresh_hr	0x8a8	0xFFD008A8	0x0c54	0	0	0
PARAM	d_maccthresh_hr	0x8aa	0xFFD008AA	0x0c55	0	0	0
PARAM	d_gu_hr	0x8ac	0xFFD008AC	0x0c56	0	0	0
PARAM	d_go_hr	0x8ae	0xFFD008AE	0x0c57	0	0	0
PARAM	d_b_hr	0x8b0	0xFFD008B0	0x0c58	0	0	0
PARAM	d_sm_hr	0x8b2	0xFFD008B2	0x0c59	0	0	0
PARAM	d_attmax_hr	0x8b4	0xFFD008B4	0x0c5a	0	0	0
PARAM	c_mldt_efr	0x8b6	0xFFD008B6	0x0c5b	0	0	0
PARAM	c_maccthresh_efr	0x8b8	0xFFD008B8	0x0c5c	0	0	0
PARAM	c_maccthresh_efr	0x8ba	0xFFD008BA	0x0c5d	0	0	0
PARAM	c_gu_efr	0x8bc	0xFFD008BC	0x0c5e	0	0	0
PARAM	c_go_efr	0x8be	0xFFD008BE	0x0c5f	0	0	0
PARAM	c_b_efr	0x8c0	0xFFD008C0	0x0c60	0	0	0
PARAM	c_sm_efr	0x8c2	0xFFD008C2	0x0c61	0	0	0
PARAM	c_attmax_efr	0x8c4	0xFFD008C4	0x0c62	0	0	0
PARAM	d_sd_min_thr_tchfs	0x8c6	0xFFD008C6	0x0c63	2619, a31d	0	0
PARAM	d_ma_min_thr_tchfs	0x8c8	0xFFD008C8	0x0c64	5238, a33b	0	0
PARAM	d_md_max_thr_tchfs	0x8ca	0xFFD008CA	0x0c65	0	0	0
PARAM	d_md1_max_thr_tchfs	0x8cc	0xFFD008CC	0x0c66	2619, a356	0	0
PARAM	d_sd_min_thr_tchhs	0x8ce	0xFFD008CE	0x0c67	0	0	0
PARAM	d_ma_min_thr_tchhs	0x8d0	0xFFD008D0	0x0c68	0	0	0
PARAM	d_sd_av_thr_tchhs	0x8d2	0xFFD008D2	0x0c69	0	0	0
PARAM	d_md_max_thr_tchhs	0x8d4	0xFFD008D4	0x0c6a	0	0	0
PARAM	d_md1_max_thr_tchhs	0x8d6	0xFFD008D6	0x0c6b	0	0	0
PARAM	d_sd_min_thr_tchefs	0x8d8	0xFFD008D8	0x0c6c	0	0	0
PARAM	d_ma_min_thr_tchefs	0x8da	0xFFD008DA	0x0c6d	0	0	0
PARAM	d_md_max_thr_tchefs	0x8dc	0xFFD008DC	0x0c6e	0	0	0
PARAM	d_md1_max_thr_tchefs	0x8de	0xFFD008DE	0x0c6f	0	0	0
PARAM	d_wed_fil_init	0x8e0	0xFFD008E0	0x0c70	0	0	0
PARAM	d_wed_fil_tc	0x8e2	0xFFD008E2	0x0c71	0	0	0
PARAM	d_x_min	0x8e4	0xFFD008E4	0x0c72	0	0	0
PARAM	d_x_max	0x8e6	0xFFD008E6	0x0c73	0	0	0
PARAM	d_slope	0x8e8	0xFFD008E8	0x0c74	0	0	0
PARAM	d_y_min	0x8ea	0xFFD008EA	0x0c75	0	0	0
PARAM	d_y_max	0x8ec	0xFFD008EC	0x0c76	0	0	0
PARAM	d_wed_diff_threshold	0x8ee	0xFFD008EE	0x0c77	0	0	0
PARAM	d_mabfi_min_thr_tchhs	0x8f0	0xFFD008F0	0x0c78	0	0	0
PARAM	d_facch_thr	0x8f2	0xFFD008F2	0x0c79	2654, 91e3	0	0
PARAM	d_max_ovsp_thr	0x8f4	0xFFD008F4	0x0c7a	0	0	0
PARAM	d_sync_thres	0x8f6	0xFFD008F6	0x0c7b	0	0	0
PARAM	d_idle_thres	0x8f8	0xFFD008F8	0x0c7c	0	0	0
PARAM	d_m1_thres	0x8fa	0xFFD008FA	0x0c7d	0	0	0
PARAM	d_max_ovsp_cnt	0x8fc	0xFFD008FC	0x0c7e	0	0	0
PARAM	d_gsm_bgd_gain	0x8fe	0xFFD008FE	0x0c7f	0	0	0
PARAM	a_fir_holes[4]	0x900	0xFFD00900	0x0c80..0x0c83	0	0	0
PARAM	a_fir31_uplink[30]	0x908	0xFFD00908	0x0c84..0x0ca1	0	0	0
PARAM	a_fir31_downlink[31]	0x946	0xFFD00946	0x0ca3..0x0cc1	0	0	0

Bits (`include/calypso/l1_environment.h`, bit **positions**): B_BLUD 15 (block present), B_AF 14, B_EMPTY_BLOCK 10, B_ECRC 9, B_SCH_CRC 8, B_FIRE1 6, B_FIRE0 5, B_BFI 2, B_UFI 0; d_dsp_page: B_GSM_PAGE 1«0, B_GSM_TASK 1«1 (masks); d_ctrl_system: B_TSQ 0..2, B_BCCH_FREQ_IND 3, B_TASK_ABORT 15; d_ctrl_tch: B_CHAN_MODE 0, B_CHAN_TYPE 4, B_RESET_SACCH 6, B_VOCODER_ON 7, B_SYNC_TCH_UL 8, B_SYNC_TCH_DL 9,

B_STOP_TCH_UL 10, B_STOP_TCH_DL 11, B_TCH_LOOP 12, B_SUBCHANNEL 15. Tasks: NO 0, FB 5, SB 6, TCH_FB 8, TCH_SB 9, RACH 10, AUL 11, DUL 12, TCHT 13, TCHA 14, NBN 17, EBN 18, NBS 19, EBS 20, NP 21, EP 22, ALLC 24, CB 25, DDL 26, ADL 27, TCHD 28, CHECKSUM 33 (bit 15 = I/Q inversion, `dsp_task_iq_swap`).

PARAM (values of `dsp_params`, read from the ELF with `print/x dsp_params`): d_transfer_rate 0x6666, d_lat_mcu_bridge 15, d_lat_mcu_hom2sam 12, d_lat_mcu_bef_fast_access 5, d_lat_dsp_after_sam 4, d_gprs_install_address 0x7002, d_misc_config 1, d_cn_sw_workaround 0xe, d_fb_margin_beg 24, d_fb_margin_end 22, d_nsubb_idle 296, d_nsubb_dedic 30, d_fb_thr_det_iacq 0x3333, d_fb_thr_det_track 0x28f6, d_dc_off_thres 0x7fff, d_dummy_thres 0x4400, d_dem_pond_gewl 0x6800, d_dem_pond_red 0x4eb8, d_maccthresh1 0x1ec0, d_mldt -4, … (rest: `firmware/calypso/dsp_params.c`).

2.6.1.4 3. Interface behaviour (replay)

- **d_dsp_page (0x08d4)**: the ARM alternates 0x0002/0x0003; the ROM reads it (1st PC 0xa51c) and sees each change at the next tick.
- **a_cu[0] (0x0a06)**: the ARM sets B_BLUD (0x8000); the ROM reads it and rewrites it (PC 0xaa5a): the ROM consumes B_BLUD (`montant.c` clears it the same way).
- **a_du_1[0] (0x096e)**: the ARM sets B_BLUD for TCH/F uplink speech; the ROM reads and writes it (PC 0xa925). **a_du_0 (0x0a24..)**: never changed by the ARM, but read and written by the ROM. See inconsistency 6.
- **a_cd[0] (0x09d2)**: written by the ROM; the ARM resets it to 0x0040 (B_FIRE1, `prim_tch.c`).
- **a_dd_0[0] (0x09f0)**: read and written by the ROM; the ARM clears it after reading.
- **d_version_number1/2 (0x08da/0x08db)**: written at boot only.

2.6.1.5 4. DSP internal memory

address	meaning	ROM reads (n, 1st PC)	ROM writes (n, 1st PC)	source
0x4280..0x429f	interleaved xCCH uplink buffer, 4 bursts x 8 words, circular BK=32	1578, 8dbd	1579, 8869	replay
0x3f8a..0x3f91	transmitted uplink burst (114 bits + hl/hu in the 2 low bits of word 7), copied by PROM0 0x8900..0x891b	1441, 85d3	13324, 8916	replay
0x3f9b..0x3fa2	uplink A5 keystream XORed onto 0x3f8a (PROM0 0x85d1)	1441, 85d4	37225, b185	`src/pont.c`
0x3f93..0x3f9a	downlink A5 keystream	11875, 8575	37225, b185	replay
0x3f92	bit 0x1000 = A5 keystream ready	168751, b04c	99947, b04c	replay
0x3d9b	read pointer of the 0x4280 circular buffer	1573, 8bd5	1577, 8820	`src/pont.c`
0x2bfa..	natural-order coded block / work buffer	305283, a2c4	61427, 8ba8	replay
0x2bf8	class 1a parity flag	338033, 8df0	84547, 8daf	replay
0x3d90	CHED quality flags (0x01 parity, 0x02 MA, 0x08 SD-min, 0x10 …)	37455, 8758	29527, 8758	replay
0x2d42.. (50 words)	SD values	16195, 9abe	16164, 9aad	replay
0x2c00..0x2c1f	Viterbi metrics	2679012, a2c4	398814, 8ba8	replay

address	meaning	ROM reads (n, 1st PC)	ROM writes (n, 1st PC)	source
0x3fad / 0x3fae	mode bits (0x3fad bit 15 = FB dispatcher lock, PC 0x8753)	343904 / 92658	27812 / 22517	`c54x_mem.c` PROBE-3FAD-GATE
0x3d89	write pointer of the 232-word circular buffer at 0x4d00 (init 0x90c0)	13281, 9169	10634, 90c0	`c54x_mem.c`
0x43d8	current handler slot (`ld *(0x43d8),A ; cala A` at 0xb01c)	13218, b01c	0	`c54x_mem.c` HANDLER-WATCH

2.6.1.6 5. I/O ports (PORTR/PORTW 0x74xx/0x75xx of the replay)

port	dir	n	1st PC	role	source
0x0002	W	26436	b572	RIF SPCX (transmit control)	`calypso_rif.h`
0x0003	R/W	81949 / 88202	a675 / a67b	RIF SPCR (receive control)	`calypso_rif.h`
0x2800	W	147204	b142	A5: command (0 stop, 0x16/0x17)	`calypso_a5.c`
0x2801, 0x2802	R	36801	b1a0, b16c	A5: trace, status	`calypso_a5.c`
0x2803..0x2806	W	36801	b145..	A5: Kc (= a_kc[0..3])	`calypso_a5.c`
0x2807, 0x2808	W	36801	b151, b154	A5: a_a5fn[0] (T3«5|T2), a_a5fn[1] (T1)	`calypso_a5.c`
0x2809..0x2818	R	36801	b1ab..	A5: keystream (downlink then uplink)	`calypso_a5.c`
0xf900	W	232373	a6c3	API Control (CAL207 7.2.2)	`calypso_xio.c`
0xfa01	W	18809	a658	DSP INTH	`calypso_xio.c`
0xfc24..0xfc28	R/W	~12500..31315	a5e8..	RHEA DMA window (AAD 0x099c = buffer 0x0cce)	`calypso_rhea_dma.c`

Interrupt vectors (`calypso_c54x.h`): TPU frame vec 28 / IMR bit 12, API vec 25 / bit 9, DMA vec 30 / bit 14, RIF RX vec 16 / bit 0, A5 (CRYPT) vec 27 / bit 11.

2.6.1.7 6. Known inconsistencies (layer1 / qosmo / c54x_exe)

1. **`calypso_bsp_tx_burst()` reads `data[0x0900 + i]`** (qosmo `calypso_bsp.c`): 0x0900 is inside the NDB (a_sch26[2..]), not a transmit buffer. The ROM's transmitted burst is at 0x3f8a (c54x_exe takes it there in `pont.c tx_rom_publier`).
2. **Same name, different units**: `NDB_D_DSP_PAGE`/`NDB_D_FB_DET` are **byte** offsets in the NDB (0x000/0x048, `calypso_api.h`) and absolute **DSP word** addresses (0x08D4/0x08F8, `l1-dsp/calypso_fbsb.h`). A unit including both headers would get a silent redefinition.
3. **B_BLUD**: bit position (15) in layer1, mask (1u«15) in qosmo, same name.
4. **CALYPSO_API_SIZE = 64 KiB** (32768 words, `calypso_api.h`) versus 16 KiB in reality (API_SIZE 0x2000 words, `firmware/calypso/dsp.c`); the ARM never writes beyond 0x2000.

5. **d_ctrl_tch**: `dsp.c` documents b_chan_type on bits 4..7, `dsp_api.h` on 4..5 with bit 6 = SACCH reset and bit 7 = vocoder ON; `dsp_load_tch_param(chan_type=SDCCH_8=4)` therefore sets bit 6. Values actually written: not analysed yet.
6. **TCH/F uplink speech in a_du_1**: `prim_tch.c` uses `tch_sub ? a_du_0 : a_du_1` (the downlink uses `tch_sub ? a_dd_1 : a_dd_0`); the ARM writes 0x096e (a_du_1), the ROM reads it but also reads and writes a_du_0 (0x0a24), which the ARM never touches. `montant.c` reads NDB_A_DU_1, consistent with layer1. Role of a_du_0 for the ROM: to be confirmed.
7. Sizes: sizeof differs between ARM and host (T_DB_MCU_TO_DSP 36/34, T_NDB 6356, T_PARAM 292: tail padding); offsets are identical.
8. Cross-checked against the ELF and correct: all WP_/*RP*_/NDB_* of `calypso_api.h`, API_VERSION/API_VERSION API_BL_STATUS 0x0FFE, `calypso_fbsb.h` (d_dsp_page, d_dsp_state 0x08E2, d_fb_det, d_fb_mode, a_sync_demod[0..3]), `calypso_arm2dsp.c` A2D_DSP_PAGE_OFF 0x01A8, `calypso_bsp.c` D_RACH_DEFAULT_OF 0x023A, `src/rejouer.c` W_PAGE/R_PAGE/NDB/PARAM/BL_*, `tools/rejeu_banc.c` (a_cd 0x1FC, a_fd 0x21A, a_dd_0 0x238, a_kc 0x2CE, d_a5mode 0x1CE), `src/montant.c` NDB_D_TCH_MODE 0x006, `dsp.c` SC_CHKSUM_VER (0x08DB = d_version_number2).

2.6.1.8 7. TODO

- Finish `tools/layer1_tester.py` (only extraction is written): automatic cross-check, static scan of the layer1 functions, PASS/FAIL per field and function, generation of this page.
- Analyse the ARM values (d_ctrl_tch, d_fn, a_a5fn, W page vs d_dsp_page, W->R echo).

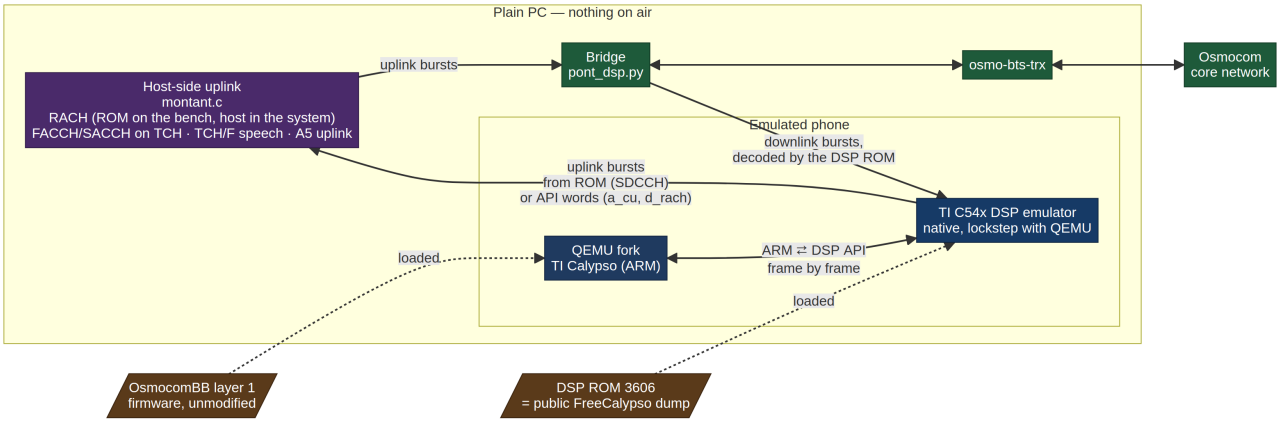
2.7 /opt/GSM/c54x_exe/docs/emulating-the-baseband.md

8368 octets, 124 lignes

2.7.1 Emulating the baseband

A complete GSM call on an ordinary PC, where the phone's real layer 1 firmware and the real baseband DSP code take part, down to the lowest layers of 2G. I forked QEMU to emulate the TI Calypso platform so that OsmocomBB's layer 1 runs on it, and I wrote an emulator for the TI C54x DSP that layer 1 talks to. Nothing goes on air: no SDR, no radio hardware, no spectrum licence. The baseband is the most closed part of a phone. Ever wanted to watch what a 2G DSP actually does, step by step, without any hardware? Now you can.

On the phone side, the QEMU Calypso machine loads the unmodified OsmocomBB layer 1 firmware as the real ARM would, and a native C54x emulator runs the DSP from its mask ROM — identical, word for word, to the public FreeCalypso dump of ROM version 3606, which is where to get it; nothing is redistributed here.



Layer 1 drives the DSP through its shared API, as on real hardware. In the downlink, the DSP ROM itself does the work: frequency burst detection, synchronisation, control channel and speech decoding, A5 deciphering. In the uplink, SDCCH is produced by the ROM too, after four bugs in the emulated core were fixed; RACH, FACCH/SACCH on traffic channels, uplink speech and uplink A5 are still encoded on the host side in the full system, and moving them into the DSP is the current work (the RACH already is, on the bench: see below). A bridge connects the phone to osmo-bts-trx, following the design of fixeria's trxcon, and an Osmocom core network handles the call on the other side. An end-to-end test bench checks the whole chain, from boot to attach, SMS, USSD, a voice call and a test tone recovered through it. An experimental mode runs layer 1 without the

TI ROM, with host-side signal processing (gr-gsm, libosmocoding): not a free baseband yet, but the layer above it running without the vendor's code.

One number to keep in mind while reading the rest: of the 230 worked examples of TI's instruction set manual (SPRU172C) that the bench extracts, the emulated C54x core passes 150, fails 58, and 22 cannot be assembled by the harness. Every « the ROM does X » below is conditional on the core executing the instructions of that path correctly; the two bugs found on 2026-10-04 were found exactly that way.

2.7.1.1 Three stages, as they appear in the code

1. **Layer 1 in QEMU.** The QEMU fork runs an unmodified `layer1.highram` on the Calypso machine (a patched build that passed bad speech frames at zero gain was used from 2026-10-03 to 2026-10-04 and has been removed). The API RAM is the real one (`/dev/shm/calypso_api_ram`), and the ARM DSP dialogue happens frame by frame.
2. **The gr-gsm shunt.** The `grgsm` mode of `run.sh`, the `shunt_*` traces and `l1-grgsm/`: a « host path » in which gr-gsm did FB/SB detection inside QEMU, the libosmocoding bridge did channel decoding and the bridge did A5. The DSP was bypassed; the host rebuilt the signal processing.
3. **The DSP that works.** `c54x_exe`: the TI mask ROM executed on the emulated C54x core, in lockstep with QEMU. State on 2026-10-03, from the bench's coverage table (one line per signal-processing step): 6 steps done by the ROM (FB, SCH, control channels, FACCH, speech decoding, A5 downlink), 7 by the host (IMMEDIATE ASSIGNMENT reference retouched — the table counts it as a host step, the ROM pattern being absent by construction —, RACH, A5 uplink, FACCH uplink, SACCH on TCH uplink, uplink speech, vocoder), 1 mixed (SDCCH uplink: 134 blocks by the ROM, 1 by the host).

2.7.1.2 What "processor fidelity" means now

Fidelity is no longer measured as « does the phone register » but as « how many API RAM words and how many bits at the antenna are produced by the ROM itself ». The capture point sets the fidelity:

- **Capturing at `0x3f8a`** (what `tx_rom_publier` does today) takes the coded bits *before* the modulator. The host still has to add the training sequence, synchronisation and tail bits, and redo A5 at the right frame number.
- **Capturing the TSP script** (`0x3cbb..`, marker `0x1c0a` = TOGBR2, then 16 BULDATA words of 10 bits) takes the stream *the ABB would receive on silicon*, with training sequence, tails, I/Q inversion and A5 already applied. The host then only converts ABB words into 148 bits and places them at the right fn/tn. In the ROM this capture point serves every burst type (NB, RACH, TCH): one pipe, the ROM decides everything.

That second capture point did not exist in emulation until 2026-10-04, and not because of the host: the ROM wrote the burst, the core did not know how to execute two of the instructions on the way. The emitter loop at PROM0 `0x8608` begins with `cmpr eq,ar2 ; rc tc` and the core had no `CMPR` at all (a no-op, so the loop returned at once on a stale flag); once that was fixed, the 16 words came out as `0xff06` sixteen times, because `LD src, ASM` ignored its shift and the bit accumulator never moved. With both fixed, the bench's new `rach` test shows the ROM executing the RACH task (`0xb6fd`, then `0x85a2`), writing its 36 coded bits to `0x3f8a`, and the emitter producing the exact 05.02 access burst in the script — extended tail `00111010`, the 41-bit synchronisation sequence copied from the data ROM at `0xa0d9`, the 36 bits, three tail bits, guard — bit for bit what libosmocoding's `gsm0503_rach_ext_encode` gives, 5 attempts out of 5. On the bench only: in the full system `montant.c` still erases `d_task_ra` before the ROM can see it, and the bridge still codes the RACH. Connecting the pipe is the next step, and it is now a plumbing step.

2.7.1.3 The hierarchy for what comes next

- **Before any new pipe: the 58 failing SPRU172C examples.** They are the manufacturer's own test vectors; each is a candidate for the next « the ROM computes wrongly ». Two were found by symptom this week (CMPR, LD src,ASM); the list finds them by construction.
- **Then the single pipe:** make the TSP script the source of all uplink bursts (RACH, SDCCH, TCH, SACCH, FACCH), and transmit at the ROM's frame number so that its A5 is the right one — after reading in the ROM which frame counter it uses for the uplink stream, which has not been done.
- **Cases where the ROM computes wrongly or not at all in the system:** downlink SACCH/8 often failing its FIRE check (the 2026-10-03 note says « ~80 % », without a count), `B_BFI` on speech, and on the bench `nb` and `fb0`/`fb1`. Instruction-level debugging, like the four bugs behind the SDCCH uplink.

- **The most distant fidelity:** the vocoder in the ROM, which means modelling the audio path from the DSP to the ABB.

In other words, the DSP has stopped being an obstacle to work around and has become the reference: whenever the host still does something, it is either because it has not yet been given the ROM's pipe, or because the core still has a bug. Both are measurable, and that is what is being measured — the evidence for every sentence above is in `docs/CLAIMS-2026-10-04.md`.

2.8 /opt/GSM/c54x_exe/tools/isa_tests.txt

32706 octets, 2208 lignes → 2206 lignes (1 groupes compactés)

```
T 1 ABDST Xmem, Ymem | ABDST *AR3+, *AR4+
W e39a
B A ffabcd0000
B AR3 100
B AR4 200
B B 0
B FRCT 0
B M 0100 0055
B M 0200 00aa
A A ffffab0000
A AR3 101
A AR4 201
A B 5433
A FRCT 0
A M 0100 0055
A M 0200 00aa
E
T 2 ABS src [, dst ] | ABS A, B
W f585
B A fffffffcb
B B fffffffc18
A A fffffffcb
A B 35
E
T 3 ABS src [, dst ] | ABS A
W f485
B A 312345678
B OVM 1
A A 7fffffff
A OVM 1
E
T 4 ABS src [, dst ] | ABS A
W f485
B A 312345678
B OVM 0
A A 312345678
A OVM 0
E
T 5 6: Class 7 (see page 3-12) | ADD *AR3+, 14, A
W 909e
B A 1200
B AR3 100
B C 1
B SXM 1
B M 0100 1500
A A 5401200
A AR3 101
A C 0
A SXM 1
A M 0100 1500
E
T 6 6: Class 7 (see page 3-12) | ADD A, –8, B
```

```
W f518
B A 1200
B B 1800
B C 1
A A 1200
A B 1812
A C 0
E
T 7 6: Class 7 (see page 3-12) | ADD #4568h, 8, A, B
W f108 4568
B A 1200
B B 1800
B C 1
A A 1200
A B 457a00
A C 0
E
T 9 ADDC Smem, src | ADDC *+AR2(5), A
W 06ea 0005
B A 13
B AR2 100
B C 1
B M 0105 0004
A A 18
A AR2 105
A C 0
A M 0105 0004
E
T 10 ADDM #lk, Smem | ADDM 0123Bh, *AR4+
W 6b94 123b
B AR4 100
B M 0100 0004
A AR4 101
A M 0100 123f
E
T 11 ADDM #lk, Smem | ADDM 0FFF8h, *AR4+
W 6b94 fff8
B AR4 100
B OVM 1
B SXM 1
B M 0100 8007
A AR4 101
A OVM 1
A SXM 1
A M 0100 8000
E
T 12 ADDS Smem, src | ADDS *AR2–, B
W 038a
B AR2 104
B B 3
B M 0104 f006
A AR2 103
A B f009
A C 0
A M 0104 f006
E
T 13 4: Class 1 (see page 3-3) | AND *AR3+, A
W 1893
B A ff1200
B AR3 100
B M 0100 1500
A A 1000
A AR3 101
A M 0100 1500
E
T 14 4: Class 1 (see page 3-3) | AND A, 3, B
```

```
W f183
B A 1200
B B 1800
A A 1200
A B 1000
E
T 15 ANDM #lk, Smem | ANDM #00FFh, *AR4+
W 6894 00ff
B AR4 100
B M 0100 0444
A AR4 101
A M 0100 0044
E
T 17 B[D] pmad | B 2000h
W f073 2000
B PC 1f45
A PC 2000
E
T 18 B[D] pmad | BD 1000h
W f273 1000
B PC 1f45
A PC 1000
E
T 19 BACC[D] src | BACC A
W f4e2
B A 3000
B PC 1f45
A A 3000
A PC 3000
E
T 20 BACC[D] src | BACCD B
W f7e2
B B 2000
B PC 1f45
A B 2000
A PC 2000
E
T 21 BANZ[D] pmad, Sind | BANZ 2000h, *AR3–
W 6c8b 2000
B AR3 5
B PC 1000
A AR3 4
A PC 2000
E
T 22 BANZ[D] pmad, Sind | BANZ 2000h, *AR3–
W 6c8b 2000
B AR3 0
B PC 1000
A AR3 ffff
A PC 1002
E
T 23 BANZ[D] pmad, Sind | BANZ 2000h, *AR3(–1)
W 6ce3 ffff 2000
B AR3 1
B PC 1000
A AR3 1
A PC 1003
E
T 24 BANZ[D] pmad, Sind | BANZD 2000h, *AR3–
W 6e8b 2000
B AR3 4
B PC 1000
A AR3 3
A PC 2000
E
T 25 BC[D] pmad, cond [, cond [, condÃ ]ã] | BC 2000h, AGT
```

```
W f846 2000
B A 53
B PC 1000
A A 53
A PC 2000
E
T 26 BC[D] pmad, cond [, cond [, condĂ ]ā] | BC 2000h, AGT
W f846 2000
B A fffffffffff
B PC 1000
A A fffffffffff
A PC 1002
E
T 27 BC[D] pmad, cond [, cond [, condĂ ]ā] | BCD 1000h, BOV
W fa78 1000
B OVB 1
B PC 3000
A OVB 1
A PC 1000
E
T 28 BC[D] pmad, cond [, cond [, condĂ ]ā] | BC 1000h, TC, NC, BIO
W f83b 1000
B C 1
B PC 3000
A C 1
A PC 3002
E
T 29 BIT Xmem, BITC | BIT *AR5+, 15-12; test bit 12
W 96b3
B AR5 100
B TC 0
B M 0100 7688
A AR5 101
A TC 1
A M 0100 7688
E
T 30 BITF Smem, #lk | BITF 5, 00FFh
W 6105 00ff
B DP 4
B M 0205 5400
A DP 4
A TC 0
A M 0205 5400
E
T 31 BITF Smem, #lk | BITF 5, 0800h
W 6105 0800
B DP 4
B M 0205 0f7f
A DP 4
A TC 1
A M 0205 0f7f
E
T 32 BITT Smem | BITT *AR7+0
W 34b7
B AR0 8
B AR7 100
B T c
B TC 0
B M 0100 0008
A AR0 8
A AR7 108
A T c
A TC 1
A M 0100 0008
E
T 33 CALA[D] src | CALA A
```

```
W f4e3
B A 3000
B PC 25
B SP 1111
B M 1110 4567
A A 3000
A PC 3000
A SP 1110
A M 1110 0026
E
T 34 CALA[D] src | CALAD B
W f7e3
B B 2000
B PC 25
B SP 1111
B M 1110 4567
A B 2000
A PC 2000
A SP 1110
A M 1110 0028
E
T 35 CALL[D] pmad | CALL 3333h
W f074 3333
B PC 25
B SP 1111
B M 1110 4567
A PC 3333
A SP 1110
A M 1110 0027
E
T 36 CALL[D] pmad | CALLD 1000h
W f274 1000
B PC 25
B SP 1111
B M 1110 4567
A PC 1000
A SP 1110
A M 1110 0029
E
T 37 CC[D] pmad, cond [, cond [, cond ] ] | CC 2222h, AGT
W f946 2222
B A 3000
B PC 25
B SP 1111
B M 1110 4567
A A 3000
A PC 2222
A SP 1110
A M 1110 0027
E
T 38 CC[D] pmad, cond [, cond [, cond ] ] | CCD 1000h, BOV
W fb78 1000
B OVB 1
B PC 25
B SP 1111
B M 1110 4567
A OVB 0
A PC 1000
A SP 1110
A M 1110 0029
E
T 39 CMPL src [, dst ] | CMPL A, B
W f593
B A fcdffaaeaa
B B 7899
A A fcdffaaeaa
```

```
A B 320055155
E
T 40 CMPM Smem, #lk | CMPM *AR4+, 0404h
W 6094 0404
B AR4 100
B TC 1
B M 0100 4444
A AR4 101
A TC 0
A M 0100 4444
E
S 41 CMPR CC, ARx | CMPR 2, AR4 | cmpr['OP_CC3', 'OP_ARX']: cond: '2'
T 42 CMPS src, Smem | CMPS A, *AR4+
W 8e94
B A 23457899
B AR4 100
B TC 0
B TRN 4444
B M 0100 0000
A A 23457899
A AR4 101
A TC 1
A TRN 8889
A M 0100 7899
E
T 43 DADD Lmem, src [, dst ] | DADD *AR3+, A, B
W 5193
B A 56788933
B AR3 100
B B 0
B C16 0
B M 0100 1534
B M 0101 3456
A A 56788933
A AR3 102
A B 6bacbd89
A C16 0
A M 0100 1534
A M 0101 3456
E
T 44 DADD Lmem, src [, dst ] | DADD *AR3–, A, B
W 518b
B A 56783933
B AR3 100
B B 0
B C16 1
B M 0100 1534
B M 0101 3456
A A 56783933
A AR3 fe
A B 6bac6d89
A C16 1
A M 0100 1534
A M 0101 3456
E
T 45 DADD Lmem, src [, dst ] | DADD *AR3–, A, B
W 518b
B A 56783933
B AR3 101
B B 0
B C16 0
B M 0100 1534
B M 0101 3456
A A 56783933
A AR3 ff
A B 8ace4e67
```

```
A C16 0
A M 0100 1534
A M 0101 3456
E
T 46 DADST Lmem, dst | DADST *AR3–, A
W 5a8b
B A 0
B AR3 100
B C16 1
B T 2345
B M 0100 1534
B M 0101 3456
A A 38791111
A AR3 fe
A C16 1
A T 2345
A M 0100 1534
A M 0101 3456
E
T 47 DADST Lmem, dst | DADST *AR3+, A
W 5a93
B A 0
B AR3 100
B C16 0
B T 2345
B M 0100 1534
B M 0101 3456
A A 3879579b
A AR3 102
A C16 0
A T 2345
A M 0100 1534
A M 0101 3456
E
T 48 DELAY Smem | DELAY *AR3
W 4d83
B AR3 100
B M 0100 6cac
B M 0101 0000
A AR3 100
A M 0100 6cac
A M 0101 6cac
E
T 49 DLD Lmem, dst | DLD *AR3+, B
W 5793
B AR3 100
B B 0
B M 0100 6cac
B M 0101 bd90
A AR3 102
A B 6cacbd90
A M 0100 6cac
A M 0101 bd90
E
T 50 DRSUB Lmem, src | DRSUB *AR3+, A
W 5893
B A 56788933
B AR3 100
B C16 0
B M 0100 1534
B M 0101 3456
A A ffbebbab23
A AR3 102
A C 0
A C16 0
A M 0100 1534
```

```
A M 0101 3456
E
T 51 DRSUB Lmem, src | DRSUB *AR3–, A
W 588b
B A 56783933
B AR3 100
B C 1
B C16 1
B M 0100 1534
B M 0101 3456
A A ffbebcfb23
A AR3 fe
A C 0
A C16 1
A M 0100 1534
A M 0101 3456
E
T 52 DSADT Lmem, dst | DSADT *AR3+, A
W 5e93
B A 0
B AR3 100
B C 0
B C16 0
B T 2345
B M 0100 1534
B M 0101 3456
A A fff1ef1111
A AR3 102
A C 0
A C16 0
A T 2345
A M 0100 1534
A M 0101 3456
E
T 53 DSADT Lmem, dst | DSADT *AR3–, A
W 5e8b
B A 0
B AR3 100
B C 0
B C16 1
B T 2345
B M 0100 1534
B M 0101 3456
A A fff1ef579b
A AR3 fe
A C 1
A C16 1
A T 2345
A M 0100 1534
A M 0101 3456
E
T 54 DST src, Lmem | DST B, *AR3+
W 4f93
B AR3 100
B B 6cacbd90
B M 0100 0000
B M 0101 0000
A AR3 102
A B 6cacbd90
A M 0100 6cac
A M 0101 bd90
E
T 55 DST src, Lmem | DST B, *AR3–
W 4f8b
B AR3 101
B B 6cacbd90
```

```
B M 0100 0000
B M 0101 0000
A AR3 ff
A B 6cacbd90
A M 0100 bd90
A M 0101 6cac
E
T 56 DSUB Lmem, src | DSUB *AR3+, A
W 5493
B A 56788933
B AR3 100
B C16 0
B M 0100 1534
B M 0101 3456
A A 414454dd
A AR3 102
A C16 0
A M 0100 1534
A M 0101 3456
E
T 57 DSUB Lmem, src | DSUB *AR3–, A
W 548b
B A 56783933
B AR3 100
B C 1
B C16 1
B M 0100 1534
B M 0101 3456
A A 414404dd
A AR3 fe
A C 1
A C16 1
A M 0100 1534
A M 0101 3456
E
T 58 DSUBT Lmem, dst | DSUBT *AR3+, A
W 5c93
B A 0
B AR3 100
B C16 0
B T 2345
B M 0100 1534
B M 0101 3456
A A fff1ef1111
A AR3 102
A C16 0
A T 2345
A M 0100 1534
A M 0101 3456
E
T 59 DSUBT Lmem, dst | DSUBT *AR3–, A
W 5c8b
B A 0
B AR3 100
B C16 1
B T 2345
B M 0100 1534
B M 0101 3456
A A fff1ef1111
A AR3 fe
A C16 1
A T 2345
A M 0100 1534
A M 0101 3456
E
T 60 EXP src | EXP A
```

```
W f48e
B A fffffffcb
B T 0
A A fffffffcb
A T 19
E
T 61 EXP src | EXP B
W f58e
B B 785432105
B T fffc
A B 785432105
A T fffc
E
T 62 FB[D] extpmad | FB 012000h
W f881 2000
B PC 1000
B XPC 0
A PC 2000
A XPC 1
E
T 63 FB[D] extpmad | FBD 7F1000h
W faff 1000
B PC 2000
B XPC 0
A PC 1000
A XPC 7f
E
T 64 FBACC[D] src | FBACC A
W f4e6
B A 13000
B PC 1000
B XPC 0
A A 13000
A PC 3000
A XPC 1
E
T 65 FBACC[D] src | FBACCD B
W f7e6
B B 7f2000
B XPC 1
A B 7f2000
A XPC 7f
E
T 66 FCALA[D] src | FCALA A
W f4e7
B A 7f3000
B PC 25
B SP 1111
B XPC 0
B M 110f 4567
B M 1110 4567
A A 7f3000
A PC 3000
A SP 110f
A XPC 7f
A M 110f 0000
A M 1110 0026
E
T 67 FCALA[D] src | FCALAD B
W f7e7
B B 202000
B PC 25
B SP 1111
B XPC 7f
B M 110f 4567
B M 1110 4567
```

```
A B 202000
A PC 2000
A SP 110f
A XPC 20
A M 110f 007f
A M 1110 0028
E
T 68 FCALL[D] extpmad | FCALL 013333h
W f981 3333
B PC 25
B SP 1111
B XPC 0
B M 110f 4567
B M 1110 4567
A PC 3333
A SP 110f
A XPC 1
A M 110f 0000
A M 1110 0027
E
T 69 FCALL[D] extpmad | FCALLD 301000h
W fbb0 1000
B PC 3001
B SP 1111
B XPC 7f
B M 110f 4567
B M 1110 4567
A PC 1000
A SP 110f
A XPC 30
A M 110f 007f
A M 1110 3005
E
S 70 FIRS Xmem, Ymem, pmad | FIRS *AR3+, *AR4+, COEFFS | firs['OP_Xmem', 'OP_Ymem', 'OP_pmad']: nombre: 'COEFFS'
T 71 FRAME K | FRAME 10h
W ee10
B SP 1000
A SP 1010
E
T 72 FRET[D] | FRET
W f4e4
B PC 2112
B SP 300
B XPC 1
B M 0300 0005
B M 0301 1000
A PC 1000
A SP 302
A XPC 5
A M 0300 0005
A M 0301 1000
E
T 73 FRETE[D] | FRETE
W f4e5
B PC 2112
B SP 300
B XPC 5
B M 0300 006e
B M 0301 0110
A PC 110
A SP 302
A XPC 6e
A M 0300 006e
A M 0301 0110
# valeur: ['xCxx']
E
```

```
T 77 INTR K | INTR 3
W f7c3
B INTM 0
B PC 25
B SP 1000
B M 0fff 9653
A INTM 1
A PC ff8c
A SP fff
A M 0fff 0026
E
T 78 4: Class 4B (see page 3-8) | LD *AR1, A
W 1081
B A 0
B AR1 200
B SXM 0
B M 0200 fedc
A A fedc
A AR1 200
A SXM 0
A M 0200 fedc
E
T 79 4: Class 4B (see page 3-8) | LD *AR1, A
W 1081
B A 0
B AR1 200
B SXM 1
B M 0200 fedc
A A ffffffedc
A AR1 200
A SXM 1
A M 0200 fedc
E
T 80 4: Class 4B (see page 3-8) | LD *AR1, TS, B
W 1581
B AR1 200
B B 0
B SXM 1
B T 8
B M 0200 fedc
A AR1 200
A B fffffedc00
A SXM 1
A T 8
A M 0200 fedc
E
T 81 4: Class 4B (see page 3-8) | LD *AR3+, 16, A
W 4493
B A 0
B AR3 300
B SXM 1
B M 0300 fedc
A A fffedc0000
A AR3 301
A SXM 1
A M 0300 fedc
# ligne registre incomplete: AR3                  0300                    AR1                  0301
E
T 82 4: Class 4B (see page 3-8) | LD #248, B
W e9f8
B B 0
B SXM 1
A B f8
A SXM 1
E
T 83 4: Class 4B (see page 3-8) | LD A, 8, B
```

```
W f548
B A 7ffd0040
B B ffff
B OVB 0
B SXM 1
B M 0200 fedc
A A 7ffd0040
A B 7ffd004000
A OVB 1
A SXM 1
A M 0200 fedc
E
T 84 2: Class 5B (see page 3-9) | LD *AR3+, T
W 3093
B AR3 300
B T 0
B M 0300 fedc
A AR3 301
A T fedc
A M 0300 fedc
E
T 85 2: Class 5B (see page 3-9) | LD *AR4, DP
W 4684
B AR4 200
B DP 1ff
B M 0200 fedc
A AR4 200
A DP dc
A M 0200 fedc
E
T 86 2: Class 5B (see page 3-9) | LD #23, DP
W ea17
B DP 1ff
A DP 17
E
T 87 2: Class 5B (see page 3-9) | LD 15, ASM
W ed0f
B ASM 0
A ASM f
E
T 88 2: Class 5B (see page 3-9) | LD 3, ARP
W f4a3
B ARP 0
A ARP 3
E
T 89 2: Class 5B (see page 3-9) | LD 0, ASM
W 3200
B ASM 0
B DP 4
B M 0200 fedc
A ASM 1c
A DP 4
A M 0200 fedc
E
T 90 LDM MMR, dst | LDM AR4, A
W 4814
B A 1111
B AR4 ffff
A A ffff
A AR4 ffff
E
T 91 LDM MMR, dst | LDM 060h, B
W 4960
B B 0
B M 0060 1234
A B 1234
```

```
A M 0060 1234
E
T 92 LD Xmem, dst | LD *AR4+, A
W a8ab
B A 1000
B AR4 100
B AR5 200
B B 1111
B FRCT 0
B T 400
B M 0100 1234
B M 0200 4321
A A 12340000
A AR4 101
A AR5 201
A B 10c9511
A FRCT 0
A T 400
A M 0100 1234
A M 0200 4321
E
T 93 LD Xmem, dst | LD *AR4+, A
W aaab
B A 1000
B AR4 100
B AR5 200
B B 1111
B FRCT 0
B T 400
B M 0100 1234
B M 0200 4321
A A 12340000
A AR4 101
A AR5 201
A B 10d0000
A FRCT 0
A T 400
A M 0100 1234
A M 0200 4321
E
T 94 LD Xmem, dst | LD *AR4+, A
W acab
B A 1000
B AR4 100
B AR5 200
B B 1111
B FRCT 0
B T 400
B M 0100 1234
B M 0200 4321
A A 12340000
A AR4 101
A AR5 201
A B fffef38d11
A FRCT 0
A T 400
A M 0100 1234
A M 0200 4321
E
T 95 LD Xmem, dst | LD *AR4+, A
W aeab
B A 1000
B AR4 100
B AR5 200
B B 1111
B FRCT 0
```

```
B T 400
B M 0100 1234
B M 0200 4321
A A 12340000
A AR4 101
A AR5 201
A B fffef40000
A FRCT 0
A T 400
A M 0100 1234
A M 0200 4321
E
T 96 LDR Smem, dst | LDR *AR1, A
W 1681
B A 0
B AR1 200
B SXM 0
B M 0200 fedc
A A fedc8000
A AR1 200
A SXM 0
A M 0200 fedc
E
T 97 LDU Smem, dst | LDU *AR1, A
W 1281
B A 0
B AR1 200
B M 0200 fedc
A A fedc
A AR1 200
A M 0200 fedc
E
T 98 LMS Xmem, Ymem | LMS *AR3+, *AR4+
W e19a
B A 77778888
B AR3 100
B AR4 200
B B 100
B FRCT 0
B M 0100 0055
B M 0200 00aa
A A 77cd0888
A AR3 101
A AR4 201
A B 3972
A FRCT 0
A M 0100 0055
A M 0200 00aa
E
T 99 LTD Smem | LTD *AR3
W 4c83
B AR3 100
B T 0
B M 0100 6cac
B M 0101 6cac
A AR3 100
A T 6cac
A M 0100 6cac
E
T 100 4: Class 6B (see page 3-11) | MAC *AR5+, A
W 2895
B A 1000
B AR5 100
B FRCT 0
B T 400
B M 0100 1234
```

```
A A 48e000
A AR5 101
A FRCT 0
A T 400
A M 0100 1234
E
T 101 4: Class 6B (see page 3-11) | MAC #345h, A, B
W f167 0345
B A 1000
B B 0
B FRCT 1
B T 400
A A 1000
A B 1a3800
A FRCT 1
A T 400
E
T 102 4: Class 6B (see page 3-11) | MAC *AR5+, #1234h, A
W 6495 1234
B A 1000
B AR5 100
B FRCT 0
B T 0
B M 0100 5678
A A 6261060
A AR5 101
A FRCT 0
A T 5678
A M 0100 5678
E
T 103 4: Class 6B (see page 3-11) | MAC *AR5+, *AR4+,A, B ;(AR6->AR4)
W b1ba
B A 1000
B AR4 200
B AR5 100
B B 4
B FRCT 1
B T 8
B M 0100 5678
B M 0200 1234
A A 1000
A AR4 201
A AR5 101
A B c4c10c0
A FRCT 1
A T 5678
A M 0100 5678
A M 0200 1234
E
T 104 4: Class 6B (see page 3-11) | MACR *AR5+, A
W 2a95
B A 1000
B AR5 100
B FRCT 0
B T 400
B M 0100 1234
A A 490000
A AR5 101
A FRCT 0
A T 400
A M 0100 1234
E
T 105 4: Class 6B (see page 3-11) | MACR *AR5+, *AR4+,A, B ;(AR6->AR4)
W b5ba
B A 1000
B AR4 200
```

```
B AR5 100
B B 4
B FRCT 1
B T 8
B M 0100 5678
B M 0200 1234
A A 1000
A AR4 201
A AR5 101
A B c4c0000
A FRCT 1
A T 5678
A M 0100 5678
A M 0200 1234
E
T 106 MACA[R] Smem [, B ] | MACA *AR5+
W 3595
B A 12340000
B AR5 100
B B 0
B FRCT 0
B T 400
B M 0100 5678
A A 12340000
A AR5 101
A B 6260060
A FRCT 0
A T 5678
A M 0100 5678
E
T 107 MACA[R] Smem [, B ] | MACA T, B, B
W f788
B A 12340000
B B 20000
B FRCT 1
B T 444
A A 12340000
A B 9d4ba0
A FRCT 1
A T 444
E
T 108 MACA[R] Smem [, B ] | MACAR *AR5+, B
W 3795
B A 12340000
B AR5 100
B B 0
B FRCT 0
B T 400
B M 0100 5678
A A 12340000
A AR5 101
A B 6260000
A FRCT 0
A T 5678
A M 0100 5678
E
T 109 MACA[R] Smem [, B ] | MACAR T, B, B
W f789
B A 12340000
B B 20000
B FRCT 1
B T 444
A A 12340000
A B 9d0000
A FRCT 1
A T 444
```

```
E
S 110 MACD Smem, pmad, src | MACD *AR3–, COEFFS, A | macd['OP_Smem', 'OP_pmad', 'OP_SRC1']: nombre: 'COEFFS'
S 111 MACP Smem, pmad, src | MACP *AR3–, COEFFS, A | macp['OP_Smem', 'OP_pmad', 'OP_SRC1']: nombre: 'COEFFS'
T 112 MACSU Xmem, Ymem, src | MACSU *AR4+, *AR5+, A
W a6ab
B A 1000
B AR4 100
B AR5 200
B FRCT 0
B T 8
B M 0100 8765
B M 0200 1234
A A 9a0aa84
A AR4 101
A AR5 201
A FRCT 0
A T 8765
A M 0100 8765
A M 0200 1234
E
T 113 MAR Smem | MAR *AR3+
W 6d93
B AR3 100
B ARP 0
B CMPT 0
A AR3 101
A ARP 0
A CMPT 0
E
T 114 MAR Smem | MAR *AR0–
W 6d88
B AR4 100
B ARP 4
B CMPT 1
A AR4 ff
A ARP 4
A CMPT 1
E
T 115 MAR Smem | MAR *AR3
W 6d83
B AR0 8
B AR3 100
B ARP 0
B CMPT 1
A AR0 8
A AR3 100
A ARP 3
A CMPT 1
E
T 116 MAR Smem | MAR *+AR3
W 6d9b
B AR3 100
B ARP 0
B CMPT 1
A AR3 101
A ARP 3
A CMPT 1
E
T 117 MAR Smem | MAR *AR3–
W 6d8b
B AR3 100
B ARP 0
B CMPT 1
A AR3 ff
A ARP 3
A CMPT 1
```

```
E
T 118 2: Class 7 (see page 3-12) | MAS *AR5+, A
W 2c95
B A 1000
B AR5 100
B FRCT 0
B T 400
B M 0100 1234
A A ffffb74000
A AR5 101
A FRCT 0
A T 400
A M 0100 1234
E
T 119 2: Class 7 (see page 3-12) | MAS *AR5+, *AR4+, A, B ;(AR6->AR4)
W b9ba
B A 1000
B AR4 200
B AR5 100
B B 4
B FRCT 1
B T 8
B M 0100 5678
B M 0200 1234
A A 1000
A AR4 201
A AR5 101
A B fff3b40f40
# errata SPRU172C: exemple MAS Xmem,Ymem calcule sans FRCT=1 (cf. MAC Xmem,Ymem test 103 qui l'applique)
A FRCT 1
A T 5678
A M 0100 5678
A M 0200 1234
E
T 120 2: Class 7 (see page 3-12) | MASR *AR5+, A
W 2e95
B A 1000
B AR5 100
B FRCT 0
B T 400
B M 0100 1234
A A ffffb70000
A AR5 101
A FRCT 0
A T 400
A M 0100 1234
E
T 121 2: Class 7 (see page 3-12) | MASR *AR5+, *AR4+, A, B ;(AR6->AR4)
W bdba
B A 1000
B AR4 200
B AR5 100
B B 4
B FRCT 1
B T 8
B M 0100 5678
B M 0200 1234
A A 1000
A AR4 201
A AR5 101
A B fff3b40000
# errata SPRU172C: idem MAS, rnd(A - 2*X*Y)
A FRCT 1
A T 5678
A M 0100 5678
A M 0200 1234
```

```
E
T 122 2: Class 1 (see page 3-3) | MASA *AR5+
W 3395
B A 12340000
B AR5 100
B B 20000
B FRCT 0
B T 400
B M 0100 5678
A A 12340000
A AR5 101
A B fff9dbffa0
A FRCT 0
A T 5678
A M 0100 5678
E
T 123 2: Class 1 (see page 3-3) | MASA T, B
W f78a
B A 12340000
B B 20000
B FRCT 1
B T 444
A A 12340000
A B ffff66b460
A FRCT 1
A T 444
E
T 124 2: Class 1 (see page 3-3) | MASAR T, B
W f78b
B A 12340000
B B 20000
B FRCT 1
B T 444
A A 12340000
A B ffff670000
A FRCT 1
A T 444
E
T 125 MAX dst | MAX A
W f486
B A fff6
B B ffcb
B C 1
A A fff6
A B ffcb
A C 0
E
T 126 MAX dst | MAX A
W f486
B A 55
B B 1234
B C 0
A A 1234
A B 1234
A C 1
E
T 127 MIN dst | MIN A
W f487
B A ffcb
B B fff6
B C 1
A A ffcb
A B fff6
A C 0
E
T 128 MIN dst | MIN A
```

```
W f487
B A 1234
B B 1234
B C 0
A A 1234
A B 1234
A C 1
E
T 129 4: Class 2 (see page 3-4) | MPY 13, A
W 200d
B A 36
B DP 8
B FRCT 1
B T 6
B M 040d 0007
A A 54
A DP 8
A FRCT 1
A T 6
A M 040d 0007
E
T 130 4: Class 2 (see page 3-4) | MPY *AR2–, *AR4+0%, B;
W a54e
B AR0 1
B AR2 1ff
B AR4 300
B B fffffffe0
B FRCT 0
B M 01ff 0010
B M 0300 0002
A AR0 1
A AR2 1fe
A AR4 301
A B 20
A FRCT 0
A M 01ff 0010
A M 0300 0002
E
T 131 4: Class 2 (see page 3-4) | MPY #0FFFEh, A
W f066 fffe
B A 0
B FRCT 0
B T 2000
A A ffffffc000
A FRCT 0
A T 2000
E
T 132 4: Class 2 (see page 3-4) | MPYR 0, B
W 2300
B B fffe000001
B DP 4
B FRCT 0
B T 1234
B M 0200 5678
A B 6260000
A DP 4
A FRCT 0
A T 1234
A M 0200 5678
E
T 133 2: Class 1 (see page 3-3) | MPYA *AR2
W 3182
B A ff87651111
B AR2 200
B B 320
B FRCT 0
```

```
B T 1234
B M 0200 5678
A A ff87651111
A AR2 200
A B ffd7436558
A FRCT 0
A T 5678
A M 0200 5678
E
T 134 2: Class 1 (see page 3-3) | MPYA B
W f58c
B A ff87651111
B B 320
B FRCT 0
B T 4567
A A ff87651111
A B ffdf4db2a3
A FRCT 0
A T 4567
E
T 135 MPYU Smem, dst | MPYU *AR0–, A
W 2488
B A ff80000000
B AR0 1000
B FRCT 0
B T 4000
B M 1000 fe00
A A 3f800000
A AR0 fff
A FRCT 0
A T 4000
A M 1000 fe00
E
T 136 MVDD Xmem, Ymem | MVDD *AR3+, *AR5+
W e59b
B AR3 8000
B AR5 200
B M 0200 abcd
B M 8000 1234
A AR3 8001
A AR5 201
A M 0200 1234
A M 8000 1234
E
T 137 MVDK Smem, dmad | MVDK 10, 8000h
W 710a 8000
B DP 4
B M 020a 1234
B M 8000 abcd
A DP 4
A M 020a 1234
A M 8000 1234
E
T 138 MVDK Smem, dmad | MVDK *AR3–, 1000h
W 718b 1000
B AR3 1ff
B M 01ff 1234
B M 1000 abcd
A AR3 1fe
A M 01ff 1234
A M 1000 1234
E
T 139 MVDM dmad, MMR | MVDM 300h, BK
W 7219 0300
B BK abcd
B M 0300 1234
```

```
A BK 1234
A M 0300 1234
E
T 140 MVDP Smem, pmad | MVDP 0, 0FE00h
W 7d00 fe00
B DP 4
B M 0200 0123
B P fe00 ffff
A DP 4
A M 0200 0123
E
T 141 MVKD dmad, Smem | MVKD 300h, 0
W 7000 0300
B DP 4
B M 0200 abcd
B M 0300 1234
A DP 4
A M 0200 1234
A M 0300 1234
E
T 142 MVKD dmad, Smem | MVKD 1000h, *+AR5
W 709d 1000
B AR5 1ff
B M 0200 abcd
B M 1000 1234
A AR5 200
A M 0200 1234
A M 1000 1234
E
T 143 MVMD MMR, dmad | MVMD AR7, 8000h
W 7317 8000
B AR7 1234
B M 8000 abcd
A AR7 1234
A M 8000 1234
E
T 144 MVMM MMRx, MMRy | MVMM SP, AR1
W e781
B AR1 3eff
B SP 200
A AR1 200
A SP 200
E
T 145 MVPD pmad, Smem | MVPD 0FE00h, 5
W 7c05 fe00
B DP 6
B M 0305 ffff
B P fe00 8a55
A DP 6
A M 0305 8a55
E
T 146 MVPD pmad, Smem | MVPD 2000h, *AR7–0
W 7caf 2000
B AR0 2
B AR7 ffe
B M 0ffe abcd
B P 2000 1234
A AR0 2
A AR7 ffc
A M 0ffe 1234
E
T 147 NEG src [, dst ] | NEG A, B
W f584
B A fffffff228
B B 1234
B OVA 0
```

```
A A fffffff228
A B dd8
A OVA 0
E
T 148 NEG src [, dst ] | NEG B, A
W f684
B A 1234
B B 80000000
B OVB 0
A A ff80000000
A B 80000000
A OVB 0
E
T 149 NEG src [, dst ] | NEG A
W f484
B A 8000000000
B OVA 0
B OVM 0
A A 8000000000
A OVA 1
A OVM 0
E
T 150 NEG src [, dst ] | NEG A
W f484
B A 8000000000
B OVA 0
B OVM 1
A A 7fffffff
A OVA 1
A OVM 1
E
T 152 NORM src [, dst ] | NORM A
W f48f
B A fffffff001
B T 13
A A ff80080000
A T 13
E
T 153 NORM src [, dst ] | NORM B, A
W f68f
B A fffffff001
B B 210a0a0a0a
B T ff9
A A 42141414
A B 210a0a0a0a
A T ff9
E
T 154 4: Class 1 (see page 3-3) | OR *AR3+, A
W 1a93
B A ff1200
B AR3 100
B M 0100 1500
A A ff1700
A AR3 101
A M 0100 1500
E
T 155 4: Class 1 (see page 3-3) | OR A, +3, B
W f1a3
B A 1200
B B 1800
A A 1200
A B 9800
E
T 156 ORM #lk, Smem | ORM 0404h, *AR4+
W 6994 0404
B AR4 100
```

```
B M 0100 4444
A AR4 101
A M 0100 4444
E
T 157 POLY Smem | POLY *AR3+%
W 36d3
B A 12340000
B AR3 200
B B 10000
B T 5678
B M 0200 2000
A A 6270000
A AR3 201
A B 20000000
A T 5678
A M 0200 2000
E
T 158 POPD Smem | POPD 10
W 8b0a
B DP 8
B SP 300
B M 0300 0092
B M 040a 0055
A DP 8
A SP 301
A M 0300 0092
A M 040a 0092
E
T 159 POPM MMR | POPM AR5
W 8a15
B AR5 55
B SP 3f0
B M 03f0 0060
A AR5 60
A SP 3f1
A M 03f0 0060
E
S 160 PORTR PA, Smem | PORTR 05, INDAT ; INDAT .equ 60h | portr['OP_PA', 'OP_Smem']: nombre: 'INDAT'
S 161 PORTW Smem, PA | PORTW OUTDAT, 5h ; OUTDAT .equ 07h | portw['OP_Smem', 'OP_PA']: nombre: 'OUTDAT'
T 162 PSHD Smem | PSHD *AR3+
W 4b93
B AR3 200
B SP 8000
B M 0200 07ff
B M 7fff 0092
A AR3 201
A SP 7fff
A M 0200 07ff
A M 7fff 07ff
E
T 163 PSHM MMR | PSHM BRC
W 4a1a
B BRC 1234
B SP 2000
B M 1fff 07ff
A BRC 1234
A SP 1fff
A M 1fff 1234
E
T 164 RC[D] cond [, cond [, condÃ] ] | RC AGEQ, ANOV              ; return is executed if the accumulator A
W fc62
B OVA 0
B PC 807
B SP 308
B M 0308 2002
A OVA 0
```

```
A PC 2002
A SP 309
A M 0308 2002
E
T 165 READA Smem | READA 6
W 7e06
B A 23
B DP 4
B M 0206 0075
B P 0023 0306
A A 23
A DP 4
A M 0206 0306
E
T 166 RESET | RESET
W f7e0
B INTM 0
B IPTR 1
B PC 25
A INTM 1
A IPTR 1
A PC 80
E
T 167 RET[D] | RET
W fc00
B PC 2112
B SP 300
B M 0300 1000
A PC 1000
A SP 301
A M 0300 1000
E
T 168 RETE[D] | RETE
W f4eb
B PC 1c3
B SP 2001
B M 2001 0110
A PC 110
A SP 2002
A M 2001 0110
# valeur: ['xCxx']
E
T 169 RETF[D] | RETF
W f49b
B INTM 1
B PC 1c3
B SP 2001
B M 2001 0110
A INTM 0
A PC 110
A SP 2002
A M 2001 0110
E
T 170 RND src [, dst ] | RND A, B
W f59f
B A fffffffff
B B 1
B OVM 0
A A fffffffff
A B 7fff
A OVM 0
E
T 171 RND src [, dst ] | RND A
W f49f
B A 7fffffff
B OVM 1
```

```
A A 7fffffff
A OVM 1
E
T 172 ROL src | ROL A
W f491
B A 5fb0001234
B C 0
A A 60002468
A C 1
E
T 173 ROLTC src | ROLTC A
W f492
B A 81c0005555
B TC 1
A A 8000aaab
A C 1
A TC 1
E
T 174 ROR src | ROR A
W f490
B A 7fb0001235
B C 0
A A 5800091a
A C 1
E
S 175 3: Class 2 (see page 3-4) | RPT DAT127 ; DAT127 .EQU 0FFF | rpt['OP_Smem']: nombre: 'DAT127'; rpt['OP_k8u']:
 ↪  nombre: 'DAT127'; rpt['OP_lku']: nombre: 'DAT127'
T 178 RPTB[D] pmad | ST #99, BRC ; RPTB end_block-1 (end_block-1 = 100Fh)
W 761a 0063 f072 100f
N 2
B BRC 1234
B PC ffe
B RSA 5678
A BRC 63
A PC 1002
A RSA 1002
A REA 100f
E
T 179 RPTB[D] pmad | ST #99, BRC ; RPTBD end_block-1 ; 2 delay-slot NOPs
W 761a 0063 f272 100f
N 4
B BRC 1234
B PC ffe
B RSA 5678
A BRC 63
A PC 1004
A RSA 1004
A REA 100f
E
T 180 RPTZ dst, #lk | RPTZ A, 1023 ; Repeat the next instruction 1024 times
W f071 03ff
B A ffe008000
A A 0
E
T 181 RSBX N, SBIT | RSBX SXM ; SXM means: n=1 and SBIT=8
W f6b8
B ST1 35cd
A ST1 34cd
E
T 182 RSBX N, SBIT | RSBX 1,8
W f6b8
B ST1 35cd
A ST1 34cd
E
T 183 SACCD src, Xmem, cond | SACCD A, *AR3+0%, ALT
W 9ed3
```

```
B A fffe004321
B AR0 2
B AR3 202
B ASM 1
B M 0202 0101
A A fffe004321
A AR0 2
A AR3 204
A ASM 1
A M 0202 fc00
E
T 184 SAT src | SAT B
W f583
B B 7123456789
A B 7fffffff
A OVB 1
E
T 185 SAT src | SAT A
W f483
B A f812345678
A A ff80000000
A OVA 1
E
T 186 SAT src | SAT B
W f583
B B 123456
A B 123456
A OVB 0
E
T 187 SFTA src, SHIFT [, dst ] | SFTA A, -5, B
W f57b
B A ff87650055
B B 43211234
B SXM 1
A A ff87650055
A B fffc3b2802
A C 1
A SXM 1
E
T 188 SFTA src, SHIFT [, dst ] | SFTA B, +5
W f765
B B 80aa001234
B C 0
B OVM 0
B SXM 0
A B 1540024680
A C 1
A OVM 0
A SXM 0
E
T 189 SFTC src | SFTC A
W f494
B A fffffff001
A A ffffffe002
A TC 0
E
T 190 SFTL src, SHIFT [, dst ] | SFTL A, -5, B
W f1fb
B A ff87650055
B B ff80000000
B C 0
A A ff87650055
A B 43b2802
A C 1
E
T 191 SFTL src, SHIFT [, dst ] | SFTL B, +5
```

```
W f3e5
B B 80aa001234
B C 0
A B 40024680
A C 1
E
S 192 SQDST Xmem, Ymem | SQDST *AR3+, AR4+ | sqdst['OP_Xmem', 'OP_Ymem']: Xmem: 'AR4+'
T 193 2: Class 1 (see page 3-3) | SQUR 30, B
W 271e
B B 1f4
B DP 6
B FRCT 0
B T 3
B M 031e 000f
A B e1
A DP 6
A FRCT 0
A T f
A M 031e 000f
E
T 194 2: Class 1 (see page 3-3) | SQUR A, B
W f58d
B A f0000
B B 1010101
B FRCT 1
A A f0000
A B 1c2
A FRCT 1
E
T 195 SQURA Smem, src | SQURA 30, B
W 391e
B B 3200000
B DP 6
B FRCT 0
B T 3
B M 031e 000f
A B 32000e1
A DP 6
A FRCT 0
A T f
A M 031e 000f
E
T 196 SQURA Smem, src | SQURA *AR3+, A
W 3893
B A 1f4
B AR3 31e
B FRCT 0
B T 3
B M 031e 000f
A A 2d5
A AR3 31f
A FRCT 0
A T f
A M 031e 000f
E
T 197 SQURS Smem, src | SQURS 9, A
W 3a09
B A 14b5db0
B DP 6
B FRCT 0
B T 8765
B M 0309 1234
A A 320
A DP 6
A FRCT 0
A T 1234
```

```
A M 0309 1234
E
T 198 SQURS Smem, src | SQURS *AR3, B
W 3b83
B AR3 309
B B 14b5db0
B FRCT 0
B T 8765
B M 0309 1234
A AR3 309
A B 320
A FRCT 0
A T 1234
A M 0309 1234
E
T 199 SRCCD Xmem, cond | SRCCD *AR5–, AGT
W 9d76
B A 70ffffff
B AR5 202
B BRC 4321
B M 0202 1234
A A 70ffffff
A AR5 201
A BRC 4321
A M 0202 4321
E
T 200 SSBX N, SBIT | SSBX SXM     ; SXM means: N=1, SBIT=8
W f7b8
B ST1 34cd
A ST1 35cd
E
T 201 SSBX N, SBIT | SSBX 1,8
W f7b8
B ST1 34cd
A ST1 35cd
E
T 202 3: Class 12B (see page 3-27) | ST FFFFh, 0
W 7600 ffff
B DP 4
B M 0200 0101
A DP 4
A M 0200 ffff
E
T 203 3: Class 12B (see page 3-27) | ST TRN, 5
W 8d05
B DP 4
B TRN 1234
B M 0205 0030
A DP 4
A TRN 1234
A M 0205 1234
E
T 204 3: Class 12B (see page 3-27) | ST T, *AR7–
W 8c8f
B AR7 321
B T 4210
B M 0321 1200
A AR7 320
A T 4210
A M 0321 4210
E
T 205 4: Class 11B (see page 3-25) | STH A, 10
W 820a
B A ff87654321
B DP 4
B M 020a 1234
```

```
A A ff87654321
A DP 4
A M 020a 8765
E
S 206 4: Class 11B (see page 3-25) | STH B, –8, *AR7– | sth['OP_SRC1', 'OP_Smem']: Smem direct hors 0..7F: '-8';
 ↪  sth['OP_SRC1', 'OP_ASM', 'OP_Smem']: ASM attendu; sth['OP_SRC1'
S 207 4: Class 11B (see page 3-25) | STH A, –4, 10 | sth['OP_SRC1', 'OP_Smem']: Smem direct hors 0..7F: '-4';
 ↪  sth['OP_SRC1', 'OP_ASM', 'OP_Smem']: ASM attendu; sth['OP_SRC1'
T 208 4: Class 11B (see page 3-25) | STL A, 11
W 800b
B A ff87654321
B DP 4
B M 020b 1234
A A ff87654321
A DP 4
A M 020b 4321
E
S 209 4: Class 11B (see page 3-25) | STL B, –8, *AR7– | stl['OP_SRC1', 'OP_Smem']: Smem direct hors 0..7F: '-8';
 ↪  stl['OP_SRC1', 'OP_ASM', 'OP_Smem']: ASM attendu; stl['OP_SRC1'
S 210 4: Class 11B (see page 3-25) | STL A, 7, 11 | stl['OP_SRC1', 'OP_Smem']: operandes en trop: ['11'];
 ↪  stl['OP_SRC1', 'OP_ASM', 'OP_Smem']: ASM attendu; stl['OP_SRC1', '
T 211 STLM src, MMR | STLM A, BRC
W 881a
B A ff87654321
A A ff87654321
E
T 212 STLM src, MMR | STLM B, *AR1–
W 8989
B AR1 3f17
B B ff84211234
A AR1 16
A B ff84211234
E
T 213 STM #lk, MMR | STM 0FFFFh, IMR
W 7700 ffff
B IMR ff01
A IMR ffff
E
T 214 STM #lk, MMR | STM 8765h, *AR7+
W 7797 8765
B AR0 0
B AR7 8010
A AR0 8765
A AR7 11
E
S 215 ST src, Ymem | ST A, *AR3 | st['OP_T', 'OP_Smem']: T attendu; st['OP_TRN', 'OP_Smem']: TRN attendu;
 ↪  st['OP_lk', 'OP_Smem']: nombre: 'A'
S 216 ST src, Ymem | ST B, *AR2– | st['OP_T', 'OP_Smem']: T attendu; st['OP_TRN', 'OP_Smem']: TRN attendu;
 ↪  st['OP_lk', 'OP_Smem']: nombre: 'B'
S 217 ST src, Ymem | ST A, *AR3 | st['OP_T', 'OP_Smem']: T attendu; st['OP_TRN', 'OP_Smem']: TRN attendu;
 ↪  st['OP_lk', 'OP_Smem']: nombre: 'A'
S 219 ST src, Ymem | ST A, *AR4– | st['OP_T', 'OP_Smem']: T attendu; st['OP_TRN', 'OP_Smem']: TRN attendu;
 ↪  st['OP_lk', 'OP_Smem']: nombre: 'A'
S ⟨1⟩ ST src, Ymem | ST A, *AR⟨2⟩+ | st['OP_T', 'OP_Smem']: T attendu; st['OP_TRN', 'OP_Smem']: TRN attendu;
 ↪  st['OP_lk', 'OP_Smem']: nombre: 'A'  ×4
    ⟨⟩ = (220,4) (221,4) (222,4) (223,3)
S 224 ST src, Ymem | ST A, *AR3– | st['OP_T', 'OP_Smem']: T attendu; st['OP_TRN', 'OP_Smem']: TRN attendu;
 ↪  st['OP_lk', 'OP_Smem']: nombre: 'A'
T 225 STRCD Xmem, cond | STRCD *AR5–, AGT
W 9c76
B A 70ffffff
B AR5 202
B T 4321
B M 0202 1234
A A 70ffffff
A AR5 201
```

```
A T 4321
A M 0202 4321
E
S 226 6: Class 7 (see page 3-12) | SUB *AR1+, 14, A | sub['OP_SRC', 'OPT|OP_SHIFT', 'OPT|OP_DST']: src attendu,
 ↪  '*AR1+'; sub['OP_SRC', 'OP_ASM', 'OPT|OP_DST']: src attendu, '
T 227 6: Class 7 (see page 3-12) | SUB A, –8, B
W f538
B A 1200
B B 1800
B SXM 1
A A 1200
A B 17ee
A C 1
A SXM 1
E
T 228 6: Class 7 (see page 3-12) | SUB #12345, 8, A, B
W f118 3039
B A 1200
B B 1800
B SXM 1
A A 1200
A B ffffcfd900
A C 0
A SXM 1
E
T 229 SUBB Smem, src | SUBB 5, A
W 0e05
B A 6
B C 0
B DP 8
B M 0405 0006
A A fffffffff
A C 0
A DP 8
A M 0405 0006
E
T 230 SUBB Smem, src | SUBB *AR1+, B
W 0f91
B AR1 405
B B ff80000006
B C 1
B OVM 1
B M 0405 0006
A AR1 406
A B ff80000000
A C 1
A OVM 1
A M 0405 0006
E
T 231 SUBC Smem, src | SUBC 2, A
W 1e02
B A 4
B DP 6
B M 0302 0001
A A 8
A C 0
A DP 6
A M 0302 0001
E
T 232 SUBC Smem, src | RPT #15 ; SUBC *AR1, B
W ec0f 1f81
B AR1 1000
B B 41
B M 1000 0007
A AR1 1000
A B 20009
```

```
A C 1
A M 1000 0007
E
T 233 SUBS Smem, src | SUBS *AR2–, B
W 0b8a
B AR2 100
B B 2
B M 0100 f006
A AR2 ff
A B ffffff0ffc
A C 0
A M 0100 f006
E
T 234 TRAP K | TRAP 10h
W f4d0
B PC 1233
B SP 3ff
B M 03fe 9653
A PC ffc0
A SP 3fe
A M 03fe 1234
E
T 235 WRITA Smem | WRITA 5
W 7f05
B A 257
B DP 32
B M 1005 4339
B P 0257 0306
A A 257
A DP 32
A M 1005 4339
E
T 236 n       Opcode N | XC 1, ALEQ ; MAR *AR1+
W fd47 6d91
N 2
B A fffffffffff
B AR1 32
A A fffffffffff
A AR1 33
E
T 237 4: Class 1 (see page 3-3) | XOR *AR3+, A
W 1c93
B A ff1200
B AR3 100
B M 0100 1500
A A ff0700
A AR3 101
A M 0100 1500
E
T 238 4: Class 1 (see page 3-3) | XOR A, +3, B
W f1c3
B A 1200
B B 1800
A A 1200
A B 8800
E
T 239 XORM #lk, Smem | XORM 0404h, *AR4–
W 6a8c 0404
B AR4 100
B M 0100 4444
A AR4 ff
A M 0100 4040
# ligne registre incomplete: C†
# ligne registre incomplete: TC†
# ligne registre incomplete: TC = 1
# ligne registre incomplete: TC = 0
```

```
# ligne registre incomplete: ST0 and ST1 contain the status of various conditions and modes; PMST con-
# ligne registre incomplete: ARP                          Auxiliary register pointer
# ligne registre incomplete: ASM                          Accumulator shift mode
# ligne registre incomplete: BRAF                         Block repeat active flag
# ligne registre incomplete: C                            Carry
# ligne registre incomplete: CMPT                         Compatibility mode
# ligne registre incomplete: CPL                          Compiler mode
# ligne registre incomplete: C16                          Dual 16-bit/double-precision arithmetic mode
# ligne registre incomplete: DP                           Data page pointer
# ligne registre incomplete: FRCT                         Fractional mode
# ligne registre incomplete: HM                                   Hold mode
# ligne registre incomplete: INTM                                 Interrupt mode
# ligne registre incomplete: OVA                                  Overflow flag A
# ligne registre incomplete: OVB                                  Overflow flag B
# ligne registre incomplete: OVM                                  Overflow mode
# ligne registre incomplete: SXM                                  Sign-extension mode
# ligne registre incomplete: TC                                   Test/control flag
# ligne registre incomplete: XF                                   External flag status
# ligne registre incomplete: ARP               TC       C       OVA     OVB                                DP
# ligne registre incomplete: BRAF      CPL        XF       HM      INTM        0      OVM        SXM       C16
 ↪  FRCT     CMPT          ASM
# ligne registre incomplete: A
# ligne registre incomplete: B
# ligne registre incomplete: C
# ligne registre incomplete: a single instruction is executed.
# ligne registre incomplete: T
# ligne registre incomplete: XF pin.
# ligne registre incomplete: a 0.
# ligne registre incomplete: a 0.
# ligne registre incomplete: A                                                    subtract instructions 2-3 to 2-4
# ligne registre incomplete: B instruction 4-14
# ligne registre incomplete: C                                                 DELAY instruction 4-41
# ligne registre incomplete: C address bus (CAB) C-3                           direct memory address, definition C-4
# ligne registre incomplete: C bus (CB), definition C-3                         DLD instruction 4-42
# ligne registre incomplete: C16 C-3                                         double (32-bit operand) instructions 2-6
# ligne registre incomplete: T                                             zero detect bit A (ZA), definition C-10
E
T 301 [banc 2026-10-16] ADD src, ASM, dst | ADD B, ASM, A ; codeur FR ROM PDROM 0xeac1 (RPE, xM << 6-exp)
W f680
B A 0
B B d990000
B ASM 3
A A 6cc80000
A B d990000
A ASM 3
E
T 302 [banc 2026-10-16] SUB src, ASM, dst | SUB B, ASM, A
W f681
B A 0
B B d990000
B ASM 3
A A ff93380000
A B d990000
A ASM 3
E
```

2.9 /opt/GSM/c54x_exe/tools/cch_ref/cch_deinterleave_inverse_ref.csv

7138 octets, 457 lignes → 80 lignes (1 groupes compactés)

```
iB_index,burst_B,pos_j,source_coded_k,lane_G
⟨1⟩,⟨2⟩,⟨3⟩,⟨4⟩,⟨5⟩  ×456
    ⟨⟩ = (0,0,0,0,0) (1,0,1,228,0) (2,0,2,64,0) (3,0,3,292,0) (4,0,4,128,0) (5,0,5,356,0) (6,0,6,192,0)
         (7,0,7,420,0) (8,0,8,256,0) (9,0,9,28,0) (10,0,10,320,0) (11,0,11,92,0) (12,0,12,384,0)
```

(13,0,13,156,0) (14,0,14,448,0) (15,0,15,220,0) (16,0,16,56,0) (17,0,17,284,0) (18,0,18,120,0)
(19,0,19,348,0) (20,0,20,184,0) (21,0,21,412,0) (22,0,22,248,0) (23,0,23,20,0) (24,0,24,312,0)
(25,0,25,84,0) (26,0,26,376,0) (27,0,27,148,0) (28,0,28,440,0) (29,0,29,212,0) (30,0,30,48,0)
(31,0,31,276,0) (32,0,32,112,0) (33,0,33,340,0) (34,0,34,176,0) (35,0,35,404,0) (36,0,36,240,0)
(37,0,37,12,0) (38,0,38,304,0) (39,0,39,76,0) (40,0,40,368,0) (41,0,41,140,0) (42,0,42,432,0)
(43,0,43,204,0) (44,0,44,40,0) (45,0,45,268,0) (46,0,46,104,0) (47,0,47,332,0) (48,0,48,168,0)
(49,0,49,396,0) (50,0,50,232,0) (51,0,51,4,0) (52,0,52,296,0) (53,0,53,68,0) (54,0,54,360,0)
(55,0,55,132,0) (56,0,56,424,0) (57,0,57,196,0) (58,0,58,32,0) (59,0,59,260,0) (60,0,60,96,0)
(61,0,61,324,0) (62,0,62,160,0) (63,0,63,388,0) (64,0,64,224,0) (65,0,65,452,0) (66,0,66,288,0)
(67,0,67,60,0) (68,0,68,352,0) (69,0,69,124,0) (70,0,70,416,0) (71,0,71,188,0) (72,0,72,24,0)
(73,0,73,252,0) (74,0,74,88,0) (75,0,75,316,0) (76,0,76,152,0) (77,0,77,380,0) (78,0,78,216,0)
(79,0,79,444,0) (80,0,80,280,0) (81,0,81,52,0) (82,0,82,344,0) (83,0,83,116,0) (84,0,84,408,0)
(85,0,85,180,0) (86,0,86,16,0) (87,0,87,244,0) (88,0,88,80,0) (89,0,89,308,0) (90,0,90,144,0)
(91,0,91,372,0) (92,0,92,208,0) (93,0,93,436,0) (94,0,94,272,0) (95,0,95,44,0) (96,0,96,336,0)
(97,0,97,108,0) (98,0,98,400,0) (99,0,99,172,0) (100,0,100,8,0) (101,0,101,236,0) (102,0,102,72,0)
(103,0,103,300,0) (104,0,104,136,0) (105,0,105,364,0) (106,0,106,200,0) (107,0,107,428,0)
(108,0,108,264,0) (109,0,109,36,0) (110,0,110,328,0) (111,0,111,100,0) (112,0,112,392,0)
(113,0,113,164,0) (114,1,0,57,1) (115,1,1,285,1) (116,1,2,121,1) (117,1,3,349,1) (118,1,4,185,1)
(119,1,5,413,1) (120,1,6,249,1) (121,1,7,21,1) (122,1,8,313,1) (123,1,9,85,1) (124,1,10,377,1)
(125,1,11,149,1) (126,1,12,441,1) (127,1,13,213,1) (128,1,14,49,1) (129,1,15,277,1) (130,1,16,113,1)
(131,1,17,341,1) (132,1,18,177,1) (133,1,19,405,1) (134,1,20,241,1) (135,1,21,13,1) (136,1,22,305,1)
(137,1,23,77,1) (138,1,24,369,1) (139,1,25,141,1) (140,1,26,433,1) (141,1,27,205,1) (142,1,28,41,1)
(143,1,29,269,1) (144,1,30,105,1) (145,1,31,333,1) (146,1,32,169,1) (147,1,33,397,1) (148,1,34,233,1)
(149,1,35,5,1) (150,1,36,297,1) (151,1,37,69,1) (152,1,38,361,1) (153,1,39,133,1) (154,1,40,425,1)
(155,1,41,197,1) (156,1,42,33,1) (157,1,43,261,1) (158,1,44,97,1) (159,1,45,325,1) (160,1,46,161,1)
(161,1,47,389,1) (162,1,48,225,1) (163,1,49,453,1) (164,1,50,289,1) (165,1,51,61,1) (166,1,52,353,1)
(167,1,53,125,1) (168,1,54,417,1) (169,1,55,189,1) (170,1,56,25,1) (171,1,57,253,1) (172,1,58,89,1)
(173,1,59,317,1) (174,1,60,153,1) (175,1,61,381,1) (176,1,62,217,1) (177,1,63,445,1) (178,1,64,281,1)
(179,1,65,53,1) (180,1,66,345,1) (181,1,67,117,1) (182,1,68,409,1) (183,1,69,181,1) (184,1,70,17,1)
(185,1,71,245,1) (186,1,72,81,1) (187,1,73,309,1) (188,1,74,145,1) (189,1,75,373,1) (190,1,76,209,1)
(191,1,77,437,1) (192,1,78,273,1) (193,1,79,45,1) (194,1,80,337,1) (195,1,81,109,1) (196,1,82,401,1)
(197,1,83,173,1) (198,1,84,9,1) (199,1,85,237,1) (200,1,86,73,1) (201,1,87,301,1) (202,1,88,137,1)
(203,1,89,365,1) (204,1,90,201,1) (205,1,91,429,1) (206,1,92,265,1) (207,1,93,37,1) (208,1,94,329,1)
(209,1,95,101,1) (210,1,96,393,1) (211,1,97,165,1) (212,1,98,1,1) (213,1,99,229,1) (214,1,100,65,1)
(215,1,101,293,1) (216,1,102,129,1) (217,1,103,357,1) (218,1,104,193,1) (219,1,105,421,1)
(220,1,106,257,1) (221,1,107,29,1) (222,1,108,321,1) (223,1,109,93,1) (224,1,110,385,1)
(225,1,111,157,1) (226,1,112,449,1) (227,1,113,221,1) (228,2,0,114,0) (229,2,1,342,0) (230,2,2,178,0)
(231,2,3,406,0) (232,2,4,242,0) (233,2,5,14,0) (234,2,6,306,0) (235,2,7,78,0) (236,2,8,370,0)
(237,2,9,142,0) (238,2,10,434,0) (239,2,11,206,0) (240,2,12,42,0) (241,2,13,270,0) (242,2,14,106,0)
(243,2,15,334,0) (244,2,16,170,0) (245,2,17,398,0) (246,2,18,234,0) (247,2,19,6,0) (248,2,20,298,0)
(249,2,21,70,0) (250,2,22,362,0) (251,2,23,134,0) (252,2,24,426,0) (253,2,25,198,0) (254,2,26,34,0)
(255,2,27,262,0) (256,2,28,98,0) (257,2,29,326,0) (258,2,30,162,0) (259,2,31,390,0) (260,2,32,226,0)
(261,2,33,454,0) (262,2,34,290,0) (263,2,35,62,0) (264,2,36,354,0) (265,2,37,126,0) (266,2,38,418,0)
(267,2,39,190,0) (268,2,40,26,0) (269,2,41,254,0) (270,2,42,90,0) (271,2,43,318,0) (272,2,44,154,0)
(273,2,45,382,0) (274,2,46,218,0) (275,2,47,446,0) (276,2,48,282,0) (277,2,49,54,0) (278,2,50,346,0)
(279,2,51,118,0) (280,2,52,410,0) (281,2,53,182,0) (282,2,54,18,0) (283,2,55,246,0) (284,2,56,82,0)
(285,2,57,310,0) (286,2,58,146,0) (287,2,59,374,0) (288,2,60,210,0) (289,2,61,438,0) (290,2,62,274,0)
(291,2,63,46,0) (292,2,64,338,0) (293,2,65,110,0) (294,2,66,402,0) (295,2,67,174,0) (296,2,68,10,0)
(297,2,69,238,0) (298,2,70,74,0) (299,2,71,302,0) (300,2,72,138,0) (301,2,73,366,0) (302,2,74,202,0)
(303,2,75,430,0) (304,2,76,266,0) (305,2,77,38,0) (306,2,78,330,0) (307,2,79,102,0) (308,2,80,394,0)
(309,2,81,166,0) (310,2,82,2,0) (311,2,83,230,0) (312,2,84,66,0) (313,2,85,294,0) (314,2,86,130,0)
(315,2,87,358,0) (316,2,88,194,0) (317,2,89,422,0) (318,2,90,258,0) (319,2,91,30,0) (320,2,92,322,0)
(321,2,93,94,0) (322,2,94,386,0) (323,2,95,158,0) (324,2,96,450,0) (325,2,97,222,0) (326,2,98,58,0)
(327,2,99,286,0) (328,2,100,122,0) (329,2,101,350,0) (330,2,102,186,0) (331,2,103,414,0)
(332,2,104,250,0) (333,2,105,22,0) (334,2,106,314,0) (335,2,107,86,0) (336,2,108,378,0)
(337,2,109,150,0) (338,2,110,442,0) (339,2,111,214,0) (340,2,112,50,0) (341,2,113,278,0)
(342,3,0,171,1) (343,3,1,399,1) (344,3,2,235,1) (345,3,3,7,1) (346,3,4,299,1) (347,3,5,71,1)
(348,3,6,363,1) (349,3,7,135,1) (350,3,8,427,1) (351,3,9,199,1) (352,3,10,35,1) (353,3,11,263,1)
(354,3,12,99,1) (355,3,13,327,1) (356,3,14,163,1) (357,3,15,391,1) (358,3,16,227,1) (359,3,17,455,1)
(360,3,18,291,1) (361,3,19,63,1) (362,3,20,355,1) (363,3,21,127,1) (364,3,22,419,1) (365,3,23,191,1)
(366,3,24,27,1) (367,3,25,255,1) (368,3,26,91,1) (369,3,27,319,1) (370,3,28,155,1) (371,3,29,383,1)
(372,3,30,219,1) (373,3,31,447,1) (374,3,32,283,1) (375,3,33,55,1) (376,3,34,347,1) (377,3,35,119,1)
(378,3,36,411,1) (379,3,37,183,1) (380,3,38,19,1) (381,3,39,247,1) (382,3,40,83,1) (383,3,41,311,1)
(384,3,42,147,1) (385,3,43,375,1) (386,3,44,211,1) (387,3,45,439,1) (388,3,46,275,1) (389,3,47,47,1)
(390,3,48,339,1) (391,3,49,111,1) (392,3,50,403,1) (393,3,51,175,1) (394,3,52,11,1) (395,3,53,239,1)
(396,3,54,75,1) (397,3,55,303,1) (398,3,56,139,1) (399,3,57,367,1) (400,3,58,203,1) (401,3,59,431,1)

(402,3,60,267,1) (403,3,61,39,1) (404,3,62,331,1) (405,3,63,103,1) (406,3,64,395,1) (407,3,65,167,1)
(408,3,66,3,1) (409,3,67,231,1) (410,3,68,67,1) (411,3,69,295,1) (412,3,70,131,1) (413,3,71,359,1)
(414,3,72,195,1) (415,3,73,423,1) (416,3,74,259,1) (417,3,75,31,1) (418,3,76,323,1) (419,3,77,95,1)
(420,3,78,387,1) (421,3,79,159,1) (422,3,80,451,1) (423,3,81,223,1) (424,3,82,59,1) (425,3,83,287,1)
(426,3,84,123,1) (427,3,85,351,1) (428,3,86,187,1) (429,3,87,415,1) (430,3,88,251,1) (431,3,89,23,1)
(432,3,90,315,1) (433,3,91,87,1) (434,3,92,379,1) (435,3,93,151,1) (436,3,94,443,1) (437,3,95,215,1)
(438,3,96,51,1) (439,3,97,279,1) (440,3,98,115,1) (441,3,99,343,1) (442,3,100,179,1) (443,3,101,407,1)
(444,3,102,243,1) (445,3,103,15,1) (446,3,104,307,1) (447,3,105,79,1) (448,3,106,371,1)
(449,3,107,143,1) (450,3,108,435,1) (451,3,109,207,1) (452,3,110,43,1) (453,3,111,271,1)
(454,3,112,107,1) (455,3,113,335,1)

2.10 /opt/GSM/c54x_exe/tools/cch_ref/cch_interleave_ref.csv

11686 octets, 457 lignes → 143 lignes (1 groupes compactés)

```
k,"lane_G(0=G0/c0,1=G1/c1)",burst_B,pos_j,iB_index,j_if_LSB_dropped,j_if_LSB_inverted,iB_index_if_LSB_inverted
⟨1⟩,⟨2⟩,⟨3⟩,⟨4⟩,⟨5⟩,⟨6⟩,⟨7⟩,⟨8⟩  ×456
    ⟨⟩ = (0,0,0,0,0,0,1,1) (1,1,1,98,212,98,99,213) (2,0,2,82,310,82,83,311) (3,1,3,66,408,66,67,409)
        (4,0,0,51,51,50,50,50) (5,1,1,35,149,34,34,148) (6,0,2,19,247,18,18,246) (7,1,3,3,345,2,2,344)
        (8,0,0,100,100,100,101,101) (9,1,1,84,198,84,85,199) (10,0,2,68,296,68,69,297)
        (11,1,3,52,394,52,53,395) (12,0,0,37,37,36,36,36) (13,1,1,21,135,20,20,134) (14,0,2,5,233,4,4,232)
        (15,1,3,103,445,102,102,444) (16,0,0,86,86,86,87,87) (17,1,1,70,184,70,71,185)
        (18,0,2,54,282,54,55,283) (19,1,3,38,380,38,39,381) (20,0,0,23,23,22,22,22) (21,1,1,7,121,6,6,120)
        (22,0,2,105,333,104,104,332) (23,1,3,89,431,88,88,430) (24,0,0,72,72,72,73,73)
        (25,1,1,56,170,56,57,171) (26,0,2,40,268,40,41,269) (27,1,3,24,366,24,25,367) (28,0,0,9,9,8,8,8)
        (29,1,1,107,221,106,106,220) (30,0,2,91,319,90,90,318) (31,1,3,75,417,74,74,416)
        (32,0,0,58,58,58,59,59) (33,1,1,42,156,42,43,157) (34,0,2,26,254,26,27,255) (35,1,3,10,352,10,11,353)
        (36,0,0,109,109,108,108,108) (37,1,1,93,207,92,92,206) (38,0,2,77,305,76,76,304)
        (39,1,3,61,403,60,60,402) (40,0,0,44,44,44,45,45) (41,1,1,28,142,28,29,143) (42,0,2,12,240,12,13,241)
        (43,1,3,110,452,110,111,453) (44,0,0,95,95,94,94,94) (45,1,1,79,193,78,78,192)
        (46,0,2,63,291,62,62,290) (47,1,3,47,389,46,46,388) (48,0,0,30,30,30,31,31) (49,1,1,14,128,14,15,129)
        (50,0,2,112,340,112,113,341) (51,1,3,96,438,96,97,439) (52,0,0,81,81,80,80,80)
        (53,1,1,65,179,64,64,178) (54,0,2,49,277,48,48,276) (55,1,3,33,375,32,32,374) (56,0,0,16,16,16,17,17)
        (57,1,1,0,114,0,1,115) (58,0,2,98,326,98,99,327) (59,1,3,82,424,82,83,425) (60,0,0,67,67,66,66,66)
        (61,1,1,51,165,50,50,164) (62,0,2,35,263,34,34,262) (63,1,3,19,361,18,18,360) (64,0,0,2,2,2,3,3)
        (65,1,1,100,214,100,101,215) (66,0,2,84,312,84,85,313) (67,1,3,68,410,68,69,411)
        (68,0,0,53,53,52,52,52) (69,1,1,37,151,36,36,150) (70,0,2,21,249,20,20,248) (71,1,3,5,347,4,4,346)
        (72,0,0,102,102,102,103,103) (73,1,1,86,200,86,87,201) (74,0,2,70,298,70,71,299)
        (75,1,3,54,396,54,55,397) (76,0,0,39,39,38,38,38) (77,1,1,23,137,22,22,136) (78,0,2,7,235,6,6,234)
        (79,1,3,105,447,104,104,446) (80,0,0,88,88,88,89,89) (81,1,1,72,186,72,73,187)
        (82,0,2,56,284,56,57,285) (83,1,3,40,382,40,41,383) (84,0,0,25,25,24,24,24) (85,1,1,9,123,8,8,122)
        (86,0,2,107,335,106,106,334) (87,1,3,91,433,90,90,432) (88,0,0,74,74,74,75,75)
        (89,1,1,58,172,58,59,173) (90,0,2,42,270,42,43,271) (91,1,3,26,368,26,27,369) (92,0,0,11,11,10,10,10)
        (93,1,1,109,223,108,108,222) (94,0,2,93,321,92,92,320) (95,1,3,77,419,76,76,418)
        (96,0,0,60,60,60,61,61) (97,1,1,44,158,44,45,159) (98,0,2,28,256,28,29,257) (99,1,3,12,354,12,13,355)
        (100,0,0,111,111,110,110,110) (101,1,1,95,209,94,94,208) (102,0,2,79,307,78,78,306)
        (103,1,3,63,405,62,62,404) (104,0,0,46,46,46,47,47) (105,1,1,30,144,30,31,145)
        (106,0,2,14,242,14,15,243) (107,1,3,112,454,112,113,455) (108,0,0,97,97,96,96,96)
        (109,1,1,81,195,80,80,194) (110,0,2,65,293,64,64,292) (111,1,3,49,391,48,48,390)
        (112,0,0,32,32,32,33,33) (113,1,1,16,130,16,17,131) (114,0,2,0,228,0,1,229) (115,1,3,98,440,98,99,441)
        (116,0,0,83,83,82,82,82) (117,1,1,67,181,66,66,180) (118,0,2,51,279,50,50,278)
        (119,1,3,35,377,34,34,376) (120,0,0,18,18,18,19,19) (121,1,1,2,116,2,3,117)
        (122,0,2,100,328,100,101,329) (123,1,3,84,426,84,85,427) (124,0,0,69,69,68,68,68)
        (125,1,1,53,167,52,52,166) (126,0,2,37,265,36,36,264) (127,1,3,21,363,20,20,362) (128,0,0,4,4,4,5,5)
        (129,1,1,102,216,102,103,217) (130,0,2,86,314,86,87,315) (131,1,3,70,412,70,71,413)
        (132,0,0,55,55,54,54,54) (133,1,1,39,153,38,38,152) (134,0,2,23,251,22,22,250) (135,1,3,7,349,6,6,348)
        (136,0,0,104,104,104,105,105) (137,1,1,88,202,88,89,203) (138,0,2,72,300,72,73,301)
        (139,1,3,56,398,56,57,399) (140,0,0,41,41,40,40,40) (141,1,1,25,139,24,24,138) (142,0,2,9,237,8,8,236)
        (143,1,3,107,449,106,106,448) (144,0,0,90,90,90,91,91) (145,1,1,74,188,74,75,189)
        (146,0,2,58,286,58,59,287) (147,1,3,42,384,42,43,385) (148,0,0,27,27,26,26,26)
        (149,1,1,11,125,10,10,124) (150,0,2,109,337,108,108,336) (151,1,3,93,435,92,92,434)
        (152,0,0,76,76,76,77,77) (153,1,1,60,174,60,61,175) (154,0,2,44,272,44,45,273)
        (155,1,3,28,370,28,29,371) (156,0,0,13,13,12,12,12) (157,1,1,111,225,110,110,224)
        (158,0,2,95,323,94,94,322) (159,1,3,79,421,78,78,420) (160,0,0,62,62,62,63,63)
```

(161,1,1,46,160,46,47,161) (162,0,2,30,258,30,31,259) (163,1,3,14,356,14,15,357)
(164,0,0,113,113,112,112,112) (165,1,1,97,211,96,96,210) (166,0,2,81,309,80,80,308)
(167,1,3,65,407,64,64,406) (168,0,0,48,48,48,49,49) (169,1,1,32,146,32,33,147)
(170,0,2,16,244,16,17,245) (171,1,3,0,342,0,1,343) (172,0,0,99,99,98,98,98) (173,1,1,83,197,82,82,196)
(174,0,2,67,295,66,66,294) (175,1,3,51,393,50,50,392) (176,0,0,34,34,34,35,35)
(177,1,1,18,132,18,19,133) (178,0,2,2,230,2,3,231) (179,1,3,100,442,100,101,443)
(180,0,0,85,85,84,84,84) (181,1,1,69,183,68,68,182) (182,0,2,53,281,52,52,280)
(183,1,3,37,379,36,36,378) (184,0,0,20,20,20,21,21) (185,1,1,4,118,4,5,119)
(186,0,2,102,330,102,103,331) (187,1,3,86,428,86,87,429) (188,0,0,71,71,70,70,70)
(189,1,1,55,169,54,54,168) (190,0,2,39,267,38,38,266) (191,1,3,23,365,22,22,364) (192,0,0,6,6,6,7,7)
(193,1,1,104,218,104,105,219) (194,0,2,88,316,88,89,317) (195,1,3,72,414,72,73,415)
(196,0,0,57,57,56,56,56) (197,1,1,41,155,40,40,154) (198,0,2,25,253,24,24,252) (199,1,3,9,351,8,8,350)
(200,0,0,106,106,106,107,107) (201,1,1,90,204,90,91,205) (202,0,2,74,302,74,75,303)
(203,1,3,58,400,58,59,401) (204,0,0,43,43,42,42,42) (205,1,1,27,141,26,26,140)
(206,0,2,11,239,10,10,238) (207,1,3,109,451,108,108,450) (208,0,0,92,92,92,93,93)
(209,1,1,76,190,76,77,191) (210,0,2,60,288,60,61,289) (211,1,3,44,386,44,45,387)
(212,0,0,29,29,28,28,28) (213,1,1,13,127,12,12,126) (214,0,2,111,339,110,110,338)
(215,1,3,95,437,94,94,436) (216,0,0,78,78,78,79,79) (217,1,1,62,176,62,63,177)
(218,0,2,46,274,46,47,275) (219,1,3,30,372,30,31,373) (220,0,0,15,15,14,14,14)
(221,1,1,113,227,112,112,226) (222,0,2,97,325,96,96,324) (223,1,3,81,423,80,80,422)
(224,0,0,64,64,64,65,65) (225,1,1,48,162,48,49,163) (226,0,2,32,260,32,33,261)
(227,1,3,16,358,16,17,359) (228,0,0,1,1,0,0,0) (229,1,1,99,213,98,98,212) (230,0,2,83,311,82,82,310)
(231,1,3,67,409,66,66,408) (232,0,0,50,50,50,51,51) (233,1,1,34,148,34,35,149)
(234,0,2,18,246,18,19,247) (235,1,3,2,344,2,3,345) (236,0,0,101,101,100,100,100)
(237,1,1,85,199,84,84,198) (238,0,2,69,297,68,68,296) (239,1,3,53,395,52,52,394)
(240,0,0,36,36,36,37,37) (241,1,1,20,134,20,21,135) (242,0,2,4,232,4,5,233)
(243,1,3,102,444,102,103,445) (244,0,0,87,87,86,86,86) (245,1,1,71,185,70,70,184)
(246,0,2,55,283,54,54,282) (247,1,3,39,381,38,38,380) (248,0,0,22,22,22,23,23) (249,1,1,6,120,6,7,121)
(250,0,2,104,332,104,105,333) (251,1,3,88,430,88,89,431) (252,0,0,73,73,72,72,72)
(253,1,1,57,171,56,56,170) (254,0,2,41,269,40,40,268) (255,1,3,25,367,24,24,366) (256,0,0,8,8,8,9,9)
(257,1,1,106,220,106,107,221) (258,0,2,90,318,90,91,319) (259,1,3,74,416,74,75,417)
(260,0,0,59,59,58,58,58) (261,1,1,43,157,42,42,156) (262,0,2,27,255,26,26,254)
(263,1,3,11,353,10,10,352) (264,0,0,108,108,108,109,109) (265,1,1,92,206,92,93,207)
(266,0,2,76,304,76,77,305) (267,1,3,60,402,60,61,403) (268,0,0,45,45,44,44,44)
(269,1,1,29,143,28,28,142) (270,0,2,13,241,12,12,240) (271,1,3,111,453,110,110,452)
(272,0,0,94,94,94,95,95) (273,1,1,78,192,78,79,193) (274,0,2,62,290,62,63,291)
(275,1,3,46,388,46,47,389) (276,0,0,31,31,30,30,30) (277,1,1,15,129,14,14,128)
(278,0,2,113,341,112,112,340) (279,1,3,97,439,96,96,438) (280,0,0,80,80,80,81,81)
(281,1,1,64,178,64,65,179) (282,0,2,48,276,48,49,277) (283,1,3,32,374,32,33,375)
(284,0,0,17,17,16,16,16) (285,1,1,1,115,0,0,114) (286,0,2,99,327,98,98,326) (287,1,3,83,425,82,82,424)
(288,0,0,66,66,66,67,67) (289,1,1,50,164,50,51,165) (290,0,2,34,262,34,35,263)
(291,1,3,18,360,18,19,361) (292,0,0,3,3,2,2,2) (293,1,1,101,215,100,100,214)
(294,0,2,85,313,84,84,312) (295,1,3,69,411,68,68,410) (296,0,0,52,52,52,53,53)
(297,1,1,36,150,36,37,151) (298,0,2,20,248,20,21,249) (299,1,3,4,346,4,5,347)
(300,0,0,103,103,102,102,102) (301,1,1,87,201,86,86,200) (302,0,2,71,299,70,70,298)
(303,1,3,55,397,54,54,396) (304,0,0,38,38,38,39,39) (305,1,1,22,136,22,23,137) (306,0,2,6,234,6,7,235)
(307,1,3,104,446,104,105,447) (308,0,0,89,89,88,88,88) (309,1,1,73,187,72,72,186)
(310,0,2,57,285,56,56,284) (311,1,3,41,383,40,40,382) (312,0,0,24,24,24,25,25) (313,1,1,8,122,8,9,123)
(314,0,2,106,334,106,107,335) (315,1,3,90,432,90,91,433) (316,0,0,75,75,74,74,74)
(317,1,1,59,173,58,58,172) (318,0,2,43,271,42,42,270) (319,1,3,27,369,26,26,368)
(320,0,0,10,10,10,11,11) (321,1,1,108,222,108,109,223) (322,0,2,92,320,92,93,321)
(323,1,3,76,418,76,77,419) (324,0,0,61,61,60,60,60) (325,1,1,45,159,44,44,158)
(326,0,2,29,257,28,28,256) (327,1,3,13,355,12,12,354) (328,0,0,110,110,110,111,111)
(329,1,1,94,208,94,95,209) (330,0,2,78,306,78,79,307) (331,1,3,62,404,62,63,405)
(332,0,0,47,47,46,46,46) (333,1,1,31,145,30,30,144) (334,0,2,15,243,14,14,242)
(335,1,3,113,455,112,112,454) (336,0,0,96,96,96,97,97) (337,1,1,80,194,80,81,195)
(338,0,2,64,292,64,65,293) (339,1,3,48,390,48,49,391) (340,0,0,33,33,32,32,32)
(341,1,1,17,131,16,16,130) (342,0,2,1,229,0,0,228) (343,1,3,99,441,98,98,440) (344,0,0,82,82,82,83,83)
(345,1,1,66,180,66,67,181) (346,0,2,50,278,50,51,279) (347,1,3,34,376,34,35,377)
(348,0,0,19,19,18,18,18) (349,1,1,3,117,2,2,116) (350,0,2,101,329,100,100,328)
(351,1,3,85,427,84,84,426) (352,0,0,68,68,68,69,69) (353,1,1,52,166,52,53,167)
(354,0,2,36,264,36,37,265) (355,1,3,20,362,20,21,363) (356,0,0,5,5,4,4,4)
(357,1,1,103,217,102,102,216) (358,0,2,87,315,86,86,314) (359,1,3,71,413,70,70,412)
(360,0,0,54,54,54,55,55) (361,1,1,38,152,38,39,153) (362,0,2,22,250,22,23,251) (363,1,3,6,348,6,7,349)
(364,0,0,105,105,104,104,104) (365,1,1,89,203,88,88,202) (366,0,2,73,301,72,72,300)
(367,1,3,57,399,56,56,398) (368,0,0,40,40,40,41,41) (369,1,1,24,138,24,25,139) (370,0,2,8,236,8,9,237)

```
(371,1,3,106,448,106,107,449) (372,0,0,91,91,90,90,90) (373,1,1,75,189,74,74,188)
(374,0,2,59,287,58,58,286) (375,1,3,43,385,42,42,384) (376,0,0,26,26,26,27,27)
(377,1,1,10,124,10,11,125) (378,0,2,108,336,108,109,337) (379,1,3,92,434,92,93,435)
(380,0,0,77,77,76,76,76) (381,1,1,61,175,60,60,174) (382,0,2,45,273,44,44,272)
(383,1,3,29,371,28,28,370) (384,0,0,12,12,12,13,13) (385,1,1,110,224,110,111,225)
(386,0,2,94,322,94,95,323) (387,1,3,78,420,78,79,421) (388,0,0,63,63,62,62,62)
(389,1,1,47,161,46,46,160) (390,0,2,31,259,30,30,258) (391,1,3,15,357,14,14,356)
(392,0,0,112,112,112,113,113) (393,1,1,96,210,96,97,211) (394,0,2,80,308,80,81,309)
(395,1,3,64,406,64,65,407) (396,0,0,49,49,48,48,48) (397,1,1,33,147,32,32,146)
(398,0,2,17,245,16,16,244) (399,1,3,1,343,0,0,342) (400,0,0,98,98,98,99,99) (401,1,1,82,196,82,83,197)
(402,0,2,66,294,66,67,295) (403,1,3,50,392,50,51,393) (404,0,0,35,35,34,34,34)
(405,1,1,19,133,18,18,132) (406,0,2,3,231,2,2,230) (407,1,3,101,443,100,100,442)
(408,0,0,84,84,84,85,85) (409,1,1,68,182,68,69,183) (410,0,2,52,280,52,53,281)
(411,1,3,36,378,36,37,379) (412,0,0,21,21,20,20,20) (413,1,1,5,119,4,4,118)
(414,0,2,103,331,102,102,330) (415,1,3,87,429,86,86,428) (416,0,0,70,70,70,71,71)
(417,1,1,54,168,54,55,169) (418,0,2,38,266,38,39,267) (419,1,3,22,364,22,23,365) (420,0,0,7,7,6,6,6)
(421,1,1,105,219,104,104,218) (422,0,2,89,317,88,88,316) (423,1,3,73,415,72,72,414)
(424,0,0,56,56,56,57,57) (425,1,1,40,154,40,41,155) (426,0,2,24,252,24,25,253) (427,1,3,8,350,8,9,351)
(428,0,0,107,107,106,106,106) (429,1,1,91,205,90,90,204) (430,0,2,75,303,74,74,302)
(431,1,3,59,401,58,58,400) (432,0,0,42,42,42,43,43) (433,1,1,26,140,26,27,141)
(434,0,2,10,238,10,11,239) (435,1,3,108,450,108,109,451) (436,0,0,93,93,92,92,92)
(437,1,1,77,191,76,76,190) (438,0,2,61,289,60,60,288) (439,1,3,45,387,44,44,386)
(440,0,0,28,28,28,29,29) (441,1,1,12,126,12,13,127) (442,0,2,110,338,110,111,339)
(443,1,3,94,436,94,95,437) (444,0,0,79,79,78,78,78) (445,1,1,63,177,62,62,176)
(446,0,2,47,275,46,46,274) (447,1,3,31,373,30,30,372) (448,0,0,14,14,14,15,15)
(449,1,1,112,226,112,113,227) (450,0,2,96,324,96,97,325) (451,1,3,80,422,80,81,423)
(452,0,0,65,65,64,64,64) (453,1,1,49,163,48,48,162) (454,0,2,33,261,32,32,260)
(455,1,3,17,359,16,16,358)
```

2.11 /opt/GSM/c54x_exe/tools/cch_ref/cch_testvectors.txt

2882 octets, 24 lignes → 26 lignes (1 groupes compactés)

```
# GSM 05.03 xCCH stage-isolation test vector
# Fixed pseudo-random 228-bit u (u[224:228]=tail=0).
# TEST A (interleaver): feed burst0..3 into YOUR deinterleaver,
#   compare the 456-bit result against the three candidates below:
#     == cB_correct       -> interleaver OK, bug is downstream
#     == cB_lsb_dropped   -> you never apply the ((k%8)>>2) term
#     == cB_lsb_inverted  -> that term is inverted / off-by-one
#     == none of them     -> different bug (burst order? half swap?)
# TEST B (Viterbi/lane): feed cB_correct into YOUR Viterbi,
#   compare the 228-bit result against u. Mismatch with cB correct
#   => c0/c1 swap, wrong polynomials, or tail handling.

u_228           =
 ↪  00101111001011011001000010100110100110100101101111010110110100111010110000000111110100101101111101100000100001010100110001010

burst⟨1⟩_114      = ⟨2⟩  ×4
    ⟨⟩ =

 ↪  (0,01011110011011111100000111101111010000101100100010011111101110010010000111100111011110001100001011001110001011100010)

 ↪  (1,11111011000011100110011000100111111110111000111000111001010111010011010111110110110100111000000101110101011010100011)

 ↪  (2,10100000011101000100000011100000100010000111100111101100011000011101000001110101100000011100001001100100001111001000)

 ↪  (3,00111001110110100001010110010110101000010011010100111011111011111100100000000001100010101011101110000111010101010100)

cB_correct      =
 ↪  00001101110101010100111011011000101011111000111111011110110001010001110001010001110010110110001001010111010001100010100001

cB_lsb_dropped  =
 ↪  00000101110100010100010111010100101010011000010011011011110001000001100101011100110010110110110001010001010010100010111101
```

```
cB_lsb_inverted =
↪  1100010111000001110101011010010011011001011101001000101111000100110010010001110001101011011111000100000101001010110011111010
```

3 Fichiers

3.1 /opt/GSM/c54x_exe/.gitignore

625 octets, 28 lignes → 28 lignes

```
# build products (make) — never commit binaries
/c54x_exe
/isa_test
/dsp_tester
/tch_now
/tools/rejeu_banc
/tools/sacch_tf_decode
*.o
__pycache__/
*.pyc
# run artefacts
/tmp/
# ROM du DSP TI : jamais dans le depot, telechargee par rom/fetch-rom.sh (FreeCalypso)
/rom/calypso_dsp.*.bin
!/rom/calypso_dsp.Registers.bin
/rom/calypso_dsp.txt
/c54x_exe.*
/tools/layer1_tester
# derive de binutils-2.21.1.tar.bz2 par repro_toa.sh
/binutils-2.21.1/
/binutils-2.21.1.tar.bz2
/build-tic54x/
# ROM du DSP et derives (desassemblage, dump FreeCalypso) : jamais dans le depot
/PROM0.bin
*.dis
/dsp-rom-3606-dump.txt
*.avant-*
.avant-*/
```

3.2 /opt/GSM/c54x_exe/Makefile

4871 octets, 102 lignes → 102 lignes

```
# c54x_exe - the Calypso DSP outside QEMU.
#
# The sources come from /opt/GSM/qosmo and are NOT copied here.
QOSMO   ?= /opt/GSM/qosmo
CAL     := $(QOSMO)/hw/arm/calypso
L1DSP   := $(CAL)/l1-dsp
HORS    := $(QOSMO)/contrib/hors-qemu

CC      ?= gcc
CFLAGS  ?= -O3 -march=native -g -Wall -Werror=format -Werror=format-extra-args -Wno-unused-function
↪  -Wno-unused-variable \
          -Wno-unused-but-set-variable -Wno-sign-compare
CPPFLAGS := -D_GNU_SOURCE -I$(HORS)/doublures -I$(L1DSP) -I$(CAL) -I$(QOSMO)/include -I$(QOSMO)
# calypso_bsp.c encodes the RACH/NB bursts: gsm0503_rach_ext_encode
OSMO    := $(shell pkg-config --cflags --libs libosmocoding libosmocore 2>/dev/null)
LDLIBS  := -lpthread -lm $(OSMO)
```

```
SRC := src/main.c src/rejouer.c src/pcb-minimal.c src/verbosite.c src/pont.c src/cellule.c src/montant.c
 ↪  src/tsp_tx.c $(HORS)/cales-qemu.c \
       $(L1DSP)/calypso_gmsk.c \
       $(L1DSP)/calypso_c54x.c \
       $(L1DSP)/c54x_exec.c \
       $(L1DSP)/c54x_decode.c \
       $(L1DSP)/c54x_mem.c \
       $(L1DSP)/c54x_irq.c \
       $(L1DSP)/c54x_probes.c \
       $(L1DSP)/calypso_bsp.c \
       $(L1DSP)/calypso_arm2dsp.c \
       $(L1DSP)/calypso_mailbox.c \
       $(L1DSP)/calypso_fbsb.c \
       $(L1DSP)/calypso_dma.c \
       $(L1DSP)/calypso_rhea_dma.c \
       $(L1DSP)/calypso_rif.c \
       $(L1DSP)/calypso_a5.c \
       $(L1DSP)/calypso_twl3025.c \
       $(L1DSP)/calypso_vsp.c \
       $(CAL)/calypso_xio.c \
       $(CAL)/calypso_iota.c \
       $(CAL)/calypso_trf6151.c \
       $(CAL)/calypso_debug.c \
       $(CAL)/calypso_invariants.c

all: c54x_exe

# Headers are dependencies too, so that a change to a .h alone
# (calypso_c54x.h, calypso_bsp.h...) rebuilds the binary.
HDR := $(wildcard src/*.h $(L1DSP)/*.h $(CAL)/*.h $(QOSMO)/include/hw/arm/calypso/*.h)

# REBUILT ON EVERY make. The sources come from another repository (qosmo),
# so c54x_exe is rebuilt on each call, as after a make clean, rather than
# trusting "make: Nothing to be done".
# A single cc command, no .o: nothing else to clean.
.PHONY: c54x_exe
c54x_exe: $(SRC) $(HDR)
    $(CC) $(CFLAGS) $(CPPFLAGS) -o $@ $(SRC) $(LDLIBS)

# ISA conformance: the SPRU172C worked examples replayed on the core.
#   make isa_test && ./isa_test tools/isa_tests.txt 2>/dev/null
COEUR := $(filter-out src/%,$(SRC))
tools/isa_tests.txt: tools/isa_examples.py
    python3 tools/isa_examples.py > $@

# Off-bench replay of a recorded TCH (tools/rejeu_banc.c): /dev/shm/calypso_rejeu_tch.bin
#   make tools/rejeu_banc && ./tools/rejeu_banc [file] [max ticks]
tools/rejeu_banc: tools/rejeu_banc.c src/tsp_tx.c src/pcb-minimal.c src/verbosite.c $(COEUR) $(HDR)
    $(CC) $(CFLAGS) $(CPPFLAGS) -Isrc -o $@ tools/rejeu_banc.c src/tsp_tx.c src/pcb-minimal.c src/verbosite.c
 ↪  $(COEUR) $(LDLIBS)

isa_test: tools/isa_test.c src/pcb-minimal.c $(COEUR) tools/isa_tests.txt
    $(CC) $(CFLAGS) $(CPPFLAGS) -o $@ tools/isa_test.c src/pcb-minimal.c $(COEUR) $(LDLIBS)

# layer1_tester: addresses and functions of the layer1 <-> ROM interface, map
# docs/carte-memoire.md. Does NOT rebuild c54x_exe. The core is linked WITHOUT
# calypso_mailbox.c: tools/layer1_tester.c provides calypso_mbx_evt, the hook
# of every DSP access; fopen/open/socket/sendto are wrapped (nothing is
# written outside the output file, no socket).
#   make layer1_tester && python3 tools/layer1_tester.py [--rejeu FILE] [--md docs/carte-memoire.md]
#   make api_carte [REJEU=FILE]         (alias: regenerates docs/carte-memoire.md)
L1T_COEUR := $(filter-out $(L1DSP)/calypso_mailbox.c,$(COEUR))
L1T_WRAP  :=
 ↪  -Wl,--wrap=fopen,--wrap=open,--wrap=socket,--wrap=sendto,--wrap=calypso_a5_portw,--wrap=calypso_a5_portr
REJEU     ?=
tools/layer1_tester: tools/layer1_tester.c src/pcb-minimal.c $(L1T_COEUR) $(HDR)
```

```
    $(CC) $(CFLAGS) $(CPPFLAGS) -o $@ tools/layer1_tester.c src/pcb-minimal.c $(L1T_COEUR) $(LDLIBS) $(L1T_WRAP)

layer1_tester: tools/layer1_tester

api_carte: tools/layer1_tester
    python3 tools/layer1_tester.py $(if $(REJEU),--rejeu $(REJEU)) --md docs/carte-memoire.md

# DSP function tester (tools/dsp_tester.c): the osmocom-bb ARM and the BTS
# are simulated (tools/dsp_banc_commun.c), one PASS/FAIL/NON-IMPL line per
# DSP task. Does NOT rebuild c54x_exe.
#   make dsp_tester && ./dsp_tester --all
# The API RAM structures and the parameter table are those of the osmocom-bb
# firmware (dsp_api.h, dsp_params.c), read from its tree.
OSMOBB  ?= /opt/GSM/osmocom-bb/src/target/firmware
BANC_CPPFLAGS := -idirafter $(OSMOBB)/include/calypso -idirafter $(OSMOBB)/calypso
BANC    := tools/dsp_banc_commun.c src/pcb-minimal.c src/verbosite.c
dsp_tester: tools/dsp_tester.c $(BANC) tools/dsp_banc_commun.h $(COEUR) $(HDR)
    $(CC) $(CFLAGS) $(CPPFLAGS) -Isrc $(BANC_CPPFLAGS) -o $@ tools/dsp_tester.c $(BANC) $(COEUR) $(LDLIBS)

clean:
    rm -f c54x_exe isa_test tools/layer1_tester dsp_tester

.PHONY: all clean layer1_tester api_carte
```

3.3 /opt/GSM/c54x_exe/balayage.sh

3758 octets, 74 lignes → 74 lignes

```
#!/usr/bin/env bash
# balayage.sh - sweep the replay parameters that are not known for sure, and
# rank the combinations by the only criterion that matters: the number of TRUE
# SBs (CRC OK AND BSIC = the injected one AND valid T3 AND FN = the frame).
#
# Do not rank on crc_ok. The SCH CRC is 10 bits: random input would pass
# ~0.1 % of the time, while a degenerate (near-constant) input makes the
# Viterbi converge to the same word every time and passes far more often.
# Ranking on crc_ok would therefore reward a frozen output. The secondary
# criterion is the number of DISTINCT decoded words: it measures whether the
# output depends on the input.
#
# Several bench knobs are unverified hypotheses: the sampling phase (0.5 =
# symbol centre), the head margin, the alignment of the SB command (the DSP
# demodulates the burst of the LAST command frame), the number of commands,
# the RX order, the DMA window length. Tuning them one at a time misses the
# interactions, hence a full grid.
#
#   ./balayage.sh              full sweep, results in balayage.tsv
#   ./balayage.sh --rapide     reduced grid
#   TRAMES=1200 ./balayage.sh  more frames per trial (default 600)
#   sort -t$'\t' -k6,6nr -k5,5nr balayage.tsv | head   re-read the ranking
set -uo pipefail
HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
EXE="$HERE/c54x_exe"
OUT="${OUT:-$HERE/balayage.tsv}"
TRAMES="${TRAMES:-600}"
[ -x "$EXE" ] || { echo "binaire absent : $EXE" >&2; exit 1; }

if [ "${1:-}" = "--rapide" ]; then
    DECALAGES="-1 0";           UNIQUES="0 1"
    PHASES="0.0 0.25 0.5 0.75"; MARGES="0 21 41"
    RXAVANT="0";                LENS="380"
else
    DECALAGES="-5 -4 -3 -2 -1 0 1";   UNIQUES="0 1"
    PHASES="0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9"
```

```
    MARGES="0 10 21 30 41";     RXAVANT="0 1"
    LENS="296 380"
fi

total=0
for a in $DECALAGES; do for b in $UNIQUES; do for c in $PHASES; do
for d in $MARGES; do for e in $RXAVANT; do for f in $LENS; do
    total=$((total+1)); done; done; done; done; done; done

printf 'decalage\tunique\tphase\tmarge\trx_avant\tdaram_len\tvraies\tmots_distincts\tcrc_ok\tsb_tentees\n' >
 ↪  "$OUT"
echo "[balayage] $total combinaisons, $TRAMES trames chacune -> $OUT"
n=0; t0=$(date +%s)
for a in $DECALAGES; do
 for b in $UNIQUES; do
  for c in $PHASES; do
   for d in $MARGES; do
    for e in $RXAVANT; do
     for f in $LENS; do
      n=$((n+1))
      res=$(REJEU_SB_FORCE=1 \
            REJEU_SB_DECALAGE="$a" REJEU_SB_UNIQUE="$b" \
            REJEU_DECALAGE_SYMB="$c" REJEU_MARGE="$d" \
            REJEU_RX_AVANT="$e" CALYPSO_BSP_DARAM_LEN="$f" \
            timeout 180 "$EXE" --rejouer --verbeux --trames "$TRAMES" 2>/dev/null)
      vraies=$(printf '%s' "$res" | sed -n 's/.*VRAIES ([^)]*): \([0-9]*\).*/\1/p' | head -1)
      crc=$(   printf '%s' "$res" | sed -n 's/.*CRC OK: \([0-9]*\).*/\1/p' | head -1)
      tent=$(  printf '%s' "$res" | sed -n 's#.*SB tentees / CRC KO: \([0-9]*\) /.*#\1#p' | head -1)
      mots=$(  printf '%s' "$res" | grep -oE '^  SB[0-9] fn=[0-9]+ : sb=0x[0-9a-f]+' | grep -oE '0x[0-9a-f]+' |
       ↪  sort -u | wc -l)
      printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \
             "$a" "$b" "$c" "$d" "$e" "$f" "${vraies:-0}" "${mots:-0}" "${crc:-0}" "${tent:-0}" >> "$OUT"
      if [ $((n % 25)) -eq 0 ]; then
          dt=$(( $(date +%s) - t0 ))
          echo "[balayage] $n/$total  ${dt}s  meilleur vraies=$(tail -n +2 "$OUT" | cut -f7 | sort -nr | head -1)
           ↪  mots_distincts=$(tail -n +2 "$OUT" | cut -f8 | sort -nr | head -1)"
      fi
     done; done; done; done; done; done
echo "[balayage] termine, $n essais."
echo "--- 15 meilleures combinaisons (vraies, puis mots distincts) ---"
{ head -1 "$OUT"; tail -n +2 "$OUT" | sort -t$'\t' -k7,7nr -k8,8nr | head -15; } | column -t -s$'\t'
```

3.4 /opt/GSM/c54x_exe/harvard.sh

62586 octets, 840 lignes → 840 lignes

```
#!/bin/sh
# harvard.sh - "Harvard" shell for the Calypso's TMS320C54x DSP (POSIX sh, dash is enough).
#
# The C54x is a Harvard machine: the P space (program, the opcodes) and the D space (data,
# registers, API RAM) are separate. This shell keeps both side by side:
#   - P side: type an instruction (mnemonic assembled by gas tic54x, or hex words),
#             disassemble the TI ROM (PROM0 at 0x7000, PDROM at 0xE000, pages 1..3 via XPC);
#   - D side: a state (registers + data memory words) that is read, set, and that every
#             executed instruction advances; the DROM (0x9000) and the PDROM can be read.
# Executing an instruction goes through ./isa_test (the real qosmo C54x core, pcb-minimal):
# it is given the "before" state and the words, it returns the "after" state. Nothing is
# simulated here. The "run" commands launch the bench tools as they are (c54x_exe,
# dsp_tester, isa_test, run.sh).
#
#   ./harvard.sh                interactive menu
#   ./harvard.sh repl           direct command prompt
#   ./harvard.sh pas 'LD #0x1234, A'    one command then exit (any REPL command)
#   ./harvard.sh aide
```

```
# Variables: HARVARD_DIR (state, default ~/.local/state/harvard-c54x), ROM_DIR (default ./rom),
#            ISA_TEST, OBJDUMP, GAS (tool paths).

set -u
HERE=$(cd "$(dirname "$0")" && pwd)
DIR=${HARVARD_DIR:-${XDG_STATE_HOME:-$HOME/.local/state}/harvard-c54x}
ROM_DIR=${ROM_DIR:-$HERE/rom}
ISA_TEST=${ISA_TEST:-$HERE/isa_test}
OBJDUMP=${OBJDUMP:-$HERE/build-tic54x/binutils/objdump}
GAS=${GAS:-$HERE/build-tic54x/gas/as-new}
ETAT=$DIR/etat.txt          # one line per item: "REG hex" | "M addr hex" | "P addr hex"
JOURNAL=$DIR/journal.txt
LOG=$DIR/coeur.log          # stderr of the core ([c54x] warnings ...)
REGS="A B T SP PC ST0 ST1 PMST AR0 AR1 AR2 AR3 AR4 AR5 AR6 AR7 BK BRC RSA REA TRN IMR IFR XPC"
SENT_REG=deadbeef1          # 36-bit sentinel: never equal to a register -> isa_test prints "obtenu"
SENT_MEM=1ffff              # 17-bit sentinel: never equal to a 16-bit word
mkdir -p "$DIR"; : >> "$ETAT"; : >> "$JOURNAL"

# ------------------------------------------------------------------ helpers
rouge() { if [ -t 1 ]; then printf '\033[31m%s\033[0m\n' "$*"; else printf '%s\n' "$*"; fi; }
vert()  { if [ -t 1 ]; then printf '\033[32m%s\033[0m\n' "$*"; else printf '%s\n' "$*"; fi; }
gras()  { if [ -t 1 ]; then printf '\033[1m%s\033[0m\n' "$*"; else printf '%s\n' "$*"; fi; }
hex() { # hex <text>: accepts 0x1234, 1234, 1:8000 (page:address); returns the integer
    case $1 in
        *:*) printf '%d' $(( ( ${1%%:*} << 16 ) | 0x${1#*:} )) ;;
        0x*|0X*) printf '%d' $(( $1 )) ;;
        *) printf '%d' $(( 0x$1 )) ;;
    esac 2>/dev/null
}
est_hex_mots() { # true if every argument is a hex word of 1 to 4 digits
    for m in "$@"; do case $m in ''|*[!0-9a-fA-F]*|?????*) return 1;; esac; done; return 0
}
verifier_outils() {
    [ -x "$ISA_TEST" ] || rouge "isa_test absent ($ISA_TEST) : make isa_test"
    [ -x "$OBJDUMP" ]  || rouge "objdump tic54x absent ($OBJDUMP) : pas de desassemblage"
    [ -x "$GAS" ]      || rouge "gas tic54x absent ($GAS) : seuls les mots hex sont acceptes"
}

# ------------------------------------------------------------------ state (D side)
etat_reg() { awk -v r="$1" '$1==r {v=$2} END {print v}' "$ETAT"; }
etat_fixe() { # etat_fixe <key...> <val>: replaces or adds the line "<keys> <val>"
    cle=$1; shift
    [ $# -gt 1 ] && { cle="$cle $1"; shift; }
    { grep -v "^$cle " "$ETAT"; echo "$cle $1"; } > "$ETAT.new" && mv "$ETAT.new" "$ETAT"
}
etat_mem() { awk -v a="$1" '$1=="M" && $2==a {v=$3} END {print v}' "$ETAT"; }
drapeaux() { # drapeaux <st0> <st1>: names of the bits set to 1
    st0=$(( 0x${1:-0} )); st1=$(( 0x${2:-0} )); d=""
    [ $(( st0 >> 12 & 1 )) = 1 ] && d="$d TC";  [ $(( st0 >> 11 & 1 )) = 1 ] && d="$d C"
    [ $(( st0 >> 10 & 1 )) = 1 ] && d="$d OVA"; [ $(( st0 >> 9 & 1 )) = 1 ]  && d="$d OVB"
    [ $(( st1 >> 15 & 1 )) = 1 ] && d="$d BRAF"; [ $(( st1 >> 14 & 1 )) = 1 ] && d="$d CPL"
    [ $(( st1 >> 13 & 1 )) = 1 ] && d="$d XF";  [ $(( st1 >> 12 & 1 )) = 1 ] && d="$d HM"
    [ $(( st1 >> 11 & 1 )) = 1 ] && d="$d INTM"; [ $(( st1 >> 9 & 1 )) = 1 ] && d="$d OVM"
    [ $(( st1 >> 8 & 1 )) = 1 ]  && d="$d SXM"; [ $(( st1 >> 7 & 1 )) = 1 ]  && d="$d C16"
    [ $(( st1 >> 6 & 1 )) = 1 ]  && d="$d FRCT"; [ $(( st1 >> 5 & 1 )) = 1 ] && d="$d CMPT"
    printf 'ARP=%d DP=%03x ASM=%02x flags:%s' $(( st0 >> 13 )) $(( st0 & 0x1ff )) $(( st1 & 0x1f )) "${d:- (aucun)}"
}
montrer_regs() {
    [ -s "$ETAT" ] || { echo "(etat vide : les registres ont les valeurs d'init du coeur ; faites un pas)";
     ↪  return; }
    printf '  A  =%12s   B  =%12s   T  =%6s   PC =%6s   SP =%6s   XPC=%4s\n' \
        "$(etat_reg A)" "$(etat_reg B)" "$(etat_reg T)" "$(etat_reg PC)" "$(etat_reg SP)" "$(etat_reg XPC)"
    printf '  AR0=%6s AR1=%6s AR2=%6s AR3=%6s AR4=%6s AR5=%6s AR6=%6s AR7=%6s\n' \
        "$(etat_reg AR0)" "$(etat_reg AR1)" "$(etat_reg AR2)" "$(etat_reg AR3)" "$(etat_reg AR4)" "$(etat_reg
         ↪  AR5)" "$(etat_reg AR6)" "$(etat_reg AR7)"
    printf '  ST0=%6s ST1=%6s PMST=%5s BK=%6s BRC=%6s RSA=%6s REA=%6s TRN=%6s IMR=%5s IFR=%5s\n' \
```

```
        "$(etat_reg ST0)" "$(etat_reg ST1)" "$(etat_reg PMST)" "$(etat_reg BK)" "$(etat_reg BRC)" "$(etat_reg
        ↪  RSA)" "$(etat_reg REA)" "$(etat_reg TRN)" "$(etat_reg IMR)" "$(etat_reg IFR)"
    printf '  %s\n' "$(drapeaux "$(etat_reg ST0)" "$(etat_reg ST1)")"
}
montrer_mem() {
    n=$(grep -c '^M ' "$ETAT"); [ "$n" = 0 ] && { echo "  D : aucun mot pose"; return; }
    echo "  D (mots poses / lus, $n) :"; awk '$1=="M" {printf "   [%04x]=%04x", strtonum("0x"$2),
    ↪  strtonum("0x"$3); if (++k%6==0) print ""} END {if (k%6) print ""}' "$ETAT" 2>/dev/null \
      || awk '$1=="M" {printf "   [%s]=%s", $2, $3; if (++k%6==0) print ""} END {if (k%6) print ""}' "$ETAT"
    n=$(grep -c '^P ' "$ETAT"); [ "$n" != 0 ] && { echo "  P (mots poses, $n) :"; awk '$1=="P" {printf "
    ↪  [%s]=%s", $2, $3; if (++k%6==0) print ""} END {if (k%6) print ""}' "$ETAT"; }
}
cmd_etat() { gras "--- registres (cote D)"; montrer_regs; montrer_mem; }
cmd_raz() { : > "$ETAT"; echo "etat remis a zero (valeurs d'init du coeur au prochain pas)"; }
cmd_reg() {
    [ $# -lt 2 ] && { montrer_regs; return; }
    r=$(printf '%s' "$1" | tr a-z A-Z); v=$(hex "$2") || { rouge "valeur invalide"; return 1; }
    etat_fixe "$r" "$(printf '%x' "$v")"; echo "$r <- $(printf '%x' "$v")"
}

# ------------------------------------------------------------------ assembly / disassembly (P side)
asm() { # asm <instruction> -> hex words on stdout, 1 on failure
    if est_hex_mots $1; then printf '%s\n' "$*" | tr A-F a-f; return 0; fi
    [ -x "$GAS" ] || { rouge "pas d'assembleur : donnez les mots hex"; return 1; }
    printf '\t.text\n\t%s\n' "$*" > "$DIR/a.s"
    if ! "$GAS" -o "$DIR/a.o" "$DIR/a.s" 2> "$DIR/a.err"; then
        rouge "assemblage refuse :"; sed 's/^/    /' "$DIR/a.err"; return 1
    fi
    "$OBJDUMP" -d "$DIR/a.o" | sed -n 's/^ *[0-9a-f]*:[[:space:]]*\([0-9a-f]\{4\}\).*/\1/p' | tr '\n' ' ' | sed
    ↪  's/ $/\n/'
}
dis_mots() { # dis_mots <address> <hex words...>: disassembles words
    a=$1; shift; : > "$DIR/d.bin"
    for m in "$@"; do v=$(( 0x$m )); printf "\\$(printf '%03o' $(( v & 255 )))\\$(printf '%03o' $(( v >> 8 )))" >>
    ↪  "$DIR/d.bin"; done
    "$OBJDUMP" -D -b binary -m tms320c54x --adjust-vma="0x${a:-9000}" "$DIR/d.bin" | sed -n 's/^
    ↪  *\([0-9a-f]*:.*\)/    \1/p'
}
rom_fichier() { # rom_fichier <linear P or D addr> <P|D> -> "file base_words"
    a=$1
    if [ "$2" = P ]; then
        page=$(( a >> 16 )); off=$(( a & 0xffff ))
        case $page in
            0) if [ $off -ge $(( 0xe000 )) ]; then echo "$ROM_DIR/calypso_dsp.PDROM.bin $(( 0xe000 ))"
               elif [ $off -ge $(( 0x7000 )) ]; then echo "$ROM_DIR/calypso_dsp.PROM0.bin $(( 0x7000 ))"; else
                ↪  return 1; fi ;;
            1|2|3) [ $off -ge $(( 0x8000 )) ] || return 1; echo "$ROM_DIR/calypso_dsp.PROM$page.bin $(( (page <<
             ↪  16) | 0x8000 ))" ;;
            *) return 1 ;;
        esac
    else
        if [ $a -ge $(( 0xe000 )) ]; then echo "$ROM_DIR/calypso_dsp.PDROM.bin $(( 0xe000 ))"
        elif [ $a -ge $(( 0x9000 )) ]; then echo "$ROM_DIR/calypso_dsp.DROM.bin $(( 0x9000 ))"; else return 1; fi
    fi
}
cmd_p() { # p <address> [n words]: disassemble the program ROM
    [ $# -ge 1 ] || { echo "usage : p <adresse> [n]   (0x7000.., 0xe000.., 1:8000 = page 1)"; return 1; }
    a=$(hex "$1") || { rouge "adresse invalide"; return 1; }; n=${2:-16}
    set -- $(rom_fichier "$a" P) || { rouge "$(printf '%x' "$a") : hors ROM programme (PROM0 7000-dfff, PDROM
    ↪  e000-ffff, pages 1-3 : 8000-ffff)"; return 1; }
    f=$1; base=$2; [ -r "$f" ] || { rouge "ROM absente : $f (rom/fetch-rom.sh)"; return 1; }
    page=$(( a >> 16 )); off=$(( a & 0xffff )); vma=$(( base & 0xffff ))
    [ $page -gt 0 ] && gras "  page $page (XPC=$page), fichier $(basename "$f")"
    "$OBJDUMP" -D -b binary -m tms320c54x --adjust-vma=$vma --start-address=$off --stop-address=$(( off + n ))
    ↪  "$f" \
        | sed -n 's/^ *\([0-9a-f]*:.*\)/    \1/p'
```

```
}
cmd_drom() { # drom <address> [n]: read the data ROM
    [ $# -ge 1 ] || { echo "usage : drom <adresse> [n]   (DROM 9000-dfff, PDROM e000-ffff)"; return 1; }
    a=$(hex "$1") || { rouge "adresse invalide"; return 1; }; n=${2:-16}
    set -- $(rom_fichier "$a" D) || { rouge "$(printf '%x' "$a") : hors ROM donnees"; return 1; }
    f=$1; base=$2; [ -r "$f" ] || { rouge "ROM absente : $f"; return 1; }
    od -An -v -tx2 -w16 -j $(( (a - base) * 2 )) -N $(( n * 2 )) "$f" | awk -v a=$a '{printf "    [%04x]", a;
     ↪  print; a+=8}'
}
cmd_asm() { m=$(asm "$@") || return 1; printf '    mots : %s\n' "$m"; dis_mots "$(etat_reg PC)" $m; }
cmd_dis() { est_hex_mots "$@" || { echo "usage : dis <mots hex>"; return 1; }; dis_mots "$(etat_reg PC)" "$@"; }

# ------------------------------------------------------------------ execution of one instruction (P -> D)
surveiller() { # surveiller <word1> [word2]: D addresses to read at each step (those a single instruction can
 ↪  touch)
    w1=$(( 0x$1 )); st0=$(etat_reg ST0); sp=$(etat_reg SP)
    {
        for rg in AR0 AR1 AR2 AR3 AR4 AR5 AR6 AR7; do v=$(etat_reg $rg); [ -n "$v" ] || continue
            v=$(( 0x$v )); printf '%04x\n%04x\n%04x\n' $(( (v - 1) & 0xffff )) $v $(( (v + 1) & 0xffff )); done
        [ -n "$sp" ] && { v=$(( 0x$sp )); printf '%04x\n%04x\n%04x\n' $(( (v - 1) & 0xffff )) $v $(( (v + 1) &
         ↪  0xffff )); }
        printf '%04x\n' $(( ((0x${st0:-0} & 0x1ff) << 7) | (w1 & 0x7f) ))      # direct addressing DP:dma
        [ $# -ge 2 ] && printf '%04x\n' $(( 0x$2 & 0xffff ))                    # long address lk
    } | sort -u > "$DIR/cand.txt"; awk '$1=="M" {print $2}' "$ETAT" > "$DIR/connu.txt"; grep -v -F -x -f
     ↪  "$DIR/connu.txt" "$DIR/cand.txt" 2>/dev/null || cat "$DIR/cand.txt"
}
executer() { # executer <keep 0|1> <title> <hex words...> [D addresses to read...]  ->  $DIR/pas.apres: lines
 ↪  "NAME value"
    garder=$1; titre=$2; mots=$3; shift 3
    t=$DIR/pas.txt; nm=$(grep -c '^M ' "$ETAT"); place=$(( 28 - nm )); [ $place -lt 0 ] && place=0
    {
        echo "T 0 $titre"; echo "W $mots"
        sed -n 's/^\([A-Z][A-Z0-9]*\) \([0-9a-fA-F]*\)$/B \1 \2/p; s/^\([MP]\) \([0-9a-fA-F]*\)
         ↪  \([0-9a-fA-F]*\)$/B \1 \2 \3/p' "$ETAT"
        for rg in $REGS; do echo "A $rg $SENT_REG"; done
        awk '$1=="M" {print "A M " $2 " '"$SENT_MEM"'"}' "$ETAT"
        { for adr in "$@"; do echo "$adr"; done; [ $# -eq 0 ] && surveiller $mots; } | head -n $place | sed "s/^/A
         ↪  M /; s/\$/ $SENT_MEM/"
        echo "E"
    } > "$t"
    "$ISA_TEST" "$t" 2>> "$LOG" > "$DIR/pas.out"
    grep -q '^FAIL' "$DIR/pas.out" || { rouge "isa_test n'a rien rendu :"; cat "$DIR/pas.out"; return 1; }
    grep -q '\[[0-9a-f ]*?\]' "$DIR/pas.out" && rouge "  (un registre de l'etat est inconnu d'isa_test, ignore)"
    # "  A: attendu deadbeef1, obtenu 12  data[0100]: attendu 1ffff, obtenu 0000" -> "A 12" / "M 0100 0000"
    sed -n 's/^FAIL[^]]*\]//p' "$DIR/pas.out" | sed 's/  /\n/g' | sed -n \
        's/^data\[\([0-9a-f]*\)\]: attendu [0-9a-f]*, obtenu \([0-9a-f]*\)$/M \1 \2/p;
         s/^prog\[\([0-9a-f]*\)\]: attendu [0-9a-f]*, obtenu \([0-9a-f]*\)$/P \1 \2/p;
         s/^\([A-Z][A-Z0-9]*\): attendu [0-9a-f]*, obtenu \([0-9a-f]*\)$/\1 \2/p' > "$DIR/pas.apres"
    if [ "$garder" = 1 ]; then
        # kept: the registers, the P words set, the D words already in the state, and any D word that became
         ↪  non-zero
        { grep '^P ' "$ETAT"
          awk 'FILENAME==ARGV[1] { if ($1=="M") connu[$2]=1; next }
               $1=="P" { next } $1!="M" { print; next } ($2 in connu) || ($3 != "0000" && $2 >= "0060") { print }'
 ↪  "$ETAT" "$DIR/pas.apres"
        } > "$ETAT.new" && mv "$ETAT.new" "$ETAT"
        printf '%s  %s  | %s\n' "$(date +%H:%M:%S)" "$mots" "$titre" >> "$JOURNAL"
    fi
}
diff_etat() { # diff_etat <before> <after>: what changed
    awk 'FILENAME==ARGV[1] { if ($1=="M"||$1=="P") av[$1" "$2]=$3; else av[$1]=$2; next }
         { if ($1=="M"||$1=="P") { k=$1" "$2; v=$3 } else { k=$1; v=$2 }
          if (!(k in av)) { if (k !~ /^M |^P /) nouveaux=nouveaux" "k"="v; else if (v != "0000") printf "    %-10s
 ↪  %12s -> %s\n", k, "(nouveau)", v }
           else if (av[k] != v) printf "    %-10s %12s -> %s\n", k, av[k], v }
         END { if (nouveaux != "") print "    (premiere lecture :" nouveaux ")" }' "$1" "$2"
```

```
}
cmd_pas() { # pas <instruction | hex words>
    [ $# -ge 1 ] || { echo "usage : pas <instruction>   ex : pas LD #0x1234, A   |   pas f020 1234"; return 1; }
    mots=$(asm "$@") || return 1
    pc=$(etat_reg PC); printf '  P[%s] %s   <- %s\n' "${pc:-9000}" "$mots" "$*"
    cp "$ETAT" "$DIR/avant.txt"
    executer 1 "$*" "$mots" || return 1
    gras "  changements :"; diff_etat "$DIR/avant.txt" "$ETAT" | grep . || echo "    (aucun registre ni mot pose
     ↪  n'a change)"
    printf '  PC=%s  A=%s  B=%s  T=%s  %s\n' "$(etat_reg PC)" "$(etat_reg A)" "$(etat_reg B)" "$(etat_reg T)"
     ↪  "$(drapeaux "$(etat_reg ST0)" "$(etat_reg ST1)")"
}
cmd_essai() { # essai <instruction>: execute without keeping the state
    [ $# -ge 1 ] || { echo "usage : essai <instruction>"; return 1; }
    mots=$(asm "$@") || return 1
    executer 0 "$*" "$mots" || return 1
    gras "  resultat (etat NON garde) :"; diff_etat "$ETAT" "$DIR/pas.apres" | grep . || echo "    (rien ne change)"
}
cmd_d() { # d <address> [n]: read the core's data memory (through a NOP that is not kept)
    [ $# -ge 1 ] || { echo "usage : d <adresse> [n<=24]"; return 1; }
    a=$(hex "$1") || { rouge "adresse invalide"; return 1; }; n=${2:-8}; [ $n -gt 24 ] && n=24
    liste=""; i=0; while [ $i -lt $n ]; do liste="$liste $(printf '%04x' $(( a + i )))"; i=$(( i + 1 )); done
    executer 0 "lecture D" f495 $liste || return 1
    awk -v a=$a 'BEGIN{k=0} $1=="M" { lu[$2]=$3 }
         END { for (i=0;i<'"$n"';i++) { k=sprintf("%04x", a+i); printf "    [%s]=%s", k, (k in lu)?lu[k]:"????";
 ↪  if ((i+1)%8==0) print "" } if (i%8) print "" }' "$DIR/pas.apres"
    st0=$(etat_reg ST0); echo "    (lecture par le coeur avec l'etat courant ; DP=$(printf '%03x' $(( 0x${st0:-0}
     ↪  & 0x1ff ))) ; les mots non poses valent 0 hors registres mappes 0000-005f)"
}
cmd_d_pose() { # d! <address> <words...>: set words in D
    [ $# -ge 2 ] || { echo "usage : d! <adresse> <mot> [mot...]"; return 1; }
    a=$(hex "$1") || { rouge "adresse invalide"; return 1; }; shift
    for m in "$@"; do v=$(hex "$m") || { rouge "mot invalide : $m"; return 1; }
        etat_fixe M "$(printf '%04x' $a)" "$(printf '%04x' $(( v & 0xffff )))"; printf '    D[%04x] <- %04x\n' $a
         ↪  $(( v & 0xffff )); a=$(( a + 1 )); done
}
cmd_p_pose() { # p! <address> <instruction | words>: set words in P (preloaded at each step)
    [ $# -ge 2 ] || { echo "usage : p! <adresse> <instruction|mots hex>   (ex. une sous-routine appelee par
     ↪  CALL)"; return 1; }
    a=$(hex "$1") || { rouge "adresse invalide"; return 1; }; shift
    mots=$(asm "$@") || return 1
    for m in $mots; do etat_fixe P "$(printf '%04x' $a)" "$m"; printf '    P[%04x] <- %s\n' $a "$m"; a=$(( a + 1
     ↪  )); done
}
cmd_journal() { gras "--- journal des pas"; tail -n ${1:-30} "$JOURNAL"; }
cmd_log() { gras "--- avertissements du coeur (dernieres lignes)"; tail -n ${1:-20} "$LOG"; }
cmd_sauve() { [ $# -ge 1 ] || { echo "usage : sauve <fichier>"; return 1; }; cp "$ETAT" "$1" && echo "etat -> $1"; }
cmd_charge() { [ $# -ge 1 ] && [ -r "$1" ] || { echo "usage : charge <fichier>"; return 1; }; cp "$1" "$ETAT" &&
 ↪  echo "etat <- $1" && montrer_regs; }

# ------------------------------------------------------------------ cours / arret: run inside the ROM (shell state)
# "arret" sets breakpoints (linear P addresses, 1:8000 = page 1); "cours" starts from the current PC
# (ff80 = reset vector if the state is empty), reads each instruction from the ROM (or from the P words
# set, which take priority), executes it through isa_test like "pas", and stops on: a breakpoint,
# IDLE, an address outside the ROM (RAM without a word set), a PC that does not move, or the step budget.
# All of this acts on the SHELL STATE, never on the running c54x_exe (for that: stop / go / dsp).
# Limits (the core restarts from a "registers + words" state at each step): the delay slot of a
# delayed branch (BD, CALLD, RETD...) and the counter of an RPT do not survive from one step to the next.
ARRETS=$DIR/arrets.txt; : >> "$ARRETS"
cmd_arret() { # arret [addr...] | arret -: set / list / clear the breakpoints
    if [ $# -eq 0 ]; then
        [ -s "$ARRETS" ] && { gras "--- points d'arret (adresses P)"; sed 's/^/    /' "$ARRETS"; } || echo "  aucun
         ↪  point d'arret (arret <adr> [adr...])"; return
    fi
    [ "$1" = - ] && { : > "$ARRETS"; echo "  points d'arret effaces"; return; }
    for s in "$@"; do a=$(hex "$s") || { rouge "adresse invalide : $s"; return 1; }
```

```
        a=$(printf '%x:%04x' $(( a >> 16 )) $(( a & 0xffff ))); grep -q -x -F "$a" "$ARRETS" || echo "$a" >>
         ↪  "$ARRETS"; echo "    arret a $a"; done
}
rom_insn() { # rom_insn <linear P> -> "hex words|mnemonic"; P words set take priority; 1 if outside the ROM
    a=$1; page=$(( a >> 16 )); off=$(( a & 0xffff )); : > "$DIR/i.bin"
    i=0; while [ $i -lt 3 ]; do
        lin=$(( (page << 16) | ((off + i) & 0xffff) )); ad=$(printf '%04x' $(( lin & 0xffff )))
        m=$(awk -v a="$ad" '$1=="P" && $2==a {v=$3} END {print v}' "$ETAT")
        if [ -z "$m" ]; then
            if fb=$(rom_fichier $lin P); then set -- $fb; m=$(od -An -v -tx2 -j $(( (lin - $2) * 2 )) -N 2 "$1" |
             ↪  tr -d ' ')
            elif [ $i -eq 0 ]; then return 1; else m=f495; fi
        fi
        v=$(( 0x$m )); printf "\\$(printf '%03o' $(( v & 255 )))\\$(printf '%03o' $(( v >> 8 )))" >> "$DIR/i.bin";
         ↪  i=$(( i + 1 ))
    done
    "$OBJDUMP" -D -b binary -m tms320c54x --adjust-vma="0x$(printf '%04x' $off)" "$DIR/i.bin" | sed -n 's/^
     ↪  *[0-9a-f]*:[[:space:]]*//p' \
      | awk 'NR==1 { w=$1; $1=""; sub(/^ +/,""); m=$0; next } NF>1 { exit } { w=w" "$1 } END { print w "|" m }'
}
cmd_cours() { # cours [N steps]: run from the current PC until a stop
    n=${1:-200}; case $n in ''|*[!0-9]*) echo "usage : cours [nombre de pas max]"; return 1;; esac
    pc=$(etat_reg PC); xpc=$(etat_reg XPC)
    [ -n "$pc" ] || { etat_fixe PC ff80; etat_fixe XPC 0; pc=ff80; xpc=0; echo "  (etat vide : depart au vecteur
     ↪  de reset ff80)"; }
    gras "  cours depuis $(printf '%x:%04x' $(( 0x${xpc:-0} )) $(( 0x$pc )))  (budget $n pas ; arrets : $(tr '\n'
     ↪  ' ' < "$ARRETS"))"
    : > "$DIR/cours_adr.txt"; k=0; motif=""
    while :; do
        pc=$(etat_reg PC); xpc=$(etat_reg XPC); lin=$(( (0x${xpc:-0} << 16) | 0x$pc )); cle=$(printf '%x:%04x' $((
         ↪  lin >> 16 )) $(( lin & 0xffff )))
        if [ $k -gt 0 ] && grep -q -x -F "$cle" "$ARRETS"; then motif="point d'arret $cle"; break; fi
        [ $k -lt $n ] || { motif="budget de $n pas epuise"; break; }
        r=$(rom_insn $lin) || { motif="$cle hors ROM programme (RAM sans mot pose)"; break; }
        mots=${r%%|*}; mn=${r#*|}
        printf '%s\n' "$cle" >> "$DIR/cours_adr.txt"
        case $mn in idle*) printf '    %-7s %-14s %s\n' "$cle" "$mots" "$mn"; motif="IDLE a $cle"; k=$(( k + 1 ));
         ↪  break ;; esac
        executer 1 "cours $cle $mn" "$mots" || { motif="isa_test en echec a $cle"; break; }
        k=$(( k + 1 )); pc2=$(etat_reg PC); xpc2=$(etat_reg XPC)
        printf '    %-7s %-14s %-28s -> %x:%04x  A=%s B=%s\n' "$cle" "$mots" "$mn" $(( 0x${xpc2:-0} )) $((
         ↪  0x${pc2:-0} )) "$(etat_reg A)" "$(etat_reg B)"
        [ "$pc2" = "$pc" ] && [ "${xpc2:-0}" = "${xpc:-0}" ] && { motif="PC immobile a $cle (boucle sur place /
         ↪  instruction non executee)"; break; }
    done
    gras "  arret : $motif  ($k pas)"
    gras "  adresses parcourues ($(sort -u "$DIR/cours_adr.txt" | wc -l) distinctes, dans l'ordre) :"
    awk '!(seen[$0]++)' "$DIR/cours_adr.txt" | tr '\n' ' ' | fold -s -w 96 | sed 's/^/    /'; echo
    printf '  PC=%s  A=%s  B=%s  T=%s  %s\n' "$(etat_reg PC)" "$(etat_reg A)" "$(etat_reg B)" "$(etat_reg T)"
     ↪  "$(drapeaux "$(etat_reg ST0)" "$(etat_reg ST1)")"
}

# ------------------------------------------------------------------ RUNNING DSP: stop / go / dsp on the c54x_exe
 ↪  --arm
# Here the LIVE PROCESS (the one run.sh launched) is acted upon, never the shell state:
#   stop   pauses the DSP. Binary with src/pont.c SIGUSR1 support: cooperative pause at the next frame
#          boundary (DONE returned, DSP in IDLE, the ARM waits for its next DONE); otherwise: SIGSTOP.
#          In both cases nothing is written into the API RAM or the core: the firmware only sees a
#          longer frame. The registers are read from the process memory (/proc/PID/mem, C54xState
#          offsets given by gdb on the executable), and THE ADDRESSES are shown: PC (where the DSP
#          stopped, IDLE) and the armed vectors (where it will resume: IPTR of PMST, frame IT vec 28 = bit 12).
#   go     resumes (SIGUSR2 or SIGCONT), then a snapshot 0.3 s later to see that it resumed.
#   dsp    snapshot without stopping (read while it runs: consistent to within one instruction).
PAUSE_FICHIER=${PAUSE_FICHIER:-/dev/shm/calypso_dsp_pause}
VIVANT=$DIR/vivant.txt
IRQ_NOMS="12=IT_trame_TPU(vec28) 14=DMA_INT10n(vec30)"   # c54x_irq.c:44-60; the other bits: not named here
```

```
vivant_pid() { # the c54x_exe --arm: comm == c54x_exe (readable by all, even if the process is root's) + cmdline
 ↪  --arm
    for p in $(pgrep -x c54x_exe 2>/dev/null); do
        [ "$(cat /proc/$p/comm 2>/dev/null)" = c54x_exe ] || continue
        grep -qa -- --arm /proc/$p/cmdline 2>/dev/null && { echo $p; return 0; }
    done; return 1
}
vivant_accessible() { # vivant_accessible <pid>: can this account drive this process (signal + /proc/mem)?
    kill -0 "$1" 2>/dev/null && [ -r "/proc/$1/mem" ]
}
vivant_diag() { # clear message when the bench runs under another account
    pid=$1; u=$(stat -c %U /proc/$pid 2>/dev/null); moi=$(id -un)
    rouge "le banc c54x_exe (pid $pid) tourne sous « ${u:-?} », toi tu es « $moi »."
    rouge "  -> stop/go/dsp exigent le meme compte. Lance harvard ainsi :  sudo ./harvard.sh"
    rouge "     (ou relance le banc sous ton compte). Lecture seule (d@) possible, ecriture (d@!) non."
}
vivant_coop() { grep -q -a calypso_dsp_pause /proc/$1/exe 2>/dev/null; }
vivant_decalages() { # vivant_decalages <pid> -> file "field offset" (gdb on the executable, once per pid)
    f=$DIR/decalages.$1.txt
    if ! [ -s "$f" ]; then
        command -v gdb > /dev/null || { rouge "gdb absent : impossible de situer C54xState dans le processus" >&2;
         ↪  return 1; }
        { for c in data prog pc xpc idle running a b ar t sp st0 st1 pmst imr ifr bk brc rsa rea trn insn_count
         ↪  last_exec_pc last_exec_op; do
              printf 'printf "%s %%ld\\n", (long)&((C54xState*)0)->%s\n' "$c" "$c"; done
          printf 'printf "FN %%ld\\n", (long)&g_c54x_exe_fn\n'; } > "$DIR/decal.gdb"
        gdb -batch -q -nx -ex 'set debuginfod enabled off' -x "$DIR/decal.gdb" /proc/$1/exe 2>/dev/null | grep -E
         ↪  '^[a-zA-Z_0-9]+ [0-9]+$' > "$f"
        [ "$(wc -l < "$f")" -ge 25 ] || { rm -f "$f"; rouge "gdb n'a pas rendu les decalages (executable sans -g
         ↪  ?)" >&2; return 1; }
    fi
    echo "$f"
}
vivant_lire() { # vivant_lire <pid> <label>: core registers of the process -> $VIVANT (lines "KEY val")
    pid=$1; d=$(vivant_decalages $pid) || return 1
    shm=$(awk '/calypso_api_ram/ {print $1; exit}' /proc/$pid/maps); [ -n "$shm" ] || { rouge "pas de fenetre API
     ↪  RAM dans le processus $pid"; return 1; }
    base=$(( 0x${shm%%-*} - 0x1000 - $(awk '$1=="data" {print $2}' "$d") ))      # data[0x0800] is mapped on
     ↪  /dev/shm
    exe=$(awk '$6 ~ /c54x_exe/ && $3=="00000000" {print $1; exit}' /proc/$pid/maps); exe=$(( 0x${exe%%-*} ))
    dec() { awk -v k="$1" '$1==k {print $2}' "$d"; }
    lire() { dd if=/proc/$pid/mem bs=1 skip=$1 count=$2 2>/dev/null | od -An -v -tx$2 | tr -d ' \n'; }   #
     ↪  little-endian -> hex
    echo "$pid $base $(dec prog) $(dec data) $(( 0x$(lire $(( base + $(dec pmst) )) 2) ))" >
     ↪  "$DIR/vivant_prog.txt"   # for vivant_prog
    pc=$(lire $(( base + $(dec pc) )) 4) || return 1
    [ -n "$pc" ] || { rouge "lecture de /proc/$pid/mem refusee (ptrace)"; return 1; }
    {
        echo "ETAT $2"; echo "FN $(( 0x$(lire $(( exe + $(dec FN) )) 4) ))"
        printf 'PC %04x\nXPC %x\nIDLE %d\nRUNNING %d\nINSN %d\nLAST_PC %04x\nLAST_OP %04x\n' $(( 0x$pc & 0xffff ))
         ↪  0x$(lire $(( base + $(dec xpc) )) 2) \
            0x$(lire $(( base + $(dec idle) )) 1) 0x$(lire $(( base + $(dec running) )) 1) 0x$(lire $(( base +
↪  $(dec insn_count) )) 4) \
            0x$(lire $(( base + $(dec last_exec_pc) )) 2) 0x$(lire $(( base + $(dec last_exec_op) )) 2)
        printf 'A %x\nB %x\n' $(( 0x$(lire $(( base + $(dec a) )) 8) & 0xFFFFFFFFFF )) $(( 0x$(lire $(( base +
         ↪  $(dec b) )) 8) & 0xFFFFFFFFFF ))
        for r in t sp st0 st1 pmst imr ifr bk brc rsa rea trn; do printf '%s %x\n' "$(printf '%s' $r | tr a-z A-Z)"
         ↪  0x$(lire $(( base + $(dec $r) )) 2); done
        i=0; for v in $(dd if=/proc/$pid/mem bs=1 skip=$(( base + $(dec ar) )) count=16 2>/dev/null | od -An -v
         ↪  -tx2); do printf 'AR%d %x\n' $i 0x$v; i=$(( i + 1 )); done
    } > "$VIVANT"
}
vivant_prog() { # vivant_prog <addr> <n>: n words of the live process's program memory, like prog_fetch
    set -- $(cat "$DIR/vivant_prog.txt") "$1" "$2"      # pid base prog data pmst adr n
    # c54x_mem.c prog_fetch: PMST.OVLY (bit 5) and 0x60 <= adr < 0x2800 -> data[adr] (overlaid DARAM), else
     ↪  prog[adr]
```

```
    if [ $(( $5 & 0x20 )) != 0 ] && [ $6 -ge 96 ] && [ $6 -lt 10240 ]; then o=$4; else o=$3; fi
    dd if=/proc/$1/mem bs=1 skip=$(( $2 + o + $6 * 2 )) count=$(( $7 * 2 )) 2>/dev/null | od -An -v -tx2 | tr -s '
     ↪  \n' ' '
}
montrer_vivant() { # montrer_vivant <pid>: $VIVANT on screen, with the stop / go addresses
    v() { awk -v k="$1" '$1==k {print substr($0, length($1) + 2)}' "$VIVANT"; }
    pc=$(v PC); xpc=$(v XPC); idle=$(v IDLE); pmst=$(v PMST); imr=$(v IMR); ifr=$(v IFR)
    gras "  DSP vivant (pid $1, $(vivant_coop $1 && echo 'pause cooperative SIGUSR1 disponible' || echo 'binaire
     ↪  ancien : SIGSTOP/SIGCONT')) : $(v ETAT)"
    printf '  fn=%s  PC=%x:%s  idle=%s running=%s  insn=%s  derniere instruction executee : P[%s]=%s\n' "$(v FN)"
     ↪  $(( 0x$xpc )) "$pc" "$idle" "$(v RUNNING)" "$(v INSN)" "$(v LAST_PC)" "$(v LAST_OP)"
    ( ETAT=$VIVANT; montrer_regs )
    iptr=$(( 0x$pmst >> 7 )); vbase=$(( iptr << 7 ))
    gras "  adresses de ce stop / go :"
    if [ "$idle" = 1 ]; then printf '    stop : PC=%x:%s  IDLE, le DSP attend une interruption (INTM=%d)\n' $((
     ↪  0x$xpc )) "$pc" $(( 0x${v_st1:-$(v ST1)} >> 11 & 1 ))
    else printf '    stop : PC=%x:%s  (en cours d execution, pas en IDLE)\n' $(( 0x$xpc )) "$pc"; fi
    printf '    go   : vecteurs armes (IMR=%04x IFR=%04x, IPTR=%03x -> table a %04x) :\n' $(( 0x$imr )) $(( 0x$ifr
     ↪  )) $iptr $vbase
    b=0; while [ $b -lt 16 ]; do
        if [ $(( 0x$imr >> b & 1 )) = 1 ]; then
            vec=$(( b + 16 )); adr=$(( vbase + vec * 4 )); nom=$(printf '%s\n' $IRQ_NOMS | awk -F= -v k=$b '$1==k
             ↪  {print $2}')
            cible=$(dis_mots "$(printf '%04x' $adr)" $(vivant_prog $adr 2) | head -1 | sed 's/^
             ↪  *[0-9a-f]*:[[:space:]]*//; s/[[:space:]]\+/ /g')
            printf '           bit %2d vec %2d -> %04x %-22s %s%s\n' $b $vec $adr "${nom:-?}" "${cible:+: $cible}"
             ↪  "$([ $(( 0x$ifr >> b & 1 )) = 1 ] && echo '   <- EN ATTENTE (IFR)')"
        fi; b=$(( b + 1 )); done
    lin=$(( (0x$xpc << 16) | 0x$pc ))
    if rom_fichier $lin P > /dev/null 2>&1; then gras "  ROM a PC :"; cmd_p "$(printf '%x:%04x' $(( lin >> 16 ))
     ↪  $(( lin & 0xffff )))" 4; fi
}
cmd_dsp() { # dsp: snapshot of the running DSP, without stopping it
    pid=$(vivant_pid) || { rouge "aucun c54x_exe --arm en marche (./run.sh)"; return 1; }
    vivant_accessible "$pid" || { vivant_diag "$pid"; return 1; }
    vivant_lire $pid "instantane sans arret$([ -f "$DIR/pause.$pid" ] && echo ' (en pause : go pour reprendre)')"
     ↪  || return 1; montrer_vivant $pid
}
cmd_stop() { # stop: pause the running DSP
    pid=$(vivant_pid) || { rouge "aucun c54x_exe --arm en marche (./run.sh)"; return 1; }
    vivant_accessible "$pid" || { vivant_diag "$pid"; return 1; }
    [ -f "$DIR/pause.$pid" ] && { echo "  deja en pause ($(cat "$DIR/pause.$pid")) : go pour reprendre"; cmd_dsp;
     ↪  return; }
    if vivant_coop $pid; then
        rm -f "$PAUSE_FICHIER"; kill -USR1 $pid || return 1; i=0
        while [ $i -lt 50 ] && ! grep -q '^ETAT pause' "$PAUSE_FICHIER" 2>/dev/null; do sleep 0.1; i=$(( i + 1 ));
         ↪  done
        grep -q '^ETAT pause' "$PAUSE_FICHIER" 2>/dev/null || { rouge "pas de pause apres 5 s (le DSP attend un GO
         ↪  de l'ARM ?) ; kill -USR2 $pid pour annuler"; return 1; }
        mode="SIGUSR1 (frontiere de trame, fn=$(awk '$1=="FN"{print $2}' "$PAUSE_FICHIER"))"
    else
        kill -STOP $pid || return 1; mode="SIGSTOP"
    fi
    echo "$mode" > "$DIR/pause.$pid"
    vivant_lire $pid "EN PAUSE par $mode" || return 1; montrer_vivant $pid
    echo "  (l'ARM/QEMU attend le DONE du DSP : toute la chaine est suspendue ; « go » pour reprendre)"
}
cmd_go() { # go: resume the running DSP
    pid=$(vivant_pid) || { rouge "aucun c54x_exe --arm en marche"; rm -f "$DIR"/pause.*; return 1; }
    [ -f "$DIR/pause.$pid" ] || { echo "  pas de pause en cours (stop d'abord)"; return 1; }
    mode=$(cat "$DIR/pause.$pid"); rm -f "$DIR/pause.$pid"
    case $mode in SIGUSR1*) kill -USR2 $pid; i=0; while [ $i -lt 30 ] && ! grep -q '^ETAT reprise'
     ↪  "$PAUSE_FICHIER" 2>/dev/null; do sleep 0.1; i=$(( i + 1 )); done ;;
                *) kill -CONT $pid ;; esac
    avant_insn=$(awk '$1=="INSN"{print $2}' "$VIVANT" 2>/dev/null); avant_fn=$(awk '$1=="FN"{print $2}' "$VIVANT"
     ↪  2>/dev/null)
```

```
    echo "  go : reprise ($mode)"; sleep 0.3
    vivant_lire $pid "0,3 s apres la reprise" || return 1
    v() { awk -v k="$1" '$1==k {print $2}' "$VIVANT"; }
    printf '  0,3 s apres : fn=%s (+%d trames)  PC=%x:%s idle=%s  insn=%s (+%d)\n' "$(v FN)" $(( $(v FN) -
     ↪  ${avant_fn:-0} )) $(( 0x$(v XPC) )) "$(v PC)" "$(v IDLE)" "$(v INSN)" $(( $(v INSN) - ${avant_insn:-0} ))
}

# ------------------------------------------------------------------ ARM: the layer1 firmware under gdb (telnet
 ↪  44444)
# run.sh launches QEMU with -gdb tcp::1234 and the console tools/gdb-telnet.py on port 44444: a
# gdb-multiarch attached to the firmware (layer1.highram.elf symbols), cmd.gdb panel + "demo".
# One session at a time (the gdbstub only accepts one client): if a telnet is already open,
# nothing is forced, the command to type there is shown.
GDB_TELNET=${GDB_TELNET:-44444}
telnet_occupe() { ss -Htn state established "( sport = :$GDB_TELNET )" 2>/dev/null | grep -q . ; }
cmd_gdb() { # gdb <command...>: send a command to the ARM's gdb through the telnet, show the output
    [ $# -ge 1 ] || { echo "usage : gdb <commande du panneau osmocom>   ex : gdb fn | gdb l1 | gdb help_osmo | gdb
     ↪  demo etat"; return 1; }
    ss -Hltn "( sport = :$GDB_TELNET )" 2>/dev/null | grep -q . || { rouge "rien n'ecoute sur le port $GDB_TELNET
     ↪  (run.sh avec GDB=1)"; return 1; }
    if telnet_occupe; then rouge "une session telnet est deja ouverte sur le port $GDB_TELNET (une seule a la
     ↪  fois) : tape dans ce telnet :   $*"; return 1; fi
    gras "  telnet 127.0.0.1:$GDB_TELNET  (gdb) $*"
    python3 - "$GDB_TELNET" "${GDB_ATTENTE:-90}" "$*" <<'PY'
import re, socket, sys, time
port, attente, cmd = int(sys.argv[1]), float(sys.argv[2]), sys.argv[3]
s = socket.create_connection(("127.0.0.1", port), timeout=5)
def lire(jusqua, t):
    buf = b""; fin = time.time() + t
    while time.time() < fin:
        s.settimeout(max(0.1, fin - time.time()))
        try: d = s.recv(4096)
        except socket.timeout: break
        if not d: break
        buf += d
        if jusqua in buf: break
    return buf
lire(b"(gdb) ", 20)                                  # gdb attached, target resumed (continue &)
s.sendall(b"\x03"); lire(b"(gdb) ", 5); time.sleep(0.5)   # Ctrl-C: the target stops, the prompt comes back
s.sendall(cmd.encode() + b"\necho __FIN__\\n\n")
out = lire(b"__FIN__", attente)
s.sendall(b"quit\n"); time.sleep(0.3); s.close()      # on close, the console detaches gdb: the ARM resumes
txt = re.sub(rb"\xff[\xfb-\xfe].|\xff\xfa.*?\xff\xf0", b"", out).decode("utf-8", "replace").replace("\r", "")
txt = txt.split("__FIN__")[0]
print("\n".join("    " + l for l in txt.split("\n") if not l.startswith("echo __FIN__") and l.strip() != "(gdb)"))
PY
    echo "  (session fermee : gdb detache, l'ARM tourne)"
}
# ------------------------------------------------------------------ demo: hands on the live DSP, then on the ARM
DEMO_PAUSE=${DEMO_PAUSE:-4}    # seconds between two demo steps, time to read (0 = chain)
DEMO_STOP=${DEMO_STOP:-1.5}    # duration of the real DSP pause: enough for the voice to cut out audibly, no more
demo_pause() { [ "$DEMO_PAUSE" != 0 ] && { echo "    ... (suite dans $DEMO_PAUSE s)"; sleep "$DEMO_PAUSE"; }; }
A_CD=09d2       # a_cd[0] (FIRE flag); a_cd[3]=0x09d5 start of the 23-byte L2 block (dsp_memcpy_from_api be=0)
# In the downlink CCCH/BCCH block: byte2 (type) = 0x09d6 low; cell_identity = 0x09d6 high (MSB) | 0x09d7 low (LSB).
shm_lire() { od -An -v -tx2 -j $(( (0x$1 - API_BASE) * 2 )) -N 2 "$API_SHM" 2>/dev/null | tr -d ' \n'; }  # 1 hex
 ↪  word
shm_ecrire() { local v=$(( 0x$2 & 0xffff )); printf "\\$(printf '%03o' $(( v & 255 )))\\$(printf '%03o' $(( v >> 8
 ↪  )))" \
    | dd of="$API_SHM" bs=1 seek=$(( (0x$1 - API_BASE) * 2 )) conv=notrunc status=none 2>/dev/null; }
api_mot() { cmd_d_chaud "$1" 1 2>/dev/null | sed -n 's/.*\]=\([0-9a-fA-F]\{4\}\).*/\1/p' | head -1; }  # EXACTLY 4
 ↪  hex digits, else empty
demo_si3() { # cell_id -> CELL_ID (default 777) in SI3: live loop, rewritten on every SI3 block that passes
    cible=${1:-777}
    [ -r "$API_SHM" ] || { rouge "    SI3 : pas de segment API RAM ; ./run.sh"; return 1; }
    [ -w "$API_SHM" ] || { rouge "    SI3 : segment en lecture seule ; chmod 0666 $API_SHM (ou sudo)"; return 1; }
```

```
    gras "    SI3 : relabellise le cell_id en $cible (0x$(printf %04x $cible)) a chaque bloc SI3 (type L2 0x1b)
     ↪  qui passe dans a_cd"
    fin=$(( $(date +%s%3N) + ${DEMO_HOLD_MS:-1500} )); n=0; av=""
    while [ $(date +%s%3N) -lt $fin ]; do
        w4=$(shm_lire 09d6); case $w4 in ''|*[!0-9a-fA-F]*) continue ;; esac
        [ $(( 0x$w4 & 0xff )) = $(( 0x1b )) ] || continue            # not an SI3 at this instant
        w5=$(shm_lire 09d7); case $w5 in ''|*[!0-9a-fA-F]*) continue ;; esac
        [ -z "$av" ] && { av=$(( (0x$w4 >> 8 & 0xff) << 8 | (0x$w5 & 0xff) )); echo "       1er SI3 vu : cell_id =
         ↪  $av (0x$(printf %04x $av))"; }
        shm_ecrire 09d6 $(printf '%04x' $(( (0x$w4 & 0x00ff) | ((cible >> 8 & 0xff) << 8) )))   # keeps the 0x1b
         ↪  type
       shm_ecrire 09d7 $(printf '%04x' $(( (0x$w5 & 0xff00) | (cible & 0xff) )))              # keeps the LAI byte
        n=$(( n + 1 ))
    done
    if [ "$n" = 0 ]; then echo "       (aucun SI3 vu en ${DEMO_HOLD_MS:-1500} ms ; le mobile doit lire la BCCH/CCCH
     ↪  -- reessaie quand il campe)"; return 1; fi
    vert "       $n blocs SI3 relabellises en $cible pendant ${DEMO_HOLD_MS:-1500} ms (le flag FIRE reste valide,
     ↪  pas de CRC L3)"
    echo "       -> a chaque SI3 que le firmware lit maintenant, le cell_id vaut $cible"
}
LAC_W=09d9      # a_cd[7]: the 2 LAC bytes of the SI3 (byte-swapped: a_cd[7] = LAC_LSB<<8 | LAC_MSB)
demo_lac() { # lac [value]: force the SI3 LAC to value (default 777) LIVE through the shm, without restart or
 ↪  rebuild
    cible=${1:-777}
    [ -r "$API_SHM" ] || { rouge "    LAC : pas de segment API RAM ; ./run.sh"; return 1; }
    [ -w "$API_SHM" ] || { rouge "    LAC : segment en lecture seule ; chmod 0666 $API_SHM (ou sudo)"; return 1; }
    sw=$(printf '%04x' $(( ((cible & 0xff) << 8) | ((cible >> 8) & 0xff) )))   # byte-swapped for a_cd[7]
    gras "    LAC : force a $cible (0x$(printf %04x $cible)) dans a_cd[7]=0x$sw, EN VIF, a chaque bloc SI3 (sans
     ↪  relance)"
    fin=$(( $(date +%s%3N) + ${DEMO_HOLD_MS:-3000} )); n=0; av=""
    while [ $(date +%s%3N) -lt $fin ]; do
        w4=$(shm_lire 09d6); case $w4 in ''|*[!0-9a-fA-F]*) continue ;; esac
        [ $(( 0x$w4 & 0xff )) = $(( 0x1b )) ] || continue             # SI3 only
        w7=$(shm_lire $LAC_W); case $w7 in ''|*[!0-9a-fA-F]*) continue ;; esac
        [ -z "$av" ] && { av=$(( ((0x$w7 & 0xff) << 8) | ((0x$w7 >> 8) & 0xff) )); echo "       1er SI3 vu : LAC =
         ↪  $av (0x$(printf %04x $av))"; }
        shm_ecrire $LAC_W $sw; n=$(( n + 1 ))
    done
    [ "$n" = 0 ] && { echo "       (aucun SI3 vu en ${DEMO_HOLD_MS:-3000} ms ; le mobile doit lire la BCCH)";
     ↪  return 1; }
    vert "       $n ecritures LAC=$cible pendant ${DEMO_HOLD_MS:-3000} ms, en vif (course avec le DSP : le mobile
     ↪  le voit si on gagne)"
    echo "       -> verifie : grep -ao 'CGI=[0-9-]*' /tmp/c54x-pont/mobile.log | tail  (champ LAC = 3e groupe)"
}
demo_sb() { # DEAD BEEF into a_sch (the decoded SB the ARM reads), live: rewritten in a short loop, read back
    [ -r "$API_SHM" ] || { rouge "    SB : pas de segment API RAM ($API_SHM) ; ./run.sh"; return 1; }
    [ -w "$API_SHM" ] || { rouge "    SB : segment en lecture seule pour ce compte ; chmod 0666 $API_SHM (ou
     ↪  sudo)"; return 1; }
    a=0837   # a_sch[0], page R0 (see d@ noms a_sch)
    gras "    SB : a_sch a $a (le SCH decode que le firmware lit en 0xffd00000), ecriture vive"
    echo "       avant : a_sch[0..1] = $(shm_lire $a) $(shm_lire 0838)"
    fin=$(( $(date +%s%3N) + ${DEMO_HOLD_MS:-1200} ))
    while [ $(date +%s%3N) -lt $fin ]; do shm_ecrire $a 40c3; shm_ecrire 0838 2026; done
    echo "       pendant : a_sch[0..1] = $(shm_lire $a) $(shm_lire 0838)   <- 40C3 2026 (CCC 2026) impose au SB
     ↪  que lit le firmware"
    vert "       (reecrit en boucle ${DEMO_HOLD_MS:-1200} ms ; le DSP reprend la main a la prochaine tache SB)"
}
demo_si3_preuve() { # waits for camping and shows the CGI seen by the mobile (must end with <cible>)
    cible=$1; log=${RUNDIR_OMBRE:-/tmp/c54x-pont}/mobile.log; dlog=${RUNDIR_OMBRE:-/tmp/c54x-pont}/dsp.log
    gras "    SI3 : preuve cote mobile -- cell_id force a $cible a la source (ombre descendante dans c54x_exe)"
    echo "       injections SI3 (DSP) : $(grep -ac 'ombre-dl.*SI3 cell_id' "$dlog" 2>/dev/null)   (0 si le mobile
     ↪  ne lit pas encore la BCCH)"
    i=0; cgi=""
    while [ $i -lt ${DEMO_CGI_ESSAIS:-40} ]; do
        cgi=$(grep -ao 'CGI=[0-9-]*' "$log" 2>/dev/null | tail -1)
        case $cgi in *-$cible) break ;; esac
```

```
        sleep 1; i=$(( i + 1 ))
    done
    case $cgi in
        *-$cible) vert "       mobile.log : $cgi   <- le telephone a decode le cell_id $cible (ombre reussie)" ;;
        '') echo "       (mobile pas encore campe apres ${DEMO_CGI_ESSAIS:-40} s ; grep CGI $log plus tard)" ;;
        *) echo "       mobile.log : $cgi (pas encore $cible ; injections=$(grep -ac 'ombre-dl.*SI3 cell_id'
         ↪  "$dlog" 2>/dev/null), laisse camper puis grep CGI $log)" ;;
    esac
}
demo_dsp() { # live DSP, WITHOUT restart: forces the SI3 LAC live (shm), CGI proof, stop/go
    pid=$(vivant_pid) || { rouge "demo dsp : aucun c54x_exe --arm en marche (./run.sh)"; return 1; }
    gras "===== DEMO DSP (c54x_exe --arm pid $pid, le vrai coeur C54x qui sert l'ARM) ====="
    log=${RUNDIR_OMBRE:-/tmp/c54x-pont}/mobile.log
    cible=${DEMO_CELL_ID:-777}
    gras "--- 1. LAC du SI3 force a $cible EN VIF (shm, SANS relance ; le banc reste campe)"
    av=$(grep -ao 'CGI=[0-9-]*' "$log" 2>/dev/null | tail -1); echo "      CGI avant : ${av:-<pas encore de camp>}"
    DEMO_HOLD_MS=${DEMO_LAC_MS:-5000} demo_lac "$cible"
    demo_pause
    gras "--- 2. preuve cote mobile : le LAC (3e groupe du CGI) doit passer a $cible"
    i=0; cgi=""
    while [ $i -lt ${DEMO_CGI_ESSAIS:-20} ]; do
        cgi=$(grep -ao 'CGI=[0-9-]*' "$log" 2>/dev/null | tail -1)
        case $cgi in CGI=*-*-$cible-*) break ;; esac
        DEMO_HOLD_MS=1500 demo_lac "$cible" >/dev/null 2>&1   # hammer again while the mobile re-reads the BCCH
        sleep 1; i=$(( i + 1 ))
    done
    case $cgi in
        CGI=*-*-$cible-*) vert "       mobile.log : $cgi   <- LAC = $cible, le telephone l'a decode (en vif, sans
         ↪  relance)" ;;
        '') echo "       (pas de CGI : le mobile n'est pas campe ; il faut qu'il lise la BCCH)" ;;
        *) echo "       mobile.log : $cgi (pas encore $cible au 3e groupe ; course avec le DSP, relance 'lac
         ↪  $cible' ou laisse camper)" ;;
    esac
    demo_pause
    if vivant_accessible "$pid"; then
        gras "--- 3. stop : pause du DSP, registres et LES ADRESSES"; cmd_stop || return 1; demo_pause
        gras "--- 4. go : reprise (fn, instructions)"; cmd_go
        gras "===== FIN DEMO DSP : LAC du SI3 force a $cible en vif, DSP relance, rien de casse, pas de relance du
         ↪  banc ====="
    else
        echo; vivant_diag "$pid"
        gras "===== FIN DEMO DSP : LAC force en vif. stop/go : harvard sous le compte du banc ====="
    fi
}
cmd_demo() { # demo [tout|arm [etat|trame|hello|go]|dsp]: first the ARM under gdb (hello world from ram), then the
 ↪  live DSP
    quoi=${1:-tout}; [ $# -gt 0 ] && shift
    case $quoi in
        dsp) demo_dsp ;;
        arm) gras "===== DEMO ARM (firmware layer1 dans QEMU, gdb par le telnet $GDB_TELNET) ====="; cmd_gdb demo
         ↪  "$@" ;;
        tout) gras "===== DEMO ARM D'ABORD (firmware layer1 dans QEMU, gdb par le telnet $GDB_TELNET : hello world
         ↪  from ram) ====="
              cmd_gdb demo "$@"; demo_pause; echo
              demo_dsp
              echo; gras "===== FIN DE LA DEMO : ARM en marche, DSP relance, rien de casse ====="; demo_pause ;;
        *) echo "usage : demo [tout | dsp | arm [etat|trame|hello|go]]"; return 1 ;;
    esac
}

# ------------------------------------------------------------------ live API RAM (the running DSP, --arm)
API_SHM=${API_SHM:-/dev/shm/calypso_api_ram}   # = data[0x0800..0x27ff] of the c54x_exe --arm, shared with the ARM
 ↪  (MAP_SHARED)
API_BASE=$(( 0x0800 )); API_MOTS=8192
CARTE=${CARTE:-$HERE/docs/carte-memoire.md}    # "API RAM — tous les champs" table (generated from the ELF:
 ↪  scalars only)
```

```
# Double pages (osmocom-bb include/calypso/dsp_api.h:18-23): DSP word = 0x0800 + (ARM address - 0xFFD00000)/2
#   W0 0x0800..0x0813  W1 0x0814..0x0827  (T_DB_MCU_TO_DSP, 17 useful words + 3 padding, dsp_api.h:32-80)
#   R0 0x0828..0x083b  R1 0x083c..0x084f  (T_DB_DSP_TO_MCU, 20 words, dsp_api.h:82-107)   NDB 0x08d4.. (268)
 ↪  PARAM 0x0c31.. (57)
api_page() { # api_page <mot DSP> -> W0|W1|R0|R1|NDB|PARAM|API
    a=$1; if [ $a -lt $(( 0x0814 )) ]; then echo W0; elif [ $a -lt $(( 0x0828 )) ]; then echo W1
    elif [ $a -lt $(( 0x083c )) ]; then echo R0; elif [ $a -lt $(( 0x0850 )) ]; then echo R1
    elif [ $a -ge $(( 0x08d4 )) ] && [ $a -lt $(( 0x08d4 + 268 )) ]; then echo NDB
    elif [ $a -ge $(( 0x0c31 )) ] && [ $a -lt $(( 0x0c31 + 57 )) ]; then echo PARAM; else echo API; fi
}
api_table() { # "name 0xADDR page": the generated map + the arrays the ELF does not name (dsp_api.h, API 36xx)
    grep '^| [^|]* | [a-z][a-z0-9_]* | 0x[0-9a-fA-F]* | 0xFFD[0-9A-F]* | 0x[0-9a-f]*' "$CARTE" 2>/dev/null | awk
     ↪  -F'|' '{gsub(/ /,"",$2); gsub(/ /,"",$3); gsub(/ /,"",$6); print $3, $6, $2}'
    for pp in 0x0800:W0 0x0814:W1; do pb=$(( ${pp%%:*} )); ppg=${pp#*:}                                  #
     ↪  dsp_api.h:67-70, padding 17..19
        printf 'a_a5fn[0] 0x%04x %s\na_a5fn[1] 0x%04x %s\n' $(( pb + 12 )) "$ppg" $(( pb + 13 )) "$ppg"
        printf '(bourrage) 0x%04x %s\n(bourrage) 0x%04x %s\n(bourrage) 0x%04x %s\n' $(( pb + 17 )) "$ppg" $(( pb +
         ↪  18 )) "$ppg" $(( pb + 19 )) "$ppg"; done
    for pp in 0x0828:R0 0x083c:R1; do pb=$(( ${pp%%:*} )); ppg=${pp#*:}; pi=0                            #
     ↪  dsp_api.h:97-99 (DSP 33..36)
        for nm in 'a_serv_demod[0]=D_TOA' 'a_serv_demod[1]=D_PM' 'a_serv_demod[2]=D_ANGLE' 'a_serv_demod[3]=D_SNR'
         ↪  'a_pm[0]' 'a_pm[1]' 'a_pm[2]' 'a_sch[0]' 'a_sch[1]' 'a_sch[2]' 'a_sch[3]' 'a_sch[4]'; do
            printf '%s 0x%04x %s\n' "$nm" $(( pb + 8 + pi )) "$ppg"; pi=$(( pi + 1 )); done; done
}
api_adr() { # api_adr <name|address> -> integer (DSP word), 1 if unknown / outside the window
    case $1 in
        [a-z]*) v=$(api_table | awk -v n="$1" '$1==n || (n !~ /\[/ && $1 ~ ("^" n "(\\[0\\])?(=|$)")) {print $2;
         ↪  exit}'); [ -n "$v" ] || { rouge "champ inconnu ($CARTE + dsp_api.h) : $1   -> d@ noms <motif>" >&2;
         ↪  return 1; }; v=$(( v )) ;;
        *) v=$(hex "$1") || { rouge "adresse invalide : $1" >&2; return 1; } ;;
    esac
    [ $v -ge $API_BASE ] && [ $v -lt $(( API_BASE + API_MOTS )) ] || { rouge "$(printf '%04x' $v) : hors fenetre
     ↪  API (0800-27ff)" >&2; return 1; }
    printf '%d' $v
}
api_nom() { api_table | awk -v a="$(printf '0x%04x' $1)" '$2==a {print $3 " " $1; exit}'; }
api_vivant() { [ -r "$API_SHM" ] || { rouge "pas de segment $API_SHM (lancer ./run.sh ou ./c54x_exe --arm)";
 ↪  return 1; }
    pgrep -f 'c54x_exe.*--arm' > /dev/null || echo "  (aucun c54x_exe --arm en marche : contenu fige du dernier
     ↪  run)"; }
# Field decoding (sources: dsp_api.h:46-79 for the bits, l1_environment.h:36-80 for the tasks,
# :339-353 and :370-386 for d_ctrl_tch, :355-367 for d_ctrl_abb / d_ctrl_system). Nothing is guessed.
TACHES="0=NO 1=IDLE1 5=FB 6=SB 8=TCH_FB 9=TCH_SB 10=RACH 11=AUL 12=DUL 13=TCHT 14=TCHA 15=TCH_DTX_UL 17=NBN 18=EBN
 ↪  19=NBS 20=EBS 21=NP 22=EP 24=ALLC 25=CB 26=DDL 27=ADL 28=TCHD 30=PNP 31=PEP 32=PALLC 33=CHECKSUM/PBS
 ↪  35=TST_NDB 36=TST_DB 37=INIT_VEGA 38=DSP_LOOP_C"
MODES="0=SIG_ONLY 1=TCH_FS 2=TCH_HS 3=TCH_96 4=TCH_48F 5=TCH_48H 6=TCH_24F 7=TCH_24H 8=TCH_EFR 9=TCH_144"
TYPES="0=INVALID 1=TCH_F 2=TCH_H 3=SDCCH_4 4=SDCCH_8"
decoder() { # decoder <name> <hex16> -> text (empty if nothing to say)
    n=${1%%=*}; v=$(( 0x$2 ))
    case $n in
        d_task_d|d_task_u|d_task_md|d_task_ra) t=$(printf '%s\n' $TACHES | awk -F= -v k=$v '$1==k {print $2}');
         ↪  printf 'tache %d%s' $v "${t:+ = $t}" ;;
        d_fn) printf 'fn_report=%d (FN dans la periode de rapport)  fn_sid=%d (FN%%104)' $(( v & 0xff )) $(( v >>
         ↪  8 )) ;;
        d_ctrl_tch) m=$(printf '%s\n' $MODES | awk -F= -v k=$(( v & 0xf )) '$1==k {print $2}'); ty=$(printf '%s\n'
         ↪  $TYPES | awk -F= -v k=$(( v >> 4 & 3 )) '$1==k {print $2}')
            printf 'chan_mode=%d%s chan_type=%d%s' $(( v & 0xf )) "${m:+($m)}" $(( v >> 4 & 3 )) "${ty:+($ty)}"
            [ $(( v >> 6 & 1 )) = 1 ] && printf ' RESET_SACCH'; [ $(( v >> 7 & 1 )) = 1 ] && printf ' VOCODER_ON'
            [ $(( v >> 8 & 1 )) = 1 ] && printf ' SYNC_TCH_UL'; [ $(( v >> 9 & 1 )) = 1 ] && printf ' SYNC_TCH_DL'
           [ $(( v >> 10 & 1 )) = 1 ] && printf ' STOP_TCH_UL'; [ $(( v >> 11 & 1 )) = 1 ] && printf ' STOP_TCH_DL'
            [ $(( v >> 12 & 3 )) != 0 ] && printf ' tch_loop=%d' $(( v >> 12 & 3 )); [ $(( v >> 15 & 1 )) = 1 ] &&
             ↪  printf ' SUBCHANNEL' ;;
        d_ctrl_abb) printf 'bits:'; [ $(( v & 1 )) = 1 ] && printf ' b_ramp(a_ramp dans la NDB)'; [ $(( v >> 3 & 1
         ↪  )) = 1 ] && printf ' b_apcdel(delais dans la NDB)'
            [ $(( v >> 4 & 1 )) = 1 ] && printf ' b_afc(d_afc valide)'; [ $(( v & ~0x19 )) != 0 ] && printf '
             ↪  +inconnus(%04x)' $(( v & ~0x19 & 0xffff )) ;;
```

```
        'a_a5fn[0]') printf 'T2=%d T3=%d (chiffrement A5)' $(( v & 0x1f )) $(( v >> 5 & 0x3f )) ;;
        'a_a5fn[1]') printf 'T1=%d (chiffrement A5)' $(( v & 0xfff )) ;;
        d_ctrl_system) printf 'b_tsq=%d (sequence d apprentissage)' $(( v & 7 )); [ $(( v >> 3 & 1 )) = 1 ] &&
         ↪  printf ' BCCH_FREQ_IND'; [ $(( v >> 15 & 1 )) = 1 ] && printf ' TASK_ABORT' ;;
        d_afc) [ $v -ge 32768 ] && v=$(( v - 65536 )); printf 'signe %d' $v ;;
        'a_serv_demod[0]') [ $v -ge 32768 ] && v=$(( v - 65536 )); printf 'TOA %d' $v ;;
        'a_serv_demod[2]') [ $v -ge 32768 ] && v=$(( v - 65536 )); printf 'angle %d (signe)' $v ;;
    esac
}
cmd_d_chaud() { # d@ <addr|name> [n]: read the API RAM of the running DSP, labelled by absolute address and decoded
    [ $# -ge 1 ] || { echo "usage : d@ <adresse|nom> [n]   ex : d@ 0800 20 | d@ d_task_d | d@ W1 | d@ noms d_task";
     ↪  return 1; }
    case $1 in
        noms) api_table | grep "${2:-.}" | awk '{printf "    %-5s %s  %s\n", $3, $2, $1}'; return ;;
        W0) set -- 0800 17 ;; W1) set -- 0814 17 ;; R0) set -- 0828 20 ;; R1) set -- 083c 20 ;;
    esac
    api_vivant || return 1; a=$(api_adr "$1") || return 1; n=${2:-8}
    api_table > "$DIR/table.txt"
    od -An -v -tx2 -w2 -j $(( (a - API_BASE) * 2 )) -N $(( n * 2 )) "$API_SHM" > "$DIR/mots.txt"
    while read -r mot; do
        [ -n "$mot" ] || continue
        set -- $(awk -v ad="$(printf '0x%04x' $a)" '$2==ad {print $3, $1; exit}' "$DIR/table.txt")
        pg=${1:-$(api_page $a)}; nom=${2:-}
        printf '    %-5s [%04x]=%s  %-18s %s\n' "$pg" $a "$mot" "$nom" "$([ -n "$nom" ] && decoder "$nom" "$mot")"
        a=$(( a + 1 ))
    done < "$DIR/mots.txt"
}
cmd_d_chaud_ecrire() { # d@! <addr|name> <words...>: write into the API RAM of the running DSP (seen by the DSP
 ↪  and the ARM)
    [ $# -ge 2 ] || { echo "usage : d@! <adresse|nom> <mot> [mot...]"; return 1; }
    api_vivant || return 1; a=$(api_adr "$1") || return 1; shift
    [ -w "$API_SHM" ] || { rouge "segment en lecture seule pour cet utilisateur"; return 1; }
    for m in "$@"; do v=$(hex "$m") || { rouge "mot invalide : $m"; return 1; }; v=$(( v & 0xffff ))
        [ $a -lt $(( API_BASE + API_MOTS )) ] || { rouge "debordement de la fenetre"; return 1; }
        printf "\\$(printf '%03o' $(( v & 255 )))\\$(printf '%03o' $(( v >> 8 )))" \
            | dd of="$API_SHM" bs=1 seek=$(( (a - API_BASE) * 2 )) conv=notrunc status=none || { rouge "ecriture
             ↪  refusee"; return 1; }
        printf '    [%04x] <- %04x  %s\n' $a $v "$(api_nom $a)"; a=$(( a + 1 )); done
    echo "  (ecrit dans le DSP en marche ; l'ARM voit le meme segment)"
}
# ------------------------------------------------------------------ "run": the bench tools as they are
lancer() { # lancer <cmd...>: run, log, show the tail
    gras "  \$ $*"; "$@" > "$DIR/dernier_run.log" 2>&1; rc=$?
    n=$(wc -l < "$DIR/dernier_run.log")
    [ $n -gt 60 ] && echo "  ... ($n lignes, tout dans $DIR/dernier_run.log)"
    tail -n 60 "$DIR/dernier_run.log"; echo "  (code de retour $rc)"; return $rc
}
cmd_rejeu() { # rejeu [frames] [-v..]: c54x_exe --rejouer, deterministic FB/SB acquisition
    t=${1:-200}; shift 2>/dev/null; (cd "$HERE" && lancer ./c54x_exe --rejouer --trames "$t" --verbeux "$@"); }
cmd_banc() { # banc [test|--all|--list] [options dsp_tester]
    [ $# -ge 1 ] || set -- --list; (cd "$HERE" && lancer ./dsp_tester "$@"); }
cmd_isa() { # isa [-k WORD]: the SPRU172C examples against the core
    (cd "$HERE" && lancer ./isa_test tools/isa_tests.txt "$@"); }
RUNDIR_OMBRE=${RUNDIR_OMBRE:-/tmp/c54x-pont}
cmd_ombre() { # ombre [cell_id|off]: restart the WHOLE chain (run_si.sh: BTS + scaffolding + bridge) with the SI3
 ↪  shadow
    n=${1:-${DEMO_CELL_ID:-777}}
    case $n in off|OFF) sv=""; lib="desactivee" ;; *[!0-9]*) rouge "usage : ombre [cell_id|off]  (defaut 777)";
     ↪  return 1 ;; *) sv="$n"; lib="LAC SI3 = $n" ;; esac
    # run_si.sh brings the mobile to decode the BCCH (native correlator not locked -> CAN_TOA/CAN_SB/AFC/LOCKSTEP
     ↪  scaffolding).
    # bare run.sh (PONT=0) delivers NO burst from the BTS: no camping, no SI3, the shadow never triggers.
    lanceur="$HERE/run_si.sh"; mode_full=1
    [ -x "$lanceur" ] || { lanceur="$HERE/run.sh"; mode_full=0; rouge "  run_si.sh absent : run.sh nu (sans pont,
     ↪  le mobile ne campera pas)"; }
```

```
    gras "  ombre : relance de la chaine complete ($(basename "$lanceur")) avec CALYPSO_SHADOW_LAC=${sv:-<off>}
     ↪  ($lib)"
    logdir=${RUNDIR_OMBRE:-/tmp/c54x-pont}
    : > "$DIR/run_ombre.log"
    # avoid stacking: kill any ghost launcher (run_si.sh/run.sh) BEFORE restarting, otherwise
    # several chains fight over the DSP socket/the shm/the BTS and the mobile never acquires
    # (FBSB result=255). ONLY the launchers are killed, not this shell.
    for q in $(pgrep -f 'run_si\.sh|c54x_exe/run\.sh' 2>/dev/null); do [ "$q" = "$$" ] || kill "$q" 2>/dev/null;
     ↪  done
    ( cd "$HERE" && ./run.sh --stop ) >/dev/null 2>&1; sleep 1
    if [ "$mode_full" = 1 ]; then
        ( cd "$HERE" && CALYPSO_SHADOW_LAC="$sv" CALYPSO_SHADOW_SB="${OMBRE_SB:-}" setsid "$lanceur" --secondes
         ↪  "${OMBRE_SECS:-300}" ) > "$DIR/run_ombre.log" 2>&1 &
    else
        ( cd "$HERE" && ./run.sh --stop ) >/dev/null 2>&1
        ( cd "$HERE" && env MODE=dsp PONT=1 IQ=none CALYPSO_BSP_DIRECT_FEED=1 CALYPSO_PONT_LOCKSTEP=1 \
            CALYPSO_SHADOW_LAC="$sv" CALYPSO_SHADOW_SB="${OMBRE_SB:-}" setsid ./run.sh ) > "$DIR/run_ombre.log"
 ↪  2>&1 &
    fi
    echo "  banc en relance (BTS + echafaudage + pont ; journal $DIR/run_ombre.log)..."
    i=0; annonce=""
    while [ $i -lt ${OMBRE_ATTENTE:-50} ]; do
        [ -z "$sv" ] && { [ -S /tmp/calypso_dsp.sock ] && { echo "  banc relance (ombre off)"; return 0; }; sleep
         ↪  1; i=$((i+1)); continue; }
        annonce=$(grep -a "ombre-dl.*LAC SI3 force a $sv" "$logdir/dsp.log" 2>/dev/null | tail -1)
        [ -n "$annonce" ] && break
        sleep 1; i=$((i+1))
    done
    [ -n "$annonce" ] && vert "  ombre active : ${annonce#  }" || { echo "  (ombre pas encore annoncee apres
     ↪  ${OMBRE_ATTENTE:-50}s ; voir $DIR/run_ombre.log et $logdir/dsp.log)"; return 1; }
}
cmd_statut() { (cd "$HERE" && lancer ./run.sh --status); }

# ------------------------------------------------------------------ REPL
aide() {
    cat <<'FIN'
  cote P (programme)                          cote D (donnees)
    pas <insn|mots>   executer, garder l'etat     reg [REG val]   voir / fixer un registre
    essai <insn>      executer sans garder        d <adr> [n]     lire la memoire donnees
    p <adr> [n]       desassembler la ROM         d! <adr> <mots> poser des mots en D
    p! <adr> <insn>   poser des mots en P         drom <adr> [n]  lire la ROM donnees
    asm <insn>        assembler seulement         etat | raz      tout afficher | etat a zero
    dis <mots hex>    desassembler des mots
    cours [N]         courir dans la ROM jusqu'a arret / IDLE / N pas (etat du shell)
    arret [adr..|-]   points d'arret de « cours »
  DSP EN MARCHE (c54x_exe --arm)              ARM (firmware layer1 sous gdb, telnet 44444)
    stop              pause du DSP vivant + adresses   demo [tout|arm|dsp]  la main sur l'ARM (gdb) puis le DSP
    go                reprise                          gdb <cmd>   une commande du panneau (fn, l1, help_osmo)
    dsp              instantane sans arreter                      (demo arm etat|trame|hello|go : etape par etape)
    sb                DEAD BEEF dans a_sch (SB)        si3 [id]    relabellise le cell_id de SI3 (defaut 777)
    lac [id]          force le LAC du SI3 EN VIF (shm, sans relance ; course avec le DSP)
    ombre [id|off]    relance le banc, LAC du SI3 force a la SOURCE (deterministe ; mobile.log CGI 3e groupe)
    d@ <adr|nom|W0..R1> [n]  API RAM decodee     d@! <adr|nom> <mots>  y ecrire     d@ noms [motif]
  run                                         divers
    rejeu [trames]    c54x_exe --rejouer          journal [n]     derniers pas
    banc [test|--all] dsp_tester                  log [n]         avertissements du coeur
    isa [-k MOT]      isa_test SPRU172C           sauve|charge <f> etat sur disque
    statut            run.sh --status             menu | aide | q
  adresses : 1234, 0x1234, 1:8000 (page 1).  insn : mnemonique TI (LD #0x1234, A) ou mots hex (f020 1234).
FIN
}
dispatch() { # dispatch <command line>
    c=$1; shift
    case $c in
        pas) cmd_pas "$@" ;;      essai) cmd_essai "$@" ;;
        p) cmd_p "$@" ;;          'p!') cmd_p_pose "$@" ;;
```

```
        d) cmd_d "$@" ;;          'd!') cmd_d_pose "$@" ;;
        'd@') cmd_d_chaud "$@" ;;  'd@!') cmd_d_chaud_ecrire "$@" ;;
        drom) cmd_drom "$@" ;;    asm) cmd_asm "$@" ;;      dis) cmd_dis "$@" ;;
        reg) cmd_reg "$@" ;;      etat) cmd_etat ;;         raz) cmd_raz ;;
        cours) cmd_cours "$@" ;;  arret) cmd_arret "$@" ;;
        stop) cmd_stop ;;         go) cmd_go ;;             dsp) cmd_dsp ;;
        demo) cmd_demo "$@" ;;    gdb) cmd_gdb "$@" ;;
        sb|demo_sb) demo_sb ;;   si3|demo_si3) demo_si3 "$@" ;;   lac|demo_lac) demo_lac "$@" ;;
        demo_dsp) demo_dsp ;;    demo_arm) cmd_gdb demo "$@" ;;
        journal) cmd_journal "$@" ;; log) cmd_log "$@" ;;
        sauve) cmd_sauve "$@" ;;  charge) cmd_charge "$@" ;;
        rejeu) cmd_rejeu "$@" ;;  banc) cmd_banc "$@" ;;    isa) cmd_isa "$@" ;;  statut) cmd_statut ;;
        ombre) cmd_ombre "$@" ;;
        aide|'?'|help) aide ;;
        *) rouge "commande inconnue : $c (aide)"; return 1 ;;
    esac
}
repl() {
    verifier_outils; gras "shell Harvard c54x - « aide » pour les commandes, « menu » pour le menu, « q » pour
     ↪  sortir"
    while :; do
        pc=$(etat_reg PC); if [ -t 1 ]; then printf '\033[1mP:%s D>\033[0m ' "${pc:-9000}"; else printf 'P:%s D> '
         ↪  "${pc:-9000}"; fi
        read -r ligne || { echo; return 0; }
        set -- $ligne; [ $# -eq 0 ] && continue
        case $1 in q|quit|exit) return 0 ;; menu) return 2 ;; esac
        dispatch "$@"
    done
}

# ------------------------------------------------------------------ interactive menu
pause() { printf '\n  [Entree pour continuer] '; read -r _; }
demander() { # demander <prompt> [default] -> REP
    printf '  %s' "$1"; [ -n "${2:-}" ] && printf ' [%s]' "$2"; printf ' : '
    read -r REP; [ -z "$REP" ] && REP=${2:-}
}
menu() {
    verifier_outils
    while :; do
        clear 2>/dev/null
        pc=$(etat_reg PC)
        gras "================  SHELL HARVARD - DSP TMS320C54x (Calypso)  ================"
        printf '  PC=%s  A=%s  B=%s  SP=%s   %s\n' "${pc:-9000 (init)}" "$(etat_reg A)" "$(etat_reg B)"
         ↪  "$(etat_reg SP)" "$(drapeaux "$(etat_reg ST0)" "$(etat_reg ST1)")"
        cat <<'FIN'

  ESPACE P (programme)                    ESPACE D (donnees)
   1) executer une instruction (un pas)     5) voir les registres et les mots poses
   2) essayer une instruction (sans garder)  6) fixer un registre
   3) desassembler la ROM programme          7) lire la memoire donnees du coeur
   4) poser des mots en P (sous-routine)     8) poser des mots en D
                                             9) lire la ROM donnees (DROM / PDROM)
  COURIR DANS LA ROM (etat du shell)        DSP EN MARCHE (c54x_exe --arm, /dev/shm)
  10) cours : jusqu'a arret / IDLE / N pas  12) stop : pause du DSP vivant, registres, adresses
  11) arret : points d'arret                13) go : reprise          14) dsp : instantane
                                            15) lire l'API RAM a chaud (adresse ou nom)
                                            16) ecrire dans l'API RAM a chaud
  ARM (firmware layer1 sous gdb, telnet 44444)
  17) demo : la main sur l'ARM puis le DSP  18) gdb : une commande du panneau osmocom
  RUN (les outils du banc, tels quels)      DIVERS
  19) c54x_exe --rejouer (acquisition FB/SB) 23) journal des pas / avertissements du coeur
  20) dsp_tester (liste, un test, --all)     24) etat a zero / sauver / charger
  21) isa_test (exemples SPRU172C, filtre)   r) invite de commandes (REPL)
  22) run.sh --status                        q) quitter
FIN
        demander "choix"
```

```
        case $REP in
            1) demander "instruction (mnemonique TI ou mots hex)" "NOP"; cmd_pas $REP; pause ;;
            2) demander "instruction" "NOP"; cmd_essai $REP; pause ;;
            3) demander "adresse (7000.., e000.., 1:8000)" "7000"; a=$REP; demander "nombre de mots" 24; cmd_p
             ↪  "$a" "$REP"; pause ;;
            4) demander "adresse P" "9100"; a=$REP; demander "instruction ou mots"; cmd_p_pose "$a" $REP; pause ;;
            5) cmd_etat; pause ;;
            6) demander "registre" "AR3"; r=$REP; demander "valeur hex" "0"; cmd_reg "$r" "$REP"; pause ;;
            7) demander "adresse D" "0100"; a=$REP; demander "nombre de mots (<=24)" 8; cmd_d "$a" "$REP"; pause ;;
            8) demander "adresse D" "0100"; a=$REP; demander "mots hex (separes par des espaces)"; cmd_d_pose "$a"
             ↪  $REP; pause ;;
            9) demander "adresse (9000.., e000..)" "9000"; a=$REP; demander "nombre de mots" 32; cmd_drom "$a"
             ↪  "$REP"; pause ;;
            10) cmd_arret; demander "nombre de pas max" 200; cmd_cours "$REP"; pause ;;
            11) cmd_arret; demander "arret <adr..> | - (effacer) | vide" ""; [ -n "$REP" ] && cmd_arret $REP;
             ↪  pause ;;
            12) cmd_stop; pause ;;
            13) cmd_go; pause ;;
            14) cmd_dsp; pause ;;
            15) demander "adresse, nom, page W0|W1|R0|R1, ou noms <motif>" "W0"; a=$REP; demander "nombre de mots"
             ↪  8; case $a in noms*) cmd_d_chaud $a ;; *) cmd_d_chaud "$a" "$REP" ;; esac; pause ;;
            16) demander "adresse ou nom" "d_task_d"; a=$REP; demander "mots hex (separes par des espaces)";
             ↪  cmd_d_chaud_ecrire "$a" $REP; pause ;;
            17) demander "demo tout | dsp | arm [etat|trame|hello|go]" "tout"; cmd_demo $REP; pause ;;
            18) demander "commande gdb (fn, l1, dsp, help_osmo...)" "help_osmo"; cmd_gdb $REP; pause ;;
            19) demander "trames" 200; t=$REP; demander "verbosite (-v .. -vvvvvv, vide = aucune)" ""; cmd_rejeu
             ↪  "$t" $REP; pause ;;
            20) cmd_banc --list; demander "test (nom, --all, ou vide)" ""; [ -n "$REP" ] && { demander "options
             ↪  (--trace N, -v, --insns N...)" ""; o=$REP; cmd_banc $REP $o; }; pause ;;
            21) demander "filtre -k (vide = tout)" ""; [ -n "$REP" ] && cmd_isa -k "$REP" || cmd_isa; pause ;;
            22) cmd_statut; pause ;;
            23) cmd_journal 20; cmd_log 10; pause ;;
            24) demander "raz | sauve <fichier> | charge <fichier>" "raz"; dispatch $REP; pause ;;
            r|R) repl; [ $? = 2 ] || return 0 ;;
            q|Q) return 0 ;;
            *) ;;
        esac
    done
}

# ------------------------------------------------------------------ entry point
case ${1:-menu} in
    menu) menu ;;
    repl) repl ;;
    aide|-h|--help) sed -n '2,/^$/p' "$0" | sed 's/^# \{0,1\}//'; aide ;;
    *) dispatch "$@" ;;
esac
```

3.5 /opt/GSM/c54x_exe/install.sh

26527 octets, 489 lignes → 489 lignes

```
#!/bin/bash
# ============================================================================
#  install.sh - builds c54x_exe (the Calypso DSP outside QEMU) and installs its ROM
# ============================================================================
#
#  Standalone: it needs the qosmo SOURCES (cloned, not built: c54x_exe compiles
#  hw/arm/calypso/l1-dsp/ of that tree) and libosmocore with libosmocoding,
#  visible to pkg-config - which anyone with an Osmocom stack already has.
#  This is also THE script called by the osmo-operator Dockerfile (stage l1),
#  its Dockerfile.run, start-docker.sh, the ISO and the native installation
#  (install_modules/45-calypso.sh): a single list of commands.
#
```

```
#      ./install.sh                everything: build, rom, verify
#      ./install.sh --check        prerequisites and state, MODIFIES NOTHING
#      ./install.sh --list         the steps and the values retained
#      ./install.sh --print-deps   the apt packages, on one line
#      sudo ./install.sh --deps    installs the missing apt packages, nothing else
#      sudo ./install.sh --with-deps   the missing packages, then everything
#      ./install.sh --only rom     one or more steps (comma-separated list)
#      ./install.sh --skip verify  all but those
#      ./install.sh --reinstall    replays even what is already done
#      ./install.sh -v             command output on screen (otherwise: log)
#
#  Options, and the variable that does the same (the option wins):
#      --qosmo DIR      QOSMO        $GSM_ROOT/qosmo     the compiled sources
#      --rom-dir DIR    ROM_DIR      $GSM_ROOT           where to put the ROM (repeatable); without
#                                                        the option: ROM_DIR AND rom/ of this repo
#                                                        (Dockerfile:736). c54x_exe reads it from
#                                                        --rom-dir, /opt/GSM by default (src/main.c:79)
#      --portable       PORTABLE=1   0                   CFLAGS of Dockerfile:735, without -march=native:
#                                                        for a binary that will run on another CPU
#      --dest DIR                                        copies this repo into DIR (without .git or binaries)
#                                                        and builds there: the current tree is untouched
#                       GSM_ROOT     /opt/GSM     ROM_VERSION 3606     LOG_DIR /tmp/c54x_exe-install
#                       FREECALYPSO_URL: ROM mirror (rom/fetch-rom.sh)
#
#  Correspondence with osmo-operator/Dockerfile: each step cites its lines,
#  those of the Dockerfile at commit 4266c8b, when it still carried these
#  commands (stage l1, 723-741; packages 156-205). It now calls this script.
# ------------------------------------------------------------------------------
set -uo pipefail
ICI="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
# ==============================================================================
#  THE CONTRACT - that of osmo-operator/install_modules/_lib/inst.sh
# ==============================================================================
#  IDENTICAL copy in qosmo/install.sh, c54x_exe/install.sh and
#  grgsm_exe/install.sh: each repo must be able to install on its own, without
#  osmo-operator. A fix here is carried over to the other two.
#
#  Four functions per step, only `run` is mandatory:
#     inst_<step>_check     prerequisites, READ ONLY        0 ok · 1 missing · 4 not applicable
#     inst_<step>_done      already done?                   0 yes · 1 no
#     inst_<step>_run       does the work                   0 ok · 1 failure · 3 already done
#     inst_<step>_verify    check AFTER the fact            0 ok · 1 failure
# ------------------------------------------------------------------------------
readonly INST_RC_OK=0 INST_RC_FAIL=1 INST_RC_DONE=3 INST_RC_NA=4
declare -a INST_ORDER=()
declare -A INST_DESC=()
INST_REGISTER() { INST_ORDER+=("$1"); INST_DESC[$1]="$2"; }

_INST_REASON=""
_INST_HINT=""
inst_ok()   { _INST_REASON=""   ; return $INST_RC_OK; }
inst_fail() { _INST_REASON="$*" ; return $INST_RC_FAIL; }
inst_done() { _INST_REASON="$*" ; return $INST_RC_DONE; }
inst_na()   { _INST_REASON="$*" ; return $INST_RC_NA; }
inst_hint() { _INST_HINT="$*"; }
inst_say()  { printf '%s\n' "$*"; }

have_cmd()  { command -v "$1" >/dev/null 2>&1; }
have_file() { [ -f "$1" ]; }
have_pkg()  { dpkg -s "$1" >/dev/null 2>&1; }
# A path is writable if it is, or if its closest existing parent is (it can
# then be created).
ecrivable() { local d="$1"; while [ ! -e "$d" ] && [ "$d" != / ]; do d="$(dirname "$d")"; done; [ -w "$d" ]; }
# Absolute path without requiring it to exist (configure, make -C and the
# --dest change directory: a relative path would mean nothing there).
absolu() { case "$1" in /*) printf '%s\n' "$1" ;; *) printf '%s\n' "$PWD/$1" ;; esac; }
```

```
# exige "what is needed" "advice if absent" command...: a prerequisite
# condition. Under --check it is printed (ok / MANQUE + advice); otherwise it
# is remembered, and the first missing one gives the reason for the failure.
_MANQUE=()
exige() {
    local quoi="$1" conseil="$2"; shift 2
    if "$@" >/dev/null 2>&1; then
        [ "$ACTION" = check ] && printf '      ok      %s\n' "$quoi"
        return 0
    fi
    if [ "$ACTION" = check ]; then
        printf '      %sMANQUE%s  %s\n' "$C_KO" "$C_Z" "$quoi"
        [ -n "$conseil" ] && printf '              → %s\n' "$conseil"
    fi
    [ ${#_MANQUE[@]} -eq 0 ] && [ -n "$conseil" ] && inst_hint "$conseil"
    _MANQUE+=("$quoi")
    return 1
}
# Verdict of a check function that called exige.
bilan_exige() {
    [ ${#_MANQUE[@]} -eq 0 ] && { inst_ok; return $INST_RC_OK; }
    local plus=""; [ ${#_MANQUE[@]} -gt 1 ] && plus=" (+$(( ${#_MANQUE[@]} - 1 )) autre(s), --check)"
    inst_fail "manque : ${_MANQUE[0]}$plus"
}
# ═══════════════════════════════════════════════════════════════════════
#  THE COMPONENT - c54x_exe
# ═══════════════════════════════════════════════════════════════════════
COMPOSANT=c54x_exe
: "${GSM_ROOT:=/opt/GSM}"
: "${QOSMO:=$GSM_ROOT/qosmo}"
: "${ROM_DIR:=$GSM_ROOT}"
: "${ROM_VERSION:=3606}"
: "${PORTABLE:=0}"
: "${LOG_DIR:=/tmp/c54x_exe-install}"
ROM_DIRS=()
DEST=""
# Dockerfile:735 - the Makefile flags, without -march=native: a -march=native
# binary built elsewhere dies with SIGILL at startup (Dockerfile:726-728).
CFLAGS_PORTABLE="-O3 -g -Wall -Werror=format -Werror=format-extra-args -Wno-unused-function -Wno-unused-variable
 ↪  -Wno-unused-but-set-variable -Wno-sign-compare"
ROMS="DROM PDROM PROM0 PROM1 PROM2 PROM3"

# apt packages, taken from the single list of osmo-operator/Dockerfile:
#   :159      build-essential pkg-config curl       compilation; curl for rom/fetch-rom.sh
#   :166      python3                               tools/dsp_txt2bin.py (ROM conversion)
# libosmocore / libosmocoding are NOT apt packages in the image: it compiles
# them (Dockerfile:260-311). Here they are only required, through pkg-config.
PAQUETS=(build-essential pkg-config python3 curl)

options_composant() {
    case "$1" in
        --qosmo)    _val "$@"; QOSMO="$2";        _n=2 ;;
        --rom-dir)  _val "$@"; ROM_DIRS+=("$2"); _n=2 ;;
        --dest)     _val "$@"; DEST="$2";         _n=2 ;;
        --portable) PORTABLE=1;                   _n=1 ;;
    esac
}

apres_options() {
    QOSMO="$(absolu "$QOSMO")"
    if [ ${#ROM_DIRS[@]} -eq 0 ]; then ROM_DIRS=("$ROM_DIR" "$ICI/rom"); fi
    local d l=(); for d in "${ROM_DIRS[@]}"; do
        d="$(absolu "${d%/}")"; in_liste "$d" "${l[@]+"${l[@]}"}" || l+=("$d")
    done
    ROM_DIRS=("${l[@]}")
```

```
    # --dest: a COPY is built. The binary in service (a bench running on
    # $ICI/c54x_exe) is not rewritten under its feet.
    if [ -n "$DEST" ] && [ "$ACTION" = install ]; then
        DEST="$(absolu "$DEST")"
        if [ "$DEST" != "$ICI" ]; then
            mkdir -p "$DEST" || { printf 'impossible de creer %s\n' "$DEST" >&2; exit 1; }
            tar -C "$ICI" --exclude=./.git --exclude=./c54x_exe --exclude='./c54x_exe.*' \
                --exclude=./isa_test -cf - . | tar -C "$DEST" -xf - \
                || { printf 'copie vers %s impossible\n' "$DEST" >&2; exit 1; }
            printf 'arbre copie dans %s (sans .git ni binaires) - la suite s y deroule\n' "$DEST"
            local a sauter=0 args=()
            for a in "${_ARGS_ORIG[@]}"; do
                [ "$sauter" = 1 ] && { sauter=0; continue; }
                [ "$a" = --dest ] && { sauter=1; continue; }
                args+=("$a")
            done
            # without --rom-dir, the ROM goes into ROM_DIR and rom/ of the COPY
            exec bash "$DEST/install.sh" "${args[@]+"${args[@]}"}"
        fi
    fi
}
in_liste() { local x="$1"; shift; local y; for y in "$@"; do [ "$x" = "$y" ] && return 0; done; return 1; }

resume_composant() {
    printf 'depot=%s\nQOSMO=%s\nROM (%s) -> %s\nCFLAGS=%s\n' "$ICI" "$QOSMO" "$ROM_VERSION" "${ROM_DIRS[*]}" \
        "$([ "$PORTABLE" = 1 ] && echo "portables (Dockerfile:735)" || echo "ceux du Makefile (-march=native)")"
}
avant_etapes() { :; }
apres_etapes() {
    [ -x "$ICI/c54x_exe" ] && printf '  binaire : %s/c54x_exe\n' "$ICI"
    printf '  ROM     : %s\n' "${ROM_DIRS[*]}"
    [ "${ROM_DIRS[0]}" = /opt/GSM ] || printf '  (hors /opt/GSM : lancer c54x_exe avec --rom-dir %s)\n'
     ↪  "${ROM_DIRS[0]}"
    printf '  lancer  : PONT=1 %s/run.sh   (wiki osmo-operator, Installer-c54x_exe)\n' "$ICI"
}

# Is the ROM of a directory complete and matching rom/SHA256SUMS.<version>?
# (the same test as bon() of rom/fetch-rom.sh, plus the register snapshot)
rom_valide() {
    local s="$ICI/rom/SHA256SUMS.$ROM_VERSION" r
    [ -f "$s" ] || return 1
    for r in $ROMS Registers; do [ -f "$1/calypso_dsp.$r.bin" ] || return 1; done
    ( cd "$1" && sha256sum -c --quiet "$s" ) >/dev/null 2>&1
}
# A copy already verified on this machine, among the destinations and rom/.
rom_source_locale() {
    local d; for d in "${ROM_DIRS[@]}" "$ICI/rom"; do rom_valide "$d" && { printf '%s\n' "$d"; return 0; }; done
    return 1
}

# ── build ──────────────────────────────────────────────────────────────
# No "already done": the Makefile recompiles on EVERY call, by design
# (Makefile:46-52, the sources come from another repo).
INST_REGISTER build "Compilation de c54x_exe (sources de qosmo)"
_conseil_osmo() {
    if [ -f /usr/local/lib/pkgconfig/libosmocoding.pc ]; then
       printf 'export PKG_CONFIG_PATH=/usr/local/lib/pkgconfig  (libosmocore est en /usr/local, Dockerfile:92-94)'
    else
        printf 'installez libosmocore (sources : https://gitea.osmocom.org/osmocom/libosmocore, ou le paquet
         ↪  libosmocore-dev de votre depot Osmocom)'
    fi
}
inst_build_check() {
    _MANQUE=()
    exige "sources qosmo ($QOSMO/hw/arm/calypso/l1-dsp/calypso_c54x.c)" \
        "git clone https://github.com/bbaranoff/qosmO $QOSMO, ou --qosmo DIR" \
```

```
        test -f "$QOSMO/hw/arm/calypso/l1-dsp/calypso_c54x.c"
    exige "cales hors QEMU ($QOSMO/contrib/hors-qemu/cales-qemu.c)" "qosmo trop ancien : git -C $QOSMO pull" \
        test -f "$QOSMO/contrib/hors-qemu/cales-qemu.c"
    exige "compilateur C (${CC:-gcc})" "apt-get install build-essential" have_cmd "${CC:-gcc}"
    exige "make" "apt-get install build-essential" have_cmd make
    exige "pkg-config" "apt-get install pkg-config" have_cmd pkg-config
    # The Makefile (line 14) swallows the pkg-config error (2>/dev/null): without
    # this check, the failure only shows at link time, on missing gsm0503_*.
    exige "libosmocoding + libosmocore vus de pkg-config" "$(_conseil_osmo)" \
        pkg-config --exists libosmocoding libosmocore
    exige "$ICI inscriptible (le binaire y est ecrit)" "--dest DIR pour construire une copie ailleurs" test -w
     ↪  "$ICI"
    bilan_exige
}
inst_build_run() {
    # Dockerfile:733-735: cd c54x_exe && make QOSMO=/opt/GSM/qosmo CFLAGS="<without -march=native>"
    local args=("QOSMO=$QOSMO")
    [ "$PORTABLE" = 1 ] && args+=("CFLAGS=$CFLAGS_PORTABLE")
    make -C "$ICI" "${args[@]}" || { inst_fail "echec de compilation"; return $INST_RC_FAIL; }
    inst_ok
}
inst_build_verify() {
    [ -x "$ICI/c54x_exe" ] || { inst_fail "$ICI/c54x_exe absent apres make"; return $INST_RC_FAIL; }
    local nf
    nf="$(ldd "$ICI/c54x_exe" 2>/dev/null | awk '/not found/ {print $1}' | tr '\n' ' ')"
    [ -z "$nf" ] || { inst_hint "sudo ldconfig, ou LD_LIBRARY_PATH=/usr/local/lib"; inst_fail "bibliotheques
     ↪  introuvables : $nf"; return $INST_RC_FAIL; }
    inst_ok
}

# ── rom: the TI mask-ROM, downloaded (not in the repo) ──────────────────
INST_REGISTER rom "ROM DSP $ROM_VERSION (FreeCalypso, somme verifiee)"
inst_rom_done() { local d; for d in "${ROM_DIRS[@]}"; do rom_valide "$d" || return 1; done; }
inst_rom_check() {
    _MANQUE=()
    exige "rom/fetch-rom.sh et rom/SHA256SUMS.$ROM_VERSION" "" test -f "$ICI/rom/fetch-rom.sh" -a -f
     ↪  "$ICI/rom/SHA256SUMS.$ROM_VERSION"
    exige "sha256sum" "apt-get install coreutils" have_cmd sha256sum
    local d; for d in "${ROM_DIRS[@]}"; do
        exige "$d inscriptible (ROM)" "sudo, ou --rom-dir DIR" ecrivable "$d"
    done
    # The network, python3 and curl are only needed if no valid copy exists anywhere.
    if ! rom_source_locale >/dev/null; then
        exige "python3 (tools/dsp_txt2bin.py)" "apt-get install python3" have_cmd python3
        exige "curl ou wget (telechargement FreeCalypso)" "apt-get install curl" sh -c 'command -v curl || command
         ↪  -v wget'
    fi
    bilan_exige
}
inst_rom_run() {
    # 1. A copy already verified on this machine? copy it over, without network.
    #    (the equivalent of cp c54x_exe/rom/*.bin /opt/GSM/ in Dockerfile.run:188 and start-docker.sh)
    local src d r args=()
    if src="$(rom_source_locale)"; then
        for d in "${ROM_DIRS[@]}"; do
            [ "$d" = "$src" ] && continue
            rom_valide "$d" && continue
            mkdir -p "$d" || { inst_fail "impossible de creer $d"; return $INST_RC_FAIL; }
            for r in $ROMS Registers; do
                install -m644 "$src/calypso_dsp.$r.bin" "$d/calypso_dsp.$r.bin" \
                    || { inst_fail "copie de calypso_dsp.$r.bin vers $d impossible"; return $INST_RC_FAIL; }
            done
            inst_say "ROM DSP $ROM_VERSION recopiee de $src vers $d (somme verifiee)"
        done
    fi
    # 2. rom/fetch-rom.sh does the rest: downloads, converts, verifies, installs
```

```
    #    (Dockerfile:736). Everything already in place: it says so and downloads nothing.
    for d in "${ROM_DIRS[@]}"; do args+=(--dest "$d"); done
    bash "$ICI/rom/fetch-rom.sh" --version "$ROM_VERSION" "${args[@]}" \
        || { inst_hint "reseau ? un miroir : FREECALYPSO_URL=..."; inst_fail "rom/fetch-rom.sh en echec"; return
        ↪  $INST_RC_FAIL; }
    inst_ok
}
inst_rom_verify() { inst_rom_done && inst_ok || inst_fail "ROM incomplete ou somme differente dans
 ↪  ${ROM_DIRS[*]}"; }

# ── verify: the DSP alone ──────────────────────────────────────────────
# README.md:66: "the ROM alone, without ARM: does it boot?". Outside --arm,
# c54x_exe works on a PRIVATE API RAM: no /dev/shm, no socket, no
# /tmp/c54x-pont - a bench running alongside sees nothing of it. The caller's
# CALYPSO_* (probes, traces) are removed so that it stays that way.
INST_REGISTER verify "Controle : c54x_exe --trames 200 (la ROM seule)"
sans_calypso_env() {
    local v u=(); for v in $(compgen -e); do case "$v" in CALYPSO_*) u+=(-u "$v") ;; esac; done
    env "${u[@]+"${u[@]}"}" "$@"
}
inst_verify_check() {
    _MANQUE=()
    etape_retenue build || exige "binaire $ICI/c54x_exe" "./install.sh --only build" test -x "$ICI/c54x_exe"
    etape_retenue rom   || exige "ROM DSP dans ${ROM_DIRS[0]}" "./install.sh --only rom" rom_valide
     ↪  "${ROM_DIRS[0]}"
    bilan_exige
}
inst_verify_run() {
    local out rc
    out="$(sans_calypso_env timeout 300 "$ICI/c54x_exe" --rom-dir "${ROM_DIRS[0]}" --trames 200 2>&1)"; rc=$?
    printf '%s\n' "$out" | tail -n 14
    [ $rc -eq 0 ] || { inst_fail "c54x_exe --trames 200 rend $rc"; return $INST_RC_FAIL; }
    grep -q 'sections chargees' <<<"$out" && grep -q 'bilan sur 200 trames' <<<"$out" \
        || { inst_fail "sortie inattendue (ni « sections chargees » ni « bilan sur 200 trames »)"; return
        ↪  $INST_RC_FAIL; }
    inst_ok
}
# ═══════════════════════════════════════════════════════════════════════
#  THE ENGINE - IDENTICAL copy in the three repos (see THE CONTRACT)
# ═══════════════════════════════════════════════════════════════════════
#  Same flow as osmo-operator/install.sh: for each retained step,
#  "already done?" (except --reinstall), prerequisites, work, check. The first
#  failure stops everything, says why and shows the tail of the step's log.
#  The component provides: COMPOSANT, PAQUETS, LOG_DIR, the steps, and the
#  hooks options_composant / apres_options / resume_composant /
#  avant_etapes / apres_etapes.
# -----------------------------------------------------------------------
ACTION=install ONLY="" SKIP="" WITH_DEPS=0
: "${REINSTALL:=0}" "${VERBOSE:=0}"
_ARGS_ORIG=("$@")
# _val OPTION [VALUE]: an option that expects a value must have one.
_val() { [ $# -ge 2 ] && [ -n "$2" ] || { printf '%s : valeur attendue   (--help)\n' "$1" >&2; exit 2; }; }
while [ $# -gt 0 ]; do
    case "$1" in
        -h|--help)    ACTION=help ;;
        --check)      ACTION=check ;;
        --list)       ACTION=list ;;
        --deps)       ACTION=deps ;;
        --print-deps) ACTION=print-deps ;;
        --with-deps)  WITH_DEPS=1 ;;
        --only)       _val "$@"; ONLY="$2"; shift ;;
        --skip)       _val "$@"; SKIP="$2"; shift ;;
        --reinstall)  REINSTALL=1 ;;
        -v|--verbose) VERBOSE=1 ;;
        *)  _n=0; options_composant "$@"
            [ "$_n" -gt 0 ] || { printf 'option inconnue : %s   (--help)\n' "$1" >&2; exit 2; }
```

```
            shift $((_n - 1)) ;;
    esac
    shift
done

if [ "$ACTION" = help ]; then
    awk 'NR > 2 && /^# -{10}/ { exit } NR > 2 && !/^# =+$/ { sub(/^# ?/, ""); print }' "$0"
    exit 0
fi
apres_options

if [ -t 1 ] && [ -z "${NO_COLOR:-}" ]; then
    TTY=1; C_OK=$'\033[32m'; C_KO=$'\033[31m'; C_SK=$'\033[33m'; C_DIM=$'\033[2m'; C_Z=$'\033[0m'
else
    TTY=0; C_OK=""; C_KO=""; C_SK=""; C_DIM=""; C_Z=""
fi
begin() { if [ $TTY -eq 1 ] && [ "$VERBOSE" != 1 ]; then printf '[ %s..%s ] %s' "$C_DIM" "$C_Z" "$1"; else printf
 ↪  '[ .. ] %s\n' "$1"; fi; }
end()   { [ $TTY -eq 1 ] && [ "$VERBOSE" != 1 ] && printf '\r\033[K'
          printf '[%s%s%s] %s' "$2" "$1" "$C_Z" "$3"
          [ -n "${4:-}" ] && printf ' %s(%s)%s' "$C_DIM" "$4" "$C_Z"; printf '\n'; }

# --- step selection (--only / --skip, comma-separated lists) -----------------
in_csv() { case ",$1," in *",$2,"*) return 0;; esac; return 1; }
for _s in ${ONLY//,/ } ${SKIP//,/ }; do
    [ -n "${INST_DESC[$_s]+x}" ] || { printf 'etape inconnue : %s   (etapes : %s)\n' "$_s" "${INST_ORDER[*]}" >&2;
     ↪  exit 2; }
done
SELECTED=()
for _s in "${INST_ORDER[@]}"; do
    [ -n "$ONLY" ] && ! in_csv "$ONLY" "$_s" && continue
    [ -n "$SKIP" ] && in_csv "$SKIP" "$_s" && continue
    SELECTED+=("$_s")
done
# A step that produces what another requires: under --check, the second one
# is not blamed for the absence of what the first one is about to build.
etape_retenue() { in_csv "$(IFS=,; echo "${SELECTED[*]}")" "$1"; }

# --- apt packages ------------------------------------------------------------
# returns the list of missing ones; code 2 without dpkg (not a Debian/Ubuntu)
paquets_absents() {
    have_cmd dpkg || return 2
    local p; for p in "${PAQUETS[@]}"; do have_pkg "$p" || printf '%s ' "$p"; done
    return 0
}
installer_paquets() {
    local manque rc
    manque="$(paquets_absents)"; rc=$?
    if [ $rc -eq 2 ]; then
        printf 'dpkg absent : installez l equivalent de : %s\n' "${PAQUETS[*]}" >&2; return 1
    fi
    if [ -z "${manque// }" ]; then
        printf 'paquets apt : les %d sont presents\n' "${#PAQUETS[@]}"; return 0
    fi
    if [ "$(id -u)" -ne 0 ]; then
        printf 'paquets apt manquants : %s\n  sudo apt-get install -y --no-install-recommends %s\n' "$manque"
         ↪  "$manque" >&2
        return 1
    fi
    printf 'apt-get install : %s\n' "$manque"
    apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y --no-install-recommends $manque ||
     ↪  return 1
    manque="$(paquets_absents)"
    [ -z "${manque// }" ] || { printf 'toujours absents apres apt-get : %s\n' "$manque" >&2; return 1; }
}
```

```
case "$ACTION" in
    print-deps) printf '%s\n' "${PAQUETS[*]}"; exit 0 ;;
    deps)       installer_paquets; exit $? ;;
    list)
        printf '%s - %d etape(s), dans cet ordre\n\n' "$COMPOSANT" "${#SELECTED[@]}"
        for _s in "${SELECTED[@]}"; do printf '  %-10s %s\n' "$_s" "${INST_DESC[$_s]}"; done
        printf '\n'; resume_composant | sed 's/^/  /'
        exit 0 ;;
    check)
        printf '%s : prerequis et etat (rien n est modifie)\n\n' "$COMPOSANT"
        resume_composant | sed 's/^/  /'
        _manque="$(paquets_absents)"; _rc=$?
        if [ $_rc -eq 2 ]; then printf '\n  paquets apt : dpkg absent, non verifies (%s)\n' "${PAQUETS[*]}"
        elif [ -n "${_manque// }" ]; then printf '\n  paquets apt absents (installes autrement ? sinon --deps) :
         ↪  %s\n' "$_manque"
        else printf '\n  paquets apt : les %d sont presents\n' "${#PAQUETS[@]}"; fi
        _rc=0
        for _s in "${SELECTED[@]}"; do
            _p="inst_${_s//-/_}"
            if ! declare -F "${_p}_done" >/dev/null; then _etat="pas de controle"
            elif "${_p}_done" >/dev/null 2>&1; then _etat="deja fait"
            else _etat="a faire"; fi
            printf '\n  %-10s %s  %s[%s]%s\n' "$_s" "${INST_DESC[$_s]}" "$C_DIM" "$_etat" "$C_Z"
            declare -F "${_p}_check" >/dev/null || continue
            _INST_REASON=""; _INST_HINT=""; _MANQUE=()
            "${_p}_check"; _r=$?
            case $_r in
                "$INST_RC_FAIL") _rc=1 ;;
                "$INST_RC_NA")   printf '      sans objet : %s\n' "$_INST_REASON" ;;
            esac
        done
        printf '\n'
        if [ $_rc -eq 0 ]; then printf '%sprerequis satisfaits%s\n' "$C_OK" "$C_Z"
        else printf '%sprerequis manquants%s (voir MANQUE ci-dessus)\n' "$C_KO" "$C_Z"; fi
        exit $_rc ;;
esac

# --- installation ------------------------------------------------------------
if [ "$WITH_DEPS" = 1 ]; then
    installer_paquets || { printf 'paquets apt : echec - rien n a ete construit\n' >&2; exit 1; }
else
    _manque="$(paquets_absents 2>/dev/null)"
    [ -n "${_manque// }" ] && printf '%snote : paquets apt absents (installes autrement ? sinon --with-deps) :
     ↪  %s%s\n' "$C_DIM" "$_manque" "$C_Z"
fi
if [ "$VERBOSE" != 1 ]; then
    mkdir -p "$LOG_DIR" || { printf 'journal impossible : %s (LOG_DIR=...)\n' "$LOG_DIR" >&2; exit 1; }
fi
# _jouer FUNCTION: the output goes to the step's log, or to the screen with -v.
# No pipe: the function runs in THIS shell (its failure reason survives).
_jouer() { if [ "$VERBOSE" = 1 ]; then "$1"; else "$1" >>"$_log" 2>&1; fi; }
_echec() {   # _echec STEP REASON [log]: says everything known, and stops
    end FAIL "$C_KO" "${INST_DESC[$1]}" "$2"
    [ -n "$_INST_HINT" ] && printf '       → %s\n' "$_INST_HINT"
    # the tail of the log, when it is the work (run, verify) that failed
    if [ -n "${3:-}" ] && [ "$VERBOSE" != 1 ] && [ -s "$_log" ]; then
        printf '       %sjournal : %s (fin ci-dessous)%s\n' "$C_DIM" "$_log" "$C_Z"
        tail -n 25 "$_log" | sed 's/^/       | /'
    fi
    printf '\n%s : installation interrompue a l etape %s\n' "$COMPOSANT" "$1"
    exit 1
}
avant_etapes
_nb_ok=0; _nb_skip=0
for _s in "${SELECTED[@]}"; do
    _p="inst_${_s//-/_}"; _log="$LOG_DIR/$_s.log"
```

```
    _INST_REASON=""; _INST_HINT=""; _MANQUE=()
    [ "$VERBOSE" = 1 ] || printf '\n===== %s  %s =====\n' "$(date '+%F %T')" "$_s" >>"$_log"
    begin "${INST_DESC[$_s]}"
    if [ "$REINSTALL" != 1 ] && declare -F "${_p}_done" >/dev/null && _jouer "${_p}_done"; then
        end SKIP "$C_SK" "${INST_DESC[$_s]}" "deja fait"; _nb_skip=$((_nb_skip + 1)); continue
    fi
    if declare -F "${_p}_check" >/dev/null; then
        _jouer "${_p}_check"; _r=$?
        case $_r in
            "$INST_RC_OK")   ;;
            "$INST_RC_NA")   end SKIP "$C_SK" "${INST_DESC[$_s]}" "${_INST_REASON:-sans objet}"
                             _nb_skip=$((_nb_skip + 1)); continue ;;
            *)               _echec "$_s" "${_INST_REASON:-prerequis non satisfait}" ;;
        esac
    fi
    _jouer "${_p}_run"; _r=$?
    case $_r in
        "$INST_RC_OK")   ;;
        "$INST_RC_DONE"|"$INST_RC_NA")
                         end SKIP "$C_SK" "${INST_DESC[$_s]}" "${_INST_REASON:-rien a faire}"
                         _nb_skip=$((_nb_skip + 1)); continue ;;
        *)               _echec "$_s" "${_INST_REASON:-echec}" journal ;;
    esac
    # installing is not having installed: checked after the fact
    if declare -F "${_p}_verify" >/dev/null && ! _jouer "${_p}_verify"; then
        _echec "$_s" "faite, mais le controle echoue : ${_INST_REASON:-}" journal
    fi
    end " OK " "$C_OK" "${INST_DESC[$_s]}" "${_INST_REASON:-}"; _nb_ok=$((_nb_ok + 1))
done
printf '\n%s : %d ok · %d ignoree(s) · 0 echec\n' "$COMPOSANT" "$_nb_ok" "$_nb_skip"
apres_etapes
exit 0
```

3.6 /opt/GSM/c54x_exe/mobile_pont.cfg

4120 octets, 166 lignes → 166 lignes

```
# mobile.cfg.template - with placeholders
# KI: "ki comp128 XX XX XX ... XX"  16 bytes, byte 15=ms_idx, 16=op_id
#
# AUDIO: same as the run without --dsp (/root/.osmocom/bb/mobile.cfg) except
# the TCH frame format, plus the data channel.
#   tch-voice  io-handler gapk, io-tch-format ti (the DSP vocodes; it is
#              "rtp" without --dsp, where pont.py does the coding), ALSA
#              gsm_out/gsm_in.
#   tch-data   io-handler unix-sock, /tmp/ms_data (CSD 9600 async, 8N1).
#   The order tch-voice then tch-data is the one the mobile itself writes.
#   `io-tch-format` is only valid behind gapk or unix-sock: under l1phy the
#   mobile rejects it ("This parameter is only valid for GAPK") and discards
#   the whole file.
#
# SOCKETS: the same as the run without --dsp (/tmp/osmocom_l2, /tmp/osmocom_sap),
# not the "_pont" variants. In the DSP setup the fork's `l2` module is not
# run, so no other mobile takes them. Running a grgsm bench AND this bench by
# hand at the same time would make them fight over the sockets: this is the
# accepted trade-off for having a single set of names.
!
line vty
 no login
 bind 127.0.0.1 4247
!
gps device /dev/ttyACM0
gps baudrate default
no gps enable
```

```
!
no hide-default
!
!
log stderr
 logging filter all 1
 logging color 1
 logging timestamp 1
 logging print category 1
 logging level mm debug
 logging level rr debug
 logging level cc debug
 logging level sms debug
!
log gsmtap 172.20.0.1
 logging filter all 1
 logging color 0
 logging timestamp 0
 logging print category 1
 logging level mm debug
 logging level rr debug
 logging level cc debug
 logging level sms debug
!
ms 1
 layer2-socket /tmp/osmocom_l2
 sap-socket /tmp/osmocom_sap
 sim test
 network-selection-mode auto
 imei 358925008967594 0
 imei-fixed
 no emergency-imsi
 sms-service-center +3366612340001
 no call-waiting
 no auto-answer
 no force-rekey
 no clip
 no clir
 tx-power auto
 no simulated-delay
 stick 1
 location-updating
 neighbour-measurement
 codec full-speed prefer
 codec half-speed
 no abbrev
 support
  sms
  a5/1
  a5/2
  p-gsm
  e-gsm
  no r-gsm
  no gsm-850
  dcs
  no pcs
  class-900 4
  class-850 4
  class-dcs 1
  class-pcs 1
  channel-capability sdcch+tchf+tchh
  full-speech-v1
  full-speech-v2
  half-speech-v1
  min-rxlev -106
  dsc-max 90
```

```
  no skip-max-per-band
 test-sim
  imsi 001010001000001
  ki comp128 00 11 22 33 44 55 66 77 88 99 aa bb cc dd 01 01
  no barred-access
  rplmn 001 01
 tch-voice
  io-handler gapk
! TCH FRAME FORMAT: "ti" HERE, "rtp" WITHOUT --dsp.
! This is not a preference, it is WHO does the vocoding. On this bench the
! real DSP runs (c54x_exe, TI mask ROM) and returns its frames in ITS own
! layout, the one of Calypso phones: "ti".
! The gr-gsm setup has no DSP: pont.py does the coding with libosmocoding
! (gsm0503_tch_fr_encode/decode), which speaks RTP/RFC3551, 33 bytes opened
! by the 0xD signature; hence the "rtp" of /root/.osmocom/bb/mobile.cfg,
! which applies to THAT setup, not to this one. Picking the wrong side raises
! no counter, the frames go through: they are simply read with the wrong
! layout, and the audio comes out robotic.
  io-tch-format ti
  alsa-output-dev gsm_out
  alsa-input-dev gsm_in
 tch-data
  io-handler unix-sock
  io-tch-format ti
  unix-socket /tmp/ms_data
  call-params type-rate 71
  call-params ce transparent
  call-params async
  call-params async nr-stop-bits 1
  call-params async nr-data-bits 8
  call-params async parity none
 no shutdown
!
! GSMTAP configuration
!
gsmtap
 remote-host 172.20.0.1
 no local-host
 lchan sacch
 lchan lsacch
 lchan sacch/4
 lchan sacch/8
 lchan sacch/f
 lchan sacch/h
 lchan unknown
 lchan bcch
 lchan ccch
 lchan rach
 lchan agch
 lchan pch
 lchan sdcch
 lchan sdcch/4
 lchan sdcch/8
 lchan facch/f
 lchan facch/h
 lchan pacch
 lchan cbch
 lchan pdch
 lchan pttch
 lchan tch/f
 lchan tch/h
 category gprs dl-unknown
 category gprs dl-dummy
 category gprs dl-ctrl
 category gprs dl-data-gprs
 category gprs dl-data-egprs
```

```
 category gprs ul-unknown
 category gprs ul-dummy
 category gprs ul-ctrl
 category gprs ul-data-gprs
 category gprs ul-data-egprs
end
```

3.7 /opt/GSM/c54x_exe/repro_toa.sh

5217 octets, 102 lignes → 102 lignes

```
#!/bin/bash
# Reproduces the "magic 23" evidence and writes everything to toa.log.
# Idempotent: skips downloads and builds already present.
# Needs: curl, python3, gcc, make; osmocom-bb is cloned if absent.
set -u
LOG=toa.log
: > "$LOG"
say() { printf '%s\n' "$*" | tee -a "$LOG"; }
run() { say "\$ $*"; "$@" 2>&1 | tee -a "$LOG"; say ""; }

say "# magic 23 reproduction — $(date -u +%Y-%m-%dT%H:%MZ) — $(uname -srm)"
say ""

# ------------------------------------------------------------------ 1. ROM
say "## 1. ROM dump -> PROM0.bin"
[ -f dsp-rom-3606-dump.txt ] || run curl -sO ftp://ftp.freecalypso.org/pub/GSM/Calypso/dsp-rom-3606-dump.txt
run sha256sum dsp-rom-3606-dump.txt
python3 - <<'E' 2>&1 | tee -a "$LOG"
import re,struct
sec=None;w={}
for l in open("dsp-rom-3606-dump.txt",errors="replace"):
    m=re.match(r"DSP dump: (\w+) \[",l)
    if m: sec=m.group(1); continue
    m=re.match(r"([0-9a-f]{5}) : ((?:[0-9a-f]{4} ?)+)",l)
    if m and sec=="PROM0":
        a=int(m.group(1),16)
        for i,x in enumerate(m.group(2).split()): w[a+i]=int(x,16)
open("PROM0.bin","wb").write(b"".join(struct.pack("<H",w[a]) for a in range(0x7000,0xe000)))
print("PROM0.bin: %d words (0x7000-0xdfff)" % len(range(0x7000,0xe000)))
for a in (0x7cbd,0x84a4,0xb21d,0xb2b1,0xb32d):
    print("  %04x: %04x %04x" % (a,w[a],w[a+1]))
E
run sha256sum PROM0.bin
say ""

# ------------------------------------------------------------ 2. disassembler
say "## 2. objdump tic54x (binutils 2.21.1)"
OBJDUMP=build-tic54x/binutils/objdump
if [ ! -x "$OBJDUMP" ]; then
    [ -f binutils-2.21.1.tar.bz2 ] || run curl -sO https://ftp.gnu.org/gnu/binutils/binutils-2.21.1.tar.bz2
    [ -d binutils-2.21.1 ] || tar xjf binutils-2.21.1.tar.bz2
    mkdir -p build-tic54x && ( cd build-tic54x && \
      ../binutils-2.21.1/configure --target=tic54x-coff --disable-werror --disable-nls --disable-gdb \
        --disable-gas --disable-ld --disable-gold --disable-gprof --disable-sim MAKEINFO=true \
        CFLAGS='-O1 -w -std=gnu89' >/dev/null && make -j8 all-binutils MAKEINFO=true >/dev/null ) \
      && say "built $OBJDUMP" || say "BUILD FAILED"
fi
run "$OBJDUMP" --version
"$OBJDUMP" -D -b binary -m tms320c54x --adjust-vma=0x7000 PROM0.bin > PROM0.dis 2>&1
say "full disassembly: PROM0.dis ($(wc -l < PROM0.dis) lines)"
say ""
say "### expected: 7cbd sub #11236,a | 84a4 stm #10791,ar3 | b21d stm #191,ar3 | b2b1 stm #151,ar3 | b32d stm
 ↪  #48,ar3"
```

```
grep -E '^ *(7cbd|84a4|b21d|b2b1|b32d):' PROM0.dis | tee -a "$LOG"
say ""
say "### SB TOA chain"
for a in 7c3d 81c8 7c49 7a1c 84a1 84ca 84dd 7c5f 7ca6 7cbc 7e8a b1e7; do
    grep -E "^ *$a:" PROM0.dis | tee -a "$LOG"
done
say ""
say "### FB TOA: 0x793e-0x795a"
awk '/^ *793e:/,/^ *795a:/' PROM0.dis | tee -a "$LOG"
say ""
say "### DMA programming routine: 0xa5cd-0xa5f6"
awk '/^ *a5cd:/,/^ *a5f6:/' PROM0.dis | tee -a "$LOG"
say ""

# ------------------------------------------------------------- 3. git
say "## 3. layer1 git history"
[ -d osmocom-bb ] || run git clone -q https://gitea.osmocom.org/phone-side/osmocom-bb
run git -C osmocom-bb log --format='%h %ad %an %s' --date=short -S"toa -= 23" -- src/target/firmware/layer1
run git -C osmocom-bb log --format='%h %ad %an %s' --date=short -S"magic 23" -- src/target/firmware/layer1
say "### cb71b972: toa -= 23 / fbinfo2cellinfo lines (expect added fbinfo2cellinfo, no removed toa -= 23)"
git -C osmocom-bb show cb71b972 -- src/target/firmware/layer1/prim_fbsb.c | grep -nE '^[-+].*(toa -=
↪  23|fbinfo2cellinfo)|^@@' | tee -a "$LOG"
say ""
say "### current constants"
grep -nE 'L1_(SB|NB)_MARGIN_Q|L1_TAIL_DURATION_Q|L1_(SB|NB)_DURATION_Q'
↪  osmocom-bb/src/target/firmware/layer1/tpu_window.c | tee -a "$LOG"
grep -nE 'toa -= 23|magic 23' osmocom-bb/src/target/firmware/layer1/prim_fbsb.c | tee -a "$LOG"
grep -nE 'tpu_shift \+= 75|SB TOA of 4' osmocom-bb/src/target/firmware/layer1/sync.c | tee -a "$LOG"
say ""

# ------------------------------------------------------------- 4. logs
say "## 4. hardware logs: qbits check ((TOA-46) mod 1250)*4"
say "sources: http://lists.osmocom.org/pipermail/baseband-devel/2011-September/002526.html"
say "         https://www.mail-archive.com/baseband-devel@lists.osmocom.org/msg01046.html"
python3 - <<'E' 2>&1 | tee -a "$LOG"
fb1 = [(9651,3420),(8751,4820),(8755,4836),(10003,4828),(10007,4844),(8755,4836)]
sb  = [(29,4932),(29,4932),(27,4924),(27,4924),(24,4912),(26,4920)]
ok = 0
for toa,q in fb1+sb:
    p46 = ((toa-46)%1250)*4; p23 = ((toa-23)%1250)*4
    ok += (p46==q)
    print("  TOA=%5d printed=%4d  -46:%4d %s  -23:%4d" % (toa,q,p46,"OK " if p46==q else "NO ",p23))
print("  %d/%d lines match the -46 rule" % (ok,len(fb1)+len(sb)))
fb0=[7200,2544,8784,10032,48,1296,8784]; m1=[9651,8751,8755,10003,10007,12507,12507,1259,10007,8755]
print("  FB mode 0 multiples of 48: %d/%d" % (sum(t%48==0 for t in fb0),len(fb0)))
print("  FB mode 1 == 3 mod 4:      %d/%d" % (sum(t%4==3 for t in m1),len(m1)))
d=[t-27.25 for t,_ in sb]
print("  first SB after FB sync: %s  mean delta %.2f" % ([t for t,_ in sb], sum(d)/len(d)))
print("  148 + 2*23 - 3 = %d ; 148 + 2*3 - 3 = %d ; (191-148+3)/2 = %g" % (148+2*23-3,148+2*3-3,(191-148+3)/2))
E
say ""
say "# done — $(wc -l < "$LOG") lines in $LOG"
```

3.8 /opt/GSM/c54x_exe/run.sh

63272 octets, 923 lignes → 923 lignes

```
#!/usr/bin/env bash
# run.sh - the MOBILE side of the Calypso bench, process by process, without the network.
#
# Two setups (MODE):
#   dsp    (default) 1. c54x_exe --arm   the TMS320C54x DSP, TI mask-ROM, outside QEMU
#                    2. qosmo            the ARM + osmocom-bb layer1 (CALYPSO_DSP_EXTERN=1)
#                    3. osmocon          romload on the serial0 pty, L1CTL relay
```

```
#                   4. mobile           osmocom-bb layer 2/3
#                   5. pont_dsp.py      (PONT=1) BTS bursts -> DSP (--dsp-port 6702);
#                                       the DSP bridge (pont/dsp/), TCH switch followed from the firmware
#   grgsm           2. qosmo            the ARM + layer1 with the gr-gsm layer 1 (shunt)
#                   3. osmocon  4. mobile  5. pont.py (PONT=1) BTS bursts -> gr-gsm L1
# The bridge needs the BTS (osmo-bts-trx, TRXD 5700): without network it starts and waits.
#   OSMO_TRX=1      THE PHONE IS THE TRX OF BTS#0 (start-native.sh --osmo-trx):
#                   2. qosmo on trx.highram.elf (osmocom-bb-transceiver, "trx" app by Sylvain Munaut)
#                   4. transceiver (host, osmocom-bb-transceiver, patches/osmocom-bb-transceiver-qemu-bench)
#                      INSTEAD OF mobile: L1CTL on $L2_SOCK, TRXD 5700-5702 facing osmo-bts-trx,
#                      --no-sync (no cell to listen to), a single RX timeslot (OSMO_TRX_TS_MASK=0x01);
#                      the TX mask stays 0xe3 (TS 5,6,7,0,1): prim_bts.c only serves the TS0 BCCH if
#                      tx_start == 5 (TX window shifted by 3 slots). -m 0x01 would also set
#                      TX=0x01 -> tx_start 0 -> dummy on every frame (DUMMY only, FB 0, SB 0).
#                   5. no bridge: it would hold the same ports 5700-5702 (PONT forced to 0)
#                  OSMO_TRX_ARGS (default "--no-sync -e 3", + "-r $OSMO_TRX_RT" SCHED_RR 20), TRANSCEIVER (binary
 ↪  path).
#   INSTANCE=2      A SECOND COMPLETE CHAIN next to the first (banc-mobile while banc-trx runs):
#                   run.sh re-executes itself in a private mount namespace where /dev/shm IS
#                   /dev/shm/calypso2 (as seen from the host), and the host's real /dev/shm is
#                   visible at /run/calypso-shm-hote; no /dev/shm/... path in the code changes.
#                   RUNDIR /tmp/c54x-pont2, L1CTL /tmp/osmocom_l2_2, DSP socket /tmp/calypso_dsp2.sock,
#                   monitor, gdb 1235/44445, keyboard /tmp/calypso_keys2.sock, BSP udp 6712, mobile VTY
#                   +100, 4347 (copy of mobile_pont.cfg; 4257 belongs to osmo-ggsn), PANNEAU_LOGS=none. AIR=1
 ↪  (default in instance 2):
#                   step 5 = pont/air.py, the air between the two Calypsos (TX bursts of one to the BSP
#                   of the other) instead of the bridge.
#   FIRMWARE_APP=rssi  a firmware app ALONE on the ARM + the DSP (rssi, menu, hello_world,
#                   snake_game...: board/compal_e88/<app>.highram.elf): steps 1-3, neither mobile
#                   nor bridge. The C123 screen and keyboard are emulated (qosmo:
#                   calypso_st7558.c -> /dev/shm/calypso_lcd, calypso_armio.c <- /tmp/calypso_keys.sock);
#                   the window: /usr/bin/python3 /opt/GSM/osmo-operator/tools/osmo-c123-gtk.py
#
#   ./run.sh                launch everything      ./run.sh --status    what is running
#   MODE=grgsm ./run.sh     gr-gsm setup           ./run.sh --logs      follow the logs
#   PONT=1 ./run.sh         with the bridge        ./run.sh --stop      stop everything, clean up
#   ./run.sh --step N       a single step (the previous ones must be running)
# Variables: MODE, PONT, LOCKSTEP (1: QEMU waits for the DSP on every frame), INSNS (80000),
#   VERB (-v), IQ (none|fcch|cell|...), AMP (30000),
#   QOSMO, FIRMWARE_ELF, FIRMWARE_BIN, OSMOCON, MOBILE, MOBILE_CFG, PONT_PY, RUNDIR, L2_SOCK.
#   UL_RAW (1: the bridge sends the ROM's final bursts as they are; 0: host coding), MONTANT_EFFACE_TASK_RA.
#   Real time: RT_NICE (-10 by default, empty = none), RT_FIFO (SCHED_FIFO, e.g. 20, root), RT_CPUS (taskset,
#   e.g. 1-3), RT_RENICE_AUTRES (e.g. 15: renice of the panel tools); ./run.sh --charge  who consumes what.
# Details, expectations and checks: LAUNCH.md alongside. Noise (BRUIT_MODE, inactive by default): BRUIT_ block at
 ↪  the end.
set -uo pipefail
HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
MODE="${MODE:-dsp}"
PONT="${PONT:-0}"
# INSTANCE=N (N != 1): private /dev/shm = the host's /dev/shm/calypsoN; the real /dev/shm stays
# readable at /run/calypso-shm-hote (the air reads the TX ring of instance 1 there). The processes
# launched here inherit the namespace; --stop/--status re-run with INSTANCE=N enter it too.
INSTANCE="${INSTANCE:-1}"
# PER-INSTANCE SETTINGS: instances/instance-$INSTANCE.env (written by the web page tools/osmo-reglages.py,
# template instances/instance.env.example). `: "${VAR:=value}"` lines: the environment (start-native.sh, command
# line) keeps priority, then the file, then the defaults below. Exported (set -a) for c54x_exe/QEMU/bridge.
INSTANCE_ENV="${INSTANCE_ENV:-/opt/GSM/osmo-operator/instances/instance-$INSTANCE.env}"
if [ -r "$INSTANCE_ENV" ]; then
    set -a; . "$INSTANCE_ENV"; set +a
    printf '\033[1m[run dsp]\033[0m instance %s : reglages de %s\n' "$INSTANCE" "$INSTANCE_ENV"
fi
HOTE_SHM=/run/calypso-shm-hote
AIR_DEMANDE="${AIR:-}"   # AIR set by the caller (otherwise default per instance / app, below)
if [ "$INSTANCE" != 1 ] && [ -z "${CALYPSO_NS:-}" ]; then
    command -v unshare >/dev/null || { echo "[run] ECHEC : unshare absent (util-linux)" >&2; exit 1; }
    _d="/dev/shm/calypso$INSTANCE"; mkdir -p "$_d" "$HOTE_SHM" && chmod 1777 "$_d"
```

```
    exec unshare -m --propagation private bash -c '
        mount --bind /dev/shm "$4" && mount --bind "$1" /dev/shm && export CALYPSO_NS="$2" && exec bash "$3"
↪   "${@:5}"' \
        _ "$_d" "$INSTANCE" "$0" "$HOTE_SHM" "$@"
fi
if [ "$INSTANCE" != 1 ]; then
    RUNDIR="${RUNDIR:-/tmp/c54x-pont$INSTANCE}"
    L2_SOCK="${L2_SOCK:-/tmp/osmocom_l2_$INSTANCE}"
    MONITOR="${MONITOR:-/tmp/qemu-monitor-pont$INSTANCE.sock}"
    DSP_SOCK="${DSP_SOCK:-/tmp/calypso_dsp$INSTANCE.sock}"
    GDB_STUB="${GDB_STUB:-$((1233 + INSTANCE))}"
    GDB_TELNET="${GDB_TELNET:-$((44443 + INSTANCE))}"
    PANNEAU_LOGS="${PANNEAU_LOGS:-none}"
    # instance >= 3: +30, because 6722 (= 6702 + 20) is the "MS" TRXD port of fake_trx.py (-p 6720, sidecar of
    # BTS 1): bound on 127.0.0.1:6722 it would steal the uplink bursts of phone 2 (instance 3: 6752).
    export CALYPSO_BSP_PORT="${CALYPSO_BSP_PORT:-$((6702 + 10 * (INSTANCE - 1) + (INSTANCE >= 3 ? 30 : 0)))}"
    export CALYPSO_KEYS_SOCK="${CALYPSO_KEYS_SOCK:-/tmp/calypso_keys$INSTANCE.sock}"
    AIR="${AIR:-1}"
fi
AIR="${AIR:-0}"
AIR_PY="${AIR_PY:-/opt/GSM/osmo-operator/pont/air.py}"
QOSMO="${QOSMO:-/opt/GSM/qosmo}"
QEMU="${QEMU:-$QOSMO/build/qemu-system-arm}"
# Firmware: the original layer1.highram, unmodified. FIRMWARE_ELF=... for another binary.
_FW=/opt/GSM/firmware/board/compal_e88
# OSMO_TRX=1: trx.highram firmware (the Calypso transmits the BTS), host app transceiver.
OSMO_TRX="${OSMO_TRX:-0}"
FIRMWARE_APP="${FIRMWARE_APP:-}"
if [ -n "$FIRMWARE_APP" ]; then
    FIRMWARE_ELF="$_FW/$FIRMWARE_APP.highram.elf"   # also overrides /etc/default/osmo-banc
    FIRMWARE_BIN="${FIRMWARE_ELF%.elf}.bin"
    PONT=0
fi
# RSSI: realistic air by default (calypso_bsp.c CALYPSO_AIR_NIVEAUX: the cell only on its ARFCN, DL/UL
# levels, noise elsewhere) and, with a classic BTS (pont_dsp.py alive), no air.py: the downlink arrives
# through the bridge's copy (PONT_DSP_COPIES, udp 6762). Set here and not in banc-rssi, so that any
# launcher gets the same behaviour.
if [ "$FIRMWARE_APP" = rssi ]; then
    export CALYPSO_AIR_NIVEAUX="${CALYPSO_AIR_NIVEAUX:-1}"
    export CALYPSO_FAUX_AIR="${CALYPSO_FAUX_AIR:-$( . /etc/osmocom/radio-plan.env 2>/dev/null; echo
↪    "${PLAN_ARFCN:-}")}"
    [ -z "$AIR_DEMANDE" ] && pgrep -f 'pont/pont_dsp\.py' >/dev/null && AIR=0
fi
# OSMO_TRX_TS1=1 (instance 1, with OSMO_TRX=1): a SECOND Calypso TRX for TS1 (TCH/F, the calls): the
# instance OSMO_TRX_TS1_INSTANCE (3) is launched before the transceiver (steps 1-3 only, OSMO_TRX_PHONE=2:
# trx firmware, L1CTL /tmp/osmocom_l2_3, no transceiver, no air) and the transceiver takes
# "-2 -L /tmp/osmocom_l2_3" (phone 2 = TS1 TX+RX, prim_bts.c SLOT 0 / tn 1). Its BSP (udp 6752) has no
# SB to align on: CALYPSO_BSP_TRX_OFFSET=-1 (firmware fn = tick - 1, measured on phone 1). The air
# (instance 2) also connects this ring (--c-ring/--c-port).
# OSMO_TRX_TS1=0 by default: the 2nd phone has an independent fn (downlink TCH ~80 bits wrong for the
# mobile, ASSIGNMENT FAILURE); only the ONE Calypso TS0+TS1 setup (OSMO_TRX_TCH=1) completes a call.
# start-native.sh --ts 2 (or OSMO_TRX_TS1=1) for the two-phone setup.
OSMO_TRX_TS1="${OSMO_TRX_TS1:-0}"
OSMO_TRX_TS1_INSTANCE="${OSMO_TRX_TS1_INSTANCE:-3}"
OSMO_TRX_PHONE="${OSMO_TRX_PHONE:-1}"
# The mobile over the air (INSTANCE=N, AIR=1) is never the TRX: start-native.sh --bts exports OSMO_TRX=1
# and instance 2 must not inherit it (trx firmware + second transceiver -> "UDP port 5700 already taken").
if [ "$OSMO_TRX" = 1 ] && [ "$AIR" = 1 ] && [ "$INSTANCE" != 1 ] && [ "$OSMO_TRX_PHONE" != 2 ]; then
    printf '\033[1m[run dsp]\033[0m INSTANCE=%s AIR=1 : OSMO_TRX=1 herite ignore, ce Calypso est le mobile (le TRX
↪    est l instance 1)\n' "$INSTANCE"
    OSMO_TRX=0
fi
if [ "$OSMO_TRX" = 1 ]; then
    # The trx firmware OVERRIDES: /etc/default/osmo-banc (GPRS profile) sets FIRMWARE_ELF=layer1.highram.gprs
    # in the environment, and a "${FIRMWARE_ELF:-trx}" would let it through: the transceiver would speak
    # BTS_MODE (0x1f) to a layer1 that does not know it. OSMO_TRX_FIRMWARE=<elf> for another trx.
```

```
    [ -n "${FIRMWARE_ELF:-}" ] && [ "${FIRMWARE_ELF##*/}" != "${OSMO_TRX_FIRMWARE:-trx.highram.elf}" ] \
        && printf '\033[1m[run dsp]\033[0m OSMO_TRX=1 : firmware %s ignore, trx.highram.elf a la place\n'
         ↪  "$FIRMWARE_ELF"
    FIRMWARE_ELF="${OSMO_TRX_FIRMWARE:-$_FW/trx.highram.elf}"
    FIRMWARE_BIN="${FIRMWARE_ELF%.elf}.bin"
    # BSIC of the cell: osmo-bts-trx only sends SETTSC (BCC), the NCC comes from bts 0 of the BSC
    OSMO_TRX_BSIC="${OSMO_TRX_BSIC:-$(awk '/^ *bts 0$/{f=1} f&&/base_station_id_code/{print $2; exit}'
     ↪  /etc/osmocom/osmo-bsc.cfg 2>/dev/null)}"
    TRANSCEIVER="${TRANSCEIVER:-/opt/GSM/osmocom-bb-transceiver/src/host/layer23/src/transceiver/transceiver}"
    [ -x "$TRANSCEIVER" ] || TRANSCEIVER="$(command -v transceiver || echo "$TRANSCEIVER")"
    # OSMO_TRX_TCH=1 (default): the phone serves TS0 AND TS1 (TCH/F, the calls): masks 0x03, the patched
    # firmware transmits both slots of its TX window (prim_bts.c) and reports TS1 from rx[2]; c54x_exe
    # publishes both bursts from the page (CALYPSO_TRX_TX_SLOTS=2) and writes the TS1 uplink into the
    # page. OSMO_TRX_TCH=0: TS0 only (0x01).
    # with phone 2 (OSMO_TRX_TS1=1) phone 1 keeps TS0 only: OSMO_TRX_TCH=0
    OSMO_TRX_TCH="${OSMO_TRX_TCH:-$([ "$OSMO_TRX_TS1" = 1 ] && echo 0 || echo 1)}"
    if [ "$OSMO_TRX_TCH" = 1 ] && [ "$OSMO_TRX_PHONE" != 2 ]; then
        OSMO_TRX_TS_MASK="${OSMO_TRX_TS_MASK:-0x03}"
        OSMO_TRX_TX_MASK="${OSMO_TRX_TX_MASK:-0x03}"
        export CALYPSO_TRX_TX_SLOTS="${CALYPSO_TRX_TX_SLOTS:-2}"
    fi
    OSMO_TRX_TS_MASK="${OSMO_TRX_TS_MASK:-0x01}"
    # TX mask 0x01: ONE TX window (tx_start 0, TPU slot 5 = downlink TS0), the one the ROM plays as a
    # RACH and that c54x_exe captures (TSP script 0x3cbb). The original 0xe3 (multiple window 5,6,7,0,1)
    # is not served by the bench: only the first burst (dummy) would go out. The fixed trx firmware
    # (prim_bts.c, SLOT/tn) targets TS0 whatever tx_start.
    OSMO_TRX_TX_MASK="${OSMO_TRX_TX_MASK:-0x01}"
    # -r PRIO (SCHED_RR, main.c rt_prio; OSMO_TRX_RT= empty to do without); -e 1 for debugging (-e 3 by default)
    OSMO_TRX_RT="${OSMO_TRX_RT-20}"
    OSMO_TRX_ARGS="${OSMO_TRX_ARGS:---no-sync -e 3}${OSMO_TRX_RT:+ -r $OSMO_TRX_RT}"
    PONT=0
    # the BTS repeats its bursts (dummy, FB): c54x_exe publishes every frame with a TX task, not just new ones
    export MONTANT_ROM_TX_CHAQUE="${MONTANT_ROM_TX_CHAQUE:-1}"
    # the BSP plays the uplink bursts (air.py, GSM fn of the mobile) at the trx firmware's frame, aligned on
    # the fn of the transmitted SB (pont.c -> calypso_bsp_trx_calage); CALYPSO_BSP_TRX_DELTA adjusts (default -2
    # with the patched firmware BTS_RX_LAG 3: uplink burst fn X is played one frame after the native window,
    # margin for the air)
    export CALYPSO_BSP_TRX="${CALYPSO_BSP_TRX:-1}"
    # trx firmware BTS_RX_LAG 12 (prim_bts.c): uplink burst fn X is played 11 frames after its native
    # window; margin for the round-trip latency of the air + the lag of the emulated mobile
    export CALYPSO_BSP_TRX_DELTA="${CALYPSO_BSP_TRX_DELTA:--11}"
    # phone 2 (TS1): no SB transmitted, fixed fn/tick offset (see OSMO_TRX_TS1)
    [ "$OSMO_TRX_PHONE" = 2 ] && export CALYPSO_BSP_TRX_OFFSET="${CALYPSO_BSP_TRX_OFFSET:--1}"
fi
FIRMWARE_ELF="${FIRMWARE_ELF:-$_FW/layer1.highram.elf}"
FIRMWARE_BIN="${FIRMWARE_BIN:-${FIRMWARE_ELF%.elf}.bin}"
# FIRMWARE_FLASH=<4 MiB image>: a REAL firmware in flash (FreeCalypso Magnetite, c139 target:
# Compal bootloader in sector 0 + fwimage.bin at 0x10000, see osmo-operator/scripts/magnetite-c139.sh
# and doc/magnetite.md) instead of the osmocom-bb layer1. QEMU starts at 0 in the flash (no -kernel,
# CPU ti925t = ARMv4T like the ARM7TDMI), the screen is the C139's (CALYPSO_LCD=c139, uWire), the
# L23 stack and the UI are INSIDE the firmware: no osmocon (rvinterf decodes the RVTMUX traces on
# the modem pty, socket $RUNDIR/rvinterf.sock for fc-shell / fc-fsio), no mobile. The DSP (step 1)
# and the air or the bridge (step 5) are unchanged. start-native.sh --ms-magnetite sets it.
FIRMWARE_FLASH="${FIRMWARE_FLASH:-}"
# the TRX of the BTS (OSMO_TRX=1) or phone 2 of the TRX are NEVER Magnetite
if [ -n "$FIRMWARE_FLASH" ] && { [ "$OSMO_TRX" = 1 ] || [ "$OSMO_TRX_PHONE" = 2 ]; }; then
    printf '\033[1m[run dsp]\033[0m OSMO_TRX=1 : FIRMWARE_FLASH=%s ignore, cette instance est le TRX de la BTS
     ↪  (firmware trx)\n' "$FIRMWARE_FLASH"
    FIRMWARE_FLASH=""
fi
# Magnetite (real TI firmware in flash): no pacer catch-up. With catch-up (20), the ARM chains the
# late frames, the L1 saturates and the watchdog resets the firmware (freeze).
if [ -n "$FIRMWARE_FLASH" ]; then
    export CALYPSO_PACER_RATTRAPAGE="${CALYPSO_PACER_RATTRAPAGE:-0}"
    export CALYPSO_PONT_ARM_LOCK="${CALYPSO_PONT_ARM_LOCK:-1}"
    # TI L1: SB window opened at the end of the previous frame (otherwise BCCH shifted by one frame,
```

```
    # CRC KO on every block) and NB TOA targeted at 3 (margin 1); see calypso_bsp.c sb_fin_trame()
    export CALYPSO_BSP_SB_FIN_TRAME="${CALYPSO_BSP_SB_FIN_TRAME:-1}"
    export CALYPSO_BSP_NB_MARGE="${CALYPSO_BSP_NB_MARGE:-1}"
    # NB TOA canned at the TI target (pont.c). Off by default: it requires a c54x_exe built with the
    # init_done guard, without which the canning overwrites the DSP code during the boot.
    export PONT_CAN_NB_TOA="${PONT_CAN_NB_TOA:-}"
    # fake air: noise outside the cell's ARFCN (calypso_bsp.c faux_air_*), ARFCN of the radio plan
    # SB modulated like FB/NB: the SB settings made for the osmocom layer1 (instant 0.35, spread 1.0,
    # noise 3000), which ignores the SB angle, bias the measured angle (+166 at 0 Hz); the TI L1 uses
    # it for the AFC (-700 -> -673, ~224 Hz of real offset) and would lose the SB then the BCCH
    export CALYPSO_BSP_SB_DEC="${CALYPSO_BSP_SB_DEC:-0.5}" CALYPSO_BSP_SB_SYM="${CALYPSO_BSP_SB_SYM:-0.3}"
     ↪  CALYPSO_BSP_SB_NOISE="${CALYPSO_BSP_SB_NOISE:-0}"
    export CALYPSO_BSP_BRUIT_FOND="${CALYPSO_BSP_BRUIT_FOND:-1}"   # noise instead of zeros (PM = 0 -> L1 recovery)
    # catch up with the BTS live feed (calypso_bsp.c): OVER THE AIR ONLY (AIR=1, the BTS has its own clock).
    # With a classic BTS pont_dsp.py paces osmo-bts on the frames delivered by the DSP: each +33 jump would
    # speed the BTS up (bridge at ~350 frames/s), hence more lag and more jumps, and IMM ASS never matched.
    export CALYPSO_BSP_RATTRAPE="${CALYPSO_BSP_RATTRAPE:-$AIR}"
    export CALYPSO_RACH_VERS_AIR="${CALYPSO_RACH_VERS_AIR:-1}"   # RACH of the TI firmware published to the air
     ↪  (pont.c)
    # classic BTS: the SDCCH/SACCH bursts of the TI firmware are in calypso_tx_rom (TSC 7, fn of the ROM),
    # but pont_dsp would drop them for lack of an announced dedicated channel (calypso_dcch_cfg comes from
    # the QEMU L1CTL tap, absent without the osmocom layer1). PONT_UL_TN_DEFAUT=0: TS0 (SDCCH/4 of the
    # bench) when nothing is announced. PONT_UL_RAW_DCCH_HOTE=1 (host coding) remains available.
    export PONT_UL_TN_DEFAUT="${PONT_UL_TN_DEFAUT:-0}"
    # uplink TCH by the HOST (FACCH, SACCH/T, speech coded by the bridge from a_fu/a_du): the ROM's TSP
    # script only yields 2 "unknown" bursts on the TI firmware's TCH, so the FACCH SABM would never reach
    # the BTS. 14-demarrage.sh exports PONT_UL_RAW_TCH=1 for the ROM vocoder of the osmocom mobile: forced
    # to 0 here (PONT_UL_RAW_TCH_MAGNETITE to override).
    export PONT_UL_RAW_TCH="${PONT_UL_RAW_TCH_MAGNETITE:-0}"
    # NO fn or RA correction (AIR_B_FN_DELTA, MONTANT_RACH_DIFFERE: 0). The TI L1's RA_R traces the REQUEST
    # (MPHC_RA_REQ); the RACH leaves after the random draw, just before RA_C, and c54x_exe publishes the
    # same ra at the RA_C fn: fn and RA are right without correction.
    export AIR_B_FN_DELTA="${AIR_B_FN_DELTA:-0}"
    export MONTANT_RACH_DIFFERE="${MONTANT_RACH_DIFFERE:-0}"
    export CALYPSO_FAUX_AIR="${CALYPSO_FAUX_AIR:-$( . /etc/osmocom/radio-plan.env 2>/dev/null; echo
     ↪  "${PLAN_ARFCN:-}")}"
    [ -r "$FIRMWARE_FLASH" ] || { printf '\033[1;31m[run] ECHEC :\033[0m image flash absente : %s\n'
     ↪  "$FIRMWARE_FLASH" >&2; exit 1; }
    QEMU_CPU="${QEMU_CPU:-ti925t}"
    export CALYPSO_LCD="${CALYPSO_LCD:-c139}"
    RVINTERF_BIN="${RVINTERF_BIN:-/opt/GSM/fc-host-tools/rvinterf/lowlevel/rvinterf}"
fi
OSMOCON="${OSMOCON:-/opt/GSM/osmocom-bb/src/host/osmocon/osmocon}"
MOBILE="${MOBILE:-$(command -v mobile || echo /usr/local/bin/mobile)}"
MOBILE_CFG="${MOBILE_CFG:-$HERE/mobile_pont.cfg}"
# Two entry points for the same pont/ package: pont_dsp.py for the dsp setup
# (--dsp-port 6702), pont.py for the grgsm setup, which keeps its own
# defaults. PONT_PY forces one or the other.
if [ "$MODE" = grgsm ]; then
    PONT_PY="${PONT_PY:-/opt/GSM/osmo-operator/pont/pont.py}"
else
    PONT_PY="${PONT_PY:-/opt/GSM/osmo-operator/pont/pont_dsp.py}"
fi
RUNDIR="${RUNDIR:-/tmp/c54x-pont}"
# REAL TIME. Under load (test campaign, panel, ffmpeg) the chain c54x_exe <-> QEMU <-> osmocon/mobile
# <-> bridge misses ticks and the sound cuts out: the frame is 4.615 ms, the work takes ~1.2 ms
# ([chrono]: A 0.33 + go 0.40 + B 0.16 + qemu <0.32), and the CFS scheduler readily sets a process
# aside for 5 to 10 ms when the cores are busy. The five processes of the chain are therefore
# launched with a priority: RT_NICE=-10 by default (RT_NICE= empty to do nothing), RT_FIFO=<prio> for
# SCHED_FIFO (chrt -f, root; stay below 50, the kernel threads are above), RT_CPUS=<list> to pin them
# (taskset -c, e.g. 1-3: core 0 stays for the rest of the system). RT_RENICE_AUTRES=<n> additionally
# lowers the panel tools (osmo-fft-snap, osmo-panel, osmo-topzone, osmo-ts-probe, conky) to nice n.
# ./run.sh --charge shows who consumes what.
RT_NICE="${RT_NICE--10}"
RT_FIFO="${RT_FIFO:-}"
RT_CPUS="${RT_CPUS:-}"
```

```
RT_RENICE_AUTRES="${RT_RENICE_AUTRES:-}"
PRIO=()
[ -n "$RT_CPUS" ] && PRIO+=(taskset -c "$RT_CPUS")
if [ -n "$RT_FIFO" ]; then PRIO+=(chrt -f "$RT_FIFO"); elif [ -n "$RT_NICE" ]; then PRIO+=(nice -n "$RT_NICE"); fi
renice_autres() {
    [ -n "$RT_RENICE_AUTRES" ] || return 0
    local pids; pids="$(pgrep -f 'osmo-fft-snap|osmo-panel|osmo-topzone|osmo-ts-probe|conky' | tr '\n' ' ')"
    [ -n "$pids" ] && renice -n "$RT_RENICE_AUTRES" -p $pids >/dev/null 2>&1 && dire "   outils du panneau a nice
     ↪  $RT_RENICE_AUTRES : $pids"
    return 0
}
charge() {
    echo "chaine (pid %cpu nice politique) :"
    local n
    for n in dsp qemu osmocon rvinterf mobile transceiver pont air; do
        if vivant "$n"; then printf '  %-8s %s\n' "$n" "$(ps -o pid=,pcpu=,ni=,cls= -p "$(pid_de "$n")" | tr -s '
         ↪  ')"; else printf '  %-8s arrete\n' "$n"; fi
    done
    echo "processus les plus gourmands :"
    ps -eo pid,pcpu,ni,cls,comm --sort=-pcpu | head -12 | sed 's/^/  /'
    echo "charge moyenne $(cut -d' ' -f1-3 /proc/loadavg) sur $(nproc) coeurs ; priorite de la chaine :
     ↪  ${PRIO[*]:-aucune}"
}
L2_SOCK="${L2_SOCK:-/tmp/osmocom_l2}"
MONITOR="${MONITOR:-/tmp/qemu-monitor-pont.sock}"
GDB="${GDB:-1}"                              # QEMU gdbstub + telnet console (step 2)
GDB_STUB="${GDB_STUB:-1234}"
GDB_TELNET="${GDB_TELNET:-44444}"
GDB_TELNET_PY="${GDB_TELNET_PY:-$QOSMO/tools/gdb-telnet.py}"
# 120000 by default: the budget of the TRX's DSP (instance 1, booted at the first IDLE); a smaller
# budget (16000) leaves the DSP of the Magnetite instance in "occupe boot" forever, and a TCH
# frame needs up to 87000 insn/frame (see start-native.sh).
INSNS="${INSNS:-120000}"
# DSP/QEMU lockstep by default (LOCKSTEP=0 for wall-clock mode): the emulated C54x costs
# ~6.7 ms per frame against 4.615 ms of real time, so on the wall clock QEMU skips 3 frames
# out of 4 ("DSP en retard, tick saute"), the ROM only sees one frame in 4 to 6, its FB
# block counter does not advance and the TOA reads 1251 whatever the distance of the FCCH.
# In lockstep QEMU only advances the frame once the DSP has finished the previous one:
# TOA = 23 + n x 1250, delay=10, SB decodable.
LOCKSTEP="${LOCKSTEP:-1}"
[ "$MODE" = dsp ] && [ "$LOCKSTEP" = 1 ] && export CALYPSO_PONT_LOCKSTEP=1
VERB="${VERB:--v}"
IQ="${IQ:-none}"
AMP="${AMP:-30000}"
DSP_SOCK="${DSP_SOCK:-/tmp/calypso_dsp.sock}"
DSP_SHM=/dev/shm/calypso_api_ram

# THE SHOWCASE: THE SAME LOGS AS IN SHUNT MODE. The panel (tmux calypso,
# osmo-fft-snap) follows /run/user/0/osmo-nitb/logs/{qemu,osmocon,mobile}.log
# and /dev/shm/pont.log, those written by the grgsm setup. The REAL file is
# therefore placed at the panel's path and $RUNDIR/<name>.log becomes a link to
# it -- not the other way round: an already open `tail -F` refuses a file
# replaced by a link ("untailable symbolic link"). PANNEAU_LOGS=none to touch
# nothing.
PANNEAU_LOGS="${PANNEAU_LOGS:-/run/user/0/osmo-nitb/logs}"
# The panel's FFT reads /tmp/iq_fft_ms.fifo, which pont.py only fills when
# recording is active: --no-record would silence it. PONT_AIRREC=0 turns it off.
PONT_AIRREC="${PONT_AIRREC:-1}"
vitrine() {   # vitrine <name>: to be called BEFORE the launch that writes $RUNDIR/<name>.log
    rm -f "$RUNDIR/$1.log"
    [ "$PANNEAU_LOGS" = none ] && return 0
    local cible="$PANNEAU_LOGS/$1.log"
    [ "$1" = pont ] && cible=/dev/shm/pont.log
    [ -d "$(dirname "$cible")" ] || return 0
    rm -f "$cible" && : > "$cible" && ln -s "$cible" "$RUNDIR/$1.log"
    return 0
```

```
}

dire()  { printf '\033[1m[run %s]\033[0m %s\n' "$MODE" "$*"; }
rater() { printf '\033[1;31m[run] ECHEC :\033[0m %s\n' "$*" >&2; exit 1; }
pid_de() { [ -f "$RUNDIR/$1.pid" ] && cat "$RUNDIR/$1.pid"; }
vivant() { local p; p="$(pid_de "$1")"; [ -n "$p" ] && kill -0 "$p" 2>/dev/null; }
attendre() { local n=$(( $1 * 10 )); shift; while [ "$n" -gt 0 ]; do "$@" && return 0; sleep 0.1; n=$((n - 1));
 ↪  done; return 1; }
[ "$MODE" = dsp ] || [ "$MODE" = grgsm ] || rater "MODE=$MODE inconnu (dsp|grgsm)"

etape1() {   # the DSP (dsp setup only)
    renice_autres
    [ "$MODE" = dsp ] || { dire "1. (montage grgsm : pas de c54x_exe, la couche 1 est dans QEMU)"; return; }
    vivant dsp && { dire "1. c54x_exe deja lance (pid $(pid_de dsp))"; return; }
    [ -x "$HERE/c54x_exe" ] || make -C "$HERE" >/dev/null || rater "make c54x_exe"
    rm -f "$DSP_SHM" "$DSP_SOCK"
    # The TCH announcement of a dead session (pont/dsp/tch.py) must not be
    # read again by montant.c at startup.
    rm -f /dev/shm/calypso_tch_cfg
    # Wait (at most 40 ms) for a frame the BTS delivers late rather than
    # playing it as an erasure (calypso_bsp.c, bsp_attendre_trame).
    ( cd "$HERE" && bruit_env_dsp && CALYPSO_IQDUMP_FCCH=1 CALYPSO_BSP_ATTENTE_MS="${CALYPSO_BSP_ATTENTE_MS:-40}"
     ↪  exec ${PRIO[@]+"${PRIO[@]}"} "${C54X_BIN:-./c54x_exe}" --arm "$DSP_SOCK" --insns "$INSNS" --iq "$IQ"
     ↪  --amp "$AMP" $VERB ) > "$RUNDIR/dsp.log" 2>&1 &
    echo $! > "$RUNDIR/dsp.pid"
    attendre 5 test -S "$DSP_SOCK" || rater "c54x_exe n'a pas ouvert $DSP_SOCK (voir $RUNDIR/dsp.log)"
    # shm at 0666: the umask lowers c54x_exe's 0666 request to 0644, which prevents a harvard.sh
    # run under the user account from writing the API RAM (d@!, sb/si3 demos). Forced here.
    attendre 3 test -e "$DSP_SHM" && chmod 0666 "$DSP_SHM" 2>/dev/null || true
    dire "1. c54x_exe --arm  pid $(pid_de dsp)  ($INSNS insn/trame, iq=$IQ, $DSP_SHM, $DSP_SOCK)"; bruit_dire_dsp
}

etape2() {   # the ARM
    vivant qemu && { dire "2. QEMU deja lance (pid $(pid_de qemu))"; return; }
    [ -x "$QEMU" ] || rater "QEMU absent : $QEMU (ninja -C $QOSMO/build)"
    [ -r "$FIRMWARE_ELF" ] || rater "firmware absent : $FIRMWARE_ELF"
    local extern=""
    if [ "$MODE" = dsp ]; then [ -S "$DSP_SOCK" ] || rater "le DSP ne tourne pas (etape 1 d'abord)";
     ↪  extern="$DSP_SOCK"; fi
    rm -f "$MONITOR"
    vitrine qemu
    # gdb on the ARM: QEMU gdbstub on tcp::$GDB_STUB, and the qosmo telnet
    # console (tools/gdb-telnet.py) on port $GDB_TELNET:
    #     telnet 0 44444      (Ctrl-C stops the ARM, "continue &" resumes it)
    # As long as nobody is connected, gdb does not run and the ARM is not
    # touched. GDB=0: neither stub nor console.
    local gdb_opt=()
    [ "$GDB" = 1 ] && gdb_opt=(-gdb "tcp:127.0.0.1:$GDB_STUB")
    # ASSEMBLY_LOGS=1 (start-native.sh --assembly-logs): asm trace of the ARM in
    # qemu-asm.log, each block once, at its translation. in_asm only by default.
    # ASSEMBLY_LOGS_FLAGS=in_asm,exec,nochain also logs every executed block
    # (nochain, otherwise chained blocks are only seen once): the ARM then takes
    # ~62 ms per frame instead of 0.4 ([chrono] go); the DSP in lockstep follows,
    # but the bridge's slaved clock (osmo-operator/pont/trx.py, ceiling
    # 4 x FRAME_DUR) runs thousands of frames ahead, the BSP no longer finds the
    # bursts and mobile sees no cell (FBSB result=255).
    # THE TRACE GOES TO THE DISK, NOT TO THE TMPFS. $RUNDIR is in /tmp (4.7 GB
    # tmpfs): with exec,nochain the trace fills /tmp within minutes, and an rm
    # frees nothing as long as QEMU keeps the file open (/proc/<pid>/fd/N must be
    # truncated). Written to ASSEMBLY_LOGS_DIR (/var/tmp/c54x-pont, on the disk)
    # and $RUNDIR/qemu-asm.log becomes a link; the archives leave it alone
    # (garder_journaux).
    if [ "${ASSEMBLY_LOGS:-0}" = 1 ]; then
        local asm_dir="${ASSEMBLY_LOGS_DIR:-/var/tmp/c54x-pont}"
        mkdir -p "$asm_dir" 2>/dev/null || asm_dir="$RUNDIR"
        rm -f "$RUNDIR/qemu-asm.log" "$asm_dir/qemu-asm.log"
```

```
    [ "$asm_dir" != "$RUNDIR" ] && ln -s "$asm_dir/qemu-asm.log" "$RUNDIR/qemu-asm.log"
    local asm_cible="$asm_dir/qemu-asm.log"
    # ASSEMBLY_LOGS_FIFO=1: QEMU writes into a FIFO on /dev/shm and a cat drains
    # it to the file on the disk. qemu_log_unlock() does an fflush() on EVERY
    # line (util/log.c): with exec,nochain that is one write(2) per executed
    # block (~830 per frame), about ten us on the disk against 1 to 2 us on a
    # pipe. The reader is launched BEFORE QEMU: its fopen(fifo, "w") blocks as
    # long as nobody reads.
    if [ "${ASSEMBLY_LOGS_FIFO:-0}" = 1 ]; then
        local asm_fifo="/dev/shm/qemu-asm.fifo"
        rm -f "$asm_fifo"; mkfifo "$asm_fifo"
        ${PRIO[@]+"${PRIO[@]}"} cat "$asm_fifo" > "$asm_cible" 2>/dev/null &
        echo $! > "$RUNDIR/asmcat.pid"
        asm_cible="$asm_fifo"
    fi
    gdb_opt+=(-d "${ASSEMBLY_LOGS_FLAGS:-in_asm}" -D "$asm_cible")
    [ -n "${ASSEMBLY_LOGS_FILTRE:-}" ] && gdb_opt+=(-dfilter "$ASSEMBLY_LOGS_FILTRE")
    # QEMU_LOG_BUFFERED (qosmo's util/log.c): fully buffered -D file, no more
    # fflush() per line. 4 MiB by default when tracing.
    export QEMU_LOG_BUFFERED="${QEMU_LOG_BUFFERED:-4194304}"
    # RING OF EXECUTED BLOCKS, BY DEFAULT WITH ASSEMBLY_LOGS=1. The TCG plugin tbring
    # (qosmo/tools/qosmo-tbring) pushes the PC of every EXECUTED block, chained blocks
    # included, into an mmap ring: 8 bytes per block, no text in the loop, no nochain,
    # the ARM runs as in production (-d exec,nochain: 150 MB/s of text, 20 GB in 130 s).
    # 64 GiB ring = 2^33 blocks, i.e. ~12 h of ARM at 180 000 blocks/s; SPARSE file on
    # the disk ($asm_dir, not the tmpfs), it only weighs what has been written. Read
    # back live, at any time, without stopping QEMU:
    #     tbring-dump -n 500 -f $asm_dir/qemu-tbring -e $FIRMWARE_ELF -a $asm_dir/qemu-asm.log
    # (the in_asm trace, one block per translation, serves as disassembly table).
    # ASSEMBLY_LOGS_RING=0 turns it off; ASSEMBLY_LOGS_RING_ENTRIES (power of 2) and
    # ASSEMBLY_LOGS_RING_FILE change the size and the location.
    if [ "${ASSEMBLY_LOGS_RING:-1}" = 1 ]; then
        local tbring="${TBRING_SO:-$QOSMO/tools/qosmo-tbring/libtbring.so}"
        [ -f "$tbring" ] || tbring=/usr/local/lib/qosmo/libtbring.so
        if [ -f "$tbring" ]; then
            local ring="${ASSEMBLY_LOGS_RING_FILE:-$asm_dir/qemu-tbring}"
            rm -f "$ring" "$RUNDIR/qemu-tbring"
            [ "$(dirname "$ring")" != "$RUNDIR" ] && ln -s "$ring" "$RUNDIR/qemu-tbring"
            # anneau= and not file=: file= is QEMU's keyword for the plugin path
            gdb_opt+=(-plugin "$tbring,anneau=$ring,entries=${ASSEMBLY_LOGS_RING_ENTRIES:-8589934592}")
        else
            dire "   anneau tbring ignore : libtbring.so introuvable (make -C $QOSMO/tools/qosmo-tbring)"
        fi
    fi
fi
# QEMU_DEBUG_FLAGS=int,guest_errors: log of the ARM's exceptions (PC of the Data
# Abort, access to an unmapped address) in $RUNDIR/qemu-debug.log. Without
# ASSEMBLY_LOGS only (a single -d/-D per QEMU command line).
if [ -n "${QEMU_DEBUG_FLAGS:-}" ] && [ "${ASSEMBLY_LOGS:-0}" != 1 ]; then
    gdb_opt+=(-d "$QEMU_DEBUG_FLAGS" -D "$RUNDIR/qemu-debug.log")
fi
# QEMU_ICOUNT=auto|shift=N,... (start-native.sh --icount): -icount.
case "${QEMU_ICOUNT:-}" in ''|0|off) ;; *) gdb_opt+=(-icount "$QEMU_ICOUNT") ;; esac
# QEMU polls the DSP every frame/64 (0.07 ms) instead of frame/16
# (0.29 ms): the serial part of a frame (DONE of phase A, GO) pays this
# latency twice -- [chrono] measurement on TCH.
CALYPSO_PONT_RETRY_DIV="${CALYPSO_PONT_RETRY_DIV:-64}" \
# FIRMWARE_FLASH: the flash (working copy in $RUNDIR, kept between two launches for the
# FFS; FLASH_RESET=1 or a newer image replace it) replaces -kernel
local fw_opt=(-kernel "$FIRMWARE_ELF")
if [ -n "$FIRMWARE_FLASH" ]; then
    if [ "${FLASH_RESET:-0}" != 0 ] || [ ! -f "$RUNDIR/flash.img" ] || [ "$FIRMWARE_FLASH" -nt
     ↪  "$RUNDIR/flash.img" ]; then
        cp "$FIRMWARE_FLASH" "$RUNDIR/flash.img" || rater "copie de $FIRMWARE_FLASH impossible"
        dire "   flash de travail $RUNDIR/flash.img (re)creee depuis $FIRMWARE_FLASH"
    fi
```

```
        fw_opt=(-drive "if=pflash,file=$RUNDIR/flash.img,format=raw")
    fi
    CALYPSO_DSP_EXTERN="$extern" ${PRIO[@]+"${PRIO[@]}"} "$QEMU" -M calypso -cpu "${QEMU_CPU:-arm946}" -display
     ↪  none -parallel none \
        -serial pty -serial pty -monitor "unix:$MONITOR,server,nowait" "${gdb_opt[@]}" \
        "${fw_opt[@]}" > "$RUNDIR/qemu.log" 2>&1 &
    echo $! > "$RUNDIR/qemu.pid"
    if [ "$GDB" = 1 ] && [ -f "$GDB_TELNET_PY" ] && ! vivant gdb; then
        python3 "$GDB_TELNET_PY" --port "$GDB_TELNET" --stub "$GDB_STUB" --elf "$FIRMWARE_ELF" \
            > "$RUNDIR/gdb.log" 2>&1 &
        echo $! > "$RUNDIR/gdb.pid"
    fi
    attendre 10 grep -aq "label serial0" "$RUNDIR/qemu.log" || rater "QEMU n'a pas publie son pty (voir
     ↪  $RUNDIR/qemu.log)"
    if [ "$MODE" = dsp ]; then
        attendre 10 grep -aq "pont DSP : API RAM partagee" "$RUNDIR/qemu.log" || rater "QEMU n'a pas rejoint le
         ↪  DSP (voir $RUNDIR/qemu.log)"
    else
        attendre 5 grep -aq "backend gr-gsm" "$RUNDIR/qemu.log" || rater "la couche 1 gr-gsm ne s'est pas annoncee
         ↪  (voir $RUNDIR/qemu.log)"
    fi
    sed -n 's/.*redirected to \(\/dev\/pts\/[0-9]*\) (label serial0).*/\1/p' "$RUNDIR/qemu.log" | head -1 >
     ↪  "$RUNDIR/modem.pty"
    dire "2. qemu-system-arm  pid $(pid_de qemu)  pty modem $(cat "$RUNDIR/modem.pty")  moniteur $MONITOR  L1=$([
     ↪  "$MODE" = dsp ] && echo "DSP externe" || echo "gr-gsm (udp 4730/4731)")"
    [ "$GDB" = 1 ] && dire "   gdb ARM : telnet 0 $GDB_TELNET  (stub tcp::$GDB_STUB, journal $RUNDIR/gdb.log)"
    case "${QEMU_ICOUNT:-}" in ''|0|off) ;; *) dire "   icount : $QEMU_ICOUNT" ;; esac
    [ -n "${QEMU_DEBUG_FLAGS:-}" ] && [ "${ASSEMBLY_LOGS:-0}" != 1 ] && dire "   journal QEMU -d $QEMU_DEBUG_FLAGS
     ↪  : $RUNDIR/qemu-debug.log"
    # mailbox monitor (CALYPSO_MAILBOX=1): two files, one per side
    if [ -n "${CALYPSO_MAILBOX:-}" ] && [ "${CALYPSO_MAILBOX}" != 0 ]; then
        local _mb="${CALYPSO_MAILBOX_FILE:-${LOG_DIR:-/tmp/calypso/logs}/mailbox.log}"
        dire "   mailbox ARM<->DSP : ${_mb} (cote DSP, c54x_exe) et ${_mb%.log}-arm.log (cote ARM, QEMU) ;
         ↪  CALYPSO_MAILBOX=$CALYPSO_MAILBOX"
    fi
    [ "${ASSEMBLY_LOGS:-0}" = 1 ] && dire "   trace asm ARM : $(readlink -f "$RUNDIR/qemu-asm.log") (-d
     ↪  ${ASSEMBLY_LOGS_FLAGS:-in_asm}${ASSEMBLY_LOGS_FILTRE:+, -dfilter
     ↪  $ASSEMBLY_LOGS_FILTRE}${ASSEMBLY_LOGS_FIFO:+, via FIFO}, QEMU_LOG_BUFFERED=$QEMU_LOG_BUFFERED)$([
     ↪  "${ASSEMBLY_LOGS_RING:-1}" = 1 ] && echo " ; anneau des blocs executes : $(readlink -f
     ↪  "$RUNDIR/qemu-tbring" 2>/dev/null) (tbring-dump)")"
}

etape3_rvinterf() {   # FIRMWARE_FLASH: rvinterf (FreeCalypso) instead of osmocon
    vivant rvinterf && { dire "3. rvinterf deja lance (pid $(pid_de rvinterf))"; return; }
    [ -x "$RVINTERF_BIN" ] || { dire "3. (rvinterf absent : $RVINTERF_BIN ; pas de traces RVTMUX)"; return; }
    local pty; pty="$(cat "$RUNDIR/modem.pty")"
    rm -f "$RUNDIR/rvinterf.sock"
    "$RVINTERF_BIN" -n -l "$RUNDIR/rvinterf.log" -s "$RUNDIR/rvinterf.sock" "$pty" > "$RUNDIR/rvinterf.out" 2>&1 &
    echo $! > "$RUNDIR/rvinterf.pid"
    dire "3. rvinterf  pid $(pid_de rvinterf)  traces RVTMUX -> $RUNDIR/rvinterf.log  socket $RUNDIR/rvinterf.sock
     ↪  (fc-shell -s / fc-fsio -s)"
}
etape3() {   # osmocon
    [ -n "$FIRMWARE_FLASH" ] && { etape3_rvinterf; return; }
    vivant osmocon && { dire "3. osmocon deja lance (pid $(pid_de osmocon))"; return; }
    vivant qemu || rater "QEMU ne tourne pas (etape 2 d'abord)"
    [ -x "$OSMOCON" ] || rater "osmocon absent : $OSMOCON"
    [ -r "$FIRMWARE_BIN" ] || rater "image .bin absente : $FIRMWARE_BIN"
    local pty; pty="$(cat "$RUNDIR/modem.pty")"
    rm -f "$L2_SOCK"
    vitrine osmocon
    ${PRIO[@]+"${PRIO[@]}"} stdbuf -oL -eL "$OSMOCON" -m romload -i 100 -p "$pty" -s "$L2_SOCK" "$FIRMWARE_BIN" >
     ↪  "$RUNDIR/osmocon.log" 2>&1 &
    echo $! > "$RUNDIR/osmocon.pid"
    if ! attendre 30 grep -aq "your code is running now" "$RUNDIR/osmocon.log"; then
        rater "osmocon n'a pas fini le romload (voir $RUNDIR/osmocon.log). Un osmocon tue a mi-bloc laisse
```

```
       le stub romload de l'UART en attente : ./run.sh --stop puis ./run.sh"
    fi
    dire "3. osmocon  pid $(pid_de osmocon)  L1CTL sur $L2_SOCK"
}

# BANC_VOCODEUR=rom: the ROM vocodes (modelled voice port, CALYPSO_VSP=1).
# The mobile must then be in `io-handler l1phy` -- the only mode where the firmware
# lifts the vocoder mutes (prim_tch.c:101-111) -- and WITHOUT io-tch-format or ALSA in
# the tch-voice block: under l1phy the mobile refuses io-tch-format and drops the WHOLE
# file. The variant is therefore generated here from MOBILE_CFG, never written by hand.
config_vocodeur_rom() {
    local src="$1" dst="$RUNDIR/mobile_pont_rom.cfg"
    awk '
        /^ tch-voice/ { voix = 1; print; next }
        /^ [^ ]/      { voix = 0 }
        voix && /^  io-handler gapk/ { print "  io-handler l1phy"; next }
        voix && /^  (io-tch-format|alsa-output-dev|alsa-input-dev)/ { next }
        { print }' "$src" > "$dst" || rater "variante l1phy impossible : $dst"
    grep -q "^  io-handler l1phy" "$dst" || rater "variante l1phy : pas de bloc tch-voice gapk dans $src"
    echo "$dst"
}

etape4_trx() {   # OSMO_TRX=1: the host app transceiver, instead of the mobile
    [ "$OSMO_TRX_PHONE" = 2 ] && { dire "4. (OSMO_TRX_PHONE=2 : telephone 2 du transceiver de l'instance 1, TS1 ;
     ↪  pas de transceiver ici)"; return; }
    vivant transceiver && { dire "4. transceiver deja lance (pid $(pid_de transceiver))"; return; }
    [ -S "$L2_SOCK" ] || rater "pas de socket L1CTL $L2_SOCK (etape 3 d'abord)"
    local args_ts1=""
    if [ "$OSMO_TRX_TS1" = 1 ]; then
        local l2_3="/tmp/osmocom_l2_$OSMO_TRX_TS1_INSTANCE"
        if [ ! -S "$l2_3" ]; then
            dire "4a. telephone 2 (TS1) : instance $OSMO_TRX_TS1_INSTANCE (c54x_exe + QEMU trx + osmocon)"
            INSTANCE="$OSMO_TRX_TS1_INSTANCE" OSMO_TRX=1 OSMO_TRX_PHONE=2 AIR=0 PONT=0 bash "$0" || rater
             ↪  "l'instance $OSMO_TRX_TS1_INSTANCE (telephone 2) n'a pas demarre"
        fi
        attendre 10 test -S "$l2_3" || rater "pas de socket L1CTL $l2_3 pour le telephone 2"
        args_ts1="-2 -L $l2_3"
    fi
    [ -x "$TRANSCEIVER" ] || rater "transceiver absent : $TRANSCEIVER (make -C
     ↪  /opt/GSM/osmocom-bb-transceiver/src/host/layer23/src/transceiver)"
    local p
    # LOCAL column of ss ($4; $5 is the peer), all states: with "-uan | grep" the peer column would match
    # too (osmo-bts-trx 5800 -> 127.0.0.1:5700 would pass for a holder), and "-uln" does not see the
    # transceiver's sockets, which are CONNECTED (ESTAB) to the BTS, not listening.
    for p in 5700 5701 5702; do
        ss -uanH 2>/dev/null | awk -v p="$p" '$4 ~ ("[:.]" p "$") {found=1} END {exit !found}' \
            && rater "port UDP $p deja pris par $(ss -uanpH 2>/dev/null | awk -v p="$p" '$4 ~ ("[:.]" p "$")' |
             ↪  grep -o 'users:.*' | head -1) (un pont.py qui traine ? ./run.sh --stop)"
    done
    vitrine transceiver
    # -l: osmocon's L1CTL socket; -m: RX timeslot mask (a single one to begin with); the TX mask
    # keeps its default 0xe3 (OSMO_TRX_TX_MASK to force it: -M), see the header
    ${PRIO[@]+"${PRIO[@]}"} stdbuf -oL -eL "$TRANSCEIVER" -l "$L2_SOCK" -m "$OSMO_TRX_TS_MASK"
     ↪  ${OSMO_TRX_TX_MASK:+-M "$OSMO_TRX_TX_MASK"} ${OSMO_TRX_BSIC:+-b "$OSMO_TRX_BSIC"} $args_ts1
     ↪  $OSMO_TRX_ARGS > "$RUNDIR/transceiver.log" 2>&1 &
    echo $! > "$RUNDIR/transceiver.pid"
    attendre 10 sh -c "ss -uanH 2>/dev/null | awk '\$4 ~ /[:.]5702\$/ {f=1} END {exit !f}'" || rater "transceiver
     ↪  n'ecoute pas en 5700-5702 (voir $RUNDIR/transceiver.log)"
    vivant transceiver || rater "transceiver s'est arrete : $(tail -1 "$RUNDIR/transceiver.log")"
    # osmo-bts-trx (BTS#0) EXITS when the transceiver disappears ("No more clock from
    # transceiver", bts_shutdown exit 1): after a --stop the BTS is gone, and the fork's plan
    # (qosmo/run.sh --only bts) refuses to restart it alone (bsc dependency). It is restarted here
    # like the 60-bts.sh module, in the panel's log. OSMO_TRX_BTS_RELANCE=0 to do nothing.
    if [ "${OSMO_TRX_BTS_RELANCE:-1}" = 1 ]; then
        local bts_cfg="${BTS_CFG:-/etc/osmocom/osmo-bts-trx.cfg}" bts_bin bts_log bts_pid _i
        # The previous BTS may still be shutting down (bts_shutdown: up to 70 s measured):
```

```
        # wait until it is gone, or until it has switched the firmware on (POWERON -> "BTS MODE: 7 514").
        for _i in $(seq 1 "${OSMO_TRX_BTS_ATTENTE:-90}"); do
            pgrep -f -- "osmo-bts-trx -c $bts_cfg\$" >/dev/null 2>&1 || break
            grep -aq 'BTS MODE: [0-9]* [1-9]' "$RUNDIR/osmocon.log" 2>/dev/null && break
            [ "$_i" = 1 ] && dire "   BTS#0 presente mais muette : on attend qu'elle allume le firmware ou qu'elle
             ↪  finisse de s'eteindre (${OSMO_TRX_BTS_ATTENTE:-90} s max)"
            sleep 1
        done
        if ! pgrep -f -- "osmo-bts-trx -c $bts_cfg\$" >/dev/null 2>&1; then
            bts_bin="${OSMO_BTS_TRX_BIN:-$(command -v osmo-bts-trx || echo /usr/local/bin/osmo-bts-trx)}"
            bts_log="$(ls -d /run/user/*/osmo-nitb/logs 2>/dev/null | head -1)";
             ↪  bts_log="${bts_log:-$RUNDIR}/bts.log"
            if [ -x "$bts_bin" ] && [ -r "$bts_cfg" ]; then
               setsid stdbuf -oL -eL $(chrt -r 1 true 2>/dev/null && echo "chrt -r 1") "$bts_bin" -c "$bts_cfg" >>
                ↪  "$bts_log" 2>&1 </dev/null &
                bts_pid=$!
                for bts_piddir in /run/user/*/osmo-nitb; do [ -d "$bts_piddir" ] && echo "$bts_pid" >
                 ↪  "$bts_piddir/bts.pid"; done
                dire "4b. osmo-bts-trx relancee (BTS#0 absente : elle sort quand le transceiver disparait)  pid
                 ↪  $bts_pid  $bts_cfg  journal $bts_log"
            else
                dire "4b. BTS#0 absente et non relancable (binaire $bts_bin ou config $bts_cfg) :
                 ↪  ./start-native.sh --osmo-trx"
            fi
        fi
    fi
    dire "4. transceiver  pid $(pid_de transceiver)  L1CTL $L2_SOCK  TRXD 5700-5702 <- osmo-bts-trx  timeslots RX
     ↪  $OSMO_TRX_TS_MASK TX ${OSMO_TRX_TX_MASK:-0xe3}  bsic ${OSMO_TRX_BSIC:-?}  ($OSMO_TRX_ARGS)${args_ts1:+  +
     ↪  telephone 2 TS1 : $args_ts1}"
}

etape4() {   # the mobile
    [ -n "$FIRMWARE_FLASH" ] && { dire "4. (FIRMWARE_FLASH : pile L23 et UI dans le firmware Magnetite, pas de
     ↪  mobile ; ecran C139 : tools/osmo-c123-gtk.py --modele c139 --instance $INSTANCE)"; return; }
    [ -n "$FIRMWARE_APP" ] && { dire "4. (FIRMWARE_APP=$FIRMWARE_APP : pas de mobile ; ecran /dev/shm/calypso_lcd,
     ↪  touches /tmp/calypso_keys.sock, fenetre : tools/osmo-c123-gtk.py)"; return; }
    [ "$OSMO_TRX" = 1 ] && { etape4_trx; return; }
    vivant mobile && { dire "4. mobile deja lance (pid $(pid_de mobile))"; return; }
    if [ "${BANC_VOCODEUR:-hote}" = rom ]; then
        MOBILE_CFG="$(config_vocodeur_rom "$MOBILE_CFG")"
        dire "4. vocodeur ROM (BANC_VOCODEUR=rom) : tch-voice en l1phy -> $MOBILE_CFG"
    fi
    [ -S "$L2_SOCK" ] || rater "pas de socket L1CTL $L2_SOCK (etape 3 d'abord)"
    if [ "$INSTANCE" != 1 ]; then
        # instance-specific sockets and VTY (layer2/sap/ms_data suffixed _N, VTY +10 per instance)
        local _src="$MOBILE_CFG" _dst="$RUNDIR/mobile_instance$INSTANCE.cfg"
        # the VTY port is shifted by 100 per instance (4257 = osmo-ggsn; awk adds, sed does not; sub() keeps
       # the indentation: "$3 = ..." would rebuild the line without the leading space and the VTY would refuse it)
        awk -v inc=$((100 * (INSTANCE - 1))) '/^[[:space:]]*bind 127\.0\.0\.1 [0-9]+/ { n = $3 + inc;
         ↪  sub(/[0-9]+$/, n) } { print }' "$_src" \
            | sed -e "s#/tmp/osmocom_l2\b#$L2_SOCK#" -e "s#/tmp/osmocom_sap\b#/tmp/osmocom_sap_$INSTANCE#" -e
             ↪  "s#/tmp/ms_data\b#/tmp/ms_data_$INSTANCE#" > "$_dst"
        # "stick" follows the ARFCN of the radio plan (BTS#0) when it has changed (template: 514)
        local _arfcn; _arfcn="$( . /etc/osmocom/radio-plan.env 2>/dev/null; echo "${PLAN_ARFCN:-}")"
        [ -n "$_arfcn" ] && sed -i "s|^\([[:space:]]*\)stick [0-9]*\$|\1stick $_arfcn|" "$_dst"
        MOBILE_CFG="$_dst"
        dire "4. instance $INSTANCE : config mobile derivee -> $MOBILE_CFG"
    fi
    [ -x "$MOBILE" ] || rater "mobile absent : $MOBILE"
    [ -r "$MOBILE_CFG" ] || rater "config mobile absente : $MOBILE_CFG"
    local vty; vty="$(sed -n 's/^ *bind 127.0.0.1 \([0-9]*\).*/\1/p' "$MOBILE_CFG" | head -1)"
    if [ -n "$vty" ] && ss -ltn 2>/dev/null | grep -q ":$vty "; then
        rater "le port VTY $vty de $MOBILE_CFG est deja pris par : $(ss -ltnp 2>/dev/null | grep ":$vty " | grep
         ↪  -o 'users:.*' | head -1)
       changez la ligne « bind 127.0.0.1 $vty » de la config (les 42xx sont ceux du reseau du banc)"
    fi
```

```
    vitrine mobile
    # Resync on the chosen cell: 8 attempts instead of 1 (gsm322.c,
    # sync_retries_selection). The DSP only passes the SB once in three to
    # five, and two failures are enough for the "no service" loop.
    # grgsm setup: binary default, unchanged.
    local retries=""
    [ "$MODE" = dsp ] && retries="${L23_SYNC_RETRIES_SELECTION:-8}"
    L23_SYNC_RETRIES_SELECTION="$retries" \
    ${PRIO[@]+"${PRIO[@]}"} stdbuf -oL "$MOBILE" -c "$MOBILE_CFG" > "$RUNDIR/mobile.log" 2>&1 &
    echo $! > "$RUNDIR/mobile.pid"
    sleep 2
    vivant mobile || rater "mobile s'est arrete : $(sed 's/\x1b\[[0-9;]*m//g' "$RUNDIR/mobile.log" | grep -iE
     ↪  'cannot|error|unable' | tail -1)"
    dire "4. mobile  pid $(pid_de mobile)  config $MOBILE_CFG  vty telnet 127.0.0.1 ${vty:-?}"
}

etape5_air() {   # AIR=1: the air between the two Calypsos (pont/air.py), instead of the bridge
    if vivant air; then dire "5. air deja lance (pid $(pid_de air))"; etape5_vsp; return; fi
    [ -r "$AIR_PY" ] || rater "air.py absent : $AIR_PY"
    local autre_shm="$HOTE_SHM" autre_port=6702
    [ "$INSTANCE" = 1 ] && { autre_shm=/dev/shm/calypso2; autre_port=6712; }
    vitrine air
    # A = the other instance (its TX ring, its BSP port), B = this one
    # phone 2 of the TRX (TS1, instance 3: /dev/shm/calypso3, BSP udp 6722) if it runs, and the TCH
    # announcement to the mobile (calypso_tch_cfg of THIS /dev/shm) when the air decodes the ASSIGNMENT COMMAND
    local args_c=""
    if [ -S "/tmp/osmocom_l2_${OSMO_TRX_TS1_INSTANCE}" ] && [ -f "/tmp/c54x-pont${OSMO_TRX_TS1_INSTANCE}/dsp.pid"
     ↪  ] \
       && kill -0 "$(cat "/tmp/c54x-pont${OSMO_TRX_TS1_INSTANCE}/dsp.pid" 2>/dev/null)" 2>/dev/null; then
        args_c="--c-ring $HOTE_SHM/calypso${OSMO_TRX_TS1_INSTANCE}/calypso_tx_rom --c-port $((6702 + 10 *
         ↪  (OSMO_TRX_TS1_INSTANCE - 1) + (OSMO_TRX_TS1_INSTANCE >= 3 ? 30 : 0)))"
    fi
    ${PRIO[@]+"${PRIO[@]}"} python3 -u "$AIR_PY" --a-ring "$autre_shm/calypso_tx_rom" --a-port "$autre_port" \
        --b-ring /dev/shm/calypso_tx_rom --b-port "${CALYPSO_BSP_PORT:-6702}" $args_c --tch-cfg
         ↪  /dev/shm/calypso_tch_cfg > "$RUNDIR/air.log" 2>&1 &
    echo $! > "$RUNDIR/air.pid"
    attendre 5 grep -aq "air :" "$RUNDIR/air.log" || rater "air.py ne s'est pas annonce (voir $RUNDIR/air.log)"
    dire "5. air  pid $(pid_de air)  $autre_shm/calypso_tx_rom -> udp $CALYPSO_BSP_PORT (ici) ;
     ↪  /dev/shm/calypso_tx_rom (ici) -> udp $autre_port (l'autre)${args_c:+ ; TS1 : $args_c}"
    etape5_vsp
}

# the voice port (CALYPSO_VSP=1): PCM rings of the DSP <-> PulseAudio, also in air mode (the call's
# sound otherwise goes through the bridge; without it the call is established but silent). Launched in
# the instance's namespace: /dev/shm/calypso_pcm_ul and _dl are those of THIS mobile.
etape5_vsp() {
    [ "${CALYPSO_VSP:-0}" = 1 ] || return 0
    vivant vsp && { dire "5b. pont audio VSP deja lance (pid $(pid_de vsp))"; return 0; }
    ( cd /opt/GSM/osmo-operator && exec python3 -m pont.vsp_audio ) > "$RUNDIR/vsp.log" 2>&1 &
    echo $! > "$RUNDIR/vsp.pid"
    dire "5b. pont audio VSP  pid $(pid_de vsp)  gsm_mic.monitor -> calypso_pcm_ul, calypso_pcm_dl -> gsm_audio
     ↪  (journal $RUNDIR/vsp.log)"
}

etape5() {   # the TRX bridge (PONT=1): BTS bursts to layer 1
    [ "$AIR" = 1 ] && { etape5_air; return; }
    [ "$OSMO_TRX" = 1 ] && { dire "5. (OSMO_TRX=1 : pas de pont, transceiver tient 5700-5702)"; return; }
    [ "$OSMO_TRX_PHONE" = 2 ] && { dire "5. (telephone 2 : rien)"; return; }
    [ -n "$FIRMWARE_APP" ] && { dire "5. (FIRMWARE_APP=$FIRMWARE_APP : pas de pont)"; return; }
    [ "$PONT" = 1 ] || { dire "5. (PONT=0 : pas de pont.py, aucun burst du BTS n'arrive)"; return; }
    vivant pont && { dire "5. pont.py deja lance (pid $(pid_de pont))"; return; }
    [ -r "$PONT_PY" ] || rater "pont.py absent : $PONT_PY"
    local extra=""; [ "$MODE" = dsp ] && extra="--dsp-port 6702"
    [ "$PONT_AIRREC" = 0 ] && extra="$extra --no-record"
    vitrine pont
    # UL_RAW=1 (default): the bridge sends the ROM's final uplink bursts (calypso_tx_rom,
```

```
    # src/tsp_tx.c) as they are, at their frame number -- RACH, SDCCH, SACCH, FACCH, speech, A5 included.
    # UL_RAW=0: coding and A5 on the host side (PONT_UL_ROM remains the xCCH path).
    # ARM asm trace: the ARM+DSP pair can run at 14 times real time (exec,nochain).
    # The bridge's slaved clock (pont/trx.py) only goes down to 4 times below
    # real time by default; it is allowed 16 times (measured: 10 to 14 times;
    # beyond 16 the pause requested from osmo-bts-trx between two IND CLOCK
    # approaches its 50-frame limit, see trx.py run_clock). PONT_HORLOGE_MAX
    # already set in the environment is honoured.
    if [ "${ASSEMBLY_LOGS:-0}" = 1 ]; then export PONT_HORLOGE_MAX="${PONT_HORLOGE_MAX:-16}"; fi
    ( cd "$(dirname "$PONT_PY")/.." && bruit_env_pont && export PONT_UL_RAW="${UL_RAW:-1}" && exec
     ↪  ${PRIO[@]+"${PRIO[@]}"} python3 "$PONT_PY" $extra ) > "$RUNDIR/pont.log" 2>&1 &
    echo $! > "$RUNDIR/pont.pid"
    attendre 10 grep -aq "pont TRX : ports" "$RUNDIR/pont.log" || rater "pont.py ne s'est pas annonce (voir
     ↪  $RUNDIR/pont.log)"
    dire "5. pont.py  pid $(pid_de pont)  TRXD 5700-5702 <- BTS ; vers $([ "$MODE" = dsp ] && echo "le DSP (udp
     ↪  6702)" || echo "la L1 gr-gsm (udp 4730/4731)")"
    [ -n "${PONT_HORLOGE_MAX:-}" ] && dire "   horloge asservie : ralentissement jusqu'a ${PONT_HORLOGE_MAX} x le
     ↪  temps reel (PONT_HORLOGE_MAX)"
    # voice port (CALYPSO_VSP=1): PCM rings of the DSP <-> PulseAudio
    if [ "${CALYPSO_VSP:-0}" = 1 ] && ! vivant vsp; then
        ( cd /opt/GSM/osmo-operator && exec python3 -m pont.vsp_audio ) > "$RUNDIR/vsp.log" 2>&1 &
        echo $! > "$RUNDIR/vsp.pid"
       dire "5b. pont audio VSP  pid $(pid_de vsp)  gsm_mic.monitor -> calypso_pcm_ul, calypso_pcm_dl -> gsm_audio"
    fi
}

arreter() {
    local n
    for n in vsp pont air bruit mobile transceiver osmocon rvinterf gdb qemu dsp asmcat; do
        if vivant "$n"; then kill "$(pid_de "$n")" 2>/dev/null; dire "arret $n (pid $(pid_de "$n"))"; fi
        rm -f "$RUNDIR/$n.pid"
    done; bruit_arreter
    sleep 1
    # FULL STOP OF AN INSTANCE N != 1 (banc-rssi --stop): processes without a pid file or whose pid was
    # lost (relaunch over a live instance) would keep the BSP port (udp 6762 held by a ghost c54x_exe, the
    # new one receiving nothing). Everything holding the instance's own resources is swept.
    if [ "$INSTANCE" != 1 ]; then
        # Not "$RUNDIR/": a mere tail -F of its logs, or the shell that launched --stop, would match.
        local motif="$DSP_SOCK|$MONITOR|$L2_SOCK|--stub $GDB_STUB |--port $GDB_TELNET "
        local p=$$ ancetres=" $$ "
        while p="$(awk '/^PPid:/ {print $2}' "/proc/$p/status" 2>/dev/null)" && [ -n "$p" ] && [ "$p" -gt 1 ]; do
            ancetres="$ancetres$p "
        done
        local restes="" r
        for r in $(pgrep -f -- "$motif"); do
            case "$ancetres" in *" $r "*) ;; *) restes="$restes$r " ;; esac
        done
        if [ -n "${restes// /}" ]; then
            dire "instance $INSTANCE : restes arretes (pid $restes)"
            kill $restes 2>/dev/null; sleep 1; kill -9 $restes 2>/dev/null
        fi
    fi
    rm -f "$DSP_SHM" "$DSP_SOCK" "$L2_SOCK" "$MONITOR" "$RUNDIR/modem.pty" /dev/shm/qemu-asm.fifo
    # Mobile sockets: they do not disappear with it. A leftover "sap" or
    # "ms_data" (tch-data, mobile_pont.cfg) file makes the bind of the next
    # run fail with EADDRINUSE, and the mobile starts without its data path
    # without saying so.
    # ONLY for instance 1: these sockets are in the SHARED /tmp and belong to the mobile of
    # instance 1; banc-rssi --stop (instance 4) must not erase them under the running mobile (SAP, tch-data).
    [ "$INSTANCE" = 1 ] && rm -f /tmp/osmocom_sap /tmp/ms_data
    # Bench clock (calypso_bsp.c -> pont/trx.py): left in place, pont.py
    # would slave its first second to the frame of a dead session and the BTS
    # would stay frozen waiting for a DSP that no longer exists.
    rm -f /dev/shm/calypso_horloge
    # Uplink side-bands (src/montant.c): otherwise pont.py re-reads the last
    # record of a previous session at startup.
```

```
    rm -f /dev/shm/calypso_rach /dev/shm/calypso_sdcch_ul \
          /dev/shm/calypso_tch_facch_ul /dev/shm/calypso_tch_sacch_ul \
          /dev/shm/calypso_tch_ul
    # TCH announcement by the DSP bridge (pont/dsp/tch.py -> montant.c
    # scruter_tch). dsp setup only: in grgsm this file belongs to QEMU's
    # gr-gsm L1, it is left alone.
    [ "$MODE" = dsp ] && rm -f /dev/shm/calypso_tch_cfg
}

statut() {
    local n
    for n in dsp qemu osmocon rvinterf mobile transceiver pont air; do
        if vivant "$n"; then printf '  %-8s pid %-7s  %s\n' "$n" "$(pid_de "$n")" "$RUNDIR/$n.log"
        else printf '  %-8s arrete\n' "$n"; fi
    done; bruit_statut
    [ -f "$RUNDIR/dsp.log" ] && grep -a "fn=" "$RUNDIR/dsp.log" | tail -1
    [ -f "$RUNDIR/osmocon.log" ] && grep -a "FB0\|FB1\|SB\|BSIC" "$RUNDIR/osmocon.log" | tail -1
}

# KEEP THE DSP AND BRIDGE LOGS. dsp.log is overwritten at every launch and
# pont.log is emptied by vitrine; neither is in the panel's archives
# (osmo-nitb/archives). The previous session is stored in
# $RUNDIR/archives/<date of dsp.log>/ and JOURNAUX_GARDES (10) of them are
# kept. JOURNAUX_GARDES=0 keeps nothing.
JOURNAUX_GARDES="${JOURNAUX_GARDES:-10}"
garder_journaux() {
    [ "$JOURNAUX_GARDES" -gt 0 ] 2>/dev/null || return 0
    [ -s "$RUNDIR/dsp.log" ] || [ -s /dev/shm/pont.log ] || return 0
    local d="$RUNDIR/archives/$(date -r "$RUNDIR/dsp.log" +%Y%m%d-%H%M%S 2>/dev/null || date +%Y%m%d-%H%M%S)"
    [ -d "$d" ] && return 0                    # already stored (--stop then relaunch)
    mkdir -p "$d" || return 0
    local n
    for n in dsp pont mobile qemu osmocon bruit; do
        [ -s "$RUNDIR/$n.log" ] && cp -L "$RUNDIR/$n.log" "$d/" 2>/dev/null
    done
    # state of the BSP's dedicated store (stored/played/missed/fallbacks/lost)
    [ -s /dev/shm/calypso_bsp_dedie ] && cp /dev/shm/calypso_bsp_dedie "$d/bsp_dedie.txt" 2>/dev/null
    ls -1dt "$RUNDIR"/archives/*/ 2>/dev/null | tail -n +$((JOURNAUX_GARDES + 1)) | xargs -r rm -rf
    dire "journaux de la session precedente ranges dans $d"
}

# NOISE INJECTOR, INACTIVE BY DEFAULT (BRUIT_MODE and BRUIT_SNR_DB empty: nothing
# changes, not one more line on screen). Placed here, at the end of the file, so that
# the line numbers cited by the wiki (Lancer-a-la-main.md...) stay right; the steps
# above only call bruit_env_dsp, bruit_dire_dsp, bruit_env_pont, bruit_arreter and
# bruit_statut, without effect by default.
# Single source: $QOSMO/tools/injecteur_bruit.py, fallback on tools/injecteur_bruit.py here
# (a link to it).
#   BRUIT_MODE     ber | souple | iq (EXPERIMENTAL) | relais: step b (--step b), between the
#                  DSP and QEMU in the sequence, between the bridge and the L1 in the flow.
#     dsp   : c54x_exe listens on BRUIT_PORT_DSP (16702, CALYPSO_BSP_PORT), the injector takes
#             127.0.0.1:6702 in its place; the bridge does not change (--dsp-port 6702). The BSP
#             learns its uplink peer from the sender of the downlink: its uplink goes through
#             the injector, which hands it unchanged to the bridge (5702) from 6702.
#     grgsm : the injector listens on BRUIT_PORT_GRGSM (14730) and relays to QEMU 4730; the bridge
#             sends there (PONT_GSMTAP_PORT, set at step 5). ber | relais only: QEMU
#             receives already decoded L2 blocks.
#     iq    : refused if CALYPSO_BSP_STREAM=1 (default of start-native.sh --dsp).
#   BRUIT_BER (0..1), BRUIT_RAFALES (L bits), BRUIT_SNR_DB (dB), BRUIT_PERTE (0..1),
#   BRUIT_TN (0 or 1,2...), BRUIT_GRAINE, BRUIT_STATS (10 s), BRUIT_OPTS (extra options),
#   BRUIT_PORT_DSP, BRUIT_PORT_GRGSM, BRUIT_PY.
#   BRUIT_SNR_DB WITHOUT BRUIT_MODE (dsp): no injector, AWGN of the C54x core -- c54x_exe
#   receives CALYPSO_BSP_SNR_DB (and CALYPSO_BSP_BRUIT_GRAINE <- BRUIT_GRAINE), calypso_c54x.c.
# A refused configuration (--verifier) stops run.sh BEFORE any launch.
# Log: $RUNDIR/bruit.log. Details: /opt/GSM/osmo-operator/wiki/Injecteur-bruit.md.
BRUIT_MODE="${BRUIT_MODE:-}"
```

```
BRUIT_SNR_DB="${BRUIT_SNR_DB:-}"
BRUIT_PORT_DSP="${BRUIT_PORT_DSP:-16702}"
BRUIT_PORT_GRGSM="${BRUIT_PORT_GRGSM:-14730}"
if [ -z "${BRUIT_PY:-}" ]; then
    BRUIT_PY="$QOSMO/tools/injecteur_bruit.py"
    [ -r "$BRUIT_PY" ] || BRUIT_PY="$HERE/tools/injecteur_bruit.py"
fi
BRUIT_ACTIF=0      # 1: the injector is inserted for this launch (bruit_preparer)
BRUIT_ARGS=()

bruit_args() {   # fills BRUIT_ARGS
    BRUIT_ARGS=(--mode "$BRUIT_MODE" --etiquette c54x --stats "${BRUIT_STATS:-10}")
    if [ "$MODE" = dsp ]; then
        BRUIT_ARGS+=(--cible dsp --ecoute 127.0.0.1:6702 --vers "127.0.0.1:$BRUIT_PORT_DSP")
    else
        BRUIT_ARGS+=(--cible grgsm --ecoute "127.0.0.1:$BRUIT_PORT_GRGSM" --vers 127.0.0.1:4730)
    fi
    [ -n "${BRUIT_BER:-}" ]     && BRUIT_ARGS+=(--ber "$BRUIT_BER")
    [ -n "${BRUIT_RAFALES:-}" ] && BRUIT_ARGS+=(--rafales "$BRUIT_RAFALES")
    [ -n "$BRUIT_SNR_DB" ]      && BRUIT_ARGS+=(--snr-db "$BRUIT_SNR_DB")
    [ -n "${BRUIT_PERTE:-}" ]   && BRUIT_ARGS+=(--perte "$BRUIT_PERTE")
    [ -n "${BRUIT_TN:-}" ]      && BRUIT_ARGS+=(--tn "$BRUIT_TN")
    [ -n "${BRUIT_GRAINE:-}" ]  && BRUIT_ARGS+=(--graine "$BRUIT_GRAINE")
    [ -n "${BRUIT_OPTS:-}" ]    && BRUIT_ARGS+=($BRUIT_OPTS)
    return 0
}

bruit_preparer() {   # before step 1: decides BRUIT_ACTIF, checks the configuration
    [ -n "$BRUIT_MODE" ] || return 0
    if vivant bruit; then BRUIT_ACTIF=1; return 0; fi
    if [ "$MODE" = dsp ] && vivant dsp; then
        # An already running c54x_exe can only be intercepted if it already listens on the moved port.
        if ! tr '\0' '\n' < "/proc/$(pid_de dsp)/environ" 2>/dev/null | grep -qx
         ↪  "CALYPSO_BSP_PORT=$BRUIT_PORT_DSP"; then
            dire "b. BRUIT_MODE=$BRUIT_MODE ignore : c54x_exe tourne deja sur 6702, sans injecteur (./run.sh
             ↪  --stop puis ./run.sh)"
            return 0
        fi
    fi
    [ -r "$BRUIT_PY" ] || rater "injecteur de bruit absent : $BRUIT_PY (posez BRUIT_PY)"
    bruit_args
    local msg
    msg="$(python3 "$BRUIT_PY" "${BRUIT_ARGS[@]}" --verifier 2>&1)" || rater "injecteur de bruit : $(printf '%s\n'
     ↪  "$msg" | tail -1)"
    BRUIT_ACTIF=1
}

bruit_env_dsp() {   # in the c54x_exe subshell; sets nothing by default
    [ "$BRUIT_ACTIF" = 1 ] && export CALYPSO_BSP_PORT="$BRUIT_PORT_DSP"
    if [ -z "$BRUIT_MODE" ] && [ -n "$BRUIT_SNR_DB" ]; then
        export CALYPSO_BSP_SNR_DB="${CALYPSO_BSP_SNR_DB:-$BRUIT_SNR_DB}"
        [ -n "${BRUIT_GRAINE:-}" ] && export CALYPSO_BSP_BRUIT_GRAINE="${CALYPSO_BSP_BRUIT_GRAINE:-$BRUIT_GRAINE}"
    fi
    return 0
}

bruit_dire_dsp() {   # after step 1; silent by default
    [ "$BRUIT_ACTIF" = 1 ] && dire "   BSP deplace sur udp/$BRUIT_PORT_DSP (CALYPSO_BSP_PORT) : 6702 est a
     ↪  l'injecteur de bruit"
    [ -z "$BRUIT_MODE" ] && [ -n "$BRUIT_SNR_DB" ] && dire "   bruit du coeur :
     ↪  CALYPSO_BSP_SNR_DB=${CALYPSO_BSP_SNR_DB:-$BRUIT_SNR_DB} dB${BRUIT_GRAINE:+, graine $BRUIT_GRAINE}"
    return 0
}

bruit_env_pont() {   # in the bridge subshell; sets nothing by default
    [ "$MODE" = grgsm ] && [ -n "$BRUIT_MODE" ] || return 0
```

```
    # pont/pont.py would target BRUIT_PORT_GRGSM by itself: it is stated explicitly, and
    # 4730 (QEMU) if the injector is not running, so as not to send into the void.
    if vivant bruit; then export PONT_GSMTAP_PORT="$BRUIT_PORT_GRGSM"; else export PONT_GSMTAP_PORT=4730; fi
    return 0
}

etapeb() {   # the noise injector (BRUIT_MODE), between the bridge and layer 1
    [ "$BRUIT_ACTIF" = 1 ] || return 0
    vivant bruit && { dire "b. injecteur de bruit deja lance (pid $(pid_de bruit))"; return; }
    bruit_args
    : > "$RUNDIR/bruit.log"
    python3 -u "$BRUIT_PY" "${BRUIT_ARGS[@]}" >> "$RUNDIR/bruit.log" 2>&1 &
    echo $! > "$RUNDIR/bruit.pid"
    attendre 5 grep -aq "PRET ecoute=" "$RUNDIR/bruit.log" || rater "l'injecteur de bruit ne s'est pas annonce
     ↪  (voir $RUNDIR/bruit.log)"
    dire "b. injecteur de bruit  pid $(pid_de bruit)  mode $BRUIT_MODE  $(sed -n 's/.*PRET \(ecoute=[^ ]*\)
     ↪  \(vers=[^ ]*\).*/\1 \2/p' "$RUNDIR/bruit.log" | head -1)  journal $RUNDIR/bruit.log"
}

bruit_arreter() {   # --stop: the injector without pid (launched by hand with --etiquette c54x)
    pkill -f -- "injecteur_bruit\.py .*--etiquette c54x" 2>/dev/null
    return 0
}

bruit_statut() {   # --status: silent by default
    if vivant bruit; then
        printf '  %-8s pid %-7s  %s\n' bruit "$(pid_de bruit)" "$RUNDIR/bruit.log"
        grep -a "STATS\|BILAN" "$RUNDIR/bruit.log" 2>/dev/null | tail -1
    elif [ -n "$BRUIT_MODE" ]; then
        printf '  %-8s arrete\n' bruit
    fi
    return 0
}

mkdir -p "$RUNDIR"
case "${1:-}" in
    --stop)   arreter; garder_journaux
              # phone 2 of the TRX (instance 3) goes down with instance 1
              if [ "$INSTANCE" = 1 ] && [ -d "/tmp/c54x-pont$OSMO_TRX_TS1_INSTANCE" ] && [ -f
               ↪  "/tmp/c54x-pont$OSMO_TRX_TS1_INSTANCE/dsp.pid" ]; then
                  INSTANCE="$OSMO_TRX_TS1_INSTANCE" OSMO_TRX=1 OSMO_TRX_PHONE=2 AIR=0 bash "$0" --stop
              fi
              # the mobile over the air (instance 2) too: left alive across start-native.sh --bts --ms
              # relaunches, the air would keep the OLD TX ring of instance 1 open (file deleted): A->B
              # frozen, no more downlink for the mobile, hence no more LU.
              if [ "$INSTANCE" = 1 ] && [ -d /tmp/c54x-pont2 ] && ls /tmp/c54x-pont2/*.pid >/dev/null 2>&1; then
                  INSTANCE=2 AIR=1 OSMO_TRX=0 bash "$0" --stop
              fi
              # emptied once stored: the next launch does not store them twice
              [ -f "$RUNDIR/dsp.log" ] && : > "$RUNDIR/dsp.log"
              [ -f /dev/shm/pont.log ] && : > /dev/shm/pont.log ;;
    --status) statut ;;
    --charge) charge ;;
    --logs)   # coloured on display (qosmo/tools/qosmo-couleurs.py, QOSMO_COULEURS=0 turns it off)
              _coul="$QOSMO/tools/qosmo-couleurs.py"
              if [ "${QOSMO_COULEURS:-1}" != 0 ] && [ -f "$_coul" ] && [ -t 1 ]; then
                  exec tail -n 5 -F "$RUNDIR"/dsp.log "$RUNDIR"/qemu.log "$RUNDIR"/osmocon.log
                   ↪  "$RUNDIR"/mobile.log "$RUNDIR"/transceiver.log "$RUNDIR"/pont.log "$RUNDIR"/air.log $([ -n
                   ↪  "$BRUIT_MODE" ] || [ -f "$RUNDIR/bruit.pid" ] && echo "$RUNDIR/bruit.log") 2>/dev/null |
                   ↪  python3 -u "$_coul" --force
              fi
              exec tail -n 5 -F "$RUNDIR"/dsp.log "$RUNDIR"/qemu.log "$RUNDIR"/osmocon.log "$RUNDIR"/mobile.log
               ↪  "$RUNDIR"/transceiver.log "$RUNDIR"/pont.log "$RUNDIR"/air.log $([ -n "$BRUIT_MODE" ] || [ -f
               ↪  "$RUNDIR/bruit.pid" ] && echo "$RUNDIR/bruit.log") 2>/dev/null ;;
    --step)   case "${2:-}" in 1) bruit_preparer; etape1;; b) bruit_preparer; etapeb;; 2) etape2;; 3) etape3;; 4)
     ↪  etape4;; 5) etape5;; a) etape5_air;; *) rater "--step 1|b|2|3|4|5|a";; esac ;;
```

```
    -h|--help) sed -n '2,22p' "$0" ;;
    "")       garder_journaux; bruit_preparer; etape1; etapeb; etape2; etape3; etape4; etape5
              dire "tout tourne. Journaux : $RUNDIR/*.log   suivre : ./run.sh --logs   arreter : ./run.sh --stop" ;;
    *)        rater "option inconnue : $1 (voir --help)" ;;
esac
```

3.9 /opt/GSM/c54x_exe/run_real.sh

3846 octets, 57 lignes → 57 lignes

```
#!/usr/bin/env bash
# run_real.sh - the REAL bench: virtual BTS (osmo-bts-trx + osmocom core) ->
# pont.py (TRX) -> DSP c54x_exe (--iq none, UDP 6702 bursts) -> QEMU/ARM ->
# osmocon -> mobile. Same executables as run.sh, without any crutch
# (no CAN_TOA / CAN_SB / forced AFC): native FB, SB and BCCH.
#
#   ./run_real.sh              start everything, observe 120 s, SI / LAI verdict
#   ./run_real.sh --secondes N observation time
#   ./run_real.sh --logs       follow the logs         ./run_real.sh --stop  stop everything
#
# Settings: INSNS=60000 (DSP pace close to the BTS real time),
# CALYPSO_BSP_STREAM=1 (one TS0 burst per frame, FN order),
# CALYPSO_RHEA_DMA_XFER=1 CALYPSO_BSP_DIRECT_FEED=1 (DMA/RIF delivery),
# CALYPSO_BSP_NB_SYM=0.3 (normal burst widening, calypso_bsp.c).
set -u
HERE="$(cd "$(dirname "$0")" && pwd)"
RUNDIR=/tmp/c54x-pont
SECS=120
case "${1:-}" in
  --stop)  "$HERE/run.sh" --stop; systemctl stop osmo-bts-trx 2>/dev/null; exit 0 ;;
  --logs)  exec "$HERE/run.sh" --logs ;;
  --secondes) SECS="$2" ;;
  -h|--help) sed -n '2,14p' "$0"; exit 0 ;;
esac
E_MCC="$(grep -m1 -oE 'network country code [0-9]+' /etc/osmocom/osmo-msc.cfg | grep -oE '[0-9]+$')"
E_MNC="$(grep -m1 -oE 'mobile network code [0-9]+'  /etc/osmocom/osmo-msc.cfg | grep -oE '[0-9]+$')"
E_LAC="$(grep -m1 -oE 'location_area_code 0x[0-9a-fA-F]+' /etc/osmocom/osmo-bsc.cfg | grep -oE '0x[0-9a-fA-F]+')"
E_LAC=$((E_LAC)); E_MCC=$((10#$E_MCC)); E_MNC=$((10#$E_MNC))
BSIC="$(grep -m1 -oE 'base_station_id_code [0-9]+' /etc/osmocom/osmo-bsc.cfg | grep -oE '[0-9]+$')"
echo "== run_real : LAI attendu MCC=$E_MCC MNC=$E_MNC LAC=$E_LAC (BSIC=${BSIC:-?}), aucune bequille"
"$HERE/run.sh" --stop >/dev/null 2>&1; sleep 1
for u in osmo-hlr osmo-stp osmo-msc osmo-mgw osmo-bsc; do systemctl is-active --quiet "$u" || systemctl start
 ↪  "$u"; done
systemctl restart osmo-bts-trx; sleep 3
mkdir -p "$RUNDIR"; : > "$RUNDIR/mobile.log"; : > "$RUNDIR/osmocon.log"; : > "$RUNDIR/pont.log"
export MODE=dsp PONT=1 IQ=none INSNS="${INSNS:-60000}" LOCKSTEP="${LOCKSTEP:-1}"
export CALYPSO_RHEA_DMA_XFER=1 CALYPSO_BSP_DIRECT_FEED=1 CALYPSO_BSP_STREAM=1
export PONT_NB_DEBUG="${PONT_NB_DEBUG:-1}"
"$HERE/run.sh" || { echo "== run.sh a echoue"; exit 1; }
echo "== observation ${SECS}s ..."
sleep "$SECS"
M="$RUNDIR/mobile.log"; O="$RUNDIR/osmocon.log"
echo "== SB decodees (osmocon) : $(sed 's/\x1b\[[0-9;]*m//g' "$O" | grep -ac '=> SB')   FBSB sans SB : $(sed
 ↪  's/\x1b\[[0-9;]*m//g' "$M" | grep -ac 'result=255')"
echo "== blocs BCCH jetes (fire) : $(sed 's/\x1b\[[0-9;]*m//g' "$M" | grep -ac 'Dropping frame')"
echo "== System Information vus :"
sed 's/\x1b\[[0-9;]*m//g' "$M" | grep -aoE "New SYSTEM INFORMATION [0-9a-z]+" | sort | uniq -c || echo "  (aucun)"
echo "== montant : RACH publies par le DSP : $(sed 's/\x1b\[[0-9;]*m//g' "$RUNDIR/dsp.log" | grep -ac '\[montant\]
 ↪  RACH')   compteurs du pont : $(grep -ao 'UL bursts=[0-9]* tard=[0-9]* rach=[0-9]*' "$RUNDIR/pont.log" | tail
 ↪  -1)"
echo "== IMM ASS : $(sed 's/\x1b\[[0-9;]*m//g' "$M" | grep -ac 'IMMEDIATE ASSIGNMENT')   LU ACCEPT : $(sed
 ↪  's/\x1b\[[0-9;]*m//g' "$M" | grep -ac 'LOCATION UPDATING ACCEPT')"
LAI="$(sed 's/\x1b\[[0-9;]*m//g' "$M" | grep -aoE "lai=[0-9]+-[0-9]+-[0-9]+" | tail -1)"
echo "== VERDICT :"
```

```
if [ -z "$LAI" ]; then echo "  ECHEC : aucun SI lu"; else
  D_MCC="$(echo "$LAI" | cut -d= -f2 | cut -d- -f1)"; D_MNC="$(echo "$LAI" | cut -d- -f2)"; D_LAC="$(echo "$LAI" |
   ↪  cut -d- -f3)"
  if [ "$((10#$D_MCC))" = "$E_MCC" ] && [ "$((10#$D_MNC))" = "$E_MNC" ] && [ "$D_LAC" = "$E_LAC" ]; then
    echo "  VRAI : $LAI = la config du reseau, le mobile a lu la BCCH du BTS"
  else echo "  FAUX POSITIF : $LAI ne correspond pas a MCC=$E_MCC MNC=$E_MNC LAC=$E_LAC"; fi
  sed 's/\x1b\[[0-9;]*m//g' "$M" | grep -aq "camping normally" && echo "  et il campe (MM IDLE)"
fi
echo "== arret : ./run_real.sh --stop"
```

3.10 /opt/GSM/c54x_exe/run_si.sh

3847 octets, 72 lignes → 72 lignes

```
#!/usr/bin/env bash
# run_si.sh - bring the mobile (c54x_exe DSP stack) up to decoding the BCCH of
# the REAL network and reading a System Information, THEN check that the
# decoded LAI matches the network config. Otherwise = FALSE POSITIVE, said so.
#
# The chain runs NATIVE: no result scaffolding (no CAN_TOA / CAN_SB / forced
# AFC / RHEA_DMA); the ROM's FB correlator, AFC and DMA do the work. Only the
# pacing and burst feeding are set, and they fabricate no result:
#   - CALYPSO_BSP_DIRECT_FEED=1  bursts delivered straight to the DSP buffers
#   - CALYPSO_PONT_LOCKSTEP=1    QEMU frame paced on the DSP
# A SI read here is an integration test of the whole chain
# (sync -> BCCH -> L2/L3) against the network's own identity.
#
#   ./run_si.sh              # real chain (BTS + bridge)
#   ./run_si.sh --secondes N # observation time (default 90)
#   ./run_si.sh --stop       # stop everything
set -u
HERE="$(cd "$(dirname "$0")" && pwd)"
RUNDIR=/tmp/c54x-pont
SECS=90
BSIC="$(grep -m1 -oE 'base_station_id_code [0-9]+' /etc/osmocom/osmo-bsc.cfg 2>/dev/null | grep -oE '[0-9]+$')";
 ↪  BSIC="${BSIC:-7}"
# expected LAI, read from the config (MCC/MNC/LAC)
E_MCC="$(grep -m1 -oE 'network country code [0-9]+' /etc/osmocom/osmo-msc.cfg | grep -oE '[0-9]+$')"
E_MNC="$(grep -m1 -oE 'mobile network code [0-9]+'  /etc/osmocom/osmo-msc.cfg | grep -oE '[0-9]+$')"
E_LAC="$(grep -m1 -oE 'location_area_code 0x[0-9a-fA-F]+' /etc/osmocom/osmo-bsc.cfg | grep -oE '0x[0-9a-fA-F]+')"
E_LAC=$((E_LAC)); E_MCC=$((10#$E_MCC)); E_MNC=$((10#$E_MNC))

case "${1:-}" in
  --stop) "$HERE/run.sh" --stop; systemctl stop osmo-bts-trx 2>/dev/null; exit 0 ;;
  --secondes) SECS="$2" ;;
  -h|--help) sed -n '2,25p' "$0"; exit 0 ;;
esac

echo "== run_si : cible LAI attendu MCC=$E_MCC MNC=$E_MNC LAC=$E_LAC (BSIC=$BSIC, ARFCN 514)"
"$HERE/run.sh" --stop >/dev/null 2>&1; sleep 1
systemctl start osmo-bts-trx 2>/dev/null; sleep 3
: > "$RUNDIR/mobile.log" 2>/dev/null; : > "$RUNDIR/osmocon.log" 2>/dev/null

# real chain, NATIVE: no result scaffolding (CAN_TOA/CAN_SB/AFC_HZ/RHEA_DMA);
# the ROM's FB correlator, AFC and DMA do the work. Only the pacing and the
# burst feeding are set, and they fabricate no result.
export MODE=dsp PONT=1 IQ=none
export CALYPSO_BSP_DIRECT_FEED=1 CALYPSO_PONT_LOCKSTEP=1   # pacing / feeding (not cheats)
echo "== NATIF : aucun echafaudage (CAN_TOA/CAN_SB/AFC_HZ/RHEA_DMA retires) ; LOCKSTEP + DIRECT_FEED"
"$HERE/run.sh" >/dev/null 2>&1 &
sleep 12
grep -aq "code is running" "$RUNDIR/osmocon.log" && echo "== firmware lancé" || echo "== firmware PAS lancé (voir
 ↪  $RUNDIR/osmocon.log)"
echo "== observation ${SECS}s ..."
```

```
sleep "$SECS"

M="$RUNDIR/mobile.log"
echo "== System Information vus (BCCH) :"
sed 's/\x1b\[[0-9;]*m//g' "$M" | grep -aoE "New SYSTEM INFORMATION [0-9a-z]+" | sort | uniq -c || echo "  (aucun)"
LAI="$(sed 's/\x1b\[[0-9;]*m//g' "$M" | grep -aoE "lai=[0-9]+-[0-9]+-[0-9]+" | tail -1)"
echo "== [can-sb] premières lignes :"; grep -a "\[can-sb\]" "$RUNDIR/dsp.log" 2>/dev/null | head -3 | cut -c1-120

echo "== VERDICT :"
if [ -z "$LAI" ]; then
  echo "  ÉCHEC : aucun SI lu -> pas de camp. (attendu tant que le démod NB/BCCH natif n'est pas bon)"
else
  D_MCC="$(echo "$LAI" | cut -d= -f2 | cut -d- -f1)"
  D_MNC="$(echo "$LAI" | cut -d- -f2)"
  D_LAC="$(echo "$LAI" | cut -d- -f3)"
  echo "  LAI décodé : MCC=$D_MCC MNC=$D_MNC LAC=$D_LAC"
  if [ "$D_MCC" = "$E_MCC" ] && [ "$D_MNC" = "$E_MNC" ] && [ "$D_LAC" = "$E_LAC" ]; then
    echo "  ✓ VRAI : le LAI correspond à la config -> le mobile a réellement lu la BCCH du réseau."
  else
    echo "  ✗ FAUX POSITIF : le LAI ne correspond pas (config MCC=$E_MCC MNC=$E_MNC LAC=$E_LAC)."
    echo "    Un bloc décodé qui ne porte pas l'identité du réseau n'est pas un camp, c'est du bruit qui a passé
    ↪  un CRC."
  fi
fi
echo "== arrêt : ./run_si.sh --stop"
```

3.11 /opt/GSM/c54x_exe/sign.sh

2209 octets, 52 lignes → 52 lignes

```
#!/usr/bin/env bash
# sign.sh - SHA-256 digest and detached GPG signature of a file (default proof_toa_23.md).
#
#   ./sign.sh                      sign proof_toa_23.md
#   ./sign.sh FILE                 sign FILE
#   ./sign.sh --verify [FILE]      verify the digest and the signature
#
# Key: if no GPG secret key matches SIGN_UID, an ed25519 key (signing only, no expiration)
# is generated. Variables: SIGN_UID (identity), SIGN_PASSPHRASE (empty by default),
# GNUPGHOME (keyring, default ~/.gnupg). Outputs, next to the file:
#   FILE.sha256     digest (sha256sum format)
#   FILE.asc        ASCII detached signature
#   FILE.pubkey.asc public key to distribute with the document
set -euo pipefail
HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SIGN_UID="${SIGN_UID:-proof_toa_23 <bastienbaranoff@gmail.com>}"
SIGN_PASSPHRASE="${SIGN_PASSPHRASE:-}"
export GNUPGHOME="${GNUPGHOME:-$HOME/.gnupg}"
mkdir -p "$GNUPGHOME"; chmod 700 "$GNUPGHOME"
GPG=(gpg --batch --yes --pinentry-mode loopback --passphrase "$SIGN_PASSPHRASE")

verify=0
[ "${1:-}" = "--verify" ] && { verify=1; shift; }
F="${1:-$HERE/proof_toa_23.md}"
[ -r "$F" ] || { echo "fichier illisible : $F" >&2; exit 2; }
D="$(cd "$(dirname "$F")" && pwd)"; B="$(basename "$F")"

if [ "$verify" = 1 ]; then
    (cd "$D" && sha256sum -c "$B.sha256")
    [ -r "$F.pubkey.asc" ] && gpg --batch --quiet --import "$F.pubkey.asc" 2>/dev/null || true
    gpg --batch --verify "$F.asc" "$F"
    exit $?
fi
```

```
# 1. digest
(cd "$D" && sha256sum "$B" > "$B.sha256")
cat "$F.sha256"

# 2. key (generated once)
if ! gpg --batch --list-secret-keys "=$SIGN_UID" >/dev/null 2>&1; then
    echo "aucune cle secrete pour « $SIGN_UID » : generation d'une cle ed25519 (signature, sans expiration)"
    "${GPG[@]}" --quick-gen-key "$SIGN_UID" ed25519 sign 0
fi
FPR="$(gpg --batch --with-colons --list-secret-keys "=$SIGN_UID" | awk -F: '$1=="fpr"{print $10; exit}')"
echo "cle : $FPR  ($SIGN_UID)"

# 3. detached signature + public key
"${GPG[@]}" --armor --detach-sign --local-user "$FPR" --output "$F.asc" "$F"
gpg --batch --armor --export "$FPR" > "$F.pubkey.asc"
echo "signature : $F.asc"
echo "cle publique : $F.pubkey.asc"
echo "verification : $0 --verify $F"
```

3.12 /opt/GSM/c54x_exe/src/carte_tx.h

2148 octets, 36 lignes → 36 lignes

```
/* Generated by tests/rom-tx/carte.py: output bit p of the ROM buffer data[0x4280] (word p/16,
 * bit 15-p%16) -> index i among the 456 bits of gsm0503_xcch_encode; -1 = unknown. */
static const short g_carte_tx[512] = {
  -1,1,2,3,4,5,6,-1,8,9,10,11,-1,13,14,15,
  16,17,18,19,20,-1,22,23,24,25,-1,27,28,29,30,31,
  32,33,34,-1,36,37,38,39,-1,41,42,43,44,45,46,47,
  48,-1,50,51,52,53,54,55,56,59,60,61,62,63,64,-1,
  66,-1,68,69,70,71,-1,73,74,75,76,77,78,-1,80,81,
  82,83,84,85,-1,87,88,89,90,91,92,-1,94,95,96,97,
  98,99,-1,101,102,103,104,105,106,107,108,109,110,111,112,113,
  -1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,
  116,117,118,119,120,-1,122,123,124,125,-1,127,128,129,130,131,
  132,133,134,-1,136,137,138,139,-1,141,142,143,144,145,146,147,
  148,-1,150,151,152,153,154,155,156,157,158,159,160,161,162,-1,
  164,-1,166,167,168,169,-1,171,172,175,176,177,178,-1,180,181,
  182,183,184,185,-1,187,188,189,190,191,192,-1,194,195,196,197,
  198,199,-1,201,202,203,204,205,206,207,208,209,210,211,212,213,
  -1,215,-1,217,218,219,220,221,222,-1,224,225,226,227,-1,229,
  230,231,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,
  232,233,234,-1,236,237,238,239,-1,241,242,243,244,245,246,247,
  248,-1,250,251,252,253,254,255,256,257,258,259,260,261,262,-1,
  264,-1,266,267,268,269,-1,271,272,273,274,275,276,-1,278,279,
  280,281,282,283,-1,285,286,287,288,291,292,-1,294,295,296,297,
  298,299,-1,301,302,303,304,305,306,307,308,309,310,311,312,313,
  -1,315,316,317,318,319,320,321,322,-1,324,325,326,327,-1,329,
  -1,331,332,333,334,335,336,-1,338,339,340,341,-1,343,344,345,
  346,347,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,
  348,-1,350,351,352,353,354,355,356,357,358,359,360,361,362,-1,
  364,-1,366,367,368,369,-1,371,372,373,374,375,376,-1,378,379,
  380,381,382,383,-1,385,386,387,388,389,390,-1,392,393,394,395,
  396,397,-1,399,400,401,402,403,404,407,408,409,410,411,412,413,
  -1,415,416,417,418,419,420,421,422,-1,424,425,426,427,-1,429,
  -1,431,432,433,434,435,436,-1,438,439,440,441,-1,443,444,445,
  446,447,448,449,450,-1,452,453,454,455,-1,-1,-1,-1,-1,-1,
  -1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,-1,
};
```

3.13 /opt/GSM/c54x_exe/src/cellule.c

37331 octets, 691 lignes → 693 lignes (2 groupes compactés)

```
/* SPDX-License-Identifier: GPL-2.0-or-later */
/*
 * cellule.c - cell synchronisation bursts for the DSP.
 *
 * FCCH = 148 zero bits (45.002 5.2.4). SCH = 3 tail + 39 + 64 extended training
 * sequence (45.002 5.2.5) + 39 + 3 tail; the 78 coded bits come from
 * gsm0503_sch_encode() of libosmocoding fed with sb_info (BSIC + T1'/T2/T3',
 * 44.018 9.1.30). Dummy burst: the fixed pattern of 45.002 5.2.6. Burst
 * assembly follows osmo-bts sched_lchan_fcch_sch.c and scheduler.c.
 */
#include <string.h>
#include <math.h>
#include "hw/arm/calypso/calypso_debug.h"
#include <stdlib.h>
#include <osmocom/core/bits.h>
#include <osmocom/coding/gsm0503_coding.h>
#include "calypso_gmsk.h"
#include "cellule.h"

static const uint8_t train_sb[64] = {
    1,0,1,1,1,0,0,1,0,1,1,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,1,1,1,
    0,0,1,0,1,1,0,1,0,1,0,0,0,1,0,1,0,1,1,1,0,1,1,0,0,0,0,1,1,0,1,1,
};
static const uint8_t factice[148] = {
    0,0,0,
    ⟨1⟩,⟨2⟩,⟨3⟩,⟨4⟩,⟨5⟩,⟨6⟩,⟨7⟩,⟨8⟩,0,⟨9⟩,⟨10⟩,1,⟨11⟩,⟨12⟩,⟨13⟩,⟨14⟩,⟨15⟩,⟨16⟩,⟨17⟩,⟨18⟩,⟨19⟩,⟨20⟩,⟨21⟩,⟨22⟩,⟨23⟩,⟨24⟩,⟨25⟩,⟨26⟩,⟨
↪ 27⟩,⟨28⟩,⟨29⟩,⟨30⟩,
↪ ×4
    ⟨⟩ = (1,1,1,1,1,0,1,1,1,1,0,1,1,0,0,0,0,0,1,0,1,0,0,1,0,0,1,1,1,0)
         (0,0,0,0,1,0,0,1,0,0,0,0,0,0,0,0,0,1,1,1,1,1,0,0,0,1,1,1,0,0)
         (0,1,0,1,1,1,0,0,1,0,1,1,0,0,0,1,0,1,0,1,1,1,0,1,1,0,0,1,0,1,0)
         (0,0,1,1,0,0,1,1,0,1,1,0,0,1,1,1,1,1,0,1,0,0,1,1,1,1,1,0,0,0,1)
    0,0,1,0,1,1,1,1,1,0,1,0,1,0,
    0,0,0,
};

int cellule_sch_partout;      /* diagnostic: emit SCH on every non-FCCH frame */

/* ---- BCCH / CCCH : normal bursts of the synthetic cell [2026-09-20] ----
 *
 * Downlink TN0 of a combined CCCH+SDCCH/4 cell (44.018 6.3.1.3, 45.002 clause
 * 7 table 3): BCCH norm on p51 2..5, CCCH on 6..9, 12..15, 16..19, SDCCH/4
 * elsewhere (left as dummy bursts). The BCCH block of 51-multiframe TC =
 * (fn / 51) % 8 carries SI1 (TC 0, 4), SI2 (1, 5), SI3 (2, 6), SI4 (3, 7);
 * the CCCH blocks carry an empty PAGING REQUEST TYPE 1. Coding is the one of
 * osmo-bts sched_lchan_xcch.c: gsm0503_xcch_encode() (Fire CRC, r=1/2
 * convolutional code, 4-burst diagonal interleaving) then 3 tail, 57 data,
 * hl, 26-bit training sequence BCC of the BSIC (45.002 5.2.3 set 1), hu, 57
 * data, 3 tail. The ROM demodulates with the TSC the ARM hands it in
 * dsp_load_rx_task(ALLC_DSP_TASK, burst_id, tsc), decodes the four bursts and
 * leaves 23 octets in a_cd[3..] with the Fire result in a_cd[0]; prim_rx_nb.c
 * copies them into an L1CTL_DATA_IND and mobile's rr reads the SI.
 *
 * Identity: MCC 001 MNC 01 LAC 1 CI 6001, ARFCN 514, the bench network of
 * /etc/osmocom (run_si.sh checks the decoded LAI against it). Override with
 * CELLULE_MCC / CELLULE_MNC / CELLULE_LAC / CELLULE_CI / CELLULE_ARFCN. */
#include <arpa/inet.h>
#include <osmocom/gsm/gsm48.h>
#include <osmocom/gsm/gsm48_ie.h>
#include <osmocom/gsm/gsm23003.h>
#include <osmocom/gsm/protocol/gsm_04_08.h>
#include <osmocom/gsm/sysinfo.h>
```

```
static const uint8_t train_nb[8][26] = {   /* 45.002 table 5.2.3a */
    { (1),(2),(3),(4),(5),(6),(7),(8),(9),(10),(11),(12),(13),(14),(15),0,(16),(17),(18),(19),(20),(21),(22),(23),(24),(25) },  ×8
    () = (0,0,1,0,0,1,0,1,1,1,0,0,0,0,1,0,0,1,0,0,1,0,1,1,1)
         (0,0,1,0,1,1,0,1,1,1,0,1,1,1,1,0,0,1,0,1,1,0,1,1,1)
         (0,1,0,0,0,0,1,1,1,0,1,1,1,0,1,0,1,0,0,0,0,1,1,1,0)
         (0,1,0,0,0,1,1,1,1,0,1,1,0,1,0,0,1,0,0,0,1,1,1,1,0)
         (0,0,0,1,1,0,1,0,1,1,1,0,0,1,0,0,0,0,1,1,0,1,0,1,1)
         (0,1,0,0,1,1,1,0,1,0,1,1,0,0,0,0,1,0,0,1,1,1,0,1,0)
         (1,0,1,0,0,1,1,1,1,1,0,1,1,0,0,1,0,1,0,0,1,1,1,1,1)
         (1,1,1,0,1,1,1,1,1,0,0,0,1,0,0,1,1,1,1,0,1,1,1,1,0,0)
};

int cellule_marge_nb = -1;    /* head margin of a normal burst; < 0: bare 148 samples */
int cellule_tsc_force = -1;   /* training sequence of the normal bursts; < 0: BCC of the BSIC */
double cellule_dec_nb = -1;
double cellule_phase_nb = 0;  /* carrier phase (deg) of the last normal burst (probe) */  /* sampling instant used
 ↪  for the last normal burst (probe) */
int cellule_fenetre_nb = 0;   /* one-shot NB window length in samples (0: 148 + 2 x margin) */
int cellule_sans_bcch;        /* 1: dummy bursts on BCCH/CCCH, the old cell */

static int env_int(const char *nom, int defaut)
{
    const char *e = calypso_getenv(nom);
    return (e && *e) ? (int)strtol(e, NULL, 0) : defaut;
}

/* The 23 octets of the L2 frame carried by BCCH block TC (0..7), or by a CCCH
 * block (tc < 0). Buffers are static, the caller copies. */
static const uint8_t *cellule_l2(int tc)
{
    static uint8_t si1[23], si2[23], si3[23], si4[23], pag[23];
    static int pret;
    if (!pret) {
        pret = 1;
        struct osmo_location_area_id lai = {
            .plmn = { .mcc = (uint16_t)env_int("CELLULE_MCC", 1),
                      .mnc = (uint16_t)env_int("CELLULE_MNC", 1), .mnc_3_digits = false },
            .lac = (uint16_t)env_int("CELLULE_LAC", 1) };
        uint16_t ci = (uint16_t)env_int("CELLULE_CI", 6001);
        int arfcn = env_int("CELLULE_ARFCN", 514) & 0x3ff;
        struct gsm48_rach_control rach = { .re = 1, .cell_bar = 0, .tx_integer = 9, .max_trans = 3,
                                           .t2 = 0x00, .t3 = 0x00 };
        struct gsm48_cell_sel_par csp = { .ms_txpwr_max_ccch = 0, .cell_resel_hyst = 2,
                                          .rxlev_acc_min = 0, .neci = 1, .acs = 0 };
        memset(si1, GSM_MACBLOCK_PADDING, 23); memset(si2, GSM_MACBLOCK_PADDING, 23);
        memset(si3, GSM_MACBLOCK_PADDING, 23); memset(si4, GSM_MACBLOCK_PADDING, 23);
        memset(pag, GSM_MACBLOCK_PADDING, 23);

        /* SI1: cell channel description in variable bitmap format (44.018
         * 10.5.2.13.7, the layout gsm48_decode_freq_list() reads): ORIG-ARFCN
         * = our carrier, no further bit set. Rest octet 0x2b: L, no NCH; L,
         * band indicator 1800. */
        struct gsm48_system_information_type_1 *s1 = (void *)si1;
        s1->header.l2_plen = (uint8_t)((21 << 2) | 1);
        s1->header.rr_protocol_discriminator = GSM48_PDISC_RR;
        s1->header.skip_indicator = 0;
        s1->header.system_information = GSM48_MT_RR_SYSINFO_1;
        memset(s1->cell_channel_description, 0, 16);
        s1->cell_channel_description[0] = (uint8_t)(0x8e | ((arfcn >> 9) & 1));
        s1->cell_channel_description[1] = (uint8_t)((arfcn >> 1) & 0xff);
        s1->cell_channel_description[2] = (uint8_t)((arfcn & 1) << 7);
        s1->rach_control = rach;
        {   /* self-check: decode what we encoded */
            static struct gsm_sysinfo_freq f[1024];
            memset(f, 0, sizeof f);
```

```
            gsm48_decode_freq_list(f, s1->cell_channel_description, 16, 0xce, 1);
            int n = 0, ok = 0;
            for (int i = 0; i < 1024; i++) if (f[i].mask) { n++; if (i == arfcn) ok = 1; }
            if (!ok || n != 1)
                printf("cellule : SI1 cell channel description FAUSSE (%d ARFCN decodes, %d attendu %s)\n",
                       n, arfcn, ok ? "present" : "ABSENT");
        }

        /* SI2: no neighbour (bitmap 0 all clear), all NCC permitted */
        struct gsm48_system_information_type_2 *s2 = (void *)si2;
        s2->header.l2_plen = (uint8_t)((22 << 2) | 1);
        s2->header.rr_protocol_discriminator = GSM48_PDISC_RR;
        s2->header.skip_indicator = 0;
        s2->header.system_information = GSM48_MT_RR_SYSINFO_2;
        memset(s2->bcch_frequency_list, 0, 16);
        s2->ncc_permitted = 0xff;
        s2->rach_control = rach;

        /* SI3: identity, combined CCCH, IMSI attach, no periodic LU. Rest
         * octets 0x2b: every optional element absent (all L). */
        struct gsm48_system_information_type_3 *s3 = (void *)si3;
        s3->header.l2_plen = (uint8_t)((18 << 2) | 1);
        s3->header.rr_protocol_discriminator = GSM48_PDISC_RR;
        s3->header.skip_indicator = 0;
        s3->header.system_information = GSM48_MT_RR_SYSINFO_3;
        s3->cell_identity = htons(ci);
        gsm48_generate_lai2(&s3->lai, &lai);
        s3->control_channel_desc.ccch_conf = 1;        /* 1 CCCH combined with SDCCH/4 */
        s3->control_channel_desc.bs_ag_blks_res = 1;
        s3->control_channel_desc.att = 1;
        s3->control_channel_desc.bs_pa_mfrms = 0;      /* 2 multiframes */
        s3->control_channel_desc.t3212 = 0;
        s3->cell_options.radio_link_timeout = 7;       /* 32 */
        s3->cell_options.dtx = 2;
        s3->cell_options.pwrc = 0;
        s3->cell_sel_par = csp;
        s3->rach_control = rach;

        /* SI4: identity again, no CBCH; rest octets all L */
        struct gsm48_system_information_type_4 *s4 = (void *)si4;
        s4->header.l2_plen = (uint8_t)((12 << 2) | 1);
        s4->header.rr_protocol_discriminator = GSM48_PDISC_RR;
        s4->header.skip_indicator = 0;
        s4->header.system_information = GSM48_MT_RR_SYSINFO_4;
        gsm48_generate_lai2(&s4->lai, &lai);
        s4->cell_sel_par = csp;
        s4->rach_control = rach;

        /* CCCH: PAGING REQUEST TYPE 1, page mode normal, no identity (the
         * fill osmo-bts sends on an idle paging block) */
        static const uint8_t vide[] = { 0x15, 0x06, 0x21, 0x00, 0x01, 0xf0 };
        memcpy(pag, vide, sizeof vide);

        printf("cellule : BCCH SI1-4 MCC=%03u MNC=%02u LAC=%u CI=%u ARFCN=%d, CCCH = paging vide\n",
               lai.plmn.mcc, lai.plmn.mnc, lai.lac, ci, arfcn);
    }
    if (tc < 0) return pag;
    switch (tc & 3) { case 0: return si1; case 1: return si2; case 2: return si3; default: return si4; }
}

/* Normal burst bid (0..3) of the block starting at fn0, or -1 if no block
 * starts there. The four bursts of one block are cached. */
static int cellule_nb(uint32_t fn0, int bid, uint8_t bsic, uint8_t bits[148])
{
    static uint32_t fn_cache = 0xffffffffu;
    static ubit_t bursts[4 * 116];
```

```
    if (fn0 != fn_cache) {
        uint32_t p51 = fn0 % 51;
        const uint8_t *l2;
        if (p51 == 2)                                   l2 = cellule_l2((int)((fn0 / 51) % 8));
        else if (p51 == 6 || p51 == 12 || p51 == 16)    l2 = cellule_l2(-1);
        else return -1;
        gsm0503_xcch_encode(bursts, l2);
        fn_cache = fn0;
    }
    /* CELLULE_NB_REPEAT=<k>: burst k of the block in all four positions, to
     * hand the ROM's decoder four bursts it is known to demodulate. */
    { static int rep = -2; if (rep == -2) rep = env_int("CELLULE_NB_REPEAT", -1); if (rep >= 0 && rep < 4) bid =
     ↪  rep; }
    const ubit_t *b = bursts + bid * 116;
    memset(bits, 0, 3);
    memcpy(bits + 3, b, 58);                    /* 57 data + hl */
    memcpy(bits + 61, train_nb[cellule_tsc_force >= 0 ? cellule_tsc_force & 7 : bsic & 7], 26);
    memcpy(bits + 87, b + 58, 58);              /* hu + 57 data */
    memset(bits + 145, 0, 3);
    return 0;
}

/* sb_info of 44.018 9.1.30 for frame fn, byte layout of osmo-bts
 * sched_lchan_fcch_sch.c. T3' = (T3 - 1) / 10: the SCH sits on T3 in
 * {1,11,21,31,41}, so T3' in 0..4. Previously computed as T3 / 10 here and as
 * (T3 - 1) / 10 in pont.c: identical on real SCH frames, different at T3 = 50
 * under cellule_sch_partout. One encoder now, shared by cellule_burst() and
 * cellule_code_attendu(). */
static void sb_info_de_fn(uint32_t fn, uint8_t bsic, uint8_t sb_info[4])
{
    uint32_t t1 = fn / 1326, t2 = fn % 26, t3 = fn % 51;
    uint32_t t3p = t3 ? (t3 - 1) / 10 : 0;
    sb_info[0] = (uint8_t)(((bsic & 0x3f) << 2) | ((t1 & 0x600) >> 9));
    sb_info[1] = (uint8_t)((t1 & 0x1fe) >> 1);
    sb_info[2] = (uint8_t)(((t1 & 0x001) << 7) | ((t2 & 0x1f) << 2) | ((t3p & 0x6) >> 1));
    sb_info[3] = (uint8_t)(t3p & 0x1);
}
/* Caller sets the head margin; the tail margin is trimmed to keep 190 complex
 * samples in total. Sliding the burst inside that fixed-size buffer separates an
 * influence window that is an ABSOLUTE buffer index (late bits lose influence,
 * early ones gain it) from one RELATIVE to the burst start (the profile moves
 * with the burst and keeps its shape). */
int cellule_marge_fin = -1;

char cellule_burst(uint32_t fn, uint8_t bsic, int amp, double decalage, int marge, int16_t *iq, int *n_iq)
{
    uint8_t bits[148];
    uint32_t p51 = fn % 51;
    char type;
    if (p51 % 10 == 0 && p51 <= 40) {
        memset(bits, 0, sizeof(bits));
        type = 'F';
    } else if ((p51 % 10 == 1 && p51 <= 41) || cellule_sch_partout) {
        uint8_t sb_info[4];
        sb_info_de_fn(fn, bsic, sb_info);
        ubit_t code[78];
        gsm0503_sch_encode(code, sb_info);
        /* Demodulator impulse response: flip exactly ONE of the 78 coded bits and
         * watch which soft-bit positions at 0x2c72 move.
         *   1 position          -> pure permutation, 78 runs give the whole table
         *   3 or 4 around one   -> normal ISI, the chain is sound there
         *   all of them         -> global, wrong traceback (on C54x: CMPS/TRN) */
        {
            /* The flip must stay inside ONE frame: otherwise it perturbs the whole
             * run history (scheduling, AFC) and the differential measurement then
             * compares two histories instead of two bursts.
```

```
         * REJEU_INVERSER_FN=<f> restricts the flip to frame f. */
        static int inv = -2; static long invfn = -2;
        if (inv == -2) { const char *e = calypso_getenv("REJEU_INVERSER_BIT"); inv = e ? atoi(e) : -1; }
        if (invfn == -2) { const char *e = calypso_getenv("REJEU_INVERSER_FN"); invfn = e ? atol(e) : -1; }
        if (inv >= 0 && inv < 78 && (invfn < 0 || (long)fn == invfn)) code[inv] ^= 1;
    }
    memset(bits, 0, 3);
    memcpy(bits + 3, code, 39);
    memcpy(bits + 42, train_sb, 64);
    memcpy(bits + 106, code + 39, 39);
    memset(bits + 145, 0, 3);
    /* Midamble probe: the 78 coded bits cannot reach the training sequence, yet
     * that sequence is what channel estimation is supposed to use. If flipping
     * one midamble bit leaves the soft bits untouched, the demodulator ignores
     * the known sequence and equalises against an estimate built elsewhere. */
    {
        /* Same one-frame confinement as the coded-bit flip; without it every SCH
         * burst of the run is hit. REJEU_INVERSER_FN is shared with
         * REJEU_INVERSER_BIT, so only one thing is probed at a time. */
        static int im = -2; static long imfn = -2;
        if (im == -2) { const char *e = calypso_getenv("REJEU_INVERSER_MIDAMBULE");
                        im = e ? atoi(e) : -1; }
        if (imfn == -2) { const char *e = calypso_getenv("REJEU_INVERSER_FN");
                          imfn = e ? atol(e) : -1; }
        if (im >= 0 && im < 64 && (imfn < 0 || (long)fn == imfn)) bits[42 + im] ^= 1;
    }
    type = 'S';
} else if (!cellule_sans_bcch && p51 >= 2 && p51 <= 49 && p51 % 10 >= 2 &&
           cellule_nb(fn - ((p51 % 10 - 2) & 3), (int)((p51 % 10 - 2) & 3), bsic, bits) == 0) {
    type = (p51 <= 5) ? 'B' : 'C';
} else {
    memcpy(bits, factice, 148);
    type = '.';
}
/* CELLULE_NB_FINE=1 : burst position swept from 2.3 to 4.7 samples in 0.1
 * steps by multiframe (head margin + sampling instant together) */
if (type == 'B' || type == 'C') {
    static int fine = -2; if (fine == -2) fine = env_int("CELLULE_NB_FINE", 0);
    if (fine && cellule_marge_nb >= 0) {
        double total = 2.3 + 0.1 * (double)((fn / 51u) % 25u);
        cellule_marge_nb = (int)total; decalage = total - (double)cellule_marge_nb;
        cellule_dec_nb = decalage;
    }
}
int m_tete = -1, m_fin = 0;
if (type == 'S' && marge > 0) {
    m_tete = marge;
    m_fin = (cellule_marge_fin >= 0) ? cellule_marge_fin : marge;
} else if ((type == 'B' || type == 'C') && cellule_marge_nb >= 0) {
    /* the ROM takes 151 samples for an NB window (ALGTH 604): 3 + 148 */
    /* exactly the window: a frame longer than the DMA window leaves
     * samples in the RIF and the next burst starts on them */
    m_tete = cellule_marge_nb;
    m_fin = cellule_fenetre_nb > m_tete + 148 ? cellule_fenetre_nb - m_tete - 148 : 0;
}
/* CELLULE_NB_AMP=<n>: amplitude of the normal bursts alone (FCCH/SCH keep
 * amp), to probe the ROM's fixed-point headroom on the NB path. */
if (type == 'B' || type == 'C') { static int nb_amp = -2; if (nb_amp == -2) nb_amp = env_int("CELLULE_NB_AMP",
 ↪  -1); if (nb_amp > 0) amp = nb_amp; }
/* CELLULE_NB_DEC=<x>|auto : sampling instant of the normal bursts alone
 * (FCCH/SCH keep decalage); auto sweeps 0, 0.25, 0.5, 0.75 by multiframe */
if (type == 'B' || type == 'C') {
    static int mode = -2; static double d = 0;
    if (mode == -2) { const char *e = calypso_getenv("CELLULE_NB_DEC"); mode = 0;
                      if (e && !strcmp(e, "auto")) mode = 2; else if (e && *e) { mode = 1; d = atof(e); } }
    if (mode == 1) decalage = d; else if (mode == 2) decalage = 0.25 * (double)((fn / 51u) % 4u);
```

```
    cellule_dec_nb = decalage;
}
/* CELLULE_NB_SHIFT=<n> (experiment): transmit the normal burst with its
 * bits shifted n positions earlier (bit i+n at position i, zeros at the
 * end). Why: the ROM reads the training sequence of its equaliser output
 * at index 58 of the decision buffer (bit 61 with 3 tail bits skipped),
 * but the emulated equaliser puts bit i at index i-2, so the 26 TSC
 * decisions agree with TSC2 at 12/26 as read and 26/26 shifted by one:
 * the residual estimate then sees no signal, the soft-bit scale collapses
 * to 2 and every quantised soft bit is +1. Shifting the bits by one is the
 * test of that reading; it is not a fix. */
if (type == 'B' || type == 'C') {
    static int nsh = -2; if (nsh == -2) nsh = env_int("CELLULE_NB_SHIFT", 0);
    if (nsh > 0) { for (int i = 0; i < 148; i++) bits[i] = (i + nsh < 148) ? bits[i + nsh] : 0; }
    else if (nsh < 0) { for (int i = 147; i >= 0; i--) bits[i] = (i + nsh >= 0) ? bits[i + nsh] : 0; }
}
/* CELLULE_NB_PHASE=<deg>|auto : carrier phase of the normal bursts (auto:
 * 0, 22.5, 45, 67.5 degrees by multiframe). Samples exactly on the I/Q
 * axes (decalage 0, phase 0) or exactly on the diagonals (decalage 0.5)
 * are what a synthetic GMSK gives and what no radio ever gives. */
double phase0 = 0.0;
if (type == 'B' || type == 'C') {
    static int pm = -2; static double pd = 0;
    if (pm == -2) { const char *e = calypso_getenv("CELLULE_NB_PHASE"); pm = 0;
                    if (e && !strcmp(e, "auto")) pm = 2; else if (e && *e) { pm = 1; pd = atof(e); } }
    if (pm == 1) phase0 = pd * M_PI / 180.0; else if (pm == 2) phase0 = 22.5 * (double)((fn / 51u) % 4u) * M_PI
     ↪  / 180.0;
    /* CELLULE_NB_PHASE=quad : 0, 90, 180, 270 degrees by multiframe */
    { static int q = -1; if (q < 0) { const char *e = calypso_getenv("CELLULE_NB_PHASE"); q = (e && !strcmp(e,
     ↪  "quad")) ? 1 : 0; }
      if (q) phase0 = 90.0 * (double)((fn / 51u) % 4u) * M_PI / 180.0; }
    cellule_phase_nb = phase0 * 180.0 / M_PI;
}
/* CELLULE_NB_MSK=1 (experiment): pure MSK for the normal bursts, no
 * Gaussian filter: the phase advances linearly by +-90 degrees per bit,
 * so a sample taken at decalage 0.5 sits EXACTLY on a diagonal, with
 * |I| = |Q|. Tests whether the ROM's NB demodulator hard-decides on the
 * signs of I and Q (measured: only decalage 0.5 ever works, 0.4 and 0.6
 * fail on every burst, an equaliser would degrade smoothly). */
static int nb_msk = -2; if (nb_msk == -2) nb_msk = env_int("CELLULE_NB_MSK", 0);
if (nb_msk && (type == 'B' || type == 'C')) {
    int16_t *o = iq + 2 * (m_tete >= 0 ? m_tete : 0);
    double ph = phase0; int prev = 1;
    for (int k = 0; k < 148; k++) {
        int d = (bits[k] & 1) ^ prev; prev = bits[k] & 1;
        double al = 1.0 - 2.0 * d;
        double pk = ph + al * (M_PI / 2.0) * decalage;     /* phase at the sampling instant */
        o[2*k] = (int16_t)lrint(amp * cos(pk)); o[2*k+1] = (int16_t)lrint(amp * sin(pk));
        ph += al * (M_PI / 2.0);
    }
    if (m_tete >= 0) {
        memset(iq, 0, (size_t)m_tete * 2 * sizeof(int16_t));
        memset(iq + 2 * (m_tete + 148), 0, (size_t)m_fin * 2 * sizeof(int16_t));
        *n_iq = 2 * (148 + m_tete + m_fin);
    } else *n_iq = 2 * 148;
    return type;
}
/* CELLULE_SB_AMP=<n>: amplitude of the SCH only (FCCH and NB keep amp).
 * The ROM's SNR on the SCH saturates at 16384. */
if (type == 'S') { static int sba = -2; if (sba == -2) sba = env_int("CELLULE_SB_AMP", -1); if (sba > 0) amp =
 ↪  sba; }
/* CELLULE_SB_PHASE=<deg> : carrier phase of the SCH burst */
if (type == 'S') { static int sp = -2; if (sp == -2) sp = env_int("CELLULE_SB_PHASE", 0); phase0 = sp * M_PI /
 ↪  180.0; }
int16_t *burst_iq = iq;
int fen_n = 148;              /* samples covered by the SCH spread and noise */
```

```
/* CELLULE_SB_GARDE=<mode>: THE GUARD IS NOT SILENCE. On the GSM downlink
 * the BTS transmits continuously: the SCH is preceded by the TS7 signal and
 * followed by TS1, not by 21 null samples. The ROM's equaliser never sees a
 * silence -> signal edge on silicon, and it misreads the FIRST information
 * bits (bits 5-15 of the word) when given one. Here the whole window
 * (margin, burst, margin) is modulated in a single pass, the margins
 * carrying guard bits: 1 = zeros, 2 = pseudo-random (seed = fn), 3 = ones.
 * 0 = plain silence margins. */
if (m_tete >= 0) {
    static int garde = -2; if (garde == -2) garde = env_int("CELLULE_SB_GARDE", 0);
    int tot = m_tete + 148 + m_fin;
    if (type == 'S' && garde > 0 && tot <= 512) {
        uint8_t ext[512]; uint32_t seed = fn * 2654435761u + 99u;
        for (int k = 0; k < tot; k++) {
            int i = k - m_tete;
            if (i >= 0 && i < 148) ext[k] = bits[i];
            else if (garde == 1) ext[k] = 0;
            else if (garde == 3) ext[k] = 1;
            else { seed = seed * 1103515245u + 12345u; ext[k] = (uint8_t)((seed >> 16) & 1); }
        }
        { static double fc = -2; if (fc == -2) { const char *e = calypso_getenv("CELLULE_SB_FC"); fc = (e &&
         ↪  *e) ? atof(e) : 0; }
          if (fc > 0) gmsk_moduler_filtre(ext, tot, amp, phase0, decalage, fc, iq);
          else gmsk_moduler(ext, tot, amp, phase0, decalage, iq); }
        *n_iq = 2 * tot;
        burst_iq = iq;            /* spread and noise over the whole window */
        fen_n = tot;
    } else {
        memset(iq, 0, (size_t)m_tete * 2 * sizeof(int16_t));
        { static double fc = -2; if (fc == -2) { const char *e = calypso_getenv("CELLULE_SB_FC"); fc = (e &&
         ↪  *e) ? atof(e) : 0; }
          /* CELLULE_SB_FC=<kHz>: modelled receive chain (see gmsk_moduler_filtre) */
         if (type == 'S' && fc > 0) gmsk_moduler_filtre(bits, 148, amp, phase0, decalage, fc, iq + 2 * m_tete);
          else gmsk_moduler(bits, 148, amp, phase0, decalage, iq + 2 * m_tete); }
        memset(iq + 2 * (m_tete + 148), 0, (size_t)m_fin * 2 * sizeof(int16_t));
        *n_iq = 2 * (148 + m_tete + m_fin);
        burst_iq = iq + 2 * m_tete;
    }
} else {
    gmsk_moduler(bits, 148, amp, phase0, decalage, iq);
    *n_iq = 2 * 148;
}
/* [2026-09-21] CELLULE_NB_ISI=<h1>[,<h2>[,<h3>]] : causal tail on the normal
 * bursts, y[n] = x[n] + h1 x[n-1] + h2 x[n-2] + h3 x[n-3]. Why: the ROM
 * correlates the 16 central TSC bits over 10 lags, then picks the 7-lag
 * window of maximum energy (0x8551, sliding sum) and cuts its 5-tap
 * channel estimate from it. Our GMSK at 1 sample/symbol has energy on 3
 * lags only (main + the +-1 ISI), so three of the four windows tie to 0.1 %
 * and the choice is decided by e[lag 8] against e[lag 1] = the GMSK +-2
 * tap (912) against the data leakage: measured over 9 bursts, the window
 * was right (s=2) exactly when e[8] > e[1]. A receiver's analogue filter
 * spreads energy over the following lags and settles it; this tail does
 * the same for the synthetic cell. */
if (type == 'B' || type == 'C') {
    static int isi_n = -2; static double h[4];
    if (isi_n == -2) { isi_n = 0; const char *e = calypso_getenv("CELLULE_NB_ISI");
        if (e && *e) { char tmp[64]; strncpy(tmp, e, sizeof tmp - 1); tmp[sizeof tmp - 1] = 0;
            for (char *t = strtok(tmp, ","); t && isi_n < 3; t = strtok(NULL, ",")) h[++isi_n] = atof(t); } }
    if (isi_n > 0) {
        double xi[148], xq[148];
        for (int k = 0; k < 148; k++) { xi[k] = burst_iq[2*k]; xq[k] = burst_iq[2*k+1]; }
        for (int k = 0; k < 148; k++) {
            double yi = xi[k], yq = xq[k];
            for (int d = 1; d <= isi_n; d++) if (k - d >= 0) { yi += h[d] * xi[k-d]; yq += h[d] * xq[k-d]; }
            if (yi > 32767) yi = 32767;
            if (yi < -32768) yi = -32768;
```

```
            if (yq > 32767) yq = 32767;
            if (yq < -32768) yq = -32768;
            burst_iq[2*k] = (int16_t)lrint(yi); burst_iq[2*k+1] = (int16_t)lrint(yq);
        }
    }
}
/* [2026-09-21] CELLULE_NB_SYM=<a> : symmetric spread y[n] = x[n] + a (x[n-1]
 * + x[n+1]) on the normal bursts, timing unchanged. Why: the ROM zeroes
 * every channel tap whose energy is below 1/16 of the window energy
 * (0x7f0c-0x7f1c, threshold = total >> 4, i.e. 25 % in amplitude). The
 * +-1 taps of our GMSK at 1 sample/symbol are 25 % (6467..7189 against a
 * main tap of 27000): measured, the pre-cursor tap was kept (e = 4.70e7 >
 * 4.50e7) on the bursts that decoded and zeroed (4.18e7 < 4.25e7) on the
 * ones that did not, and a 5-tap model without its 25 % pre-cursor gives
 * 45 % errors. A receiver's channel filter widens the pulse; a = 0.3 puts
 * the +-1 taps near 55 % and the +-2 taps near 10 %, both far from the
 * threshold, whatever the data. */
if (type == 'B' || type == 'C') {
    /* default 0.3 since 2026-09-21 (the value that decodes SI1-4); 0 disables */
    static double a = -2; if (a == -2) { const char *e = calypso_getenv("CELLULE_NB_SYM"); a = (e && *e) ?
     ↪  atof(e) : 0.3; }
    gmsk_elargir(burst_iq, 148, a);
}
/* CELLULE_SB_SYM=<a>: the same spread on the SCH. Without it the ROM
 * returns CRC OK on about half the SCHs, with only bits 0-2 of the SB word
 * wrong (...1b/1f/1d instead of ...1c): the same burst positions flip
 * depending on the content, the "razor's edge" profile of the NBs without
 * CELLULE_NB_SYM. 0 by default. */
if (type == 'S') {
    static double a = -2; if (a == -2) { const char *e = calypso_getenv("CELLULE_SB_SYM"); a = (e && *e) ?
     ↪  atof(e) : 0.0; }
    if (a != 0.0) gmsk_elargir(burst_iq, fen_n, a);
}
/* CELLULE_SB_ISI=<h1>[,<h2>[,<h3>]]: the CAUSAL tail of CELLULE_NB_ISI, on
 * the SCH. Same mechanism as for the NBs: the ROM picks the window of its
 * channel estimate at the energy maximum over lags, our GMSK at 1
 * sample/symbol leaves the windows tied and the neighbouring data decides;
 * the misses then fall on the half BEFORE the training sequence and are
 * fixed by the content. A causal tail (like the analogue filter of a real
 * receiver) breaks the tie. Normalised by 1+sum(h) to stay within 16 bits. */
if (type == 'S') {
    static int isi_n = -2; static double h[4];
    if (isi_n == -2) { isi_n = 0; const char *e = calypso_getenv("CELLULE_SB_ISI");
        if (e && *e) { char tmp[64]; strncpy(tmp, e, sizeof tmp - 1); tmp[sizeof tmp - 1] = 0;
            for (char *t = strtok(tmp, ","); t && isi_n < 3; t = strtok(NULL, ",")) h[++isi_n] = atof(t); } }
    if (isi_n > 0) {
        double norm = 1.0; for (int d = 1; d <= isi_n; d++) norm += fabs(h[d]);
        double xi[512], xq[512];
        for (int k = 0; k < fen_n; k++) { xi[k] = burst_iq[2*k]; xq[k] = burst_iq[2*k+1]; }
        for (int k = 0; k < fen_n; k++) {
            double yi = xi[k], yq = xq[k];
            for (int d = 1; d <= isi_n; d++) if (k - d >= 0) { yi += h[d] * xi[k-d]; yq += h[d] * xq[k-d]; }
            burst_iq[2*k] = (int16_t)lrint(yi / norm); burst_iq[2*k+1] = (int16_t)lrint(yq / norm);
        }
    }
}
/* CELLULE_SB_NOISE=<sigma>: the same Gaussian noise on the SCH (see
 * CELLULE_NB_NOISE below: without noise, the scale of the ROM's soft bits
 * collapses). 0 = unchanged. */
if (type == 'S') {
    static double sigma = -2; if (sigma == -2) { const char *e = calypso_getenv("CELLULE_SB_NOISE"); sigma =
     ↪  (e && *e) ? atof(e) : 0.0; }
    if (sigma > 0) {
        static uint32_t graine = 0xffffffffu;   /* CELLULE_SB_NOISE_SEED: another noise realisation */
        if (graine == 0xffffffffu) graine = (uint32_t)env_int("CELLULE_SB_NOISE_SEED", 0);
        uint32_t seed = fn * 2654435761u + 777u + graine * 7919u;
```

```
            for (int k = 0; k < 2 * fen_n; k++) {
                seed = seed * 1103515245u + 12345u; double u1 = ((seed >> 8) & 0xffff) / 65536.0 + 1e-6;
                seed = seed * 1103515245u + 12345u; double u2 = ((seed >> 8) & 0xffff) / 65536.0;
                double g = sqrt(-2.0 * log(u1)) * cos(2.0 * M_PI * u2);
                double v = burst_iq[k] + sigma * g;
                if (v > 32767) v = 32767;
                if (v < -32768) v = -32768;
                burst_iq[k] = (int16_t)lrint(v);
            }
        }
    }
    /* [2026-09-21] CELLULE_NB_NOISE=<sigma> : Gaussian noise on the normal
     * bursts (deterministic seed per frame). Why: the ROM scales its soft bits
     * by a noise estimate before the 4-bit quantiser (0x8168 -> 0x82d0, a
     * 129-entry table indexed by soft >> 8); with a noiseless burst the scaled
     * values are 2..5 instead of thousands, every index is 0 and all 116
     * quantised soft bits come out +1 (measured in the storage at 0x4200 +
     * 29 x burst): the sign is lost before the deinterleaver. A radio always
     * carries noise; the cell now does too. */
    if (type == 'B' || type == 'C') {
        static double sigma = -2; if (sigma == -2) { const char *e = calypso_getenv("CELLULE_NB_NOISE"); sigma =
        ↪  (e && *e) ? atof(e) : 0.0; }
        if (sigma > 0) {
            uint32_t seed = fn * 2654435761u + 12345u;
            for (int k = 0; k < 296; k++) {
                /* Box-Muller on a small LCG */
                seed = seed * 1103515245u + 12345u; double u1 = ((seed >> 8) & 0xffff) / 65536.0 + 1e-6;
                seed = seed * 1103515245u + 12345u; double u2 = ((seed >> 8) & 0xffff) / 65536.0;
                double g = sqrt(-2.0 * log(u1)) * cos(2.0 * M_PI * u2);
                double v = burst_iq[k] + sigma * g;
                if (v > 32767) v = 32767;
                if (v < -32768) v = -32768;
                burst_iq[k] = (int16_t)lrint(v);
            }
        }
    }
    /* [2026-09-21] CELLULE_NB_ZERO_DC=1 (experiment): remove the mean of the
     * 148 samples of a normal burst. Measured: the ROM demodulates a normal
     * burst perfectly when the data-induced mean of its samples is below ~7 %
     * of the amplitude and loses it entirely above ~10 % (16 bursts of SI1-4,
     * no exception), the SCH being immune. */
    if (type == 'B' || type == 'C') {
        static int zdc = -2; if (zdc == -2) zdc = env_int("CELLULE_NB_ZERO_DC", 0);
        if (zdc) {
            long si = 0, sq = 0;
            for (int k = 0; k < 148; k++) { si += burst_iq[2*k]; sq += burst_iq[2*k+1]; }
            int mi = (int)(si / 148), mq = (int)(sq / 148);
            for (int k = 0; k < 148; k++) { burst_iq[2*k] = (int16_t)(burst_iq[2*k] - mi); burst_iq[2*k+1] =
            ↪  (int16_t)(burst_iq[2*k+1] - mq); }
        }
    }
    return type;
}

/* Expected 78 coded bits for a frame, to compare sign by sign with the soft bits
 * the DSP produces. */
void cellule_code_attendu(uint32_t fn, uint8_t bsic, unsigned char *code78)
{
    uint8_t sb_info[4];
    sb_info_de_fn(fn, bsic, sb_info);
    ubit_t code[78];
    gsm0503_sch_encode(code, sb_info);
    for (int i = 0; i < 78; i++) code78[i] = (unsigned char)code[i];
}

/* Reference demodulator, in C, to prove the samples handed to the DSP do carry the
```

```
 * message. GMSK at 1 sample/symbol: the phase advances by (pi/2)*alpha_n per symbol,
 * so alpha_n = sign(arg(x[n] * conj(x[n-1]))). Recover the alpha sequence, correlate
 * it with the midamble to find the offset, then read the data bits from there. */
#include <math.h>
int cellule_demod_reference(const int16_t *x, int n_ech, unsigned char *bits148, int *offset)
{
    static double alpha[512];
    if (n_ech > 512) n_ech = 512;
    for (int n = 1; n < n_ech; n++) {
        double i0 = x[2*(n-1)], q0 = x[2*(n-1)+1];
        double i1 = x[2*n],     q1 = x[2*n+1];
        /* x[n] * conj(x[n-1]) */
        double re = i1*i0 + q1*q0, im = q1*i0 - i1*q0;
        alpha[n] = atan2(im, re);
    }
    alpha[0] = 0;
    /* reference alpha of the midamble: d_j = t_j XOR t_{j-1}, burst positions 43..105 */
    double ref[63];
    for (int j = 1; j < 64; j++) ref[j-1] = (train_sb[j] ^ train_sb[j-1]) ? -1.0 : 1.0;
    int best = -1; double bestv = -1e30;
    for (int k = 0; k + 63 < n_ech; k++) {
        double acc = 0;
        for (int j = 0; j < 63; j++) acc += ref[j] * alpha[k + j];
        if (acc > bestv) { bestv = acc; best = k; }
    }
    if (best < 0) return -1;
    /* the midamble starts 43 symbols into the burst */
    int b0 = best - 43;
    *offset = b0;
    if (b0 < 0 || b0 + 148 > n_ech) return -1;
    int prev = 1;
    for (int i = 0; i < 148; i++) {
        int d = (alpha[b0 + i] < 0) ? 1 : 0;   /* alpha = 1-2d */
        bits148[i] = (unsigned char)(d ^ prev);
        prev = bits148[i];
    }
    return 0;
}

/* Raw DIFFERENTIAL bits d_i, with no differential decoding: an isolated error does
 * not propagate here, unlike in b_i = d_i XOR b_{i-1}. This is the honest measure of
 * demodulator quality. */
int cellule_demod_d(const int16_t *x, int n_ech, int b0, unsigned char *d148)
{
    if (b0 < 1 || b0 + 148 > n_ech) return -1;
    for (int i = 0; i < 148; i++) {
        int n = b0 + i;
        double i0 = x[2*(n-1)], q0 = x[2*(n-1)+1];
        double i1 = x[2*n],     q1 = x[2*n+1];
        double im = q1*i0 - i1*q0, re = i1*i0 + q1*q0;
        d148[i] = (unsigned char)(atan2(im, re) < 0 ? 1 : 0);
    }
    return 0;
}

int cellule_train_sb(int i) { return (i >= 0 && i < 64) ? train_sb[i] : -1; }

/* The dummy burst (45.002 5.2.6) as 148 GMSK samples: what the BCCH carrier
 * transmits on every timeslot that carries nothing else. The FB search of the
 * ROM receives the whole frame, so the seven other timeslots must look like a
 * real C0 carrier, not like silence. */
void cellule_factice(int amp, double decalage, int16_t *iq)
{
    gmsk_moduler(factice, 148, amp, 0.0, decalage, iq);
}
```

```c
/* The 148 bits of the TN0 burst of frame fn as cellule_burst() sends them, for
 * probes that look for them inside the DSP memory. 0 if it is a normal burst. */
int cellule_bits_attendus(uint32_t fn, uint8_t bsic, uint8_t bits[148])
{
    uint32_t p51 = fn % 51;
    if (cellule_sans_bcch || p51 < 2 || p51 > 49 || p51 % 10 < 2) return -1;
    return cellule_nb(fn - ((p51 % 10 - 2) & 3), (int)((p51 % 10 - 2) & 3), bsic, bits);
}

/* Expected decoder-side vectors for the block that ends at frame fn (burst 3):
 * the 456 convolutionally coded bits in deinterleaved order (45.003 4.1.4:
 * bit k of the coded block sits in burst k mod 4 at position 2*((49k) mod 57)
 * + ((k mod 8) div 4) of the 114 data bits) and the 184 information bits +
 * 40 parity + 4 tail (the 23 octets as sent, MSB first, then the Fire parity
 * as gsm0503 computes it: we only need the 184 here). */
int cellule_bloc_attendu(uint32_t fn, uint8_t bsic, uint8_t code456[456], uint8_t info184[184])
{
    uint32_t p51 = fn % 51;
    if (cellule_sans_bcch || p51 < 2 || p51 > 49 || p51 % 10 < 2) return -1;
    uint32_t fn0 = fn - ((p51 % 10 - 2) & 3);
    uint8_t bits[148]; uint8_t data[4][114];
    for (int b = 0; b < 4; b++) {
        if (cellule_nb(fn0, b, bsic, bits) < 0) return -1;
        memcpy(data[b], bits + 3, 57); memcpy(data[b] + 57, bits + 88, 57);
    }
    for (int k = 0; k < 456; k++) {
        int b = k & 3, j = 2 * ((49 * k) % 57) + ((k % 8) / 4);
        code456[k] = data[b][j];
    }
    uint32_t tc = (fn0 / 51) % 8;
    const uint8_t *l2 = (p51 <= 5) ? cellule_l2((int)tc) : cellule_l2(-1);
    /* [2026-09-21] LSB first within each octet: that is how gsm0503_xcch_encode
     * unpacks the L2 frame before the Fire parity and the convolutional code
     * (osmo_pbit2ubit_ext(..., lsb_mode=1)), so it is the bit order the ROM's
     * Viterbi has to reproduce. MSB first gave a false "78 faux" on a correct
     * block. */
    for (int i = 0; i < 184; i++) info184[i] = (l2[i / 8] >> (i % 8)) & 1;
    return 0;
}

/* The 228 bits the Viterbi decoder must output: 184 information bits, 40 Fire
 * parity bits, 4 tail bits (45.003 4.1.1-4.1.2). */
#include <osmocom/core/crc64gen.h>
#include <osmocom/coding/gsm0503_parity.h>
int cellule_u228_attendu(uint32_t fn, uint8_t bsic, uint8_t u228[228])
{
    uint8_t code[456], info[184];
    if (cellule_bloc_attendu(fn, bsic, code, info) < 0) return -1;
    memcpy(u228, info, 184);
    osmo_crc64gen_set_bits(&gsm0503_fire_crc40, info, 184, u228 + 184);
    memset(u228 + 224, 0, 4);
    return 0;
}
```

3.14 /opt/GSM/c54x_exe/src/cellule.h

1880 octets, 29 lignes → 29 lignes

```c
/* SPDX-License-Identifier: GPL-2.0-or-later */
#ifndef CELLULE_H
#define CELLULE_H
#include <stdint.h>
/* Downlink TN0 burst of a GSM cell for frame fn (51-multiframe: FCCH on
 * 0/10/20/30/40, SCH on 1/11/21/31/41, dummy burst elsewhere), GMSK modulated,
```

```
 * 148 int16 I/Q samples. Returns the burst type: 'F', 'S', 'B' (BCCH, SI1-4),
 * 'C' (CCCH, empty paging) or '.' (dummy).
 * marge = silence samples prepended and appended to an SCH burst: the ROM SB
 * decoder reads a 190-sample window, 148 + 2 x 21 (BSP_IQ_MAX_I16 in
 * calypso_bsp.c, "SB en demande 190"). *n_iq gets the number of int16 written. */
extern int cellule_sch_partout;
extern int cellule_marge_fin;
extern int cellule_marge_nb;     /* head/tail margin of BCCH/CCCH normal bursts, < 0 = none (pont.c sets it per
 ↪  frame) */
extern int cellule_sans_bcch;
extern double cellule_dec_nb;
extern double cellule_phase_nb;
extern int cellule_fenetre_nb;   /* NB window length in samples, the frame is padded to it exactly */
extern int cellule_tsc_force;    /* TSC of the normal bursts, < 0 = BCC (CELLULE_TSC, pont.c) */    /* 1: the old
 ↪  cell, dummy bursts on BCCH/CCCH */
int cellule_train_sb(int i);
int cellule_demod_d(const int16_t *x, int n_ech, int b0, unsigned char *d148);
int cellule_demod_reference(const int16_t *x, int n_ech, unsigned char *bits148, int *offset);
void cellule_code_attendu(uint32_t fn, uint8_t bsic, unsigned char *code78);
char cellule_burst(uint32_t fn, uint8_t bsic, int amp, double decalage, int marge, int16_t *iq, int *n_iq);
int cellule_u228_attendu(uint32_t fn, uint8_t bsic, uint8_t u228[228]);
int cellule_bloc_attendu(uint32_t fn, uint8_t bsic, uint8_t code456[456], uint8_t info184[184]);
int cellule_bits_attendus(uint32_t fn, uint8_t bsic, uint8_t bits[148]);
void cellule_factice(int amp, double decalage, int16_t *iq);   /* 148 GMSK samples of the dummy burst */
#endif
```

3.15 /opt/GSM/c54x_exe/src/main.c

12823 octets, 279 lignes → 279 lignes

```
/* SPDX-License-Identifier: GPL-2.0-or-later */
/*
 * c54x_exe - the Calypso TMS320C54x DSP, run without QEMU and without the ARM.
 *
 * Bench for the symptom recorded in the qemu-calypso README [2026-09-16]:
 * a_sch[0] = 0x8100 (B_BLUD | B_SCH_CRC), and a_sch[3] = 0xf8d8 CONSTANT over
 * 21/21 writes while 10 distinct burst contents were presented. A decoder
 * whose output does not depend on its input is not decoding.
 *
 * Reaching that point used to mean booting QEMU, the ARM and the osmocom-bb
 * firmware: seconds per run. The DSP needs almost none of it - out of 26631
 * lines of C54x L1, 14 symbols come from QEMU (two of them mutexes in the
 * core), and calypso_{arm2dsp,dma,fbsb,mailbox}.c need none. Here the TI mask
 * ROM runs alone in milliseconds and we watch what the DSP writes into API
 * RAM, which makes the question "does the output depend on the input?"
 * answerable in a loop, hence in CI.
 *
 * The sources are NOT copied here: this binary compiles those of
 * /opt/GSM/qosmo. Copying them would recreate the divergence this bench exists
 * to remove.
 */
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include "qemu/thread.h"
#include "calypso_c54x.h"
#include "hw/arm/calypso/calypso_api.h"
#include "hw/arm/calypso/calypso_dsp_pont.h"
#include "calypso_dma.h"
#include "calypso_bsp.h"
#include "verbosite.h"
#include "rejouer.h"
```

```c
#include "pont.h"

/* ── what the platform would otherwise provide ──────────────────────── */
extern int c54x_rapide;      /* calypso_c54x.h : fast path of the core */
extern int c54x_sondes;      /* c54x_internal.h: pure probes, off by default */
uint32_t g_c54x_exe_fn;      /* non-static: pont.c updates it on every TICK */
uint32_t calypso_trx_get_fn(void) { return g_c54x_exe_fn; }

/* No interrupt controller here, so the ack the DSP raises has nowhere to go.
 * If DSP behaviour ever turns out to depend on that ack, this binary diverges
 * from QEMU at exactly this point. */
void calypso_inth_arm_ack(void) { }

/* The DARAM lock normally lives in l1-dsp/calypso_full_pcb.c, excluded from
 * this binary because it includes hw/core/cpu.h - all of QEMU behind it. Same
 * pthread_mutex: the DSP really locks. */
QemuMutex calypso_pcb_daram_lock;

/* Guest memory: with no ARM nobody writes into it, but shared sources
 * reference it. */
void cpu_physical_memory_rw(uint64_t addr, void *buf, uint64_t len, bool wr)
{
    (void)addr;
    if (!wr) {
        memset(buf, 0, len);
    }
}

/* ── ROMs at their silicon addresses (cf. calypso_trx.c of qosmo-dsp) ──── */
static const struct { const char *suffixe; uint32_t adresse; bool programme; }
ROMS[] = {
    { "PROM0",  0x07000, true  },
    { "PROM1",  0x18000, true  },   /* page 1, reached with XPC=1 */
    { "PROM2",  0x28000, true  },
    { "PROM3",  0x38000, true  },
    { "DROM",   0x09000, false },
    { "PDROM",  0x0E000, false },   /* mapped on the DATA side ... */
    { "PDROM",  0x0E000, true  },   /* ... AND on the PROGRAM side (IT vectors) */
};

static void usage(const char *prog)
{
    fprintf(stderr,
        "usage: %s [options]\n"
        "  --rom-dir DIR     ou sont calypso_dsp.*.bin   (defaut /opt/GSM)\n"
        "  --trames N        nombre de trames TDMA       (defaut 100, 4000 avec --rejouer)\n"
        "  --insns N         instructions par trame      (defaut 2300, 200000 avec --arm)\n"
        "  --verbeux         une ligne par trame\n"
        "  --iq MODE         avec --arm : injecter un burst I/Q a chaque trame :\n"
        "                    fcch (rotation +pi/2/ech.), noise, tone:<dphi>, none,\n"
        "                    cell[:bsic[:decalage]] = FCCH+SCH+factice GMSK (multitrame 51)\n"
        "  --amp N           amplitude int16 des echantillons injectes (defaut 30000)\n"
        "  --rejouer         rejeu DETERMINISTE de l'acquisition FB/SB (sans QEMU)\n"
        "  --bsic N          BSIC de la cellule injectee en rejeu (defaut 7)\n"
        "  --arm [SOCKET]    servir l'ARM de QEMU (qosmo, CALYPSO_DSP_EXTERN=1) :\n"
        "                    API RAM partagee dans /dev/shm%s, trame verrouillee\n"
        "                    sur %s\n"
        "  -v .. -vvvvvv     traces du coeur C54x (stderr), par niveau :\n"
        "                    (rien)  erreurs seulement\n"
        "                    -v      + avertissements\n"
        "                    -vv     + cycle de vie : boot, reset, gates ACTIF/INACTIVE\n"
        "                    -vvv    + taches, API RAM, interruptions, chemins FB/SB\n"
        "                    -vvvv   + sondes memoire : WATCH, DUMP, SCAN, MAP, SP-*\n"
        "                    -vvvvv  + pas a pas : BRANCH-TRACE, LOOPTRACE, TERM, CYCLE\n"
        "                    -vvvvvv tout, stderr brut\n",
        prog, CALYPSO_PONT_SHM, CALYPSO_PONT_SOCK);
```

```
}

int main(int argc, char **argv)
{
    const char *rom_dir = "/opt/GSM";
    const char *arm_sock = NULL;
    int rejeu = 0, bsic_rej = 7;
    const char *iq_mode = "none";
    int amp = 30000;
    long trames = -1, insns = -1;
    bool verbeux = false;
    int niveau = 0;

    for (int i = 1; i < argc; i++) {
        const char *a = argv[i];
        if (!strcmp(a, "--rom-dir") && i + 1 < argc)      rom_dir = argv[++i];
        else if (!strcmp(a, "--trames") && i + 1 < argc)  trames = atol(argv[++i]);
        else if (!strcmp(a, "--insns") && i + 1 < argc)   insns = atol(argv[++i]);
        else if (!strcmp(a, "--verbeux"))                 verbeux = true;
        else if (!strcmp(a, "--iq") && i + 1 < argc)      iq_mode = argv[++i];
        else if (!strcmp(a, "--amp") && i + 1 < argc)     amp = atoi(argv[++i]);
        else if (!strcmp(a, "--rejouer")) { rejeu = 1; }
        else if (!strcmp(a, "--bsic") && i + 1 < argc) bsic_rej = atoi(argv[++i]);
        else if (!strcmp(a, "--arm")) {
            arm_sock = (i + 1 < argc && argv[i + 1][0] != '-') ? argv[++i] : CALYPSO_PONT_SOCK;
        }
        else if (a[0] == '-' && a[1] == 'v' && strspn(a + 1, "v") == strlen(a + 1)) {
            niveau = (int)strlen(a + 1);
        }
        else if (!strcmp(a, "-h") || !strcmp(a, "--help")) { usage(argv[0]); return 0; }
        else { usage(argv[0]); return 2; }
    }
    if (insns < 0) {
        /* [2026-09-20] 200000 under --arm: the FB search over whole 1250-symbol
         * frames costs the ROM 24-36k instructions per frame, and a frame cut
         * short by the budget leaves the ARM reading half-written results (a
         * zeroed a_sch read as a CRC-OK SB). Silicon has ~360k cycles per frame
         * at 78 MHz; 32000 was a bench constant, not a hardware one. */
        insns = arm_sock ? 200000 : 2300;
    }
    if (trames < 0) {
        trames = rejeu ? 4000 : 100;       /* an explicit --trames is honoured as is */
    }
    verbosite_installer(niveau);
    /* Fast path of the core (calypso_c54x.h): on unless a probe is armed. */
    { const char *d = getenv("CALYPSO_DEBUG"), *r = getenv("CALYPSO_C54X_RAPIDE");
      c54x_rapide = (r && *r) ? (*r != '0') : !(d && *d); }
    /* Pure probes of the core (c54x_internal.h): off by default; -vvvv and
     * above turn them on, since that is the level at which they are read.
     * Otherwise the core resolves CALYPSO_SONDES / CALYPSO_DEBUG at the first
     * c54x_init. */
    if (niveau >= 4) c54x_sondes = 1;

    qemu_mutex_init(&calypso_pcb_daram_lock);

    /* API RAM: private, or shared with the QEMU ARM. It must be installed
     * BEFORE the ROMs, since the loader copies into it whatever falls inside
     * the 0x0800 window. Under --arm the shared segment IS data[0x0800..] and
     * api_ram aliases it, so every core write reaches the ARM by either
     * path. */
    static uint16_t api_ram_privee[CALYPSO_API_WORDS];
    uint16_t *api_ram = api_ram_privee;
    C54xState *dsp;
    if (arm_sock || rejeu) {
        /* A pending interrupt (IFR&IMR) is taken as soon as INTM drops: real
         * C54x behaviour (SPRU131 ch.6), not a workaround. The core gates it
```

```
     * behind CALYPSO_C54X_IRQ_LEVEL; turn it on here unless explicitly
     * overridden (empty CALYPSO_C54X_IRQ_LEVEL turns it off). Applied to
     * BOTH benches: the replay used to run the core without it and could
     * not reproduce the bridge. */
    const char *e = getenv("CALYPSO_C54X_IRQ_LEVEL");
    if (!e) setenv("CALYPSO_C54X_IRQ_LEVEL", "1", 1);
    else if (!*e) unsetenv("CALYPSO_C54X_IRQ_LEVEL");
}
if (arm_sock) {
    dsp = pont_allouer_dsp();
    if (!dsp) { return 1; }
    api_ram = &dsp->data[C54X_API_BASE];
} else {
    dsp = c54x_init();
    if (!dsp) { fprintf(stderr, "c54x_init a echoue\n"); return 1; }
}
c54x_set_api_ram(dsp, api_ram);

int charges = 0;
for (unsigned i = 0; i < sizeof(ROMS) / sizeof(ROMS[0]); i++) {
    char chemin[512];
    snprintf(chemin, sizeof(chemin), "%s/calypso_dsp.%s.bin", rom_dir, ROMS[i].suffixe);
    int n = c54x_load_section(dsp, chemin, ROMS[i].adresse, ROMS[i].programme);
    if (n < 0) {
        fprintf(stderr, "ROM manquante : %s\n", chemin);
        return 1;
    }
    printf("  %-6s %-5s 0x%05x  %6d mots\n", ROMS[i].suffixe,
           ROMS[i].programme ? "prog" : "data", ROMS[i].adresse, n);
    charges++;
}
{ char chemin[512];
  snprintf(chemin, sizeof(chemin), "%s/calypso_dsp.Registers.bin", rom_dir);
  int n = c54x_load_registers(dsp, chemin);
  printf("  %-6s %-5s %-7s  %6d mots\n", "Regs", "mmr", "", n); }

printf("%d sections chargees, reset...\n", charges);
c54x_reset(dsp);

if (rejeu) {
    calypso_dma_init();
    calypso_bsp_init(dsp);
    if (insns < 32000) insns = 32000;
    if (!iq_mode || !*iq_mode || !strcmp(iq_mode, "none")) iq_mode = "cell";
    int rc = rejouer(dsp, api_ram, trames, insns, iq_mode, amp, bsic_rej, verbeux);
    verbosite_retirer();
    verbosite_bilan(stdout);
    return rc;
}
if (arm_sock) {
    /* Same sequence as calypso_trx_init() of qosmo-dsp after reset. */
    calypso_dma_init();
    calypso_bsp_init(dsp);
    int rc = pont_serveur(dsp, api_ram, arm_sock, insns, verbeux, iq_mode, amp);
    verbosite_retirer();
    verbosite_bilan(stdout);
    return rc;
}

/* The README observables, sampled once per frame. */
uint16_t *d_fb_det = &api_ram[(API_NDB + NDB_D_FB_DET) / 2];
uint16_t *a_sch0   = &api_ram[(API_R_PAGE(0) + RP_A_SCH) / 2];

unsigned fb_vus = 0, sch_ecrits = 0;
uint16_t fb_precedent = 0, sch_sig_prec = 0xFFFF;
uint16_t sch3_premier = 0; bool sch3_varie = false, sch3_vu = false;
```

```
    if (verbeux) {
        printf("\n  trame    d_fb_det   a_sch[0]  a_sch[1]  a_sch[2]  a_sch[3]\n");
    }
    for (long t = 0; t < trames; t++) {
        g_c54x_exe_fn = (uint32_t)t;
        c54x_run(dsp, (int)insns);

        /* Count TRANSITIONS, not frames where the value is non-zero: the bit
         * stays raised, so counting per frame reported "50 out of 50", which
         * only measured how long the run lasted. */
        if (*d_fb_det && !fb_precedent) fb_vus++;
        fb_precedent = *d_fb_det;

        uint16_t sch_sig = (uint16_t)(a_sch0[0] ^ a_sch0[3]);
        if ((a_sch0[0] || a_sch0[3]) && sch_sig != sch_sig_prec) {
            sch_ecrits++;
            if (!sch3_vu) { sch3_premier = a_sch0[3]; sch3_vu = true; }
            else if (a_sch0[3] != sch3_premier) sch3_varie = true;
            sch_sig_prec = sch_sig;
        }
        if (verbeux) {
            printf("  %5ld    %6u     0x%04x    0x%04x    0x%04x    0x%04x\n",
                   t, *d_fb_det, a_sch0[0], a_sch0[1], a_sch0[2], a_sch0[3]);
        }
    }

    verbosite_retirer();
    printf("\n─── bilan sur %ld trames ───\n", trames);
    printf("  AUCUN burst injecte : le DSP tourne sur une API RAM vierge, sans\n"
           "  ARM ni TPU. Les valeurs ci-dessous disent que la mask-ROM EXECUTE,\n"
           "  pas encore qu'elle decode. Injecter des bursts est l'etape suivante.\n\n");
    printf("  d_fb_det leve      : %u transition(s) 0 -> 1\n", fb_vus);
    printf("  a_sch change       : %u fois\n", sch_ecrits);
    if (sch3_vu) {
        printf("  a_sch[3]           : 0x%04x%s\n", sch3_premier,
               sch3_varie ? " puis VARIE" : " CONSTANT sur toutes les ecritures");
        if (!sch3_varie) {
            printf("\n  ^ c'est le symptome du README : une sortie qui ne depend\n"
                   "    pas de l'entree. Ici l'entree est vide, donc attendu.\n");
        }
    } else {
        printf("  a_sch              : jamais ecrit (la tache SB n'a pas tourne)\n");
    }
    verbosite_bilan(stdout);
    return 0;
}
```

3.16 /opt/GSM/c54x_exe/src/montant.c

67082 octets, 1536 lignes → 1536 lignes

```
/* SPDX-License-Identifier: GPL-2.0-or-later */
/*
 * montant.c - uplink side of the DSP setup: RACH, SDCCH, SACCH, FACCH, speech.
 *
 * WHY THIS FILE EXISTS
 * --------------------
 * Under CALYPSO_DSP_EXTERN=1, calypso_l1_do_init() calls calypso_l1_disable():
 * no layer 1 is registered in QEMU, so calypso_l1_do_rach_written() and
 * calypso_l1_do_page_written() (calypso_l1_dispatch.c) are no-ops. In the
 * grgsm setup those were the functions publishing the uplink into the
 * /dev/shm side-bands that pont.py consumes (pont/uplink.py):
 *
```

```
 *   /dev/shm/calypso_rach           RACH        (ra, bsic)
 *   /dev/shm/calypso_sdcch_ul       SDCCH UL    (23-byte L2 block)
 *   /dev/shm/calypso_tch_facch_ul   FACCH UL
 *   /dev/shm/calypso_tch_sacch_ul   SACCH UL
 *   /dev/shm/calypso_tch_ul         speech (ring of FR frames)
 *
 * Without the RACH there is no IMM ASS, hence no SDCCH and never a LOCATION
 * UPDATING ACCEPT.
 *
 * HOW
 * ---
 * This process holds the shared API RAM, so the captures of
 * qosmo-grgsm/hw/arm/calypso/calypso_l1_grgsm.c are reproduced as is, but
 * triggered by POLLING once per frame instead of QEMU's write callbacks. The
 * call site (pont.c, end of PONT_TICK) matches the end of the ARM scenario:
 * dsp_end_scenario() has just written d_dsp_page = B_GSM_TASK | page, and the
 * task words of that W page are current.
 *
 * The RACH is the only case where polling is not equivalent to a callback:
 * the firmware writes d_rach (prim_rach.c:72) then d_task_ra, and nobody
 * clears them afterwards (sync.c:307 only zeroes them on ABORT). Triggering
 * is therefore EDGE based: first non-zero value, or a value different from
 * the previous one. Accepted blind spot: two consecutive attempts with the
 * same (RA, BSIC), about 1/256 since gsm48_rr draws the RA at random; the
 * next attempt goes through. MONTANT_CONSOMME_RACH=1 zeroes d_rach after
 * publication, which makes the trigger exact at the cost of a write into the
 * API window the ROM might read.
 *
 * This module does not speak TRXD: calypso_bsp_send_rach_ra() exists (dead
 * code) but would send the access burst to 127.0.0.1:5702, the DOWNLINK
 * socket of pont.py, whose run_data() does "self.bts_data = addr" on every
 * packet received: the burst would be read back as downlink and the BTS
 * address overwritten. The side-band path goes through the uplink machinery
 * of pont.py, the one already proven in the grgsm setup.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <stdint.h>
#include <unistd.h>
#include <fcntl.h>
#include "hw/arm/calypso/calypso_api.h"
#include "hw/arm/calypso/calypso_debug.h"
#include "calypso_bsp.h"
#include "montant.h"
#include <osmocom/core/bits.h>
#include <osmocom/coding/gsm0503_coding.h>   /* downlink FACCH prosthesis */

#define SHM_RACH        "/dev/shm/calypso_rach"
#define SHM_SDCCH_UL    "/dev/shm/calypso_sdcch_ul"
#define SHM_FACCH_UL    "/dev/shm/calypso_tch_facch_ul"
#define SHM_SACCH_UL    "/dev/shm/calypso_tch_sacch_ul"
#define SHM_TCH_UL      "/dev/shm/calypso_tch_ul"
#define SHM_KC          "/dev/shm/calypso_kc_l1"

/* Sizes and layouts: pont/uplink.py. */
#define REC_RACH        16      /* 12 read: seq(4) ra(1) bsic(1) ..(2) fn(4)   */
#define REC_L2          48      /* 39 read: seq(4) l1s(4) fn(4) task(2) . p51(1) . l2(23) */
#define TCH_UL_SLOTS    16
#define TCH_UL_SLOT_SZ  64      /* TCH_UL_SLOT       */
#define TCH_UL_FR_OFS   16      /* TCH_UL_FR_OFS     */
#define FR_BYTES        33

/* Search window for the L2 header in a_cu (SDCCH_UL_WINDOW_OFS of
 * calypso_l1_grgsm.c). */
```

```
#define SDCCH_UL_WINDOW_OFS  6

/* Uplink SDCCH duplicate filter (heuristic path).
 *
 * The firmware leaves its block in a_cu; republishing it would make the bridge
 * resend it and the BSC would see a second SABM on an already established
 * link ("SABM frame with information not allowed in this state"), dropping
 * the channel in the middle of the procedure.
 *
 * So a block is published ONLY when its content changes. MONTANT_SDCCH_REPETE
 * = N re-enables republication of the same block after N frames (0 = never,
 * the default). A LAPDm retransmission by the mobile carries the same bytes
 * and is therefore swallowed; this is the accepted trade-off. */
#define SDCCH_UL_REPETE_DEFAUT 0

/* Minimum number of frames between two published RACHs: one attempt by the
 * mobile spans several frames and the same d_rach stays readable meanwhile. */
#define RACH_GARDE_TRAMES 4

static struct {
    unsigned long rach, sdcch, facch, sacch, parole;
    uint16_t prev_rach;
    uint32_t fn_rach;
    bool     rach_vu;      /* at least one RACH published          */
    bool     base_rach;    /* the reference value has been taken   */
    /* Last uplink SDCCH block published, for the duplicate filter. */
    uint8_t  sdcch_dernier[23];
    uint32_t sdcch_trame;
    bool     sdcch_a_dernier;
    bool     blud_vu;          /* a_cu has already announced a block via B_BLUD */
    unsigned sans_blud;        /* uplink tasks seen without B_BLUD         */
    bool     dedie_arme;
    /* SDCCH <-> TCH switch followed from the firmware tasks (see
     * suivre_tache_tch). The last SDCCH read from calypso_dcch_cfg is kept to
     * return to it; the TCH announcement comes from the bridge
     * (calypso_tch_cfg). */
    int      sd_tn, sd_genre, sd_ss;
    bool     sd_valide;        /* an SDCCH is known (dcch_cfg armed)       */
    int      tch_tn, tch_tsc;  /* bridge announcement: 0 = no TCH announced */
    bool     sur_tch;          /* the BSP plays the TCH timeslot           */
} g;
static int g_tn_tch_sb;   /* TCH TS seen on the side-band (0: TS0 / no TCH) */

/* Read tasks the ARM posts in d_task_d of the W page (osmocom-bb firmware,
 * include/calypso/l1_environment.h:45-52). Reading an SDCCH/SACCH block goes
 * through ALLC (prim_rx_nb.c:200), like the BCCH and the CCCH; DDL/ADL are
 * kept as a precaution. */
#ifndef DDL_DSP_TASK
#define DDL_DSP_TASK 26
#endif
#ifndef ADL_DSP_TASK
#define ADL_DSP_TASK 27
#endif
/* d_tch_mode: fourth word of the NDB (dsp_api.h:121, rejouer.c:623). */
#define NDB_D_TCH_MODE 0x006u

static int journal(void)
{
    static int n = -1;
    if (n < 0) {
        const char *e = calypso_getenv("MONTANT_DEBUG");
        n = (e && *e) ? atoi(e) : 20;   /* the first 20 events, by default */
    }
    return n;
}
```

```
static int sb_ouvrir(const char *chemin, off_t taille)
{
    int fd = open(chemin, O_CREAT | O_RDWR, 0644);
    if (fd >= 0 && ftruncate(fd, taille) < 0) {
        close(fd);
        return -1;
    }
    return fd;
}

static void sb_ecrire(int fd, const void *buf, size_t n, off_t off)
{
    if (fd >= 0 && pwrite(fd, buf, n, off) < 0) {
        return;
    }
}

static void publier_l2(int *fdp, const char *chemin, uint32_t *seq,
                       const uint8_t *l2, uint16_t task_u, uint32_t fn)
{
    if (*fdp == -2) {
        *fdp = sb_ouvrir(chemin, REC_L2);
    }
    uint8_t buf[REC_L2];
    memset(buf, 0, sizeof(buf));
    (*seq)++;
    memcpy(buf + 0, seq, 4);
    memcpy(buf + 4, &fn, 4);
    memcpy(buf + 8, &fn, 4);
    memcpy(buf + 12, &task_u, 2);
    buf[14] = (uint8_t)(fn % 51u);
    memcpy(buf + 16, l2, 23);
    sb_ecrire(*fdp, buf, sizeof(buf), 0);
}

/* UPLINK SPEECH RECORDING. Every uplink frame is logged here, raw TI (type 1)
 * and converted to FR (type 0), in the same 48-byte record format as noter_dl;
 * tools/decoder_add.py decodes them with libgsm and looks for the test tone.
 * The file is reset at each call (more than 500 frames without a record),
 * 16000 records at most. */
static void noter_ul(uint32_t fn, uint8_t type, uint16_t etat, const uint8_t *data, int n)
{
    static FILE *f;
    static unsigned nrec;
    static uint32_t fn_prec;
    if (!f || (uint32_t)(fn - fn_prec) > 500u) {
        if (f) fclose(f);
        f = fopen("/dev/shm/calypso_add_ul.bin", "wb");
        nrec = 0;
    }
    fn_prec = fn;
    if (!f || nrec >= 16000) return;
    uint8_t r[48];
    memset(r, 0, sizeof r);
    memcpy(r, &fn, 4);
    r[4] = type; r[5] = (uint8_t)n;
    memcpy(r + 6, &etat, 2);
    memcpy(r + 12, data, n > 36 ? 36 : n);
    fwrite(r, 1, sizeof r, f);
    if ((++nrec % 32) == 0) fflush(f);
}
static void publier_parole(const uint8_t *fr, uint32_t fn)
{
    static int fd = -2;
    static uint32_t seq;
    if (fd == -2) {
```

```
        fd = sb_ouvrir(SHM_TCH_UL, 8 + TCH_UL_SLOTS * TCH_UL_SLOT_SZ);
        uint32_t hdr[2] = { 0, TCH_UL_SLOTS };
        sb_ecrire(fd, hdr, sizeof(hdr), 0);
    }
    uint8_t buf[TCH_UL_SLOT_SZ];
    memset(buf, 0, sizeof(buf));
    seq++;
    memcpy(buf + 0, &seq, 4);
    memcpy(buf + 4, &fn, 4);
    memcpy(buf + 8, &fn, 4);
    memcpy(buf + TCH_UL_FR_OFS, fr, FR_BYTES);
    /* Header (the write counter) last: pont.py reads the header first, then
     * the slot; the other order would hand it a half-written slot. */
    sb_ecrire(fd, buf, sizeof(buf), 8 + (off_t)((seq - 1) % TCH_UL_SLOTS) * TCH_UL_SLOT_SZ);
    sb_ecrire(fd, &seq, 4, 0);
}

/* Uplink block deposited by the firmware in the NDB: word 0 = header (B_BLUD
 * flags "block ready"), data from word 3 on. The 33 speech bytes are stored
 * high byte first, the 23 L2 bytes low byte first.
 *
 * B_BLUD BELONGS TO THE ROM, NOT TO THE HOST. For a_cu (SDCCH, SACCH) the ROM
 * encodes the block as soon as the DUL/AUL task is posted and the flag drops
 * at the next tick (posted at T, cleared by the ROM at T+1, bursts from T+1 to
 * T+4). For a_fu (FACCH) and a_du (speech) the TCHT task uses the flag to
 * choose between FACCH and speech, and the ROM only reads it at the next
 * tick: clearing it here at the end of tick T would prevent the ROM from ever
 * sending a FACCH or speech burst. The flag is therefore left to the ROM; a
 * block is published once per posting (flag seen dropping in between, or
 * different content). MONTANT_CONSOMME_BLUD=1: the host clears the flag after
 * reading. */
static bool prendre_ul(uint16_t *api_ram, unsigned off, uint8_t *out, int n)
{
    uint16_t *w = &api_ram[(API_NDB + off) / 2];
    static int consomme = -1;
    static struct { unsigned off; bool publie; uint8_t contenu[FR_BYTES]; } vu[4];
    static int nvu;
    if (consomme < 0) {
        const char *e = calypso_getenv("MONTANT_CONSOMME_BLUD");
        consomme = (e && *e == '1') ? 1 : 0;
        printf("  [montant] B_BLUD des tampons montants (a_cu, a_fu, a_du) %s\n",
               consomme ? "efface par l'hote apres lecture (MONTANT_CONSOMME_BLUD=1) : la ROM n'emet ni FACCH ni
↪  parole"
                        : "laisse a la ROM, qui le consomme (MONTANT_CONSOMME_BLUD=1 pour l'ancien comportement)");
    }
    int k;
    for (k = 0; k < nvu && vu[k].off != off; k++) { }
    if (k == nvu && nvu < 4) { vu[nvu].off = off; vu[nvu].publie = false; nvu++; }
    if (!(w[0] & B_BLUD)) {
        if (k < nvu) vu[k].publie = false;     /* the ROM consumed it: the next posting is published */
        return false;
    }
    for (int i = 0; i < n; i += 2) {
        uint16_t v = w[3 + i / 2];
        uint8_t premier = (n == FR_BYTES) ? (uint8_t)(v >> 8) : (uint8_t)(v & 0xff);
        uint8_t second  = (n == FR_BYTES) ? (uint8_t)(v & 0xff) : (uint8_t)(v >> 8);
        out[i] = premier;
        if (i + 1 < n) {
            out[i + 1] = second;
        }
    }
    if (consomme) {
        w[0] &= (uint16_t)~B_BLUD;
        return true;
    }
    if (k < nvu) {
```

```
        if (vu[k].publie && memcmp(vu[k].contenu, out, (size_t)n) == 0) return false;   /* already published, not
        ↪  yet consumed */
        vu[k].publie = true;
        memcpy(vu[k].contenu, out, (size_t)n);
    }
    return true;
}

/* UPLINK SPEECH IS IN TI FORMAT, THE BRIDGE EXPECTS STANDARD FR.
 * mobile_pont.cfg: io-tch-format ti. gapk converts the microphone voice to
 * the TI DSP format (osmo-gapk fmt_ti.c, ti_fr_from_canon), the firmware
 * copies it as is into a_du, and the bridge passes it to
 * gsm0503_tch_fr_encode(..., net_order=1), which expects TS 101 318 FR (the
 * RTP "gsm" format, leading nibble 0xd). The 260-bit order differs, so the
 * block is converted here as in fmt_ti.c (ti_fr_to_canon) then fmt_gsm.c
 * (gsm_from_canon). The downlink stays TI end to end. MONTANT_PAROLE_TI=0:
 * raw pass-through. */
#include <osmocom/codec/codec.h>
static int bit_msb(const uint8_t *b, int i) { return (b[i >> 3] >> (7 - (i & 7))) & 1; }
static void mettre_bit_msb(uint8_t *b, int i, int v)
{
    if (v) b[i >> 3] |= (uint8_t)(0x80 >> (i & 7));
    else   b[i >> 3] &= (uint8_t)~(0x80 >> (i & 7));
}
static void parole_ti_vers_fr(uint8_t fr[FR_BYTES])
{
    static int conv = -1;
    if (conv < 0) { const char *e = calypso_getenv("MONTANT_PAROLE_TI"); conv = !(e && *e == '0'); }
    if (!conv) return;
    uint8_t canon[FR_BYTES];
    memset(canon, 0, sizeof canon);
    for (int i = 0; i < 260; i++) {                /* ti_fr_to_canon */
        int si = i >= 182 ? i + 4 : i;
        mettre_bit_msb(canon, gsm610_bitorder[i], bit_msb(fr, si));
    }
    fr[0] = (uint8_t)(0xd0 | (canon[0] >> 4));      /* gsm_from_canon: 0xd + 260 bits */
    for (int i = 1; i < FR_BYTES; i++)
        fr[i] = (uint8_t)((canon[i - 1] << 4) | (canon[i] >> 4));
}

static int fd_facch = -2;
static uint32_t seq_facch;

/* UPLINK FACCH AT THE GO. The ARM posts a_fu (B_BLUD) between phase A and the GO;
 * the ROM reads it and clears B_BLUD during phase B of the SAME frame, so
 * montant_scruter (end of frame) mostly finds the flag already dropped. Called
 * by pont.c at the GO, before the ROM runs; the duplicate filter of
 * prendre_ul() keeps montant_scruter from republishing the same block.
 * MONTANT_FU_AU_GO=0 disables it. */
void montant_capter_fu(uint16_t *api_ram, uint32_t fn)
{
    static int on = -1;
    if (on < 0) { const char *e = calypso_getenv("MONTANT_FU_AU_GO"); on = !(e && *e == '0'); }
    if (!on || !g.sur_tch) return;
    uint8_t l2[23];
    if (prendre_ul(api_ram, NDB_A_FU, l2, 23)) {
        publier_l2(&fd_facch, SHM_FACCH_UL, &seq_facch, l2, TCHT_DSP_TASK, fn);
        if (g.facch++ < (unsigned long)journal())
            printf("  [montant] FACCH UL fn=%u (au GO)\n", fn);
    }
    /* UPLINK SPEECH AT THE GO. a_du (NDB_A_DU_1) behaves like a_fu: B_BLUD is
     * cleared by the ROM during phase B of the frame, so capture_tch_ul
     * (montant_scruter, end of frame) comes too late. It is captured here at
     * the GO, symmetric to the FACCH above. The duplicate filter of
     * prendre_ul() (content dedup, vu[]) prevents a double send when
     * capture_tch_ul retries at the end of the frame. MONTANT_DU_AU_GO=0
```

```
     * disables it. */
    static int du = -1;
    if (du < 0) { const char *e = calypso_getenv("MONTANT_DU_AU_GO"); du = !(e && *e == '0'); }
    if (du) {
        uint8_t fr[FR_BYTES];
        if (prendre_ul(api_ram, NDB_A_DU_1, fr, FR_BYTES)) {
            noter_ul(fn, 1, TCHT_DSP_TASK, fr, FR_BYTES);   /* raw, TI format (a_du) */
            parole_ti_vers_fr(fr);
            noter_ul(fn, 0, TCHT_DSP_TASK, fr, FR_BYTES);   /* converted to FR, what the bridge gets */
            publier_parole(fr, fn);
            if (g.parole++ < (unsigned long)journal())
                printf("  [montant] parole UL fn=%u (au GO)\n", fn);
        }
    }
}

static bool capture_tch_ul(uint16_t *api_ram, uint16_t task_u, uint32_t fn)
{
    static int fd_sacch = -2;
    static uint32_t seq_sacch;
    uint8_t l2[23], fr[FR_BYTES];

    switch (task_u & 0x7FFF) {
    case TCHT_DSP_TASK:
        if (prendre_ul(api_ram, NDB_A_FU, l2, 23)) {
            publier_l2(&fd_facch, SHM_FACCH_UL, &seq_facch, l2, task_u, fn);
            if (g.facch++ < (unsigned long)journal())
                printf("  [montant] FACCH UL fn=%u task=0x%04x\n", fn, task_u);
        }
        if (prendre_ul(api_ram, NDB_A_DU_1, fr, FR_BYTES)) {
            noter_ul(fn, 1, task_u, fr, FR_BYTES);      /* raw, TI format (a_du) */
            parole_ti_vers_fr(fr);
            noter_ul(fn, 0, task_u, fr, FR_BYTES);      /* converted to FR, what the bridge gets */
            publier_parole(fr, fn);
            if (g.parole++ < (unsigned long)journal())
                printf("  [montant] parole UL fn=%u\n", fn);
        }
        return true;
    case TCHA_DSP_TASK:
        if (prendre_ul(api_ram, NDB_A_CU, l2, 23)) {
            publier_l2(&fd_sacch, SHM_SACCH_UL, &seq_sacch, l2, task_u, fn);
            if (g.sacch++ < (unsigned long)journal())
                printf("  [montant] SACCH UL fn=%u task=0x%04x\n", fn, task_u);
        }
        return true;
    case TCHD_DSP_TASK:
        return true;
    default:
        return false;
    }
}

/* Uplink SDCCH / SACCH.
 *
 * The firmware ANNOUNCES its block, nothing has to be guessed:
 * prim_tx_nb.c:80-101 writes into a_cu the header `(1 << B_BLUD)`, two zero
 * words, then the 23 L2 bytes from word 3 on -- exactly the layout that
 * prendre_ul() reads. The flag is single-use: one block = one publication.
 *
 * MONTANT_SDCCH_FENETRE=1 forces the heuristic path (LAPDm header scan in
 * a_cu+6 with a content-based duplicate filter, as in the gr-gsm layer 1), and
 * that path takes over by itself if B_BLUD never rises while the firmware
 * posts uplink tasks (the case where the ROM would consume the flag first).
 */
/* DUPLICATE FILTER EXPIRY (heuristic path only).
 *
```

```
 * The gr-gsm layer 1 republishes an identical block after 60 ticks
 * (calypso_l1_grgsm.c:673). This is NOT a retransmission engine: in that
 * setup QEMU clears d_task_u at every tick (calypso_trx.c, non-bridge branch),
 * so layer 1 only sees an uplink task on the frames where the firmware has
 * just posted it. The 60 ticks only prevent publishing the same block four
 * times (once per burst) while letting a REAL retransmission by the mobile
 * through, when its T200 makes it re-post.
 *
 * On the B_BLUD path this problem does not exist: the flag is single-use, one
 * posting = one publication, and a retransmission by the mobile raises the
 * flag again. Nothing to delay. */
#define SDCCH_TTL_DEFAUT      60   /* frames, like SDCCH_UL_DEDUP_TICKS */

static int sabm_ttl(void)
{
    static int v = -1;
    if (v < 0) {
        const char *e = calypso_getenv("MONTANT_SDCCH_TTL");
        if (!e || !*e) {
            e = calypso_getenv("MONTANT_SDCCH_REPETE");   /* alias */
        }
        v = (e && *e) ? atoi(e) : SDCCH_TTL_DEFAUT;
    }
    return v;
}

static void publier_sdcch(uint16_t task_u, uint32_t fn, const uint8_t *l2)
{
    static int fd = -2;
    static uint32_t seq;
    if (l2[1] == 0x03) {      /* empty frame (UI without data) */
        return;
    }
    publier_l2(&fd, SHM_SDCCH_UL, &seq, l2, task_u, fn);
    if (g.sdcch++ < (unsigned long)journal()) {
        printf("  [montant] SDCCH UL fn=%u task=0x%04x L2=", fn, task_u);
        for (int k = 0; k < 23; k++) printf("%02x%s", l2[k], k == 22 ? "" : " ");
        printf("\n");
    }
}

/* Heuristic path: window scan, duplicate filter on the useful content,
 * republication after MONTANT_SDCCH_REPETE frames (0 = never). */
static void capture_sdcch_fenetre(uint16_t *api_ram, uint16_t task_u, uint32_t fn)
{
    uint8_t fen[30];
    const uint8_t *src = (const uint8_t *)api_ram + API_NDB + NDB_A_CU + SDCCH_UL_WINDOW_OFS;
    memcpy(fen, src, sizeof(fen));

    int kk = 0;
    for (int j = 0; j <= 6; j++) {
        uint8_t a = fen[j], c = fen[j + 1], l = fen[j + 2];
        int sapi = (a >> 2) & 7;
        bool addr_ok = (a & 0x01) && ((a & 0x60) == 0) && (sapi == 0 || sapi == 3);
        bool ctrl_ok = (c != 0x2b) && (c != 0xff);
        bool len_ok = (l & 0x01) && ((l >> 2) <= 20);
        if (addr_ok && ctrl_ok && len_ok) {
            kk = j;
            break;
        }
    }
    const uint8_t *l2 = fen + kk;
    int repete = sabm_ttl();
    unsigned utile = 3u + (unsigned)(l2[2] >> 2);
    if (utile > 23u) {
        utile = 23u;
```

```
    }
    if (g.sdcch_a_dernier && !memcmp(g.sdcch_dernier, l2, utile) &&
        (repete <= 0 || (uint32_t)(fn - g.sdcch_trame) < (uint32_t)repete)) {
        return;
    }
    memcpy(g.sdcch_dernier, l2, 23);
    g.sdcch_trame = fn;
    g.sdcch_a_dernier = true;
    publier_sdcch(task_u, fn, l2);
}

static void capture_sdcch_ul(uint16_t *api_ram, uint16_t task_u, uint32_t fn)
{
    static int fenetre = -1;
    if (fenetre < 0) {
        const char *e = calypso_getenv("MONTANT_SDCCH_FENETRE");
        fenetre = (e && *e == '1') ? 1 : 0;
    }
    if (!fenetre) {
        uint8_t l2[23];
        if (prendre_ul(api_ram, NDB_A_CU, l2, 23)) {
            g.blud_vu = true;
            publier_sdcch(task_u, fn, l2);
            return;
        }
        if (g.blud_vu) {
            return;               /* the flag works: nothing to publish */
        }
        /* B_BLUD never seen while the firmware posts uplink tasks: the ROM
         * may consume it first. Switch to the window scan. */
        if (++g.sans_blud == 400) {
            printf("  [montant] a_cu : B_BLUD jamais vu en %u taches montantes, "
                   "bascule sur la fenetre heuristique\n", g.sans_blud);
        }
        if (g.sans_blud < 400) {
            return;
        }
    }
    capture_sdcch_fenetre(api_ram, task_u, fn);
}

/* The dedicated channel, read directly from the side-band written by the
 * QEMU L1CTL tap (calypso_dcch_tap.c) and already read by pont.py.
 *
 * A PONT_DCCH message from the bridge would be lost in the two-phase lock
 * step (the PONT_GO wait loop discards everything that is not a GO). The
 * file depends on no protocol: one 8-byte read per frame, and the same
 * sequence number pont.py uses to know whether it changed. */
static void scruter_dcch(uint32_t fn)
{
    static int fd = -2;
    static uint32_t seq;
    static uint32_t dernier_essai = UINT32_MAX;

    if (fd < 0) {
        /* At most one open attempt per frame, compared by equality so that
         * the hyperframe wrap-around (~3 h 29) never blocks a reopen. */
        if (fn == dernier_essai) {
            return;
        }
        /* Retried EVERY frame: run.sh clears the side-bands at startup and
         * the file only appears at the first write (QEMU tap here, bridge for
         * calypso_tch_cfg); any slower polling would see the announcement
         * frames too late. A failing open() costs a microsecond. */
        dernier_essai = fn;
        fd = open("/dev/shm/calypso_dcch_cfg", O_RDONLY);
```

```
        if (fd < 0) {
            return;
        }
    }
    uint8_t b[16];
    if (pread(fd, b, sizeof(b), 0) != (ssize_t)sizeof(b)) {
        return;
    }
    uint32_t s2;
    memcpy(&s2, b, 4);
    if (!s2 || s2 == seq) {
        return;
    }
    seq = s2;
    int genre = b[4], ss = b[5], tn = b[6];
    if (genre == 4) {
        /* PDCH (GPRS, firmware l1gprs_fw.c, tap calypso_dcch_tap.c): the BSP
         * plays the whole TN, like a TCH; no SDCCH memorised, no TCH switch. */
        printf("  [montant] canal dedie (side-band seq=%u) : PDCH TS%d (GPRS), toutes les trames au BSP\n", seq,
↪   tn);
        fflush(stdout);
        calypso_bsp_set_dedie(tn, 4, 0);
        g.dedie_arme = true;
        return;
    }
    /* Kind 2/3: the QEMU tap (calypso_dcch_tap.c) announces the TCH (TCH/F,
     * TCH/H). Only noted here: the BSP switch follows the firmware task
     * (suivre_tache_tch), the timeslot and TSC come from the bridge
     * (scruter_tch), and the memorised SDCCH must NOT be overwritten -- it is
     * the one an ASSIGNMENT FAILURE returns to (the tap republishes it when
     * the firmware goes back). */
    if (genre == 2 || genre == 3) {
        g_tn_tch_sb = tn;   /* TCH TS, to label the published uplink bursts (air) */
        /* With an A5/1-ciphered SDCCH neither the bridge nor the air reads the
         * ASSIGNMENT COMMAND, so calypso_tch_cfg never comes. The QEMU tap sees
         * the TS the firmware programs: it is taken as the TCH timeslot when
         * nobody announced one; suivre_tache_tch switches the BSP at the next
         * task. */
        if (!g.tch_tn && tn >= 1 && tn <= 7) {
            g.tch_tn = tn;
           printf("  [montant] TCH : intervalle TS%d pris sur le side-band (pas d'annonce calypso_tch_cfg)\n", tn);
        }
        printf("  [montant] canal dedie (side-band seq=%u) : TCH/%c TS%d SS=%d, firmware sur le TCH "
               "(%s%s)\n", seq, genre == 2 ? 'F' : 'H', tn, ss,
               g.sur_tch ? "BSP deja sur le TCH" : "BSP pas encore bascule",
               g.sd_valide ? ", SDCCH memorise" : "");
        fflush(stdout);
        return;
    }
    printf("  [montant] canal dedie (side-band seq=%u) : %s TS%d SDCCH/%d SS=%d%s\n",
           seq, genre == 0xFF ? "libere" : "arme", tn, genre == 1 ? 8 : 4, ss,
           (genre != 0xFF && g.sur_tch) ? " (memorise : le firmware est sur le TCH)" : "");
    g_tn_tch_sb = 0;   /* SDCCH (TS0) armed or channel released: the mobile no longer sends on the TCH */
    if (genre == 0xFF) {
        calypso_bsp_set_dedie(0, 0xFF, 0);
        g.dedie_arme = false;
        g.sd_valide = false;
        g.sur_tch = false;
        montant_canal_libere();
        return;
    }
    /* The SDCCH is MEMORISED: the BSP returns to it when the firmware leaves
     * the TCH (ASSIGNMENT FAILURE). While the firmware is on the TCH the BSP
     * is left alone. */
    g.sd_tn = tn;
    g.sd_genre = genre;
```

```
    g.sd_ss = ss;
    g.sd_valide = true;
    if (g.sur_tch) {
        return;
    }
    calypso_bsp_set_dedie(tn, genre, ss);
    g.dedie_arme = true;
}

/* Hand the memorised SDCCH back to the BSP (or release it if there is none). */
static void revenir_sdcch(const char *raison, uint32_t fn)
{
    g.sur_tch = false;
    if (g.sd_valide) {
        printf("  [montant] %s a fn=%u : BSP rendu au SDCCH TS%d SS=%d\n",
               raison, fn, g.sd_tn, g.sd_ss);
        calypso_bsp_set_dedie(g.sd_tn, g.sd_genre, g.sd_ss);
        g.dedie_arme = true;
    } else {
        printf("  [montant] %s a fn=%u : aucun SDCCH connu, canal dedie libere\n",
               raison, fn);
        calypso_bsp_set_dedie(0, 0xFF, 0);
        g.dedie_arme = false;
    }
    fflush(stdout);
}

/* MONTANT_TCH_TACHE=0: arm the BSP as soon as the bridge announces the TCH.
 * See scruter_tch. */
static bool tch_sur_tache(void)
{
    static int v = -1;
    if (v < 0) {
        const char *e = calypso_getenv("MONTANT_TCH_TACHE");
        v = (e && *e == '0') ? 0 : 1;
    }
    return v != 0;
}

/* THE TCH TIMESLOT.
 *
 * pont.py publishes the traffic channel timeslot in /dev/shm/calypso_tch_cfg
 * (pont/state.py, Tch._write_cfg: seq, tn, tsc, arfcn) as soon as it decodes
 * the downlink ASSIGNMENT COMMAND. MONTANT_TCH=0 disables reading it.
 *
 * THE ANNOUNCEMENT IS NOT THE SWITCH.
 * The bridge writes this file when it DECODES the ASSIGNMENT COMMAND, i.e. at
 * BTS time, while the BSP plays frames at DSP time -- 12 to ~200 frames
 * behind. Arming the BSP here would replace with TS2 bursts ALL the frames
 * not yet played (bsp_dedie_trame is true everywhere for a TCH), the
 * ASSIGNMENT COMMAND included, and TS0 with them.
 *
 * So this file is only an ANNOUNCEMENT (timeslot, TSC). The switch comes from
 * the firmware itself: suivre_tache_tch() arms the TCH at the first
 * TCHT/TCHA/TCHD task it posts, and hands the SDCCH back as soon as it posts
 * a block read task (ALLC) again. A non-zero seq with tn=0 is the bridge
 * giving up (pont/dsp/tch.py, TchDsp.abandon): back to the SDCCH without
 * dropping the Kc. seq=0 remains the full release.
 * MONTANT_TCH_TACHE=0 restores arming at the announcement. */
static void scruter_tch(uint32_t fn)
{
    static int fd = -1, coupe = -1;
    static uint32_t seq, dernier_essai = UINT32_MAX;

    if (coupe < 0) {
        const char *e = calypso_getenv("MONTANT_TCH");
```

```
        coupe = (e && *e == '0') ? 1 : 0;
    }
    if (coupe) {
        return;
    }
    if (fd < 0) {
        if (fn == dernier_essai) {          /* see scruter_dcch */
            return;
        }
        dernier_essai = fn;
        fd = open("/dev/shm/calypso_tch_cfg", O_RDONLY);
        if (fd < 0) {
            return;
        }
    }
    uint8_t b[16];
    if (pread(fd, b, sizeof(b), 0) != (ssize_t)sizeof(b)) {
        return;
    }
    uint32_t s2;
    memcpy(&s2, b, 4);
    if (s2 == seq) {
        return;
    }
    seq = s2;
    int tn = b[4], tsc = b[5];
    if (!s2) {
        /* Full release (Tch.close). */
        int etait = g.tch_tn;
        g.tch_tn = 0;
        if (g.sur_tch) {
            printf("  [montant] TCH (seq=0) : libere TS%d\n", etait);
            calypso_bsp_set_dedie(0, 0xFF, 0);
            g.sur_tch = false;
            g.dedie_arme = false;
            montant_canal_libere();
        } else if (etait) {
            printf("  [montant] TCH (seq=0) : annonce TS%d retiree (le firmware n'y etait pas)\n", etait);
        }
        return;
    }
    if (tn <= 0 || tn > 7) {
        /* Given up by the bridge (ASSIGNMENT FAILURE, mobile back on the
         * SDCCH): the SDCCH is still alive, and so is the Kc. */
        int etait = g.tch_tn;
        g.tch_tn = 0;
        printf("  [montant] TCH (seq=%u) : annonce TS%d abandonnee par le pont\n", seq, etait);
        if (g.sur_tch) {
            revenir_sdcch("abandon du TCH", fn);
        }
        return;
    }
    g.tch_tn = tn;
    g.tch_tsc = tsc;
    if (!tch_sur_tache()) {
        printf("  [montant] TCH (seq=%u) : arme TS%d TSC=%d - toutes ses trames (MONTANT_TCH_TACHE=0)\n",
               seq, tn, tsc);
        calypso_bsp_set_dedie(tn, 2 /* BSP_DEDIE_TCH */, 0);
        g.sur_tch = true;
        g.dedie_arme = true;
        return;
    }
    printf("  [montant] TCH (seq=%u) : TS%d TSC=%d annonce par le pont, "
           "bascule du BSP a la premiere tache TCH du firmware\n", seq, tn, tsc);
    fflush(stdout);
}
```

```
/* THE SDCCH <-> TCH SWITCH FOLLOWS THE FIRMWARE.
 *
 * d_task_d of the W page the ARM has just written says what it listens to on
 * the next frame: TCHT (13, traffic and FACCH), TCHA (14, SACCH/T) or TCHD
 * (28) on a TCH (prim_tch.c:520-585, 791-802), ALLC (24) for an SDCCH/SACCH
 * block (prim_rx_nb.c:200). This task is posted at DSP pace, so the BSP
 * changes timeslot on the frame the ARM really reads, whatever the DSP lag
 * behind the BTS. The other frames (neighbour PM, idle frame) change
 * nothing. */
/* d_fn PROBE: the position the firmware gives the DSP in the 104-multiframe
 * (dsp.c:537, d_fn = fn_report | (fn%104) << 8), on TCH tasks. To compare with
 * the BTS fn the BSP plays at the same tick ([sacch_tf]): an offset that is a
 * multiple of 26 lets speech and FACCH through but de-interleaves the SACCH
 * out of order (a_cd FIRE KO on every block, LOS). 24 lines. */
static void sonde_dfn(uint16_t *api_ram, unsigned pg, bool taches, uint32_t fn)
{
    static unsigned n;
    if (!taches || !g.sur_tch || n >= 24) {
        return;
    }
    uint16_t t = api_ram[(API_W_PAGE(pg) + WP_D_TASK_D) / 2] & 0x7fffu;
    uint16_t dfn = api_ram[(API_W_PAGE(pg) + WP_D_FN) / 2];
    if (t != TCHA_DSP_TASK) {
        return;
    }
    n++;
    printf("  [d_fn] tick=%u tache=%u fn_report=%u fn%%104=%u\n",
           fn, t, dfn & 0xffu, (dfn >> 8) & 0xffu);
}

static void suivre_tache_tch(uint16_t *api_ram, unsigned pg, bool taches, uint32_t fn)
{
    static bool sans_annonce_dit;
    if (!taches || !tch_sur_tache()) {
        return;
    }
    uint16_t t = api_ram[(API_W_PAGE(pg) + WP_D_TASK_D) / 2] & 0x7fffu;
    bool tache_tch = (t == TCHT_DSP_TASK || t == TCHA_DSP_TASK || t == TCHD_DSP_TASK);
    bool tache_bloc = (t == ALLC_DSP_TASK || t == DDL_DSP_TASK || t == ADL_DSP_TASK);

    if (tache_tch && !g.sur_tch) {
        if (!g.tch_tn) {
            /* No announcement (A5/1-ciphered SDCCH: the air cannot read the
             * ASSIGNMENT COMMAND): the timeslot is the one of the RX window the
             * firmware programmed in the TPU. */
            int tn_tpu = calypso_bsp_fenetre_tn();
            if (tn_tpu >= 1 && tn_tpu <= 7) {
                g.tch_tn = tn_tpu;
                printf("  [montant] TCH : tache %u postee a fn=%u sans annonce (calypso_tch_cfg) : "
                       "intervalle TS%d lu sur la fenetre TPU\n", t, fn, tn_tpu);
            } else {
                if (!sans_annonce_dit) {
                    sans_annonce_dit = true;
                    printf("  [montant] TCH : tache %u postee a fn=%u sans annonce du pont "
                           "(calypso_tch_cfg) : intervalle inconnu (TPU %d), BSP inchange\n", t, fn, tn_tpu);
                }
                return;
            }
        }
        sans_annonce_dit = false;
        printf("  [montant] TCH : le firmware poste la tache %u a fn=%u, BSP bascule sur TS%d (TSC=%d)\n",
               t, fn, g.tch_tn, g.tch_tsc);
        fflush(stdout);
        calypso_bsp_set_dedie(g.tch_tn, 2 /* BSP_DEDIE_TCH */, 0);
        g.sur_tch = true;
```

```
        g.dedie_arme = true;
        return;
    }
    /* TI FIRMWARE: ALLC AFTER THE TCH IS NOT NECESSARILY THE SDCCH. The TI L1
     * also reads the BCCH and the paging in ALLC: after the CHANNEL RELEASE of
     * a call, handing the BSP back to SDCCH/8 TS1 (which replaces TS0 on every
     * frame while the TI dedicated mode lasts, CALYPSO_BSP_SDCCH8_TOUT) would
     * lose the cell. Here the BSP first returns to TS0; the SDCCH is only
     * restored if an UPLINK task follows within 102 frames (assignment
     * failure: the L1 goes back to its SDCCH); in idle there is none.
     * MONTANT_TI_ALLC_DIRECT=1: return to the SDCCH directly. */
    static int ti = -1, direct = -1;
    static bool attente_ul; static uint32_t attente_fn;
    if (ti < 0) { ti = calypso_getenv("MS_MAGNETITE") != NULL;
                  const char *e = calypso_getenv("MONTANT_TI_ALLC_DIRECT"); direct = (e && *e == '1'); }
    if (tache_bloc && g.sur_tch) {
        if (!ti || direct) {
            revenir_sdcch("le firmware est revenu sur le SDCCH (tache ALLC)", fn);
        } else {
            g.sur_tch = false;
            g.dedie_arme = false;
            calypso_bsp_set_dedie(0, 0xFF, 0);
            attente_ul = g.sd_valide;
            attente_fn = fn;
            printf("  [montant] firmware TI : tache ALLC apres le TCH a fn=%u : BSP rendu a TS0, retour au SDCCH "
                   "seulement sur une tache montante%s\n", fn, g.sd_valide ? "" : " (aucun SDCCH connu)");
            fflush(stdout);
        }
        return;
    }
    if (attente_ul) {
        uint16_t u = api_ram[(API_W_PAGE(pg) + WP_D_TASK_U) / 2] & 0x7fffu;
        if (u) {
            attente_ul = false;
            revenir_sdcch("firmware TI : tache montante apres le TCH (echec d'assignation)", fn);
        } else if ((uint32_t)(fn - attente_fn) > 102u) {
            attente_ul = false;
            printf("  [montant] firmware TI : aucune tache montante 102 trames apres le TCH (fn=%u) : en
↪  veille\n", fn);
        }
    }
}

/* a_fd PROBE: DOWNLINK FACCH ON THE TCH.
 * The firmware only passes a FACCH up if ((fn%13)%4)==3 AND a_fd[0] carries
 * B_BLUD (prim_tch.c:235-246), then resets a_fd[0] = B_FIRE1: only the DSP
 * sets B_BLUD. This probe tells a ROM problem (B_BLUD never set) from a
 * plumbing one (set but FIRE). MONTANT_AFD=0 disables it. */
/* DOWNLINK FRAMES DELIVERED BY THE ROM, FOR COMPARISON.
 * Every speech frame (a_dd_0, converted from TI format to standard FR like
 * the uplink) and every FACCH (a_fd) seen with B_BLUD, with the tick, the
 * status word and the reported error count (word 2). 48-byte records: fn
 * LE32, type (0 speech, 1 FACCH), n, status LE16, err LE16, 2 padding bytes,
 * 36 data bytes. File reset at each call (more than 500 frames without a
 * record), 8000 records at most. tools/comparer_parole.py compares it with
 * /dev/shm/calypso_tch_dl.bin (BSP bursts) and /dev/shm/pont_tch_dl.bin
 * (bridge decoding). */
static void noter_dl(uint32_t fn, uint8_t type, uint16_t etat, uint16_t err,
                     const uint8_t *data, int n)
{
    static FILE *f;
    static unsigned nrec;
    static uint32_t fn_prec;
    if (!f || (uint32_t)(fn - fn_prec) > 500u) {
        if (f) fclose(f);
        f = fopen("/dev/shm/calypso_add_dl.bin", "wb");
```

```
        nrec = 0;
    }
    fn_prec = fn;
    if (!f || nrec >= 8000) return;
    uint8_t r[48];
    memset(r, 0, sizeof r);
    memcpy(r, &fn, 4);
    r[4] = type; r[5] = (uint8_t)n;
    memcpy(r + 6, &etat, 2);
    memcpy(r + 8, &err, 2);
    memcpy(r + 12, data, n > 36 ? 36 : n);
    fwrite(r, 1, sizeof r, f);
    if ((++nrec % 32) == 0) fflush(f);
}

static void sonde_afd(uint16_t *api_ram, uint32_t fn)
{
    static int sonde = -1;
    static uint16_t prec;
    static unsigned long ok, ko;
    if (sonde < 0) {
        const char *e = calypso_getenv("MONTANT_AFD");
        sonde = (e && *e == '0') ? 0 : 1;
    }
    if (!sonde || !g.sur_tch) {
        return;
    }
    uint16_t etat = api_ram[(API_NDB + NDB_A_FD) / 2];
    if ((etat & B_BLUD) && etat != prec) {
        const uint8_t *d = (const uint8_t *)api_ram + API_NDB + NDB_A_FD + 6;
        uint16_t mode = api_ram[(API_NDB + NDB_D_TCH_MODE) / 2];
        bool fire = (etat & 0x0040) != 0;          /* B_FIRE1 */
        if (fire) ko++; else ok++;
        noter_dl(fn, 1, etat, api_ram[(API_NDB + NDB_A_FD) / 2 + 2], d, 23);
        if ((ok + ko) <= 60 || ((ok + ko) % 50) == 0) {
            printf("  [a_fd] fn=%u fn%%13=%u etat=%04x BLUD=1 FIRE=%d d_tch_mode=%04x "
                   "L2=%02x %02x %02x %02x | ok=%lu ko=%lu\n",
                   fn, fn % 13u, etat, fire ? 1 : 0, mode, d[0], d[1], d[2], d[3], ok, ko);
        }
    }
    prec = etat;
}

/* a_dd PROBE: DOWNLINK SPEECH ON THE TCH.
 * The firmware only passes a speech frame up to the mobile
 * (L1CTL_TRAFFIC_IND) if a_dd_0[0] carries B_BLUD (prim_tch.c:322-327); GAPK
 * only encodes the uplink in response. This probe tells whether the DSP still
 * delivers downlink speech. One line for the first 20, then every 100.
 * MONTANT_ADD=0 disables it.
 *
 * 0x0040 is B_FIRE1 and the Fire code only protects control channels, so
 * speech is judged by bit 2 = B_BFI (l1_environment.h:272). The firmware only
 * tests B_BLUD (prim_tch.c:327) and does not pass the BFI up: GAPK decodes
 * the 33 bytes as they are. The probe therefore counts the BFI too and shows
 * a_dd_0[2], the error count the ROM reports (num_biterr on the firmware
 * side). */
static void sonde_add(uint16_t *api_ram, uint32_t fn)
{
    static int sonde = -1;
    static uint16_t prec;
    static unsigned long vues, ko, bfi;
    static uint32_t fn_dernier;
    if (sonde < 0) {
        const char *e = calypso_getenv("MONTANT_ADD");
        sonde = (e && *e == '0') ? 0 : 1;
    }
```

```
    if (!sonde || !g.sur_tch) {
        return;
    }
    uint16_t etat = api_ram[(API_NDB + NDB_A_DD_0) / 2];
    if ((etat & B_BLUD) && etat != prec) {
        bool fire = (etat & 0x0040) != 0;
        bool mauvaise = (etat & 0x0004) != 0;      /* B_BFI */
        uint16_t erreurs = api_ram[(API_NDB + NDB_A_DD_0) / 2 + 2];
        {
            const uint16_t *w = &api_ram[(API_NDB + NDB_A_DD_0) / 2];
            uint8_t fr[FR_BYTES];
            for (int k = 0; k < FR_BYTES; k += 2) {       /* high byte first, like prendre_ul */
                fr[k] = (uint8_t)(w[3 + k / 2] >> 8);
                if (k + 1 < FR_BYTES) fr[k + 1] = (uint8_t)(w[3 + k / 2] & 0xff);
            }
            parole_ti_vers_fr(fr);
            noter_dl(fn, 0, etat, erreurs, fr, FR_BYTES);
        }
        vues++;
        if (fire) ko++;
        if (mauvaise) bfi++;
        if (vues <= 20 || vues % 100 == 0) {
            printf("  [a_dd] fn=%u etat=%04x FIRE=%d BFI=%d err=%u (ecart %u trames) | vues=%lu ko=%lu bfi=%lu\n",
                   fn, etat, fire ? 1 : 0, mauvaise ? 1 : 0, erreurs, fn - fn_dernier, vues, ko, bfi);
        }
        fn_dernier = fn;
    }
    prec = etat;
}

bool montant_sur_tch(void)
{
    return g.sur_tch;
}

/* Timeslot the firmware transmits on: the TCH TS announced by the side-band
 * (kind 2/3) until it is released, 0 otherwise (SDCCH/4 on TS0, RACH). Used to
 * label the bursts published in calypso_tx_rom (bits 4-6 of the src byte) so
 * that the air carries them on the right TRX timeslot. */
int montant_tn_dedie(void)
{
    return g_tn_tch_sb;
}

void tx_rach_vers_air(uint8_t ra, uint8_t bsic, uint32_t tick);   /* pont.c */

static void publier_rach(uint8_t ra, uint8_t bsic, uint32_t fn)
{
    tx_rach_vers_air(ra, bsic, fn);   /* Magnetite: to the air (CALYPSO_RACH_VERS_AIR) */
    static int fd = -2;
    static uint32_t seq;
    if (fd == -2) {
        fd = sb_ouvrir(SHM_RACH, REC_RACH);
    }
    uint8_t buf[REC_RACH];
    memset(buf, 0, sizeof(buf));
    seq++;
    memcpy(buf + 0, &seq, 4);
    buf[4] = ra;
    buf[5] = bsic;
    memcpy(buf + 8, &fn, 4);
    sb_ecrire(fd, buf, sizeof(buf), 0);
    /* An access attempt is a new connection: the duplicate filter memory of
     * the heuristic path restarts (no effect on the B_BLUD path, which does
     * not need it). */
    g.sdcch_a_dernier = false;
```

```
    if (g.rach++ < (unsigned long)journal())
        printf("  [montant] RACH ra=0x%02x bsic=%u fn=%u -> %s\n", ra, bsic, fn, SHM_RACH);
}

/* ── THE Kc: THE BRIDGE CAN NEITHER CIPHER NOR DECIPHER WITHOUT IT ──────────
 *
 * Under CALYPSO_DSP_EXTERN=1 the gr-gsm layer 1 is disabled, and it was the
 * one publishing /dev/shm/calypso_kc_l1 (calypso_l1_grgsm.c, publish_kc).
 * Without the file, `Cipher.current()` of pont.py returns None and
 * `cipher.apply()` leaves the burst UNCHANGED in both directions. There is no
 * A5 in the Calypso model either (`d_a5mode` only exists in l1-grgsm/, not in
 * l1-dsp/): the bridge does it, as it already does the channel coding.
 *
 * Layout taken as is from publish_kc() so that pont/cipher.py (KC_RECLEN=32)
 * reads it unchanged: seq(4) algo(1) length(1) Kc[8] 0xFF. The four words of
 * a_kc come out big-endian AND reversed, as in the original -- a layout the
 * reader expects is not "corrected".
 *
 * MONTANT_KC=0 disables the publication. */
#define KC_RECLEN        32
#define KC_PUBLIER_TOUTES 22   /* frames between two polls, like grgsm */
#define KC_GRACE_CLAIR    5    /* cf. publish_kc: the firmware clears d_a5mode
                                * at every DM_REL_REQ, including when the Kc
                                * comes right back (Assignment Command),
                                * while the BTS keeps ciphering. */

/* Raised by montant_canal_libere(): the next poll must publish the return to
 * clear WITHOUT waiting for the grace period. See publier_kc(). */
static bool g_kc_liberer;

static void publier_kc(uint16_t *api_ram)
{
    static int actif = -1, fd = -1, tick, clair_en_attente;
    static uint32_t seq;
    static uint8_t dernier[KC_RECLEN];
    static bool a_dernier;

    if (actif < 0) {
        const char *e = calypso_getenv("MONTANT_KC");
        actif = (e && *e == '0') ? 0 : 1;
    }
    if (!actif) {
        return;
    }
    /* A MODE CHANGE CANNOT WAIT FOR THE PERIODIC POLL. The mobile switches as
     * soon as it processes the CIPHERING MODE COMMAND and sends its CIPHERING
     * MODE COMPLETE right away; an uplink SDCCH block falls every 51 frames,
     * and that block -- the FIRST ciphered uplink message -- must not leave
     * in clear while the BTS already deciphers it. d_a5mode is therefore read
     * EVERY frame (two memory accesses) and the full poll is only deferred
     * while the mode does not change. */
    uint16_t mode = api_ram[(API_NDB + NDB_D_A5MODE) / 2];
    uint8_t mode_algo = (mode >= 1 && mode <= 3) ? (uint8_t)mode : 0;
    bool bascule = (a_dernier && mode_algo != dernier[4]) || g_kc_liberer;
    if (!bascule && ++tick < KC_PUBLIER_TOUTES) {
        return;
    }
    tick = 0;
    const uint16_t *kw = &api_ram[(API_NDB + NDB_A_KC) / 2];
    uint8_t rec[KC_RECLEN] = {0};
    bool nul = true;
    for (int i = 0; i < 4; i++) {
        rec[6 + 6 - 2 * i] = (uint8_t)(kw[i] >> 8);
        rec[6 + 7 - 2 * i] = (uint8_t)(kw[i] & 0xFF);
    }
    for (int i = 6; i < 14; i++) {
```

```
        if (rec[i]) {
            nul = false;
        }
    }
    uint8_t algo = (mode >= 1 && mode <= 3 && !nul) ? (uint8_t)mode : 0;
    if (!algo) {
        memset(rec + 6, 0, 8);
    }
    rec[4] = algo;
    rec[5] = algo ? 8 : 0;
    rec[14] = 0xFF;

    if (a_dernier && !memcmp(dernier + 4, rec + 4, KC_RECLEN - 4)) {
        clair_en_attente = 0;
        /* This early return must CONSUME g_kc_liberer, otherwise the release
         * stays armed indefinitely: every following poll would compute
         * bascule=true, republish the same record and bump `seq`, which
         * pont.py reads as "new key". */
        g_kc_liberer = false;
        return;
    }
    /* THE GRACE PERIOD MUST NOT SURVIVE THE CHANNEL RELEASE. It comes from
     * publish_kc() and serves the INTRA-connection case: the firmware clears
     * d_a5mode at every DM_REL_REQ, including during an Assignment Command
     * where the Kc comes right back, while the BTS keeps ciphering. Between
     * TWO connections it is harmful: the algo=1 record would stay readable
     * five polls longer, and pont.py -- which drops its key at every
     * IMMEDIATE ASSIGNMENT (downlink.py) -- would read it back from the file
     * and cipher the uplink of the NEXT connection from its first block while
     * the mobile still sends in clear. The channel release is the right
     * signal: montant_canal_libere(), called on PONT_DCCH kind 0xFF. */
    if (!algo && a_dernier && dernier[4] && !g_kc_liberer &&
        ++clair_en_attente < KC_GRACE_CLAIR) {
        return;   /* ciphered -> clear: wait, the Kc may come back */
    }
    clair_en_attente = 0;
    g_kc_liberer = false;

    if (fd < 0 && (fd = open(SHM_KC, O_WRONLY | O_CREAT, 0644)) < 0) {
        actif = 0;
        return;
    }
    seq++;
    memcpy(rec, &seq, 4);
    if (pwrite(fd, rec, sizeof rec, 0) != (ssize_t)sizeof rec) {
        close(fd); fd = -1; seq--;
        return;
    }
    memcpy(dernier, rec, sizeof rec);
    a_dernier = true;
    if (algo) {
        printf("  [montant] chiffrement A5/%u : Kc publie vers %s (seq=%u)\n",
               algo, SHM_KC, seq);
    } else {
        printf("  [montant] retour en clair (seq=%u)\n", seq);
    }
    fflush(stdout);
}

void calypso_bsp_set_dedie_ti(int actif);   /* calypso_bsp.c */
static bool g_ti_dedie;                      /* TI firmware in dedicated mode (ALLC tasks), see below */

/* DOWNLINK FACCH PROSTHESIS (TI firmware). Under the Magnetite TI L1 the ROM
 * decodes the FACCH/F with the last two bursts of the block missing (Fire KO,
 * RR Assignment Failure), while the TS2 bursts the BSP delivers carry the
 * block intact. They are decoded here at every end of a FACCH/F block
```

```
 * (fn % 13 % 4 == 3) and, if it is a FACCH with a good CRC, posted into
 * a_fd: the TI L1 reads it on the next frame (l1_cmplx.c, fn_mod13 % 4 == 0),
 * after the end of this tick. The rest (speech) stays with the ROM.
 * CALYPSO_PROTHESE_FACCH_DL=0 disables it (live setting). */
/* DOWNLINK SPEECH PROSTHESIS (TI firmware), same mechanism as the FACCH: the
 * FR block decoded here from the BSP bursts (good CRC) is posted into a_dd in
 * TI format (inverse of parole_ti_vers_fr: gsm_to_canon then
 * ti_fr_from_canon), BFI at 0, so that the ROM speech decoder takes it
 * instead of its own. Only replaces the frames the ROM judged bad.
 * CALYPSO_PROTHESE_PAROLE_DL=0 disables it (live setting), =2 also replaces
 * the good ones. */
static void prothese_parole_dl(uint16_t *api_ram, const uint8_t fr[FR_BYTES], int ne, uint32_t e)
{
    static unsigned n_ok;
    int mode = calypso_reglage("CALYPSO_PROTHESE_PAROLE_DL", 1);
    if (!mode) return;
    uint16_t *dd = &api_ram[(API_NDB + NDB_A_DD_0) / 2];
    if (mode == 1 && (dd[0] & B_BLUD) && !(dd[0] & 0x0004)) return;   /* the ROM already has it good */
    uint8_t canon[FR_BYTES], ti[FR_BYTES + 1];
    memset(canon, 0, sizeof canon);
    for (int i = 0; i < 260; i++)                   /* gsm_to_canon: the leading 0xd nibble is skipped */
        mettre_bit_msb(canon, i, bit_msb(fr, i + 4));
    memset(ti, 0, sizeof ti);
    for (int i = 0; i < 260; i++) {                 /* ti_fr_from_canon */
        int si = i >= 182 ? i + 4 : i;
        mettre_bit_msb(ti, si, bit_msb(canon, gsm610_bitorder[i]));
    }
    dd[0] = (uint16_t)((dd[0] & ~0x0044u) | B_BLUD);   /* block present, neither FIRE nor BFI */
    dd[2] = (uint16_t)ne;
    for (int k = 0; k < FR_BYTES; k += 2)              /* high byte first, like sonde_add */
        dd[3 + k / 2] = (uint16_t)((ti[k] << 8) | (k + 1 < FR_BYTES ? ti[k + 1] : 0));
    if (n_ok++ < 40 || n_ok % 500 == 0)
        printf("  [parole-dl] fn=%u : trame FR de la BTS posee dans a_dd (%d err) [%u]\n", e, ne, n_ok);
}

static void prothese_facch_dl(uint16_t *api_ram)
{
    static uint32_t deja = 0xffffffffu;
    static unsigned n_ok;
    uint32_t e;
    if (!calypso_reglage("CALYPSO_PROTHESE_FACCH_DL", 1) || !g_ti_dedie) return;
    if (!calypso_bsp_tch_dernier_fn(&e) || e == deja) return;
    deja = e;
    if (e % 26u == 12u || e % 26u == 25u || (e % 13u) % 4u != 3u) return;
    uint32_t seq[8]; int n = 0;
    for (uint32_t f = e; n < 8; f--) {
        if (f % 26u == 12u || f % 26u == 25u) continue;
        seq[7 - n++] = f;
    }
    sbit_t soft[8 * 116];
    for (int k = 0; k < 8; k++) {
        uint8_t b[148];
        if (!calypso_bsp_tch_burst(seq[k], b)) return;
        for (int j = 0; j < 58; j++) {
            soft[k * 116 + j]      = b[3 + j]  ? -127 : 127;
            soft[k * 116 + 58 + j] = b[87 + j] ? -127 : 127;
        }
    }
    uint8_t l2[33]; int ne = 0, nb = 0;
    int rc = gsm0503_tch_fr_decode(l2, soft, 1, 0, &ne, &nb);
    if (rc == FR_BYTES) { prothese_parole_dl(api_ram, l2, ne, e); return; }
    if (rc != 23) return;
    uint16_t *fd = &api_ram[(API_NDB + NDB_A_FD) / 2];
    fd[0] = B_BLUD;                 /* block present, B_FIRE1/B_FIRE0 at 0: CRC good */
    fd[2] = (uint16_t)ne;
    for (int k = 0; k < 12; k++)
```

```
        fd[3 + k] = (uint16_t)(l2[2 * k] | (2 * k + 1 < 23 ? l2[2 * k + 1] << 8 : 0));
    if (n_ok++ < 40 || n_ok % 100 == 0)
        printf("  [facch-dl] fn=%u : FACCH de la BTS posee dans a_fd (%02x %02x %02x %02x, %d err) [%u]\n",
               e, l2[0], l2[1], l2[2], l2[3], ne, n_ok);
    fflush(stdout);
}

void montant_scruter(uint16_t *api_ram, uint32_t fn, unsigned page)
{
    static int coupe = -1;
    if (coupe < 0) {
        const char *e = calypso_getenv("MONTANT");
        coupe = (e && *e == '0') ? 1 : 0;
    }
    if (coupe || !api_ram) {
        return;
    }

    /* Which W page carries the tasks? dsp_end_scenario() (firmware,
     * calypso/dsp.c:471) writes d_dsp_page = B_GSM_TASK | w_page BEFORE
     * flipping w_page: the NDB word is the fresh source, including when
     * l1_sync() ran between the TICK and the GO. The `page` argument (the
     * d_dsp_page QEMU sampled at the TICK) is only a fallback. */
    uint16_t v_page = api_ram[(API_NDB + NDB_D_DSP_PAGE) / 2];
    bool taches = (v_page & B_GSM_TASK) != 0;
    unsigned pg = taches ? ((v_page & B_GSM_PAGE) ? 1u : 0u) : (page & 1u);

    {   /* CIPHERING PROBE: every change of d_a5mode or a_kc as the ARM leaves
         * them (NDB 0x1ce / 0x2ce), with the tick. The DSP ciphers and
         * deciphers by itself (calypso_a5.c) from these two fields; a reset
         * on the TCH stops deciphering while the BTS keeps ciphering. */
        static uint16_t a5_prec = 0xffff, kc_prec[4];
        uint16_t a5 = api_ram[(API_NDB + 0x1ce) / 2];
        const uint16_t *kc = &api_ram[(API_NDB + 0x2ce) / 2];
        if (a5 != a5_prec || memcmp(kc, kc_prec, sizeof kc_prec)) {
            printf("  [a5-arm] tick=%u d_a5mode=%u a_kc=%04x %04x %04x %04x%s\n", fn, a5, kc[0], kc[1], kc[2],
↪   kc[3],
                   g.sur_tch ? "  (BSP sur le TCH)" : "");
            fflush(stdout);
            a5_prec = a5; memcpy(kc_prec, kc, sizeof kc_prec);
        }
    }
    scruter_dcch(fn);
    scruter_tch(fn);      /* the TCH announcement by the bridge            */
    suivre_tache_tch(api_ram, pg, taches, fn);   /* the switch, by the firmware */
    sonde_dfn(api_ram, pg, taches, fn);
    if (taches) calypso_bsp_dfn_observe(fn, api_ram[(API_W_PAGE(pg) + WP_D_FN) / 2], api_ram[(API_W_PAGE(pg) +
↪   WP_D_TASK_D) / 2]);

    /* B_SYNC_TCH_DL FOR THE TI L1. osmocom (calypso/dsp.c dsp_load_tch_param)
     * sets at the first TCH both B_SYNC_TCH_UL (bit 8 of d_ctrl_tch) AND
     * B_SYNC_TCH_DL (bit 9); the Magnetite TI L1 sets bit 8 alone, its bit 9
     * is named B_SYNC_AMR and stays 0 (d_ctrl_tch 0x01d1 then 0x0091). Without
     * downlink sync the ROM keeps an arbitrary TCH block phase and decodes
     * every FACCH with its last two bursts missing (Fire KO, RR Assignment
     * Failure). Set here, at the end of the tick, on the W page the ROM reads
     * at the next tick. CALYPSO_TI_SYNC_DL=0 disables it. */
    {
        /* n = 1: on the page carrying bit 8; n > 1: on the (n-1)-th following
         * task page (shifts the phase). */
        int sync_dl = calypso_reglage("CALYPSO_TI_SYNC_DL", 1);   /* live, /dev/shm/calypso_reglages */
        static int attente = -1;
        static bool fait_tch;   /* once per TCH */
        uint16_t *ctrl = &api_ram[(API_W_PAGE(pg) + 0x12u) / 2];   /* d_ctrl_tch, word 9 of the W page */
        uint16_t td = api_ram[(API_W_PAGE(pg) + WP_D_TASK_D) / 2] & 0x7fffu;
        if (!g.sur_tch) fait_tch = false;
```

```
        if (sync_dl > 0 && taches) {
            /* ALSO ON THE FIRST TCH TASK: the TI L1 may set d_ctrl_tch=0x0091
             * (without bit 8) from the first TCH page on, so bit 8 alone is
             * not a reliable trigger. */
            if ((*ctrl & 0x0100) && !(*ctrl & 0x0200)) attente = sync_dl - 1;
            else if (td == TCHT_DSP_TASK && !fait_tch && !(*ctrl & 0x0200)) { attente = sync_dl - 1; fait_tch =
             ↪  true; }
            else if (attente > 0) attente--;
            if (attente == 0) {
                *ctrl |= 0x0200;
               printf("  [sync-dl] tick=%u page W%u : d_ctrl_tch |= B_SYNC_TCH_DL -> %04x (reglage %d)\n", fn, pg,
↪  *ctrl, sync_dl);
                fflush(stdout);
                attente = -1;
            }
        }
    }
    /* VOICE BY THE HOST: NO ROM CODEC. With PONT_VOIX_HOTE=1 (pont/voix_hote.py,
     * libgsm) speech no longer goes through the ROM, but the TI L1 sets
     * B_VOCODER_ON (bit 7 of d_ctrl_tch, 0x0091) and the ROM would still
     * encode and decode every frame (8.4 ms per frame for 4.62 available,
     * with the bridge clock slaved to the DSP: the whole network slows down).
     * The bit is cleared on the task W page. CALYPSO_TI_VOCODEUR_OFF (live
     * setting): default 1 if PONT_VOIX_HOTE=1, else 0. */
    {
        static int defaut = -1;
        if (defaut < 0) { const char *e = calypso_getenv("PONT_VOIX_HOTE"); defaut = (e && *e == '1'); }
        if (taches && calypso_reglage("CALYPSO_TI_VOCODEUR_OFF", defaut)) {
            uint16_t *ctrl = &api_ram[(API_W_PAGE(pg) + 0x12u) / 2];   /* d_ctrl_tch, word 9 of the W page */
            if (*ctrl & 0x0080) {
                static unsigned n_voc;
                *ctrl &= (uint16_t)~0x0080u;
                if (n_voc++ == 0) {
                    printf("  [vocodeur-off] tick=%u : B_VOCODER_ON efface de d_ctrl_tch (voix par l'hote) ->
↪  %04x\n", fn, *ctrl);
                    fflush(stdout);
                }
            }
        }
    }
    prothese_facch_dl(api_ram);   /* see above */

    /* THE DEDICATED CHANNEL IS ALSO RELEASED WHEN THE ARM SEARCHES FOR SYNC.
     * The QEMU L1CTL tap does not always announce the release; the BSP would
     * then keep replacing TS0 on the channel frames -- for SS=0 those are
     * fn%%51 = 0..3, and fn%%51=0 carries the FCCH, fn%%51=1 the SCH -- and
     * the mobile would lose its sync for good.
     *
     * An FB (5) or SB (6) task posted by the ARM means it searches for sync on
     * TS0: it is no longer in dedicated mode, whatever the tap says. That
     * signal comes from the firmware itself. */
    {
        uint16_t md0 = api_ram[(API_W_PAGE(0) + WP_D_TASK_MD) / 2] & 0xff;
        uint16_t md1 = api_ram[(API_W_PAGE(1) + WP_D_TASK_MD) / 2] & 0xff;
        bool cherche_synchro = (md0 == FB_DSP_TASK || md0 == SB_DSP_TASK ||
                                md1 == FB_DSP_TASK || md1 == SB_DSP_TASK);
        if (g.dedie_arme && cherche_synchro) {
            printf("  [montant] tache %s postee : le mobile cherche la synchro, "
                   "canal dedie libere (TS0 rendu au FCCH/SCH)\n",
                   (md0 == FB_DSP_TASK || md1 == FB_DSP_TASK) ? "FB" : "SB");
            g.dedie_arme = false;
            g.sur_tch = false;
            g.sd_valide = false;
            calypso_bsp_set_dedie(0, 0xFF, 0);
            montant_canal_libere();
        }
```

```
    }
    sonde_afd(api_ram, fn);
    sonde_add(api_ram, fn);

    /* a_cd PROBE: block by block verdict of the dedicated downlink, the a_cd
     * status word (bit 15 = block present, bit 6 = Fire error) with the frame
     * and its position in the multiframe, to see whether failures follow the
     * SDCCH, the SACCH, or a particular position. MONTANT_ACD=0 disables
     * it. */
    {
        static int sonde = -1;
        if (sonde < 0) { const char *e = calypso_getenv("MONTANT_ACD"); sonde = (e && *e == '0') ? 0 : 1; }
        if (sonde && (g.dedie_arme || g_ti_dedie)) {
            static uint16_t prec;
            static unsigned long ok, ko;
            uint16_t etat = api_ram[(API_NDB + NDB_A_CD) / 2];
            /* TI dedicated mode: every block (etat != prec alone would hide consecutive FIRE KO) */
            static uint8_t d_prec[4];
            const uint8_t *d = (const uint8_t *)api_ram + API_NDB + NDB_A_CD + 6;
            bool nouveau = (etat != prec) || (g_ti_dedie && memcmp(d, d_prec, 4) != 0);
            if ((etat & B_BLUD) && nouveau) {
                memcpy(d_prec, d, 4);
                bool fire = (etat & 0x0040) != 0;
                if (fire) ko++; else ok++;
                if ((ok + ko) <= 60 || ((ok + ko) % 50) == 0) {
                    printf("  [a_cd] fn=%u p51=%u p102=%u etat=%04x %s "
                           "L2=%02x %02x %02x %02x | ok=%lu ko=%lu\n",
                           fn, fn % 51u, fn % 102u, etat, fire ? "FIRE KO" : "ok",
                           d[0], d[1], d[2], d[3], ok, ko);
                }
            }
            prec = etat;
        }
    }

    uint16_t *wp = &api_ram[API_W_PAGE(pg) / 2];
    uint16_t task_u  = taches ? wp[WP_D_TASK_U / 2] : 0;
    uint16_t task_ra = wp[WP_D_TASK_RA / 2];
    uint16_t d_rach  = api_ram[(API_NDB + NDB_D_RACH) / 2];

    /* DEDICATED UPLINK PROBE (Magnetite, classic BTS). At the first ALLC task
     * seen in a W page, 300 lines: tasks of both pages, d_dsp_page, chosen
     * page, a_cu header. And if a_cu carries B_BLUD without an uplink task
     * seen, it is taken anyway (the TI L1 posts it with the block to send,
     * l1ddsp_load_info). */
    {
        static unsigned reste;
        static bool arme;
        uint16_t td0 = api_ram[(API_W_PAGE(0) + WP_D_TASK_D) / 2] & 0x7fffu;
        uint16_t td1 = api_ram[(API_W_PAGE(1) + WP_D_TASK_D) / 2] & 0x7fffu;
        uint16_t cu0 = api_ram[(API_NDB + NDB_A_CU) / 2];
        if (!arme && (td0 == ALLC_DSP_TASK || td1 == ALLC_DSP_TASK)) {
            arme = true;
            reste = 300;
            printf("  [ul-dedie] premiere tache ALLC (tick %u) : sonde du montant sur 300 trames\n", fn);
        }
        if (reste) {
            reste--;
            printf("  [ul-dedie] tick=%u dsp_page=%04x pg=%u W0 d=%u u=%u W1 d=%u u=%u a_cu[0]=%04x\n", fn,
↪   v_page, pg,
                   td0, api_ram[(API_W_PAGE(0) + WP_D_TASK_U) / 2] & 0x7fffu,
                   td1, api_ram[(API_W_PAGE(1) + WP_D_TASK_U) / 2] & 0x7fffu, cu0);
        }
        if (arme && task_u == 0 && (cu0 & B_BLUD))
            capture_sdcch_ul(api_ram, 0, fn);
        /* Dedicated state of the TI firmware: active while an uplink task has
```

```
         * been seen in the last 102 frames. Arms the a_cd probe and the BSP
         * offset in dedicated mode (CALYPSO_BSP_DEDIE_DECAL). */
        /* Based on the UPLINK TASKS (d_task_u, 11/12): ALLC (24) also serves
         * the BCCH and the CCCH in idle, so it would trigger the offset in
         * idle too. */
        static uint32_t derniere_allc;
        static bool vu_allc;
        uint16_t tu0 = api_ram[(API_W_PAGE(0) + WP_D_TASK_U) / 2] & 0x7fffu;
        uint16_t tu1 = api_ram[(API_W_PAGE(1) + WP_D_TASK_U) / 2] & 0x7fffu;
        if (tu0 || tu1) { derniere_allc = fn; vu_allc = true; }
        bool dedie = vu_allc && (uint32_t)(fn - derniere_allc) < 102u;
        if (dedie != g_ti_dedie) {
            g_ti_dedie = dedie;
            calypso_bsp_set_dedie_ti(dedie);
        }
    }

    /* RACH: the task, not the value.
     *
     * The NDB word d_rach (byte 0x474 on the ARM side) is ALSO C54x memory
     * (data[0x0A3A]): the ROM leaves residue there (typically 0xfe00), so
     * triggering on the d_rach value would publish a bogus access burst
     * (ra=0xfe, bsic=0) at every start. The right signal is d_task_ra:
     * prim_rach.c:77 writes dsp_task_iq_swap(RACH_DSP_TASK=10, arfcn, 1)
     * there when it posts the RA, and nothing else is worth 10. Optionally
     * the word is cleared after publication (like
     * qosmo-dsp/calypso_trx.c:1955): the firmware never reads it back (only
     * prim_rach.c:77 and sync.c:307 write it), which gives a clean edge for
     * the next attempt.
     *
     * MONTANT_RACH_SUR_DRACH=1 selects the value trigger (d_rach transition,
     * first value taken as reference) for the case where something would
     * consume d_task_ra before this poll. */
    static int sur_drach = -1;
    if (sur_drach < 0) {
        const char *e = calypso_getenv("MONTANT_RACH_SUR_DRACH");
        sur_drach = (e && *e == '1') ? 1 : 0;
    }
    bool tache_rach = ((task_ra & 0x7fffu) == RACH_DSP_TASK);
    bool front_valeur = false;
    if (!g.base_rach) {
        g.base_rach = true;
        g.prev_rach = d_rach;
    } else if (d_rach != g.prev_rach) {
        front_valeur = (d_rach != 0);
        g.prev_rach = d_rach;
    }
    /* TASK EDGE when the ROM keeps the task (MONTANT_EFFACE_TASK_RA=0): nobody
     * clears d_task_ra then, it stays at RACH_DSP_TASK in the W page, and the
     * RACH_GARDE_TRAMES guard alone would republish the same RACH every 4
     * frames. Publication only happens on the edge (task absent -> present)
     * or on a new d_rach. MONTANT_RACH_DEDUP=0: publish on the guard alone. */
    static int front_seul = -1;
    static bool tache_avant;
    if (front_seul < 0) {
        const char *ea = calypso_getenv("MONTANT_EFFACE_TASK_RA");
        const char *ed = calypso_getenv("MONTANT_RACH_DEDUP");
        front_seul = (ea && *ea == '0') && !(ed && *ed == '0');
        if (front_seul)
            printf("  [montant] RACH publie sur le front de d_task_ra (MONTANT_RACH_DEDUP=0 : toutes les %d
↪ trames)\n",
                   RACH_GARDE_TRAMES);
    }
    bool nouveau_rach = !front_seul || !tache_avant || front_valeur;
    tache_avant = tache_rach;
    /* DEFERRED RACH (MONTANT_RACH_DIFFERE=N frames, Magnetite: 1 via run.sh).
```

```
     * The TI L1 posts d_task_ra BEFORE writing the d_rach of the attempt: read
     * at the edge, d_rach would be the one of the PREVIOUS attempt. The tick of
     * the edge is kept and d_rach is read N frames later. */
    static int differe = -1;
    static bool rach_attente;
    static uint32_t fn_attente;
    if (differe < 0) {
        const char *e = calypso_getenv("MONTANT_RACH_DIFFERE");
        differe = (e && *e) ? atoi(e) : 0;
        if (differe > 0) printf("  [montant] RACH differe de %d trame(s) : d_rach lu apres le front de
         ↪  d_task_ra\n", differe);
    }
    if (rach_attente && (uint32_t)(fn - fn_attente) >= (uint32_t)differe) {
        rach_attente = false;
        if (d_rach != 0)
            publier_rach((uint8_t)(d_rach >> 8), (uint8_t)((d_rach & 0xff) >> 2), fn_attente);
        g.prev_rach = d_rach;
    }
    if (d_rach != 0 && (tache_rach || (sur_drach && front_valeur)) && nouveau_rach &&
        (!g.rach_vu || (uint32_t)(fn - g.fn_rach) >= RACH_GARDE_TRAMES)) {
        if (differe > 0) { rach_attente = true; fn_attente = fn; }
        else publier_rach((uint8_t)(d_rach >> 8), (uint8_t)((d_rach & 0xff) >> 2), fn);
        g.prev_rach = d_rach;
        g.fn_rach = fn;
        g.rach_vu = true;
        if (g.rach <= (unsigned long)journal())
            printf("  [montant]   declencheur : %s (task_ra=0x%04x d_rach=0x%04x)\n",
                   tache_rach ? "d_task_ra=RACH_DSP_TASK" : "transition de d_rach",
                   task_ra, d_rach);
        static int consomme = -1;
        if (consomme < 0) {
            const char *e = calypso_getenv("MONTANT_CONSOMME_RACH");
            consomme = (e && *e == '1') ? 1 : 0;
        }
        if (consomme) {
            api_ram[(API_NDB + NDB_D_RACH) / 2] = 0;
            g.prev_rach = 0;
        }
        /* MONTANT_EFFACE_TASK_RA=1 clears d_task_ra in the W page: the ROM
         * never sees the RACH task (it would read it on the next frame) and
         * the bridge sends the access burst (side-band calypso_rach). */
        /* By default the ROM KEEPS the task: it encodes the access burst
         * (0x3f8a, 36 bits = gsm0503_rach_ext_encode) and its transmitter
         * serialises it in the TSP script, published by pont.c
         * (calypso_tx_rom). */
        static int efface_ra = -1;
        if (efface_ra < 0) {
            const char *e = calypso_getenv("MONTANT_EFFACE_TASK_RA");
            efface_ra = (e && *e == '1') ? 1 : 0;
            printf("  [montant] d_task_ra %s\n", efface_ra ? "efface avant la ROM (MONTANT_EFFACE_TASK_RA=1) :
↪  RACH code par le pont"
                                                           : "laisse a la ROM : elle code l'access-burst
                                                            ↪  (MONTANT_EFFACE_TASK_RA=1 pour l'ancien
                                                            ↪  comportement)");
        }
        if (task_ra && efface_ra) {
            wp[WP_D_TASK_RA / 2] = 0;   /* like qosmo-dsp/calypso_trx.c:1955 */
        }
    }

    /* SDCCH / SACCH / FACCH / speech: same dispatch as the gr-gsm layer 1. */
    if (task_u != 0 && !capture_tch_ul(api_ram, task_u, fn)) {
        capture_sdcch_ul(api_ram, task_u, fn);
    }

    publier_kc(api_ram);
```

```
}

/* The dedicated channel has just been released (PONT_DCCH, kind 0xFF): the
 * duplicate filter memory must restart, otherwise the SABM of the NEXT
 * connection, byte for byte identical to the previous one, would be taken
 * for a duplicate and never leave. */
void montant_canal_libere(void)
{
    g.sdcch_a_dernier = false;
    g_kc_liberer = true;   /* the Kc of THIS connection is worthless now */
}

void montant_bilan(void)
{
    if (g.rach || g.sdcch || g.facch || g.sacch || g.parole) {
        printf("  montant publie : RACH %lu, SDCCH %lu, FACCH %lu, SACCH %lu, parole %lu\n",
               g.rach, g.sdcch, g.facch, g.sacch, g.parole);
    } else {
        printf("  montant : rien publie (aucun d_rach ni d_task_u vu dans l'API RAM)\n");
    }
}
```

3.17 /opt/GSM/c54x_exe/src/montant.h

1074 octets, 29 lignes → 29 lignes

```
/* SPDX-License-Identifier: GPL-2.0-or-later */
/*
 * montant.h - the uplink side of the DSP setup (RACH, SDCCH, SACCH, FACCH, speech).
 */
#ifndef C54X_EXE_MONTANT_H
#define C54X_EXE_MONTANT_H

#include <stdint.h>
#include <stdbool.h>

/* To be called once per frame, after the ARM has handed over (end of
 * scenario: d_dsp_page written, tasks posted in the W page). `page` is bit 0
 * of d_dsp_page, the one QEMU already carries in PONT_TICK.m.b. */
void montant_scruter(uint16_t *api_ram, uint32_t fn, unsigned page);

/* Uplink FACCH read at the GO, before the ROM clears B_BLUD from a_fu. */
void montant_capter_fu(uint16_t *api_ram, uint32_t fn);

/* The dedicated channel is released: forget the last published SDCCH block. */
void montant_canal_libere(void);

/* One summary line at the end of the session. */
void montant_bilan(void);

/* True while the BSP plays the TCH timeslot (switch followed from the firmware). */
bool montant_sur_tch(void);
int  montant_tn_dedie(void);   /* TS of the TCH (side-band), 0 otherwise: label of the published bursts */

#endif
```

3.18 /opt/GSM/c54x_exe/src/pcb-minimal.c

1380 octets, 44 lignes → 44 lignes

```c
/* SPDX-License-Identifier: GPL-2.0-or-later */
/*
 * The four DARAM helpers of calypso_full_pcb.c, copied verbatim.
 *
 * calypso_full_pcb.c as a whole does not build outside QEMU: it includes
 * hw/core/cpu.h and all of QEMU behind it. The helpers the DSP needs do not
 * depend on any of that - one mutex and two array accesses - so they are copied
 * word for word from qosmo-dsp/hw/arm/calypso/calypso_full_pcb.c.
 *
 * This is the only copy in this binary: if the original changes, this file lies.
 * It goes away once calypso_full_pcb.c is decoupled from C54xState.
 */
#include "qemu/osdep.h"
#include "qemu/thread.h"
#include "calypso_c54x.h"

extern QemuMutex calypso_pcb_daram_lock;

uint16_t calypso_dsp_daram_read(void *dsp_void, uint16_t addr)
{
    C54xState *dsp = (C54xState *)dsp_void;
    qemu_mutex_lock(&calypso_pcb_daram_lock);
    uint16_t v = dsp->data[addr];
    qemu_mutex_unlock(&calypso_pcb_daram_lock);
    return v;
}

void calypso_dsp_daram_write(void *dsp_void, uint16_t addr, uint16_t val)
{
    C54xState *dsp = (C54xState *)dsp_void;
    qemu_mutex_lock(&calypso_pcb_daram_lock);
    dsp->data[addr] = val;
    qemu_mutex_unlock(&calypso_pcb_daram_lock);
}

void calypso_pcb_daram_lock_acquire(void)
{
    qemu_mutex_lock(&calypso_pcb_daram_lock);
}

void calypso_pcb_daram_lock_release(void)
{
    qemu_mutex_unlock(&calypso_pcb_daram_lock);
}
```

3.19 /opt/GSM/c54x_exe/src/pont.c

173510 octets, 2905 lignes → 2903 lignes (2 groupes compactés)

```c
/* SPDX-License-Identifier: GPL-2.0-or-later */
/*
 * pont.c - server side of the ARM/DSP bridge.
 *
 * The ARM (osmocom-bb layer1) runs under QEMU (qosmo, CALYPSO_DSP_EXTERN=1);
 * the C54x runs in this process. Both share the API RAM through
 * /dev/shm/calypso_api_ram and lock step frame by frame over
 * /tmp/calypso_dsp.sock. The per-frame sequence below mirrors, section by
 * section, what qosmo-dsp/hw/arm/calypso/calypso_trx.c:calypso_tdma_tick()
 * does with its internal DSP; divergences between the two originate here.
 */
#include <stdio.h>
#include <stdlib.h>
#include <stddef.h>
#include <stdint.h>
```

```c
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <fcntl.h>
#include <poll.h>
#include <signal.h>
#include <sys/mman.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <math.h>
#include <sys/stat.h>
#include <osmocom/core/bits.h>
#include <osmocom/coding/gsm0503_coding.h>
#include "calypso_c54x.h"
#include "calypso_dma.h"
#include "calypso_bsp.h"
#include "calypso_twl3025.h"
#include "calypso_rhea_dma.h"
#include "calypso_a5.h"
#include "calypso_vsp.h"
#include "calypso_rif.h"
#include "hw/arm/calypso/calypso_api.h"
#include "hw/arm/calypso/calypso_dsp_pont.h"
#include "pont.h"
#include "tsp_tx.h"
#include "calypso_gmsk.h"
#include "cellule.h"
#include "montant.h"
#include "carte_tx.h"
#include "hw/arm/calypso/calypso_debug.h"

extern int g_toa_grille, g_toa_valeur;   /* c54x_mem.c: provenance of the TOA */
extern uint32_t g_c54x_exe_fn;          /* main.c: value returned by calypso_trx_get_fn() */

static volatile sig_atomic_t g_stop;
static void sur_signal(int sig) { (void)sig; g_stop = 1; }

/* COOPERATIVE STOP / GO ON THE RUNNING DSP (harvard.sh stop / go).
 * SIGUSR1 requests a pause, taken at the next frame boundary (top of the
 * TICK loop: the DSP has returned its DONE, it is in IDLE, the ARM waits);
 * SIGUSR2 resumes. During the pause, the core state is written to
 * PONT_PAUSE_FICHIER (one "KEY value" line per item, readable by a shell);
 * on resume the first line becomes "ETAT reprise". Nothing is written into
 * the API RAM or the core: the firmware only sees a longer frame, like a
 * SIGSTOP, but at a clean point and with the registers. */
#define PONT_PAUSE_FICHIER "/dev/shm/calypso_dsp_pause"
static volatile sig_atomic_t g_pause;
static void sur_pause(int sig) { g_pause = (sig == SIGUSR1); }
static void pause_ecrire(const C54xState *dsp, const char *etat)
{
    FILE *f = fopen(PONT_PAUSE_FICHIER ".tmp", "w");
    if (!f) return;
    fprintf(f, "ETAT %s\nFN %u\nPC %04x\nXPC %x\nIDLE %d\nRUNNING %d\nINSN %u\nLAST_PC %04x\nLAST_OP %04x\n",
            etat, g_c54x_exe_fn, dsp->pc & 0xffff, dsp->xpc, dsp->idle, dsp->running, dsp->insn_count,
            dsp->last_exec_pc, dsp->last_exec_op);
    fprintf(f, "A %llx\nB %llx\nT %x\nSP %x\nST0 %x\nST1 %x\nPMST %x\nIMR %x\nIFR %x\nBK %x\nBRC %x\nRSA %x\nREA
↪  %x\nTRN %x\n",
            (unsigned long long)(dsp->a & 0xFFFFFFFFFULL), (unsigned long long)(dsp->b & 0xFFFFFFFFFULL),
            dsp->t, dsp->sp, dsp->st0, dsp->st1, dsp->pmst, dsp->imr, dsp->ifr, dsp->bk, dsp->brc, dsp->rsa,
↪  dsp->rea, dsp->trn);
    for (int i = 0; i < 8; i++) fprintf(f, "AR%d %x\n", i, dsp->ar[i]);
    fprintf(f, "RPT %d %x %x\nDELAY %d %x\n", dsp->rpt_active, dsp->rpt_count, dsp->rpt_pc, dsp->delay_slots,
↪  dsp->delayed_pc);
    fclose(f);
    rename(PONT_PAUSE_FICHIER ".tmp", PONT_PAUSE_FICHIER);
}
```

```
static void pause_cooperative(C54xState *dsp)
{
    pause_ecrire(dsp, "pause");
    printf("pont : PAUSE (SIGUSR1) fn=%u pc=%04x xpc=%x idle=%d : etat dans %s, SIGUSR2 pour reprendre\n",
           g_c54x_exe_fn, dsp->pc & 0xffff, dsp->xpc, dsp->idle, PONT_PAUSE_FICHIER);
    fflush(stdout);
    while (g_pause && !g_stop) usleep(10000);
    pause_ecrire(dsp, "reprise");
    printf("pont : REPRISE fn=%u pc=%04x\n", g_c54x_exe_fn, dsp->pc & 0xffff);
    fflush(stdout);
}

C54xState *pont_allouer_dsp(void)
{
    /* Shift the start of the struct so that data[] lands on a page boundary,
     * and therefore data[C54X_API_BASE] too: the API window must be mmap-able
     * with MAP_FIXED. */
    size_t off = offsetof(C54xState, data) % 4096;
    size_t decal = off ? 4096 - off : 0;
    void *brut = NULL;
    if (posix_memalign(&brut, 4096, sizeof(C54xState) + 4096) != 0) {
        fprintf(stderr, "pont : posix_memalign\n");
        return NULL;
    }
    memset(brut, 0, sizeof(C54xState) + 4096);
    C54xState *s = (C54xState *)((char *)brut + decal);
    uint16_t *fenetre = &s->data[C54X_API_BASE];
    if (((uintptr_t)fenetre & 4095) != 0) {
        fprintf(stderr, "pont : fenetre API non alignee (%p)\n", (void *)fenetre);
        return NULL;
    }

    int fd = shm_open(CALYPSO_PONT_SHM, O_RDWR | O_CREAT, 0666);
    if (fd < 0) {
        fprintf(stderr, "pont : shm_open(%s) : %s\n", CALYPSO_PONT_SHM, strerror(errno));
        return NULL;
    }
    /* shm_open applies the umask: 0666 becomes 0644 under root, and a harvard.sh
     * run under the user's account could then no longer WRITE the API RAM (d@!,
     * sb/si3 demos). fchmod forces 0666 so that the shm is writable by everyone. */
    if (fchmod(fd, 0666) < 0) {
        fprintf(stderr, "pont : fchmod(%s, 0666) : %s\n", CALYPSO_PONT_SHM, strerror(errno));
    }
    if (ftruncate(fd, CALYPSO_PONT_SHM_BYTES) < 0) {
        fprintf(stderr, "pont : ftruncate : %s\n", strerror(errno));
        close(fd);
        return NULL;
    }
    void *map = mmap(fenetre, CALYPSO_PONT_SHM_BYTES, PROT_READ | PROT_WRITE,
                     MAP_SHARED | MAP_FIXED, fd, 0);
    close(fd);
    if (map == MAP_FAILED || map != (void *)fenetre) {
        fprintf(stderr, "pont : mmap MAP_FIXED : %s\n", strerror(errno));
        return NULL;
    }
    memset(fenetre, 0, CALYPSO_PONT_SHM_BYTES);
    return s;
}

static bool envoyer(int fd, uint32_t type, uint32_t a, uint32_t b)
{
    CalypsoPontMsg m = { type, a, b };
    return send(fd, &m, sizeof(m), MSG_NOSIGNAL) == (ssize_t)sizeof(m);
}

/* PC profile: c54x_run is driven in slices of 64 instructions and the PC
```

```
 * sampled between two slices is charged to a 64-word bucket. No sampling
 * thread, no cost. Published every 217 frames. */
#define PROFIL_SEAU 64
static unsigned long g_profil[0x10000 / PROFIL_SEAU];
static uint32_t g_profil_hw;     /* high-water mark: highest PC seen (boot excluded) */
/* Trace window: PONT_TRACE_FB=N logs the next N instructions (pc, opcode, A,
 * AR3, DP, INTM, IMR/IFR) to /tmp/c54x-pont/trace-fb.txt, starting at the
 * first frame whose write page carries d_task_md=5 (FB_DSP_TASK). Exact
 * single-stepping (c54x_run with n=1). */
static long g_trace_reste = -1;
static FILE *g_trace_f;
static bool g_trace_ouverte;
static uint32_t g_trace_pc_lo, g_trace_pc_hi;   /* PONT_TRACE_PC=lo-hi: open the trace when PC enters [lo,hi] */
static void trace_armer(void)
{
    static bool fait;
    if (fait) return;
    fait = true;
    const char *e = calypso_getenv("PONT_TRACE_FB");
    const char *r = calypso_getenv("PONT_TRACE_PC");
    if (e && *e) { g_trace_reste = atol(e); g_trace_f = fopen("/tmp/c54x-pont/trace-fb.txt", "w"); }
    if (r && *r) {
        char *fin = NULL; g_trace_pc_lo = (uint32_t)strtoul(r, &fin, 0);
        g_trace_pc_hi = (fin && *fin == '-') ? (uint32_t)strtoul(fin + 1, NULL, 0) : g_trace_pc_lo;
        if (g_trace_reste <= 0) g_trace_reste = 4000;
        if (!g_trace_f) g_trace_f = fopen("/tmp/c54x-pont/trace-fb.txt", "w");
    }
}
/* DARAM cells watched during the trace: every change is logged with the PC of
 * the instruction that made it (before/after diff on each step). */
/* Cells watched during the trace. Rebuilt [2026-09-19]: the previous list had
 * accreted 13 addresses (0x3fc1 0x3fde 0x3fd3 0x0c3f 0x0906 0x058a 0x0e60
 * 0x435b 0x0e4f 0x2a01 ...) that appear NOWHERE else in the tree and whose
 * introducing commit (812c343) says only "commit". An address whose meaning
 * cannot be restated is not evidence: when it moves, nothing follows. Every
 * entry below carries where its meaning comes from.
 *
 * API window cells are derived from calypso_api.h rather than written raw, so
 * they cannot drift from the map: API base = C54X_API_BASE = 0x0800 words, and
 * the API_* offsets are in BYTES (hence the /2). */
#define CEL_W(p, off)   (uint16_t)(C54X_API_BASE + (API_W_PAGE(p) + (off)) / 2)
#define CEL_R(p, off)   (uint16_t)(C54X_API_BASE + (API_R_PAGE(p) + (off)) / 2)
#define CEL_NDB(off)    (uint16_t)(C54X_API_BASE + (API_NDB + (off)) / 2)

static const uint16_t g_cellules[] = {
    /* --- ARM -> DSP pages (calypso_api.h) ------------------------------- */
    CEL_W(0, WP_D_TASK_MD), CEL_W(1, WP_D_TASK_MD),   /* 0x0804/0x0818 the posted task */
    CEL_W(0, WP_D_TASK_D),  CEL_W(1, WP_D_TASK_D),    /* 0x0800/0x0814 */
    CEL_W(0, WP_D_FN),      CEL_W(1, WP_D_FN),        /* 0x0808/0x081c */

    /* --- NDB (calypso_api.h + calypso_fbsb.h) --------------------------- */
    CEL_NDB(NDB_D_DSP_PAGE),                          /* 0x08d4 page toggle */
    CEL_NDB(NDB_D_FB_DET),                            /* 0x08f8 FB found flag */
    CEL_NDB(NDB_A_SYNC_DEMOD + 2 * D_TOA),            /* 0x08fa */
    CEL_NDB(NDB_A_SYNC_DEMOD + 2 * D_PM),             /* 0x08fb */
    CEL_NDB(NDB_A_SYNC_DEMOD + 2 * D_ANGLE),          /* 0x08fc  <- was MISSING */
    CEL_NDB(NDB_A_SYNC_DEMOD + 2 * D_SNR),            /* 0x08fd  <- was MISSING */

    /* --- DSP -> ARM pages: a_sch[0,1,3,4], BOTH pages ------------------- */
    CEL_R(0, RP_A_SCH + 0), CEL_R(0, RP_A_SCH + 2),
    CEL_R(0, RP_A_SCH + 6), CEL_R(0, RP_A_SCH + 8),   /* 0x0837 38 3a 3b */
    CEL_R(1, RP_A_SCH + 0), CEL_R(1, RP_A_SCH + 2),
    CEL_R(1, RP_A_SCH + 6), CEL_R(1, RP_A_SCH + 8),   /* 0x084b 4c 4e 4f */

    /* --- FB correlator input, read out of the mask ROM [2026-09-19] ------ */
    0x3fb5,   /* pointer to the input buffer; PROM0 0xb2c4 ST #0x0cce,*(0x3fb5)
```

```
             *                              and 0xb2c9 ST #0x0d2e,*(0x3fb5) */
    0x0cce,   /* buffer A, same two instructions; also the AAD the DMA uses */
    0x0d2e,   /* buffer B, idem */
    0x0e4e,   /* second DARAM target seen in the rhea-dma RX transfers */
    0x2a00,   /* CALYPSO_BSP_DARAM_ADDR env default, before AAD_FOLLOW */

    /* --- cells whose meaning is stated elsewhere in the tree ------------- */
    0x3f92,   /* calypso_dma.h:9   source of d_error_status (0x08d5) */
    0x43d8,   /* calypso_bsp.c:1567  poked per burst on the FB/SB mission */
    0x43d5,   /* c54x_mem.c:2229   PROM 0xb4be stm #0x43d5 ; reada *AR1+ */
    0x098c,   /* calypso_mailbox.c:46  mailbox poll (0xde86 ld *(0x098c)) */
    0x435e,   /* calypso_c54x.c:4574  bit13 = DMA config lock */
    0x4368,   /* c54x_mem.c:1835   DISPATCH-CELL-RESEED */
    0x3fb0,   /* c54x_probes.c:505  BSP read window 0x3fb0..0x3fbf */
    0x0000 };
#define N_CELLULES (sizeof(g_cellules) / sizeof(g_cellules[0]))
static uint16_t g_cell_prev[N_CELLULES];
static uint16_t g_trace_pc_prev;
/* Extra areas watched during an injection trace (PONT_TX_TRACE_BLOC): a_cu (coder input)
 * and the output buffer 0x4280.. + 0x2be4..; every write is logged with the PC of the
 * instruction that made it. */
static uint16_t g_zone_prev[0x10000];
static bool g_zone_active;
static void trace_zones(C54xState *dsp, bool init)
{
    static const struct { unsigned a, n; } z[] = { {0x0a06, 16}, {0x4280, 32}, {0x2be4, 52}, {0x2bc0, 8}, {0x3f8a,
     ↪  8} };
    for (unsigned r = 0; r < sizeof z / sizeof z[0]; r++)
        for (unsigned k = 0; k < z[r].n; k++) {
            unsigned a = z[r].a + k; uint16_t v = dsp->data[a];
            if (!init && v != g_zone_prev[a] && g_trace_f)
                fprintf(g_trace_f, "      Z data[%04x] %04x -> %04x   (par pc=%04x)\n", a, g_zone_prev[a], v,
 ↪  g_trace_pc_prev);
            g_zone_prev[a] = v;
        }
}
static void trace_cellules(C54xState *dsp, bool init)
{
    if (g_zone_active) trace_zones(dsp, init);
    for (unsigned i = 0; i < N_CELLULES; i++) {
        uint16_t v = dsp->data[g_cellules[i]];
        if (!init && v != g_cell_prev[i] && g_trace_f)
            fprintf(g_trace_f, "      W data[%04x] %04x -> %04x   (par pc=%04x)\n",
                    g_cellules[i], g_cell_prev[i], v, g_trace_pc_prev);
        g_cell_prev[i] = v;
    }
}

static inline void trace_pas(C54xState *dsp)
{
    if (g_trace_reste <= 0 || !g_trace_f || !g_trace_ouverte) return;
    trace_cellules(dsp, false);
    g_trace_pc_prev = dsp->pc & 0xffff;
    uint16_t op = dsp->prog[dsp->pc & 0x3ffff];
    uint16_t op2 = dsp->prog[(dsp->pc + 1) & 0x3ffff];
    fprintf(g_trace_f, "%04x  %04x %04x  A=%010llx B=%010llx AR1=%04x AR2=%04x AR3=%04x AR4=%04x DP=%03x INTM=%d
 ↪  IMR=%04x IFR=%04x SP=%04x T=%04x BRC=%04x RSA=%04x REA=%04x ST0=%04x ST1=%04x insn=%u\n",
            dsp->pc & 0xffff, op, op2,
            (unsigned long long)(dsp->a & 0xfffffffffULL), (unsigned long long)(dsp->b & 0xfffffffffULL),
            dsp->ar[1], dsp->ar[2], dsp->ar[3], dsp->ar[4], dsp->st0 & 0x1ff, !!(dsp->st1 & 0x800),
            dsp->imr, dsp->ifr, dsp->sp, dsp->t, dsp->brc, dsp->rsa, dsp->rea, dsp->st0, dsp->st1,
 ↪  dsp->insn_count);
    if (--g_trace_reste == 0) { fclose(g_trace_f); g_trace_f = NULL; printf("pont : trace FB terminee
     ↪  (/tmp/c54x-pont/trace-fb.txt)\n"); }
}
```

```c
/* PONT_PC_COUNT=pc1,pc2,... : exact hit counters, published with the profile.
 * Forces single-stepping, about 3x slower; diagnostic use only. */
static uint16_t g_pcc[16]; static unsigned long g_pcc_n[16]; static int g_pcc_k = -1;
static void pcc_armer(void)
{
    if (g_pcc_k >= 0) return;
    g_pcc_k = 0;
    const char *e = calypso_getenv("PONT_PC_COUNT");
    while (e && *e && g_pcc_k < 16) {
        char *fin = NULL; long v = strtol(e, &fin, 0);
        if (fin == e) break;
        g_pcc[g_pcc_k++] = (uint16_t)v;
        e = (*fin == ',') ? fin + 1 : fin;
    }
}
/* PONT_DUMP_DATA=addr:n,addr:n : DARAM words printed along with the profile. */
static void dump_publier(C54xState *dsp)
{
    const char *e = calypso_getenv("PONT_DUMP_DATA");
    while (e && *e) {
        char *fin = NULL; long a = strtol(e, &fin, 0); long n = 8;
        if (fin == e) break;
        if (*fin == ':') n = strtol(fin + 1, &fin, 0);
        printf("  data[%04lx..] :", a);
        for (long i = 0; i < n && i < 32; i++) printf(" %04x", dsp->data[(a + i) & 0xffff]);
        printf("\n");
        e = (*fin == ',') ? fin + 1 : fin;
    }
}
static void pcc_publier(void)
{
    if (g_pcc_k <= 0) return;
    printf("  passages exacts :");
    for (int i = 0; i < g_pcc_k; i++) { printf(" %04x:%lu", g_pcc[i], g_pcc_n[i]); g_pcc_n[i] = 0; }
    printf("\n");
}

/* ── TX PROBE: does the ROM execute the uplink tasks? ──────────────────
 * PONT_TX_SONDE=N (N events, 0/absent = inert). On every frame where the ARM
 * posted d_task_u (or d_task_ra) in one of the W pages, the DSP data memory is
 * compared before and after the frame's execution, looking for what the ROM
 * would have written: (a) the modified ranges, (b) the runs of >= 100 words
 * holding only 0 or 1 — the shape of the 148 hard bits of an uplink burst,
 * which calypso_bsp_tx_burst() reads at the CANDIDATE address data[0x0900].
 * Modifies nothing. */
static uint16_t *g_tx_snap;
static int g_tx_n = -1, g_tx_arme;
static int tx_sonde_active(void)
{
    if (g_tx_n < 0) { const char *e = calypso_getenv("PONT_TX_SONDE"); g_tx_n = (e && *e) ? atoi(e) : 0; }
    return g_tx_n > 0;
}
static void tx_sonde_avant(const C54xState *dsp, const uint16_t *api_ram)
{
    (void)api_ram;
    /* Snapshot on EVERY frame: the ARM posts its tasks AFTER the TICK (l1_sync
     * runs between the TICK and the GO), so d_task_u is still 0 here and
     * arming on its value at this moment never triggers. The decision is
     * taken in tx_sonde_apres(). */
    g_tx_arme = 0;
    if (!tx_sonde_active()) return;
    if (!g_tx_snap) g_tx_snap = malloc(sizeof(uint16_t) * C54X_DATA_SIZE);
    if (!g_tx_snap) return;
    memcpy(g_tx_snap, dsp->data, sizeof(uint16_t) * C54X_DATA_SIZE);
    g_tx_arme = 1;
}
```

```
static void tx_sonde_apres(const C54xState *dsp, const uint16_t *api_ram, uint32_t fn)
{
    if (!g_tx_arme || !g_tx_snap || g_tx_n <= 0) return;
    g_tx_arme = 0;
    /* triggers if an uplink task is posted in one of the W pages, and ALSO
     * on the next frame (the ROM may execute it one frame later). */
    static int suite;
    int pose = api_ram[(API_W_PAGE(0) + WP_D_TASK_U) / 2] || api_ram[(API_W_PAGE(1) + WP_D_TASK_U) / 2] ||
               api_ram[(API_W_PAGE(0) + 0x06u) / 2] || api_ram[(API_W_PAGE(1) + 0x06u) / 2];
    if (!pose && !suite) return;
    suite = pose ? 1 : 0;
    unsigned changes = 0, a, nplages = 0, runs = 0;
    for (a = 0; a < C54X_DATA_SIZE; a++) if (dsp->data[a] != g_tx_snap[a]) changes++;
    /* a frame where the ROM touches nothing (task already handled, ROM at
     * rest) does not count among the N events: it would drown the measurement. */
    if (changes == 0) return;
    g_tx_n--;
    printf("  [tx-sonde] fn=%u task_u: page0=0x%04x page1=0x%04x task_ra: page0=0x%04x page1=0x%04x pc=0x%04x\n",
↪   fn,
           api_ram[(API_W_PAGE(0) + WP_D_TASK_U) / 2], api_ram[(API_W_PAGE(1) + WP_D_TASK_U) / 2],
           api_ram[(API_W_PAGE(0) + 0x06u) / 2], api_ram[(API_W_PAGE(1) + 0x06u) / 2], dsp->pc);
    printf("  [tx-sonde]   %u mots de donnees modifies pendant la trame\n", changes);
    /* The modified ranges of at least 8 words, AND THEIR CONTENT in
     * /tmp/tx-sonde/fn<fn>_<adresse>.txt (one hex word per line): enough to
     * search offline for the expected encoded block, in all its forms (one
     * bit per word, 16 bits per word...). */
    mkdir("/tmp/tx-sonde", 0777);
    for (a = 0; a < C54X_DATA_SIZE && nplages < 12; ) {
        if (dsp->data[a] == g_tx_snap[a]) { a++; continue; }
        unsigned b = a; unsigned n = 0;
        while (b < C54X_DATA_SIZE && (dsp->data[b] != g_tx_snap[b] || (b + 1 < C54X_DATA_SIZE && dsp->data[b + 1]
        ↪  != g_tx_snap[b + 1]))) { b++; n++; }
        if (n >= 8) {
            char nom[96]; snprintf(nom, sizeof nom, "/tmp/tx-sonde/fn%u_%04x.txt", fn, a);
            FILE *f = fopen(nom, "w");
            if (f) { for (unsigned k = a; k < b; k++) fprintf(f, "%04x\n", dsp->data[k]); fclose(f); }
            printf("  [tx-sonde]   modifie data[0x%04x..0x%04x] (%u mots) -> %s\n", a, b - 1, n, nom);
            nplages++;
        }
        a = b + 1;
    }
    /* runs of >= 100 words holding 0 or 1, at least 40 of them modified */
    for (a = 0; a < C54X_DATA_SIZE && runs < 4; ) {
        if (dsp->data[a] > 1) { a++; continue; }
        unsigned b = a, ch = 0, uns = 0;
        while (b < C54X_DATA_SIZE && dsp->data[b] <= 1) { if (dsp->data[b] != g_tx_snap[b]) ch++; if
        ↪  (dsp->data[b]) uns++; b++; }
        if (b - a >= 100 && ch >= 40 && uns > 0) {
            printf("  [tx-sonde]   SUITE 0/1 data[0x%04x..0x%04x] long=%u modifies=%u uns=%u :", a, b - 1, b - a,
↪   ch, uns);
            for (unsigned k = 0; k < 24 && a + k < b; k++) printf(" %u", dsp->data[a + k]);
            printf("\n"); runs++;
        }
        a = b + 1;
    }
    if (!runs) printf("  [tx-sonde]   aucune suite de bits 0/1 (mot a mot)\n");
    fflush(stdout);
}

/* ── INJECTION OF L2 BLOCKS INTO THE ROM (linearity experiment) ──────────────
 * PONT_TX_INJECT=<file>: one line per 23-byte hex block. On every uplink
 * block (a_cu) the firmware posts, AFTER montant.c has published it (so the
 * real link is untouched: the BTS gets the real block), the test block is
 * written into a_cu in its place, so that the ROM encodes it on the next
 * frame. The coding being linear, a null block then single-bit blocks give
 * the map between input bits and output bits of the buffer read by the TX
```

```
 * probe (PONT_TX_SONDE). Same byte order as prendre_ul() (montant.c):
 * word 3 + i/2, low byte first. Does nothing if the variable is absent. */
#define TX_INJ_MAX 600
static uint8_t g_inj_bloc[TX_INJ_MAX][23];
static C54xState *dsp_global_tx;
static int g_inj_n = -1, g_inj_i, g_inj_attente = -1;
static void tx_inject_charger(void)
{
    const char *f = calypso_getenv("PONT_TX_INJECT");
    g_inj_n = 0;
    if (!f || !*f) return;
    FILE *fp = fopen(f, "r");
    if (!fp) { printf("  [tx-inject] %s illisible\n", f); return; }
    char ligne[256];
    while (g_inj_n < TX_INJ_MAX && fgets(ligne, sizeof ligne, fp)) {
       unsigned v[23]; int k = sscanf(ligne, "%x %x %x %x %x %x %x %x %x %x %x %x %x %x %x %x %x %x %x %x %x %x %x",
            &v[0],&v[1],&v[2],&v[3],&v[4],&v[5],&v[6],&v[7],&v[8],&v[9],&v[10],&v[11],&v[12],&v[13],&v[14],&v[15
            ↪  ],&v[16],&v[17],&v[18],&v[19],&v[20],&v[21],&v[22]);
        if (k != 23) continue;
        for (int i = 0; i < 23; i++) g_inj_bloc[g_inj_n][i] = (uint8_t)v[i];
        g_inj_n++;
    }
    fclose(fp);
    printf("  [tx-inject] %d blocs de test charges depuis %s\n", g_inj_n, f);
}
/* avant = B_BLUD of a_cu BEFORE montant_scruter(): a block has just been posted. */
static void tx_inject_apres(uint16_t *api_ram, uint32_t fn, int avant)
{
    if (g_inj_n < 0) tx_inject_charger();
    if (g_inj_n == 0 || !avant) return;
    if (g_inj_i >= g_inj_n) {
       if (g_inj_i == g_inj_n) { printf("  [tx-inject] fn=%u fin : %d blocs injectes\n", fn, g_inj_n); g_inj_i++; }
        return;
    }
    uint16_t *w = &api_ram[(API_NDB + NDB_A_CU) / 2];
    const uint8_t *b = g_inj_bloc[g_inj_i];
    for (int i = 0; i < 23; i += 2)
        w[3 + i / 2] = (uint16_t)(b[i] | ((i + 1 < 23 ? b[i + 1] : 0) << 8));
    g_inj_attente = g_inj_i;   /* its output will be read at the end of the next frame */
    { static long cible = -2;
      if (cible == -2) { const char *e = calypso_getenv("PONT_TX_TRACE_BLOC"); cible = (e && *e) ? atol(e) : -1; }
      if (cible == g_inj_i && !g_trace_f) {
          const char *e = calypso_getenv("PONT_TX_TRACE_N");
          g_trace_reste = (e && *e) ? atol(e) : 250000;
          g_trace_f = fopen("/tmp/c54x-pont/trace-tx.txt", "w");
          if (g_trace_f) { g_zone_active = true; g_trace_ouverte = true; trace_cellules(dsp_global_tx, true);
              fprintf(g_trace_f, "# trace du bloc injecte %d a fn=%u\n", g_inj_i, fn);
              printf("  [tx-inject] trace ouverte (bloc %d, %ld instructions) -> /tmp/c54x-pont/trace-tx.txt\n",
↪  g_inj_i, g_trace_reste); }
      } }
    printf("  [tx-inject] fn=%u bloc=%d ->", fn, g_inj_i);
    for (int i = 0; i < 23; i++) printf(" %02x", b[i]);
    printf("\n");
    g_inj_i++;
}

/* End of the frame following the injection: for this block, write the ranges
 * WHERE THE ROM DEPOSITED its output (found by the TX probe: 0x4280.. and
 * 0x2be4..), at FIXED ADDRESSES, whether they changed or not.
 * /tmp/tx-sonde/inj_<bloc>.txt: "address word" per line. Independent of the
 * probe's event counter. */
static void tx_inject_fulldump(const C54xState *dsp, int bloc, uint32_t fn, int d)
{
    static int actif=-1;
    if (actif<0){const char*e=calypso_getenv("PONT_TX_FULLDUMP");actif=(e&&*e=='1');}
    if(!actif) return;
```

```
    if (d > 6) return;
    mkdir("/tmp/tx-full",0777);
    char nom[96]; snprintf(nom,sizeof nom,"/tmp/tx-full/full_%d_%d.txt",bloc,d);
    FILE*f=fopen(nom,"w");
    if(f){fprintf(f,"# fn=%u bloc=%d\n",fn,bloc);
        for(unsigned a=0x2800;a<0x5c00;a++) fprintf(f,"%04x %04x\n",a,dsp->data[a]);
        fclose(f);}
}
static void tx_inject_sortie(const C54xState *dsp, uint32_t fn)
{
    /* the ROM may finish its work over SEVERAL frames (the FIRE parity, the filling of the next
     * bursts): the same ranges are recorded during the 8 frames following the injection,
     * /tmp/tx-sonde/injf_<bloc>_<d>.txt (d = frames after the injection), in addition to
     * inj_<bloc>.txt of the first frame. */
    static int restant = 0, bloc = -1, d = 0;
    if (g_inj_attente >= 0) { bloc = g_inj_attente; restant = 8; d = 0; }
    if (restant <= 0) { g_inj_attente = -1; return; }
    tx_inject_fulldump(dsp, bloc, fn, d + 1);
    static const struct { unsigned a, n; } plages[] = {
        {0x4280, 32}, {0x2be4, 52}, {0x2bc0, 8}, {0x3dc6, 9}, {0x5abb, 12}, {0x3f8a, 8}, {0x0809, 8}, {0x081d, 8},
         ↪  {0x0a06, 15}
    };
    mkdir("/tmp/tx-sonde", 0777);
    d++;
    char nom[96];
    if (d == 1) snprintf(nom, sizeof nom, "/tmp/tx-sonde/inj_%d.txt", bloc);
    else        snprintf(nom, sizeof nom, "/tmp/tx-sonde/injf_%d_%d.txt", bloc, d);
    FILE *f = fopen(nom, "w");
    if (f) {
        fprintf(f, "# fn=%u bloc=%d d=%d\n", fn, bloc, d);
        for (unsigned r = 0; r < sizeof plages / sizeof plages[0]; r++)
            for (unsigned k = 0; k < plages[r].n; k++)
                fprintf(f, "%04x %04x\n", plages[r].a + k, dsp->data[plages[r].a + k]);
        fclose(f);
    }
    restant--;
    g_inj_attente = -1;
}

/* ── SHADOW MODE: the ROM encodes, the host encodes, both are compared ──────────
 * PONT_TX_OMBRE=1. On every REAL uplink SDCCH block (a_cu, B_BLUD set by the
 * firmware), after montant.c has published it, its 23 bytes are kept; at the
 * end of the next frame the buffer the ROM filled is read (data[0x4280],
 * 4 groups of 8 words = 4 bursts) and compared with the bits of
 * gsm0503_xcch_encode (the one the bridge uses). Nothing is written into the
 * DSP or sent: the link stays the host's. Layout: bit j of burst g at column
 * q = j - 2g in the group (first half, q < 57) and q = j - 2g - 2 (second half,
 * after the stealing bit at q = 57). Compares q in [0,50) and [58,106): safe
 * zone for g = 0..3. Output: "[ombre] fn=.. g0 a/b g1 .." + totals, and the
 * detail in /tmp/tx-sonde/ombre.txt. */
static int g_omb = -1, g_omb_att;
static int g_txf_restant, g_txf_d; static uint8_t g_txf_l2[23];
static int g_rt_restant, g_rt_d, g_rt_n; static uint8_t g_rt_l2[23];
static uint8_t g_omb_l2[23];
static unsigned long g_omb_blocs, g_omb_parfaits, g_omb_ok[4], g_omb_n[4];
/* ── DOWNLINK SHADOW: rewrite at the source what the ARM is about to read ──────
 * CALYPSO_SHADOW_LAC=<n> and CALYPSO_SHADOW_SB=1. a_cd (NDB) carries the
 * decoded downlink L2 block, a_sch the decoded SCH; the ARM reads them AFTER
 * the DONE. In lockstep the DSP writes then the ARM reads: this steps in just
 * before the DONE, so what is set is exactly what the firmware receives
 * (deterministic, no race).
 *   - SI3: if a_cd is a System Information 3 (L2 type byte = 0x1b), the 2 LAC
 *     bytes (LAI) are replaced by n -> mobile.log: CGI=MCC-MNC-n-CI. FIRE flag
 *     unchanged, no CRC protects the L3. (The cell_id is not touched.)
 *   - SB: a_sch (R0/R1) <- DEAD BEEF. DESTRUCTIVE: the firmware loses SCH sync;
 *     only enable it to show the loss of the cell.
```

```
 * Offsets: a_cd[3]=start of L2 (dsp_memcpy_from_api be=0, 2 bytes/word, low byte first)
 * so type=low byte of a_cd[4], cell_id = high byte of a_cd[4] (MSB) | low byte of a_cd[5] (LSB). */
static void ombre_descendante(uint16_t *api_ram, uint32_t fn)
{
    static int lac = -2, sb = -2;
    if (lac == -2) { const char *e = calypso_getenv("CALYPSO_SHADOW_LAC");
                     lac = (e && *e) ? atoi(e) : -1;
                     if (lac >= 0) printf("  [ombre-dl] LAC SI3 force a %d (0x%04x)\n", lac, lac & 0xffff); }
    if (sb == -2)  { const char *e = calypso_getenv("CALYPSO_SHADOW_SB");
                     sb = (e && *e == '1') ? 1 : 0;
                     if (sb) printf("  [ombre-dl] SB a_sch force a 40C3 2026 (CCC 2026 ; DESTRUCTIF : perte de
                      ↪  synchro)\n"); }
    if (!api_ram) return;
    if (lac >= 0) {
        /* LAI in SI3: L2 bytes 8-9 = LAC (big-endian). dsp_memcpy be=0 puts both into a_cd[7]:
         * byte 8 (LAC MSB) = low byte of a_cd[7], byte 9 (LAC LSB) = high byte. Hence
         * a_cd[7] = (LAC_LSB << 8) | LAC_MSB = byte-swapped LAC. The cell_id (a_cd[4..5]) is NOT touched. */
        uint16_t *w = &api_ram[(API_NDB + NDB_A_CD) / 2];
        if ((w[4] & 0xff) == 0x1b) {                      /* SI3 (L2 type = low byte of a_cd[4]) */
            uint16_t cible = (uint16_t)(((lac & 0xff) << 8) | ((lac >> 8) & 0xff));
            if (w[7] != cible) {
                int av = ((w[7] & 0xff) << 8) | ((w[7] >> 8) & 0xff);
                w[7] = cible;
                static unsigned long n;
                if (++n <= 5 || n % 200 == 0)
                    printf("  [ombre-dl] fn=%u SI3 LAC %d -> %d (#%lu)\n", fn, av, lac & 0xffff, n);
            }
        }
    }
    if (sb) {
       for (unsigned base = 0x0037; ; base = 0x004b) {    /* a_sch[0] R0=0x0837, R1=0x084b; index = adr - 0x0800 */
            api_ram[base] = 0x40c3; api_ram[base + 1] = 0x2026;   /* CCC 2026, readable marker */
            if (base == 0x004b) break;
        }
    }
}
static void tx_ombre_pose(const uint16_t *api_ram, int blud_avant)
{
    if (g_omb < 0) { const char *e = calypso_getenv("PONT_TX_OMBRE"); g_omb = (e && *e == '1'); if (g_omb)
     ↪  printf("  [ombre] actif\n"); }
    if (!g_omb || !blud_avant) return;
    const uint16_t *w = &api_ram[(API_NDB + NDB_A_CU) / 2];
    for (int i = 0; i < 23; i += 2) {
        g_omb_l2[i] = (uint8_t)(w[3 + i / 2] & 0xff);
        if (i + 1 < 23) g_omb_l2[i + 1] = (uint8_t)(w[3 + i / 2] >> 8);
    }
    g_omb_att = 1;
    { int a=-1; static int actif=-1; if(actif<0){const
     ↪  char*e=calypso_getenv("PONT_TX_TXFRAMES");actif=(e&&*e=='1');}
      if(actif){ memcpy(g_txf_l2,g_omb_l2,23); g_txf_restant=10; g_txf_d=0; } (void)a; }
    { static int ar=-1; if(ar<0){const char*e=calypso_getenv("PONT_TX_REELFULL");ar=(e&&*e=='1');}
      if(ar){ memcpy(g_rt_l2,g_omb_l2,23); g_rt_restant=6; g_rt_d=0; } }
}
/* PONT_TX_TXFRAMES=1: after each real block posted (a_cu), data[0x4280] (32 words) is recorded
 * on EVERY frame for 10 frames, with the L2 and the offset d, in /tmp/tx-sonde/txf.txt.
 * Purpose: the parity lanes of the burst may only be written at the TRANSMIT frame of each
 * burst (one per burst), after the coding frame. */
static void tx_txframes(const C54xState *dsp)
{
    static int actif=-1;
    if(actif<0){const char*e=calypso_getenv("PONT_TX_TXFRAMES");actif=(e&&*e=='1');}
    if(!actif||g_txf_restant<=0) return;
    g_txf_d++;
    mkdir("/tmp/tx-sonde",0777);
    FILE*f=fopen("/tmp/tx-sonde/txf.txt","a");
    if(f){for(int i=0;i<23;i++)fprintf(f,"%02x",g_txf_l2[i]);
```

```
        fprintf(f," %d ",g_txf_d);
        for(int w=0;w<32;w++)fprintf(f,"%04x",dsp->data[0x4280+w]);
        fprintf(f,"\n");fclose(f);}
    g_txf_restant--;
}
static void tx_reelfull(const C54xState *dsp, uint32_t fn)
{
    static int actif=-1;
    if(actif<0){const char*e=calypso_getenv("PONT_TX_REELFULL");actif=(e&&*e=='1');}
    if(!actif||g_rt_restant<=0||g_rt_n>=600) return;
    g_rt_d++;
    mkdir("/tmp/tx-rt",0777);
    char nom[96]; snprintf(nom,sizeof nom,"/tmp/tx-rt/rt_%04d_%d.txt",g_rt_n,g_rt_d);
    FILE*f=fopen(nom,"w");
    if(f){for(int i=0;i<23;i++)fprintf(f,"%02x",g_rt_l2[i]);fprintf(f,"\n");
        for(unsigned a=0x0800;a<0x0e00;a++)fprintf(f,"%04x %04x\n",a,dsp->data[a]);
        for(unsigned a=0x2a00;a<0x2d00;a++)fprintf(f,"%04x %04x\n",a,dsp->data[a]);
        for(unsigned a=0x4200;a<0x4300;a++)fprintf(f,"%04x %04x\n",a,dsp->data[a]);
        fclose(f);}
    g_rt_restant--; if(g_rt_restant==0) g_rt_n++;
}
static void tx_ombre_compare(const C54xState *dsp, uint32_t fn)
{
    tx_txframes(dsp);
    tx_reelfull(dsp, fn);
    if (g_omb <= 0 || !g_omb_att) return;
    g_omb_att = 0;
    ubit_t e[1024] = {0};
    if (gsm0503_xcch_encode(e, g_omb_l2) != 0) return;
    unsigned ok = 0, n = 0, okg[4] = {0}, ng[4] = {0};
    mkdir("/tmp/tx-sonde", 0777);
    FILE *f = fopen("/tmp/tx-sonde/ombre.txt", "a");
    if (f) { fprintf(f, "# fn=%u L2=", fn); for (int i = 0; i < 23; i++) fprintf(f, "%02x ", g_omb_l2[i]);
     ↪  fprintf(f, "\n"); }
    for (int p = 0; p < 512; p++) {
        int i = g_carte_tx[p];
        if (i < 0) continue;
        unsigned bit = (dsp->data[0x4280 + p / 16] >> (15 - p % 16)) & 1u, att = e[i] & 1u;
        n++; ng[p / 128]++;
        if (bit == att) { ok++; okg[p / 128]++; }
        else if (f) fprintf(f, "  p=%d (g%d q=%d) i=%d rom=%u hote=%u\n", p, p / 128, p % 128, i, bit, att);
    }
    if (f) fclose(f);
    FILE *g = fopen("/tmp/tx-sonde/reel.txt", "a");
    if (g) {
        for (int i = 0; i < 23; i++) fprintf(g, "%02x", g_omb_l2[i]);
        fprintf(g, " ");
        for (int w = 0; w < 32; w++) fprintf(g, "%04x", dsp->data[0x4280 + w]);
        fprintf(g, "\n"); fclose(g);
    }
    g_omb_blocs++; g_omb_parfaits += (ok == n);
    for (int g = 0; g < 4; g++) { g_omb_ok[g] += okg[g]; g_omb_n[g] += ng[g]; }
    printf("  [ombre] fn=%u L2=%02x %02x %02x.. %u/%u bits connus identiques (g0 %u/%u g1 %u/%u g2 %u/%u g3 %u/%u)
 ↪  | blocs=%lu parfaits=%lu\n", fn,
           g_omb_l2[0], g_omb_l2[1], g_omb_l2[2], ok, n, okg[0], ng[0], okg[1], ng[1], okg[2], ng[2], okg[3],
 ↪  ng[3], g_omb_blocs, g_omb_parfaits);
    fflush(stdout);
}

/* UPLINK xCCH BURSTS SENT BY THE ROM -> pont.py.
 * On every transmit frame, the ROM (PROM0 0x890e-0x891b) copies the current burst from the circular
 * buffer 0x4280..0x429f (BK=32, 4 bursts of 8 words) to data[0x3f8a..0x3f91]: 114 bits + hl/hu in
 * the 2 low bits of word 7, MSB first; then zeroes word 7 of the burst in 0x4280 and stores the
 * pointer advanced by 8 in data[0x3d9b] (mvkd ar2,@0x1b, DP=0x7b). Coding a block resets this
 * pointer to 0x4280 (0x8820). A change of data[0x3d9b] at the end of a frame = a burst has just
 * left; its rank is ((p - 0x4280)/8 + 3) & 3. The 4 copies in 0x3f8a are gsm0503_xcch_encode bit
```

```
 * for bit.
 * The 4 bursts of a block are published together in /dev/shm/calypso_xcch_ul_rom (ring, format:
 * pont/dsp/uplink.py), with the L2 posted in a_cu just before burst 0 (for the journal; the bridge
 * does not rely on it, it re-decodes the bursts). In ciphered mode the ROM XORs the uplink A5 key
 * stream (data[0x3f9b..0x3fa2], calypso_a5.c) onto 0x3f8a itself: this stream is published next
 * to the bits, the bridge undoes the XOR (the ROM ciphers at the DSP's time, not at the fn where
 * the bridge transmits) and re-ciphers at the air fn. MONTANT_ROM_UL=0 turns the publication off. */
#define ROM_UL_SHM      "/dev/shm/calypso_xcch_ul_rom"
#define ROM_UL_SLOTS    8
#define ROM_UL_SLOT_SZ  1024    /* seq(4) fn(4) l2(23) .(1) bits(4 x 116 ubits) flux(4 x 116 ubits) */
#define ROM_UL_FLUX     0x3f9b  /* uplink A5 key stream the ROM XORs onto 0x3f8a (PROM0 0x85d1) */
#define ROM_UL_PTR      0x3d9b
#define ROM_UL_BURST    0x3f8a
static uint8_t g_romul_l2_pose[23];     /* last L2 posted in a_cu (tx_rom_pose) */
static void tx_rom_pose(const uint16_t *api_ram, int blud_avant)
{
    if (!blud_avant) return;
    const uint16_t *w = &api_ram[(API_NDB + NDB_A_CU) / 2];
    for (int i = 0; i < 23; i++)
        g_romul_l2_pose[i] = (uint8_t)(i & 1 ? w[3 + i / 2] >> 8 : w[3 + i / 2] & 0xff);
}

/* THE ROM'S FINAL UPLINK BURST (TSP script, see src/tsp_tx.h). On every frame where the script
 * carries a new burst, it is published in /dev/shm/calypso_tx_rom (ring: header w(4) slots(4),
 * then 160-byte slots: seq(4) fn(4) type(1) tsc(1) offset(1) .(1) bits(148)).
 * fn = the GSM frame of this burst, the one the ARM gave the DSP for it: it is also the frame
 * number the ROM ciphered with (dsp_tester tx-sdcch-a5: XOR = osmo_a5(kc, fn of the command),
 * 24/24). The bridge can therefore transmit this burst as is at this fn, without coding or A5 on
 * the host side (PONT_UL_RAW=1 in pont/dsp/uplink.py). MONTANT_ROM_TX=0 turns the publication off. */
/* THE FRAME NUMBER OF THE BURST. The lockstep tick (m.a) is NOT the GSM fn: the firmware aligns
 * itself on the SCH. The true fn is in the W page the firmware hands to the DSP:
 * a_a5fn[0] = (T3 << 5) | T2, a_a5fn[1] = T1 (calypso/dsp.c dsp_load_tch_param, written with
 * every RX/TX task of next_time). It is read at the frame IT, on the page d_dsp_page designates;
 * pages without an RX/TX task (RACH) keep a stale a_a5fn: tick + offset is used then, the offset
 * being that of the last dated page. */
#define GSM_HYPERFRAME 2715648u
static struct { bool valide; uint32_t fn; bool dec_valide; int32_t dec; uint32_t tick, tick_dec; bool tache_tx; }
 ↪  g_tsp_fn;
static uint32_t gsmtime2fn(unsigned t1, unsigned t2, unsigned t3)
{
    return (1326u * t1 + 51u * ((t3 + 26u - t2) % 26u) + t3) % GSM_HYPERFRAME;
}
static void rach_dfn_noter(const C54xState *dsp, uint32_t tick);
static void scruter_ra_c(void);
static void rach_pend_echeance(uint32_t tick);
static struct { bool actif; uint32_t tick; uint8_t bits[148]; } g_rach_pend;
static uint32_t g_rom_emis_tick = 0xffffffffu;   /* last non-NB burst sent by the ROM (RACH) */
static void rach_pend_publier(uint32_t tick_emis, const char *quoi);
static void tx_tsp_dater(const C54xState *dsp, uint32_t tick)
{
    const uint16_t *api = &dsp->data[0x800];
    uint16_t v = api[(API_NDB + NDB_D_DSP_PAGE) / 2];
    unsigned pg = (v & B_GSM_PAGE) ? 1u : 0u;
    const uint16_t *w = &api[API_W_PAGE(pg) / 2];
    bool datee = (w[WP_D_TASK_D / 2] & 0x7fff) != 0 || (w[WP_D_TASK_U / 2] & 0x7fff) != 0;
    g_tsp_fn.tick = tick; g_tsp_fn.valide = false;
    /* a transmit task commanded this frame (RACH: the one the trx firmware's extcode hijacks
     * to send FB/SB/dummy/NB): tx_tsp_publier republishes even a burst identical to the
     * previous one when MONTANT_ROM_TX_CHAQUE=1 */
    g_tsp_fn.tache_tx = (v & B_GSM_TASK) && ((w[WP_D_TASK_RA / 2] & 0x7fff) != 0 || (w[WP_D_TASK_U / 2] & 0x7fff)
 ↪  != 0);
    if (v & B_GSM_TASK) rach_dfn_noter(dsp, tick);
    rach_pend_echeance(tick);
    if ((tick % 64u) == 0) scruter_ra_c();   /* RA_C read as it comes, not only at the next RACH */
    if (!(v & B_GSM_TASK) || !datee) return;
    unsigned t2 = w[12] & 0x1f, t3 = (w[12] >> 5) & 0x3f, t1 = w[13] & 0x7ff;
```

```
    if (t2 >= 26 || t3 >= 51) return;
    /* trx firmware (BTS mode): it never sets a_a5fn (no dsp_load_tch_param), the page says
     * t1=t2=t3=0 on every frame and every TRX burst would carry fn=0 in the calypso_tx_rom
     * ring. The bare tick (source 0), monotonic, is kept instead. */
    if (t1 == 0 && t2 == 0 && t3 == 0) return;
    g_tsp_fn.fn = gsmtime2fn(t1, t2, t3); g_tsp_fn.valide = true;
    g_tsp_fn.dec = (int32_t)((g_tsp_fn.fn + GSM_HYPERFRAME - (tick % GSM_HYPERFRAME)) % GSM_HYPERFRAME);
    g_tsp_fn.dec_valide = true;
    g_tsp_fn.tick_dec = tick;
    calypso_bsp_off_mobile_a5fn(tick, g_tsp_fn.fn);   /* the idle offset follows the dedicated one */
}
/* RACH DATED TO WITHIN ONE FRAME. In idle the mobile's fn is only known through the last SB or the last
 * a_a5fn; the TI L1 realigns its time base (tpu_offset) and comes back from a dedicated channel with one
 * or the other, to within one frame. A wrongly dated RACH = IMM ASS ignored, the mobile sends another
 * RACH. Hence: a new RACH without a dedicated channel since the previous one (less than 1000 frames)
 * -> next correction among 0, +1, -1; a dedicated channel obtained -> the correction stays.
 * CALYPSO_RACH_CORR=0 to turn it off. */
/* RA_C: the TI L1 trace (rvinterf, CALYPSO_RVINTERF_LOG) says at which fn the MOBILE sent each RACH.
 * Read at every RACH (the RA_C of the previous one has arrived since); the tick of the matching RACH
 * comes from the queue of the last RACHs. Learns the mobile's offset (calypso_bsp_off_mobile_rac). */
static uint32_t g_rach_ticks[16]; static unsigned g_rach_w;
static int rach_corr_idx_reset;
static void scruter_ra_c(void)
{
    static FILE *f; static long pos = -1;
    if (!f) {
        const char *e = calypso_getenv("CALYPSO_RVINTERF_LOG");
        f = fopen(e && *e ? e : "/tmp/c54x-pont/rvinterf.log", "r");
        if (!f) return;
        fseek(f, 0, SEEK_END); pos = ftell(f);
    }
    char l[256];
    clearerr(f); fseek(f, pos, SEEK_SET);
    while (fgets(l, sizeof l, f)) {
        pos = ftell(f);
        const char *p = strstr(l, "L1: RA_C");
        if (!p) continue;
        unsigned r = (unsigned)strtoul(p + 8, NULL, 10) % 42432u;
        /* the RACH whose tick is closest to r (mod 42432), among the last 16 */
        int best = -1; long bd = 1L << 30;
        for (unsigned k = 0; k < 16 && k < g_rach_w; k++) {
            uint32_t t = g_rach_ticks[(g_rach_w - 1 - k) % 16];
            uint32_t f0; if (!calypso_bsp_fn_mobile_du_tick(t, &f0)) f0 = t;
            long d = (long)((r + 42432u - f0 % 42432u) % 42432u); if (d > 21216) d -= 42432; if (d < 0) d = -d;
            if (d < bd) { bd = d; best = (int)((g_rach_w - 1 - k) % 16); }
        }
        if (best >= 0 && bd < 40) { calypso_bsp_off_mobile_rac(g_rach_ticks[best], r); rach_corr_idx_reset = 1; }
    }
}
static int rach_corr(uint32_t tick)
{
    static int on = -1, idx; static uint32_t t_prec; static unsigned dedie_prec; static bool vu;
    /* RA_C learning does not reset the rotation: the RA_C offset oscillates by +-1, so the rotation
     * keeps going as long as no dedicated channel is obtained, and keeps the winning value. */
    rach_corr_idx_reset = 0;
    static const int seq[5] = { 0, +1, -1, +2, -2 };   /* +-2 seen between two accesses */
    if (on < 0) { const char *e = calypso_getenv("CALYPSO_RACH_CORR"); on = !(e && *e == '0'); }
    if (!on) return 0;
    /* ROTATION OFF BY DEFAULT: with RA_C - tick constant over the RACHs of one access and the offset
     * learnt from the 2nd one, a rotation added on top dates the following ones wrong. The RACH queue
     * and the RA_C reading (learning) are kept, without correction. CALYPSO_RACH_ROTATION=1 enables it. */
    static int rot = -1;
    if (rot < 0) { const char *e = calypso_getenv("CALYPSO_RACH_ROTATION"); rot = (e && *e == '1'); }
    if (!vu || (uint32_t)(tick - t_prec) > 8) {      /* a new RACH (both paths land on the same tick) */
        scruter_ra_c();
        rach_corr_idx_reset = 0;
```

```
        g_rach_ticks[g_rach_w++ % 16] = tick;
        unsigned d = calypso_bsp_dedie_entrees();
        if (vu && d == dedie_prec && (uint32_t)(tick - t_prec) < 1000) {
            idx = (idx + 1) % 5;
           if (rot) printf("  [rach-corr] RACH sans canal depuis le tick %u : correction %+d\n", t_prec, seq[idx]);
        }
        dedie_prec = d; t_prec = tick; vu = true;
    }
    return rot ? seq[idx] : 0;
}
/* RACH DATED BY d_fn. The fn of the tick (RA_C offset, 768..770) drifts from one frame to the next and
 * the rotation adds to it. The W page of the RACH carries d_fn = fn_report | (fn%104) << 8, the fn of
 * the MOBILE: the fn closest to the estimate that has this remainder modulo 104 is taken
 * (+ CALYPSO_RACH_DFN_K). CALYPSO_RACH_DFN=0: estimate + rotation only. */
static struct { bool valide; uint32_t tick; unsigned fn104; } g_rach_dfn;
static void rach_dfn_noter(const C54xState *dsp, uint32_t tick)
{
    if (!dsp) return;
    const uint16_t *api = &dsp->data[0x800];
    uint16_t v = api[(API_NDB + NDB_D_DSP_PAGE) / 2];
    const uint16_t *w = &api[API_W_PAGE((v & B_GSM_PAGE) ? 1u : 0u) / 2];
    if (!(w[WP_D_TASK_RA / 2] & 0x7fff)) return;
    unsigned f104 = (unsigned)(w[WP_D_FN / 2] >> 8);
    if (f104 >= 104 || w[WP_D_FN / 2] == 0) return;
    g_rach_dfn.valide = true; g_rach_dfn.tick = tick; g_rach_dfn.fn104 = f104;
}
static bool rach_dfn_caler(uint32_t tick, uint32_t estime, uint32_t *fn)
{
    static int on = -1, k = 0;
    /* the TI firmware does not set d_fn on the RACH task (0 -> wrong alignment of +6): off by default */
    if (on < 0) { const char *e = calypso_getenv("CALYPSO_RACH_DFN"); on = (e && *e == '1');
                  e = calypso_getenv("CALYPSO_RACH_DFN_K"); k = (e && *e) ? atoi(e) : 0; }
    if (!on || !g_rach_dfn.valide || (uint32_t)(tick - g_rach_dfn.tick + 2) > 4) return false;
    int d = ((int)g_rach_dfn.fn104 + k - (int)(estime % 104) + 2 * 104) % 104;
    if (d >= 52) d -= 104;
    if (d < -6 || d > 6) return false;
    *fn = (estime + GSM_HYPERFRAME + (uint32_t)d) % GSM_HYPERFRAME;
    static unsigned n;
    if (++n <= 40 || n % 100 == 0)
        printf("  [rach-dfn] tick=%u d_fn%%104=%u estime=%u -> fn %u (%+d)\n", tick, g_rach_dfn.fn104, estime,
↪   *fn, d);
    return true;
}
static uint32_t tx_tsp_fn(uint32_t tick, int *source)
{
    if (g_tsp_fn.valide && g_tsp_fn.tick == tick) { *source = 1; return g_tsp_fn.fn; }
    /* a_a5fn offset STALE beyond 102 frames (CALYPSO_TSP_DEC_TTL; 0 = never): the TI L1 only sets a_a5fn
     * in a ciphered connection; in idle the mobile resynchronises (SB) and its fn drifts by one frame
     * from the offset frozen at the last connection, which would date the RACH fn+1 (IMM ASS ignored).
     * Beyond that, the BSP alignment (source 3), which follows the offset realignments. */
    static int ttl = -1;
    if (ttl < 0) { const char *e = calypso_getenv("CALYPSO_TSP_DEC_TTL"); ttl = (e && *e) ? atoi(e) : 102; }
    if (g_tsp_fn.dec_valide && (ttl == 0 || (uint32_t)(tick - g_tsp_fn.tick_dec) <= (uint32_t)ttl)) {
        *source = 2; return (uint32_t)((tick + (uint32_t)g_tsp_fn.dec) % GSM_HYPERFRAME);
    }
    /* firmware without a_a5fn in idle (TI L1): the fn the MOBILE believes, learnt at the last SB delivered
     * in the SB window (calypso_bsp.c g_off_mobile); the IMM ASS must match that one. */
    scruter_ra_c();   /* the RA_C of the previous RACH BEFORE estimating */
    { uint32_t f, c; if (calypso_bsp_fn_mobile_du_tick(tick, &f)) { *source = 4;
          if (rach_dfn_caler(tick, f, &c)) { rach_corr(tick); return c; }   /* rach_corr: RACH queue for RA_C */
          return (uint32_t)((f + GSM_HYPERFRAME + rach_corr(tick)) % GSM_HYPERFRAME); } }
    /* firmware without a_a5fn (TI L1, Magnetite): the BTS frame of the TS0 alignment */
    { uint32_t f; if (calypso_bsp_fn_bts_du_tick(tick, &f)) { *source = 3; return f; } }
    *source = 0; return tick;
}
#define ROM_TX_SHM   "/dev/shm/calypso_tx_rom"
```

```
#define ROM_TX_SLOTS 256
#define ROM_TX_SZ    160
/* The calypso_tx_rom ring is shared by tx_tsp_publier (bursts captured from the TSP script) and
 * tx_page_publier (TRX mode: bursts synthesised from the extcode page, one per TX slot). 160-byte
 * slot: seq(4) fn(4) type(1) tsc(1) offset(1) src(1) bits(148); src: bit 7 = A5, bits 4-6 = tn. */
static int g_ring_fd = -1;
static uint32_t g_ring_w;
static bool ring_publier(int type, const uint8_t *bits, int tsc, int offset, uint8_t src, uint32_t fn_gsm)
{
    if (g_ring_fd < 0) {
        g_ring_fd = open(ROM_TX_SHM, O_RDWR | O_CREAT, 0666);
        if (g_ring_fd < 0 || ftruncate(g_ring_fd, 8 + ROM_TX_SLOTS * ROM_TX_SZ) < 0) return false;
        uint32_t h[2] = { 0, ROM_TX_SLOTS };
        if (pwrite(g_ring_fd, h, sizeof h, 0) < 0) return false;
    }
    uint8_t slot[ROM_TX_SZ] = {0};
    g_ring_w++;
    memcpy(slot, &g_ring_w, 4); memcpy(slot + 4, &fn_gsm, 4);
    slot[8] = (uint8_t)type; slot[9] = (uint8_t)(tsc < 0 ? 0xff : tsc); slot[10] = (uint8_t)(offset < 0 ? 0xff :
 ↪  offset);
    slot[11] = src;
    memcpy(slot + 12, bits, 148);
    off_t off = 8 + (off_t)((g_ring_w - 1) % ROM_TX_SLOTS) * ROM_TX_SZ;
    return pwrite(g_ring_fd, slot, sizeof slot, off) >= 0 && pwrite(g_ring_fd, &g_ring_w, 4, 0) >= 0;
}
static int trx_page_mode(void)
{
    static int m = -1;
    if (m < 0) { const char *e = calypso_getenv("CALYPSO_BSP_TRX"); m = (e && *e == '1') ? 1 : 0; }
    return m;
}
static void tx_tsp_publier(const C54xState *dsp, uint32_t fn)
{
    static int actif = -1;
    static unsigned n_rach, n_nb, n_inconnu, n_sb, n_dummy, n_fb;
    static int chaque = -1;
    if (actif < 0) {
        const char *e = calypso_getenv("MONTANT_ROM_TX"); actif = !(e && *e == '0');
        e = calypso_getenv("MONTANT_ROM_TX_CHAQUE"); chaque = (e && *e == '1') ? 1 : 0;
        if (chaque) printf("  [rom-tx] MONTANT_ROM_TX_CHAQUE=1 : un burst publie a CHAQUE trame avec tache TX
         ↪  (mode TRX)\n");
        if (trx_page_mode()) { actif = 0; printf("  [rom-tx] mode TRX : les bursts sont publies depuis la page de
         ↪  l'extcode (tx_page_publier), pas du script TSP\n"); }
        else printf("  [rom-tx] burst montant final de la ROM (script TSP 0x3cbb) -> %s : %s\n", ROM_TX_SHM,
               actif ? "actif" : "coupe (MONTANT_ROM_TX=0)");
    }
    if (!actif) return;
    struct tsp_tx_burst b; bool nouveau = false;
    if (!tsp_tx_lire(dsp->data, &b, &nouveau)) return;
    if (!nouveau && !(chaque && g_tsp_fn.tache_tx && g_tsp_fn.tick == fn)) return;
    /* THE EXTCODE'S SCH (trx firmware) COMES OUT HALF WRONG ON THIS BENCH. Compared with
     * gsm0503_sch_encode() of the sch[0..1] words of the extended page (0x2000/0x2200 + 32), the
     * first 39 coded bits of the burst are EXACT and the last 39 (bits 106..144: top of the info,
     * CRC-10 parity, tail) are ~50 % wrong. The SCH encoder is in the DSP extcode (dsp_extcode.c,
     * 0x015c), not in the ROM: an instruction of that code (the parity loop) is mis-emulated by the
     * core. Prosthesis behind MONTANT_ROM_TX_SB_RECODE (1 by default in TRX mode, 0 to turn it
     * off): the SCH is re-encoded from the sch words of the page whose first half matches the
     * captured burst bit for bit, and the second half is replaced in what is published. The
     * counter says how many bits differed: the gap to watch once the core is fixed (0 = the
     * prosthesis is no longer needed). */
    if (b.type == TSP_TX_SB && chaque) {
        static int recode = -1; static unsigned n_sb, n_recodes, n_sans_page, n_bits_faux;
       if (recode < 0) { const char *e = calypso_getenv("MONTANT_ROM_TX_SB_RECODE"); recode = !(e && *e == '0'); }
        if (recode) {
            n_sb++;
            /* Both pages each carry an sch word: that of this SB and that of the previous SB (10 or
```

```
           * 11 frames earlier). The first coded half does not always tell them apart (it carries
           * neither T2 nor the low bit of T1/T3'): among the pages whose first half matches, the
           * one with the most recent fn is taken, otherwise one SB in three would carry the fn of
           * the previous SB. */
          int fait = 0; uint32_t fn_best = 0; ubit_t code_best[78];
          for (unsigned base = 0x2000; base <= 0x2200; base += 0x200) {
              uint32_t sch = dsp->data[base + 32] | ((uint32_t)dsp->data[base + 33] << 16);
              uint8_t info[4] = { sch & 0xff, (sch >> 8) & 0xff, (sch >> 16) & 0xff, (sch >> 24) & 0xff };
              ubit_t code[78];
              if (gsm0503_sch_encode(code, info) < 0) continue;
              if (memcmp(code, b.bits + 3, 39) != 0) continue;          /* not the page of this burst */
            /* inverse of sb_build() (prim_bts.c): t1&1 -> bit 23, t1&0x1fe -> bits 8..15, t1&0x600 -> bits 0..1,
               * t2 -> bits 18..22, t3'&1 -> bit 24, t3'&6 -> bits 16..17, bsic -> bits 2..7.
               * Checked on 0x005a1c1c = BSIC 7, T1 56, T2 22, T3 41. */
              uint32_t t1 = ((sch >> 23) & 1) | ((sch >> 7) & 0x1fe) | ((sch & 3) << 9);
              uint32_t t2 = (sch >> 18) & 0x1f, t3 = (((sch >> 24) & 1) | (((sch >> 16) & 3) << 1)) * 10 + 1;
              /* 05.02 3.3.2.2.1: FN = 51 ((T3 - T2) mod 26) + T3 + 51 x 26 x T1 (T3 - T2, not T2 - T3:
               * the latter gives a decoded fn that goes BACK by 51 every 51 frames) */
              uint32_t fnsb = 51 * 26 * t1 + 51 * ((t3 + 26 - t2) % 26) + t3;
            if (!fait || (int32_t)(fnsb - fn_best) > 0) { fn_best = fnsb; memcpy(code_best, code, sizeof code); }
              fait = 1;
          }
          if (fait) {
              int faux = 0;
              for (int k = 0; k < 39; k++) faux += (b.bits[106 + k] != code_best[39 + k]);
              n_bits_faux += (unsigned)faux;
              if (faux) { memcpy(b.bits + 106, code_best + 39, 39); n_recodes++; }
              /* TRX UPLINK ALIGNMENT. This SB (GSM fn fn_best) leaves at tick `fn`: the trx
               * firmware's clock is therefore known. The BSP of this instance receives the mobile's
               * bursts tagged with their GSM fn (the mobile's a_a5fn) and must play them at the
               * frame where the firmware expects them (calypso_bsp_trx_calage, CALYPSO_BSP_TRX=1).
               * Only applied when two consecutive SBs give the same gap (the two pages do not
               * always carry the sch of THIS burst). */
              static int32_t ecart_prec = INT32_MIN; static unsigned n_cal;
              int32_t ecart = (int32_t)(fn_best - fn);
              if (ecart == ecart_prec) {
                  if (calypso_bsp_trx_calage(fn_best, fn) && (++n_cal <= 5 || n_cal % 500 == 0))
                      printf("  [rom-tx] calage TRX : SB fn=%u emis au tick %u (fn - tick = %d) -> BSP montant
↪ cale (%u)\n", fn_best, fn, ecart, n_cal);
              }
              ecart_prec = ecart;
          } else n_sans_page++;
          if (n_sb <= 3 || n_sb % 1000 == 0)
              printf("  [rom-tx] SB recode (prothese SCH) : %u SB, %u recodes, %u sans page sch, %.1f bits
↪ faux/SB en moyenne\n",
                     n_sb, n_recodes, n_sans_page, n_sb ? (double)n_bits_faux / n_sb : 0.0);
      }
  }
  if (b.type != TSP_TX_NB) g_rom_emis_tick = fn;
  if (g_rach_pend.actif && b.type != TSP_TX_NB && (uint32_t)(fn - g_rach_pend.tick) <= 2)
      rach_pend_publier(fn, "emission ROM");
  int src = 0;
  uint32_t fn_gsm = tx_tsp_fn(fn, &src);
  uint8_t octet_src = (uint8_t)src;
  /* bit 7: NB burst ciphered by the ROM -- the A5 coprocessor computed an
   * A5/1-2 stream for this fn (to within one frame: SACCH realignment,
   * calypso_a5.c). Read by the bridge for the "A5 ul rom=" counter. */
  {
      uint32_t fa;
      if (b.type == TSP_TX_NB && calypso_a5_dernier(&fa)) {
          int32_t d = (int32_t)((fn_gsm + GSM_HYPERFRAME - fa) % GSM_HYPERFRAME);
          if (d <= 1 || d >= (int32_t)GSM_HYPERFRAME - 1) octet_src |= 0x80;
      }
  }
  /* bits 4-6: the timeslot the firmware transmits on (TCH TS1..7; 0 = TS0): the air
   * carries the burst at this TRX timeslot */
```

```
    octet_src |= (uint8_t)((montant_tn_dedie() & 7) << 4);
    if (!ring_publier(b.type, b.bits, b.tsc, b.offset, octet_src, fn_gsm)) { actif = 0; return; }
    uint32_t w_compte = g_ring_w;
    if (b.type == TSP_TX_RACH) n_rach++; else if (b.type == TSP_TX_NB) n_nb++; else if (b.type == TSP_TX_SB)
     ↪  n_sb++; else if (b.type == TSP_TX_DUMMY) n_dummy++; else if (b.type == TSP_TX_FB) n_fb++; else
     ↪  n_inconnu++;
    static int nlog;
    if (nlog++ < 6 || (b.type == TSP_TX_RACH && n_rach <= 3))
        printf("  [rom-tx] #%u tick=%u fn=%u (%s) %s%s offset=%d @%04x : %s...\n", w_compte, fn, fn_gsm,
               src == 1 ? "a_a5fn" : src == 2 ? "tick+dec" : src == 3 ? "calage BSP" : "TICK NU",
              b.type == TSP_TX_RACH ? "RACH" : b.type == TSP_TX_NB ? "NB" : b.type == TSP_TX_SB ? "SB" : b.type ==
 ↪  TSP_TX_DUMMY ? "DUMMY" : b.type == TSP_TX_FB ? "FB" : "?", b.type == TSP_TX_NB ? (char[]){' ', 'T', 'S', 'C',
 ↪  (char)('0' + b.tsc), 0} : "",
               b.offset, b.adresse, (char[]){ '0' + b.bits[0], '0' + b.bits[1], '0' + b.bits[2], '0' + b.bits[3],
 ↪  '0' + b.bits[4], '0' + b.bits[5], '0' + b.bits[6], '0' + b.bits[7], 0 });
    if ((n_rach + n_nb + n_sb + n_dummy + n_fb + n_inconnu) % 2000 == 0)
        printf("  [rom-tx] %u bursts publies : RACH %u, NB %u, SB %u, DUMMY %u, FB %u, inconnus %u\n", n_rach +
 ↪  n_nb + n_sb + n_dummy + n_fb + n_inconnu, n_rach, n_nb, n_sb, n_dummy, n_fb, n_inconnu);
}

/* RACH TO THE AIR (CALYPSO_RACH_VERS_AIR=1, Magnetite). The capture at the ROM's TSP script
 * (0x3cbb/0x3ccd, the osmocom firmware path) does not see the TI firmware's access burst: montant.c
 * detects the RACHs (d_task_ra, ra/bsic) but they only go to /dev/shm/calypso_rach, which the air
 * does not read. The access burst is coded here (05.03, gsm0503_rach_ext_encode) and published in
 * calypso_tx_rom, dated with the BTS frame of the alignment. */
/* a_pm NEVER ZERO (CALYPSO_BSP_BRUIT_FOND=1, Magnetite). The TI L1 reads pm = a_pm >> 5
 * (l1ddsp_meas_read) and counts every zero pm as a failure (l1_check_pm_error: 100 -> recovery, DSP
 * restarted). The bench power model (calypso_bsp_rssi_apm) only runs on DSP writes; an a_pm left
 * at 0 by the ROM would go out as is. Before handing the frame back to the ARM, every zero a_pm
 * word of both R pages receives the model (floor 64). */
/* probe [apmT] (see calypso_apm_trace_on): R pages at the TICK (what the DSP left after the
 * previous frame and what the L1 zeroed), then at the DONE before/after the model. */
static uint16_t g_apm_tick_r[2][4];
static void apm_trace_tick(const uint16_t *api_ram, uint32_t tick, uint32_t b)
{
    for (int pg = 0; pg < 2; pg++) {
        const uint16_t *r = &api_ram[API_R_PAGE(pg) / 2];
        g_apm_tick_r[pg][0] = r[4]; g_apm_tick_r[pg][1] = r[12]; g_apm_tick_r[pg][2] = r[13]; g_apm_tick_r[pg][3]
 ↪  = r[14];
    }
    (void)tick; (void)b;
}
static void apm_trace_done(const uint16_t *api_ram, uint16_t avant[2][4]);
static void apm_jamais_nul_corps(uint16_t *api_ram);
static void apm_jamais_nul(uint16_t *api_ram)
{
    uint16_t done_avant[2][4];
    for (int pg = 0; pg < 2; pg++) {
        const uint16_t *r = &api_ram[API_R_PAGE(pg) / 2];
        done_avant[pg][0] = r[4]; done_avant[pg][1] = r[12]; done_avant[pg][2] = r[13]; done_avant[pg][3] = r[14];
    }
    apm_jamais_nul_corps(api_ram);
    apm_trace_done(api_ram, done_avant);
}
static void apm_jamais_nul_corps(uint16_t *api_ram)
{
    static int on = -1;
    if (on < 0) { const char *e = calypso_getenv("CALYPSO_BSP_BRUIT_FOND"); on = (e && *e == '1'); }
    if (!on) return;
    for (int pg = 0; pg < 2; pg++) {
        uint16_t *a = &api_ram[(API_R_PAGE(pg) + RP_A_PM) / 2];
        uint16_t avant[3] = { a[0], a[1], a[2] };
        int depile[3] = { -2, -2, -2 };
        /* one measurement per window, in the order of the delivered windows (queue, see calypso_bsp.c) */
        /* words 12..14 are written by the DSP and served in order by c54x_mem.c
         * (calypso_bsp_rssi_apm_suivante); a zero here is a page zeroed by the L1: floor without
```

```
        * consuming the queue (never 0: l1_check_pm_error). */
       for (int k = 0; k < 3; k++)
           if ((a[k] >> 5) == 0) a[k] = calypso_bsp_rssi_apm();
       /* PROBE [apm-fort]: PMs replaced with the MAV of a REAL burst (idle, ARFCN 1), or a frame
        * where a window received one. 200 lines. */
       /* summary every 255 frames (paging period of the bench): PMs replaced with a burst, with noise,
        * or with no window queued -- idle is what matters, the detailed probe runs out during the scan */
       {
           static unsigned long b_burst, b_bruit, b_vide, b_t0; static uint32_t b_tick;
           for (int k = 0; k < 3; k++) {
               if (depile[k] >= 5000) b_burst++; else if (depile[k] >= 0) b_bruit++; else if (depile[k] == -1)
                ↪  b_vide++;
           }
           if (g_c54x_exe_fn - b_tick >= 255) {
               b_tick = g_c54x_exe_fn;
               printf("  [apm-bilan] fn=%u sur 255 trames : PM <- burst %lu, <- bruit %lu, file vide %lu\n",
                      g_c54x_exe_fn, b_burst, b_bruit, b_vide);
               b_burst = b_bruit = b_vide = 0; (void)b_t0;
           }
       }
       static unsigned n_fort;
       if ((calypso_bsp_pm_mav_fort() || depile[0] >= 5000 || depile[1] >= 5000 || depile[2] >= 5000) &&
           (n_fort++ < 20 || n_fort % 50 == 0)) {
           char t[256]; calypso_bsp_pm_mav_texte(t, sizeof t);
           uint16_t *r = &api_ram[API_R_PAGE(pg) / 2];
           printf("  [apm-fort] page=%d d_task_md=%04x d_task_d=%04x a_pm %04x->%04x %04x->%04x %04x->%04x
↪  depiles %d %d %d | mav %s\n",
                  pg, r[4], r[0], avant[0], a[0], avant[1], a[1], avant[2], a[2], depile[0], depile[1],
↪  depile[2], t);
       }
   }
   /* FULL-LIST MEASUREMENT (RXL_R 548): the TI L1 (DSP 36) programs d_task_md = 0x200 + n
    * (n <= 8 measurements, l1_sync.c) and reads the n PMs in words 12..12+n-1 of the R page
    * (l1ddsp_meas_read, nbmeas = 8): beyond the 3 a_pm words, these are the a_sch words. The power
    * model therefore has to cover words 12..12+n-1, not just 12..14, otherwise 5 zero PMs out of 8
    * on every scan frame trigger a recovery. The R page echoes d_task_md (word 4): that is used. */
   /* DEDICATED MODE: the SDCCH/SACCH tasks (ADL 22, DDL 24 on the TI L1 side) read the power in
    * a_serv_demod[D_PM], word 9 of the R page for DSP 36 (l1_defty.h: a_serv_demod 8..11). Left at 0
    * by the emulated DSP after the IMM ASS, it gives "> PM 1778 22", "> PM 1808 24" on every frame,
    * ABORT, back to idle. Same floor as a_pm, as soon as a downlink task (d_task_d, word 0) is reported. */
   for (int pg = 0; pg < 2; pg++) {
       uint16_t *r = &api_ram[API_R_PAGE(pg) / 2];
       /* + the sync tasks (d_task_md: SB2, SBCONF...): the TI L1 also reads their PM in
        * a_serv_demod[D_PM] (l1_cmplx.c, l1s_read_mon_result); word 9 left at 0 -> 4 "> PM" (task 8) at
        * every SB re-read of the idle cell, cumulative counter -> RECOVERY after a few minutes. */
       if (((r[0] & 0x7fff) || (r[4] & 0xff)) && (r[9] >> 5) == 0) r[9] = calypso_bsp_rssi_apm();
   }
   for (int pg = 0; pg < 2; pg++) {
       uint16_t *r = &api_ram[API_R_PAGE(pg) / 2];
       uint16_t md = r[4];
       if (!(md & 0x200)) continue;
       int n = md & 0xff; if (n > 8) n = 8;
       uint16_t avant[8];
       for (int k = 0; k < n; k++) avant[k] = r[12 + k];
       for (int k = 3; k < n; k++)      /* 15..19: not written by the DSP on this path, rest of the queue */
           if ((r[12 + k] >> 5) == 0) r[12 + k] = calypso_bsp_rssi_apm_suivante(NULL);
       /* PROBE [apm]: in idle (Magnetite) the BA measurement of ARFCN 1 can fall to the floor even
        * with one MAV per window. PMs returned by the DSP (before) and after the model, MAV of the windows. */
       static unsigned long n_apm;
       if (++n_apm <= 60 || n_apm % 200 == 0) {
           char t[256]; calypso_bsp_pm_mav_texte(t, sizeof t);
           printf("  [apm] #%lu page=%d md=%04x n=%d :", n_apm, pg, md, n);
           for (int k = 0; k < n; k++) printf(" %04x->%04x", avant[k], r[12 + k]);
           printf(" | mav %s\n", t);
       }
   }
```

```
}

static void apm_trace_done(const uint16_t *api_ram, uint16_t avant[2][4])
{
    if (!calypso_apm_trace_on()) return;
    const uint16_t *w0 = &api_ram[0], *w1 = &api_ram[0x14];
    printf("  [apmT] fn=%u dsp_page=%04x W md %04x/%04x |", g_c54x_exe_fn,
           api_ram[(API_NDB + NDB_D_DSP_PAGE) / 2], w0[4], w1[4]);
    for (int pg = 0; pg < 2; pg++) {
        const uint16_t *r = &api_ram[API_R_PAGE(pg) / 2];
        printf(" R%d tick md=%04x pm=%04x,%04x,%04x done md=%04x pm=%04x,%04x,%04x -> %04x,%04x,%04x |", pg,
               g_apm_tick_r[pg][0], g_apm_tick_r[pg][1], g_apm_tick_r[pg][2], g_apm_tick_r[pg][3],
               avant[pg][0], avant[pg][1], avant[pg][2], avant[pg][3], r[12], r[13], r[14]);
    }
    printf("\n");
}

/* THE RACH DATED BY ITS TRANSMISSION. RA_C = tick where the ROM TRANSMITS the burst (TSP script) + the
 * delivery offset. The tick where the task is POSTED (montant.c) precedes the transmission by 0 or 1
 * frame depending on the access: that is the whole jitter. The burst therefore waits for the ROM's
 * transmission (tx_tsp_publier); without a transmission within 3 ticks, it leaves at the posting
 * tick + 1. CALYPSO_RACH_EMISSION=0: date by the posting tick. */
static int rach_emission_on(void)
{
    static int on = -1;
    if (on < 0) { const char *e = calypso_getenv("CALYPSO_RACH_EMISSION"); on = !(e && *e == '0'); }
    return on;
}
static void rach_pend_publier(uint32_t tick_emis, const char *quoi)
{
    if (!g_rach_pend.actif) return;
    g_rach_pend.actif = false;
    uint32_t f; int src = 3;
    if (!calypso_bsp_fn_bts_du_tick(tick_emis, &f)) { f = tx_tsp_fn(tick_emis, &src); }
    rach_corr(g_rach_pend.tick);   /* RACH queue (RA_C reading) */
    if (!ring_publier(TSP_TX_RACH, g_rach_pend.bits, -1, 8, (uint8_t)src, f)) return;
    static unsigned n;
    if (++n <= 40 || n % 100 == 0)
        printf("  [rach-air] RACH pose tick=%u, emis tick=%u (%s) -> fn BTS %u publie vers l'air [%u]\n",
               g_rach_pend.tick, tick_emis, quoi, f, n);
}
static void rach_pend_echeance(uint32_t tick)
{
    if (g_rach_pend.actif && (uint32_t)(tick - g_rach_pend.tick) >= 3)
        rach_pend_publier(g_rach_pend.tick + 1, "sans emission vue, pose+1");
}
void tx_rach_vers_air(uint8_t ra, uint8_t bsic, uint32_t tick)
{
    static int actif = -1;
    if (actif < 0) { const char *e = calypso_getenv("CALYPSO_RACH_VERS_AIR"); actif = (e && *e == '1'); }
    if (!actif) return;
    /* the WHOLE ACCESS BURST (05.02 5.2.7), like the ROM path (tsp_tx.c, offset 8): 8 extended tail
     * bits, 41 of sync, the 36 coded ones, 3 of tail. gsm0503_rach_ext_encode only returns the 36
     * coded bits: published alone at the head, the TRX would have no sync to correlate. */
    static const char queue[] = "00111010", sync[] = "01001011011111111100110011010101000111000";
    uint8_t code[36], bits[148] = {0};
    if (gsm0503_rach_ext_encode(code, ra, bsic, false) < 0) return;
    for (int i = 0; i < 8; i++) bits[i] = (uint8_t)(queue[i] - '0');
    for (int i = 0; i < 41; i++) bits[8 + i] = (uint8_t)(sync[i] - '0');
    memcpy(bits + 49, code, 36);
    if (rach_emission_on()) {
        if (g_rach_pend.actif) rach_pend_publier(g_rach_pend.tick + 1, "remplace");
        g_rach_pend.actif = true; g_rach_pend.tick = tick; memcpy(g_rach_pend.bits, bits, 148);
        if (g_rom_emis_tick == tick) rach_pend_publier(tick, "emission ROM, meme tick");   /* already transmitted
        ↪  */
        return;
```

```
    }
    int src = 0;
    rach_dfn_noter(dsp_global_tx, tick);
    uint32_t fn_gsm = tx_tsp_fn(tick, &src);
    if (!ring_publier(TSP_TX_RACH, bits, -1, 8, (uint8_t)src, fn_gsm)) return;
    static unsigned n;
    if (++n <= 20 || n % 100 == 0)
        printf("  [rach-air] RACH ra=0x%02x bsic=%u tick=%u -> fn BTS %u (%s) publie vers l'air [%u]\n",
               ra, bsic, tick, fn_gsm, src == 1 ? "a_a5fn" : src == 2 ? "tick+dec" : src == 3 ? "calage BSP" : src
↪   == 4 ? "fn du mobile (SB)" : "TICK NU", n);
}

/* TRX MODE: PUBLICATION FROM THE EXTCODE PAGE. The trx firmware fills tx[p] (cmd, data pointer) for
 * each slot p of its TX window (patched prim_bts.c: slot p = TS (tx_start + p)), and the DSP extcode
 * modulates them; the emulated ROM only plays one burst per frame (the TSP script) and the extcode's
 * SCH coder is wrong on this core. The ROM is not needed: the bits follow from the page (FB = 148
 * zeros, dummy = 05.02 5.2.6, SB = gsm0503_sch_encode(sch[0..1]), NB = 8 data words + TSC of the NDB).
 * Called at phase A of tick T+1 with the page the DSP processed at tick T (index d_dsp_page): the
 * published bursts carry fn = T. The SB aligns the uplink BSP (calypso_bsp_trx_calage).
 * CALYPSO_TRX_TX_SLOTS (1): number of slots published; CALYPSO_TRX_TX_START (0): TS of slot 0. */
static void tx_page_publier(const C54xState *dsp, uint32_t tick, int w_page)
{
    static int nslots = -1, tx_start = 0; static unsigned n_pub, n_cal; static int32_t ecart_prec = INT32_MIN;
    static const char DUMMY_148[] =
↪       "00011111011011101100000101001001110000010010001000000011111000111000101110001011100010101110100101000110011001110011111
↪       ";
    static const char *const TSC8[8] = {
        "0010010111000010001001010111", "0010110111011111000101101111", "0100001110111010010000011110",
        "0100011110110100010001111110", "0001101011100010000011010111", "0100111010110000001001110101",
        "1010011111011000101001111111", "1110111100010010111011111100" };
    if (nslots < 0) {
        const char *e = calypso_getenv("CALYPSO_TRX_TX_SLOTS"); nslots = (e && *e) ? atoi(e) : 1;
        if (nslots < 1) nslots = 1;
        if (nslots > 8) nslots = 8;
        e = calypso_getenv("CALYPSO_TRX_TX_START"); tx_start = (e && *e) ? atoi(e) & 7 : 0;
        printf("  [rom-tx] publication depuis la page de l'extcode : %d slot(s) TX a partir du TS%d\n", nslots,
↪   tx_start);
    }
    const uint16_t *db = &dsp->data[w_page ? 0x2200 : 0x2000];
    uint32_t fn_tag = tick - 1;                        /* the DSP frame where these bursts left */
    int tsc = dsp->data[0x2401] & 7;
    for (int p = 0; p < nslots; p++) {
        unsigned cmd = db[2 * p], ptr = db[2 * p + 1];
        uint8_t bits[148]; int type, offset = 0, tscp = -1;
        memset(bits, 0, sizeof bits);
        switch (cmd) {
        case 1: type = TSP_TX_FB; break;                                          /* 148 zeros */
        case 2: {                                                                  /* SB */
            uint32_t sch = db[32] | ((uint32_t)db[33] << 16);
            uint8_t info[4] = { sch & 0xff, (sch >> 8) & 0xff, (sch >> 16) & 0xff, (sch >> 24) & 0xff };
            ubit_t code[78];
            if (gsm0503_sch_encode(code, info) < 0) continue;
            static const char SYNC_SB[] = "1011100101100010000001000000111100101101010001010111011000011011";
            for (int k = 0; k < 39; k++) { bits[3 + k] = code[k]; bits[106 + k] = code[39 + k]; }
            for (int k = 0; k < 64; k++) bits[42 + k] = (uint8_t)(SYNC_SB[k] - '0');
            type = TSP_TX_SB;
            /* uplink alignment: this SB (GSM fn) left at tick fn_tag; applied when two SBs in a row agree */
            uint32_t t1 = ((sch >> 23) & 1) | ((sch >> 7) & 0x1fe) | ((sch & 3) << 9);
            uint32_t t2 = (sch >> 18) & 0x1f, t3 = (((sch >> 24) & 1) | (((sch >> 16) & 3) << 1)) * 10 + 1;
            uint32_t fnsb = 51 * 26 * t1 + 51 * ((t3 + 26 - t2) % 26) + t3;
            int32_t ecart = (int32_t)(fnsb - fn_tag);
            /* the alignment keeps the relation measured at the TSP script (fn of the SB = transmit tick - 1):
             * the phase A tick (T + 1) is passed, not fn_tag, so that CALYPSO_BSP_TRX_DELTA keeps its meaning */
            if (ecart == ecart_prec && calypso_bsp_trx_calage(fnsb, tick) && (++n_cal <= 3 || n_cal % 500 == 0))
                printf("  [rom-tx] calage TRX (page) : SB fn=%u emis au tick %u (fn - tick = %d) -> BSP montant
↪   cale (%u)\n", fnsb, fn_tag, ecart, n_cal);
```

```
            ecart_prec = ecart;
            break; }
        case 3: for (int k = 0; k < 148; k++) bits[k] = (uint8_t)(DUMMY_148[k] - '0'); type = TSP_TX_DUMMY; break;
        case 5: {                                                       /* NB: 116 bits + TSC */
            if (ptr + 8 > 0x200) continue;
            const uint16_t *w = &db[ptr];
            uint8_t pb[15];
            for (int k = 0; k < 7; k++) { pb[2 * k] = w[k] >> 8; pb[2 * k + 1] = w[k] & 0xff; }
            pb[14] = w[7] >> 8;
           for (int k = 0; k < 58; k++) bits[3 + k] = (pb[k >> 3] >> (7 - (k & 7))) & 1;          /* 57 data + hl */
            for (int k = 0; k < 58; k++) bits[87 + k] = (pb[(58 + k) >> 3] >> (7 - ((58 + k) & 7))) & 1;   /* hu +
             ↪  57 data */
            for (int k = 0; k < 26; k++) bits[61 + k] = (uint8_t)(TSC8[tsc][k] - '0');
            type = TSP_TX_NB; tscp = tsc;
            break; }
        default: continue;                                              /* 0 (nothing) or AB */
        }
        uint8_t src = (uint8_t)(((tx_start + p) & 7) << 4);
        ring_publier(type, bits, tscp, offset, src, fn_tag);
        n_pub++;
        if (n_pub <= 4 || n_pub % 20000 == 0)
            printf("  [rom-tx] page : #%u tick=%u slot %d (TS%d) cmd=%u -> %s\n", n_pub, fn_tag, p, (tx_start + p)
↪  & 7, cmd,
                   type == TSP_TX_FB ? "FB" : type == TSP_TX_SB ? "SB" : type == TSP_TX_DUMMY ? "DUMMY" : "NB");
    }
}

static void tx_rom_publier(const C54xState *dsp, uint32_t fn)
{
    static int actif = -1, fd = -1;
    static uint16_t prev;
    static unsigned masque;
    static uint8_t l2[23], bits[4 * 116], flux[4 * 116];
    static uint32_t w_compte, fn0;
    if (actif < 0) {
        const char *e = calypso_getenv("MONTANT_ROM_UL"); actif = !(e && *e == '0');
        prev = dsp->data[ROM_UL_PTR];
        printf("  [rom-ul] bursts xCCH montants de la ROM (0x3f8a) -> %s : %s\n", ROM_UL_SHM, actif ? "actif" :
↪  "coupe (MONTANT_ROM_UL=0)");
    }
    if (!actif) return;
    uint16_t p = dsp->data[ROM_UL_PTR];
    if (p == prev) return;
    prev = p;
    if (p < 0x4280 || p >= 0x42a0 || (p - 0x4280) % 8) { masque = 0; return; }
    unsigned b = ((p - 0x4280) / 8 + 3) & 3;
    if (b == 0) { masque = 0; memcpy(l2, g_romul_l2_pose, 23); fn0 = fn; }
    uint8_t *o = &bits[b * 116], *x = &flux[b * 116];
    for (int i = 0; i < 116; i++) {
        /* 57 data, hl, hu, 57 data. The 114 data bits occupy words 0..7 MSB first (bits 0..113);
         * hl/hu are the 2 LOW bits of word 7 (0x3f91 = 0x?003 on SDCCH, where hl = hu = 1), i.e.
         * k = 126 and 127 -- not 114/115 (bits 13/12, always 0). */
        int k = i < 57 ? i : (i < 59 ? 126 + (i - 57) : i - 2);
        o[i] = (dsp->data[ROM_UL_BURST + k / 16] >> (15 - k % 16)) & 1;
        x[i] = (dsp->data[ROM_UL_FLUX + k / 16] >> (15 - k % 16)) & 1;
    }
    masque |= 1u << b;
    if (b != 3) return;
    if (masque != 0xf) { masque = 0; return; }
    masque = 0;
    if (fd < 0) {
        fd = open(ROM_UL_SHM, O_RDWR | O_CREAT, 0666);
        if (fd < 0 || ftruncate(fd, 8 + ROM_UL_SLOTS * ROM_UL_SLOT_SZ) < 0) { actif = 0; return; }
        uint32_t h[2] = { 0, ROM_UL_SLOTS };
        if (pwrite(fd, h, sizeof h, 0) < 0) { actif = 0; return; }
    }
```

```
    uint8_t slot[ROM_UL_SLOT_SZ] = {0};
    w_compte++;
    memcpy(slot, &w_compte, 4); memcpy(slot + 4, &fn0, 4);
    memcpy(slot + 8, l2, 23); memcpy(slot + 32, bits, sizeof bits); memcpy(slot + 32 + sizeof bits, flux, sizeof
 ↪  flux);
    off_t off = 8 + (off_t)((w_compte - 1) % ROM_UL_SLOTS) * ROM_UL_SLOT_SZ;
    if (pwrite(fd, slot, sizeof slot, off) < 0 || pwrite(fd, &w_compte, 4, 0) < 0) return;
    static int nlog;
    if (nlog++ < 4) printf("  [rom-ul] bloc %u publie (fn burst0=%u, L2=%02x %02x %02x..)\n", w_compte, fn0,
     ↪  l2[0], l2[1], l2[2]);
}

static int c54x_run_profile(C54xState *dsp, int budget)
{
    int fait = 0;
    pcc_armer();
    while (fait < budget && !dsp->idle && dsp->running) {
        int n = budget - fait < PROFIL_SEAU ? budget - fait : PROFIL_SEAU;
        static long minfn = -1;
        if (minfn < 0) { const char *e = calypso_getenv("PONT_TRACE_MINFN"); minfn = (e && *e) ? atol(e) : 0; }
        if (g_trace_reste > 0 && !g_trace_ouverte && g_trace_pc_hi && (long)g_c54x_exe_fn >= minfn &&
            (dsp->pc & 0xffff) >= g_trace_pc_lo && (dsp->pc & 0xffff) <= g_trace_pc_hi && g_trace_f) {
            g_trace_ouverte = true;
            trace_cellules(dsp, true);
            fprintf(g_trace_f, "# PC=%04x dans [%04x,%04x] a fn=%u ; IMR=%04x IFR=%04x\n",
                    dsp->pc & 0xffff, g_trace_pc_lo, g_trace_pc_hi, g_c54x_exe_fn, dsp->imr, dsp->ifr);
            printf("pont : trace ouverte sur PC=%04x fn=%u\n", dsp->pc & 0xffff, g_c54x_exe_fn);
        }
        if (g_trace_reste > 0 && g_trace_ouverte) n = 1;
        if (g_pcc_k > 0) {
            n = 1;
            uint16_t pc = dsp->pc & 0xffff;
            for (int i = 0; i < g_pcc_k; i++) if (g_pcc[i] == pc) g_pcc_n[i]++;
        }
        if (n == 1) trace_pas(dsp);
        /* Budget accounting uses the count c54x_run returns, not the insn_count
         * delta: RPT iterations do not bump insn_count, so a step or a whole
         * bucket landing inside a several-hundred-iteration "rpt *(lk); nop"
         * reads as a stalled DSP and aborts the frame mid-ISR. */
        int ex = c54x_run(dsp, n);
        fait += ex;
        g_profil[(dsp->pc & 0xffff) / PROFIL_SEAU]++;
        if ((dsp->pc & 0xffff) > g_profil_hw) g_profil_hw = dsp->pc & 0xffff;
        if (ex <= 0) break;
    }
    return fait;
}

static void profil_publier(void)
{
    unsigned long tot = 0; unsigned n = 0;
    for (unsigned i = 0; i < 0x10000 / PROFIL_SEAU; i++) tot += g_profil[i];
    if (!tot) return;
    printf("  profil PC (%lu tranches, high-water 0x%04x) :", tot, g_profil_hw);
    /* the 12 hottest buckets, in decreasing order */
    for (int k = 0; k < 12; k++) {
        unsigned best = 0; unsigned long bv = 0;
        for (unsigned i = 0; i < 0x10000 / PROFIL_SEAU; i++)
            if (g_profil[i] > bv) { bv = g_profil[i]; best = i; }
        if (!bv) break;
        printf(" %04x:%lu%%", best * PROFIL_SEAU, bv * 100 / tot);
        g_profil[best] = 0; n++;
    }
    printf("\n");
    memset(g_profil, 0, sizeof(g_profil)); g_profil_hw = 0;
}
```

```
/* One TDMA frame, in the order calypso_tdma_tick() uses: DMA tick, then boot
 * (run to the first IDLE) while init is pending, then the TPU-frame interrupt
 * if IMR arms it followed by one run budget. Returns the PONT_DONE flags and
 * the executed instruction count in *insns. */
/* [2026-09-20] MID-FRAME INJECTION (PONT_RX_MODE=milieu, the default). The ROM
 * arms DMA2 in its frame ISR and the receiver only keeps samples while a
 * window is open (calypso_rif: no window, no sample). Delivering the burst
 * AFTER the frame's run (the old PONT_RX_APRES=1) found the channel closed on
 * most frames: measured 13 transfers on one frame in six, the FB block counter
 * starved, TOA = 1251 for an FCCH four frames away. The replay bench delivers
 * after a short slice of the frame, once the ISR has armed; the bridge now does
 * the same: budget/8, inject (synthetic cell and UDP bursts), then the rest. */
static bool g_tick_irq_trame = true;   /* TICK.b bit 16: the ARM armed the DSP frame interrupt */
static struct { bool actif; const char *iq_mode; int amp; uint32_t fn; unsigned long *injectes; bool udp; char
 ↪  dernier_type; int dernier_n_iq; } g_inj;
static void injecter_burst(C54xState *dsp, const char *iq_mode, int amp, uint32_t fn, unsigned long *injectes);

uint16_t prog_fetch(C54xState *s, uint16_t pc);
static unsigned g_flags_entree; static uint16_t g_data_avant_b[C54X_DATA_SIZE];
static uint16_t g_snap_2be2[148]; static int g_snap_ok;
static uint16_t g_snap_2a00[456]; static int g_snap_2a00_ok;
#define s_or_dsp_st0(d) ((d)->st0)
static int16_t g_dernier_iq[2 * 256]; static int g_dernier_n_iq;
static uint16_t g_daram_apres_dma[384]; static uint16_t g_daram_aad;
/* phase : 0 = whole frame ; 1 = frame ISR only, up to the arming of the RX
 * window (then DONE|PHASE_A and wait for PONT_GO) ; 2 = burst delivery and
 * the rest of the frame. See CALYPSO_PONT_TICK_DEUX_PHASES. */
/* TCH RECORDER (ARM side). The ARM writes into the API RAM, by difference
 * with the snapshot taken when the DSP handed over: 'A', tick BE32, window
 * (0 = before the TICK, 1 = between phase A and GO), count BE16, then
 * (word BE16, value BE16). And at every TICK: 'T', tick BE32, flags (bit0
 * frame irq, bit1 two phases), budget BE32. Same file as the BSP's I/Q
 * deliveries (calypso_bsp_enreg_fichier), same guard: TCH active,
 * CALYPSO_REJEU_ENREG=0 turns it off. Replayed by tools/rejeu_banc.c. */
static uint16_t g_enreg_api[CALYPSO_API_WORDS];
static bool g_enreg_api_ok;
static void enreg_api_prendre(const uint16_t *api_ram)
{
    memcpy(g_enreg_api, api_ram, sizeof g_enreg_api);
    g_enreg_api_ok = true;
}
static void enreg_api_diff(const uint16_t *api_ram, uint32_t tick, int fenetre)
{
    FILE *f = calypso_bsp_enreg_fichier();
    if (!f || !g_enreg_api_ok) return;
    static uint8_t buf[8 + 4 * CALYPSO_API_WORDS];
    unsigned n = 0;
    for (unsigned i = 0; i < CALYPSO_API_WORDS; i++) {
        if (api_ram[i] == g_enreg_api[i]) continue;
        uint8_t *q = buf + 8 + 4 * n++;
        q[0] = i >> 8; q[1] = i; q[2] = api_ram[i] >> 8; q[3] = api_ram[i];
    }
    buf[0] = 'A'; buf[1] = tick >> 24; buf[2] = tick >> 16; buf[3] = tick >> 8; buf[4] = tick;
    buf[5] = (uint8_t)fenetre; buf[6] = n >> 8; buf[7] = n;
    fwrite(buf, 1, 8 + 4 * n, f);
    fflush(f);
}
/* Starting state, once, at the first recorded TICK (DSP handed over, at rest):
 * 'S' whole API RAM, 'D' registers then data memory of the C54x. */
typedef struct {
    int64_t a, b; uint16_t ar[8], t, trn, sp, bk, brc, rsa, rea, st0, st1, pmst, imr, ifr, xpc;
    uint32_t pc; uint8_t idle, running;
} EnregRegs;
static void enreg_base(FILE *f, const C54xState *dsp, const uint16_t *api_ram, uint32_t tick)
{
```

```
    uint8_t h[5] = { 'S', tick >> 24, tick >> 16, tick >> 8, tick };
    fwrite(h, 1, 5, f);
    fwrite(api_ram, sizeof(uint16_t), CALYPSO_API_WORDS, f);
    EnregRegs r = { .a = dsp->a, .b = dsp->b, .t = dsp->t, .trn = dsp->trn, .sp = dsp->sp, .bk = dsp->bk,
                    .brc = dsp->brc, .rsa = dsp->rsa, .rea = dsp->rea, .st0 = dsp->st0, .st1 = dsp->st1,
                    .pmst = dsp->pmst, .imr = dsp->imr, .ifr = dsp->ifr, .xpc = dsp->xpc, .pc = dsp->pc,
                    .idle = dsp->idle, .running = dsp->running };
    memcpy(r.ar, dsp->ar, sizeof r.ar);
    h[0] = 'D';
    fwrite(h, 1, 5, f);
    fwrite(&r, sizeof r, 1, f);
    fwrite(dsp->data, sizeof(uint16_t), C54X_DATA_SIZE, f);
    /* 'P': the low PROGRAM memory 0x0000-0x7FFF. The ROM writes code there at run time
     * (prog_write: 0x2800-0x7FFF only goes into prog[]); without it the replay would call an empty
     * 0x2EC5 at the 1st TCH task, losing the stack. */
    h[0] = 'P';
    fwrite(h, 1, 5, f);
    fwrite(dsp->prog, sizeof(uint16_t), 0x8000, f);
}
static void enreg_tick(const C54xState *dsp, const uint16_t *api_ram, uint32_t tick, bool irq, bool deux, long
 ↪  budget)
{
    FILE *f = calypso_bsp_enreg_fichier();
    if (!f) return;
    static bool base;
    if (!base) { base = true; enreg_base(f, dsp, g_enreg_api_ok ? g_enreg_api : api_ram, tick); }
    uint8_t h[10] = { 'T', tick >> 24, tick >> 16, tick >> 8, tick, (uint8_t)((irq ? 1 : 0) | (deux ? 2 : 0)),
                      (uint8_t)(budget >> 24), (uint8_t)(budget >> 16), (uint8_t)(budget >> 8), (uint8_t)budget };
    fwrite(h, 1, sizeof h, f);
}

/* THE DSP AND THE ARM IN PARALLEL (PONT_DONE_TOT, 1 by default).
 * In lockstep a frame would cost the SUM of QEMU (the ARM) and the C54x: on a
 * TCH the DSP reaches ~4 ms per frame (demodulation, Viterbi, speech), QEMU
 * takes ~2, and the bench would fall below the 216.7 frames/s the speech path
 * needs. On silicon the ARM and the DSP run at the same time and the ARM only
 * reads the results back at the next frame. PONT_DONE is therefore returned
 * as soon as the burst is deposited, and the frame is finished (rest of the
 * budget, DMA pump, uplink, probes) while QEMU advances; the next TICK waits
 * in the socket. A frame then costs the longer of the two, not their sum.
 * Only side effect: PONT_DONE_API_IRQ is not known at the time of the DONE
 * (never raised on this bench: irq=0 in every summary). PONT_DONE_TOT=0
 * finishes the frame before the DONE. */
static int g_done_tot = -1;
static int g_reste_b;

/* PER-FRAME TIMER: where a bench frame goes, in average ms, printed every
 * 1000 frames ("[chrono]"). qemu = wait for the TICK after the DONE (the ARM),
 * A = DSP phase A, go = wait for the GO (the ARM's l1_sync), B = phase B up
 * to the DONE, apres = end of the DSP frame after the DONE. */
#include <time.h>
static double chrono_ms(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); return t.tv_sec * 1e3 +
 ↪  t.tv_nsec / 1e6; }
/* + WORST FRAME: the average cannot exceed 4.62 (the frame is paced by the
 * QEMU TICK, the wait fills the rest); only the max and the number of frames
 * over budget say whether the margin holds. dt = end of frame to end of
 * frame; trop = dt > 5.54 ms (+20 %: the frame is paced at 4.615, half the
 * frames exceed it by plain jitter); dsp_trop = TICK -> end of frame > 4.615 ms
 * (the DSP alone does not keep up with a frame). */
static struct { double t_fin, qemu, a, go, b, apres, dt_max, travail_max; unsigned n, trop, dsp_trop; } g_chrono;
static void chrono_trame(double t_tick, double t_done_a, double t_go, double t_done, double t_fin, uint32_t fn)
{
    if (g_chrono.t_fin > 0) {
        double dt = t_fin - g_chrono.t_fin;
        g_chrono.qemu += t_tick - g_chrono.t_fin;
        if (dt > g_chrono.dt_max) g_chrono.dt_max = dt;
```

```
        if (dt > 5.538) g_chrono.trop++;
    }
    if (t_fin - t_tick > g_chrono.travail_max) g_chrono.travail_max = t_fin - t_tick;
    if (t_fin - t_tick > 4.615) g_chrono.dsp_trop++;
    g_chrono.a += t_done_a - t_tick; g_chrono.go += t_go - t_done_a;
    g_chrono.b += t_done - t_go; g_chrono.apres += t_fin - t_done;
    g_chrono.t_fin = t_fin;
    if (++g_chrono.n == 1000) {
        double k = 1.0 / g_chrono.n;
        printf("  [chrono] fn=%u sur 1000 trames (ms) : qemu %.2f | A %.2f | go %.2f | B %.2f | apres DONE %.2f "
              "| trame %.2f (temps reel 4.62) | pire trame %.2f, DSP pire %.2f, trames >5.54 ms %u/1000, DSP >4.62
               ↪  ms %u/1000\n",
               fn, g_chrono.qemu * k, g_chrono.a * k, g_chrono.go * k,
               g_chrono.b * k, g_chrono.apres * k,
               (g_chrono.qemu + g_chrono.a + g_chrono.go + g_chrono.b + g_chrono.apres) * k,
               g_chrono.dt_max, g_chrono.travail_max, g_chrono.trop, g_chrono.dsp_trop);
        fflush(stdout);
        double tf = g_chrono.t_fin; memset(&g_chrono, 0, sizeof g_chrono); g_chrono.t_fin = tf;
    }
}

static uint32_t jouer_trame(C54xState *dsp, long budget, bool *init_done, uint32_t *insns, int phase)
{
    uint32_t drapeaux = 0;
    uint32_t avant = dsp->insn_count;
    static int fait_a;          /* phase A instructions, charged to phase B's budget */
    static bool etait_idle_a;   /* idle state at the frame start, for PONT_DONE_API_IRQ */

    if (phase == 2) goto phase_b;
    calypso_dma_tick(dsp);

    if (dsp->running && !*init_done) {
        if (!dsp->idle) {
            c54x_run(dsp, (int)budget);
        }
        if (dsp->idle) {
            *init_done = true;
            drapeaux |= PONT_DONE_INIT;
        }
    }
    if (dsp->running) {
        etait_idle_a = dsp->idle;
        /* Wake on a level-held or pending interrupt. The core only vectors a
         * pending interrupt (IFR&IMR, INTM=0) on the next instruction
         * (c54x_irq_level_check, CALYPSO_C54X_IRQ_LEVEL=1), and an idle DSP
         * executes none; on silicon the interrupt itself wakes it (SPRU131).
         * INT10n (RHEA DMA completion, bit 14) is a level line: it stays
         * asserted while a channel holds IRQ_STATE, so present it on wake. */
        if (dsp->idle && calypso_rhea_dma_irq_level() && (dsp->imr & (1u << 14)) &&
            !(dsp->ifr & (1u << 14)))
            c54x_interrupt_ex(dsp, 30, 14);
        if (dsp->idle && (dsp->ifr & dsp->imr) && !(dsp->st1 & 0x800))
            dsp->idle = false;
        /* [2026-09-20] The DSP frame interrupt is TPU_CTRL_DSP_EN, a bit the
         * firmware sets in dsp_end_scenario() on EVERY scenario and that the
         * TPU consumes: the ROM gets a frame interrupt only on frames where
         * the ARM handed it a page. Raising it on every tick made the ROM
         * re-read the same page for 10+ frames (the firmware only flips the
         * write page on frames with a DSP item and the ROM never clears
         * d_task_md): the FB task restarted at each FCCH and the SB job never
         * ran (0xaba4 dispatched every frame, no 764-byte window armed).
         * QEMU now says in TICK.b bit 16 whether the ARM armed it.
         * PONT_IRQ_TRAME=1 restores the interrupt on every tick (A/B). */
        static int irq_chaque = -1;
        if (irq_chaque < 0) { const char *e = calypso_getenv("PONT_IRQ_TRAME"); irq_chaque = (e && *e == '1') ? 1
         ↪  : 0; }
```

```
        if ((dsp->imr & (1u << C54X_IT_TPU_FRAME_BIT)) && (irq_chaque || g_tick_irq_trame)) {
            tx_tsp_dater(dsp, g_c54x_exe_fn);     /* the GSM fn of the page the ROM is about to read */
            c54x_interrupt_ex(dsp, C54X_IT_TPU_FRAME_VEC, C54X_IT_TPU_FRAME_BIT);
        }
        if (calypso_getenv("PONT_IRQ_DEBUG") && g_c54x_exe_fn > 5000 && g_c54x_exe_fn < 5012)
            printf("  [irq] fn=%u APRES vec28 : idle=%d pc=%04x INTM=%d IMR=%04x IFR=%04x\n",
                   g_c54x_exe_fn, dsp->idle, dsp->pc & 0xffff, !!(dsp->st1 & 0x800), dsp->imr, dsp->ifr);
        if (g_inj.actif || phase == 1) {
            int fait = 0;
            if (!dsp->idle) fait = c54x_run_profile(dsp, (int)budget / 8);   /* the ISR arms DMA2 */
            /* [2026-09-20] Deliver only once the receive window is armed. On a
             * frame where the ROM first finishes the previous burst's demod
             * (20-30k instructions of Viterbi) before it programs the one-shot
             * NB window, the fixed budget/8 slice delivered the frame BEFORE the
             * arming, the RIF discarded it (no window) and the result page stayed
             * empty: measured on the BTS bench, one normal-burst result in four
             * missing (b0 absent in 23 of 97 BCCH blocks), read by the firmware
             * as EMPTY / BURST ID n!=m. Keep running, in slices, until DMA2 is
             * armed or the DSP idles, half the budget at most. */
            /* ON THE TCH, RUN TO THE IDLE BEFORE THE DEPOSIT.
             * For a TCHA task (SACCH/TF), the ROM arms the window of frame N+1
             * THEN demodulates, at the start of N+1, the SACCH burst of N it
             * left in 0x0cce; on silicon the burst of N+1 only arrives when
             * TS2 comes round, later. Depositing as soon as the window is
             * armed would overwrite it and the SACCH would be decoded on a
             * traffic burst (a_cd FIRE KO on every block). Outside the TCH
             * (FB, SB, SDCCH) the stop at the arming is kept.
             * PONT_TCH_DEPOT_IDLE=0 deposits at the arming on the TCH too. */
            int tch_idle = calypso_reglage("PONT_TCH_DEPOT_IDLE", 1);   /* read live */
            bool jusqua_idle = tch_idle && montant_sur_tch();
            while (!dsp->idle && (jusqua_idle || !calypso_rhea_dma_rx_armed()) && fait < (int)budget / 2)
                fait += c54x_run_profile(dsp, 256);
            if (jusqua_idle) {
                static unsigned n_tch;
                /* + irq_trame/IMR/IFR/INTM/pc: tells a frame IT not raised (QEMU) from a TPU frame
                 * bit missing from the IMR (ROM) when the DSP sits at "0 insn, idle=1" on a TCH.
                 * 200 lines to cover several calls. */
                if (n_tch++ < 200)
                    printf("  [depot_tch] fn=%u phase A : %d insn, idle=%d, fenetre armee=%d irq_trame=%d "
                           "IMR=%04x IFR=%04x INTM=%d pc=%04x%s\n",
                           g_c54x_exe_fn, fait, dsp->idle, calypso_rhea_dma_rx_armed(), g_tick_irq_trame,
                           dsp->imr, dsp->ifr, !!(dsp->st1 & 0x800), dsp->pc & 0xffff,
                           dsp->idle ? "" : "  <- BUDGET/2 ATTEINT, depot avant la fin");
            }
            fait_a = fait;
            if (phase == 1) {
                /* ISR played, R page written, window armed: the ARM may run
                 * l1_sync now. The burst comes with PONT_GO. */
                if (dsp->idle)    drapeaux |= PONT_DONE_IDLE;
                if (dsp->running) drapeaux |= PONT_DONE_RUNNING;
                if (*init_done)   drapeaux |= PONT_DONE_INIT;
                *insns = dsp->insn_count - avant;
                return drapeaux | PONT_DONE_PHASE_A;
            }
    phase_b:
            fait = fait_a;
            if (dsp->api_ram) montant_capter_fu(dsp->api_ram, g_c54x_exe_fn);   /* before the ROM reads a_fu */
            /* PONT_NB_DEBUG: flags at the burst's entry, and a snapshot of the
             * data memory to list what the demod writes (regions), see
             * sonde_bits(). */
            { static int on = -1; if (on < 0) on = calypso_getenv("PONT_NB_DEBUG") ? 1 : 0;
              if (on) { g_flags_entree = ((s_or_dsp_st0(dsp) & ST0_OVA) ? 1 : 0) | ((dsp->st0 & ST0_OVB) ? 2 : 0) |
                ↪  ((dsp->st0 & ST0_C) ? 4 : 0) | ((dsp->st0 & ST0_TC) ? 8 : 0) | ((dsp->st1 & ST1_OVM) ? 16 : 0) |
                ↪  ((dsp->st1 & ST1_FRCT) ? 32 : 0) | ((dsp->st1 & ST1_SXM) ? 64 : 0);
                        memcpy(g_data_avant_b, dsp->data, sizeof g_data_avant_b); } }
            if (g_inj.udp) calypso_bsp_service(g_inj.fn);
```

```
            if (g_inj.iq_mode) injecter_burst(dsp, g_inj.iq_mode, g_inj.amp, g_inj.fn, g_inj.injectes);
            { uint16_t aad = calypso_rhea_dma_get_daram();
              memcpy(g_daram_apres_dma, &dsp->data[aad], sizeof g_daram_apres_dma); g_daram_aad = aad; }
            if (dsp->idle && calypso_rhea_dma_irq_level() && (dsp->imr & (1u << 14)) && !(dsp->ifr & (1u << 14)))
                c54x_interrupt_ex(dsp, 30, 14);
            if (dsp->idle && (dsp->ifr & dsp->imr) && !(dsp->st1 & 0x800)) dsp->idle = false;
            /* PONT_NB_HIST=<dir>: opcode histogram of phase B (the burst's demod)
             * for the first 12 normal bursts, one file per frame, to name the
             * instructions the NB path leans on and cross them with the ISA
             * scorecard (isa_test). Single-stepped: prog_fetch() before each. */
            static const char *hist_dir = NULL; static int hist_init = 0; static unsigned hist_n;
            if (!hist_init) { hist_init = 1; hist_dir = calypso_getenv("PONT_NB_HIST"); }
            /* PONT_NB_HIST_SB=1: trace the SCH bursts (SB task, d_task_md = 6) instead */
            static int hist_sb = -1; if (hist_sb < 0) hist_sb = calypso_getenv("PONT_NB_HIST_SB") ? 1 : 0;
            bool tache_nb = dsp->api_ram && (dsp->api_ram[API_R_PAGE(0) / 2] == 24 || dsp->api_ram[API_R_PAGE(1) /
             ↪  2] == 24);
            bool tache_sb = dsp->api_ram && ((dsp->api_ram[API_W_PAGE(0) / 2 + 4] & 0xff) == 6 ||
             ↪  (dsp->api_ram[API_W_PAGE(1) / 2 + 4] & 0xff) == 6);
            if (hist_dir && hist_n < 12 && !dsp->idle && dsp->api_ram &&
                ((!hist_sb && g_inj.dernier_type == 'B' && tache_nb) || (hist_sb && g_inj.dernier_type == 'S' &&
                 ↪  tache_sb))) {
                static unsigned hist[65536]; memset(hist, 0, sizeof hist);
                int reste = (int)budget - fait, k = 0;
                /* data watch: every write into the demod's working cells, with
                 * the PC of the instruction (taps 0x2cbb.., tracker 0x5aaa..,
                 * result cells 0x3fa4.., reference 0x2b28..) */
                static const struct { uint16_t lo, hi; } W[] = { {0x2cbb, 0x2d04}, {0x5aaa, 0x5ac8}, {0x3fa4,
                 ↪  0x3fa8}, {0x2b28, 0x2b58}, {0x2f00, 0x2f2c}, {0x2a00, 0x2bc8}, {0x2be0, 0x2c80} };
                #define NW 7
                static uint16_t prev[0x600]; int nw = 0;
                for (unsigned r = 0; r < NW; r++) for (unsigned a = W[r].lo; a < W[r].hi; a++) prev[nw++] =
                 ↪  dsp->data[a];
                char nomw[256]; snprintf(nomw, sizeof nomw, "%s/watch_%u.txt", hist_dir, g_inj.fn);
                FILE *fw = fopen(nomw, "w");
                char nomt[256]; snprintf(nomt, sizeof nomt, "%s/trace_%u.txt", hist_dir, g_inj.fn);
                FILE *ft = fopen(nomt, "w");
                while (!dsp->idle && k < reste) {
                    uint16_t w = prog_fetch(dsp, (uint16_t)dsp->pc);
                    uint16_t pc0 = (uint16_t)dsp->pc; uint16_t ar2 = dsp->ar[2], ar3 = dsp->ar[3];
                    hist[w]++;
                   if (k == 12800) { memcpy(g_snap_2be2, &dsp->data[0x2be2], sizeof g_snap_2be2); g_snap_ok = 1; }
                    if (pc0 == 0x9a78 && !g_snap_2a00_ok) { memcpy(g_snap_2a00, &dsp->data[0x2a00], sizeof
                     ↪  g_snap_2a00); g_snap_2a00_ok = 1;
                      char nm[256]; snprintf(nm, sizeof nm, "%s/mem_%u_9a78.bin", hist_dir, g_inj.fn); FILE *fm =
                       ↪  fopen(nm, "wb"); if (fm) { fwrite(dsp->data, 2, C54X_DATA_SIZE, fm); fclose(fm); } }
                    if (ft) fprintf(ft, "%04x %04x %010llx %010llx %04x %04x %04x %04x %04x %04x %04x %04x %04x
                     ↪  %04x %04x %04x %04x %04x\n", pc0, w,
                                    (unsigned long long)(dsp->a & 0xFFFFFFFFFFULL), (unsigned long long)(dsp->b &
                                     ↪  0xFFFFFFFFFFULL),
                                    dsp->t, dsp->st0, dsp->st1, dsp->ar[1], ar2, ar3, dsp->ar[4], dsp->ar[5],
↪  dsp->ar[0],
                                    dsp->data[ar2], dsp->data[ar3], dsp->data[dsp->ar[4]], dsp->data[dsp->ar[5]],
↪  dsp->ar[6]);
                    c54x_run(dsp, 1); k++;
                    if (fw) { int i = 0;
                        for (unsigned r = 0; r < NW; r++) for (unsigned a = W[r].lo; a < W[r].hi; a++, i++)
                           if (dsp->data[a] != prev[i]) { fprintf(fw, "%d pc=%04x op=%04x %04x: %04x -> %04x (%d)
                            ↪  AR2=%04x AR3=%04x\n", k, pc0, w, a, prev[i], dsp->data[a], (int16_t)dsp->data[a],
                            ↪  ar2, ar3); prev[i] = dsp->data[a]; } }
                }
                if (fw) fclose(fw);
                if (ft) fclose(ft);
                { char nm[256]; snprintf(nm, sizeof nm, "%s/mem_%u_fin.bin", hist_dir, g_inj.fn); FILE *fm =
                 ↪  fopen(nm, "wb"); if (fm) { fwrite(dsp->data, 2, C54X_DATA_SIZE, fm); fclose(fm); } }
                char nom[256]; snprintf(nom, sizeof nom, "%s/hist_%u.txt", hist_dir, g_inj.fn);
                FILE *f = fopen(nom, "w");
```

```
                if (f) { for (unsigned w = 0; w < 65536; w++) if (hist[w]) fprintf(f, "%04x %u\n", w, hist[w]);
                 ↪  fclose(f); }
                hist_n++;
            }
            /* PONT_DONE_TOT: the rest of the frame (phase B) is played AFTER the
             * PONT_DONE, while QEMU runs the ARM -- see servir(). */
            if (g_done_tot && phase == 2) g_reste_b = (int)budget - fait;
            else if (!dsp->idle) c54x_run_profile(dsp, (int)budget - fait);
        } else if (!dsp->idle) {
            c54x_run_profile(dsp, (int)budget);
        }
        if (!etait_idle_a && dsp->idle) {
            drapeaux |= PONT_DONE_API_IRQ;
        }
    }
    if (dsp->idle)    drapeaux |= PONT_DONE_IDLE;
    if (dsp->running) drapeaux |= PONT_DONE_RUNNING;
    if (*init_done)   drapeaux |= PONT_DONE_INIT;
    *insns = dsp->insn_count - avant;
    return drapeaux;
}

/* Synthetic I/Q injection.
 *
 * On silicon the DSP gets 148 complex int16 samples per burst, one per symbol,
 * written to DARAM 0x2a00 (296 words) by the BSP DMA and followed by the RX
 * interrupt; calypso_bsp_rx_burst() reproduces that sequence. An FCCH burst is
 * GMSK over 148 zero bits, i.e. a tone at +1625/24 kHz, which is +pi/2 of
 * phase per symbol at 270.833 ksym/s (GSM 45.004). */
#define IQ_N 148
static void iq_synthese(const char *mode, int amp, uint32_t fn, int16_t *iq)
{
    static unsigned seed = 12345;
    double dphi = 0;
    bool bruit = false;
    if (!strcmp(mode, "fcch")) {
        dphi = M_PI / 2;
    } else if (!strcmp(mode, "noise")) {
        bruit = true;
    } else if (!strncmp(mode, "tone:", 5)) {
        dphi = atof(mode + 5);
    }
    for (int k = 0; k < IQ_N; k++) {
        if (bruit) {
            seed = seed * 1103515245u + 12345u; int a = (int)((seed >> 16) % (2 * amp + 1)) - amp;
            seed = seed * 1103515245u + 12345u; int b = (int)((seed >> 16) % (2 * amp + 1)) - amp;
            iq[2 * k] = (int16_t)a; iq[2 * k + 1] = (int16_t)b;
        } else {
            double ph = dphi * k;
            iq[2 * k]     = (int16_t)lrint(amp * cos(ph));
            iq[2 * k + 1] = (int16_t)lrint(amp * sin(ph));
        }
    }
    (void)fn;
}

/* Recorded-cell replay, paced by the DSP frame clock.
 *
 * /opt/GSM/cellule_reelle.bin holds the TS0 of 311 CONSECUTIVE frames captured
 * off the air at 4 samples/symbol, each tagged with its real frame number, so
 * the 51-multiframe is intact: FCCH at fn%51 in {0,10,20,30,40}, SCH at
 * {1,11,21,31,41}.
 *
 * Why it is fed from here and not over UDP: the source and the DSP have
 * independent clocks. Measured on this bench, the DSP runs between 72 and 160
 * frames/s depending on load while rejouer_cellule.py streams at a rate of its
```

```
 * own, so the DSP swallowed about four cell frames per frame of its own. The
 * FB task survives that — it only ever correlates ONE window — but the SB task
 * needs the burst of the frame IMMEDIATELY AFTER the FCCH, and that frame was
 * never the SCH. Pulled from here, one burst per frame, the multiframe reaches
 * the DSP intact whatever the emulated clock does. */
typedef struct { uint32_t fn; int16_t *iq; int fcch; } ReelBurst;
static ReelBurst *g_reel;
static unsigned   g_reel_n, g_reel_util;
static int        g_reel_nsym;
static unsigned   g_reel_base;

/* Decimate a stored burst to the 1 sample/symbol the correlator works at. The
 * same decimation the UDP path applies through CALYPSO_BSP_IQ_DECIM, and the
 * same one reelle_injecter delivers: what is measured here is what the DSP
 * gets. Returns the number of int16 written. */
static int reelle_decimer(const ReelBurst *b, int16_t *iq, int max_i16)
{
    int decim = g_reel_nsym / 148;
    if (decim < 1) decim = 1;
    int n = 0;
    for (int k = 0; k * decim < g_reel_nsym && n <= max_i16 - 2; k++) {
        iq[n++] = b->iq[2 * (k * decim)];
        iq[n++] = b->iq[2 * (k * decim) + 1];
    }
    return n;
}

/* Is this burst an FCCH? At 1 sample/symbol an all-zeros GMSK burst is a pure
 * tone rotating by exactly +pi/2 per sample — the criterion calypso_bsp.c's
 * FCCH-PROBE already uses (coh ~ 1, dphi ~ +1.571). Nothing else on TS0 comes
 * close, so this labels the multiframe without decoding anything. */
static int reelle_est_fcch(const ReelBurst *b)
{
    int16_t iq[2 * 256];
    int n = reelle_decimer(b, iq, (int)(sizeof(iq) / sizeof(iq[0])));
    int ns = n / 2;
    if (ns < 32) return 0;
    double accr = 0, acci = 0, den = 0;
    for (int k = 1; k < ns; k++) {
        double i0 = iq[2*(k-1)], q0 = iq[2*(k-1)+1];
        double i1 = iq[2*k],     q1 = iq[2*k+1];
        accr += i1*i0 + q1*q0;
        acci += q1*i0 - i1*q0;
        den  += sqrt((i0*i0 + q0*q0) * (i1*i1 + q1*q1));
    }
    if (den <= 0) return 0;
    double coh  = sqrt(accr*accr + acci*acci) / den;
    double dphi = atan2(acci, accr);
    return coh > 0.90 && fabs(dphi - M_PI_2) < 0.30;
}

/* Where does the recording sit in the 51-multiframe?
 *
 * The frame numbers stored in the file cannot be trusted for this: measured on
 * cellule_reelle.bin, the FCCH bursts carry tags whose residues are
 * {0,10,20,31,41} where GSM 05.02 puts the FCCH at fn%51 in {0,10,20,30,40} —
 * a constant offset of 31. So the phase is taken from the SIGNAL instead: the
 * FCCH bursts are located by reelle_est_fcch(), and the one followed by a gap
 * of 11 frames is the last of its multiframe, i.e. true phase 40 (the sequence
 * of gaps is 10,10,10,10,11). g_reel_base then satisfies
 *
 *     entry_index  ==  (arm_fn - g_reel_base)  (mod 51)
 *
 * which is what reelle_injecter uses to pick a burst BY FRAME NUMBER. */
static void reelle_caler(void)
{
```

```
    unsigned fcch[64], nf = 0;
    for (unsigned i = 0; i < g_reel_n && nf < 64; i++)
        if (g_reel[i].fcch) fcch[nf++] = i;

    unsigned i40 = 0; int trouve = 0;
    for (unsigned k = 0; k + 1 < nf; k++)
        if (fcch[k + 1] - fcch[k] == 11) { i40 = fcch[k]; trouve = 1; break; }

    if (!trouve) {
        g_reel_base = 0;
        printf("pont : ATTENTION — phase de multitrame indeterminee (%u FCCH reperees, "
               "aucun ecart de 11) ; calage a 0, la lecture reste cadencee par fn\n", nf);
        return;
    }
    g_reel_base = (40u + 51u - (i40 % 51u)) % 51u;   /* i40 == 40 - base  (mod 51) */
    unsigned ecart_tag = (g_reel[0].fn + 51u - g_reel_base) % 51u;
    printf("pont : %u FCCH reperees dans la capture (ecarts 10/11), phase calee sur le signal : "
           "base=%u ; les tags fn du fichier sont decales de +%u mod 51\n",
           nf, g_reel_base, ecart_tag);
}

/* Recorded-cell replay, addressed by the ARM frame number.
 *
 * /opt/GSM/cellule_reelle.bin holds the TS0 of 311 CONSECUTIVE frames captured
 * off the air at 4 samples/symbol, each tagged with its real frame number.
 *
 * Why it is fed from here and not over UDP: the source and the DSP have
 * independent clocks. Measured on this bench, the DSP runs between 72 and 160
 * frames/s depending on load while rejouer_cellule.py streams at a rate of its
 * own, so the DSP swallowed about four cell frames per frame of its own. The
 * FB task survives that — it only ever correlates ONE window — but the SB task
 * needs the burst of the frame IMMEDIATELY AFTER the FCCH, and that frame was
 * never the SCH.
 *
 * [2026-09-19] Why it is addressed by fn and no longer by a running index:
 * calypso_trx.c:pont_echange() SKIPS a tick whenever the DSP has not returned
 * its DONE — about 3% of frames once the traces are off, 85% with -vvv on a
 * terminal. A running index does not advance on a skipped tick while the ARM
 * clock does, so every skip shifted the recording against the ARM by one frame,
 * FOR GOOD: measured 4202 frames of accumulated drift over a 5-minute run, i.e.
 * a multiframe phase wandering without bound. The ARM then armed its SB one
 * frame after an FCCH and got a burst from somewhere else entirely (11 SB CRC
 * OK in 68000 frames), and the deinterleaver assembled BCCH blocks out of
 * unrelated bursts (205-221 bit errors per block, every block dropped).
 * Indexing by fn makes a skipped tick skip a RECORDING burst too: the phase is
 * held whatever the emulated clock does. */
static void reelle_injecter(C54xState *dsp, uint32_t fn, unsigned long *injectes)
{
    if (!g_reel_util) return;
    unsigned idx = (fn + g_reel_util - (g_reel_base % g_reel_util)) % g_reel_util;
    const ReelBurst *b = &g_reel[idx];

    int16_t iq[2 * 256];
    int n_iq = reelle_decimer(b, iq, (int)(sizeof(iq) / sizeof(iq[0])));

    /* The burst keeps its OWN frame number: it is the provenance of the samples,
     * and calypso_bsp_rx_burst only ever logs it. */
    calypso_bsp_rx_burst(0, b->fn, iq, n_iq);
    (*injectes)++;
}

static int reelle_charger(const char *chemin)
{
    FILE *f = fopen(chemin, "rb");
    if (!f) { fprintf(stderr, "pont : cellule reelle introuvable : %s\n", chemin); return 0; }
    uint32_t hdr[4];
```

```
    if (fread(hdr, sizeof(hdr), 1, f) != 1) { fclose(f); return 0; }
    unsigned n = hdr[0], nsym = hdr[1], bsic = hdr[2];
    if (!n || !nsym || nsym > 4096) { fclose(f); return 0; }
    g_reel = calloc(n, sizeof(*g_reel));
    if (!g_reel) { fclose(f); return 0; }
    for (unsigned i = 0; i < n; i++) {
        uint32_t e[2];
        if (fread(e, sizeof(e), 1, f) != 1) { n = i; break; }
        int16_t *iq = malloc((size_t)nsym * 2 * sizeof(int16_t));
        if (!iq || fread(iq, sizeof(int16_t) * 2, nsym, f) != nsym) { free(iq); n = i; break; }
        g_reel[i].fn = e[0];
        g_reel[i].iq = iq;
    }
    fclose(f);
    g_reel_n = n; g_reel_nsym = (int)nsym;

    /* Only a WHOLE number of 51-multiframes may be looped: 311 = 6*51 + 5, so
     * wrapping on all 311 shifted the phase by 5 at every turn. */
    g_reel_util = (n / 51u) * 51u;

    for (unsigned i = 0; i < g_reel_n; i++)
        g_reel[i].fcch = reelle_est_fcch(&g_reel[i]);

    printf("pont : cellule REELLE %s — %u trames consecutives a %d ech/symbole, BSIC=%u, "
           "fn %u..%u, %u trames jouees (%u multitrames de 51, %u ecartees), adressage par fn\n",
           chemin, n, nsym / 148, bsic, n ? g_reel[0].fn : 0, n ? g_reel[n - 1].fn : 0,
           g_reel_util, g_reel_util / 51u, n - g_reel_util);
    reelle_caler();
    return (int)g_reel_util;
}

/* Inject one burst (synthetic cell or plain signal) into the RIF/BSP path. */
static int g_verif_sonde = -1;

static void injecter_burst(C54xState *dsp, const char *iq_mode, int amp, uint32_t fn,
                           unsigned long *injectes)
{
        int16_t iq[2 * 256];
        int n_iq = 2 * IQ_N;
        char t_dbg = '?';
        if (!strncmp(iq_mode, "reelle", 6)) {
            static int charge = 0;
            if (!charge) { const char *p = strchr(iq_mode, ':');
                charge = reelle_charger(p ? p + 1 : "/opt/GSM/cellule_reelle.bin"); if (!charge) charge = -1; }
            if (charge > 0) reelle_injecter(dsp, fn, injectes);
            return;
        }
        if (!strncmp(iq_mode, "cell", 4)) {
            /* cell[:bsic[:offset[:margin]]] : full cell, FCCH/SCH/dummy bursts */
            static int bsic = -1; static double dec = 0.5; static int marge = 21;
            if (bsic < 0) { bsic = 42; const char *p = strchr(iq_mode, ':');
                if (p) { bsic = atoi(p + 1) & 0x3f; p = strchr(p + 1, ':');
                    if (p) { dec = atof(p + 1); p = strchr(p + 1, ':'); if (p) marge = atoi(p + 1); } }
                if (marge > 50) marge = 50;
                if (strstr(iq_mode, "all")) cellule_sch_partout = 1;
                printf("pont : cellule BSIC=%d (NCC=%d BCC=%d), echantillonnage a %.2f symbole, "
                       "SCH dans une fenetre de %d echantillons (DARAM len=%u mots)%s\n",
                       bsic, bsic >> 3, bsic & 7, dec, 148 + 2 * marge, calypso_bsp_get_daram_len(),
                       cellule_sch_partout ? ", SCH sur toutes les trames non-FCCH" : ""); }
            /* the SCH burst is delivered as the 190-sample window block only when
             * the DSP has its one-shot SB window armed; otherwise it is a frame of
             * the FB stream like any other (same rule as rejouer.c) */
            int marge_eff = calypso_rhea_dma_one_shot() ? marge : 0;
            /* [2026-09-20] BCCH/CCCH normal bursts: framed like the BSP stream
             * assembler (bsp_livrer_trame), 3 silent samples ahead when the
             * one-shot window is the 151-sample NB one (tpu_window.c
```

```
             * L1_NB_MARGIN_Q), bare 148 samples in the continuous FB stream. */
            { int nwin = calypso_rhea_dma_one_shot() ? calypso_rhea_dma_get_len_words() / 2 : 0;
              /* PONT_NB_MARGE=<n> overrides the 3-sample head margin of a normal
               * burst; "auto" sweeps 0..7 by 51-multiframe (the [scan] probe
               * prints it), one run to find where the ROM's TSC search lands. */
              static int mnb = -2; static int mauto = 0;
              if (mnb == -2) { const char *e = calypso_getenv("PONT_NB_MARGE"); mnb = 3;
                               if (e && !strcmp(e, "auto")) mauto = 1; else if (e && *e) mnb = atoi(e); }
              int m_nb = mauto ? (int)((fn / 51u) % 8u) : mnb;
              /* CELLULE_TSC=<k>|auto : training sequence written in the bursts
               * (the ARM still tells the ROM tsc = BCC); auto sweeps 0..7 */
              static int tsc = -2; static int tauto = 0;
              if (tsc == -2) { const char *e = calypso_getenv("CELLULE_TSC"); tsc = -1;
                               if (e && !strcmp(e, "auto")) tauto = 1; else if (e && *e) tsc = atoi(e) & 7; }
              cellule_tsc_force = tauto ? (int)((fn / 51u) % 8u) : tsc;
              cellule_marge_nb = (nwin >= 150 && nwin < 190) ? m_nb : (nwin >= 190 ? marge : -1);
              cellule_fenetre_nb = nwin; }
            char t = cellule_burst(fn, (uint8_t)bsic, amp, dec, marge_eff, iq, &n_iq);
            t_dbg = t; g_inj.dernier_type = t; g_inj.dernier_n_iq = n_iq;
            static unsigned nS, nF, nB, nC, tot;
            if (t == 'S') nS++; else if (t == 'F') nF++; else if (t == 'B') nB++; else if (t == 'C') nC++;
            if (++tot % 5000 == 1) printf("pont : bursts injectes FCCH=%u SCH=%u BCCH=%u CCCH=%u (fn=%u)\n", nF,
             ↪  nS, nB, nC, fn);
        } else {
            iq_synthese(iq_mode, amp, fn, iq);
        }
        static int irqdbg = -1; static unsigned irqdbg_n;
        if (irqdbg < 0) irqdbg = calypso_getenv("PONT_IRQ_DEBUG") ? 1 : 0;
        bool dbg = irqdbg && fn > 5000 && irqdbg_n < 12;
        if (dbg) printf("  [irq] fn=%u AVANT rx_burst : idle=%d pc=%04x INTM=%d IMR=%04x IFR=%04x PMST=%04x
         ↪  SP=%04x\n",
                        fn, dsp->idle, dsp->pc & 0xffff, !!(dsp->st1 & 0x800), dsp->imr, dsp->ifr, dsp->pmst,
↪  dsp->sp);
        { static int dj = -1; if (dj < 0) dj = calypso_getenv("PONT_DEBUG_INJ") ? 1 : 0;
          int md0 = dsp->api_ram ? (dsp->api_ram[4] & 0xff) : 0, md1 = dsp->api_ram ? (dsp->api_ram[0x18] & 0xff)
           ↪  : 0;
          if (dj && (calypso_rhea_dma_one_shot() || md0 == 6 || md1 == 6 || (fn >= 300 && fn <= 312)))
              printf("  [inj] fn=%u p51=%u AVANT : type=%c n_iq=%d dma armee=%d one_shot=%d task_md=%d/%d rif=%d
↪  mots idle=%d pc=%04x"
                     " | W0=%04x %04x %04x %04x %04x ..%04x %04x  W1=%04x %04x %04x %04x %04x ..%04x %04x  NDB
                      ↪  page=%04x fb_mode=%04x fb_det=%04x\n",
                     fn, fn % 51u, t_dbg, n_iq, calypso_rhea_dma_rx_armed(), calypso_rhea_dma_one_shot(), md0,
↪  md1, calypso_rif_level(), dsp->idle, dsp->pc & 0xffff,
                     dsp->api_ram[0], dsp->api_ram[1], dsp->api_ram[2], dsp->api_ram[3], dsp->api_ram[4],
↪  dsp->api_ram[15], dsp->api_ram[16],
                     dsp->api_ram[0x14], dsp->api_ram[0x15], dsp->api_ram[0x16], dsp->api_ram[0x17],
↪  dsp->api_ram[0x18], dsp->api_ram[0x14+15], dsp->api_ram[0x14+16],
                     dsp->api_ram[0xd4], dsp->api_ram[0xd4+37], dsp->api_ram[0xd4+36]); }
        memcpy(g_dernier_iq, iq, (size_t)n_iq * sizeof(int16_t)); g_dernier_n_iq = n_iq;
        calypso_bsp_rx_burst(0, fn, iq, n_iq);
        { static int dj2 = -1; if (dj2 < 0) dj2 = calypso_getenv("PONT_DEBUG_INJ") ? 1 : 0;
          if (dj2 && fn >= 300 && fn <= 312)
              printf("  [inj] fn=%u APRES : rif=%d mots idle=%d IFR=%04x\n", fn, calypso_rif_level(), dsp->idle,
↪  dsp->ifr); }
        if (dbg) { printf("  [irq] fn=%u APRES rx_burst : idle=%d pc=%04x INTM=%d IMR=%04x IFR=%04x SP=%04x\n",
                          fn, dsp->idle, dsp->pc & 0xffff, !!(dsp->st1 & 0x800), dsp->imr, dsp->ifr, dsp->sp);
↪  irqdbg_n++; }
        (*injectes)++;
}


/* Register of active hacks. Every departure from native behaviour (canned TOA,
 * forced AFC rotation, direct feed, locks, ...) is listed by hacks_actifs() and
 * printed on every milestone line: a milestone reached with a non-empty list is
 * not a native milestone. The stimulus (synthetic cell) is reported separately,
 * being test input rather than a crutch. */
```

```
/* SB encoder: exact inverse of l1s_decode_sb (prim_fbsb.c), identical to
 * shunt_encode_sb in qemu-calypso. Maps {bsic, T1, T2, T3} to the 25-bit sb
 * word; the firmware reads sb = a_sch[3] | a_sch[4]<<16, then bsic = (sb>>2)
 * & 0x3f, and so on (GSM 45.002 SCH layout). */
static uint32_t pont_encode_sb(uint8_t bsic, uint16_t t1, uint8_t t2, uint8_t t3)
{
    uint8_t t3p = (t3 == 0) ? 0 : (uint8_t)((t3 - 1) / 10);
    uint32_t sb = 0;
    sb |= ((uint32_t)(bsic & 0x3f)) << 2;
    sb |= ((uint32_t)(t1 & 0x001)) << 23;
    sb |= ((uint32_t)(t1 & 0x1fe)) << 7;
    sb |= ((uint32_t)(t1 & 0x600)) >> 9;
    sb |= ((uint32_t)(t2 & 0x1f))  << 18;
    sb |= ((uint32_t)(t3p & 1))    << 24;
    sb |= ((uint32_t)(t3p & 6))    << 15;
    return sb;
}

/* PACING, NOT A CRUTCH.
 *
 * STREAM and LOCKSTEP are not listed among the crutches. Neither falsifies a
 * measurement: they let an emulated DSP, which costs ~6.7 ms per frame,
 * coexist with a real-time BTS whose frame lasts 4.615 ms.
 *   - without LOCKSTEP, QEMU advances on the wall clock and skips the frames
 *     the DSP has not finished (about 39 % loss);
 *   - without STREAM, every burst goes to the RIF as soon as it arrives: 217
 *     bursts per second for ~89 ticks, the ROM counts ~1526 symbols per frame
 *     instead of 1250, its TOA never settles and the pipeline misaligns
 *     ("BURST ID 3!=2", "EMPTY").
 * The paced mode delivers what the BTS actually sent on all eight timeslots
 * (calypso_bsp.c, bsp_autres_stocker/bsp_autres_bits), not just TS0 with the
 * other seven padded with zeros.
 *
 * They are still displayed, but on their own line. */
static const char *cadencement_actif(void)
{
    static char buf[128]; buf[0] = 0;
    const char *s1 = calypso_getenv("CALYPSO_BSP_STREAM");
    const char *s2 = calypso_getenv("CALYPSO_PONT_LOCKSTEP");
    if (s1 && *s1 == '1') strcat(buf, "STREAM ");
    if (s2 && *s2 == '1') strcat(buf, "LOCKSTEP ");
    return buf[0] ? buf : "aucun (le DSP ne suivra pas le temps reel)";
}

static const char *hacks_actifs(void)
{
    static char buf[512]; buf[0] = 0;
    struct { const char *env, *tag; int mode; } t[] = {   /* mode 0: set and non-empty; 1: =="1"; 2: =="0"; 3:
     ↪  integer != 0; 4: integer >= 0 */
        {"PONT_CAN_TOA","CAN_TOA",4}, {"PONT_CAN_SB_TOA","CAN_SB_TOA",4}, {"PONT_CAN_SB","CAN_SB_FULL",0},
        {"CALYPSO_TWL3025_AFC_HZ","AFC_HZ",3}, {"CALYPSO_TWL3025_AFC","AFC_OFF",2},
        {"CALYPSO_TWL3025_AFC_SIGN_OLD","AFC_SIGN_OLD",0},
        {"CALYPSO_BSP_VEC30","VEC30",1},
        {"CALYPSO_BSP_RX_LEAD","RX_LEAD",3}, {"CALYPSO_BSP_TPU_TRACK","TPU_TRACK",1},
        {"CALYPSO_BSP_TOA_LOCK","TOA_LOCK",1},
        {"CALYPSO_BSP_IQ_PASSTHROUGH","IQ_SYNTH",2},
        {"CALYPSO_RHEA_DMA_XFER","RHEA_DMA",1}, {"CALYPSO_BSP_RX_VEC","RX_VEC",0},
        {"CELLULE_SCH_ONLY","SCH_ONLY",1}, {"CELLULE_SCH_AMPDIV","SCH_AMPDIV",3},
        {"CALYPSO_FIXES","FIXES",0},
    };
    for (unsigned i = 0; i < sizeof t / sizeof t[0]; i++) {
        const char *v = calypso_getenv(t[i].env); if (!v || !*v) continue;
        int on = 0; long n = atol(v);
        switch (t[i].mode) { case 0: on = 1; break; case 1: on = (*v=='1'); break; case 2: on = (*v=='0'); break;
                             case 3: on = (n != 0); break; case 4: on = (n >= 0); break; }
```

```
        if (!on) continue;
        size_t l = strlen(buf);
        if (t[i].mode == 3 || t[i].mode == 4) snprintf(buf + l, sizeof buf - l, "%s%s=%ld", l ? "," : "",
         ↪  t[i].tag, n);
        else snprintf(buf + l, sizeof buf - l, "%s%s", l ? "," : "", t[i].tag);
    }
    /* a decoder fix turned off is a departure from native too */
    static const char *fx[] = {"NORM_SD","F7_DELAYED","MPY_MAC_LK","MACP_MACD","PAR_ST_DSTBAR","STL_STH_SHFT",
     ↪  "XCCD","ADDSUB_XSHFT","FIRS_RPT","RPT_COUNT"};
    for (unsigned i = 0; i < sizeof fx / sizeof fx[0]; i++) {
        char e[64]; snprintf(e, sizeof e, "CALYPSO_FIX_%s", fx[i]); const char *v = calypso_getenv(e);
       if (v && *v == '0') { size_t l = strlen(buf); snprintf(buf + l, sizeof buf - l, "%sFIX_%s=0", l ? "," : "",
         ↪  fx[i]); }
    }
    return buf[0] ? buf : "aucun";
}

/* PONT_NB_DEBUG=1: pages as seen at one instant of the frame (A = end of the
 * ROM's frame ISR, G = PONT_GO received i.e. after the ARM's l1_sync, B = end
 * of the frame). Who reads which W page and who writes which R page, when. */
static void sonde_pages(const char *quand, uint32_t fn, C54xState *dsp, const uint16_t *api_ram)
{
    static int on = -1; static unsigned n;
    if (on < 0) on = calypso_getenv("PONT_NB_DEBUG") ? 1 : 0;
    if (!on || n >= 240) return;
    const uint16_t *w0 = &api_ram[API_W_PAGE(0) / 2], *w1 = &api_ram[API_W_PAGE(1) / 2];
    const uint16_t *r0 = &api_ram[API_R_PAGE(0) / 2], *r1 = &api_ram[API_R_PAGE(1) / 2];
    if (!(w0[0] == 24 || w1[0] == 24 || r0[0] == 24 || r1[0] == 24)) return;
    n++;
    printf("  [pg%s] fn=%u p51=%u W0=%u/%u W1=%u/%u R0=%u/%u R1=%u/%u dsp_page=%04x idle=%d pc=%04x IFR=%04x
 ↪  IMR=%04x INTM=%d irq_trame=%d dma_armee=%d\n",
           quand, fn, fn % 51u, w0[0], w0[1], w1[0], w1[1], r0[0], r0[1], r1[0], r1[1],
           api_ram[(API_NDB + NDB_D_DSP_PAGE) / 2], dsp->idle, dsp->pc & 0xffff, dsp->ifr, dsp->imr,
           !!(dsp->st1 & 0x800), g_tick_irq_trame, calypso_rhea_dma_rx_armed());
}

/* PONT_NB_DEBUG=1, after a normal burst: look for the demodulated bits in the
 * DSP data memory. Two templates, the 116 data bits (57+hl, hu+57) and the
 * whole 148-bit burst, matched against the sign of each int16 word (soft bits,
 * both polarities) and against hard 0/1 words. Reports the best run. */
static uint32_t g_insn_a, g_insn_b;
static void sonde_bits(uint32_t fn, C54xState *dsp, uint8_t bsic)
{
    static int on = -1; static unsigned n;
    if (on < 0) on = calypso_getenv("PONT_NB_DEBUG") ? 1 : 0;
    if (!on || n >= 40) return;
    uint8_t bits[148];
    if (cellule_bits_attendus(fn, bsic, bits) < 0) return;
    uint8_t t116[116]; memcpy(t116, bits + 3, 58); memcpy(t116 + 58, bits + 87, 58);
    const struct { const uint8_t *t; int len; const char *nom; } tpl[2] = { { t116, 116, "116 data" }, { bits,
     ↪  148, "148 burst" } };
    n++;
    printf("  [scan] fn=%u p51=%u burst %d :", fn, fn % 51u, (int)((fn % 51u % 10 - 2) & 3));
    for (int k = 0; k < 2; k++) {
        int best[3] = {0,0,0}; unsigned bad[3] = {0,0,0};
        for (unsigned a = 0x60; a + tpl[k].len < C54X_DATA_SIZE; a++) {
            int m0 = 0, m1 = 0, m2 = 0;
            for (int i = 0; i < tpl[k].len; i++) {
                int16_t v = (int16_t)dsp->data[a + i];
                int neg = v < 0, one = (v == 1), zero = (v == 0);
                if ((v < 0) == (tpl[k].t[i] != 0)) m0++;
                if ((v > 0) == (tpl[k].t[i] != 0)) m1++;
                if ((one && tpl[k].t[i]) || (zero && !tpl[k].t[i])) m2++;
            }
            if (m⟨1⟩ > best[⟨2⟩]) { best[⟨3⟩] = m⟨4⟩; bad[⟨5⟩] = a; }  ×3
    ⟨⟩ = (0,0,0,0,0) (1,1,1,1,1) (2,2,2,2,2)
```

```
      }
      printf("  %s: neg=1 %d/%d @%04x  pos=1 %d/%d @%04x  hard %d/%d @%04x |", tpl[k].nom,
             best[0], tpl[k].len, bad[0], best[1], tpl[k].len, bad[1], best[2], tpl[k].len, bad[2]);
    }
    /* the ROM's burst buffer at 0x2be2 (found by this scan: 146/148 on good
     * bursts): mismatch map, one char per bit, '.' ok, 'x' wrong, '|' at the
     * data/TSC boundaries */
    { char map[160]; int k = 0, err = 0;
      for (int i = 0; i < 148; i++) {
          if (i == 3 || i == 61 || i == 87 || i == 145) map[k++] = '|';
          int16_t v = (int16_t)dsp->data[0x2be2 + i];
          int ok = ((v > 0) == (bits[i] != 0)); if (!ok) err++;
          map[k++] = ok ? '.' : 'x';
      }
      map[k] = 0;
      /* the same buffer against the two previous bursts: a demod deferred to
       * the next frame's ISR would leave burst N-1 here at the end of frame N */
      int e1 = -1, e2 = -1; uint8_t pb[148];
      if (cellule_bits_attendus(fn - 1, bsic, pb) == 0) { e1 = 0; for (int i = 0; i < 148; i++) if
       ↪  ((((int16_t)dsp->data[0x2be2 + i]) > 0) != (pb[i] != 0)) e1++; }
      if (cellule_bits_attendus(fn - 2, bsic, pb) == 0) { e2 = 0; for (int i = 0; i < 148; i++) if
       ↪  ((((int16_t)dsp->data[0x2be2 + i]) > 0) != (pb[i] != 0)) e2++; }
      /* regions of the data memory the demod wrote during phase B (beyond the
       * DARAM burst buffer and the API pages), with a few values each */
      { char z[900]; int k = 0; unsigned a = 0x60;
        while (a < C54X_DATA_SIZE && k < 800) {
            if (dsp->data[a] != g_data_avant_b[a]) {
                unsigned b = a; while (b < C54X_DATA_SIZE && b - a < 4096 && (dsp->data[b] != g_data_avant_b[b] ||
                 ↪  (b + 1 < C54X_DATA_SIZE && dsp->data[b+1] != g_data_avant_b[b+1]))) b++;
                k += snprintf(z + k, sizeof z - k, " %04x+%u", a, b - a); a = b;
            } else a++;
        }
        { const char *d = calypso_getenv("PONT_NB_HIST"); static unsigned nz;
          if (d && nz < 12) { char nom[256]; snprintf(nom, sizeof nom, "%s/zones_%u.txt", d, fn); FILE *f =
           ↪  fopen(nom, "w");
              if (f) { for (unsigned q = 0x60; q < C54X_DATA_SIZE; q++) if (dsp->data[q] != g_data_avant_b[q])
               ↪  fprintf(f, "%04x %04x %04x\n", q, g_data_avant_b[q], dsp->data[q]); fclose(f); nz++; } } }
        printf("  [zones] fn=%u flags(OVA=%d OVB=%d C=%d TC=%d OVM=%d FRCT=%d SXM=%d) ecrites:%s\n", fn,
               g_flags_entree & 1, !!(g_flags_entree & 2), !!(g_flags_entree & 4), !!(g_flags_entree & 8),
↪  !!(g_flags_entree & 16), !!(g_flags_entree & 32), !!(g_flags_entree & 64), z); }
      if (g_snap_ok) { int es = 0; for (int i = 0; i < 148; i++) if ((((int16_t)g_snap_2be2[i]) > 0) != (bits[i]
       ↪  != 0)) es++;
                       printf("  [scan] fn=%u 2be2 au pas 12800 (avant le decodeur) : erreurs=%d\n", fn, es);
↪  g_snap_ok = 0; }
      printf("  [scan] fn=%u marge=%d tsc=%d dec=%.2f phase=%.1f rif=%d 2be2 erreurs=%d (vs N-1: %d, N-2: %d)
↪  insnA=%u insnB=%u %s  v[0..3]=%d %d %d %d\n", fn, cellule_marge_nb, cellule_tsc_force, cellule_dec_nb,
↪  cellule_phase_nb, calypso_rif_level(), err, e1, e2, g_insn_a, g_insn_b, map,
             (int16_t)dsp->data[0x2be2], (int16_t)dsp->data[0x2be3], (int16_t)dsp->data[0x2be4],
              ↪  (int16_t)dsp->data[0x2be5]); }
    /* after burst 3: the decoder's vectors. Search the whole data memory for
     * the 456 coded bits in deinterleaved order and for the 184 information
     * bits, as signs of int16 words and as hard 0/1 words. */
    { uint8_t code[456], info[184];
      if (((fn % 51u) % 10u - 2) % 4 == 3 && cellule_bloc_attendu(fn, bsic, code, info) == 0) {
          const struct { const uint8_t *t; int len; const char *nom; } tp[2] = { { code, 456, "456 codes" }, {
           ↪  info, 184, "184 info" } };
          printf("  [bloc] fn=%u :", fn);
          for (int k = 0; k < 2; k++) {
              int best[3] = {0,0,0}; unsigned bad[3] = {0,0,0};
              for (unsigned a = 0x60; a + tp[k].len < C54X_DATA_SIZE; a++) {
                  int m0 = 0, m1 = 0, m2 = 0;
                  for (int i = 0; i < tp[k].len; i++) {
                      int16_t v = (int16_t)dsp->data[a + i];
                      if ((v < 0) == (tp[k].t[i] != 0)) m0++;
                      if ((v > 0) == (tp[k].t[i] != 0)) m1++;
                      if ((v == 1 && tp[k].t[i]) || (v == 0 && !tp[k].t[i])) m2++;
```

```
                }
                if (m[1] > best[(2)]) { best[(3)] = m[4]; bad[(5)] = a; }  ×3
    () = (0,0,0,0,0) (1,1,1,1,1) (2,2,2,2,2)
            }
            printf("  %s: neg=1 %d/%d @%04x  pos=1 %d/%d @%04x  hard %d/%d @%04x |", tp[k].nom,
                   best[0], tp[k].len, bad[0], best[1], tp[k].len, bad[1], best[2], tp[k].len, bad[2]);
        }
        /* also the coded bits packed 16 per word (MSB first) */
        { int bestp = 0; unsigned badp = 0;
          for (unsigned a = 0x60; a + 29 < C54X_DATA_SIZE; a++) {
              int m = 0;
              for (int i = 0; i < 456; i++) { int bit = (dsp->data[a + i / 16] >> (15 - i % 16)) & 1; if (bit ==
               ↪  code[i]) m++; }
              if (m > bestp) { bestp = m; badp = a; } }
          printf("  packed %d/456 @%04x", bestp, badp);
          int pb[4] = {0,0,0,0};
          for (int i = 0; i < 456; i++) { int bit = (dsp->data[badp + i / 16] >> (15 - i % 16)) & 1; if (bit ==
           ↪  code[i]) pb[i & 3]++; }
          printf(" par burst %d %d %d %d /114", pb[0], pb[1], pb[2], pb[3]); }
        printf("\n");
        /* the decoder input at 0x2a00 (the trellis loop reads pairs from AR1 = 0x2a00) */
        if (g_snap_2a00_ok) { const char *d = calypso_getenv("PONT_NB_HIST"); if (d) { char nom[256];
         ↪  snprintf(nom, sizeof nom, "%s/entree_%u.txt", d, fn); FILE *f = fopen(nom, "w"); if (f) { for (int i
         ↪  = 0; i < 456; i++) fprintf(f, "%d\n", (int16_t)g_snap_2a00[i]); fclose(f); } } }
        if (g_snap_2a00_ok) { int mp = 0, mn = 0, m1 = 0, nz = 0; g_snap_2a00_ok = 0;
          for (int i = 0; i < 456; i++) { int16_t v = (int16_t)g_snap_2a00[i]; if (v) nz++;
              if ((v > 0) == (code[i] != 0)) mp++;
              if ((v < 0) == (code[i] != 0)) mn++;
              if ((v == 1) == (code[i] != 0)) m1++; }
          printf("  [entree] fn=%u 0x2a00..+456 : nonzero=%d  pos=1 %d/456  neg=1 %d/456  one=1 %d/456 |
↪  premiers:", fn, nz, mp, mn, m1);
          for (int i = 0; i < 48; i++) printf(" %d", (int16_t)g_snap_2a00[i]);
          printf("\n  [entree] attendu :"); for (int i = 0; i < 48; i++) printf(" %d", code[i]); printf("\n");
          /* per-burst view: bits k with k%4==b */
          for (int b = 0; b < 4; b++) { int m = 0, n = 0; for (int i = b; i < 456; i += 4) { int16_t v =
           ↪  (int16_t)g_snap_2a00[i]; if (v) { n++; if ((v < 0) == (code[i] != 0)) m++; } }
              printf("  [entree] burst %d : neg=1 %d/%d\n", b, m, n); } }
        /* the 228 decoder output bits: best match over memory, hard words and LSB */
        { uint8_t u[228];
          if (cellule_u228_attendu(fn, bsic, u) == 0) {
              int best[2] = {0,0}; unsigned bad[2] = {0,0};
              for (unsigned a = 0x60; a + 228 < C54X_DATA_SIZE; a++) {
                  int m0 = 0, m1 = 0;
                  for (int i = 0; i < 228; i++) { uint16_t v = dsp->data[a + i];
                      if ((v == 1 && u[i]) || (v == 0 && !u[i])) m0++;
                      if ((v & 1) == u[i]) m1++; }
                if (m0 > best[0]) { best[0] = m0; bad[0] = a; } if (m1 > best[1]) { best[1] = m1; bad[1] = a; } }
              printf("  [u228] fn=%u hard %d/228 @%04x  lsb %d/228 @%04x | ", fn, best[0], bad[0], best[1],
↪  bad[1]);
              /* mismatch map at 0x2d66 (hard) */
              unsigned a = 0x2d66; int e = 0; char map[240]; int k = 0;
              for (int i = 0; i < 228; i++) { uint16_t v = dsp->data[a + i]; int ok = (v == u[i]); if (!ok) e++;
               ↪  if (i == 184 || i == 224) map[k++] = '|'; map[k++] = ok ? '.' : (v > 1 ? '?' : 'x'); }
              map[k] = 0; printf("2d66: %d faux %s\n", e, map);
              /* packed forms of the 228 bits anywhere in memory: 16 per word MSB
               * first, LSB first, and the same with each word bit-reversed */
             { const char *nm[4] = { "msb", "lsb", "msb-rev", "lsb-rev" }; printf("  [u228] fn=%u packed:", fn);
                for (int f = 0; f < 4; f++) { int best = 0; unsigned bad = 0;
                  for (unsigned a = 0x60; a + 15 < C54X_DATA_SIZE; a++) { int m = 0;
                      for (int i = 0; i < 228; i++) { uint16_t w = dsp->data[a + i / 16]; int bi = i % 16;
                         int bit = (f == 0) ? (w >> (15 - bi)) & 1 : (f == 1) ? (w >> bi) & 1 : (f == 2) ? (w >>
                          ↪  bi) & 1 : (w >> (15 - bi)) & 1;
                          if (bit == u[i]) m++; }
                      if (m > best) { best = m; bad = a; } }
                  printf(" %s %d/228 @%04x", nm[f], best, bad); }
```

```
                printf(" | 2c3c:"); for (int i = 0; i < 16; i++) printf(" %04x", dsp->data[0x2c3c + i]);
↪  printf("\n"); } } }
        } }
    /* where did the delivered samples land in DARAM (AAD)? offset in words at
     * which the ROM's buffer equals our frame, and how many words match */
    { uint16_t aad = calypso_rhea_dma_get_daram(); int best = 0, boff = 0;
      for (int off = -8; off <= 8; off++) {
          int m = 0;
          for (int i = 0; i < g_dernier_n_iq; i++) {
              int a = (int)aad + off + i; if (a < 0 || a >= C54X_DATA_SIZE) continue;
              if ((int16_t)dsp->data[a] == g_dernier_iq[i]) m++;
          }
          if (m > best) { best = m; boff = off; }
      }
      int p0 = -1; for (int i = 0; i < 40; i++) if ((int16_t)dsp->data[aad + i] != 0) { p0 = i; break; }
      { /* runs of words that differ from what was delivered, at offset 0 */
        char runs[256]; int k = 0, i = 0;
        while (i < g_dernier_n_iq && k < 200) {
            if ((int16_t)dsp->data[aad + i] != g_dernier_iq[i]) {
                int j = i; while (j < g_dernier_n_iq && (int16_t)dsp->data[aad + j] != g_dernier_iq[j]) j++;
              k += snprintf(runs + k, sizeof runs - k, " %d+%d(%d)", i, j - i, (int16_t)dsp->data[aad + i]); i = j;
            } else i++;
        }
        runs[k] = 0;
        int d_dma = 0; for (int q = 0; q < g_dernier_n_iq && q < 384; q++) if ((int16_t)g_daram_apres_dma[q] !=
         ↪  g_dernier_iq[q]) d_dma++;
        printf("  [daram] fn=%u differences apres la trame (mot+longueur(valeur)):%s | juste apres le DMA, avant
↪  la ROM : %d mots differents (aad=%04x)\n", fn, runs, d_dma, g_daram_aad); }
      printf("  [daram] fn=%u aad=%04x livres=%d mots, identiques=%d a l'offset %d, premier mot non nul a +%d,
↪  mots 0..7: %d %d %d %d %d %d %d %d\n",
             fn, aad, g_dernier_n_iq, best, boff, p0,
             (int16_t)dsp->data[aad], (int16_t)dsp->data[aad+1], (int16_t)dsp->data[aad+2],
              ↪  (int16_t)dsp->data[aad+3],
             (int16_t)dsp->data[aad+4], (int16_t)dsp->data[aad+5], (int16_t)dsp->data[aad+6],
              ↪  (int16_t)dsp->data[aad+7]); }
    printf("\n");
}

static void servir(int fd, C54xState *dsp, uint16_t *api_ram, long insns, bool verbeux,
                   const char *iq_mode, int amp)
{
    bool injecter = iq_mode && *iq_mode && strcmp(iq_mode, "none") != 0;
    unsigned long injectes = 0;
    const uint16_t *a_sync = &api_ram[(API_NDB + NDB_A_SYNC_DEMOD) / 2];
    bool init_done = false;
    if (g_verif_sonde < 0) g_verif_sonde = calypso_getenv("CALYPSO_BSP_VERIF") ? 1 : 0;
    unsigned long trames = 0, irqs = 0, resets = 0;
    uint64_t insns_total = 0;
    const uint16_t *d_fb_det = &api_ram[(API_NDB + NDB_D_FB_DET) / 2];
    /* [2026-09-19] a_sch was read from R page 0 ONLY, hardcoded, while the DSP
     * writes its result to the page d_dsp_page designates, alternating. Stale
     * page-0 content read as a result is exactly what produces SB decodes that
     * are wrong yet REPEATABLE (measured: BSIC 44 six times, 22 five times out
     * of 17, never the 32 the capture carries). The PONT_CAN_SB hack in this
     * same file already writes BOTH pages, so the page was known to matter.
     * Both are kept here and the live one is picked per frame. */
    const uint16_t *a_sch_pg[2] = {
        &api_ram[(API_R_PAGE(0) + RP_A_SCH) / 2],
        &api_ram[(API_R_PAGE(1) + RP_A_SCH) / 2] };
    const uint16_t *a_sch0   = a_sch_pg[0];

    CalypsoPontMsg m;
    if (recv(fd, &m, sizeof(m), 0) != (ssize_t)sizeof(m) || m.type != PONT_HELLO ||
        m.a != CALYPSO_API_WORDS || m.b != CALYPSO_PONT_MAGIC) {
        fprintf(stderr, "pont : poignee de main invalide (type=%u a=%u)\n", m.type, m.a);
        return;
```

```
    }
    envoyer(fd, PONT_HELLO_OK, CALYPSO_API_WORDS, CALYPSO_PONT_MAGIC);
    printf("pont : ARM connecte, API RAM %u mots, %ld insn/trame\n", CALYPSO_API_WORDS, insns);
    fflush(stdout);

    /* PONT_RESET arriving while waiting for the PONT_GO (two-phase frame): the TI firmware
     * restarts the DSP mid-way (fn ~290) and then waits for LEAD_READY; the reset is kept and
     * applied at the end of the frame. */
    unsigned reset_differe = 0;
    uint32_t reset_differe_dl = 0;
    while (!g_stop) {
        if (g_pause) pause_cooperative(dsp);
        if (reset_differe) {
            reset_differe--;
            memset(&m, 0, sizeof m);
            m.type = PONT_RESET;
            m.a = reset_differe_dl;
            printf("pont : reset garde pendant l'attente du GO, applique maintenant\n");
        } else {
            struct pollfd pfd = { .fd = fd, .events = POLLIN };
            if (poll(&pfd, 1, 200) <= 0) {
                continue;
            }
            ssize_t n = recv(fd, &m, sizeof(m), 0);
            if (n != (ssize_t)sizeof(m)) {
                printf("pont : ARM deconnecte (%zd)\n", n);
                break;
            }
        }
        switch (m.type) {
        case PONT_RESET: {
            /* THE TI bootCode. On the Calypso C035 the API is also the DSP's PROGRAM space
             * 0xE000-0xFFFF (leadapi.h BASE_API_LEAD 0xE000). The TI firmware (Magnetite,
             * LA_InitialLeadBoot, leadboot.c) places there, before releasing the reset, a loader
             * (0xf800), an init (0xfe00) and the RESET VECTOR (0xff80: F073 FE00 = b 0xfe00): every
             * reset goes through this loader, which reports LEAD_READY. Here 0xE000-0xFFFF is the
             * fixed "PDROM", and c54x_reset copies a ROM image into the API window of the data
             * memory, which would restart the ROM hot (0xb3e7) at the 2nd reset, without LEAD_READY,
             * leaving the ARM in dsp_power_on+0x34 (NO_DWNLD). Signature present: the API is kept as
             * the ARM wrote it (a DSP reset does not touch the shared RAM) and the DSP restarts in
             * the loader, as with the bootCode. osmocom firmware (no signature): unchanged. */
            static uint16_t api_avant[0x2000];
            const bool boot_ti = api_ram[0xff80 - 0xe000] == 0xF073 && api_ram[0xff81 - 0xe000] == 0xFE00;
            if (boot_ti) memcpy(api_avant, api_ram, sizeof api_avant);
            c54x_reset(dsp);
            if (boot_ti) {
                /* Executing the bootCode itself (program window 0xE000) crashes the DSP (SP-CORRUPT
                 * at PC 0: C035 memory modes not emulated). Its FUNCTION is kept: DSP placed in the
                 * ROM's loader (0xb410: st #1,*(0xfff) then wait for 2 and jump), with the registers
                 * the ROM sets there (0xb3ff-0xb403). API kept as the ARM wrote it. */
                memcpy(api_ram, api_avant, sizeof api_avant);
                for (int i = 0; i < 0x2000; i++) dsp->data[0x0800 + i] = api_avant[i];
                dsp->st1 = 0x2900; dsp->st0 = 0x1800; dsp->sp = 0x5AC8;
                dsp->pc = 0xb410;
                printf("pont : bootCode TI dans l'API (0xff80 = b 0xfe00) : API gardee, DSP dans le chargeur ROM
↪   0xb410\n");
            }
            dsp->running = true;
            init_done = false;
            api_ram[(API_NDB + NDB_D_DSP_PAGE) / 2] = 0;
            resets++;
            printf("pont : RESET #%lu (DL_STATUS=0x%04x) fn=%u pc=0x%04x\n",
                   resets, m.a, g_c54x_exe_fn, dsp->pc);
            break;
        }
        case PONT_TICK: {
```

```
g_c54x_exe_fn = m.a;
g_tick_irq_trame = (m.b & CALYPSO_PONT_TICK_IRQ_TRAME) != 0;
apm_trace_tick(api_ram, m.a, m.b);
bool deux_phases = (m.b & CALYPSO_PONT_TICK_DEUX_PHASES) != 0;
bool done_envoye = false;
double t_tick = chrono_ms(), t_done_a = t_tick, t_go = t_tick, t_done = 0;
m.b &= 1u;
enreg_tick(dsp, api_ram, m.a, g_tick_irq_trame, deux_phases, insns);
enreg_api_diff(api_ram, m.a, 0);
tx_sonde_avant(dsp, api_ram);
calypso_bsp_set_tpu_offset((int)m.c);   /* firmware RX window */
/* AFC relay, closing the loop. The ARM writes d_afc (word 15 of the W
 * page) into the shared API RAM; on silicon the DSP serialises it to
 * the TWL3025 over the TSP. Without this relay the sample rotation
 * stays frozen and the frequency error never converges below the SB
 * threshold. m.b carries d_dsp_page, bit 0 selecting the W page. */
{ unsigned wp = m.b & 1u;
  int16_t dac = (int16_t)api_ram[(wp ? 0x14u : 0x00u) + 15u];
  static int16_t prev; static int first = 1;
  /* [2026-09-19] Measurement before any fix: the AFC DAC never settles,
   * every correction is followed by a write of exactly -700
   * (afc_initial_dac_value). The suspicion is the dual-page write the
   * set_afc_dac filter already documents — page A holding the inherited
   * init while page B carries the correction — with the filter guarding
   * only against 0, not against -700. Print BOTH pages so the claim can
   * be checked instead of assumed. */
  { static unsigned na;
    int16_t d0 = (int16_t)api_ram[0x00u + 15u], d1 = (int16_t)api_ram[0x14u + 15u];
    static int16_t p0 = 0x7fff, p1 = 0x7fff;
    if ((d0 != p0 || d1 != p1) && na++ < 40)
        printf("  [afc] fn=%u w_page=%u relaye=%d | page0=%d page1=%d\n",
               m.a, wp, dac, d0, d1);
    p0 = d0; p1 = d1; }
  if (first || dac != prev) { calypso_twl3025_set_afc_dac(dac); prev = dac; first = 0; } }
{ static unsigned _to=0; if (calypso_getenv("PONT_TPU_DEBUG") && (_to<5 || _to%2000==0)) printf("
↪  [tpu] fn=%u tpu_offset=%u\n", m.a, m.c); _to++; }
trace_armer();
if (g_trace_reste > 0 && g_trace_f && !g_trace_ouverte && !g_trace_pc_hi) {
    /* wait for the first FB task posted by the ARM (W page 0 or 1) */
    static int md_cible = -1;
   if (md_cible < 0) { const char *e = calypso_getenv("PONT_TRACE_MD"); md_cible = (e && *e) ? atoi(e)
    ↪  : 5; }
    if (api_ram[(API_W_PAGE(0) + WP_D_TASK_MD) / 2] == md_cible ||
        api_ram[(API_W_PAGE(1) + WP_D_TASK_MD) / 2] == md_cible) {
        g_trace_ouverte = true;
        trace_cellules(dsp, true);
        fprintf(g_trace_f, "# d_task_md=%d vu a fn=%u page=%u ; IMR=%04x IFR=%04x pc=%04x\n",
                md_cible, m.a, m.b, dsp->imr, dsp->ifr, dsp->pc & 0xffff);
        printf("pont : trace FB ouverte a fn=%u\n", m.a);
    }
}
/* PONT_RX_APRES (default 1): deliver the burst AFTER the frame
 * interrupt, as on silicon, where the TPU programs the RX window
 * within the frame, the DSP arms DMA2 in the frame ISR, and the
 * samples land afterwards. Injecting before the interrupt instead
 * leaves DMA2 with ENABLE=0 when the RIF issues its RX request, and
 * the SB job then decodes the previous tick's window. */
/* PONT_RX_MODE : milieu (default, see jouer_trame) | apres | avant.
 * PONT_RX_APRES=0/1 is still honoured as avant/apres. */
static int rx_mode = -1;   /* 0 = milieu, 1 = apres, 2 = avant */
if (rx_mode < 0) { const char *e = calypso_getenv("PONT_RX_MODE"); const char *a =
↪  calypso_getenv("PONT_RX_APRES");
                   rx_mode = (e && !strcmp(e, "apres")) ? 1 : (e && !strcmp(e, "avant")) ? 2
                           : (a && *a == '1') ? 1 : (a && *a == '0') ? 2 : 0; }
bool rx_apres = (rx_mode == 1);
if (rx_mode == 2 && injecter && init_done) injecter_burst(dsp, iq_mode, amp, m.a, &injectes);
```

```
/* Real chain: drain UDP socket 6702 (bursts from the bridge/BTS) and
 * hand them to the DSP. This is the only source when synthetic
 * injection is off, and a no-op when the socket is empty. */
if (init_done && rx_mode != 0) calypso_bsp_service(m.a);
g_inj.actif = (rx_mode == 0 && init_done);
g_inj.iq_mode = (rx_mode == 0 && injecter) ? iq_mode : NULL;
g_inj.amp = amp; g_inj.fn = m.a; g_inj.injectes = &injectes; g_inj.udp = (rx_mode == 0 && init_done);
uint32_t ninsn = 0;
bool init_avant = init_done;
uint32_t drapeaux = jouer_trame(dsp, insns, &init_done, &ninsn, deux_phases ? 1 : 0);
if (deux_phases) {
    /* [2026-09-21] Phase A done: the ROM's frame ISR has written the
     * R page (previous burst) and armed the window. Tell QEMU, which
     * raises the ARM frame IRQ, waits for the end of l1_sync() and
     * sends PONT_GO; only then is the burst of this frame delivered.
     * That is the silicon order, and what keeps "BURST ID n!=m" and
     * "EMPTY" (prim_rx_nb.c) away. */
    sonde_pages("A", m.a, dsp, api_ram);
    enreg_api_prendre(api_ram);
    ombre_descendante(api_ram, m.a);
    /* TRX mode: the exact bits of the uplink burst played at the previous frame, in
     * the result page the ARM reads after this DONE (calypso_bsp.c), and the downlink
     * bursts of all the TX slots of this page, published in the ring */
    {
        int w_page = (api_ram[(API_NDB + NDB_D_DSP_PAGE) / 2] & B_GSM_PAGE) ? 1 : 0;
        if (trx_page_mode()) tx_page_publier(dsp, m.a, w_page);
        calypso_bsp_trx_ul_prothese(dsp->data, m.a, w_page);
    }
    apm_jamais_nul(api_ram);
    envoyer(fd, PONT_DONE, drapeaux & ~PONT_DONE_API_IRQ, ninsn);
    t_done_a = chrono_ms();
    bool go = false;
    while (!g_stop && !go) {
        struct pollfd pg = { .fd = fd, .events = POLLIN };
       if (poll(&pg, 1, 2000) <= 0) { static unsigned nt; if (nt++ < 3) printf("pont : PONT_GO attendu
        ↪  (fn=%u)\n", m.a); continue; }
        CalypsoPontMsg g;
        ssize_t ng = recv(fd, &g, sizeof(g), 0);
        if (ng != (ssize_t)sizeof(g)) { printf("pont : ARM deconnecte en attente de GO (%zd)\n", ng);
        ↪  g_stop = 1; break; }
        if (g.type == PONT_GO) { go = true; t_go = chrono_ms(); }
        else if (g.type == PONT_RESET) { reset_differe = 1; reset_differe_dl = g.a; }   /* see
        ↪  reset_differe */
        else { static unsigned nx; if (nx++ < 3) printf("pont : message %u recu en attente de GO,
        ↪  ignore\n", g.type); }
    }
    if (!go) break;
    sonde_pages("G", m.a, dsp, api_ram);
    enreg_api_diff(api_ram, m.a, 1);
    if (calypso_getenv("PONT_NB_DEBUG") && ((m.a % 51u) % 10u - 2) % 4 == 3 && m.a % 51u <= 5 &&
        (api_ram[API_R_PAGE(0) / 2] == 24 || api_ram[API_R_PAGE(1) / 2] == 24)) {
        static unsigned nq;
        if (nq++ < 8) {
            printf("  [garde] fn=%u avant le burst 3, bursts precedents en memoire :", m.a);
            for (int b = 1; b <= 3; b++) {
                uint8_t bits[148]; if (cellule_bits_attendus(m.a - b, 42, bits) < 0) continue;
                uint8_t t116[116]; memcpy(t116, bits + 3, 58); memcpy(t116 + 58, bits + 87, 58);
                int best[3] = {0,0,0}; unsigned bad[3] = {0,0,0};
                for (unsigned a = 0x60; a + 116 < C54X_DATA_SIZE; a++) {
                    int m0 = 0, m1 = 0, m2 = 0;
                    for (int i = 0; i < 116; i++) { int16_t v = (int16_t)dsp->data[a + i];
                        if ((v < 0) == (t116[i] != 0)) m0++;
                        if ((v > 0) == (t116[i] != 0)) m1++;
                        if ((v == 1 && t116[i]) || (v == 0 && !t116[i])) m2++; }
                    if (m0 > best[0]) { best[0] = m0; bad[0] = a; } if (m1 > best[1]) { best[1] = m1;
                    ↪  bad[1] = a; } if (m2 > best[2]) { best[2] = m2; bad[2] = a; }
```

```
                        }
                        /* packed 16 per word too */
                        int bestp = 0; unsigned badp = 0;
                        for (unsigned a = 0x60; a + 8 < C54X_DATA_SIZE; a++) { int mm = 0;
                          for (int i = 0; i < 116; i++) { int bit = (dsp->data[a + i / 16] >> (15 - i % 16)) &
                            ↪  1; if (bit == t116[i]) mm++; }
                           if (mm > bestp) { bestp = mm; badp = a; } }
                       printf(" b%d(fn %u): neg %d@%04x pos %d@%04x hard %d@%04x packed %d@%04x |", 3 - b, m.a
↪  - b, best[0], bad[0], best[1], bad[1], best[2], bad[2], bestp, badp);
                    }
                    printf("\n");
                }
            }
            uint32_t n2 = 0;
            g_insn_a = ninsn;
            if (g_done_tot < 0) { const char *e = calypso_getenv("PONT_DONE_TOT"); g_done_tot = !(e && *e ==
             ↪  '0'); }
            g_reste_b = 0;
            drapeaux = jouer_trame(dsp, insns, &init_done, &n2, 2);
            ninsn += n2; g_insn_b = n2;
            if (g_done_tot) {
                /* DONE right away: QEMU resumes, the DSP finishes the frame here. */
                ombre_descendante(api_ram, m.a);
                envoyer(fd, PONT_DONE, drapeaux & ~PONT_DONE_API_IRQ, ninsn);
                done_envoye = true;
                t_done = chrono_ms();
                if (g_reste_b > 0 && !dsp->idle && dsp->running) {
                    uint32_t av = dsp->insn_count;
                    c54x_run_profile(dsp, g_reste_b);
                    ninsn += dsp->insn_count - av;
                }
                g_reste_b = 0;
            }
        }
        g_inj.actif = false;
        sonde_pages("B", m.a, dsp, api_ram);
        if (g_inj.dernier_type == 'B' && (api_ram[API_R_PAGE(0) / 2] == 24 || api_ram[API_R_PAGE(1) / 2] ==
         ↪  24)) sonde_bits(m.a, dsp, 42);
        /* Reference probe (CALYPSO_BSP_VERIF=1): compare DARAM against the
         * burst the BSP was handed, AFTER the DSP has run — the samples
         * only reach DARAM through the DSP's own DMA draining the RIF, so
         * there is nothing to compare before jouer_trame. */
        if (g_verif_sonde) {
            uint32_t vfn = 0; uint16_t vad = 0; int vn = 0, vage = 0;
            int ident = calypso_bsp_verif_compare(&vfn, &vad, &vn, &vage);
            if (ident >= 0 && vn > 0) {
                static unsigned nv;
                if (nv++ < 4000) {
                    uint16_t vpg = calypso_bsp_verif_last_page();
                    printf("  [verif] fn=%u p51=%u age=%d page=0x%04x w_page=%d : "
                           "%d/%d en 0x%04x %s\n",
                           vfn, vfn % 51u, vage, vpg, (int)(vpg & 1u),
                           ident, vn, vad, ident == vn ? "VALIDE" : "partiel");
                }
            }
        }
        if (rx_apres && injecter && init_done && dsp->running) {
            injecter_burst(dsp, iq_mode, amp, m.a, &injectes);
            /* DMA completion inside the same frame: wake on the held INT10n
             * line, then let the DSP run the completion ISR and background
             * work through to IDLE. */
            uint32_t avant = dsp->insn_count;
            if (dsp->idle && calypso_rhea_dma_irq_level() && (dsp->imr & (1u << 14)) &&
                !(dsp->ifr & (1u << 14)))
                c54x_interrupt_ex(dsp, 30, 14);
            if (dsp->idle && (dsp->ifr & dsp->imr) && !(dsp->st1 & 0x800)) dsp->idle = false;
```

```
            if (!dsp->idle) c54x_run_profile(dsp, (int)insns);
            ninsn += dsp->insn_count - avant;
        }
        /* [2026-09-20] STREAM PUMP (same as rejouer.c). DMA2 now fills its
         * double buffer with full pages and interrupts once per pair; the
         * ROM's ISR consumes both halves and the DSP idles. Then the next
         * pair is handed over, until the receiver holds less than a pair. A
         * 156.25-symbol frame is 3.25 pages, so this runs 1 or 2 times. */
        for (int k = 0; k < 40 && dsp->running; k++) {   /* 13 page pairs per 1250-symbol frame */
            /* an SCH held back because the DMA was not armed at the deposit
             * is delivered again here, as soon as the ROM (phase B) has armed it. */
            int relivre = calypso_bsp_sb_retenter();
            /* TI dedicated mode: measurement window served / held NB delivered again (calypso_bsp.c);
             * the DSP is then left running (end of DMA -> ISR -> next window) within the same tick */
            relivre |= calypso_bsp_nb_retenter();
            if (!calypso_rhea_dma_pump(dsp) && !relivre) break;
            if (dsp->idle && (dsp->ifr & dsp->imr) && !(dsp->st1 & 0x800)) dsp->idle = false;
            uint32_t av2 = dsp->insn_count;
            if (!dsp->idle) c54x_run_profile(dsp, (int)insns / 4);
            ninsn += dsp->insn_count - av2;
            drapeaux = (drapeaux & ~PONT_DONE_IDLE) | (dsp->idle ? PONT_DONE_IDLE : 0);
        }
        /* VOICE PORT (VSP, calypso_vsp.c, CALYPSO_VSP=1): the 36 or 37 PCM samples
         * of the frame, each through the ROM's ISR 0xc14a (IT 20). */
        if (init_done && dsp->running)
            ninsn += calypso_vsp_trame(dsp, c54x_run_profile, m.a);
        drapeaux = (drapeaux & ~PONT_DONE_IDLE) | (dsp->idle ? PONT_DONE_IDLE : 0);
        /* [2026-09-20] NB probe (PONT_NB_DEBUG=1): what the ROM leaves in the
         * R pages for a normal-burst task (ALLC/BCCH), per burst: d_task_d,
         * d_burst_d, a_serv_demod (TOA, PM, ANGLE, SNR) and, on burst 3,
         * the a_cd header of the NDB (Fire/CRC word, bit errors) plus the
         * first decoded octets. Read alongside mobile's "Dropping frame
         * with N bit errors". */
        {
            static int nbdbg = -1; static unsigned nbn;
            if (nbdbg < 0) nbdbg = calypso_getenv("PONT_NB_DEBUG") ? 1 : 0;
            /* Ceiling of 20000 lines: a smaller one would be used up by
             * camping before any dedicated connection, leaving the probe
             * silent when it is needed. */
            if (nbdbg && nbn < 20000) {
                static uint16_t prev[2][4];
                for (int pg = 0; pg < 2; pg++) {
                    const uint16_t *r = &api_ram[API_R_PAGE(pg) / 2];
                    uint16_t cur[4] = { r[RP_D_TASK_D/2], r[RP_D_BURST_D/2], r[RP_A_SERV_DEMOD/2 + D_TOA],
                     ↪  r[RP_A_SERV_DEMOD/2 + D_PM] };
                    if (r[RP_D_TASK_D/2] && memcmp(cur, prev[pg], sizeof cur)) {
                        const uint16_t *w0 = &api_ram[API_W_PAGE(0) / 2], *w1 = &api_ram[API_W_PAGE(1) / 2];
                       printf("  [nb] fn=%u p51=%u R%d task_d=%u burst_d=%u TOA=%d PM=%d ANGLE=%d SNR=%u | W0
↪  task_d=%u burst=%u W1 task_d=%u burst=%u | livre=%c n_iq=%d one_shot=%d len=%u mots",
                                m.a, m.a % 51u, pg, r[RP_D_TASK_D/2], r[RP_D_BURST_D/2],
                                (int16_t)r[RP_A_SERV_DEMOD/2 + D_TOA], (int16_t)r[RP_A_SERV_DEMOD/2 + D_PM],
                                (int16_t)r[RP_A_SERV_DEMOD/2 + D_ANGLE], r[RP_A_SERV_DEMOD/2 + D_SNR],
                                w0[0], w0[1], w1[0], w1[1], g_inj.dernier_type, g_inj.dernier_n_iq,
                                calypso_rhea_dma_one_shot(), calypso_rhea_dma_get_len_words());
                        if (r[RP_D_BURST_D/2] == 3) {
                            const uint16_t *cd = &api_ram[(API_NDB + NDB_A_CD) / 2];
                            printf(" | a_cd=%04x %04x %04x :", cd[0], cd[1], cd[2]);
                            for (int k = 3; k < 15; k++) printf(" %04x", cd[k]);
                        }
                        printf("\n"); nbn++;
                    }
                    memcpy(prev[pg], cur, sizeof cur);
                }
            }
        }
        /* Canned results: scaffolding to prove the pipeline through to the
```

```
 * LU, not an end state. PONT_CAN_TOA=23 forces the reported TOA
 * (a_sync_demod[D_TOA]) to the on-time value the firmware expects
 * (prim_fbsb.c subtracts 23). Drop it once the correlator's native
 * TOA is correct. -1, the default, disables canning. */
{
    static int can_toa = -2;
    if (can_toa == -2) { const char *e = calypso_getenv("PONT_CAN_TOA"); can_toa = (e && *e) ? atoi(e)
     ↪  : -1; }
    if (can_toa >= 0 && *d_fb_det)
        api_ram[(API_NDB + NDB_A_SYNC_DEMOD) / 2 + D_TOA] = (uint16_t)can_toa;
    /* SB: the firmware reads a_serv_demod[D_TOA] from the read page
     * and expects about 4. PONT_CAN_SB_TOA=4 pins it on both R
     * pages, which frames the SCH correctly. */
    /* PONT_CAN_NB_TOA=<v> (Magnetite: 3): the TI L1 does not converge on the TOA of the
     * normal bursts (l1ctl_toa_update: ISH = (TOA+1)*4 - 16 qbits, the bench does not
     * move the burst when tpu_offset moves); it is pinned on the R pages carrying an NB
     * task (d_task_d != 0). The TOA of the SB (a_sch, 23) stays native: sync depends on it. */
    static int can_nb = -2;
    if (can_nb == -2) { const char *e = calypso_getenv("PONT_CAN_NB_TOA"); can_nb = (e && *e) ? atoi(e)
     ↪  : -1;
                        if (can_nb >= 0) printf("  [pont] PONT_CAN_NB_TOA=%d : TOA des NB fixe\n",
                         ↪  can_nb); }
    /* init_done: during the boot these API RAM words carry the DSP code the ARM
     * downloads; writing there would break the download. */
    if (can_nb >= 0 && init_done) {
        for (int pg = 0; pg < 2; pg++) {
            uint16_t *r = &api_ram[API_R_PAGE(pg) / 2];
            if (r[RP_D_TASK_D / 2] && r[RP_D_TASK_D / 2] <= 40) r[RP_A_SERV_DEMOD / 2 + D_TOA] =
             ↪  (uint16_t)can_nb;
        }
    }
    static int can_sb = -2;
    if (can_sb == -2) { const char *e = calypso_getenv("PONT_CAN_SB_TOA"); can_sb = (e && *e) ? atoi(e)
     ↪  : -1; }
    if (can_sb >= 0) {
        api_ram[(API_R_PAGE(0) + RP_A_SERV_DEMOD) / 2 + D_TOA] = (uint16_t)can_sb;
        api_ram[(API_R_PAGE(1) + RP_A_SERV_DEMOD) / 2 + D_TOA] = (uint16_t)can_sb;
    }
    /* Full SB canning (PONT_CAN_SB=<bsic>): write the SB result
     * (a_sch) with CRC OK, the BSIC and the real frame number, the
     * way qemu-calypso's shunt_encode_sb does. The frame number comes
     * from the last delivered burst (real BTS, via
     * calypso_bsp_get_last_fn) or from m.a. Applies only when the ARM
     * asks for SB (d_task_md=6 on a W page) and on an SCH frame,
     * fn%51 in {1,11,21,31,41}. */
    static int can_sb_full = -2, can_sb_bsic = 7;
    if (can_sb_full == -2) { const char *e = calypso_getenv("PONT_CAN_SB"); can_sb_full = (e && *e) ? 1
     ↪  : 0; if (e && *e) can_sb_bsic = atoi(e) & 0x3f; }
    if (can_sb_full) {
        int md0 = api_ram[4] & 0xff, md1 = api_ram[0x18] & 0xff;   /* d_task_md on W pages 0 and 1 */
        if (md0 == 6 || md1 == 6) {
            /* [2026-09-19] The frame MUST be the tick's own (m.a), not
             * calypso_bsp_get_last_fn(): measured, the two diverged badly
             * (the canned SB announced fn=5662 while the firmware sat at
             * 11261), and Synchronize_TDMA then locked the ARM onto a frame
             * number unrelated to what the injector delivers — every burst
             * after the sync misaligned. A canned SB has to carry the clock
             * the cell is actually generated from, or it proves nothing. */
            uint32_t bfn = m.a;
            uint32_t p51 = bfn % 51;
          if (!(p51 % 10 == 1 && p51 <= 41)) bfn += (51 + 1 - (int)p51) % 51;  /* snap to an SCH frame */
            uint32_t sb = pont_encode_sb((uint8_t)can_sb_bsic, bfn / 1326, bfn % 26, bfn % 51);
            for (int pg = 0; pg < 2; pg++) {
                uint16_t *a = &api_ram[(API_R_PAGE(pg) + RP_A_SCH) / 2];
                a[0] = 0x8000;                /* B_SCH_CRC clear = CRC OK */
                a[1] = 0x2034;                /* echo value, as the DSP writes it */
```

```
                        a[3] = (uint16_t)(sb & 0xffff);
                        a[4] = (uint16_t)(sb >> 16);
                    }
                    static int nlog = 0;
                    if (nlog < 8) { printf("  [can-sb] fn=%u -> sb=0x%08x BSIC=%d (T1=%u T2=%u T3=%u)
                     ↪  hacks=%s\n",
                                            bfn, sb, can_sb_bsic, bfn/1326, bfn%26, bfn%51, hacks_actifs());
↪  nlog++; }
                }
            }
        }
        if (*d_fb_det) calypso_bsp_toa_feedback((int)(int16_t)a_sync[0]);  /* native TOA tracking loop */
        /* UPLINK. Under CALYPSO_DSP_EXTERN=1 the gr-gsm layer 1 is disabled,
         * so the hooks that published the uplink (calypso_l1_do_rach_written /
         * _page_written) are no-ops and the mobile's RACH would never leave
         * the API RAM. The W page the ARM has just filled (m.b = d_dsp_page)
         * is polled here and the same /dev/shm side-bands as in the grgsm
         * setup are fed. */
        tx_inject_sortie(dsp, m.a);
        tx_ombre_compare(dsp, m.a);
        tx_rom_publier(dsp, m.a);
        tx_tsp_publier(dsp, m.a);
        tx_sonde_apres(dsp, api_ram, m.a);
        dsp_global_tx = dsp;
        { int blud_avant = (api_ram[(API_NDB + NDB_A_CU) / 2] & B_BLUD) != 0;
          montant_scruter(api_ram, m.a, m.b & 1u);
          tx_inject_apres(api_ram, m.a, blud_avant);
          tx_ombre_pose(api_ram, blud_avant && g_inj_n <= 0);
          tx_rom_pose(api_ram, blud_avant); }
        trames++;
        insns_total += ninsn;
        if (drapeaux & PONT_DONE_API_IRQ) irqs++;
        enreg_api_prendre(api_ram);
        if (!done_envoye) { apm_jamais_nul(api_ram); ombre_descendante(api_ram, m.a); envoyer(fd, PONT_DONE,
         ↪  drapeaux, ninsn); }
        { double t_fin = chrono_ms(); chrono_trame(t_tick, t_done_a, t_go, t_done > 0 ? t_done : t_fin, t_fin,
         ↪  m.a); }
        if (!init_avant && init_done) {
            printf("pont : DSP boote (premier IDLE) fn=%u insn=%u\n", m.a, dsp->insn_count);
        }
        /* frame at which the ARM posts the FB/SB task (W0 word 4, W1 word 0x18) */
        { static int prev5 = -1, prev6 = -1, ncmd = 0;
          int md0 = api_ram[4] & 0xff, md1 = api_ram[0x18] & 0xff;
          int has5 = (md0 == 5 || md1 == 5), has6 = (md0 == 6 || md1 == 6);
          if (has5 && prev5 != 1 && ncmd < 24) { printf("  [cmd] fn=%u tache FB postee par l'ARM\n", m.a);
           ↪  ncmd++; }
         { static int ncmd6; if (has6 && prev6 != 1 && ncmd6 < 60) { printf("  [cmd] fn=%u tache SB postee par
           ↪  l'ARM (W0 md=%d W1 md=%d)\n", m.a, md0, md1); ncmd6++; } }
          if (has6 && prev6 != 1 && ncmd < 24) { printf("  [cmd] fn=%u tache SB postee par l'ARM\n", m.a);
           ↪  ncmd++; }
          if (has6 && prev6 != 1) calypso_bsp_sb_trace(8);   /* and around every SB task */
          prev5 = has5; prev6 = has6; }
        { static int fb_prev = 0; if (*d_fb_det && !fb_prev) { printf("  [jalon] fn=%u d_fb_det=1
         ↪  hacks=%s\n", m.a, hacks_actifs());
                                                               calypso_bsp_sb_trace(60); }   /* every SCH traced
↪  up to the SB attempts */
          fb_prev = *d_fb_det != 0; }
        /* [2026-09-19] ONE LINE PER SB ATTEMPT. Everything upstream of the SB
         * task is now measured correct — right frame, right window, right
         * sample offset — yet the CRC passes in 0.4% of attempts. A decode
         * that were simply broken would give 0%, so something DISCRIMINATES
         * the rare successes. This logs, for every frame the ARM has an SB
         * task posted, the TOA the FB left behind (a_sync[0], the value the
         * firmware positions the window from) next to the CRC outcome, so
         * the two populations can be compared directly. */
        /* The a_sync cells are already cleared by the time the SB task is
```

```
 * posted (measured: toa=pm=ang=0 on every attempt), so the FB result
 * has to be LATCHED when d_fb_det rises and carried to the attempt. */
static int lat_toa, lat_pm, lat_ang; static uint32_t lat_fn;
{ static int fbl = 0;
  if (*d_fb_det && !fbl) {
      lat_toa = (int)(int16_t)a_sync[0]; lat_pm = a_sync[1];
      lat_ang = (int)(int16_t)a_sync[2]; lat_fn = m.a;
  }
  fbl = *d_fb_det != 0; }
{ static int sb_prev = 0;
  int md_0 = api_ram[4] & 0xff, md_1 = api_ram[0x18] & 0xff;
  int sb_now = (md_0 == 6 || md_1 == 6);
  if (sb_now && !sb_prev) {
      static unsigned nsb;
      if (nsb++ < 4000)
          printf("  [sb] fn=%u p51=%u | FB a fn=%u toa=%d pm=%u ang=%d "
                 "| TOA-ROM=%d src=%s "
                 "| a_sch=%04x %s\n",
                 m.a, m.a % 51u, lat_fn, lat_toa, lat_pm, lat_ang,
                 g_toa_valeur,
                 g_toa_grille < 0 ? "?" : (g_toa_grille ? "GRILLE(0x0cce)" : "fine"),
                 a_sch0[0],
                 ((a_sch0[0] & 0x8100) == 0x8000) ? "CRC_OK" : "crc_ko");
  }
  sb_prev = sb_now; }
/* CRC watched on BOTH R pages, and the page reported: reading page 0
 * alone cannot tell a real decode from stale content. A genuine SCH
 * gives the SAME BSIC every time (32 for cellule_reelle.bin); a
 * value that changes at each hit is a 10-bit CRC passing by chance
 * (1/1024) or a stale cell. */
/* [2026-09-19] Who writes a_sch at all? The DSP never does: the
 * A_SCH-WR probe in c54x_mem.c (live, its ANGLE-WR neighbour fires)
 * counted ZERO stores by the ROM to 0x0837..0x083b / 0x084b..0x084f.
 * The only host writer is the PONT_CAN_SB block below, which is off.
 * Yet the cells change. The remaining writer is the ARM, through the
 * shared mapping, which no DSP-side probe can see -- so watch the
 * VALUES from here and name the frame. */
/* [2026-09-19] Who maintains d_dsp_page? calypso_api.h: an armed page
 * is B_GSM_TASK|page = 0x0002 or 0x0003; 0x0000 is the reset state
 * l1s_reset_hw() writes (sync.c:165), and it also puts the firmware
 * back on R page 0. Measured at 0x0000 on 39% of samples, because
 * the FBSB loop restarts ~12000 times. On silicon the DSP re-arms
 * the page itself; this names every transition and its writer. */
{ static uint16_t dpp; static int dfirst = 1; static unsigned ndp;
  uint16_t dp = api_ram[(API_NDB + NDB_D_DSP_PAGE) / 2];
  if (!dfirst && dp != dpp && ndp < 40) {
      printf("  [page] fn=%u d_dsp_page %04x -> %04x  (%s)\n", m.a, dpp, dp,
             (dp & 0x0002) ? ((dp & 1) ? "arme page 1" : "arme page 0")
                           : "NON ARME (etat de reset)");
      ndp++;
  }
  dpp = dp; dfirst = 0; }
{ static uint16_t prev[2][5]; static int first = 1; static unsigned nch;
  for (int pg = 0; pg < 2; pg++) {
      const uint16_t *a = a_sch_pg[pg];
      /* ceiling 600 and TOA of the R page: this is the SB framing
       * measurement (a well-placed SCH reads TOA=23). */
      if (!first && nch < 600 &&
          (a[0] != prev[pg][0] || a[3] != prev[pg][3] || a[4] != prev[pg][4])) {
          const uint16_t *rp = &api_ram[API_R_PAGE(pg) / 2];
          printf("  [a_sch] fn=%u page=%d : %04x %04x %04x %04x -> "
                 "%04x %04x %04x %04x  (d_dsp_page=%04x) TOA=%d PM=%d SNR=%u %s\n",
                 m.a, pg, prev[pg][0], prev[pg][1], prev[pg][3], prev[pg][4],
                 a[0], a[1], a[3], a[4],
                 api_ram[(API_NDB + NDB_D_DSP_PAGE) / 2],
                 (int16_t)rp[RP_A_SERV_DEMOD/2 + D_TOA], (int16_t)rp[RP_A_SERV_DEMOD/2 + D_PM],
```

```
                    rp[RP_A_SERV_DEMOD/2 + D_SNR],
                    ((a[0] & 0x8100) == 0x8000) ? "CRC_OK" : (a[0] & 0x0100) ? "crc_ko" : "");
                nch++;
            }
            prev[pg][0]=a[0]; prev[pg][1]=a[1]; prev[pg][3]=a[3]; prev[pg][4]=a[4];
        }
        first = 0; }
    { static int crc_prev[2] = {1, 1};
        for (int pg = 0; pg < 2; pg++) {
            const uint16_t *a = a_sch_pg[pg];
            /* [2026-09-19] The criterion (a[0] & 0x8100) == 0x8000 counts
             * SATURATED ACCUMULATORS as CRC OK: a_sch[0] has been seen
             * carrying plain numbers, and 0x8000 is exactly what a
             * saturated accumulator stores (rejouer.c:923). Every "SB CRC
             * OK" of this session rested on it, so it is replaced by a
             * test the arithmetic cannot pass by accident: GSM 04.08
             * bounds T2 <= 25 and T3' <= 4, so 6 of 32 T2 values and 3 of
             * 8 T3' values are IMPOSSIBLE in a real SCH. */
            uint32_t _sb = (uint32_t)a[3] | ((uint32_t)a[4] << 16);
            unsigned _t2  = (_sb >> 18) & 0x1f;
            unsigned _t3p = ((_sb >> 24) & 1) | ((_sb >> 15) & 6);
            int _plausible = (_t2 <= 25) && (_t3p <= 4);
            int crc_ok = ((a[0] & 0x8100) == 0x8000) && _plausible;
            if (crc_ok && !crc_prev[pg]) {
                uint32_t sb = (uint32_t)a[3] | ((uint32_t)a[4] << 16);
                printf("  [jalon] fn=%u SB PLAUSIBLE page=%d BSIC=%u "
                       "a_sch=%04x %04x %04x %04x  hacks=%s\n",
                       m.a, pg, (unsigned)((sb >> 2) & 0x3f),
                       a[0], a[1], a[3], a[4], hacks_actifs());
            }
            crc_prev[pg] = crc_ok;
        } }
    if ((trames % 217) == 0) {
        profil_publier();
        pcc_publier();
        dump_publier(dsp);
    }
    if (verbeux || (trames % 217) == 0) {
        printf("  fn=%-7u page=%u insn=%-7u %s%s%s  d_fb_det=%-5u a_sch=%04x %04x %04x %04x"
               " sync=%04x %04x %04x  | trames=%lu irq=%lu iq=%lu\n",
               m.a, m.b, ninsn,
               (drapeaux & PONT_DONE_IDLE) ? "IDLE " : "occupe ",
               (drapeaux & PONT_DONE_API_IRQ) ? "IRQ-API " : "",
               (drapeaux & PONT_DONE_INIT) ? "" : "boot ",
               *d_fb_det, a_sch0[0], a_sch0[1], a_sch0[2], a_sch0[3],
               a_sync[0], a_sync[1], a_sync[2], trames, irqs, injectes);
        if ((trames % (217*8)) == 0) printf("  hacks actifs : %s ; cadencement : %s ; stimulus : %s\n",
                                            hacks_actifs(), cadencement_actif(), iq_mode ? iq_mode : "none");
    }
    fflush(stdout);
    break;
}
case PONT_DCCH:
    /* QEMU read the dedicated channel in the firmware's L1CTL stream
     * (calypso_dcch_tap.c): a = TN, b = kind, c = subchannel. The BSP
     * needs it to know which timeslot to deliver: the bridge sends it
     * all eight, it only plays one per tick, and without this it is
     * always TS0 - hence nothing of the mobile's SDCCH. */
    if (m.b == 4)
        printf("pont : canal dedie arme : TS%u PDCH (GPRS)\n", m.a);
    else
        printf("pont : canal dedie %s : TS%u SDCCH/%s SS=%u\n",
               m.b == 0xFF ? "libere" : "arme", m.a, m.b ? "8" : "4", m.c);
    calypso_bsp_set_dedie((int)m.a, (int)m.b, (int)m.c);
    if (m.b == 0xFF || (int)m.a <= 0) {
        montant_canal_libere();
```

```
            }
            break;
        case PONT_BYE:
            printf("pont : BYE\n");
            return;
        default:
            fprintf(stderr, "pont : message inconnu type=%u\n", m.type);
            break;
        }
    }
    printf("pont : bilan de la session : %lu trames, %lu IRQ API, %lu reset, %llu insn\n",
           trames, irqs, resets, (unsigned long long)insns_total);
    montant_bilan();
}

int pont_serveur(C54xState *dsp, uint16_t *api_ram, const char *socket_path,
                 long insns, bool verbeux, const char *iq_mode, int amp)
{
    signal(SIGINT, sur_signal);
    signal(SIGTERM, sur_signal);
    signal(SIGUSR1, sur_pause);
    signal(SIGUSR2, sur_pause);
    { static int16_t rempl[2 * 148];         /* TS1..TS7 of the C0 carrier: dummy bursts */
      cellule_factice(amp, 0.5, rempl);
      calypso_bsp_set_remplissage(rempl, 2 * 148); }

    int srv = socket(AF_UNIX, SOCK_SEQPACKET, 0);
    if (srv < 0) {
        fprintf(stderr, "pont : socket : %s\n", strerror(errno));
        return 1;
    }
    struct sockaddr_un sa = { .sun_family = AF_UNIX };
    snprintf(sa.sun_path, sizeof(sa.sun_path), "%s", socket_path);
    unlink(socket_path);
    if (bind(srv, (struct sockaddr *)&sa, sizeof(sa)) < 0 || listen(srv, 1) < 0) {
        fprintf(stderr, "pont : bind/listen(%s) : %s\n", socket_path, strerror(errno));
        close(srv);
        return 1;
    }
    chmod(socket_path, 0666);
    if (iq_mode && *iq_mode && strcmp(iq_mode, "none") != 0)
        printf("pont : injection I/Q « %s » amplitude %d a chaque trame (DARAM 0x%04x, %u mots)\n",
               iq_mode, amp, calypso_bsp_get_daram_addr(), calypso_bsp_get_daram_len());
    printf("pont : en attente de l'ARM sur %s (API RAM : /dev/shm%s)\n"
           "       cote QEMU : CALYPSO_DSP_EXTERN=1 qemu-system-arm -M calypso ...\n",
           socket_path, CALYPSO_PONT_SHM);
    fflush(stdout);

    while (!g_stop) {
        struct pollfd pfd = { .fd = srv, .events = POLLIN };
        if (poll(&pfd, 1, 200) <= 0) {
            continue;
        }
        int fd = accept(srv, NULL, NULL);
        if (fd < 0) {
            continue;
        }
        servir(fd, dsp, api_ram, insns, verbeux, iq_mode, amp);
        close(fd);
        fflush(stdout);
    }
    close(srv);
    unlink(socket_path);
    return 0;
}
```

3.20 /opt/GSM/c54x_exe/src/pont.h

1062 octets, 23 lignes → 23 lignes

```
/* SPDX-License-Identifier: GPL-2.0-or-later */
#ifndef PONT_H
#define PONT_H
#include <stdint.h>
#include <stdbool.h>
#include "calypso_c54x.h"

/* Allocates the C54xState with data[] page-aligned, then maps the shared segment
 * OVER data[0x0800..0x27FF] (protocol in calypso_dsp_pont.h). Replaces
 * c54x_init() in --arm mode; api_ram must then be &dsp->data[C54X_API_BASE]. */
C54xState *pont_allouer_dsp(void);

/* Serves QEMU's ARM: one connection after another, runs one frame per TICK,
 * answers DONE. Returns only on SIGINT/SIGTERM.
 * iq_mode: NULL/"none" = nothing, "fcch" = synthetic FCCH burst (+pi/2 rotation
 * per sample, 1 sample/symbol), "noise" = noise, "tone:<dphi>" = dphi radians
 * per sample, "cell[:bsic[:decalage[:marge]]]" = full 51-multiframe cell.
 * amp = int16 amplitude (real FCCH bursts run at ~32500 rms). Injected on every
 * frame through calypso_bsp_rx_burst(). */
int pont_serveur(C54xState *dsp, uint16_t *api_ram, const char *socket_path,
                 long insns, bool verbeux, const char *iq_mode, int amp);

#endif
```

3.21 /opt/GSM/c54x_exe/src/rejouer.c

161814 octets, 2596 lignes → 2593 lignes (2 groupes compactés)

```
/* SPDX-License-Identifier: GPL-2.0-or-later */
/*
 * rejouer.c - deterministic replay of FB/SB acquisition on the real DSP.
 *
 * Two identical runs of the full bench (QEMU + osmocon + mobile) diverged: the
 * ARM (TCG) advances at host speed between frame interrupts, so the frame on
 * which it posts the FB task changes from run to run. Here the ARM is NOT
 * emulated: its layer 1 is replayed in C (l1_sync + fb_sched_set/sb_sched_set
 * from prim_fbsb.c/sync.c), injection is locked to the frame counter, and
 * nothing reads the host clock. Same input => same output, always.
 *
 * This is not an operating mode: it is a DSP measurement bench (FB then SB)
 * driven by the same command sequence as the real firmware.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <math.h>
#include "calypso_c54x.h"
#include "calypso_bsp.h"
#include "hw/arm/calypso/calypso_api.h"
#include "cellule.h"
#include "calypso_twl3025.h"
#include "calypso_rhea_dma.h"
#include "rejouer.h"
#include <osmocom/core/bits.h>
#include <osmocom/core/crcgen.h>
#include <osmocom/coding/gsm0503_parity.h>
#include "hw/arm/calypso/calypso_debug.h"
```

```
/* ---- API offsets in WORDS from the API base (= DSP 0x800) --------------- */
#define W_PAGE(p)     ((p) ? 0x14u : 0x00u)   /* T_DB_MCU_TO_DSP, 17 words */
#define R_PAGE(p)     ((p) ? 0x3Cu : 0x28u)   /* T_DB_DSP_TO_MCU, 20 words */
#define W_SIZE        17u
#define R_SIZE        20u
#define W_TASK_D      0u
#define W_TASK_MD     4u
#define W_CTRL_ABB    11u
#define W_AFC         15u
#define W_CTRL_SYS    16u
#define NDB           0xD4u
#define NDB_PAGE      (NDB + 0u)
#define NDB_ERRSTAT   (NDB + 1u)
#define NDB_FB_DET    (NDB + 36u)
#define NDB_FB_MODE   (NDB + 37u)
#define NDB_SYNC      (NDB + 38u)   /* a_sync_demod[TOA,PM,ANGLE,SNR] */
#define R_SERV        8u            /* a_serv_demod[4] */
/* [2026-09-18] SYMBOL SAMPLING OFFSET. Replay sampled the GMSK at 0.0, i.e. on
 * the symbol BOUNDARY, while the bridge uses 0.5, the CENTER. With the gaussian
 * pulse of gmsk.c (BT=0.3), the weight of the target symbol in the phase
 * difference between two consecutive samples is:
 *
 *   offset   target symbol   previous neighbour   next neighbour
 *    0.0         0.454            0.023               0.492
 *    0.25        0.590            0.069               0.333
 *    0.5         0.653            0.158               0.189
 *
 * At 0.0 the NEXT neighbour outweighs the symbol being read: the offset is not
 * noisy, it is UNDETERMINED, and the demodulator may latch onto either one. That
 * accounts for offset=22 at margin=21, midamble at 56/63, data at 71-73/78 with
 * otherwise perfect samples, and the correlation peak jumping between adjacent
 * lags. */
/* Tunable for sweeps: 0.5 (symbol center) is argued above, but it is a
 * hypothesis and the bench must be able to test it.
 * REJEU_DECALAGE_SYMB=<x>, default 0.5. */
static double decalage_symb(void)
{
    static double d = -1.0;
    if (d < 0) { const char *e = calypso_getenv("REJEU_DECALAGE_SYMB");
                 d = (e && *e) ? atof(e) : 0.5;
                 if (d < 0 || d >= 1.0) d = 0.5; }
    return d;
}
#define DECALAGE_SYMB decalage_symb()
/* Tunable head margin (REJEU_MARGE, default 21); the tail fills out to 190 complex samples */
static int marge_tete(void)
{
    static int m = -1;
    if (m < 0) { const char *e = calypso_getenv("REJEU_MARGE"); m = e ? atoi(e) : 21;
                 if (m < 0 || m > 41) m = 21;
                 cellule_marge_fin = 190 - 148 - m; }
    return m;
}

#define R_SCH         15u           /* a_sch[5] */
#define B_SCH_CRC     8
/* B_GSM_TASK comes from calypso_api.h (bit mask, (1u << 1)) */
#define B_AFC         4
#define FB_DSP_TASK   5
#define SB_DSP_TASK   6
#define BITS_PER_TDMA 1250   /* tpu.h: a TDMA frame is 8 x 156.25 = 1250 bit periods */

/* firmware thresholds (prim_fbsb.c: SNR not gating, #else FB*_SNR_THRESH=0) */
#define THRESH1       (11000 - 1000)
#define THRESH2       (1000 - 200)
#define AFC_RETRY_MAX 30
```

```
#define FB0_RETRY_MAX 3
/* sync.h: ANGLE_TO_FREQ(a) = a * BITFREQ_DIV_PI / ANG2FREQ_SCALING */
#ifndef BITFREQ_DIV_PI
#define BITFREQ_DIV_PI   86208   /* sync.h:203: 270kHz/pi */
#endif
#ifndef ANG2FREQ_SCALING
#define ANG2FREQ_SCALING (2<<15) /* sync.h:204: fx1.15 */
#endif
#define ANGLE2FREQ(a) ((int)(int16_t)(a) * BITFREQ_DIV_PI / ANG2FREQ_SCALING)

/* ---- replayed ARM state ------------------------------------------------ */
typedef void (*cb_t)(int attempt);
struct item { int frame; cb_t cb; int attempt; };

static uint16_t *api;
static C54xState *dsp;
static unsigned w_page, r_page, r_page_used;
static uint32_t fn_cur;              /* l1s.current_time.fn */
static unsigned long histo_pc[64];
static unsigned long insn_total;
static unsigned char *vu_sb, *vu_hors;
static int16_t g_livre_iq[2*256]; static int g_livre_niq;
static char g_livre_type; static uint32_t g_livre_fn; static int g_livre_n;
static int      afc_dac = -700;      /* afc_initial_dac_value (compal) */
static int      fb_mode, afc_retries, fb0_retries;
static struct   item sched[32]; static int n_sched;
static struct   { int toa, pm, angle, snr, freq_diff, attempt; uint32_t fnr; } fb;
static int      verdict;             /* 0 running, 1 SB OK, -1 gave up */
static uint32_t sb_word; static int sb_bsic; static uint32_t sb_fn;
static int      n_fb_ok, n_sb_try, n_sb_crcfail, n_crc_ok, n_sb_vraies;
static int      n_err_dsp, n_err8;
static long     n_cmps_sb, n_cmps_hors, n_e0_sb, n_sttrn_sb;
static long     n_e0x_sb[4];
static int      n_firs_vus;
static long     n_cmps_reg[64];
static long     n_xc_reg[64];
static int      n_xc_vus;
static int      n_bal;
static int      n_sat_vus;
static int      n_ecr;
static int      n_2a0;
static int      g_bsic_injecte;   /* BSIC actually transmitted by the cell */
static int      trace;
static unsigned long hit_7c31, hit_9841, hit_84a1, hit_770a, hit_b219, hit_7a16;
static unsigned long hit_8478, hit_8492, hit_8493, hit_8497;
static unsigned long hit_7d1c, hit_7d1d, hit_7d1e, hit_81e4;
/* [2026-09-19] SB PATH OPCODE INVENTORY. The fault is confined to the window
 * 0x84a1..0x8497, but which instruction is mis-emulated is unknown. Record the
 * distinct opcodes executed inside the SB demod with their pass count and one
 * witness PC: that gives a finite list to audit against SPRU172C instead of an
 * intuition. REJEU_OPCODES=1. */
/* Environment flags read ONCE in rejouer(): calypso_getenv() inside the per-instruction
 * loop was a linear scan of environ per emulated instruction. */
static int env_div, env_softs_continu, env_firs, env_probe_t, env_decodeur;
static uint16_t g_ad_avant;
static uint32_t g_vie_fn; static uint16_t g_vie_ad;
static int g_dans_sb;            /* true between 0x7c31 (demod) and 0x9841 (decoder) */
static unsigned long g_op_n[65536];
static unsigned long g_op_dec[65536];   /* DECODER opcodes, region 0x9800-0x9bff */
static unsigned long g_op_eq[65536];    /* EQUALIZER opcodes, region 0x8400-0x84ff */
static unsigned long g_n_dec;           /* number of decodes (passes at 0x9841) */
static uint16_t      g_op_pc[65536];
static unsigned long g_op_tous[65536];   /* every executed opcode word, both paths */

/* [2026-09-18] REAL SCH BURSTS. Until now the DSP only ever saw our own
 * fixture, a synthetic GMSK SCH at 1 sample/symbol, so "is the ROM sound, or is
```

```
 * the fixture too poor for a coherent equalizer?" stayed open. Inject SCH bursts
 * extracted from a REAL capture (ptrkrysik/test_data, ARFCN 725, USRP decimation
 * 174 -> 574712.6 Hz) whose answer is known: BSIC=32, Viterbi decode with 0
 * errors, valid CRC. REJEU_SCH_REEL=<file>. The framing is unchanged -- same
 * margins, same timing -- ONLY the burst content differs. */
static int16_t (*g_reels)[296];
static uint8_t (*g_reels_code)[78];      /* the 78 bits actually transmitted */
static uint32_t *g_reels_fn; static uint16_t *g_reels_bsic;
static unsigned g_n_reels, g_reel_i;
static int g_reel_pour_fn[64];           /* which frame received which burst */
static int reels_charger(const char *chemin)
{
    FILE *f = fopen(chemin, "rb");
    if (!f) { fprintf(stderr, "SCH reels : %s illisible\n", chemin); return -1; }
    uint32_t n = 0, nsym = 0, ncode = 0;
    if (fread(&n, 4, 1, f) != 1 || fread(&nsym, 4, 1, f) != 1 ||
        fread(&ncode, 4, 1, f) != 1 || nsym != 148 || ncode != 78 || !n) {
        fprintf(stderr, "SCH reels : entete invalide\n"); fclose(f); return -1; }
    g_reels = calloc(n, sizeof *g_reels);
    g_reels_code = calloc(n, sizeof *g_reels_code);
    g_reels_fn = calloc(n, sizeof *g_reels_fn);
    g_reels_bsic = calloc(n, sizeof *g_reels_bsic);
    if (!g_reels || !g_reels_code || !g_reels_fn || !g_reels_bsic) { fclose(f); return -1; }
    for (uint32_t i = 0; i < n; i++) {
        uint32_t fn; uint16_t bsic; uint8_t pad[2];
        if (fread(&fn,4,1,f)!=1 || fread(&bsic,2,1,f)!=1 || fread(pad,1,2,f)!=2 ||
            fread(g_reels[i], sizeof(int16_t), 296, f) != 296 ||
            fread(g_reels_code[i], 1, 78, f) != 78) { n = i; break; }
        g_reels_fn[i] = fn; g_reels_bsic[i] = bsic;
    }
    fclose(f); g_n_reels = n;
    for (int i = 0; i < 64; i++) g_reel_pour_fn[i] = -1;
    printf("SCH reels : %u bursts charges depuis %s (BSIC attendu %u)\n",
           n, chemin, n ? g_reels_bsic[0] : 0);
    return n ? 0 : -1;
}
/* Read PROGRAM space the way the core does: with OVLY set, DARAM 0x0060..0x27FF
 * is aliased into program space, so reading dsp->prog[] directly misses the
 * alias -- exactly the trap the FIRS probe must avoid. */
static uint16_t prog_ovly(C54xState *d, uint16_t a)
{
    if ((d->pmst & (1u << 5)) && a >= 0x0060 && a < 0x2800) return d->data[a];
    return d->prog[a];
}

static void plan(int delay, cb_t cb, int attempt)
{
    if (n_sched >= 32) return;
    sched[n_sched].frame = (int)fn_cur + delay;
    sched[n_sched].cb = cb; sched[n_sched].attempt = attempt; n_sched++;
}
static void sched_reset(void) { n_sched = 0; }

static uint16_t *dbw(void) { return &api[W_PAGE(w_page)]; }
static uint16_t *dbr(void) { return &api[R_PAGE(r_page)]; }

/* ---- callbacks: faithful copy of prim_fbsb.c ---------------------------- */
static void fbdet_cmd(int unused);
static void fbdet_resp(int attempt);
static void sbdet_cmd(int attempt);

/* calypso_getenv("X") alone is TRUE even for X=0, so a line written
 * `REJEU_SCH_PARTOUT=0 REJEU_SB_FORCE=0` enabled every hack instead of cutting
 * them. Here "0", "", "non", "no" and "off" are false. */
static int drapeau_env(const char *nom)
{
```

```
    const char *e = calypso_getenv(nom);
    if (!e || !*e) return 0;
    if (e[0] == '0' && e[1] == 0) return 0;
    if (!strcmp(e, "non") || !strcmp(e, "no") || !strcmp(e, "off")) return 0;
    return 1;
}

static void plan_sb_annule(void) { }

static void sbdet_resp(int attempt);

static void plan_fb_set(int delay, int mode)
{
    fb_mode = mode;
    plan(delay + 0, fbdet_cmd, 0);
    for (int a = 1; a <= 12; a++) plan(delay + 1 + a, fbdet_resp, a);
}
static int sb_uniq(void)
{
    static int u = -1;
    if (u < 0) u = drapeau_env("REJEU_SB_UNIQUE") ? 1 : 0;
    return u;
}
/* Last scheduled SB attempt: that one must restart an acquisition. Hardcoding
 * it to "attempt == 2" leaves the bench silent under a single command, where
 * that attempt does not exist. */
static int sb_dernier_essai(void) { return sb_uniq() ? 1 : 2; }

static void plan_sb_set(int delay)
{
/* [2026-09-18] The DSP demodulates the burst of the frame FOLLOWING the command
     * (measured by perturbation: command at fn=11 -> only a perturbation at fn=12
     * changes the softs). Posting on TWO consecutive frames therefore consumes f and
     * f+1; SCH frames are 10 apart, so the second attempt always reads a DUMMY
     * burst, which is half the measurements taken on a constant input.
     * REJEU_SB_UNIQUE=1 posts a single command so every attempt sees a real SCH. */
    plan(delay + 0, sbdet_cmd, 1);
    if (!sb_uniq()) plan(delay + 1, sbdet_cmd, 2);
    plan(delay + 3, sbdet_resp, 1);
    if (!sb_uniq()) plan(delay + 4, sbdet_resp, 2);
}

static uint32_t g_sb_cmd_fn;   /* frame of the LAST SB command = the one the DSP demodulates */
static uint32_t fb_cmd_fn;      /* frame on which the ARM posted the FB task */
static void fbdet_cmd(int unused)
{
    (void)unused;
    fb_cmd_fn = fn_cur;
    dbw()[W_TASK_MD]  = FB_DSP_TASK;
    api[NDB_FB_MODE]  = (uint16_t)fb_mode;
}

static void fbdet_resp(int attempt)
{
    if (!api[NDB_FB_DET]) {
        if (attempt < 12) return;
        sched_reset();
        if (fb0_retries < FB0_RETRY_MAX) { fb0_retries++; plan_fb_set(1, 0); }
        else verdict = -1;
        return;
    }
    /* read_fb_result */
    fb.toa   = (int16_t)api[NDB_SYNC + 0];
    fb.pm    = (int16_t)api[NDB_SYNC + 1] >> 3;
    fb.angle = (int16_t)api[NDB_SYNC + 2];
    fb.snr   = (int16_t)api[NDB_SYNC + 3];
```

```
    fb.freq_diff = ANGLE2FREQ(fb.angle);
    fb.fnr = fn_cur; fb.attempt = attempt;
    api[NDB_FB_DET] = 0; api[NDB_SYNC + 0] = 0;
    n_fb_ok++;
    if (trace) {
        /* real FCCH = first frame >= cmd with p51 in {0,10,20,30,40} */
        uint32_t f = fb_cmd_fn; while ((f % 51) % 10 != 0 || (f % 51) > 40) f++;
        int dframe = (int)(f - fb_cmd_fn);                /* FCCH frame inside the window */
        int toa_attendu = dframe * BITS_PER_TDMA + 23;      /* what the firmware can read */
        int ntdma_lu = (fb.toa - 23) / BITS_PER_TDMA;
        printf("  FB%d att=%d fn=%u TOA=%d (df=%d) | cmd=%u FCCH reelle=%u (+%d trames) "
               "=> TOA attendu ~%d, ntdma lu=%d au lieu de %d\n",
               fb_mode, attempt, fn_cur, fb.toa, fb.freq_diff,
               fb_cmd_fn, f, dframe, toa_attendu, ntdma_lu, dframe);
    }
    /* afc_correct: delta = (norm * err)/slope; slope compal_e88 = 287 */
    afc_dac += (int)(((32768 / 947) * (long)fb.freq_diff) / 287);
    if (afc_dac > 4095) afc_dac = 4095;
    if (afc_dac < -4096) afc_dac = -4096;
    sched_reset();
    if (fb_mode == 0) {
        if (abs(fb.freq_diff) < THRESH1) plan_fb_set(1, 1);
        else if (afc_retries < AFC_RETRY_MAX) { afc_retries++; plan_fb_set(1, 0); }
        else verdict = -1;
    } else {
        int toa = fb.toa - 23, ntdma, qbits;
        if (toa < 0) { qbits = (toa + BITS_PER_TDMA) * 4; ntdma = -1; }
        else { ntdma = toa / BITS_PER_TDMA; qbits = (toa - ntdma * BITS_PER_TDMA) * 4; }
        int fn_offset = (int)fn_cur - attempt + ntdma;
        int delay = fn_offset + 11 - (int)fn_cur - 1;
        if (trace) printf("    -> toa-23=%d ntdma=%d qbits=%d delay=%d\n", toa, ntdma, qbits, delay);
        if (abs(fb.freq_diff) < THRESH2) {
            if (delay < 0) delay = 0;
            if (delay > 20) delay = 20;
            /* REJEU_SB_FORCE=1: aim at the NEXT SCH frame (p51 in {1,11,21,31,41})
             * instead of the computed delay. Separates the DEMODULATOR from the
             * TIMING: a CRC passing here means only the timing (ntdma) is at
             * fault; a failure means the SB demodulation really is wrong. */
            static int force = -1;
            if (force < 0) force = drapeau_env("REJEU_SB_FORCE") ? 1 : 0;
            if (force) {
                uint32_t f = fn_cur + 1;
                while (!((f % 51) % 10 == 1 && (f % 51) <= 41)) f++;
                delay = (int)(f - fn_cur);
                /* [2026-09-18] THE CONSUMED FRAME IS NOT THE FIRST COMMAND.
                 * plan_sb_set posts the task on TWO frames (delay+0 and delay+1) and the
                 * DSP demodulates the SECOND: measured by perturbing one raw sample,
                 * frame by frame, over fn 9..15 -- only fn=12 changes the soft bits read
                 * at fn=14/15, while [force] aimed at fn=11. The SCH was thus placed on
                 * frame 11 while the DSP demodulated frame 12, which carries a DUMMY
                 * burst (fixed pattern, 45.002 5.2.6), hence a CONSTANT input: frozen
                 * softs, identical range from frame to frame, and CRC OKs all returning
                 * the same word tens of frames apart. Stepping back one frame aligns the
                 * CONSUMED frame with the SCH. REJEU_SB_DECALAGE=<n> (default -1). */
                { static int d = -2; if (d == -2) { const char *e = calypso_getenv("REJEU_SB_DECALAGE");
                                                    d = e ? atoi(e) : -1; }
                  delay += d; if (delay < 0) delay = 0; }
                if (trace) printf("    [force] SB vise fn=%u (p51=%u), delay=%d\n", f, f % 51, delay);
            }
            plan_sb_set(delay);
        } else plan_fb_set(1, 1);
    }
}

static void sbdet_cmd(int attempt)
{
```

```
    g_sb_cmd_fn = fn_cur;
    if (trace) { unsigned p = fn_cur % 51;
        printf("  SBcmd att=%d fn=%u p51=%u %s\n", attempt, fn_cur, p,
               (p % 10 == 1 && p <= 41) ? "<- trame SCH (bon)" : "<- PAS une trame SCH"); }
    dbw()[W_TASK_MD] = SB_DSP_TASK;
    api[NDB_FB_MODE] = 0;
    if (trace) printf("  [SBcmd W] page=%u W=%04x %04x %04x %04x %04x ..%04x %04x  NDB page=%04x fb_mode=%04x
    ↪  fb_det=%04x\n",
                      w_page, dbw()[0], dbw()[1], dbw()[2], dbw()[3], dbw()[4], dbw()[15], dbw()[16],
↪   api[NDB_PAGE], api[NDB_FB_MODE], api[NDB_FB_DET]);
}

static void sbdet_resp(int attempt)
{
    n_sb_try++;
    r_page_used = 1;
    if (trace) {
        /* SB demodulator internals (RE 9.4 and 9.19):
         * 0x2f06 = correlation peak index
         * 0x2bf8 = internal CRC flag
         * 0x2c72 = THE 78 SOFT BITS. 0x2a00 is the 296-word FB correlator buffer, so
         * reading it there yields only 0x0000 and 0xffff; the real softs at 0x2c72
         * have varied magnitudes. Their range is printed too. */
        {
            int16_t mn = 32767, mx = -32768, nnul = 0;
            for (int k = 0; k < 78; k++) {
                int16_t v = (int16_t)dsp->data[0x2c72 + k];
                if (v) nnul++;
                if (v < mn) mn = v;
                if (v > mx) mx = v;
            }
            /* THE FEED. Does the buffer the DSP demodulates really hold the
             * delivered samples, and at which shift? Compare the BSP drop word by word
             * with the last delivered burst and look for the best alignment: a maximum
             * away from 0 means the burst is shifted; low everywhere means it is not
             * our burst at all. */
            if (calypso_getenv("REJEU_FEED")) {
                uint16_t ad = calypso_bsp_get_daram_addr();
                int nl = g_livre_niq;
                int best = 0, bestn = -1, n0 = 0;
                for (int sh = -80; sh <= 80; sh++) {
                    int m = 0;
                    for (int k = 0; k < nl; k++) {
                        int j = k + sh; if (j < 0 || j >= 2048) continue;
                        if ((int16_t)dsp->data[(ad + j) & 0x3fff] == g_livre_iq[k]) m++;
                    }
                    if (sh == 0) n0 = m;
                    if (m > bestn) { bestn = m; best = sh; }
                }
                printf("    [feed] fn=%u (livre fn=%u type=%c) addr=0x%04x len=%u nl=%d"
                       "  identiques a shift0=%d/%d  meilleur shift=%d avec %d/%d\n",
                       fn_cur, g_livre_fn, g_livre_type ? g_livre_type : '?', ad,
                       calypso_bsp_get_daram_len(), nl, n0, nl, best, bestn, nl);
            }
            /* [2026-09-18] PER-POSITION ERROR PROFILE. An aggregate percentage hides
             * what discriminates: WHERE the errors fall. The 78 coded bits occupy
             * symbols [3..41] and [106..144], the midamble sits at [42..105]. Errors at
             * the EDGES = channel estimate right at the midamble and drifting away from
             * it (phase ramp / too short an estimate). Uniform errors = wrong equalizer.
             * ~0 errors under one polarity = good softs and a bug downstream. Compared
             * against the frame ACTUALLY demodulated (last SB command), not the current
             * frame. */
            /* [2026-09-19] FIRS reads its input at 0x2a8e..0x2a9a (zone 0x2a00) while
             * the BSP drops the burst at 0x0cce. Is that intermediate zone filled, and
             * does it carry our burst? */
            if (calypso_getenv("REJEU_ZONE2A")) {
```

```
    uint16_t ad = calypso_bsp_get_daram_addr();
    int nz0 = 0, nz2 = 0;
    for (int k = 0; k < 296; k++) {
        if (dsp->data[(ad + k) & 0x3fff]) nz0++;
        if (dsp->data[(0x2a00 + k) & 0x3fff]) nz2++;
    }
    printf("    [zone] depot 0x%04x : %d/296 non nuls | zone 0x2a00 : %d/296 non nuls\n",
           ad, nz0, nz2);
    printf("      0x%04x : %04x %04x %04x %04x %04x %04x\n", ad,
           dsp->data[ad&0x3fff], dsp->data[(ad+1)&0x3fff], dsp->data[(ad+2)&0x3fff],
           dsp->data[(ad+3)&0x3fff], dsp->data[(ad+4)&0x3fff], dsp->data[(ad+5)&0x3fff]);
    printf("      0x2a8e : %04x %04x %04x %04x %04x %04x   (entree lue par FIRS)\n",
           dsp->data[0x2a8e], dsp->data[0x2a8f], dsp->data[0x2a90],
           dsp->data[0x2a91], dsp->data[0x2a92], dsp->data[0x2a93]);
}
if (calypso_getenv("REJEU_PROFIL")) {
    unsigned char att78[78];
    int ri = g_n_reels ? g_reel_pour_fn[g_sb_cmd_fn & 63] : -1;
    if (ri >= 0) memcpy(att78, g_reels_code[ri], 78);   /* real ground truth */
    else cellule_code_attendu(g_sb_cmd_fn, (uint8_t)g_bsic_injecte, att78);
    int acc[2] = {0, 0};
    char map[2][79];
    for (int pol = 0; pol < 2; pol++) {
        for (int k = 0; k < 78; k++) {
            int16_t v = (int16_t)dsp->data[0x2c72 + k];
            int bit = pol ? (v > 0) : (v < 0);   /* both sign conventions */
            int ok = (bit == (att78[k] & 1));
            map[pol][k] = ok ? '.' : 'X';
            acc[pol] += ok;
        }
        map[pol][78] = 0;
    }
    /* [2026-09-18] ALIGNMENT SWEEP. The peak moves from frame to frame
     * (23, 20, 19...): if it places the read window, comparing at the
     * canonical alignment compares good softs against a wrong framing, and
     * "uncorrelated" would be a false conclusion. Rebuild the full expected
     * burst (3 tail + 39 + 64 TSC + 39 + 3 tail) and find the shift s that
     * maximizes agreement. An s reaching ~78/78 means the softs are GOOD and
     * only the framing is wrong. */
    {
        unsigned char burst[148];
        memset(burst, 0, 3);
        for (int k = 0; k < 39; k++) burst[3 + k] = att78[k];
        for (int k = 0; k < 64; k++) burst[42 + k] = (unsigned char)cellule_train_sb(k);
        for (int k = 0; k < 39; k++) burst[106 + k] = att78[39 + k];
        memset(burst + 145, 0, 3);
        int bs = 0, bp = 0, bv = -1;
        for (int sh = -40; sh <= 40; sh++) {
            for (int pol = 0; pol < 2; pol++) {
                int m = 0, n = 0;
                for (int k = 0; k < 78; k++) {
                    int pos = (k < 39 ? 3 + k : 106 + (k - 39)) + sh;
                    if (pos < 0 || pos >= 148) continue;
                    int16_t v = (int16_t)dsp->data[0x2c72 + k];
                    int bit = pol ? (v > 0) : (v < 0);
                    if (bit == (burst[pos] & 1)) m++;
                    n++;
                }
                if (n >= 60 && m > bv) { bv = m; bs = sh; bp = pol; }
            }
        }
        printf("    [align] meilleur decalage=%+d polarite=%d -> %d/78\n", bs, bp, bv);
    }
    int best = acc[1] > acc[0] ? 1 : 0;
    printf("    [profil] fn_demod=%u (p51=%u) pic=%u : concordance pol0=%d/78 pol1=%d/78\n"
           "      premiers39 |%.39s|\n"
```

```
                    "      derniers39 |%s|\n",
                    g_sb_cmd_fn, g_sb_cmd_fn % 51, dsp->data[0x2f06],
                    acc[0], acc[1], map[best], map[best] + 39);
            }
            if (calypso_getenv("REJEU_DUMP_SOUPLES")) {
                printf("    [souples] fn=%u pic=%u :", fn_cur, dsp->data[0x2f06]);
                for (int k = 0; k < 78; k++) printf(" %04x", dsp->data[0x2c72 + k]);
                printf("\n");
            }
            printf("    [sb-int] pic(2f06)=%u  crc(2bf8)=%u  souples(2c72)[0..5]=%04x %04x %04x %04x %04x %04x"
                   "  non nuls=%d/78  etendue=[%d..%d]\n",
                   dsp->data[0x2f06], dsp->data[0x2bf8],
                   dsp->data[0x2c72], dsp->data[0x2c73], dsp->data[0x2c74],
                   dsp->data[0x2c75], dsp->data[0x2c76], dsp->data[0x2c77], nnul, mn, mx);
        }
    }
    if (trace) { unsigned p = fn_cur % 51;
        /* Decoded word EVEN on a failed CRC, plus the ROM's measurements on the
         * burst (a_serv_demod: TOA, PM, ANGLE, SNR): the material for comparing
         * decoded SCHs against missed ones. */
        printf("  SBresp att=%d fn=%u p51=%u crc=%s a_sch=%04x %04x %04x %04x %04x toa=%d pm=%d angle=%d snr=%d
↪   sb_cmd_fn=%u\n", attempt, fn_cur, p,
               (dbr()[R_SCH + 0] & (1 << B_SCH_CRC)) ? "FAUX" : "OK",
               dbr()[R_SCH + 0], dbr()[R_SCH + 1], dbr()[R_SCH + 2], dbr()[R_SCH + 3], dbr()[R_SCH + 4],
               (int16_t)dbr()[R_SERV + 0], (int16_t)dbr()[R_SERV + 1], (int16_t)dbr()[R_SERV + 2],
                ↪  (int16_t)dbr()[R_SERV + 3], g_sb_cmd_fn); }
    if (dbr()[R_SCH + 0] & (1 << B_SCH_CRC)) {
        n_sb_crcfail++;
        if (attempt == sb_dernier_essai()) { sched_reset(); plan_fb_set(1, 0); }
        return;
    }
    sb_word = dbr()[R_SCH + 3] | ((uint32_t)dbr()[R_SCH + 4] << 16);
    sb_bsic = (sb_word >> 2) & 0x3f;
    unsigned t1 = ((sb_word >> 23) & 1) | ((sb_word >> 7) & 0x1fe) | ((sb_word << 9) & 0x600);
    unsigned t2 = (sb_word >> 18) & 0x1f;
    unsigned t3p = ((sb_word >> 24) & 1) | ((sb_word >> 15) & 6);
    unsigned t3 = t3p * 10 + 1;
    sb_fn = 51u * ((t3 - t2 + 26u) % 26u) + t3 + 26u * 51u * t1;
    /* A CRC OK is a decode only if it yields the INJECTED BSIC, a valid T3 (T3 <= 50
     * by construction) and a reconstructed FN equal to the current frame. Each pass
     * is qualified and the run continues instead of stopping on the first.
     * REJEU_ARRET_1ER=1 restores stopping.
     *
     * REJEU_SCH_PARTOUT=1 CORRUPTS THE REFERENCE TRUTH: T3' has three bits, so
     * a SCH emitted on p51=45 encodes T3'=4, which the decoder turns back into
     * T3=41 and sb_fn != fn_cur even for a PERFECT decode. FN can only be
     * validated without that flag. */
    int t3_ok = (t3 <= 50), bsic_ok = (sb_bsic == (unsigned)g_bsic_injecte);
    /* [2026-09-20] The SB word carries the frame number of the BURST that was
     * demodulated, i.e. the frame of the last SB command (g_sb_cmd_fn), not the
     * frame on which the ARM reads the result (fn_cur, 2-3 frames later). The
     * first genuine decodes (BSIC 7, FN 31 read at fn 34; FN 62 read at fn 64)
     * were being counted as false positives by the old fn_cur comparison. */
    int fn_ok = (sb_fn == g_sb_cmd_fn);
    n_crc_ok++;
    if (bsic_ok && t3_ok && fn_ok) n_sb_vraies++;
    printf("  SB%d fn=%u : sb=0x%08x BSIC=%d (injecte %d) T1=%u T2=%u T3=%u -> FN=%u (burst demodule fn=%u)
↪   %s%s\n",
           attempt, fn_cur, sb_word, sb_bsic, g_bsic_injecte, t1, t2, t3, sb_fn, g_sb_cmd_fn,
           (bsic_ok && t3_ok && fn_ok) ? "** VRAIE **"
           : !bsic_ok ? "FAUX POSITIF (BSIC ne colle pas)"
           : !t3_ok   ? "FAUX POSITIF (T3 > 50, impossible)"
           :            "FAUX POSITIF (FN != trame du burst demodule)",
           cellule_sch_partout ? "  [FN non qualifiable sous SCH_PARTOUT]" : "");
    if (drapeau_env("REJEU_ARRET_1ER") || ((bsic_ok && t3_ok && fn_ok) && !drapeau_env("REJEU_CONTINUER"))) {
     ↪  verdict = 1; return; }
```

```
    if (bsic_ok && t3_ok && fn_ok) { sched_reset(); plan_fb_set(1, 0); return; }   /* REJEU_CONTINUER: restart the
     ↪  acquisition */
    /* [2026-09-18] An unqualified CRC OK (false positive) must reschedule, or the
     * bench freezes and the freeze reads as a result: the item queue drains, no
     * command is posted again, and the replay crosses the remaining thousands of
     * frames asking the DSP nothing. The summary was then IDENTICAL at 1200 and at
     * 12000 frames -- not because the DSP stalled, but because the replayed ARM had
     * gone silent. The CRC failure branch already restarted (plan_fb_set on attempt
     * 2); only the "CRC OK but false positive" exit was a dead end. */
    sched_reset(); plan_fb_set(1, 0);
}


/* ---- ARM-side DSP init: what the firmware's dsp_power_on() does ---------
 * Without it the DSP runs on a blank API RAM: the FB detection thresholds
 * (d_fb_thr_det_iacq/track) and the margins (d_fb_margin_beg/end, which feed the
 * TOA formula at 0x794b/0x7956) are 0, and FB detection diverges from the first
 * frame. Values from dsp_params.c / dsp_ndb_init(). */
#define PARAM   0x431u          /* BASE_API_PARAM (API 0x862) in words */
#define A_SCH26 (NDB + 42u)
/* DSP bootloader: API offsets in WORDS (dsp.c: BASE_API_RAM + 0x0ff8..0x0ffe) */
#define BL_ADDR_HI_W  0x7FCu
#define BL_SIZE_W     0x7FDu
#define BL_ADDR_LO_W  0x7FEu
#define BL_STATUS_W   0x7FFu
#define DSP_START     0x7000u

static long pump(long max, int stop_on_idle)
{
    long b = 0;
    while (b < max && dsp->running) {
        int ex = c54x_run(dsp, 256);
        if (ex <= 0) break;
        b += ex;
        if (stop_on_idle && dsp->idle) break;
    }
    return b;
}

static void arm_dsp_init(void)
{
    memset(api, 0, 0x2000u * sizeof(uint16_t));          /* dsp_api_memset(API) */

    /* dsp_pre_boot(): the DSP has just been reset; wait for BL_STATUS_IDLE */
    long b = 0;
    while (api[BL_STATUS_W] != 1 && b < 8000000) {
        int ex = c54x_run(dsp, 256); if (ex <= 0) break; b += ex;
    }
    printf("  bootloader DSP : BL_STATUS=%u apres %ld insn\n", api[BL_STATUS_W], b);

    /* dsp_set_params(): NDB first */
    api[NDB + 8]  = 0x0074;  api[NDB + 9]  = 0x0001;
    api[NDB + 10] = 0x0154;  api[NDB + 11] = 0x17ff;
    api[NDB + 12] = 7;       api[NDB + 13] = 0;
    api[NDB + 14] = 3;                                  /* d_dsp_state = IDLE3 */
    /* then the parameter table (dsp_params.c). The first four FB fields feed the
     * TOA formula (0x794b/0x7956) and the detection thresholds. */
    static const int16_t P[] = {
        0x6666, 15, 12, 5, 4, 0x7002, 1, 0xE, 0, 0, 0, 0,
        24, 22, 296, 30,                                /* margin_beg/end, nsubb_idle/dedic */
        0x3333, (int16_t)0x28f6,                        /* fb_thr_det_iacq / _track */
        0x7fff, 17408, 26624, 20152,
        7872, -4, 7872, 5772, 7872, 53, -892, 208,
    };
    for (unsigned i = 0; i < sizeof P / sizeof P[0]; i++) api[PARAM + i] = (uint16_t)P[i];
```

```
    /* dsp_bl_start_at(DSP_START) */
    api[BL_ADDR_HI_W] = 0; api[BL_ADDR_LO_W] = DSP_START; api[BL_SIZE_W] = 0;
    api[BL_STATUS_W]  = 2;                              /* BL_CMD_COPY_BLOCK */
    b = pump(4000000, 1);
    printf("  DSP demarre : %ld insn, idle=%d, version=0x%04x%04x\n",
           b, dsp->idle, api[NDB + 6], api[NDB + 7]);

    /* dsp_ndb_init(): what matters for FB/SB */
    api[NDB + 2]  = 0x0179;                             /* d_spcx_rif */
    api[NDB + 3]  = 0x0800 | ((8 - 4) << 7);            /* d_tch_mode */
    api[NDB_FB_MODE] = 1;
    api[NDB_FB_DET]  = 0;
    api[A_SCH26]     = (1u << B_SCH_CRC);
    /* dsp_db_init() */
    memset(&api[W_PAGE(0)], 0, W_SIZE * sizeof(uint16_t));
    memset(&api[W_PAGE(1)], 0, W_SIZE * sizeof(uint16_t));
    memset(&api[R_PAGE(0)], 0, R_SIZE * sizeof(uint16_t));
    memset(&api[R_PAGE(1)], 0, R_SIZE * sizeof(uint16_t));
    /* [2026-09-20] REJEU_DUMP_PARAM=1: the first eight parameter words after
     * the DSP has started, to compare with the live bench's shared API RAM
     * (od on /dev/shm/calypso_api_ram at word 0x431). */
    if (drapeau_env("REJEU_DUMP_PARAM"))
        printf("  PARAM apres demarrage : %04x %04x %04x %04x %04x %04x %04x %04x  (attendu 6666 000f 000c 0005
↪   0004 7002 0001 000e)\n",
               api[PARAM], api[PARAM+1], api[PARAM+2], api[PARAM+3], api[PARAM+4], api[PARAM+5], api[PARAM+6],
↪   api[PARAM+7]);
}

/* ---- one frame: l1_sync() then the DSP -------------------------------- */
static void l1_sync(void)
{
    r_page_used = 0;
    memset(dbw(), 0, W_SIZE * sizeof(uint16_t));          /* memset db_w */
    dbw()[W_AFC] = (uint16_t)(int16_t)afc_dac;            /* afc_load_dsp */
    dbw()[W_CTRL_ABB] |= (1u << B_AFC);
    /* [2026-09-17] CLOSED AFC LOOP. qemu-src relays the d_afc write to the TWL3025
     * model (calypso_trx.c, offsets 0x001E/0x0046); qosmo does it NOWHERE
     * (set_afc_dac is never called there), so the loop stayed open, sample rotation
     * never moved, and the frequency error never converged below the SB threshold
     * (800 Hz). */
    calypso_twl3025_set_afc_dac((int16_t)afc_dac);
    if (api[NDB_ERRSTAT]) {                               /* as in sync.c:249 */
        static int n; if (trace && n < 6) { printf("  DSP Error Status: %u\n", api[NDB_ERRSTAT]); n++; }
        api[NDB_ERRSTAT] = 0;
    }
    /* Run the items of THIS frame. A callback may schedule a new one for the
     * CURRENT frame (delay=0, which is what the firmware computes after FB1), so
     * rescan until exhaustion or that item is silently lost -- and it is precisely
     * the SB command aiming at the right SCH frame. */
    for (int tour = 0; tour < 8; tour++) {
        int fait = 0;
        for (int i = 0; i < n_sched; i++) {
            if (sched[i].frame == (int)fn_cur && sched[i].cb) {
                cb_t cb = sched[i].cb; int at = sched[i].attempt;
                sched[i].cb = NULL;
                cb(at); fait = 1;
                if (verdict) return;
            }
        }
        if (!fait) break;
    }
    if (r_page_used) {
        memset(dbr(), 0, R_SIZE * sizeof(uint16_t));
        dbr()[R_SCH + 0] = (1u << B_SCH_CRC);
        r_page ^= 1;
    }
```

```
    api[NDB_PAGE] = (uint16_t)(B_GSM_TASK | w_page);      /* dsp_end_scenario() */
    w_page ^= 1;
}

int rejouer(C54xState *d, uint16_t *api_ram, long trames, long insns,
            const char *iq_mode, int amp, int bsic, int verbeux)
{
    dsp = d; api = api_ram; trace = verbeux;
    env_div = calypso_getenv("REJEU_DIV") != NULL;
    env_softs_continu = calypso_getenv("REJEU_SOFTS_CONTINU") != NULL;
    env_firs = calypso_getenv("REJEU_FIRS") != NULL;
    env_probe_t = drapeau_env("REJEU_PROBE_T");
    env_decodeur = calypso_getenv("REJEU_DECODEUR") != NULL;
    w_page = r_page = r_page_used = 0; fn_cur = 0; afc_dac = -700;
    fb_mode = 0; afc_retries = fb0_retries = 0; n_sched = 0; verdict = 0;
    n_fb_ok = n_sb_try = n_sb_crcfail = n_crc_ok = n_sb_vraies = 0;
    n_err_dsp = n_err8 = 0;
    g_bsic_injecte = bsic;
    { const char *r = calypso_getenv("REJEU_SCH_REEL");
      if (r && *r && reels_charger(r) == 0) g_bsic_injecte = g_reels_bsic[0]; }
    memset(&fb, 0, sizeof fb);

    if (drapeau_env("REJEU_SCH_PARTOUT")) { cellule_sch_partout = 1; printf("  [stimulus] SCH sur toutes les
     ↪  trames non-FCCH (masque le cadencage)\n"
               "  [stimulus] ATTENTION : t3p = p51/10 n'est juste que sur p51 in {1,11,21,31,41},\n"
               "               donc la FN reconstruite ne peut PAS etre validee sous ce drapeau.\n"); }
    printf("rejeu deterministe : %ld trames max, %ld insn/trame, cellule BSIC=%d, iq=%s\n",
           trames, insns, bsic, iq_mode ? iq_mode : "cell");
    /* firmware dsp_power_on(): bootloader boot + parameters + NDB */
    arm_dsp_init();
    { static int16_t rempl[2 * 148];         /* TS1..TS7 of the C0 carrier: dummy bursts */
      cellule_factice(amp, DECALAGE_SYMB, rempl);
      calypso_bsp_set_remplissage(rempl, 2 * 148); }
    plan_fb_set(1, 0);                       /* first FBSB_REQ */

    int16_t iq[2 * 256]; int n_iq;
    for (long t = 0; t < trames && !verdict; t++) {
        fn_cur = (uint32_t)t;
        g_c54x_exe_fn = fn_cur;
        uint32_t insn_debut_trame = dsp->insn_count;
        l1_sync();
        /* [2026-09-18] The firmware (sync.c) reads d_error_status every frame, prints
         * it and clears it; replay ignored it entirely, hence its silence about the
         * "DSP Error Status: 8" seen on the firmware side. 8 = DSP_ERR_DMA_PROG:
         * overflow of the DMA job ring at 0x4330 (orm *(0x3f92),#8 at 0xaa83). */
        if (api[NDB_ERRSTAT]) {
            unsigned e = api[NDB_ERRSTAT];
            n_err_dsp++;
            if (e & 8) n_err8++;
            if (n_err_dsp <= 8)
                printf("  [erreur DSP] status=%u%s a fn=%u\n", e,
                       (e & 8) ? " (bit 3 = DSP_ERR_DMA_PROG, anneau DMA 0x4330 sature)" : "",
                       fn_cur);
            api[NDB_ERRSTAT] = 0;
        }
        if (verdict) break;
        /* [2026-09-17] HARDWARE ORDER. On silicon: the ARM posts the task -> the DSP
         * reads it on the frame interrupt and ARMS its RX window (DMA) -> the samples
         * arrive -> the DSP processes them. Injecting BEFORE the DSP has armed makes
         * the transfer use the PREVIOUS task's DMA programming (the FB one), so the SB
         * receives the wrong burst. REJEU_RX_AVANT=1 restores the old order for
         * comparison. */
        static int rx_avant = -1;
        if (rx_avant < 0) rx_avant = drapeau_env("REJEU_RX_AVANT") ? 1 : 0;
        n_iq = 2 * 148;
        int injecter = (!iq_mode || strcmp(iq_mode, "none") != 0);
```

```
        if (injecter && rx_avant) {
            g_livre_type = cellule_burst(fn_cur, (uint8_t)g_bsic_injecte, amp, DECALAGE_SYMB, marge_tete(), iq,
↪  &n_iq);
            g_livre_fn = fn_cur; g_livre_n = n_iq;
            g_ad_avant = calypso_bsp_get_daram_addr();
            { static int da = -1; static unsigned nd;
              if (da < 0) da = calypso_getenv("REJEU_ADR") ? 1 : 0;
              if (da && nd < 14) { nd++;
                  printf("  [adr] fn=%-4u type=%c n_iq=%-4d -> depot 0x%04x len=%u\n",
                         fn_cur, g_livre_type ? g_livre_type : '?', n_iq,
                         calypso_bsp_get_daram_addr(), calypso_bsp_get_daram_len()); } }
            calypso_bsp_rx_burst(0, fn_cur, iq, n_iq);
        }
        /* frame interrupt: the DSP reads the task and arms its RX window.
         * [2026-09-20] REJEU_IRQ_SCENARIO=1: raise it only on frames where the
         * replayed ARM ended a DSP scenario (a task on the page just handed
         * over), as dsp_end_scenario() does with tpu_dsp_frameirq_enable(),
         * a bit the firmware sets again on every scenario. Every other frame
         * the ROM gets no frame interrupt and does not re-read the page. */
        { static int irq_sc = -1;
          if (irq_sc < 0) irq_sc = drapeau_env("REJEU_IRQ_SCENARIO") ? 1 : 0;
          unsigned wp_donnee = (api[NDB_PAGE] & 1u);
          bool scenario = api[W_PAGE(wp_donnee) + W_TASK_MD] != 0 || api[W_PAGE(wp_donnee) + 0] != 0;
          if ((dsp->imr & (1u << 12)) && (!irq_sc || scenario)) c54x_interrupt_ex(dsp, 28, 12); }
        if (dsp->idle && (dsp->ifr & dsp->imr) && !(dsp->st1 & 0x800)) dsp->idle = false;
        if (injecter && !rx_avant) {
            /* let the DSP arm (short budget), THEN deliver the samples */
            long arm = 0;
            while (arm < insns / 4 && dsp->running && !dsp->idle) {
                int ex = c54x_run(dsp, 64); if (ex <= 0) break; arm += ex;
            }
            g_livre_type = cellule_burst(fn_cur, (uint8_t)g_bsic_injecte, amp, DECALAGE_SYMB, marge_tete(), iq,
↪  &n_iq);
            /* [2026-09-20] FULL FRAME for the FB search. On silicon the FB task
             * streams 156.25 symbols per TDMA frame (148 burst + 8.25 guard); the
             * TOA it reports counts frames in those units and the firmware turns
             * it back into frames with BITS_PER_TDMA = 1250. Delivering 148
             * samples per frame made a 9-frame distance read as 5. Pad every
             * non-SCH burst with silence to 156 samples (157 on one frame in four,
             * so the average is 156.25). REJEU_TRAME_PLEINE=0 restores 148. */
            /* The SCH frame is delivered as the 190-sample WINDOW block (margins
             * 21/21) only while the DSP has its one-shot SB window armed; inside
             * an FB search (continuous DMA) it is a plain frame of the stream like
             * any other, or the stream would gain 34 symbols on every SCH frame.
             * The TOA origin (the firmware's "23") is not set here but by the DMA
             * model at arm time (CALYPSO_RHEA_DMA_ARM_SKIP): the burst position
             * inside the frame can only move within the 8.25 idle symbols. */
            { static int pleine = -1;
             if (pleine < 0) { const char *e = calypso_getenv("REJEU_TRAME_PLEINE"); pleine = (e && *e == '0') ? 0
↪  : 1; }
             bool fenetre_sb = calypso_rhea_dma_one_shot();
             if (pleine && !(g_livre_type == 'S' && fenetre_sb)) {
                 if (g_livre_type == 'S') {          /* drop the window margins: burst only */
                     int m = marge_tete();
                     memmove(iq, iq + 2 * m, 2 * 148 * sizeof(int16_t));
                     n_iq = 2 * 148;
                 }
                 int cible = 156 + ((fn_cur & 3) == 3 ? 1 : 0);
                 if (n_iq < 2 * cible) { memset(iq + n_iq, 0, (size_t)(2 * cible - n_iq) * sizeof(int16_t)); n_iq
↪  = 2 * cible; }
             } }
            g_livre_fn = fn_cur; g_livre_n = n_iq;
            /* Replace the SCH CONTENT with a real burst, without touching the framing. */
            if (g_n_reels && g_livre_type == 'S') {
                int m = marge_tete();
                unsigned r = g_reel_i % g_n_reels;
```

```
    { static double gn = -1;
      if (gn < 0) { const char *e = calypso_getenv("REJEU_REEL_GAIN"); gn = e ? atof(e) : 1.0;
                    if (gn <= 0) gn = 1.0; }
      if (gn == 1.0) memcpy(iq + 2 * m, g_reels[r], 296 * sizeof(int16_t));
      else for (int q = 0; q < 296; q++) {
          long v = lrint(g_reels[r][q] * gn);
          iq[2 * m + q] = (int16_t)(v > 32767 ? 32767 : v < -32768 ? -32768 : v); } }
    /* Check: is the burst dropped really the REAL one? */
    static unsigned nv;
    if (nv < 4) { nv++;
        printf("  [reel] trame %u <- burst #%u (fn reelle %u, BSIC %u) I/Q %d %d %d %d\n",
               fn_cur, r, g_reels_fn[r], g_reels_bsic[r],
               iq[2*m], iq[2*m+1], iq[2*m+2], iq[2*m+3]); }
    g_reel_pour_fn[fn_cur & 63] = (int)r;
    g_reel_i++;
}
/* [2026-09-18] Perturb ONE raw sample rather than a bit: this maps
 * influence index by index, with no confusion from burst packing nor from
 * the scheduling change a coded-bit flip causes. REJEU_PERTURBER_ECH=<n>
 * adds a delta to complex sample n of the delivered buffer. */
{ static int pe = -2; static long pf = -2;
  if (pe == -2) { const char *e = calypso_getenv("REJEU_PERTURBER_ECH"); pe = e ? atoi(e) : -1; }
  if (pf == -2) { const char *e = calypso_getenv("REJEU_PERTURBER_FN");  pf = e ? atol(e) : -1; }
  if (pe >= 0 && 2 * pe + 1 < n_iq && (pf < 0 || (long)fn_cur == pf)) {
      iq[2 * pe]     = (int16_t)(iq[2 * pe]     + 3000);
      iq[2 * pe + 1] = (int16_t)(iq[2 * pe + 1] - 3000);
  } }
/* [2026-09-18] DELIVERED SAMPLE CONVENTION. Transport is exact ([feed]
 * probe: 380/380 identical at shift 0), so if the feed is at fault it is the
 * CONVENTION, not the routing. Four mutually exclusive hypotheses:
 *   derot-  : the DSP expects a signal DEROTATED by -pi/2 per symbol
 *   derot+  : ... by +pi/2
 *   swap    : I and Q swapped
 *   conj    : Q negated (conjugate)
 * REJEU_FEED_XFORM=derot-|derot+|swap|conj|none (default none). */
{ static const char *xf = NULL; static int init = 0;
  if (!init) { xf = calypso_getenv("REJEU_FEED_XFORM"); init = 1; }
  if (xf && *xf && strcmp(xf, "none")) {
      int ns = n_iq / 2;
      if (!strcmp(xf, "swap")) {
          for (int k = 0; k < ns; k++) { int16_t t = iq[2*k]; iq[2*k] = iq[2*k+1]; iq[2*k+1] = t; }
      } else if (!strcmp(xf, "conj")) {
          for (int k = 0; k < ns; k++) iq[2*k+1] = (int16_t)(-iq[2*k+1]);
      } else if (!strcmp(xf, "derot-") || !strcmp(xf, "derot+")) {
          double sgn = (xf[5] == '-') ? -1.0 : 1.0;
          for (int k = 0; k < ns; k++) {
              double ph = sgn * (M_PI / 2.0) * k;
              double c = cos(ph), sn = sin(ph);
              double i0 = iq[2*k], q0 = iq[2*k+1];
              iq[2*k]   = (int16_t)lrint(i0 * c - q0 * sn);
              iq[2*k+1] = (int16_t)lrint(i0 * sn + q0 * c);
          }
      }
  } }
memcpy(g_livre_iq, iq, (size_t)n_iq * sizeof(int16_t)); g_livre_niq = n_iq;
{ static int da = -1; static unsigned nd;
  if (da < 0) da = calypso_getenv("REJEU_ADR") ? 1 : 0;
  if (da && nd < 14) { nd++;
      printf("  [adr] fn=%-4u type=%c n_iq=%-4d -> depot 0x%04x len=%u\n",
             fn_cur, g_livre_type ? g_livre_type : '?', n_iq,
             calypso_bsp_get_daram_addr(), calypso_bsp_get_daram_len()); } }
/* [2026-09-20] PAGE-BY-PAGE DELIVERY (REJEU_PAGES=<words>, e.g. 96).
 * On silicon the RIF streams continuously and DMA2 completes one
 * 96-word page at a time, each completion interrupting the DSP; the
 * FB correlator runs per page. Handing the whole frame in one call
 * drains 3-4 pages in one pass with ONE interrupt, so the ROM
```

```
             * processes one page per frame and its TOA advanced by 96 per frame
             * instead of 156 symbols. Here the frame's samples are delivered in
             * chunks, the DSP running a slice after each, so every page gets its
             * own completion. */
            { static long pages_mots = -1;
              if (pages_mots < 0) { const char *e = calypso_getenv("REJEU_PAGES"); pages_mots = (e && *e) ?
               ↪  atol(e) : 0; }
              if (pages_mots > 0) {
                  int pos = 0, npage = 0;
                  while (pos < n_iq) {
                      int m = n_iq - pos < pages_mots ? n_iq - pos : (int)pages_mots;
                      uint16_t b34 = dsp->data[0x3fb4], b35 = dsp->data[0x3fb5];
                      calypso_bsp_rx_burst(0, fn_cur, iq + pos, m);
                      pos += m; npage++;
                      { static unsigned nb; if (nb < 40 && fn_cur >= 2 && fn_cur <= 14) { nb++;
                         printf("  [bloc] fn=%u page %d (%d mots) : 0x3fb4=%04x 0x3fb5=%04x avant", fn_cur, npage,
↪  m, b34, b35); } }
                      /* completion interrupt already pending: let the DSP serve it */
                      if (dsp->idle && (dsp->ifr & dsp->imr) && !(dsp->st1 & 0x800)) dsp->idle = false;
                      long slice = 0;
                      while (slice < insns / 8 && dsp->running && !dsp->idle) {
                          int ex = c54x_run(dsp, 64); if (ex <= 0) break; slice += ex;
                      }
                      { static unsigned nb2; if (nb2 < 40 && fn_cur >= 2 && fn_cur <= 14) { nb2++;
                          printf(" -> apres 0x3fb4=%04x 0x3fb5=%04x (idle=%d, %ld insn)\n", dsp->data[0x3fb4],
↪  dsp->data[0x3fb5], dsp->idle, slice); } }
                  }
                  { static unsigned np; if (np < 3) { np++;
                      printf("  [pages] fn=%u : %d mots livres en %d pages de %ld\n", fn_cur, n_iq, npage,
↪  pages_mots); } }
              } else
                  calypso_bsp_rx_burst(0, fn_cur, iq, n_iq);
            }
            /* [2026-09-20] STREAM PUMP. The DMA now fills its double buffer with
             * full pages only and interrupts once per pair; the ROM's ISR consumes
             * both halves and the DSP goes idle. Then the next pair is handed
             * over, as the hardware would once the ISR is out of the way. A frame
             * carries 3.25 pages, so this runs 1 or 2 times per frame. */
            for (int k = 0; k < 40; k++) {          /* a 1250-symbol frame is 26 pages = 13 pairs */
                if (dsp->idle && (dsp->ifr & dsp->imr) && !(dsp->st1 & 0x800)) dsp->idle = false;
                long sl = 0;
                while (sl < insns / 4 && dsp->running && !dsp->idle) {
                    int ex = c54x_run(dsp, 64); if (ex <= 0) break; sl += ex;
                }
                if (!calypso_rhea_dma_pump(dsp)) break;
            }
            if (calypso_getenv("CALYPSO_BSP_VERIF")) {
                static int dit;
                if (!dit) { dit = 1;
                    printf("  [mem] api est-il un alias de data[0x0800] ? %s\n",
                           (void *)api == (void *)&dsp->data[0x0800] ? "OUI" : "NON — deux memoires distinctes");
                    printf("        api[0x0cce-0x0800]=%04x   data[0x0cce]=%04x\n",
                           api[0x0cce - 0x0800], dsp->data[0x0cce]); }
                static unsigned nvr;
                uint32_t vfn; uint16_t vad; int vn, vage;
                int id = calypso_bsp_verif_compare(&vfn, &vad, &vn, &vage);
                if (id >= 0 && nvr < 8) { nvr++;
                    printf("  [ref] fn=%-4u type=%c age=%d : %d/%d identiques en 0x%04x%s\n",
                           vfn, g_livre_type ? g_livre_type : '?', vage, id, vn, vad,
                           id == vn ? "   VALIDE" : "   <<< ECRITURE FAUSSE"); }
            }
            { static int vv = -1; static unsigned nv;
              if (vv < 0) vv = calypso_getenv("REJEU_VIE") ? 1 : 0;
              if (vv && g_livre_type == 'S' && nv < 6) { nv++;
                  uint16_t ad = calypso_bsp_get_daram_addr();
                  printf("  [vie] fn=%-4u adresse APRES l'appel : 0x%04x (avant : 0x%04x)\n",
```

```
                       fn_cur, ad, g_ad_avant);
                int ex = 0, best = 0, bex = -1;
                for (int sh = -8; sh <= 44; sh++) {
                    int e = 0;
                    for (int k = 0; k < 280; k++) {
                        int j = k + sh; if (j < 0 || j >= n_iq) continue;
                        if ((int16_t)dsp->data[(ad + k) & 0x3fff] == iq[j]) e++;
                    }
                    if (e > bex) { bex = e; best = sh; }
                    if (sh == 0) ex = e;
                }
               printf("  [vie] fn=%-4u APRES depot en 0x%04x : shift0=%d/280  meilleur shift=%+d avec %d/280\n",
                       fn_cur, ad, ex, best, bex);
                if (bex < 200) {   /* not there: locate the burst */
                    int ba = -1, bn = 0;
                    for (unsigned a = 0; a + 280 < 0x4000; a++) {
                        int e = 0;
                        for (int k = 0; k < 64; k++)
                            if ((int16_t)dsp->data[a + k] == iq[k]) e++;
                        if (e > bn) { bn = e; ba = (int)a; }
                    }
                    if (bn >= 60) {
                        int tot = 0;
                        for (int k = 0; k < 280; k++)
                            if ((int16_t)dsp->data[(ba + k) & 0x3fff] == iq[k]) tot++;
                        printf("        -> burst TROUVE en 0x%04x : %d/280 identiques\n", ba, tot);
                    } else printf("        -> burst introuvable en DARAM (meilleur %d/64 en 0x%04x)\n", bn, ba);
                }
                g_vie_fn = fn_cur; g_vie_ad = ad; } }
            if (dsp->idle && (dsp->ifr & dsp->imr) && !(dsp->st1 & 0x800)) dsp->idle = false;
        }
        { static int tf = -1; if (tf < 0) tf = calypso_getenv("REJEU_TRACE_FB") ? 1 : 0;
          if (tf && fn_cur < 60)
              printf("  [fb] fn=%-3u apres pompe : 0x3fb4=%04x 0x3fb3=%04x d_fb_det=%u fb_mode=%u task_md=%u/%u
↪  idle=%d insn_trame=%u  sync=%04x %04x %04x %04x\n",
                     fn_cur, dsp->data[0x3fb4], dsp->data[0x3fb3], api[NDB_FB_DET], api[NDB_FB_MODE],
                     api[W_PAGE(0) + W_TASK_MD], api[W_PAGE(1) + W_TASK_MD], dsp->idle, dsp->insn_count -
↪  insn_debut_trame,
                     api[NDB_SYNC], api[NDB_SYNC+1], api[NDB_SYNC+2], api[NDB_SYNC+3]); }
        long done = 0;
        static int probe = -1;
        if (probe < 0) probe = drapeau_env("REJEU_PROBE_TOA") ? 1 : 0;
        if (!probe) {
            /* Always count, step by step: running 256 instructions at a time without
             * looking at a single PC left hit_b219/7c31/84a1/9841/770a at zero while the
             * summary still printed them, so "job b219=0 ... corr FB 770a=0" only meant
             * that nobody was counting. The PC histogram says what the DSP really
             * executes. */
            while (done < insns && dsp->running && !dsp->idle) {
                uint16_t pc = dsp->pc & 0xffff;
                histo_pc[pc >> 10]++;
                switch (pc) {
                case 0x7c31: hit_7c31++; break;
                case 0x9841: hit_9841++; break;
                case 0x84a1: hit_84a1++; break;
                case 0x770a: hit_770a++; break;
                case 0xb219: hit_b219++; break;
                case 0x7a16: hit_7a16++; break;
                /* The two FIRS sites of the SB. The one at 0x8493 is followed by
                 * `sth *AR6+,B` at 0x8497: THAT is what writes the soft bits. If it does
                 * not run, the softs come out of a stale B. */
                case 0x⟨1⟩: hit_⟨2⟩++; break;  ×4
⟨⟩ = (8478,8478) (8492,8492) (8493,8493) (8497,8497)
                /* [2026-09-19] THE 16-STEP DIVISION. 0x7d1c RPT #15; 0x7d1d SUBC
                 * *(0x0b),A; 0x7d1e STL A,*(0x0a) = the quotient. A null divisor gives a
                 * degenerate quotient and the whole cascade follows. */
```

```
/* [2026-09-19] The dividend is born here. It should be 1 shifted
 * (ld #1,A; sfta A,<n>) and it arrives NULL at the division. Trace A over
 * the whole prologue to see which instruction zeroes it. */
case 0x7d10: case 0x7d11: case 0x7d12: case 0x7d13:
case 0x7d14: case 0x7d15: case 0x7d16: case 0x7d17:
case 0x7d18: case 0x7d19: case 0x7d1a: case 0x7d1b:
    if (env_div) {
        static int dumpe;
        if (!dumpe) { dumpe = 1;
            printf("    [rom] 0x7d10-0x7d20 :");
            for (unsigned a = 0x7d10; a <= 0x7d20; a++)
                printf(" %04x", prog_ovly(dsp, (uint16_t)a));
            printf("\n"); }
        static unsigned np;
        if (np < 26) { np++;
            printf("    [pro] pc=%04x op=%04x  A=%010llx  T=%04x ST0=%04x ST1=%04x\n",
                   pc, prog_ovly(dsp, pc),
                   (unsigned long long)(dsp->a & 0xfffffffffULL),
                   dsp->t, dsp->st0, dsp->st1); }
    }
    break;
case 0x7d1c: hit_7d1c++;
    if (env_div) {
        unsigned dp = dsp->st0 & 0x1FF;
        uint16_t dv = dsp->data[(uint16_t)((dp << 7) | 0x0B)];
        static unsigned nd;
        if (nd < 12) { nd++;
            printf("    [div] avant : dividende A=%010llx  diviseur=0x%04x (%d)%s\n",
                   (unsigned long long)(dsp->a & 0xfffffffffULL), dv, (int16_t)dv,
                   dv == 0 ? "   <<< DIVISEUR NUL" : ""); }
    }
    break;
case 0x7d1d: hit_7d1d++; break;
case 0x7d1e: hit_7d1e++;
    if (env_div) {
        static unsigned nq;
        if (nq < 12) { nq++;
            printf("    [div] apres : quotient A=%010llx  (mot bas = %d)\n",
                   (unsigned long long)(dsp->a & 0xfffffffffULL),
                   (int16_t)(dsp->a & 0xffff)); }
    }
    break;
case 0x81e4: hit_81e4++; break;
default: break; }
/* [2026-09-19] WHO WRITES a_sch? The status word a_sch[0] sometimes
 * receives plain numbers (0x1111, 0x1388=5000, 0x142e) where only B_BLUD
 * (bit15) and B_SCH_CRC (bit8) have a meaning -- and the 0x8000 read as
 * "CRC OK" is more likely a saturated accumulator. a_sch[0..4] =
 * R_PAGE+15.., i.e. data[0x0837..0x083b] (page 0) and
 * data[0x084b..0x084f] (page 1). Record the PC of each write.
 * REJEU_QUI_ASCH=1. */
{ static int qa = -1; static uint16_t sh[10]; static int ini; static unsigned nqa;
  if (qa < 0) qa = calypso_getenv("REJEU_QUI_ASCH") ? 1 : 0;
  if (qa) {
      static const uint16_t adr[10] = {0x0837,0x0838,0x0839,0x083a,0x083b,
                                       0x084b,0x084c,0x084d,0x084e,0x084f};
      if (!ini) { for (int k=0;k<10;k++) sh[k]=dsp->data[adr[k]]; ini=1; }
      for (int k=0;k<10;k++) {
          uint16_t v = dsp->data[adr[k]];
          if (v != sh[k]) {
              /* On a CRC OK, print the FULL word: eight events
               * carrying the same word are not eight independent
               * draws but one attractor reached eight times, which
               * changes the statistics completely. */
              if (k % 5 == 0 && v == 0x8000) {
                  const uint16_t *b = &dsp->data[adr[k]];
```

```
                printf("    [crcok] fn=%u  a_sch = %04x %04x %04x %04x %04x"
                       "  -> mot 0x%04x%04x  crc_interne(2bf8)=%u\n",
                       fn_cur, b[0], b[1], b[2], b[3], b[4],
                       b[4], b[3], dsp->data[0x2bf8]);
            }
            if (nqa < 4000) { nqa++;
                int j = k % 5;
                printf("    [asch] a_sch[%d] (page %d, 0x%04x) : %04x -> %04x"
                       "   ecrit juste avant pc=%04x  fn=%u%s\n",
                       j, k/5, adr[k], sh[k], v, pc, fn_cur,
                       (j==0 && v && v!=0x0100 && v!=0x8000 && v!=0x8100)
                         ? "   <<< PAS UN DRAPEAU" : ""); }
            sh[k] = v;
        }
    }
  } }
/* [2026-09-19] WHICH PC PINS A AT +-0x40000000 (A_high = +-16384)?
 * Everything else follows from it: the clipping at 0x2ac0, B zeroed by
 * the ADD at 0x8389, the degenerate softs. Record the PC of each
 * transition to that value. REJEU_QUI_A=1. */
{ static int qA = -1; static int64_t aprec; static unsigned long parpc[0x10000];
  static unsigned long tA; static int armA;
  if (qA < 0) qA = calypso_getenv("REJEU_QUI_A") ? 1 : 0;
  if (qA) {
      int64_t a40 = dsp->a & 0xfffffffffLL;
      int pin = (a40 == 0x0040000000LL || a40 == 0xffc0000000LL);
      int pin0 = (aprec == 0x0040000000LL || aprec == 0xffc0000000LL);
      static uint16_t pcp;
      if (pin && !pin0 && armA) parpc[pcp]++;
      aprec = a40; pcp = pc; armA = 1;
      if (++tA % 400000 == 0) {
          printf("  [quiA] PC qui amenent A a +-0x40000000 :\n");
          for (int rang=0; rang<8; rang++) {
              unsigned best=0; unsigned long bv=0;
              for (unsigned i=0;i<0x10000;i++) if (parpc[i]>bv){bv=parpc[i];best=i;}
              if (!bv) break;
              printf("    pc=%04x op=%04x : %lu fois\n", best,
                     prog_ovly(dsp,(uint16_t)best), bv);
              parpc[best]=0;
          }
          tA = 1;
      }
  } }
/* [2026-09-19] WHO WRITES THE +-16384 INTO 0x2ac0? Watch the buffer
 * word by word and record the PC of each write, separating clipped
 * values from the rest. REJEU_QUI2AC0=1. */
{ static int qc = -1; static uint16_t sh2[64]; static int ini3;
  static unsigned long par_pc_butee[0x10000], par_pc_autre[0x10000];
  static unsigned long tot2;
  if (qc < 0) qc = calypso_getenv("REJEU_QUI2AC0") ? 1 : 0;
  if (qc) {
      if (!ini3) { for (int k=0;k<64;k++) sh2[k]=dsp->data[0x2ac0+k]; ini3=1; }
      for (int k=0;k<64;k++) {
          uint16_t v = dsp->data[0x2ac0+k];
          if (v != sh2[k]) {
              int16_t sv = (int16_t)v;
              if (sv == 16384 || sv == -16384) par_pc_butee[pc]++;
              else par_pc_autre[pc]++;
              sh2[k] = v;
          }
      }
      if (++tot2 % 300000 == 0) {
          printf("  [2ac0] PC ecrivains (butee +-16384 | autres) :\n");
          for (int rang=0; rang<8; rang++) {
              unsigned best=0; unsigned long bv=0;
              for (unsigned i=0;i<0x10000;i++)
```

```
                            if (par_pc_butee[i]+par_pc_autre[i] > bv) { bv=par_pc_butee[i]+par_pc_autre[i];
                             ↪  best=i; }
                        if (!bv) break;
                        printf("    pc=%04x op=%04x : butee=%lu  autres=%lu\n",
                               best, prog_ovly(dsp,(uint16_t)best),
                               par_pc_butee[best], par_pc_autre[best]);
                        par_pc_butee[best]=0; par_pc_autre[best]=0;
                    }
                    tot2 = 1;
                }
              } }
            /* [2026-09-19] DOES THE WORKING BUFFER HOLD THE BURST? The injected
             * samples are known (g_livre_iq); after the copy, 0x2a00 and 0x2ac0
             * should carry the I and Q channels. Compare instead of assuming.
             * REJEU_CMP2A=1: at the first pass at 0x84a0 (correlator entry, so the
             * copy is done) on an SCH frame. */
            { static int cm = -1; static int fait2;
              if (cm < 0) cm = calypso_getenv("REJEU_CMP2A") ? 1 : 0;
              if (cm && fait2 < 3 && pc == 0x84a0 && g_n_reels &&
                  g_reel_pour_fn[g_sb_cmd_fn & 63] >= 0) {
                  fait2++;
                  int ri = g_reel_pour_fn[g_sb_cmd_fn & 63];
                  const int16_t *bi = g_reels[ri];
                  int m = 0;
                 printf("  [cmp] burst reel #%d (fn %u, BSIC %u) vs tampons de travail\n", ri, g_reels_fn[ri],
↪  g_reels_bsic[ri]);
                  printf("    I injecte : "); for (int k=0;k<10;k++) printf(" %6d", bi[2*(m+k)]);
                  printf("\n    0x2a00    : "); for (int k=0;k<10;k++) printf(" %6d",
↪  (int16_t)dsp->data[0x2a00+k]);
                  printf("\n    0x2ac0    : "); for (int k=0;k<10;k++) printf(" %6d",
↪  (int16_t)dsp->data[0x2ac0+k]);
                  printf("\n    Q injecte : "); for (int k=0;k<10;k++) printf(" %6d", bi[2*(m+k)+1]);
                  printf("\n");
                  /* normalized correlation between each buffer and each channel */
                  /* how far the constancy goes: how many words are identical? */
                  for (int buf=0; buf<2; buf++) {
                      uint16_t base = buf ? 0x2ac0 : 0x2a00;
                      int16_t v0 = (int16_t)dsp->data[base];
                      int same = 0, n = 0; int16_t mn=32767, mx=-32768;
                      for (int k=0;k<190;k++) { int16_t v=(int16_t)dsp->data[base+k];
                          if (v==v0) same++;
                          if(v<mn)mn=v;
                          if(v>mx)mx=v;
                          n++; }
                      printf("    0x%04x : %d/%d mots egaux au premier (%d) ; etendue [%d..%d]\n",
                             base, same, n, v0, mn, mx);
                  }
                  for (int buf=0; buf<2; buf++) {
                      uint16_t base = buf ? 0x2ac0 : 0x2a00;
                      for (int voie=0; voie<2; voie++) {
                          double sxy=0, sxx=0, syy=0;
                          for (int k=0;k<128;k++) {
                              double x=(int16_t)dsp->data[base+k], y=bi[2*(m+k)+voie];
                              sxy+=x*y; sxx+=x*x; syy+=y*y; }
                          printf("    correlation 0x%04x vs %c : %+.3f\n", base, voie?'Q':'I',
                                 (sxx>0&&syy>0)? sxy/sqrt(sxx*syy) : 0.0);
                      }
                  }
              } }
            /* [2026-09-19] THE burst -> working buffer COPY, the last link never
             * examined: the burst arrives exact at 0x0cce and buffer 0x2a80 is
             * filled by 0x81d0..0x81da. What does that loop READ? */
            { static int rc = -1; static unsigned n;
              if (rc < 0) rc = calypso_getenv("REJEU_RECOPIE") ? 1 : 0;
              if (rc && pc >= 0x81c8 && pc <= 0x81e0) {
                  /* Does AR5 sweep a table, or stay on two cells? */
```

```
                    static unsigned lo = 0xffff, hi = 0, vus[64], nv;
                    if (dsp->ar[5] < lo) lo = dsp->ar[5];
                    if (dsp->ar[5] > hi) hi = dsp->ar[5];
                    { int trouve = 0; for (unsigned q = 0; q < nv; q++) if (vus[q] == dsp->ar[5]) trouve = 1;
                      if (!trouve && nv < 64) vus[nv++] = dsp->ar[5]; }
                    /* Does the multiplicand change? A phasor kept in place would
                     * change value at every step; a constant would not. */
                    { static unsigned long nval; static uint16_t vu_v[32]; static unsigned nvv;
                      uint16_t v = dsp->data[dsp->ar[5] & 0x3fff];
                      int t2 = 0; for (unsigned q = 0; q < nvv; q++) if (vu_v[q] == v) t2 = 1;
                      if (!t2 && nvv < 32) vu_v[nvv++] = v;
                      if (++nval % 4000 == 0) {
                          printf("    [mul] %u valeurs distinctes lues via AR5 :", nvv);
                          for (unsigned q = 0; q < nvv && q < 12; q++) printf(" %04x", vu_v[q]);
                          printf("\n"); } }
                    static unsigned long tot;
                    if (++tot % 4000 == 0)
                        printf("    [ar5] apres %lu pas : plage 0x%04x..0x%04x, %u cellules distinctes\n",
                               tot, lo, hi, nv);
                }
                if (rc && n < 16 && pc >= 0x81c8 && pc <= 0x81e0) {
                    n++;
                    printf("    [cp] pc=%04x op=%04x  A=%010llx B=%010llx"
                           "  AR2=%04x->%04x AR3=%04x->%04x AR4=%04x->%04x AR5=%04x->%04x\n",
                           pc, prog_ovly(dsp, pc),
                           (unsigned long long)(dsp->a & 0xffffffffffULL),
                           (unsigned long long)(dsp->b & 0xffffffffffULL),
                           dsp->ar[(1)], dsp->data[dsp->ar[(2)] & 0x3fff],  ×3
⟨⟩ = (2,2) (3,3) (4,4)
                           dsp->ar[5], dsp->data[dsp->ar[5] & 0x3fff]);
                } }
              /* [2026-09-19] THE BLOCK THAT OVERWRITES THE LOWER HALF OF THE FIRS
               * WINDOW. 0x832b..0x8330 writes 0000 x4 then 4000 x2 there. Boundary
               * conditions, or clobbering? Print the program words of the area and the
               * registers on entry, once. REJEU_DUMP832=1. */
              { static int d8 = -1; static int fait;
                if (d8 < 0) d8 = calypso_getenv("REJEU_DUMP832") ? 1 : 0;
                if (d8 && !fait && pc == 0x8320) {
                    fait = 1;
                    printf("  [832] programme 0x8318-0x8340 (alias OVLY compris) :\n");
                    for (unsigned a = 0x8318; a <= 0x8340; a += 8) {
                        printf("    %04x:", a);
                        for (int k = 0; k < 8 && a + k <= 0x8340; k++)
                            printf(" %04x", prog_ovly(dsp, (uint16_t)(a + k)));
                        printf("\n");
                    }
                    printf("    registres : A=%010llx B=%010llx T=%04x\n",
                           (unsigned long long)(dsp->a & 0xffffffffffULL),
                           (unsigned long long)(dsp->b & 0xffffffffffULL), dsp->t);
                    printf("    AR0=%04x AR1=%04x AR2=%04x AR3=%04x AR4=%04x AR5=%04x AR6=%04x AR7=%04x\n",
                           dsp->ar[0],dsp->ar[1],dsp->ar[2],dsp->ar[3],
                           dsp->ar[4],dsp->ar[5],dsp->ar[6],dsp->ar[7]);
                } }
              /* [2026-09-19] THE CORRELATOR BLOCK, PC BY PC, OVER A SINGLE DECODE.
               * 3100 MACs per decode reads either as "50 x 62" or "31 x 100"; only a
               * per-pass count decides. Count each PC in 0x84a0..0x84d0 during the
               * FIRST decode, then print. REJEU_BLOC=1. */
              { static int bl = -1; static unsigned long cnt[0x40]; static int fini, vu_dec;
                if (bl < 0) bl = calypso_getenv("REJEU_BLOC") ? 1 : 0;
                if (bl && !fini) {
                    if (pc >= 0x84a0 && pc <= 0x84df) cnt[pc - 0x84a0]++;
                    if (pc == 0x9841) {
                        if (!vu_dec) { vu_dec = 1; }
                        else {
                            fini = 1;
                            printf("  [bloc] correlateur 0x84a0-0x84df sur UN decodage :\n");
```

```
            for (int k = 0; k < 0x40; k++)
                if (cnt[k]) printf("    pc=%04x  op=%04x  %6lu fois\n",
                                   0x84a0 + k, prog_ovly(dsp, 0x84a0 + k), cnt[k]);
        }
    }
  } }
/* [2026-09-19] IMPULSE RESPONSE OF 0x2c72. Force ONE position to an
 * extreme value, held over the whole window, and compare the output
 * between +v and -v. How many decoded bits move:
 *   1 position       -> a permutation, 0x2c72 really is one soft per bit
 *   3 or 4 adjacent  -> normal ISI, the chain is sound here
 *   all / none       -> these are not per-bit softs
 * REJEU_IMPULSION=<k> REJEU_IMPULSION_VAL=<v>. */
{ static int ik = -2, iv;
  if (ik == -2) { const char *e = calypso_getenv("REJEU_IMPULSION");
                  ik = e ? atoi(e) : -1;
                  const char *w = calypso_getenv("REJEU_IMPULSION_VAL");
                  iv = w ? atoi(w) : 20000; }
  if (ik >= 0 && ik < 78 && g_dans_sb)
      dsp->data[0x2c72 + ik] = (uint16_t)(int16_t)iv; }
/* [2026-09-19] IS THE PEAK USED AT ALL? The addresses FIRS reads do
 * not move when the burst moves, while the peak does follow. Force the
 * peak to an arbitrary value over the whole demodulation window: if
 * nothing downstream changes, it is not consumed and the equalizer is
 * aligned on nothing. REJEU_FORCER_PIC=<n>. */
{ static int fp = -2;
  if (fp == -2) { const char *e = calypso_getenv("REJEU_FORCER_PIC"); fp = e ? atoi(e) : -1; }
  if (fp >= 0 && g_dans_sb) dsp->data[0x2f06] = (uint16_t)fp; }
/* [2026-09-19] PERFECT SOFTS. The fault lies somewhere between the
 * correlator (sound) and the soft-bit write. Split the space in two:
 * just before the decoder (0x9841) reads 0x2c72, write the ideal soft
 * bits there ourselves, derived from the 78 bits the injected burst
 * REALLY carries. If the DSP then returns BSIC=32, everything downstream
 * (Viterbi, CRC, a_sch packing) is proven and the fault is strictly in
 * soft PRODUCTION; otherwise it is downstream.
 * REJEU_SOFTS_PARFAITS=<amplitude>, sign tested both ways via
 * REJEU_SOFTS_POLARITE=0|1. REJEU_SOFTS_CONTINU=1 holds the ideal softs
 * at EVERY step of the demodulation window, not only at the decoder
 * entry -- otherwise a read before 0x9841 escapes the injection and "no
 * effect" means nothing. */
if (pc == 0x9841 || (env_softs_continu && g_dans_sb)) {
    static int amp = -2, pol = -1;
    if (amp == -2) { const char *e = calypso_getenv("REJEU_SOFTS_PARFAITS");
                     amp = e ? atoi(e) : -1;
                     const char *q = calypso_getenv("REJEU_SOFTS_POLARITE");
                     pol = q ? atoi(q) : 0; }
    if (amp > 0) {
        int ri = g_n_reels ? g_reel_pour_fn[g_sb_cmd_fn & 63] : -1;
        unsigned char att[78];
        if (ri >= 0) memcpy(att, g_reels_code[ri], 78);
        else cellule_code_attendu(g_sb_cmd_fn, (uint8_t)g_bsic_injecte, att);
        /* REJEU_SOFTS_PERM: the order of the 78 values is a hypothesis.
         * none = as is; swap = the two halves of 39 exchanged;
         * rev = reversed order; entrelace = even/odd separated. */
        static const char *perm; static int perm_lu;
        if (!perm_lu) { perm = calypso_getenv("REJEU_SOFTS_PERM"); perm_lu = 1; }
        /* [2026-09-20] The decoder reads its 78 softs at 0x2a00
         * (ROM 0x984a `stm #0x2a00,AR1`; packing 0x7e65-0x7e7a),
         * NOT at 0x2c72. Writing 0x2c72 tested nothing.
         * REJEU_SOFTS_ADDR overrides (default 0x2a00). */
        static long sa = -1;
        if (sa < 0) { const char *e = calypso_getenv("REJEU_SOFTS_ADDR");
                      sa = (e && *e) ? strtol(e, NULL, 0) : 0x2a00; }
        for (int k = 0; k < 78; k++) {
            int j = k;
            if (perm && !strcmp(perm, "swap"))       j = (k < 39) ? k + 39 : k - 39;
```

```
            else if (perm && !strcmp(perm, "rev"))   j = 77 - k;
            else if (perm && !strcmp(perm, "entrelace")) j = (k < 39) ? 2*k : 2*(k-39)+1;
            int bit = att[j] & 1;
            int v = (pol ? bit : !bit) ? amp : -amp;
            dsp->data[(uint16_t)(sa + k)] = (uint16_t)(int16_t)v;
        }
        static unsigned ns; if (ns < 3) { ns++;
            printf("  [softs] fn_demod=%u : 78 souples IDEAUX ecrits en 0x%04lx "
                   "(amp=%d pol=%d, source=%s)\n", g_sb_cmd_fn, sa, amp, pol,
                   ri >= 0 ? "burst reel" : "fixture"); }
    }
}
/* [2026-09-19] WHO WRITES THE EQUALIZER INPUT WINDOW? FIRS reads
 * 0x2a8e..0x2a9a and its lower half is zero or 0x4000. Watch the area
 * word by word and record the PC of each write: that says who fills it,
 * in what order, and where the filling stops. REJEU_QUI2A=1. */
{ static int q2 = -1; static uint16_t shadow[0x30]; static int init2;
  static unsigned nq;
  if (q2 < 0) q2 = calypso_getenv("REJEU_QUI2A") ? 1 : 0;
  if (q2) {
      if (!init2) { for (int k = 0; k < 0x30; k++) shadow[k] = dsp->data[0x2a80 + k]; init2 = 1; }
      for (int k = 0; k < 0x30; k++) {
          uint16_t v = dsp->data[0x2a80 + k];
          if (v != shadow[k]) {
              if (nq < 4000) { nq++;
                  printf("    [qui] 0x%04x : %04x -> %04x   ecrit juste avant pc=%04x (fn=%u)\n",
                         0x2a80 + k, shadow[k], v, pc, fn_cur); }
              shadow[k] = v;
          }
      }
  } }
/* [2026-09-19] WHO FILLS B? The correlator is sound (the peak follows
 * the margin), FIRS carries nothing, and yet `sth *AR6+,B` at 0x8497 is
 * what writes the soft bits. Trace A and B over the whole window
 * 0x8470..0x84a0 to see where B takes its value. REJEU_TRACE_B=1. */
/* 0x847c / 0x8498 : op=0x4485 = LD Smem,16,A per tic54x-opc.c
 * (0x4400/0xFE00). No handler matches this mask in c54x_exec.c,
 * yet A changes across it. Dump A before/after and every AR with
 * the word it points at, to find where the value comes from. */
/* Dump the DSP work buffers at the correlator entry, together with
 * the real burst that was fed, so an external model can identify what
 * each buffer holds instead of guessing. REJEU_DUMP_BUF=<path>. */
/* Who writes the BSP deposit window 0x0cce, and when? At correlator
 * entry the window holds none of the injected bursts and its content
 * does not change between frames. Shadow the window, log each change
 * with the PC that made it. REJEU_QUI_CCE=1. */
/* Timeline of the deposit window on one SB frame: every change with
 * the PC and the instruction count, plus the correlator entry. Tells
 * whether the burst is destroyed before or after the SB reads it.
 * REJEU_CHRONO=<fn>. */
{ static long cf = -2; static uint16_t sh2[380]; static int ini2; static unsigned nl;
  if (cf == -2) { const char *e = calypso_getenv("REJEU_CHRONO"); cf = e ? atol(e) : -1; }
  if (cf >= 0 && (long)fn_cur == cf) {
      if (!ini2) { for (int k=0;k<380;k++) sh2[k]=dsp->data[0x0cce + k]; ini2=1;
                   printf("  [chrono] trame %ld\n", cf); }
      unsigned chg=0;
      for (int k=0;k<380;k++) { uint16_t v=dsp->data[0x0cce + k];
          if (v!=sh2[k]) { chg++; sh2[k]=v; } }
      if (chg && nl < 24) { nl++;
          printf("      insn=%-8u pc=%04x  %4u mots changes\n",
                 dsp->insn_count, pc, chg); }
      if (pc == 0x84a0 && nl < 30) { nl++;
          printf("      insn=%-8u pc=84a0  <== ENTREE DU CORRELATEUR SB\n",
                 dsp->insn_count); }
      if (pc == 0x7c31 && nl < 30) { nl++;
          printf("      insn=%-8u pc=7c31  <== entree du demodulateur SB\n",
```

```
                dsp->insn_count); }
  } }
{ static int qc = -1; static uint16_t sh[380]; static int ini;
  static unsigned long par_pc[0x10000], tot; static unsigned long nfr[64];
  if (qc < 0) qc = calypso_getenv("REJEU_QUI_CCE") ? 1 : 0;
  if (qc) {
      if (!ini) { for (int k=0;k<380;k++) sh[k]=dsp->data[0x0cce + k]; ini=1; }
      unsigned chg = 0;
      for (int k=0;k<380;k++) {
          uint16_t v = dsp->data[0x0cce + k];
          if (v != sh[k]) { chg++; sh[k]=v; }
      }
      if (chg) { par_pc[pc] += chg; nfr[fn_cur & 63] += chg; }
      if (++tot % 500000 == 0) {
          printf("  [cce] ecrivains de 0x0cce (mots modifies) :\n");
          for (int r=0;r<6;r++) {
              unsigned best=0; unsigned long bv=0;
              for (unsigned i=0;i<0x10000;i++) if (par_pc[i]>bv){bv=par_pc[i];best=i;}
              if (!bv) break;
              printf("      pc=%04x op=%04x : %lu mots\n", best,
                     prog_ovly(dsp,(uint16_t)best), bv);
              par_pc[best]=0;
          }
          tot=1;
      }
  } }
{ static int vv2 = -1; static unsigned nv2;
  if (vv2 < 0) vv2 = calypso_getenv("REJEU_VIE") ? 1 : 0;
  if (vv2 && pc == 0x84a0 && g_vie_ad && nv2 < 5) { nv2++;
      int ex = 0;
      for (int k = 0; k < 296; k++)
          if ((int16_t)dsp->data[(g_vie_ad + k) & 0x3fff] == g_livre_iq[2*marge_tete() + k]) ex++;
      printf("  [vie] fn=%-4u ENTREE CORRELATEUR (depot de fn=%u) : %d/296 identiques\n",
             fn_cur, g_vie_fn, ex); } }
{ static int db = -1; static FILE *fb;
  if (db < 0) { const char *e = calypso_getenv("REJEU_DUMP_BUF");
                db = e ? 1 : 0; if (db) fb = fopen(e, "wb"); }
  if (db && fb && pc == 0x84a0 && g_n_reels) {
      int ri = g_reel_pour_fn[g_sb_cmd_fn & 63];
      if (ri >= 0) {
          static int nb;
          if (nb < 8) { nb++;
              uint32_t hdr[4] = { g_sb_cmd_fn, (uint32_t)ri,
                                  g_reels_fn[ri], g_reels_bsic[ri] };
              fwrite(hdr, 4, 4, fb);
              fwrite(g_reels[ri], 2, 296, fb);          /* injected burst */
              fwrite(&dsp->data[0x0cce], 2, 380, fb);   /* BSP deposit */
              fwrite(&dsp->data[0x2a00], 2, 256, fb);   /* buffer 1 */
              fwrite(&dsp->data[0x2ac0], 2, 256, fb);   /* buffer 2 */
              fwrite(&dsp->data[0x2c72], 2, 78, fb);    /* soft bits */
              fwrite(g_reels_code[ri], 1, 78, fb);      /* transmitted bits */
              fflush(fb);
          }
      }
  } }
{ static int q4 = -1; static unsigned n4; static int64_t avant; static int arme;
  if (q4 < 0) q4 = calypso_getenv("REJEU_Q4485") ? 1 : 0;
  if (q4) {
      if (arme) { arme = 0;
          printf("        -> A apres = %010llx\n",
                 (unsigned long long)(dsp->a & 0xfffffffffULL)); }
      if ((pc == 0x847c || pc == 0x8498) && n4 < 6) {
          n4++; avant = dsp->a; arme = 1;
          printf("    [4485] pc=%04x op=%04x  A avant = %010llx\n",
                 pc, prog_ovly(dsp, pc), (unsigned long long)(avant & 0xfffffffffULL));
          for (int k = 0; k < 8; k++)
```

```
                printf("        AR%d=%04x -> %04x\n", k, dsp->ar[k],
                       dsp->data[dsp->ar[k] & 0x3fff]);
        }
    } }
{ static int tb = -1; static unsigned ntb;
    if (tb < 0) tb = calypso_getenv("REJEU_TRACE_B") ? 1 : 0;
    if (tb && ntb < 70 && pc >= 0x8470 && pc <= 0x84a0) {
        ntb++;
        printf("    [B] pc=%04x op=%04x A=%010llx B=%010llx AR2=%04x->%04x AR3=%04x->%04x\n",
               pc, prog_ovly(dsp, pc),
               (unsigned long long)(dsp->a & 0xfffffffffULL),
               (unsigned long long)(dsp->b & 0xfffffffffULL),
               dsp->ar[2], dsp->data[dsp->ar[2] & 0x3fff],
               dsp->ar[3], dsp->data[dsp->ar[3] & 0x3fff]);
    } }
{ uint16_t o = prog_ovly(dsp, pc); uint8_t h = o >> 8;
    static int dans_sb2;
    if (pc == 0x7c31) dans_sb2 = 1;
    if (pc == 0x9841) dans_sb2 = 0;
    g_dans_sb = dans_sb2;
    /* [2026-09-18] WHAT FIRS MULTIPLIES. FIRS (0xE0, 2 words) takes its
     * pmad from the following word and reads its coefficients in PROGRAM
     * space. Print pmad, the coefficients as the core sees them (OVLY alias
     * included) and data[] at the same address: null or constant
     * coefficients mean the equalizer output CANNOT depend on its input,
     * which would be the root cause. */
    if (env_firs && dans_sb2 && h == 0xE0) {
        static unsigned nf;
        if (nf < 10) {
            uint16_t pmad = prog_ovly(dsp, (uint16_t)(pc + 1));
            printf("    [firs] #%u pc=%04x pmad=%04x  coef(prog+ovly)=", ++nf, pc, pmad);
            for (int k = 0; k < 6; k++) printf(" %04x", prog_ovly(dsp, (uint16_t)(pmad + k)));
            printf("   data[pmad..]=");
            for (int k = 0; k < 6; k++) printf(" %04x", dsp->data[(pmad + k) & 0x3fff]);
            printf("\n");
        }
    }
    if (pc >= 0x9800 && pc <= 0x9bff) g_op_dec[o]++;
    if (pc >= 0x8400 && pc <= 0x84ff) g_op_eq[o]++;
    if (pc == 0x9841) g_n_dec++;
    if (dans_sb2) {
        if (g_op_n[o]++ == 0) g_op_pc[o] = pc;
        if (h == 0x8E || h == 0x8F) n_cmps_sb++;
        if (h >= 0xE0 && h <= 0xE3) { n_e0_sb++; n_e0x_sb[h - 0xE0]++; }
        /* [2026-09-18] FIRS reads its coefficients at Pmem[pmad], with
         * pmad ~ 0x0061. But 0x0060-0x007F is the C54x DARAM SCRATCH-PAD
         * and the core's OVLY window only starts at 0x0080, so
         * prog_read(0x61) falls back on prog[] where nothing is loaded
         * below 0x7000. If the coefficients only live in data[], FIRS
         * multiplies by nothing and its output cannot depend on its input.
         * Read BOTH spaces at the FIRS. */
        if (h == 0xE0 && n_firs_vus < 6) {
            uint16_t pmad = dsp->prog[(pc + 1) & 0xffff];
            int nzp = 0, nzd = 0;
            for (int k = 0; k < 6; k++) {
                if (dsp->prog[(pmad + k) & 0xffff]) nzp++;
                if (dsp->data[(pmad + k) & 0x3fff]) nzd++;
            }
            /* [2026-09-18] Reading prog[] RAW is misleading: with
             * PMST_OVLY set and the alias floor at 0x0060 (gate
             * CALYPSO_OVLY_SCRATCH, default 1), a PROGRAM read in
             * 0x0060-0x27FF is redirected to data[]. Reproduce the core's
             * translation to know what FIRS REALLY reads, rather than what
             * the prog[] array holds. */
            int ovly = (dsp->pmst & 0x0020) != 0;
            int alias = ovly && pmad >= 0x0060 && pmad < 0x2800;
```

```
            printf("    [firs] pc=%04x pmad=%04x PMST=%04x OVLY=%d alias=%s"
                   " -> FIRS lit", pc, pmad, dsp->pmst, ovly,
                   alias ? "data[] (scratch-pad visible)" : "prog[] (PAS d'alias)");
            for (int k = 0; k < 6; k++)
                printf(" %04x", alias ? dsp->data[(pmad + k) & 0x3fff]
                                      : dsp->prog[(pmad + k) & 0xffff]);
            printf("\n");
            printf("    [firs] pc=%04x pmad=%04x | prog[pmad..+5]=", pc, pmad);
            for (int k = 0; k < 6; k++) printf(" %04x", dsp->prog[(pmad + k) & 0xffff]);
            printf(" (%d non nuls)\n                        | data[pmad..+5]=", nzp);
            for (int k = 0; k < 6; k++) printf(" %04x", dsp->data[(pmad + k) & 0x3fff]);
            printf(" (%d non nuls)\n", nzd);
            n_firs_vus++;
        }
    } else if (h == 0x8E || h == 0x8F) n_cmps_hors++;
    /* [2026-09-18] The dans_sb window closes at 0x9841, so it EXCLUDES
     * the SCH decoder and its Viterbi (0x9a78): "0 CMPS in the SB demod"
     * said nothing about the Viterbi. CMPS is therefore also counted per PC
     * region, with no window, which is unambiguous. */
    if (h == 0x8E || h == 0x8F) n_cmps_reg[pc >> 10]++;
    /* [2026-09-18] WHO WRITES 0x2a00? The decoder reads 78 words at
     * 0x2a00 and finds them ALL NULL on entry: either the equalization
     * stage never writes them, or something erases them first. Watch
     * 0x2a00..0x2a8d over the WHOLE SB job (not just the demod), recording
     * the PC of each change and its direction (to a value, or to zero). */
/* [2026-09-18] 0x8389 = 0x4594 = `ADD *AR4+,16,A,B` (0x4400/0xFC00,
 * bit9=src, bit8=dst). The core has NO handler for 0x4400-0x47FF: the
 * only one in the area is (op & 0xFC00) == 0x4000, which covers SUB
 * alone. Check that B really changes across that instruction. */
{ static int nav; static int64_t bavant;
    if (pc == 0x8389) { bavant = dsp->b; nav = 1; }
    else if (nav == 1 && pc == 0x838a) {
        static int n=0;
        if (n < 4) {
            printf("    [add-4594] B avant=%010llx  B apres=%010llx  A=%010llx  %s\n",
                   (unsigned long long)(bavant & 0xfffffffffULL),
                   (unsigned long long)(dsp->b & 0xfffffffffULL),
                   (unsigned long long)(dsp->a & 0xfffffffffULL),
                   (dsp->b == bavant) ? "B INCHANGE => instruction NON EXECUTEE" : "B modifie");
            n++;
        }
        nav = 0;
    } }
{ static int nb838d;
    if (pc == 0x838e && nb838d < 6) {
        printf("    [B@838d] B=%010llx  A=%010llx  AR6=%04x  BRC=%u  ST1=%04x\n",
               (unsigned long long)(dsp->b & 0xfffffffffULL),
               (unsigned long long)(dsp->a & 0xfffffffffULL),
               dsp->ar[6], dsp->brc, dsp->st1);
        nb838d++;
    } }
{ static uint16_t omb2[142]; static int arme6; static long ecr[64], vers0[64];
    static uint16_t pmin[64], pmax[64];
    static struct { uint16_t pc; long v, z; } parpc2[12];
    if (pc == 0xb219) {
        for (int k = 0; k < 142; k++) omb2[k] = dsp->data[0x2a00 + k];
        arme6 = 1;
        for (int b = 0; b < 64; b++) { ecr[b] = 0; vers0[b] = 0; }
        for (int t = 0; t < 12; t++) { parpc2[t].pc = 0; parpc2[t].v = 0; parpc2[t].z = 0; }
    }
    if (arme6) {
        for (int k = 0; k < 142; k++) {
            if (dsp->data[0x2a00 + k] != omb2[k]) {
                /* [2026-09-18] Bucket by EXACT PC, and separate value
                 * writes from zeroing: 284 changes of which 142 to zero
                 * means something writes the 142 soft bits then ERASES
```

```
                 * them all, and both instructions must be named. The PC
                 * printed is that of the FOLLOWING instruction. */
                { int z = (dsp->data[0x2a00 + k] == 0);
                  for (int t = 0; t < 12; t++) {
                      if (parpc2[t].pc == 0 || parpc2[t].pc == pc) {
                          parpc2[t].pc = pc;
                          if (z) parpc2[t].z++; else parpc2[t].v++;
                          break;
                      }
                  } }
                int b = pc >> 10;
                if (!ecr[b]) { pmin[b] = pc; pmax[b] = pc; }
                if (pc < pmin[b]) pmin[b] = pc;
                if (pc > pmax[b]) pmax[b] = pc;
                ecr[b]++;
                if (dsp->data[0x2a00 + k] == 0) vers0[b]++;
                omb2[k] = dsp->data[0x2a00 + k];
            }
        }
    }
    if (pc == 0x9841 && arme6 && n_2a0 < 3) {
        printf("    [qui-2a00] modifications de 0x2a00..0x2a8d pendant le job SB :\n");
        long tot = 0;
        for (int b = 0; b < 64; b++)
            if (ecr[b]) {
              printf("       region 0x%04x : %ld modifs (dont %ld vers zero), PC 0x%04x a 0x%04x\n",
                     b << 10, ecr[b], vers0[b], pmin[b], pmax[b]);
                tot += ecr[b];
            }
        if (!tot) printf("        AUCUNE : rien n'ecrit jamais dans 0x2a00\n");
        printf("        par PC exact (PC imprime = instruction SUIVANTE) :\n");
        for (int t = 0; t < 12; t++)
            if (parpc2[t].pc)
                printf("           pc=%04x (donc ecrivain %04x) : %ld valeurs, %ld mises a ZERO\n",
                       parpc2[t].pc, parpc2[t].pc - 1, parpc2[t].v, parpc2[t].z);
        n_2a0++;
    } }
/* [2026-09-18] WHO WRITES THE 78 SOFT BITS, AND AT WHICH ADDRESS?
 * Sweeping the 78 coded bits shows the first half landing at position b+3
 * and the second at TWO positions 51 apart (b-38 and b+13), i.e. the two
 * data blocks OVERLAP instead of concatenating. Shadow the area and
 * record the PC as soon as its content changes. */
{ static uint16_t ombre[78]; static int arme5; static long parpc[64];
  static uint16_t pcmin[64], pcmax[64]; static long seq;
  if (dans_sb2) {
     if (!arme5) { for (int k = 0; k < 78; k++) ombre[k] = dsp->data[0x2c72 + k]; arme5 = 1; seq =
      ↪  0; }
      for (int k = 0; k < 78; k++) {
          if (dsp->data[0x2c72 + k] != ombre[k]) {
              int b = pc >> 10;
              if (!parpc[b]) { pcmin[b] = pc; pcmax[b] = pc; }
              if (pc < pcmin[b]) pcmin[b] = pc;
              if (pc > pcmax[b]) pcmax[b] = pc;
              parpc[b]++;
              /* [2026-09-18] EXACT CHRONOLOGY: sequence number, PC,
               * index touched, old and new value. Tells (a) in which
               * ORDER the two passes write, hence which destroys which,
               * and (b) whether two writes at the same index carry the
               * SAME value (mis-addressed copy) or DIFFERENT ones (two
               * halves of a sum that should have landed together). */
              if (n_ecr == 0 && seq < 100)
                  printf("    [w] %3ld pc=%04x k=%2d  %04x -> %04x  "
                         "AR0=%04x AR1=%04x AR2=%04x AR3=%04x AR4=%04x AR5=%04x AR6=%04x AR7=%04x
                          ↪  BRC=%u\n",
                         seq, pc, k, ombre[k], dsp->data[0x2c72 + k],
                         dsp->ar[0], dsp->ar[1], dsp->ar[2], dsp->ar[3],
```

```
                        dsp->ar[4], dsp->ar[5], dsp->ar[6], dsp->ar[7], dsp->brc);
                seq++;
                ombre[k] = dsp->data[0x2c72 + k];
            }
        }
    } else if (arme5 && n_ecr < 2) {
        printf("    [ecrit-2c72] sites qui modifient les 78 bits souples :\n");
        for (int b = 0; b < 64; b++)
            if (parpc[b])
                printf("        region 0x%04x : %ld ecritures, PC de 0x%04x a 0x%04x\n",
                       b << 10, parpc[b], pcmin[b], pcmax[b]);
        n_ecr++; arme5 = 0;
        for (int b = 0; b < 64; b++) parpc[b] = 0;
    } }
/* [2026-09-18] WHY THE EDGES ARE LOST. The max|delta| profile shows
 * three orders of magnitude between the middle of the burst (13452,
 * 19534, extreme values REPEATED) and its edges (2 to 30). Identical
 * extremes coming back means SATURATION. In a fixed-point MLSE the path
 * metrics MUST be renormalized at every step; without that they grow,
 * saturate, and the relative contribution of edge symbols collapses.
 * Count accumulator saturations and flag state during the SB demod. */
{ static long nsat, nova, novb; static int ovm_vu;
    if (dans_sb2) {
        int64_t a = dsp->a, b = dsp->b;
        if (a > 0x7FFFFFFFLL || a < -0x80000000LL) nsat++;
        if (b > 0x7FFFFFFFLL || b < -0x80000000LL) nsat++;
        if (dsp->st0 & 0x0400) nova++;      /* OVA */
        if (dsp->st0 & 0x0200) novb++;      /* OVB */
        if (dsp->st1 & 0x0200) ovm_vu = 1;  /* OVM: saturation mode */
    } else if ((nsat || nova || novb) && n_sat_vus < 3) {
        printf("    [saturation] demod SB : %ld depassements 32 bits,"
               " OVA pose %ld fois, OVB %ld fois, mode OVM %s\n",
               nsat, nova, novb, ovm_vu ? "ACTIF" : "inactif");
        n_sat_vus++; nsat = nova = novb = 0;
    } }
/* [2026-09-18] WHERE DOES THE FIXED WINDOW ADDRESS COME FROM? Watch
 * the address registers that SWEEP the input buffer 0x0cce during the SB
 * demod and print, per register, the sample index range covered and the
 * PC that set it. If that range does NOT move by 10 when the margin goes
 * from 21 to 31, the address is computed once and reused as is. BK is
 * printed too: a mis-emulated circular block size would loop the access
 * over a fixed window of the right length with a correct base. */
{ static uint16_t amin[8], amax[8], apc[8]; static long an[8];
    static int arme4;
    if (dans_sb2) {
       if (!arme4) { for (int k = 0; k < 8; k++) { amin[k] = 0xffff; amax[k] = 0; an[k] = 0; } arme4
        ↪  = 1; }
        for (int k = 0; k < 8; k++) {
            uint16_t v = dsp->ar[k];
            if (v >= 0x0cce && v < 0x0cce + 380) {
                uint16_t idx = (uint16_t)((v - 0x0cce) / 2);
                if (idx < amin[k]) { amin[k] = idx; apc[k] = pc; }
                if (idx > amax[k]) amax[k] = idx;
                an[k]++;
            }
        }
    } else if (arme4 && n_bal < 3) {
        printf("    [tampon-AR] parcours de 0x0cce pendant le demod SB (BK=%u) :\n", dsp->bk);
        for (int k = 0; k < 8; k++)
            if (an[k])
                printf("        AR%d : echantillons %u..%u (%u larges), %ld acces, 1er a pc=%04x\n",
                       k, amin[k], amax[k], amax[k] - amin[k] + 1, an[k], apc[k]);
        n_bal++; arme4 = 0;
    } }
/* [2026-09-18] XC PIPELINE HAZARD. On the C54x the XC condition is
   * sampled two cycles before execution, so an instruction setting the
```

```
                 * flag just before the XC is not yet visible; the core evaluates at
                 * execution time. The hazard therefore only bites if the ROM places the
                 * flag setter within two slots of the XC (code written for real silicon
                 * normally does not). Measure the real DISTANCE instead of assuming it:
                 * keep the last 4 PCs and the ST0 before each. */
                { static uint16_t ring_pc[4], ring_op[4], ring_st0[4]; static int ri;
                  if ((h == 0xFD || h == 0xFF) && pc >= 0x8400 && pc < 0x8800 && n_xc_vus < 10) {
                      printf("    [xc] pc=%04x op=%04x cc=%02x | 3 precedentes :", pc, o, o & 0xff);
                      for (int k = 3; k >= 1; k--) {
                          int q = (ri - k) & 3;
                          printf("  %04x:%04x(ST0=%04x)", ring_pc[q], ring_op[q], ring_st0[q]);
                      }
                      printf(" | ST0 au XC=%04x\n", dsp->st0);
                      n_xc_vus++;
                  }
                  ring_pc[ri] = pc; ring_op[ri] = o; ring_st0[ri] = dsp->st0;
                  ri = (ri + 1) & 3;
                  if (h == 0xFD || h == 0xFF) n_xc_reg[pc >> 10]++; } }
               /* [2026-09-20] DECODER ORACLE (REJEU_DECODEUR=1). Ideal softs at
                * 0x2a00 still give no CRC OK, so the fault is inside 0x9841..: ACS
                * (0x9a78), traceback (0x9aaf) or CRC (0x9887). Dump each stage's
                * output next to what libosmocoding says it should be: u[0..34] =
                * 25 info bits of sb_info + 10 parity bits (gsm0503_sch_crc10). */
               if (env_decodeur) {
                   static unsigned nd;
                   { static unsigned nt;
                     if (pc == 0x9a7f && nt < 6) { nt++;
                         printf("  [dec] apres `sth @0x0e,B` (pc=9a7e) : T=%04x  B=%010llx (hi=%04x)  DP=%03x
↪  CPL=%d  ST0=%04x ST1=%04x  AR1=%04x\n",
                                dsp->t, (unsigned long long)(dsp->b & 0xfffffffffULL), (unsigned)((dsp->b >> 16)
↪  & 0xffff),
                                dsp->st0 & 0x1ff, !!(dsp->st1 & 0x4000), dsp->st0, dsp->st1, dsp->ar[1]); } }
                   { static int cpl_prev = -1; static uint16_t pc_prev; static unsigned ncpl;
                     int cpl = !!(dsp->st1 & 0x4000);
                     if (cpl_prev >= 0 && cpl != cpl_prev && ncpl < 16) { ncpl++;
                         printf("  [cpl] CPL %d -> %d apres l'instruction pc=%04x op=%04x  (fn=%u ST1=%04x
↪  SP=%04x)\n",
                                cpl_prev, cpl, pc_prev, prog_ovly(dsp, pc_prev), fn_cur, dsp->st1, dsp->sp); }
                     cpl_prev = cpl; pc_prev = pc; }
                   if (pc == 0x9866 && nd < 3) {
                       printf("  [dec] fn_demod=%u ACS termine : TRN[0..38] @0x2c82 =", g_sb_cmd_fn);
                       for (int k = 0; k < 39; k++) printf(" %04x", dsp->data[0x2c82 + k]);
                       printf("\n        metriques @0x2c00..0x2c1f =");
                       for (int k = 0; k < 32; k++) printf(" %04x", dsp->data[0x2c00 + k]);
                       printf("\n        softs @0x2a00[0..7] = %04x %04x %04x %04x %04x %04x %04x %04x  BK=%04x
↪  AR0=%04x ST1=%04x\n",
                              dsp->data[0x2a00], dsp->data[0x2a01], dsp->data[0x2a02], dsp->data[0x2a03],
                              dsp->data[0x2a04], dsp->data[0x2a05], dsp->data[0x2a06], dsp->data[0x2a07],
                              dsp->bk, dsp->ar[0], dsp->st1);
                   }
                   if (pc == 0x987b && nd < 3) {
                       int ri = g_n_reels ? g_reel_pour_fn[g_sb_cmd_fn & 63] : -1;
                       unsigned char u[35];
                       if (ri >= 0) { /* decode the real codeword back to u: not available, print softs only */
                           memset(u, 9, sizeof u);
                       } else {
                           uint32_t fn = g_sb_cmd_fn;
                           uint32_t t1 = fn / 1326, t2 = fn % 26, t3 = fn % 51, t3p = t3 ? (t3 - 1) / 10 : 0;
                           uint8_t sb_info[4] = {
                               (uint8_t)(((g_bsic_injecte & 0x3f) << 2) | ((t1 & 0x600) >> 9)),
                               (uint8_t)((t1 & 0x1fe) >> 1),
                               (uint8_t)(((t1 & 0x001) << 7) | ((t2 & 0x1f) << 2) | ((t3p & 0x6) >> 1)),
                               (uint8_t)(t3p & 0x1) };
                           ubit_t ub[35];
                           osmo_pbit2ubit_ext(ub, 0, sb_info, 0, 25, 1);
                           osmo_crc16gen_set_bits(&gsm0503_sch_crc10, ub, 25, ub + 25);
```

```
                        for (int k = 0; k < 35; k++) u[k] = ub[k];
                    }
                    unsigned long long um = 0, ul = 0;
                   for (int k = 0; k < 35; k++) { um = (um << 1) | u[k]; ul |= (unsigned long long)u[k] << k; }
                    printf("  [dec] traceback termine : mots @0x2c00 = %04x %04x %04x %04x | attendu u[0..34]
↪  MSB-first=0x%09llx LSB-first=0x%09llx\n",
                          dsp->data[0x2c00], dsp->data[0x2c01], dsp->data[0x2c02], dsp->data[0x2c03], um, ul);
                    printf("        u = "); for (int k = 0; k < 35; k++) printf("%d", u[k]); printf("\n");
                }
                if (pc == 0x98a2 && nd < 3) {
                    printf("  [dec] CRC : drapeau 0x2bf8=%u  A=%010llx  mots @0x2c00 = %04x %04x %04x\n",
                           dsp->data[0x2bf8], (unsigned long long)(dsp->a & 0xfffffffffULL),
                           dsp->data[0x2c00], dsp->data[0x2c01], dsp->data[0x2c02]);
                    nd++;
                }
            }
            g_op_tous[prog_ovly(dsp, pc)]++;
            int ex = c54x_run(dsp, 1);
            if (ex <= 0) break;
            done += ex;
            insn_total++;
        }
    } else {
        /* single-step around the TOA computation (0x7940..0x795c): A, B, T, 0x3fb4 */
        static int nlog;
        while (done < insns && dsp->running && !dsp->idle) {
            uint16_t pc = dsp->pc & 0xffff;
            /* The SB demod writes 78 soft bits at 0x2a00 (repack at 0x7e94) and
             * the decoder at 0x9841 reads them back. The FB correlator uses THE SAME
             * buffer. Compare both instants to prove the overwrite. */
            { static uint16_t apres_demod[8]; static int arme, np;
              if (pc == 0x7e94 && !arme) {
                  for (int k = 0; k < 8; k++) apres_demod[k] = dsp->data[0x2a00 + k];
                  arme = 1;
              } else if (pc == 0x9841 && arme && np < 6) {
                  int diff = 0;
                  for (int k = 0; k < 8; k++) if (dsp->data[0x2a00 + k] != apres_demod[k]) diff = 1;
                 printf("    [2a00] apres demod: %04x %04x %04x %04x | a l'entree du decodeur: %04x %04x %04x
↪  %04x  => %s\n",
                        apres_demod[0], apres_demod[1], apres_demod[2], apres_demod[3],
                        dsp->data[0x2a00], dsp->data[0x2a01], dsp->data[0x2a02], dsp->data[0x2a03],
                        diff ? "ECRASE" : "intact");
                  np++; arme = 0;
              } }
            /* [2026-09-17] Proof that the samples carry the message: demodulate
             * the SAME buffer 0x0cce the DSP reads, with a reference demodulator
             * written in C. If it recovers the code word and the DSP does not, the
             * signal is good and the fault is entirely in the emulated DSP. */
            /* Does the buffer hold EXACTLY the samples delivered for this frame,
             * or a mix of several bursts? The midamble is identical in every SCH: a
             * mix would reinforce it while drowning the data, which is precisely the
             * symptom observed. */
            { static int nx;
              if (pc == 0x9841 && nx < 4 && g_livre_niq) {
                  int ident = 0, n = g_livre_niq < 380 ? g_livre_niq : 380;
                  int prem_diff = -1;
                  for (int k = 0; k < n; k++) {
                      if ((int16_t)dsp->data[0x0cce + k] == g_livre_iq[k]) ident++;
                      else if (prem_diff < 0) prem_diff = k;
                  }
                  printf("    [tampon] identique aux echantillons livres : %d/%d mots"
                         " (1re difference au mot %d)\n", ident, n, prem_diff);
                  nx++;
              } }
            /* [2026-09-18] At the decoder entry, measure what it actually READS,
             * job by job: the signs of the 78 words packed at 0x2a00 against the
```

```
 * expected code word, with the best frame searched as in [ref] (under
 * REJEU_SCH_PARTOUT the delivered FN is not reliable). Both polarities
 * are printed: on differential GMSK the sign convention is not known a
 * priori. Links "the stage rewriting 0x2a00 outputs values" to "the
 * decoder has something to work on". */
{ static int nb2;
  if (pc == 0x9841 && nb2 < 20) {
      int16_t sb[78]; int nz = 0, mn = 32767, mx = -32768;
      for (int k = 0; k < 78; k++) {
          sb[k] = (int16_t)dsp->data[0x2a00 + k];
          if (!sb[k]) nz++;
          if (sb[k] < mn) mn = sb[k];
          if (sb[k] > mx) mx = sb[k];
      }
      int meil_f = -1, meil_s = -1, s0 = -1;
      for (int df = -12; df <= 3; df++) {
          long f = (long)g_livre_fn + df; if (f < 0) continue;
          unsigned char a2[78]; cellule_code_attendu((uint32_t)f, (uint8_t)g_bsic_injecte, a2);
          int ok = 0;
          for (int k = 0; k < 78; k++) { int bit = sb[k] < 0 ? 1 : 0; if (bit == a2[k]) ok++; }
          int meilleur = ok > 78 - ok ? ok : 78 - ok;
          if (meilleur > meil_s) { meil_s = meilleur; meil_f = (int)f; }
          if (df == 0) s0 = ok;
      }
      printf("    [2a00-lu] job %d : zeros=%d/78 etendue=[%d..%d] |"
             " signes trame livree %d/78 | meilleure trame %d avec %d/78\n",
             nb2 + 1, nz, mn, mx, s0, meil_f, meil_s);
      nb2++;
  } }
{ static int nr;
  if (pc == 0x9841 && nr < 4) {
      int16_t ech[2 * 190];
      for (int k = 0; k < 380; k++) ech[k] = (int16_t)dsp->data[0x0cce + k];
      unsigned char b148[148]; int off = 0;
      unsigned char att[78];
      cellule_code_attendu(g_livre_fn, (uint8_t)g_bsic_injecte, att);
      if (cellule_demod_reference(ech, 190, b148, &off) == 0) {
          /* sanity of the reference demod: the midamble must come back out */
          int okm = 0;
          for (int i = 0; i < 64; i++) if (b148[42 + i] == cellule_train_sb(i)) okm++;
          /* raw differential bits: no error propagation */
          unsigned char d148[148]; int okd = 0, okdd = 0;
          if (cellule_demod_d(ech, 190, off, d148) == 0) {
              for (int i = 1; i < 64; i++) {
                  int att_d = cellule_train_sb(i) ^ cellule_train_sb(i - 1);
                  if (d148[42 + i] == att_d) okd++;
              }
              /* same bits, opposite polarity */
              for (int i = 1; i < 64; i++) {
                  int att_d = cellule_train_sb(i) ^ cellule_train_sb(i - 1);
                  if (d148[42 + i] != att_d) okdd++;
              }
          }
          /* The DATA too, as raw differential bits, against the full
           * expected burst (3 tail + 39 + midamble + 39 + 3 tail).
           * Comparing on b148 is meaningless: differential decoding
           * propagates any single error over the rest of the burst. */
          int okdat = 0, ndat = 0, meil_f = -1, meil_s = -1;
          for (int df = -12; df <= 3; df++) {
              long f = (long)g_livre_fn + df; if (f < 0) continue;
            unsigned char a2[78]; cellule_code_attendu((uint32_t)f, (uint8_t)g_bsic_injecte, a2);
              unsigned char burst[148];
              memset(burst, 0, 3);
              memcpy(burst + 3, a2, 39);
              for (int i = 0; i < 64; i++) burst[42 + i] = (unsigned char)cellule_train_sb(i);
              memcpy(burst + 106, a2 + 39, 39);
```

```
                memset(burst + 145, 0, 3);
                int ok2 = 0, n2 = 0, prev2 = 1;
                for (int i = 0; i < 148; i++) {
                    int att_d = burst[i] ^ prev2; prev2 = burst[i];
                    int est_donnee = (i >= 3 && i < 42) || (i >= 106 && i < 145);
                    if (!est_donnee) continue;
                    n2++; if (d148[i] == att_d) ok2++;
                }
                if (ok2 > meil_s) { meil_s = ok2; meil_f = (int)f; }
                if (df == 0) { okdat = ok2; ndat = n2; }
            }
            printf("    [d-brut] midambule %d/63 (inverse %d) | DONNEES trame livree %d/%d"
                   " | meilleure trame %d avec %d/%d\n",
                   okd, okdd, okdat, ndat, meil_f, meil_s, ndat);
            /* the midamble is IDENTICAL for every SCH frame: if it locks
             * but the data does not, the buffer holds the burst of ANOTHER
             * frame. Find out which. */
            int meilleure = -1, meilleur_score = -1;
            for (int df = -12; df <= 3; df++) {
                long f = (long)g_livre_fn + df; if (f < 0) continue;
              unsigned char a2[78]; cellule_code_attendu((uint32_t)f, (uint8_t)g_bsic_injecte, a2);
                int ok2 = 0;
                for (int i = 0; i < 39; i++) if (b148[3 + i] == a2[i]) ok2++;
                for (int i = 0; i < 39; i++) if (b148[106 + i] == a2[39 + i]) ok2++;
                if (ok2 > meilleur_score) { meilleur_score = ok2; meilleure = (int)f; }
            }
            int ok = 0;
            for (int i = 0; i < 39; i++) if (b148[3 + i] == att[i]) ok++;
            for (int i = 0; i < 39; i++) if (b148[106 + i] == att[39 + i]) ok++;
            printf("    [ref] fn livree=%u offset=%d | midambule %d/64 |"
                   " mot de code de la trame livree %d/78 |"
                   " MEILLEURE trame = %d avec %d/78\n",
                   g_livre_fn, off, okm, ok, meilleure, meilleur_score);
        } else {
            printf("    [ref] fn=%u  demod de reference : pas de pic exploitable\n", g_livre_fn);
        }
        nr++;
    } }
/* [2026-09-17] Hunting the next ISA bug. The FB path works, so every
 * opcode it executes is valid by construction. List the opcodes only the
 * SB demodulator executes: the one family never validated, hence the pool
 * of suspects. */
{ static int dans_sb;
  if (!vu_sb) { vu_sb = calloc(65536, 1); vu_hors = calloc(65536, 1); }
  if (pc == 0x7c31) dans_sb = 1;
  if (pc == 0x9841) dans_sb = 0;
  /* [2026-09-18] Flipping ONE coded bit moves 78 positions out of 78
   * in 0x2c72: the traceback is global. On the C54x that points at CMPS
   * (0x8E/0x8F) and the TRN register. So count what really executes in
   * the SB path: the true CMPS, or the 0xE0 family (FIRS/LMS/SQDST/ABDST)
   * whose variant still carries pseudo-CMPS semantics in the core. */
  { uint16_t o = dsp->prog[pc]; uint8_t h = o >> 8;
    if (dans_sb) {
        if (h == 0x8E || h == 0x8F) n_cmps_sb++;
        if (h >= 0xE0 && h <= 0xE3) { n_e0_sb++; n_e0x_sb[h - 0xE0]++; }
        if (o == 0x8D00 || (o & 0xFF00) == 0x8D00) n_sttrn_sb++;
    } else {
        if (h == 0x8E || h == 0x8F) n_cmps_hors++;
    } }
  { uint16_t o = dsp->prog[pc];
    if (dans_sb) vu_sb[o] = 1; else vu_hors[o] = 1; }
}
/* [2026-09-17] The 78 soft bits are at 0x2c72 (NOT 0x2a00, which is
 * the 296-word FB correlator buffer). Compare the SIGN of each with the
 * bit actually transmitted: matching signs mean the demodulator is sound
 * and the fault is in the decoder; matching with a shift means the
```

```
 * correlator offset is wrong. */
{ static int nb;
  if (pc == 0x9841 && nb < 400) {
      unsigned char att[78];
      cellule_code_attendu(g_livre_fn, (uint8_t)g_bsic_injecte, att);
      /* Try the plausible conventions: polarity, halves swapped (the
       * DSP may return 39+39 in the other order), and a shift. A
       * combination clearly above chance means the demodulator is sound
       * and it is a matter of convention. */
          /* [2026-09-18] The family tested did NOT include even/odd
           * de-interleaving. The SCH convolutional code emits its 78 bits
           * in the order C(2k), C(2k+1) per trellis step, and the burst
           * stores them as two halves of 39. If the DSP stores its softs
           * in the INTERNAL order (all evens then all odds), no cyclic
           * shift, polarity or half swap recovers it: every combination
           * lands exactly at chance, which is precisely the plateau
           * observed. Add the permutation and its inverse.
           *   0 identity   1 halves swapped
           *   2 evens first   3 odds first */
          int best = -99, bestd = 0, bestv = 0;
          for (int v = 0; v < 8; v++) {
              int inv = v & 1, perm = v >> 1;
              for (int d = -8; d <= 8; d++) {
                  int ok = 0, tot = 0;
                  for (int i = 0; i < 78; i++) {
                      int src;
                      switch (perm) {
                      case 1:  src = (i + 39) % 78; break;
                      case 2:  src = (i % 2 == 0) ? (i / 2) : (39 + (i - 1) / 2); break;
                      case 3:  src = (i % 2 == 1) ? ((i - 1) / 2) : (39 + i / 2); break;
                      default: src = i; break;
                      }
                  int j = src + d; if (j < 0 || j >= 78) continue;
                  int16_t sv = (int16_t)dsp->data[0x2c72 + j];
                  if (!sv) continue;
                  tot++;
                  int bit = att[i] != 0; if (inv) bit = !bit;
                  if ((sv < 0) == bit) ok++;
              }
              if (tot >= 40) { int pc2 = ok * 100 / tot;
                  if (pc2 > best) { best = pc2; bestd = d; bestv = v; } }
          }
      }
      int nznb = 0; for (int i = 0; i < 78; i++) if (dsp->data[0x2c72 + i]) nznb++;
          /* Is the DSP output a SLICE of the burst at some arbitrary
           * offset? Correlate the 78 signs against the differential
           * sequence of the 148 burst bits, at every shift, in both
           * polarities. */
          { unsigned char burst[148]; int prev3 = 1; unsigned char dref[148];
            memset(burst, 0, 3); memcpy(burst + 3, att, 39);
            for (int i = 0; i < 64; i++) burst[42 + i] = (unsigned char)cellule_train_sb(i);
            memcpy(burst + 106, att + 39, 39); memset(burst + 145, 0, 3);
            for (int i = 0; i < 148; i++) { dref[i] = burst[i] ^ prev3; prev3 = burst[i]; }
            int bs = -1, bo = 0, bp = 0;
            for (int o = -78; o <= 148; o++) {
                for (int pol = 0; pol < 2; pol++) {
                    int ok = 0, tot = 0;
                    for (int i = 0; i < 78; i++) {
                        int j = o + i; if (j < 0 || j >= 148) continue;
                        int16_t sv = (int16_t)dsp->data[0x2c72 + i];
                        if (!sv) continue;
                        tot++; int b = dref[j]; if (pol) b = !b;
                        if ((sv < 0) == b) ok++;
                    }
                  if (tot >= 50 && ok * 100 / tot > bs) { bs = ok * 100 / tot; bo = o; bp = pol; }
                }
```

```
                    }
                    printf("    [tranche] sortie DSP vs burst complet : max %d%% a l'offset %d (polarite
↪  %d)\n",
                           bs, bo, bp); }
                /* [2026-09-18] Peak and agreement on the SAME line, so they can
                 * be correlated frame by frame: if the frames whose peak is 21 (the
                 * real burst start) agree better, peak PRECISION is still a problem
                 * and the channel estimate sits beside the burst, which is enough
                 * to ruin an MLSE equalizer on GMSK whose ISI spans three symbols.
                 * A flat agreement whatever the peak clears the correlator. */
                /* fingerprint of the 78 soft bits, to diff two runs */
                { static int nemq; static long cible = -2;
                  if (cible == -2) { const char *e = calypso_getenv("REJEU_EMPREINTE_FN");
                                     cible = e ? atol(e) : -1; }
                  /* [2026-09-18] THE FRAME MUST BE LOCKED. Taking whichever
                   * occurrence comes first does not compare the same thing across
                   * runs: the frame the SB attempt lands on depends on FB
                   * scheduling, and under SCH_PARTOUT a different frame means a
                   * different T1/T2/T3, hence an entirely different code word --
                   * 190/380 samples differing and 38 signs out of 78 flipping,
                   * i.e. chance. REJEU_EMPREINTE_FN=<n> prints frame n only. */
                  if (nemq < 1 && (cible < 0 || (long)g_livre_fn == cible)) {
                      /* [2026-09-18] Check the MODULATOR before blaming the
                       * demodulator. Flipping coded bit 0 touches burst bit 3, so
                       * alpha_3 and alpha_4 both change sign and their SUM is
                       * preserved: the phase realigns after two symbols, and only
                       * ~6 complex samples around index 24 should differ out of
                       * 380. If all 380 differ, the modulator is at fault. */
                      printf("    [empreinte-iq] fn=%u :", g_livre_fn);
                      for (int k = 0; k < 380; k++) printf(" %04x", dsp->data[0x0cce + k]);
                      printf("\n"); nemq++;
                  } }
                { static int nemp; static long cible2 = -2;
                  if (cible2 == -2) { const char *e = calypso_getenv("REJEU_EMPREINTE_FN");
                                      cible2 = e ? atol(e) : -1; }
                  if (nemp < 3 && (cible2 < 0 || (long)g_livre_fn == cible2)) {
                      printf("    [empreinte] fn=%u (trame courante %u%s) pic=%u :", g_livre_fn, fn_cur,
                             g_livre_fn == fn_cur ? "" : " DESALIGNE", dsp->data[0x2f06]);
                      for (int k = 0; k < 78; k++) printf(" %04x", dsp->data[0x2c72 + k]);
                      printf("\n"); nemp++;
                  } }
                /* [2026-09-18] Four independent decodings of the block
                 * 0x84a0-0x84d8 show it is NOT the soft-bit writer but the MIDAMBLE
                 * CORRELATOR: 50 lags, 64 taps of a fixed reference at 0x2cea
                 * (64 = midamble length), Re output at 0x2c56 and Im at 0x2c88,
                 * then |corr|^2 at 0x2be4 for the argmax. Its index is the LAG, not
                 * a coded-bit rank, so there is nothing to look for there. They
                 * point at 0x2a00 as the real buffer (loop 0x848d-0x849f, BRC=141
                 * hence 142 iterations, `sth B,*AR6+` at 0x8497). 142 is close to
                 * the 148 burst bits, hence the hypothesis "0x2a00 indexed by
                 * POSITION IN THE BURST", where coded bit b is at 3+b for b<39 and
                 * at 106+(b-39) beyond. */
                /* [2026-09-18] SEARCH RATHER THAN GUESS. At the decoder entry,
                 * sweep ALL of data memory for a zone whose SIGNS match the
                 * transmitted code word, under two layouts: 78 contiguous words
                 * indexed by coded-bit rank, and indexing by burst position (3+b
                 * then 106+b-39). Print the best bases. If none stands clearly
                 * above chance, the soft bits are not in data memory in either
                 * form. */
                /* [2026-09-18] INPUT / OUTPUT DISCRIMINANT for the 0x2ad5
                 * candidate. Agreement does not separate the two: the derotated
                 * input and the demodulator output are both indexed by burst
                 * position and would answer a bit flip alike. What separates them
                 * is CHANNEL dependence: a demodulator output depends on the
                 * channel estimate, hence on the peak; an input representation does
                 * not. So perturb the MIDAMBLE alone (which moves the peak without
```

```
 * touching the data) and see whether 0x2ad5 moves AT THE DATA
 * POSITIONS. Invariant => input. Moving => output, buffer found. */
/* [2026-09-18] THE REAL BUFFER, found in ROM: 0x2a00.
 * Two successive stages, both at 0x2a00:
 *   - equalizer output, 142 words, index = burst position - 3
 *     (`sth B,*AR6+`, AR6 init 0x2a00, BRC=0x8d hence 142 turns);
 *   - PACKED codeword, 78 contiguous words, index = coded-bit rank,
 *     produced by 0x7e65-0x7e7a: 39 copies, then
 *     `mar *+AR2(0x0040)` at 0x7e76 which SKIPS THE 64 MIDAMBLE
 *     BITS, then 39 copies. The +64 is literal in ROM.
 * The decoder confirms it: 0x984a `stm #0x2a00,AR1`, BRC=0x26, and
 * the body at 0x9a78 advances AR1 by 2 per trellis step, 39 steps =
 * 78 words.
 *
 * Zeros must NOT be discarded here: these soft bits take the values
 * 0x0000 and 0xffff, so an `if (!v) continue;` plus a `n >= 60`
 * requirement threw away half the samples and eliminated 0x2a00
 * before it could be scored. */
{ static int n2a0;
  if (n2a0 < 4) {
      int meil_c = -1, pol_c = 0, meil_b = -1, pol_b = 0;
      for (int pol = 0; pol < 2; pol++) {
          int ok = 0;
          for (int b = 0; b < 78; b++) {
              int16_t v = (int16_t)dsp->data[0x2a00 + b];
              int bit = att[b] != 0; if (pol) bit = !bit;
              if ((v < 0) == bit) ok++;
          }
          if (ok * 100 / 78 > meil_c) { meil_c = ok * 100 / 78; pol_c = pol; }
          ok = 0;
          for (int b = 0; b < 78; b++) {
              int j = (b < 39) ? b : (64 + b);
              int16_t v = (int16_t)dsp->data[0x2a00 + j];
              int bit = att[b] != 0; if (pol) bit = !bit;
              if ((v < 0) == bit) ok++;
          }
          if (ok * 100 / 78 > meil_b) { meil_b = ok * 100 / 78; pol_b = pol; }
      }
      int nz = 0, nff = 0, naut = 0;
      for (int k = 0; k < 142; k++) {
          uint16_t v = dsp->data[0x2a00 + k];
          if (v == 0) nz++; else if (v == 0xffff) nff++; else naut++;
      }
      printf("    [2a00] COMPACTE (0x2a00+b) : %d%% (polarite %d) | "
             "BRUT (saut de 64) : %d%% (polarite %d)\n",
             meil_c, pol_c, meil_b, pol_b);
      printf("    [2a00] contenu sur 142 mots : %d nuls, %d a 0xffff, %d autres |",
             nz, nff, naut);
      for (int k = 0; k < 10; k++) printf(" %04x", dsp->data[0x2a00 + k]);
      printf("\n");
      n2a0++;
  } }
{ static int n2d;
  if (n2d < 1) {
      printf("    [2ad5] pic=%u valeurs aux positions de burst 0..147 :\n      ",
             dsp->data[0x2f06]);
      for (int k = 0; k < 148; k++) {
          printf(" %04x", dsp->data[0x2ad5 + k]);
          if (k % 12 == 11) printf("\n      ");
      }
      printf("\n");
      n2d++;
  } }
{ static int nch;
  if (nch < 1) {
      struct { int sc, base, disp, pol; } top[6];
```

```
        for (int t = 0; t < 6; t++) { top[t].sc = -1; top[t].base = 0; }
        for (int base = 0; base < 0x3f00; base++)
          for (int disp = 0; disp < 2; disp++)
            for (int pol = 0; pol < 2; pol++) {
                int ok = 0, n = 0;
                for (int b = 0; b < 78; b++) {
                    int j = disp ? ((b < 39) ? (3 + b) : (106 + b - 39)) : b;
                    int16_t v = (int16_t)dsp->data[(base + j) & 0x3fff];
                    if (!v) continue;
                    n++; int bit = att[b] != 0; if (pol) bit = !bit;
                    if ((v < 0) == bit) ok++;
                }
                if (n < 60) continue;
                int sc = ok * 100 / n;
                for (int t = 0; t < 6; t++)
                    if (sc > top[t].sc) {
                        for (int u = 5; u > t; u--) top[u] = top[u-1];
                     top[t].sc = sc; top[t].base = base; top[t].disp = disp; top[t].pol = pol;
                        break;
                    }
            }
        printf("    [chasse] meilleure zone : base 0x%04x %s pol %d -> %d%%"
               "   (2e: 0x%04x %d%%, 3e: 0x%04x %d%%)\n",
               top[0].base, top[0].disp ? "burst" : "contigu", top[0].pol, top[0].sc,
               top[1].base, top[1].sc, top[2].base, top[2].sc);
        nch++;
  } }
{ static int n2a;
  if (n2a < 3) {
        int meil = -1, mo = 0, mp = 0;
        for (int o = -8; o <= 8; o++)
          for (int pol = 0; pol < 2; pol++) {
              int ok = 0, n = 0;
              for (int b = 0; b < 78; b++) {
                  int bp = (b < 39) ? (3 + b) : (106 + b - 39);
                  int j = bp + o; if (j < 0 || j >= 148) continue;
                  int16_t v = (int16_t)dsp->data[0x2a00 + j]; if (!v) continue;
                  n++; int bit = att[b] != 0; if (pol) bit = !bit;
                  if ((v < 0) == bit) ok++;
              }
              if (n >= 50) { int pc2 = ok * 100 / n;
                  if (pc2 > meil) { meil = pc2; mo = o; mp = pol; } }
          }
        int nz = 0; for (int k = 0; k < 148; k++) if (dsp->data[0x2a00 + k]) nz++;
        printf("    [2a00] indexe par position de burst : concordance max %d%%"
               " (decalage %d, polarite %d) | %d/148 mots non nuls\n",
               meil, mo, mp, nz);
        printf("    [2a00] valeurs :");
        for (int k = 0; k < 14; k++) printf(" %04x", dsp->data[0x2a00 + k]);
        printf("\n");
        n2a++;
  } }
{ static int nv;
  if (nv < 3) {
        int m1b = -1, m1o = 0, m1s = 0, m2b = -1, m2o = 0, m2s = 0;
        for (int o = 0; o < 100; o++)
          for (int sens = -1; sens <= 1; sens += 2)
            for (int pol = 0; pol < 2; pol++) {
                int ok1 = 0, n1 = 0, ok2 = 0, n2 = 0;
                for (int b = 0; b < 39; b++) {
                    int j = o + sens * b; if (j < 0 || j >= 100) continue;
                    int16_t v = (int16_t)dsp->data[0x2c56 + j]; if (!v) continue;
                    n1++; int bit = att[b] != 0; if (pol) bit = !bit;
                    if ((v < 0) == bit) ok1++;
                }
                for (int b = 39; b < 78; b++) {
```

```
                                int j = o + sens * (b - 39); if (j < 0 || j >= 100) continue;
                                int16_t v = (int16_t)dsp->data[0x2c56 + j]; if (!v) continue;
                                n2++; int bit = att[b] != 0; if (pol) bit = !bit;
                                if ((v < 0) == bit) ok2++;
                            }
                            if (n1 >= 30) { int pc1 = ok1 * 100 / n1;
                                if (pc1 > m1b) { m1b = pc1; m1o = o; m1s = sens * (pol ? -2 : 1); } }
                            if (n2 >= 30) { int pc2 = ok2 * 100 / n2;
                                if (pc2 > m2b) { m2b = pc2; m2o = o; m2s = sens * (pol ? -2 : 1); } }
                        }
                    printf("    [2c56] moitie 1 : meilleur %d%% a l'offset %d (code %d) | "
                           "moitie 2 : meilleur %d%% a l'offset %d (code %d)\n",
                           m1b, m1o, m1s, m2b, m2o, m2s);
                    int nz = 0; for (int k = 0; k < 100; k++) if (dsp->data[0x2c56 + k]) nz++;
                    printf("    [2c56] %d/100 mots non nuls, carte des nuls (. = nul, X = non nul) :\n
↪  ", nz);
                    for (int k = 0; k < 100; k++) {
                        printf("%c", dsp->data[0x2c56 + k] ? 'X' : '.');
                        if (k % 50 == 49) printf("\n      ");
                    }
                    printf("\n");
                    for (int lig = 0; lig < 10; lig++) {
                        printf("      +%2d :", lig * 10);
                     for (int k = lig * 10; k < lig * 10 + 10; k++) printf(" %04x", dsp->data[0x2c56 + k]);
                        printf("\n");
                    }
                    nv++;
                  } }
                printf("    [pic-conc] pic=%u concordance=%d%%\n", dsp->data[0x2f06], best);
                printf("    [bits] fn=%u  0x2c72 non-nuls=%d/78 : %04x %04x %04x %04x %04x %04x\n"
                       "           concordance max = %d%% (decalage %d, polarite %d, ordre %s)\n",
                       g_livre_fn, nznb, dsp->data[0x2c72], dsp->data[0x2c73], dsp->data[0x2c74],
                       dsp->data[0x2c75], dsp->data[0x2c76], dsp->data[0x2c77],
                       best, bestd, bestv & 1,
                       (bestv >> 1) == 1 ? "moities echangees"
                       : (bestv >> 1) == 2 ? "pairs d'abord"
                       : (bestv >> 1) == 3 ? "impairs d'abord" : "identite");
                nb++;
              } }
            /* [2026-09-18] The `sb` word returned is IDENTICAL for BSIC 7, 14 and
             * 49, while the soft bits at 0x2c72 do follow the payload: the word the
             * ARM reads back does NOT come from the decoder. Does the DSP ever write
             * the a_sch words of the R page? API at 0x0800, R_PAGE0=0x28 and
             * R_PAGE1=0x3C, a_sch at word +15. */
            { static uint16_t vu0[10], vu1[10]; static int arme3, nw;
              const uint16_t A0 = 0x800 + 0x28 + 15, A1 = 0x800 + 0x3C + 15;
              if (!arme3) {
                  for (int k = 0; k < 10; k++) { vu0[k] = dsp->data[A0 + k]; vu1[k] = dsp->data[A1 + k]; }
                  arme3 = 1;
              } else if (nw < 12) {
                  for (int k = 0; k < 5; k++) {
                      if (dsp->data[A0 + k] != vu0[k]) {
                          printf("    [a_sch] page0 mot %d : %04x -> %04x  ecrit par pc=%04x\n",
                                 k, vu0[k], dsp->data[A0 + k], pc);
                          vu0[k] = dsp->data[A0 + k]; nw++;
                      }
                      if (dsp->data[A1 + k] != vu1[k]) {
                          printf("    [a_sch] page1 mot %d : %04x -> %04x  ecrit par pc=%04x\n",
                                 k, vu1[k], dsp->data[A1 + k], pc);
                          vu1[k] = dsp->data[A1 + k]; nw++;
                      }
                  }
              } }
            /* [2026-09-18] The magnitudes are present and the argmax is still
             * wrong. Next question: WHICH zone does it sweep? Follow the span of the
             * address registers during peak selection (0x84c8..0x84ef) and compare it
```

```
 * with the magnitude zones. */
{ static uint16_t armin[8], armax[8]; static int arme2;
  if (pc >= 0x84c8 && pc <= 0x84ef) {
      if (!arme2) { for (int k = 0; k < 8; k++) { armin[k] = 0xffff; armax[k] = 0; } arme2 = 1; }
      for (int k = 0; k < 8; k++) {
          uint16_t v = dsp->ar[k];
          if (v < armin[k]) armin[k] = v;
          if (v > armax[k]) armax[k] = v;
      }
  }
{ static int nq;
  if (pc == 0x84ef && nq < 5) {
      printf("    [balayage] registres d'adresse pendant 0x84c8-0x84ef :");
      for (int k = 0; k < 8; k++)
          if (armax[k] >= armin[k] && armax[k] > armin[k])
              printf(" AR%d=[0x%04x..0x%04x]", k, armin[k], armax[k]);
      printf("\n");
      arme2 = 0;
      printf("    [magn@84ef] plages non nulles entre 0x2b00 et 0x2d00 :\n");
      int deb = -1, fin = -1;
      for (int a = 0x2b00; a <= 0x2d00; a++) {
          int nz = (a <= 0x2cff) && dsp->data[a] != 0;
          if (nz && deb < 0) deb = a;
          if (nz) fin = a;
          if (!nz && deb >= 0) {
              /* [2026-09-18] Compare as int16_t, not uint16_t: the
               * reported "maxima" (65502, 65104, 65318) were in fact
               * -34, -432, -218, so the probe reported the value closest
               * to -1 and the rank with it was noise. The maximum in
               * absolute value is printed too: if 0x2be4 carries a SIGNED
               * correlation, |v| (or v*v) must decide, not v. The two
               * ranks side by side say which. */
              int n = fin - deb + 1;
              int16_t vmax = -32768; int imax = -1;
              int amax = -1, iamax = -1;
              for (int k = deb; k <= fin; k++) {
                  int16_t v = (int16_t)dsp->data[k];
                  int a = v < 0 ? -(int)v : (int)v;
                  if (v > vmax) { vmax = v; imax = k - deb; }
                  if (a > amax) { amax = a; iamax = k - deb; }
              }
              printf("        0x%04x-0x%04x : %3d mots, max signe=%d au rang %d,"
                     " max |v|=%d au rang %d\n",
                     deb, fin, n, vmax, imax, amax, iamax);
              deb = -1;
          }
      }
      printf("        argmax DSP 0x2f06=%u\n", dsp->data[0x2f06]);
      nq++;
  } } }
/* [2026-09-17] Where does the correlator ACTUALLY write? Snapshot
 * data RAM on entry to 0x84a1 and list what changed on exit, rather than
 * guessing the address of the energy buffer. */
{ static uint16_t *snap; static int armec, ns;
  if (pc == 0x84a1 && !armec && ns < 3) {
      if (!snap) snap = malloc(0x4000 * sizeof(uint16_t));
      memcpy(snap, dsp->data, 0x4000 * sizeof(uint16_t));
      armec = 1;
  } else if (pc == 0x7e94 && armec) {
      int deb = -1, fin = -1, nch = 0;
      printf("    [ecrit] plages modifiees par le correlateur SB :\n");
      for (int k = 0; k < 0x4000; k++) {
          int d = (dsp->data[k] != snap[k]);
          if (d && deb < 0) deb = k;
          if (d) { fin = k; nch++; }
          if (!d && deb >= 0 && k > fin + 8) {
```

```
                printf("        0x%04x-0x%04x (%d mots)  ex: %04x->%04x\n",
                       deb, fin, fin - deb + 1, snap[deb], dsp->data[deb]);
                deb = -1;
            }
        }
        if (deb >= 0) printf("        0x%04x-0x%04x\n", deb, fin);
        printf("        total %d mots changes\n", nch);
        armec = 0; ns++;
    } }
/* [2026-09-17] With a REAL SCH burst on the input, the soft bits at
 * 0x2a00 come out null. Look at the middle link: the midamble
 * correlation energy (0x2be4) and the argmax (0x2f06). */
{ static int nc;
  if (pc == 0x84ef && nc < 8) {
      /* same type fix as above: int16_t, plus |v| */
      int nzc = 0, emax = -1, iemax = -1, samax = -32768, isamax = -1;
      for (int k = 0; k < 64; k++) {
          int16_t v = (int16_t)dsp->data[0x2be4 + k];
          int a = v < 0 ? -(int)v : (int)v;
          if (v) nzc++;
          if (a > emax) { emax = a; iemax = k; }
          if (v > samax) { samax = v; isamax = k; }
      }
      int nzin = 0; for (int k = 0; k < 380; k++) if (dsp->data[0x0cce + k]) nzin++;
      printf("    [corr] entree 0x0cce non-nuls=%d/380 | 0x2be4 non-nuls=%d/64"
             " | max |v|=%d au rang %d, max signe=%d au rang %d"
             " | argmax 0x2f06=%u | 0x2be4: %04x %04x %04x %04x %04x %04x\n",
             nzin, nzc, emax, iemax, samax, isamax, dsp->data[0x2f06],
             dsp->data[0x2be4], dsp->data[0x2be5], dsp->data[0x2be6],
             dsp->data[0x2be7], dsp->data[0x2be8], dsp->data[0x2be9]);
      nc++;
  } }
/* SB demod input: the DMA must have dropped 190 complex samples at 0x0cce */
{ static int ne;
  if (pc == 0x7c31 && ne < 6) {
      int nz = 0; for (int k = 0; k < 380; k++) if (dsp->data[0x0cce + k]) nz++;
      int z0 = 0; while (z0 < 380 && !dsp->data[0x0cce + z0]) z0++;
      printf("    [in-sb] fn=%u p51=%u  DERNIER BURST LIVRE: type=%c fn=%u n=%d\n"
             "            0x0cce non-nuls=%d/380 zeros_tete=%d : %04x %04x %04x %04x %04x %04x\n",
             fn_cur, fn_cur % 51, g_livre_type ? g_livre_type : '?', g_livre_fn, g_livre_n,
             nz, z0, dsp->data[0x0cce], dsp->data[0x0ccf], dsp->data[0x0cd0],
             dsp->data[0x0cd1], dsp->data[0x0cd2], dsp->data[0x0cd3]);
      ne++;
  } }
switch (pc) {                       /* SB chain (RE 9.4) */
case 0x7c31: hit_7c31++; break;     /* SB demodulator */
/* Counters must be fed in BOTH execution paths: one that stays at zero
 * under REJEU_PROBE_TOA while the code runs reads as a measurement. */
case 0x7d1c: hit_7d1c++; break;     /* rpt #15, the division */
case 0x7d1d: hit_7d1d++; break;     /* subc */
case 0x7d1e: hit_7d1e++; break;     /* the quotient */
case 0x81e4: hit_81e4++; break;     /* the mpy that depends on T */
case 0x8478: hit_8478++; break;     /* FIRS site 1 */
case 0x8492: hit_8492++; break;     /* rpt of FIRS site 2 */
case 0x8493: hit_8493++; break;     /* FIRS site 2 */
case 0x8497: hit_8497++; break;     /* sth B -> soft bits */
case 0x9841: hit_9841++; break;     /* SCH decoder */
case 0x84a1: hit_84a1++; break;     /* midamble correlator */
case 0x770a: hit_770a++; break;     /* FB correlator */
case 0xb219: hit_b219++; break;     /* SB job (arms the DMA) */
case 0x7a16: hit_7a16++; break;     /* fcall into the SB demod */
default: break; }
histo_pc[pc >> 10]++; insn_total++;
{ static uint16_t tprev, tpc, ppc; static int tn;
  if (dsp->t != tprev) { tpc = ppc; tprev = dsp->t; }
  ppc = pc;
```

```
              if (pc == 0x7947 && tn < 10 && env_probe_t) {
                  printf("    [T] au mpya (0x7947) : T=%04x (%u), dernier ecrit par pc=%04x\n",
                         dsp->t, dsp->t, tpc); tn++; } }
              if (!env_probe_t && pc >= 0x7940 && pc <= 0x795c && nlog < 60) {
                  printf("    [toa] pc=%04x A=%010llx B=%010llx T=%04x 3fb4=%04x AR4=%04x\n",
                         pc, (unsigned long long)(dsp->a & 0xfffffffffULL),
                         (unsigned long long)(dsp->b & 0xfffffffffULL),
                         dsp->t, dsp->data[0x3fb4], dsp->ar[4]);
                  nlog++;
              }
              g_op_tous[prog_ovly(dsp, pc)]++;
              int ex = c54x_run(dsp, 1);
              if (ex <= 0) break;
              done += ex;
          }
      }
  }
  {   /* [2026-09-18] Raw dump of the program words around the soft-bit writer,
       * to read the ADDRESS COMPUTATION of both passes. */
      printf("  mots programme 0x8378-0x8396 (l'ecrivain 0x838d qui met 142 zeros) :\n");
      for (uint16_t a = 0x8378; a <= 0x8396; a++)
          printf("    %04x: %04x%s", a, dsp->prog[a], ((a - 0x8378) % 8 == 7) ? "\n" : "");
      printf("\n  mots programme 0x81c8-0x81e0 (les ecrivains des 142 valeurs) :\n");
      for (uint16_t a = 0x81c8; a <= 0x81e0; a++)
          printf("    %04x: %04x%s", a, dsp->prog[a], ((a - 0x81c8) % 8 == 7) ? "\n" : "");
      printf("\n");
      printf("  mots programme 0x84a0-0x84d8 (correlateur de midambule) :\n");
      for (uint16_t a = 0x84a0; a <= 0x84d8; a++)
          printf("    %04x: %04x%s", a, dsp->prog[a], ((a - 0x84a0) % 8 == 7) ? "\n" : "");
      printf("\n  mots programme 0x7cb0-0x7cc0 (site qui remet a zero) :\n");
      for (uint16_t a = 0x7cb0; a <= 0x7cc0; a++)
          printf("    %04x: %04x%s", a, dsp->prog[a], ((a - 0x7cb0) % 8 == 7) ? "\n" : "");
      printf("\n");
  }
  { const char *e = calypso_getenv("REJEU_OPCODES_TOUS");
    if (e && *e) { FILE *fo = fopen(e, "w");
        if (fo) { for (unsigned o = 0; o < 65536; o++) if (g_op_tous[o]) fprintf(fo, "%04x %lu\n", o,
         ↪  g_op_tous[o]);
                  fclose(fo); printf("  opcodes executes ecrits dans %s\n", e); } } }
  printf("\n─── bilan rejeu (deterministe) ───\n");
  {
      printf("  instructions DSP executees : %lu\n", insn_total);
      printf("  histogramme des PC (plages de 1024 mots, non vides) :\n");
      for (int b = 0; b < 64; b++)
          if (histo_pc[b])
              printf("      0x%04x-0x%04x : %10lu  (%4.1f%%)\n",
                     b << 10, ((b + 1) << 10) - 1, histo_pc[b],
                     insn_total ? 100.0 * histo_pc[b] / insn_total : 0.0);
  }
  if (vu_sb && vu_hors) {
      int n = 0;
      printf("  opcodes executes UNIQUEMENT par le demodulateur SB (suspects d'ISA,\n"
             "  jamais valides par le chemin FB qui, lui, fonctionne) :\n    ");
      int fam[256] = {0}, famt[256] = {0};
      for (int o = 0; o < 65536; o++) {
          if (vu_sb[o]) famt[o >> 8]++;
          if (vu_sb[o] && !vu_hors[o]) { fam[o >> 8]++; n++; }
      }
      printf("\n");
      for (int f = 0; f < 256; f++)
          if (fam[f]) printf("    famille %02x.. : %3d opcodes exclusifs SB / %3d executes en SB\n",
                             f, fam[f], famt[f]);
      printf("    (%d opcodes distincts exclusifs au demodulateur SB)\n", n);
  }
  /* [2026-09-19] WHERE DOES 0x01dbf46a COME FROM? The same word is returned by
 * the synthetic fixture, by the real capture in replay and by the bridge: three
```

```
 * inputs with nothing in common. A constant word looks like a CONSTANT, not a
 * computation. Sweep data space AND program space for the pattern. */
if (calypso_getenv("REJEU_CHERCHER_MOT")) {
    unsigned long v = strtoul(calypso_getenv("REJEU_CHERCHER_MOT"), NULL, 0);
    uint16_t lo = (uint16_t)(v & 0xffff), hi = (uint16_t)(v >> 16);
    printf("  recherche de 0x%04x%04x (lo=%04x hi=%04x) :\n", hi, lo, lo, hi);
    int n = 0;
    for (unsigned a = 0; a + 1 < 0x4000; a++) {
        if (dsp->data[a] == lo && dsp->data[a+1] == hi && n < 12) {
            printf("    data[0x%04x] = %04x %04x   (lo puis hi)\n", a, lo, hi); n++; }
        if (dsp->data[a] == hi && dsp->data[a+1] == lo && n < 12) {
            printf("    data[0x%04x] = %04x %04x   (hi puis lo)\n", a, hi, lo); n++; }
    }
    for (unsigned a = 0; a + 1 < 0x10000; a++) {
        if (dsp->prog[a] == lo && dsp->prog[a+1] == hi && n < 24) {
            printf("    prog[0x%04x] = %04x %04x\n", a, lo, hi); n++; }
    }
    int nlo = 0, nhi = 0;
    for (unsigned a = 0; a < 0x4000; a++) { if (dsp->data[a]==lo) nlo++; if (dsp->data[a]==hi) nhi++; }
    printf("    occurrences isolees en data : lo(%04x) x%d, hi(%04x) x%d\n", lo, nlo, hi, nhi);
    if (!n) printf("    motif absent de la memoire : le mot est CALCULE, pas stocke\n");
}
if (calypso_getenv("REJEU_OPCODES")) {
    printf("  %lu decodages\n", g_n_dec);
    printf("  opcodes de l'EGALISEUR (0x8400-0x84ff), par decodage :\n");
    for (int rang = 0; rang < 16; rang++) {
        unsigned best = 0; unsigned long bv = 0;
        for (unsigned i = 0; i < 65536; i++) if (g_op_eq[i] > bv) { bv = g_op_eq[i]; best = i; }
        if (!bv) break;
        double q = g_n_dec ? (double)bv / g_n_dec : 0;
        printf("    op=%04x  %8lu   /dec = %8.2f %s\n", best, bv, q,
               (g_n_dec && bv % g_n_dec == 0) ? "  (entier)" : "  <<< NON ENTIER");
        g_op_eq[best] = 0;
    }
    printf("\n");
    printf("  opcodes du DECODEUR (0x9800-0x9bff) :\n");
    { unsigned long st_trn = 0, cmps = 0;
      for (unsigned i = 0; i < 65536; i++) {
          if ((i & 0xFF00) == 0x8D00) st_trn += g_op_dec[i];
          if ((i >> 8) == 0x8E || (i >> 8) == 0x8F) cmps += g_op_dec[i];
      }
      printf("    CMPS (0x8E/0x8F) : %lu     ST TRN (0x8D00) : %lu\n", cmps, st_trn); }
    for (int rang = 0; rang < 14; rang++) {
        unsigned best = 0; unsigned long bv = 0;
        for (unsigned i = 0; i < 65536; i++) if (g_op_dec[i] > bv) { bv = g_op_dec[i]; best = i; }
        if (!bv) break;
        printf("    op=%04x  %8lu fois\n", best, bv);
        g_op_dec[best] = 0;
    }
    printf("  opcodes du demod SB, par nombre de passages :\n");
    for (int rang = 0; rang < 26; rang++) {
        unsigned best = 0; unsigned long bv = 0;
        for (unsigned i = 0; i < 65536; i++) if (g_op_n[i] > bv) { bv = g_op_n[i]; best = i; }
        if (!bv) break;
        printf("    op=%04x  %8lu fois   vu en pc=%04x\n", best, bv, g_op_pc[best]);
        g_op_n[best] = 0;
    }
}
printf("  division 16 pas : rpt(7d1c)=%lu subc(7d1d)=%lu quotient(7d1e)=%lu mpy(81e4)=%lu\n",
       hit_7d1c, hit_7d1d, hit_7d1e, hit_81e4);
printf("  sites FIRS du SB : 0x8478=%lu  0x8492(rpt)=%lu  0x8493=%lu  0x8497(sth B->softs)=%lu\n",
       hit_8478, hit_8492, hit_8493, hit_8497);
printf("  trames jouees      : %u\n", fn_cur + 1);
  printf("  chaine SB (pas-a-pas) : job b219=%lu  fcall 7a16=%lu  demod 7c31=%lu  corr 84a1=%lu  decodeur
↪  9841=%lu  | corr FB 770a=%lu\n",
         hit_b219, hit_7a16, hit_7c31, hit_84a1, hit_9841, hit_770a);
```

```
    printf("  FB acceptes (ARM)  : %d\n", n_fb_ok);
    printf("  SB tentees / CRC KO: %d / %d\n", n_sb_try, n_sb_crcfail);
    printf("  CMPS (0x8E/0x8F) : %ld fois dans le demod SB, %ld hors | famille 0xE0-0xE3 en SB : %ld | ST TRN :
↪  %ld\n",
           n_cmps_sb, n_cmps_hors, n_e0_sb, n_sttrn_sb);
    {
        printf("  XC (0xFD/0xFF) par region de PC :");
        for (int b = 0; b < 64; b++) if (n_xc_reg[b]) printf(" 0x%04x=%ld", b << 10, n_xc_reg[b]);
        printf("\n");
        printf("  CMPS (0x8E/0x8F) par region de PC :");
        for (int b = 0; b < 64; b++)
            if (n_cmps_reg[b]) printf(" 0x%04x=%ld", b << 10, n_cmps_reg[b]);
        printf("\n");
    }
    printf("      detail : FIRS(E0)=%ld  LMS(E1)=%ld  SQDST(E2)=%ld  ABDST(E3)=%ld"
           "   <- inertes si CALYPSO_ISA_E0_FAM=1\n",
           n_e0x_sb[0], n_e0x_sb[1], n_e0x_sb[2], n_e0x_sb[3]);
    printf("  erreurs DSP signalees: %d, dont erreur 8 (anneau DMA): %d\n", n_err_dsp, n_err8);
    printf("  CRC OK: %d, dont VRAIES (BSIC injecte + T3 valide + FN = trame): %d%s\n",
           n_crc_ok, n_sb_vraies,
           cellule_sch_partout ? "   [FN non qualifiable : SCH_PARTOUT actif]" : "");
    if (verdict == 1) printf("  VERDICT : SB DECODEE  BSIC=%d FN=%u (sb=0x%08x)\n", sb_bsic, sb_fn, sb_word);
    else if (verdict == -1) printf("  VERDICT : ABANDON (le firmware a renonce)\n");
    else printf("  VERDICT : AUCUNE SB (ni decodee ni abandon) en %ld trames\n", trames);
    return verdict == 1 ? 0 : 1;
}
```

3.22 /opt/GSM/c54x_exe/src/rejouer.h

306 octets, 9 lignes → 9 lignes

```
/* SPDX-License-Identifier: GPL-2.0-or-later */
#ifndef REJOUER_H
#define REJOUER_H
#include <stdint.h>
#include "calypso_c54x.h"
extern uint32_t g_c54x_exe_fn;
int rejouer(C54xState *dsp, uint16_t *api_ram, long trames, long insns,
            const char *iq_mode, int amp, int bsic, int verbeux);
#endif
```

3.23 /opt/GSM/c54x_exe/src/tsp_tx.c

3581 octets, 65 lignes → 65 lignes

```
/* SPDX-License-Identifier: GPL-2.0-or-later */
/* tsp_tx.c - see tsp_tx.h. */
#include <string.h>
#include "tsp_tx.h"

/* DROM 0xa0c9..0xa0d8: the 8 training sequences of the normal bursts (26 bits, 2 words each), as
 * the ROM copies them into its transmitter (0x85d6: 0xa0c9 + 2 * tsc). */
static const char *const TSC_ROM[8] = {
    "00100101110000100010010111", "00101101110111100010110111", "01000011101110100100001110",
    "01000111101101000100011110", "00011010111001000001101011", "01001110101100000100111010",
    "10100111110110001010011111", "11101111000100101110111100",
};
/* DROM 0xa0d9..0xa0db: the synchronisation sequence of the access burst (41 bits, 05.02 5.2.7). */
static const char SYNC_RACH[] = "01001011011111111100110011010101000111100 0";
/* 05.02 5.2.5: extended training sequence of the synchronisation burst (64 bits, at offset 42 of
 * the burst); 5.2.6: the whole dummy burst (148 bits, tail bits included). */
static const char SYNC_SB[] = "10111001011000100000010000001111001011010100010101110110000110 11";
```

```
static const char DUMMY_BURST[] =
 ↪  "00011111011011101100000101001001110000010010001000000011111000111000101110001011100010101110100101000110011001110011111010000
 ↪  ;

static bool motif_a(const uint8_t *flux, int pos, const char *m, int n)
{
    if (pos < 0 || pos + n > 160) return false;
    for (int i = 0; i < n; i++) if (flux[pos + i] != (uint8_t)(m[i] - '0')) return false;
    return true;
}

bool tsp_tx_lire(const uint16_t *data, struct tsp_tx_burst *b, bool *nouveau)
{
    static uint16_t prec[16];
    for (unsigned a = 0x3cbb; a + 18 <= 0x3d00; a++) {
        if (data[a] != 0x1c0a) continue;
        const uint16_t *w = &data[a + 2];
        int ok = 1;
        for (int i = 0; i < 16; i++) if ((w[i] & 0x3e) != 0x06 && (w[i] & 0x3e) != 0x04) { ok = 0; break; }
        if (!ok) continue;
        memset(b, 0, sizeof *b);
        b->adresse = (uint16_t)a;
        memcpy(b->mots, w, sizeof b->mots);
        for (int i = 0; i < 16; i++) for (int k = 0; k < 10; k++) b->flux[10 * i + k] = (w[i] >> (15 - k)) & 1;
        *nouveau = memcmp(w, prec, sizeof prec) != 0;
        memcpy(prec, w, sizeof prec);
        b->tsc = -1; b->type = TSP_TX_INCONNU; b->offset = -1;
        /* access burst: [8 guard][8 extended tail][41 sync]... -> sync at 16, burst at 8 */
        if (motif_a(b->flux, 16, SYNC_RACH, 41)) { b->type = TSP_TX_RACH; b->offset = 8; }
        else {
            /* normal burst: 3 tail, 57, hl, 26 TSC, hu, 57, 3 -> TSC at offset + 61 */
            for (int t = 0; t < 8 && b->type == TSP_TX_INCONNU; t++)
                for (int p = 61; p + 26 <= 160 && p - 61 + 148 <= 160; p++)
                   if (motif_a(b->flux, p, TSC_ROM[t], 26)) { b->type = TSP_TX_NB; b->tsc = t; b->offset = p - 61;
                     ↪  break; }
            /* BTS bursts (trx firmware): SB (64-bit training at +42), dummy, FB (148 zeros) */
            for (int p = 0; b->type == TSP_TX_INCONNU && p + 148 <= 160; p++) {
                if (motif_a(b->flux, p + 42, SYNC_SB, 64)) { b->type = TSP_TX_SB; b->offset = p; }
                else if (motif_a(b->flux, p, DUMMY_BURST, 148)) { b->type = TSP_TX_DUMMY; b->offset = p; }
            }
            if (b->type == TSP_TX_INCONNU) {
                int zeros = 0; for (int i = 0; i < 160; i++) zeros += !b->flux[i];
                if (zeros >= 148) { b->type = TSP_TX_FB; b->offset = 0; }
            }
        }
        if (b->offset >= 0) memcpy(b->bits, b->flux + b->offset, 148);
        else memcpy(b->bits, b->flux, 148);
        return true;
    }
    return false;
}
```

3.24 /opt/GSM/c54x_exe/src/tsp_tx.h

2216 octets, 39 lignes → 39 lignes

```
/* SPDX-License-Identifier: GPL-2.0-or-later */
/*
 * tsp_tx.h - the FINAL uplink burst of the ROM, as it hands it to the ABB.
 *
 * Every frame, the ROM builds in data[0x3cbb..] the TSP script for the TWL3025 (PROM0
 * 0xb5b8 `stm #15547,ar2`). When a burst goes out, the script holds the word TOGBR2 = 0x1c0a, a
 * power word, then the 16 BULDATA words written by the transmitter 0x8605: (10 data bits) << 6 |
 * (register BULDATA1 = 3) << 1, i.e. `xxxx xxxx xx00 0110`. The 160 bits serialised this way are
```

```
 * the ROM output after training sequence (DROM 0xa0c9), RACH sync (DROM 0xa0d9), tail bits
 * (DROM 0xa0dc), A5 ciphering (XOR at 0x85d1) and I/Q inversion (XOR 0xaa80 at 0xb773): what the
 * silicon sends to the antenna, up to the format. Measured (dsp_tester rach, tx-sdcch,
 * tx-sdcch-a5): bit-identical to libosmocoding.
 */
#ifndef TSP_TX_H
#define TSP_TX_H
#include <stdint.h>
#include <stdbool.h>

/* SB, DUMMY, FB: the bursts of a BTS (trx firmware, start-native.sh --osmo-trx), sent by the
 * RACH task that the extcode hijacks; published in calypso_tx_rom for the air. */
enum tsp_tx_type { TSP_TX_INCONNU = 0, TSP_TX_RACH = 1, TSP_TX_NB = 2, TSP_TX_SB = 3, TSP_TX_DUMMY = 4, TSP_TX_FB
 ↪  = 5 };

struct tsp_tx_burst {
    uint16_t mots[16];      /* the 16 BULDATA words as they are */
    uint8_t  flux[160];     /* the 160 serialised bits (10 per word, MSB first) */
    uint8_t  bits[148];     /* the 148-bit burst (0/1): access burst or normal burst */
    int      type;          /* enum tsp_tx_type */
    int      tsc;           /* NB: recognised training sequence (0..7), -1 otherwise */
    int      offset;        /* position of the burst in flux[] (8 for the RACH, else TSC - 61) */
    uint16_t adresse;       /* address of the 0x1c0a marker in data memory */
};

/* Looks for the TSP script in data[0x3cbb..0x3d00[ and reads the burst from it. Returns true if a
 * complete burst (16 BULDATA words) is present; `nouveau` tells whether it differs from the 16 words
 * of the previous call (same process): this is the "a burst has just been sent" signal when no ring
 * pointer has moved (RACH). */
bool tsp_tx_lire(const uint16_t *data, struct tsp_tx_burst *b, bool *nouveau);

#endif
```

3.25 /opt/GSM/c54x_exe/src/verbosite.c

5631 octets, 153 lignes → 153 lignes

```
/* SPDX-License-Identifier: GPL-2.0-or-later */
/*
 * verbosite.c - the -v levels of c54x_exe.
 *
 * The C54x core (qosmo/hw/arm/calypso/l1-dsp/calypso_c54x.c) carries about a
 * hundred "[c54x] NAME ..." probes on stderr, some of them unconditional
 * (BRANCH-TRACE alone emits 700 lines at boot) [2026-09-16]. They serve QEMU
 * as much as this binary, so gating them one by one through environment
 * variables would mean touching the shared sources for every new probe.
 *
 * Instead the core is left alone: stderr is redirected into a pipe and a
 * thread re-reads the lines, passing through only those whose level is <= the
 * requested one. Classification is by keyword on the probe NAME rather than an
 * exhaustive table, so a new probe lands at a sensible level without being
 * declared. The summary reports how many lines each level hid, so the caller
 * knows what to ask for.
 */
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include "verbosite.h"

static int       g_niveau = 0;
static int       g_stderr_reel = -1;   /* dup(2) of the original */
```

```
static int       g_tube_lecture = -1;
static pthread_t g_fil;
static bool      g_actif = false;
static unsigned long g_masquees[VERBOSITE_MAX + 1];
static pthread_mutex_t g_mu = PTHREAD_MUTEX_INITIALIZER;

static bool contient(const char *l, const char *mot) { return strstr(l, mot) != NULL; }

/* Lowest -v level at which a line is shown. */
static int niveau_ligne(const char *l)
{
    /* memory probes whose NAME contains an error keyword: classify first */
    if (contient(l, "ERRWATCH"))
        return 4;
    /* 0: what breaks */
    if (contient(l, "FATAL") || contient(l, "ERR") || contient(l, "abort") ||
        contient(l, "CORRUPT") || contient(l, "TRAP") || contient(l, "manquante") ||
        contient(l, "cannot") || contient(l, "impossible") || contient(l, "a echoue"))
        return 0;
    /* 1: what is worrying */
    if (contient(l, "WARN") || contient(l, "echou") || contient(l, "failed") ||
        contient(l, "pas parque") || contient(l, "perdue"))
        return 1;
    /* 5: single-stepping */
    if (contient(l, "TRACE") || contient(l, "LOOP") || contient(l, "CYCLE") ||
        contient(l, "HIST") || contient(l, "RING") || contient(l, "-STACK"))
        return 5;
    /* 4: memory probes */
    if (contient(l, "WATCH") || contient(l, "DUMP") || contient(l, "SCAN") ||
        contient(l, "PROBE") || contient(l, "MAP") || contient(l, "SP-") ||
        contient(l, "FLOW") || contient(l, "HOT-OPS") || contient(l, "FIRST"))
        return 4;
    /* 3: tasks, API RAM, interrupts, FB/SB paths */
    if (contient(l, "TASK") || contient(l, "DISPATCH") || contient(l, "] FB") ||
        contient(l, "-FB") || contient(l, "FBDET") || contient(l, "FBWATCH") ||
        contient(l, "FBSB") || contient(l, "FBCALL") || contient(l, "FBROUTE") ||
        contient(l, "FBENTRY") || contient(l, "FBGATE") || contient(l, "FBMODE") ||
        contient(l, "SYNC") || contient(l, "FEED") || contient(l, "VEC") ||
        contient(l, "INTM") || contient(l, "IFR") || contient(l, "IMR") ||
        contient(l, "PMST") || contient(l, "MMR") || contient(l, "AFC") ||
        contient(l, "IRQ") || contient(l, "ACK") || contient(l, "IDLE") ||
        contient(l, "DMA") || contient(l, "RIF") || contient(l, "bsp") ||
        contient(l, "BSP") || contient(l, "TPU") || contient(l, "API"))
        return 3;
    /* 2: the rest - gate banners (ACTIF/INACTIVE), CALYPSO_* reads, boot,
     *    reset, [calypso-debug], and anything not coming from the core */
    return 2;
}

static void *fil_filtre(void *arg)
{
    (void)arg;
    FILE *in = fdopen(g_tube_lecture, "r");
    FILE *out = fdopen(g_stderr_reel, "w");
    if (!in || !out) {
        return NULL;
    }
    setvbuf(out, NULL, _IOLBF, 0);
    char ligne[4096];
    while (fgets(ligne, sizeof(ligne), in)) {
        int n = niveau_ligne(ligne);
        if (n <= g_niveau) {
            fputs(ligne, out);
        } else {
            pthread_mutex_lock(&g_mu);
            g_masquees[n]++;
```

```
            pthread_mutex_unlock(&g_mu);
        }
    }
    fflush(out);
    return NULL;
}

void verbosite_installer(int niveau)
{
    if (niveau < 0) niveau = 0;
    if (niveau > VERBOSITE_MAX) niveau = VERBOSITE_MAX;
    g_niveau = niveau;
    if (niveau >= VERBOSITE_MAX) {
        return;                         /* raw: nothing to install */
    }
    int tube[2];
    if (pipe(tube) < 0) {
        return;
    }
    fflush(stderr);
    g_stderr_reel = dup(STDERR_FILENO);
    g_tube_lecture = tube[0];
    dup2(tube[1], STDERR_FILENO);
    close(tube[1]);
    if (pthread_create(&g_fil, NULL, fil_filtre, NULL) != 0) {
        dup2(g_stderr_reel, STDERR_FILENO);
        return;
    }
    g_actif = true;
}

void verbosite_retirer(void)
{
    if (!g_actif) {
        return;
    }
    fflush(stderr);
    /* restore fd 2: the pipe write end closes, the thread sees EOF */
    dup2(g_stderr_reel, STDERR_FILENO);
    pthread_join(g_fil, NULL);
    g_actif = false;
}

void verbosite_bilan(FILE *out)
{
    unsigned long total = 0;
    for (int i = 0; i <= VERBOSITE_MAX; i++) total += g_masquees[i];
    if (!total) {
        return;
    }
    fprintf(out, "  traces DSP masquees : %lu  (", total);
    const char *nom[] = { "erreurs", "-v", "-vv", "-vvv", "-vvvv", "-vvvvv", "brut" };
    bool premier = true;
    for (int i = 1; i <= 5; i++) {
        if (!g_masquees[i]) continue;
        fprintf(out, "%s%lu avec %s", premier ? "" : ", ", g_masquees[i], nom[i]);
        premier = false;
    }
    fprintf(out, ")\n");
}
```

3.26 /opt/GSM/c54x_exe/src/verbosite.h

749 octets, 18 lignes → 18 lignes

```c
/* SPDX-License-Identifier: GPL-2.0-or-later */
#ifndef VERBOSITE_H
#define VERBOSITE_H
#include <stdio.h>

/* Levels: 0 = errors only (default)    -v = + warnings
 *         -vv    = + lifecycle (boot, reset, gate banner)
 *         -vvv   = + tasks, API RAM, interrupts
 *         -vvvv  = + memory probes (WATCH, DUMP, SCAN, MAP, SP-*)
 *         -vvvvv = + per-instruction traces (BRANCH, LOOP, TERM, CYCLE)
 *         -vvvvvv = everything, raw stderr, unfiltered */
#define VERBOSITE_MAX 6

void verbosite_installer(int niveau);   /* call before any DSP call           */
void verbosite_retirer(void);           /* drain the pipe, restore stderr     */
void verbosite_bilan(FILE *out);        /* count of suppressed lines, per level */

#endif
```

3.27 /opt/GSM/c54x_exe/tools/analyse_sb.py

1096 octets, 14 lignes → 14 lignes

```python
#!/usr/bin/env python3
# analyse_sb.py - summary of the SB attempts of a verbose replay (c54x_exe --rejouer --verbeux):
#   SCH present (burst inside the window), CRC OK rate, mean angle OK/KO, TOA.
# Usage: tools/analyse_sb.py rejeu.log "label"
import re,sys,collections
L=open(sys.argv[1],errors='replace').read().split('\n')
att=[]
for l in L:
    m=re.search(r'SBresp att=(\d) fn=(\d+) p51=(\d+) crc=(\w+) a_sch=(\w+) (\w+) (\w+) (\w+) (\w+) toa=(-?\d+)
 ↪ pm=(-?\d+) angle=(-?\d+) snr=(-?\d+) sb_cmd_fn=(\d+)',l)
    if m: att.append(dict(att=int(m.group(1)),crc=m.group(4),toa=int(m.group(10)),angle=int(m.group(12)),snr=int
     ↪ (m.group(13))))
pres=[a for a in att if a['snr']>1000]           # burst present inside the window
ok=[a for a in pres if a['crc']=='OK']; ko=[a for a in pres if a['crc']!='OK']
ma=lambda g: (sum(a['angle'] for a in g)/len(g)) if g else float('nan')
print(f"{sys.argv[2]:28s} SCH presents={len(pres):3d} OK={len(ok):3d} ({100*len(ok)/max(1,len(pres)):3.0f}%)
 ↪ angle OK={ma(ok):5.0f} KO={ma(ko):5.0f}  toa OK={collections.Counter(a['toa'] for a in ok).most_common(2)}")
```

3.28 /opt/GSM/c54x_exe/tools/cmp_codeur.py

2814 octets, 49 lignes → 49 lignes

```python
"""cmp_codeur.py <trace> : codeur FR de la ROM contre libgsm, sur les memes 160 echantillons."""
import ctypes, sys, numpy as np
gsm = ctypes.CDLL("libgsm.so.1"); oc = ctypes.CDLL("/usr/local/lib/libosmocodec.so.4")
gsm.gsm_create.restype = ctypes.c_void_p
order = (ctypes.c_uint16 * 260).in_dll(oc, "gsm610_bitorder")
def bit(b, i): return (b[i >> 3] >> (7 - (i & 7))) & 1
def ti_vers_fr(words):
    ti = bytearray()
    for w in words[3:20]: ti += bytes([w >> 8, w & 0xff])
    canon = bytearray(33)
    for i in range(260):
        si = i + 4 if i >= 182 else i
        if bit(ti, si): canon[order[i] >> 3] |= 0x80 >> (order[i] & 7)
    fr = bytearray(33); fr[0] = 0xd0 | (canon[0] >> 4)
    for i in range(1, 33): fr[i] = ((canon[i-1] << 4) | (canon[i] >> 4)) & 0xff
    return bytes(fr)
def explode(g, fr):
```

```
    p = (ctypes.c_short * 76)(); gsm.gsm_explode(ctypes.c_void_p(g), fr, p); return list(p)
NOMS = ['LAR1','LAR2','LAR3','LAR4','LAR5','LAR6','LAR7','LAR8'] + [f'{n}{k}' for k in range(4) for n in
 ↪  ['Nc','bc','Mc','xmax'] + [f'x{j}' for j in range(13)]]
so, paires = None, []
for l in open(sys.argv[1]):
    if l.startswith('SO'): so = [int(v) for v in l.split()[1:]]
    elif l.startswith('ENC') and so is not None:
        a = l.split(' A ')[1].split(' B ')[0].split()
        paires.append((so, [int(w, 16) for w in a])); so = None
genc = gsm.gsm_create(); gdec = gsm.gsm_create(); gx = gsm.gsm_create()
diff = np.zeros(76); n = 0; sortie_rom, sortie_ref, entree = [], [], []
for so, w in paires:
    x = (ctypes.c_short * 160)(*[max(-32768, min(32767, v * 2)) for v in so])
    ref = (ctypes.c_ubyte * 33)(); gsm.gsm_encode(ctypes.c_void_p(genc), x, ref)
    fr = ti_vers_fr(w)
    pr, pf = explode(gx, bytes(ref)), explode(gx, fr)
    if max(abs(v) for v in so) > 200:
        diff += np.array(pr) != np.array(pf); n += 1
    o = (ctypes.c_short * 160)(); gsm.gsm_decode(ctypes.c_void_p(gdec), (ctypes.c_ubyte * 33)(*fr), o); sortie_rom
 ↪  += list(o)
    entree += [v * 2 for v in so]
print(f"{len(paires)} trames, {n} avec du signal ; fraction de trames ou le parametre ROM != libgsm :")
print('  ' + '  '.join(f"{NOMS[i]}={diff[i]/max(n,1):.2f}" for i in range(76) if i < 12 or i % 17 in (8, 9, 10,
 ↪  11)))
e = np.array(entree, float); r = np.array(sortie_rom, float)
def spec(x):
    S = np.zeros(513)
    for i in range(0, len(x) - 1024, 1024):
        s = x[i:i+1024]
        if np.sqrt((s**2).mean()) > 300: S += abs(np.fft.rfft(s * np.hanning(1024)))**2
    f = np.arange(513) * 8000 / 1024; t = S.sum() or 1
    return [round(100 * S[(f >= a) & (f < a + 1000)].sum() / t, 1) for a in (0, 1000, 2000, 3000)]
print('entree micro  energie % [0-1k,1-2k,2-3k,3-4k]', spec(e))
print('codeur ROM decode par libgsm               ', spec(r))
np.array(r, '<i2').tofile(sys.argv[1] + '.ul_rom_decode.raw')
```

3.29 /opt/GSM/c54x_exe/tools/comparer_parole.py

7764 octets, 199 lignes → 199 lignes

```
#!/usr/bin/env python3
# comparer_parole.py - downlink speech and FACCH: ROM against reference.
#
# Decides between "the bursts delivered to the DSP are bad" and "the DSP
# decodes badly" by decoding the same bursts with libosmocoding and comparing
# with what the ROM returned:
#
#   /dev/shm/calypso_tch_dl.bin   TCH bursts the BSP delivered to the DSP
#                                 (tick BE32, BTS fn BE32, 148 bits 0/1)
#   /dev/shm/calypso_add_dl.bin   frames the ROM returned (montant.c,
#                                 noter_dl: speech converted TI -> FR, FACCH)
#
# The bursts are deciphered with the session Kc ([a5] lines of dsp.log) and
# the COUNT of their own fn, then decoded by gsm0503_tch_fr_decode, as the
# bridge does. Each block is paired with the frame the ROM delivered right
# after it; for speech the wrong bits are counted per class (Ia 50, Ib 132,
# II 78), for the FACCH the 23 bytes are compared.
#
# Usage: tools/comparer_parole.py [--dsp-log /tmp/c54x-pont/dsp.log] [--kc HEX]
import argparse
import collections
import ctypes
import re
import struct
```

```
import sys

sys.path.insert(0, "/opt/GSM/osmo-operator")
from pont import gsm  # noqa: E402

FR = gsm.FR_BYTES
MAC = gsm.MACBLOCK_LEN


def lire_bursts(path):
    raw = open(path, "rb").read()
    out = []
    for o in range(0, len(raw) - 155, 156):
        tick, fn = struct.unpack(">II", raw[o:o + 8])
        out.append((tick, fn, bytes(raw[o + 8:o + 156])))
    return out


def lire_rom(path):
    raw = open(path, "rb").read()
    out = []
    for o in range(0, len(raw) - 47, 48):
        fn, typ, n, etat, err = struct.unpack_from("<IBBHH", raw, o)
        out.append((fn, typ, etat, err, bytes(raw[o + 12:o + 12 + n])))
    return out


def lire_kc(dsp_log):
    """(fn, Kc) of each new Kc seen by the A5 coprocessor."""
    kcs = []
    for line in open(dsp_log, errors="replace"):
        m = re.search(r"\[a5\] #\d+ A5/\d fn=(\d+) .*Kc=([0-9a-f]{16})", line)
        if m:
            fn, kc = int(m.group(1)), bytes.fromhex(m.group(2))
            if not kcs or kcs[-1][1] != kc:
                kcs.append((fn, kc))
    return kcs


def kc_pour(kcs, fn):
    best = None
    for f, kc in kcs:
        if f <= fn + 2:
            best = kc
    return best


def classes_fr():
    """class (0 Ia, 1 Ib, 2 II) of each bit of the standard FR format (260 bits
    after the 0xd header), from gsm610_bitorder: d[k] = s[bitorder[k]]."""
    lib = ctypes.CDLL("libosmocodec.so")
    order = (ctypes.c_uint16 * 260).in_dll(lib, "gsm610_bitorder")
    cl = [0] * 260
    for k in range(260):
        cl[order[k]] = 0 if k < 50 else (1 if k < 182 else 2)
    return cl


def bits260(fr):
    v = int.from_bytes(fr[:FR], "big")
    s = bin(v)[2:].zfill(8 * FR)
    return s[4:4 + 260]


def main():
    ap = argparse.ArgumentParser()
```

```
ap.add_argument("--dsp-log", default="/tmp/c54x-pont/dsp.log")
ap.add_argument("--bursts", default="/dev/shm/calypso_tch_dl.bin")
ap.add_argument("--rom", default="/dev/shm/calypso_add_dl.bin")
ap.add_argument("--kc", help="Kc en hexa (sinon lu dans dsp.log)")
ap.add_argument("-v", action="store_true", help="une ligne par trame")
a = ap.parse_args()

bursts = lire_bursts(a.bursts)
rom = lire_rom(a.rom)
if not bursts or not rom:
    sys.exit("rien a comparer : %d bursts, %d trames ROM" % (len(bursts), len(rom)))
kcs = [(0, bytes.fromhex(a.kc))] if a.kc else lire_kc(a.dsp_log)
offs = collections.Counter(t - f for t, f, _ in bursts)
off = offs.most_common(1)[0][0]
print("bursts : %d (fn BTS %d..%d, tick - fn = %s)" % (
    len(bursts), bursts[0][1], bursts[-1][1], dict(offs.most_common(3))))
print("trames ROM : %d (parole %d, FACCH %d) ; Kc connus : %s" % (
    len(rom), sum(1 for r in rom if r[1] == 0), sum(1 for r in rom if r[1] == 1),
    [k.hex() for _, k in kcs]))

# Reference decoding: a block ends on the 4th burst of a half-block.
par_fn = {fn: b for _, fn, b in bursts}
blocs = {}   # end fn -> (rc, data, errors)
for fn in sorted(par_fn):
    if not gsm.is_tch_carrier(fn) or gsm.tch_burst_index(fn) % 4 != 3:
        continue
    seq, f = [fn], fn
    while len(seq) < 8:
        f -= 1
        while not gsm.is_tch_carrier(f):
            f -= 1
        seq.append(f)
    seq.reverse()
    if any(x not in par_fn for x in seq):
        continue
    kc = kc_pour(kcs, fn)
    coded = []
    for x in seq:
        b = par_fn[x]
        if kc:
            ks, _ = gsm.a5_keystream(1, kc, x)
            b = gsm.a5_xor(b, ks)
        coded.append(gsm.coded_from_burst(b))
    buf = (gsm.sbit * (8 * 116)).from_buffer_copy(gsm._soft(coded))
    out = (ctypes.c_uint8 * FR)()
    ne, nb = ctypes.c_int(), ctypes.c_int()
    rc = gsm._cod.gsm0503_tch_fr_decode(out, buf, 1, 0, ctypes.byref(ne), ctypes.byref(nb))
    blocs[fn] = (rc, bytes(out), ne.value)
n_fr = sum(1 for r in blocs.values() if r[0] == FR)
n_fa = sum(1 for r in blocs.values() if r[0] == MAC)
print("reference : %d blocs, %d parole, %d FACCH, %d en echec" % (
    len(blocs), n_fr, n_fa, len(blocs) - n_fr - n_fa))

cl = classes_fr()
fins = sorted(blocs)
stats = collections.defaultdict(list)
facch = collections.Counter()
for fn_rom, typ, etat, err, data in rom:
    cible = fn_rom - off
    cand = [f for f in fins if cible - 8 <= f <= cible]
    if not cand:
        stats["sans_reference"].append(0)
        continue
    f = cand[-1]
    rc, ref, ne = blocs[f]
    if typ == 0:
```

```
            if rc != FR:
                stats["parole_ref_pas_parole"].append(rc)
                continue
            b1, b2 = bits260(ref), bits260(data)
            diff = [i for i in range(260) if b1[i] != b2[i]]
            par = [sum(1 for i in diff if cl[i] == c) for c in range(3)]
            stats["parole"].append((len(diff), par, err, ne, etat))
            if a.v:
                print("  parole fn=%d ref_fin=%d etat=%04x err_rom=%d err_ref=%d faux=%d (Ia %d, Ib %d, II %d)"
                      % (fn_rom, f, etat, err, ne, len(diff), *par))
        else:
            fire = bool(etat & 0x0040)
            if rc == MAC:
                facch["ref FACCH, ROM %s" % ("FIRE KO" if fire else
                      ("identique" if data[:MAC] == ref[:MAC] else "DIFFERENTE"))] += 1
            else:
                facch["ref pas FACCH (rc=%d), ROM %s" % (rc, "FIRE KO" if fire else "ok")] += 1
            if a.v:
                print("  facch  fn=%d ref_fin=%d etat=%04x rc_ref=%d ROM=%s REF=%s" % (
                    fn_rom, f, etat, rc, data[:6].hex(" "), ref[:6].hex(" ")))

    p = stats["parole"]
    if p:
        tot = [x[0] for x in p]
        print("\nPAROLE : %d trames appariees" % len(p))
        print("  identiques aux bits pres : %d" % sum(1 for x in tot if x == 0))
        print("  bits faux par trame : moyenne %.1f, max %d (sur 260)" % (sum(tot) / len(tot), max(tot)))
        for c, nom in enumerate(("Ia (50)", "Ib (132)", "II (78)")):
            v = [x[1][c] for x in p]
            print("  classe %-8s : moyenne %.2f, trames touchees %d" % (nom, sum(v) / len(v), sum(1 for x in v if
             ↪  x)))
        print("  erreurs canal : ROM (a_dd[2]) moy %.1f | reference (Viterbi) moy %.1f" % (
            sum(x[2] for x in p) / len(p), sum(x[3] for x in p) / len(p)))
    for k in ("parole_ref_pas_parole", "sans_reference"):
        if stats[k]:
            print("  %s : %d" % (k, len(stats[k])))
    if facch:
        print("\nFACCH (trames a_fd vues par la ROM) :")
        for k, v in facch.most_common():
            print("  %-40s %d" % (k, v))


if __name__ == "__main__":
    main()
```

3.30 /opt/GSM/c54x_exe/tools/decoder_add.py

2413 octets, 46 lignes → 46 lignes

```
#!/usr/bin/env python3
# decoder_add.py - decode with libgsm the FR frames recorded by montant.c and look for a tone.
#
#   /dev/shm/calypso_add_dl.bin   downlink speech returned by the ROM (a_dd), converted TI -> FR
#   /dev/shm/calypso_add_ul.bin   uplink speech (a_du): type 1 = raw TI, type 0 = converted FR
#
# 48-byte records: fn u32, type u8, n u8, state u16, err u16, [12..12+n] data.
# The downlink a_dd is bit-exact with what the BTS transmits (comparer_parole.py): if it
# does not carry the tone injected at the microphone, the uplink (or the echo) lost it.
#
# Usage: tools/decoder_add.py [dl|ul] [--ton 1000] [--out file.raw]
import argparse, ctypes, math, struct, sys
import numpy as np
ap = argparse.ArgumentParser()
ap.add_argument("sens", nargs="?", default="dl", choices=["dl", "ul"])
```

```
ap.add_argument("--ton", type=float, default=1000.0, help="frequence cherchee (Hz)")
ap.add_argument("--out", help="PCM s16le 8 kHz decode")
a = ap.parse_args()
path = "/dev/shm/calypso_add_%s.bin" % a.sens
raw = open(path, "rb").read()
recs = []
for o in range(0, len(raw) - 47, 48):
    fn, typ, n, etat, err = struct.unpack_from("<IBBHH", raw, o)
    recs.append((fn, typ, n, etat, err, raw[o + 12:o + 12 + n]))
fr = [r for r in recs if r[1] == 0 and r[2] == 33]
print("%s : %d enregistrements, %d trames FR (fn %d..%d), nibble de tete %s" % (
    path, len(recs), len(fr), fr[0][0] if fr else 0, fr[-1][0] if fr else 0,
    sorted({hex(x[5][0] >> 4) for x in fr})))
g = ctypes.CDLL("libgsm.so.1"); g.gsm_create.restype = ctypes.c_void_p
g.gsm_decode.argtypes = [ctypes.c_void_p, ctypes.POINTER(ctypes.c_ubyte), ctypes.POINTER(ctypes.c_short)]
h = g.gsm_create()
pcm = []
for _, _, _, _, _, d in fr:
    out = (ctypes.c_short * 160)()
    g.gsm_decode(h, (ctypes.c_ubyte * 33)(*d), out)
    pcm.extend(out)
x = np.array(pcm, dtype=float); sr = 8000
if a.out:
    open(a.out, "wb").write(np.array(pcm, dtype="<i2").tobytes())
print(" sec    rms  raie(Hz)  %%dans %.0f+-50  fn" % a.ton)
for s in range(int(len(x) // sr)):
    v = x[s * sr:(s + 1) * sr]
    rms = math.sqrt(np.mean(v * v)) + 1e-9
    f = np.fft.rfftfreq(len(v), 1 / sr); sp = np.abs(np.fft.rfft(v * np.hanning(len(v)))) ** 2
    k = int(np.argmax(sp[1:])) + 1; band = (f >= a.ton - 50) & (f <= a.ton + 50)
    print("%4d %6.0f %9.0f %12.0f  %d" % (s, rms, f[k], 100 * sp[band].sum() / (sp[1:].sum() + 1e-9), fr[min(s *
     ↪  50, len(fr) - 1)][0]))
```

3.31 /opt/GSM/c54x_exe/tools/dsp_banc_commun.c

33892 octets, 825 lignes → 825 lignes (3 groupes compactés)

```
/* SPDX-License-Identifier: GPL-2.0-or-later */
/*
 * dsp_banc_commun.c - the Calypso DSP driven as by the osmocom-bb ARM.
 * See dsp_banc_commun.h. The sequences are copied from the firmware
 * (osmocom-bb src/target/firmware: calypso/dsp.c, layer1/sync.c,
 * layer1/tdma_sched.c, layer1/mframe_sched.c) and from the bench (src/pont.c:
 * jouer_trame, servir; qosmo calypso_bsp.c: bsp_ts0_livrer, sb_moduler).
 * Every deviation is stated where it is made.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include "qemu/thread.h"
#include "dsp_banc_commun.h"
#include "calypso_dma.h"
#include "calypso_bsp.h"
#include "calypso_rhea_dma.h"
#include "calypso_rif.h"
#include "calypso_twl3025.h"
#include "calypso_gmsk.h"
#include <osmocom/core/bits.h>
#include <osmocom/coding/gsm0503_coding.h>

/* ---- what main.c of c54x_exe normally provides ---------------------------- */
uint32_t g_c54x_exe_fn;
uint32_t calypso_trx_get_fn(void) { return g_c54x_exe_fn; }
```

```
void calypso_inth_arm_ack(void) { }
QemuMutex calypso_pcb_daram_lock;
void cpu_physical_memory_rw(uint64_t addr, void *buf, uint64_t len, bool wr)
{ (void)addr; if (!wr) memset(buf, 0, len); }
extern int c54x_rapide;

/* ---- state ------------------------------------------------------------- */
struct banc_dsp_api dsp_api;
C54xState *banc_dsp;
uint16_t *banc_api;
struct banc_temps banc_courant, banc_suivant;
struct banc_config banc_cfg = { .rom_dir = "/opt/GSM", .budget = 300000, .verbeux = 0, .afc_dac = -700 };
struct banc_stats banc_stats;
banc_source_fn banc_source = banc_cellule;
const uint8_t *(*banc_source_autre)(uint32_t fn, unsigned tn);
uint8_t banc_bsic = 7;
banc_cellule_nb_fn banc_cellule_nb;
int (*banc_crochet_l1s)(uint32_t fn);
void (*banc_crochet_fin)(uint32_t fn);
bool banc_sur_tch;
struct banc_ul_burst banc_ul[BANC_UL_MAX];
struct banc_tsp_burst banc_tsp[BANC_UL_MAX];
int banc_n_tsp;
static uint16_t g_tsp_prec[16];
int banc_n_ul;

static int g_irq_armee;            /* dsp_end_scenario() at the last l1_sync */
static uint16_t g_ul_prec;
static uint16_t g_ul_prec_ptr[6];   /* last pointers of the TX rings: 0x3d91, 0x3d93, 0x3d95, 0x3d97, 0x3d99,
 ↪  0x3d9b */

/* API RAM, in words from data[0x0800] (dsp_api.h: BASE_API_*) */
#define W_PAGE_MOT(p)   ((p) ? 0x14u : 0x00u)
#define R_PAGE_MOT(p)   ((p) ? 0x3Cu : 0x28u)
#define NDB_MOT         0xD4u
#define PARAM_MOT       0x431u
#define BL_ADDR_HI_W    0x7FCu     /* BL_ADDR_HI = BASE_API_RAM + 0x0ff8 */
#define BL_SIZE_W       0x7FDu
#define BL_ADDR_LO_W    0x7FEu
#define BL_STATUS_W     0x7FFu
#define BL_STATUS_IDLE  1
#define BL_CMD_COPY_BLOCK 2
#define DSP_START       0x7000

/* TWL3025 registers (osmocom-bb include/abb/twl3025.h; ABB_VAL only keeps
 * the 5 low bits, the page is dropped). */
#define PAGE(n) ((n) << 7)
enum { R_AFCCTLADD = PAGE(1) | 21, R_APCDEL1 = PAGE(0) | 2, R_APCDEL2 = PAGE(1) | 26,
       R_APCRAM = PAGE(0) | 10, R_APCOFF = PAGE(0) | 11, R_AUXDAC = PAGE(0) | 12,
       R_BULIOFF = PAGE(1) | 2, R_BULQOFF = PAGE(1) | 3, R_BULGCAL = PAGE(1) | 14,
       R_BBCTRL = PAGE(1) | 6, R_VBCTRL1 = PAGE(1) | 8, R_VBCTRL2 = PAGE(1) | 11,
       R_VBUCTRL = PAGE(1) | 7, R_VBDCTRL = PAGE(0) | 6 };
#define ABB_RAMP_VAL(up, down)  ((((down) & 0x1F) << 5) | ((up) & 0x1F))

/* calypso/dsp_params.c of the firmware, included as is: the parameter table
 * of a real phone (ROM 3306/3606) that dsp_set_params() copies into the API. */
#include "dsp_params.c"

void banc_fn2temps(struct banc_temps *t, uint32_t fn)
{
    fn %= 2715648u;
    t->fn = fn;
    t->t1 = (uint16_t)(fn / (26 * 51));
    t->t2 = (uint8_t)(fn % 26);
    t->t3 = (uint8_t)(fn % 51);
}
```

```
long banc_executer(long n)
{
    long k = 0;
    while (k < n && banc_dsp->running && !banc_dsp->idle) {
        long m = n - k;
        int r = c54x_run(banc_dsp, (int)(m > 0x40000000L ? 0x40000000L : m));
        if (r <= 0) break;
        k += r;
    }
    return k;
}

/* A DSP at IDLE is woken up by a pending interrupt (SPRU131); INT10n (end of
 * RHEA DMA, IMR bit 14) is a level line. pont.c. */
static void reveil(void)
{
    C54xState *d = banc_dsp;
    if (d->idle && calypso_rhea_dma_irq_level() && (d->imr & (1u << 14)) && !(d->ifr & (1u << 14)))
        c54x_interrupt_ex(d, 30, 14);
    if (d->idle && (d->ifr & d->imr) && !(d->st1 & 0x800))
        d->idle = false;
}

static void pages(void)
{
    dsp_api.db_w = (T_DB_MCU_TO_DSP *)&banc_api[W_PAGE_MOT(dsp_api.w_page)];
    dsp_api.db_r = (T_DB_DSP_TO_MCU *)&banc_api[R_PAGE_MOT(dsp_api.r_page)];
}

int banc_init(void)
{
    /* Nothing towards the outside: no TRXD socket (an empty replay file
     * bypasses the UDP listener of calypso_bsp_init), no published clock,
     * no recorder, no mailbox monitor. */
    setenv("CALYPSO_BSP_REPLAY_FILE", "/dev/null", 1);
    setenv("CALYPSO_BSP_HORLOGE", "0", 1);
    setenv("CALYPSO_REJEU_ENREG", "0", 1);
    setenv("CALYPSO_MAILBOX", "0", 1);
    /* Pending interrupt taken as soon as INTM drops: C54x behaviour
     * (SPRU131 ch. 6), enabled on both benches by main.c. */
    setenv("CALYPSO_C54X_IRQ_LEVEL", "1", 0);
    /* The A5 coprocessor model realigns the COUNT of SACCH/TF frames when it
     * targets the frame already deposited (calypso_a5.c, CALYPSO_A5_RECALE): a
     * workaround for a bench timing defect. The tester turns it off to measure
     * the ROM as it is (unless the caller has set it). */
    setenv("CALYPSO_A5_RECALE", "0", 0);
    qemu_mutex_init(&calypso_pcb_daram_lock);
    { const char *d = getenv("CALYPSO_DEBUG"), *r = getenv("CALYPSO_C54X_RAPIDE");
      c54x_rapide = (r && *r) ? (*r != '0') : !(d && *d); }
    banc_dsp = c54x_init();
    if (!banc_dsp) { fprintf(stderr, "c54x_init a echoue\n"); return -1; }
    banc_api = &banc_dsp->data[C54X_API_BASE];
    c54x_set_api_ram(banc_dsp, banc_api);       /* BEFORE the ROMs (main.c) */
    static const struct { const char *s; uint32_t a; bool p; } R[] = {
        { "PROM0", 0x07000, true }, { "PROM1", 0x18000, true }, { "PROM2", 0x28000, true },
        { "PROM3", 0x38000, true }, { "DROM", 0x09000, false }, { "PDROM", 0x0E000, false },
        { "PDROM", 0x0E000, true } };
    for (unsigned i = 0; i < sizeof R / sizeof R[0]; i++) {
        char c[512];
        snprintf(c, sizeof c, "%s/calypso_dsp.%s.bin", banc_cfg.rom_dir, R[i].s);
        if (c54x_load_section(banc_dsp, c, R[i].a, R[i].p) < 0) {
            fprintf(stderr, "ROM manquante : %s\n", c);
            return -1;
        }
    }
```

```
    { char c[512]; snprintf(c, sizeof c, "%s/calypso_dsp.Registers.bin", banc_cfg.rom_dir);
      c54x_load_registers(banc_dsp, c); }
    c54x_reset(banc_dsp);
    calypso_dma_init();
    calypso_bsp_init(banc_dsp);
    calypso_twl3025_set_afc_dac(banc_cfg.afc_dac);
    dsp_api.ndb = (T_NDB_MCU_DSP *)&banc_api[NDB_MOT];
    dsp_api.param = (T_PARAM_MCU_DSP *)&banc_api[PARAM_MOT];
    dsp_api.r_page = dsp_api.w_page = dsp_api.r_page_used = 0;
    pages();
    banc_fn2temps(&banc_courant, 0);
    banc_fn2temps(&banc_suivant, 0);
    return 0;
}

/* ---- calypso/dsp.c ------------------------------------------------------- */
static void dsp_audio_init(void)
{
    T_NDB_MCU_DSP *ndb = dsp_api.ndb;
    ndb->d_vbctrl1 = ABB_VAL_T(R_VBCTRL1, 0x00B);
    ndb->d_vbctrl2 = ABB_VAL_T(R_VBCTRL2, 0x000);
    ndb->d_vbuctrl = ABB_VAL_T(R_VBUCTRL, 0x009);
    ndb->d_vbdctrl = ABB_VAL_T(R_VBDCTRL, 0x066);
    ndb->d_toneskb_init = 0;
    ndb->d_toneskb_status = 0;
    ndb->d_shiftul = 0x100;
    ndb->d_shiftdl = 0x100;
    ndb->d_melo_osc_used = 0;
    ndb->d_melo_osc_active = 0;
    ndb->a_melo_note0[0] = ndb->a_melo_note1[0] = ndb->a_melo_note2[0] = ndb->a_melo_note3[0] = 0xfffe;
    ndb->a_melo_note4[0] = ndb->a_melo_note5[0] = ndb->a_melo_note6[0] = ndb->a_melo_note7[0] = 0xfffe;
    dsp_api.param->a_fir31_downlink[0] = 0x4000;
    dsp_api.param->a_fir31_uplink[0] = 0x4000;
    for (int i = 1; i < 31; i++) {
        dsp_api.param->a_fir31_downlink[i] = 0;
        dsp_api.param->a_fir31_uplink[i] = 0;
    }
#define B_GSM_ONLY      ((1L << 13) | (1L << 11))
#define B_BT_CORDLESS   (1L << 12)
#define B_BT_HEADSET    (1L << 14)
#define B_FIR_LOOP      (1L << 1)
    ndb->d_audio_init &= ~(B_FIR_LOOP | B_GSM_ONLY | B_BT_HEADSET | B_BT_CORDLESS);
    ndb->d_audio_init |= B_GSM_ONLY;
    ndb->d_aec_ctrl = 0;
    dsp_api.param->d_gsm_bgd_mgt = 0;
    ndb->d_audio_compressor_ctrl = 0x0401;
    ndb->d_melody_selection = 0;
}

static void dsp_ndb_init(void)
{
    T_NDB_MCU_DSP *ndb = dsp_api.ndb;
#define APCDEL_DOWN (2 + 0)
#define APCDEL_UP   (6 + 3 + 1)
    for (int i = 0; i < 16; i++)
        ndb->a_ramp[i] = ABB_VAL(R_APCRAM, ABB_RAMP_VAL(0, 0));
    ndb->d_debug1 = ABB_VAL_T(0, 0x000);
    ndb->d_afcctladd = ABB_VAL_T(R_AFCCTLADD, 0x000);
    ndb->d_vbuctrl = ABB_VAL_T(R_VBUCTRL, 0x0C9);
    ndb->d_vbdctrl = ABB_VAL_T(R_VBDCTRL, 0x006);
    ndb->d_bbctrl = ABB_VAL_T(R_BBCTRL, 0x2C1);
    ndb->d_bulgcal = ABB_VAL_T(R_BULGCAL, 0x000);
    ndb->d_apcoff = ABB_VAL_T(R_APCOFF, 0x040);
    ndb->d_bulioff = ABB_VAL_T(R_BULIOFF, 0x0FF);
    ndb->d_bulqoff = ABB_VAL_T(R_BULQOFF, 0x0FF);
    ndb->d_dai_onoff = ABB_VAL_T(R_APCOFF, 0x000);
```

```
    ndb->d_auxdac = ABB_VAL_T(R_AUXDAC, 0x000);
    ndb->d_vbctrl1 = ABB_VAL_T(R_VBCTRL1, 0x00B);
    ndb->d_vbctrl2 = ABB_VAL_T(R_VBCTRL2, 0x000);
    ndb->d_apcdel1 = ABB_VAL_T(R_APCDEL1, ((APCDEL_DOWN - 2) << 5) | (APCDEL_UP - 6));
    ndb->d_apcdel2 = ABB_VAL_T(R_APCDEL2, 0x000);
    ndb->d_fb_mode = 1;
    ndb->d_fb_det = 0;
    ndb->a_cd[0] = (1 << B_FIRE1);
    ndb->a_dd_0[0] = 0;
    ndb->a_dd_0[2] = 0xffff;
    ndb->a_dd_1[0] = 0;
    ndb->a_dd_1[2] = 0xffff;
    ndb->a_du_0[0] = 0;
    ndb->a_du_0[2] = 0xffff;
    ndb->a_du_1[0] = 0;
    ndb->a_du_1[2] = 0xffff;
    ndb->a_fd[0] = (1 << B_FIRE1);
    ndb->a_fd[2] = 0xffff;
    ndb->d_a5mode = 0;
    ndb->d_tch_mode = 0x0800;
#define GUARD_BITS 8
    ndb->d_tch_mode |= (((GUARD_BITS - 4) & 0x000F) << 7);
    ndb->a_sch26[0] = (1 << B_SCH_CRC);
    ndb->d_spcx_rif = 0x179;
    dsp_audio_init();
}

int banc_dsp_power_on(struct banc_boot *b)
{
    struct banc_boot vide;
    if (!b) b = &vide;
    memset(b, 0, sizeof *b);
    memset(banc_api, 0, 0x2000u * sizeof(uint16_t));        /* dsp_api_memset(API) */

    /* dsp_pre_boot(): the DSP leaves reset and waits in its loader
     * (BL_STATUS_IDLE). Deviation: the firmware first uploads dsp_bootcode
     * there through the 0xe000 mirror; here the ROM's loader runs. */
    long n = 0;
    while (banc_api[BL_STATUS_W] != BL_STATUS_IDLE && n < 8000000) {
        int r = c54x_run(banc_dsp, 256); if (r <= 0) break; n += r;
    }
    b->insn_chargeur = n;
    b->bl_status = banc_api[BL_STATUS_W];

    /* dsp_set_params() */
    T_NDB_MCU_DSP *ndb = dsp_api.ndb;
    ndb->d_background_enable = 0;
    ndb->d_background_abort = 0;
    ndb->d_background_state = 0;
    ndb->d_debug_ptr = 0x0074;
    ndb->d_debug_bk = 0x0001;
    ndb->d_pll_config = 0x154;
    ndb->p_debug_buffer = 0x17ff;
    ndb->d_debug_buffer_size = 7;
    ndb->d_debug_trace_type = 0;
    ndb->d_dsp_state = 3;
    ndb->d_audio_gain_ul = 0;
    ndb->d_audio_gain_dl = 0;
    ndb->d_es_level_api = 0x5213;
    ndb->d_mu_api = 0x5000;
    memcpy(dsp_api.param, &dsp_params, sizeof dsp_params);
    /* dsp_bl_start_at(DSP_START) */
    banc_api[BL_ADDR_HI_W] = 0;
    banc_api[BL_ADDR_LO_W] = DSP_START;
    banc_api[BL_SIZE_W] = 0;
    banc_api[BL_STATUS_W] = BL_CMD_COPY_BLOCK;
```

```
    n = 0;
    while (n < 4000000 && banc_dsp->running) {
        int r = c54x_run(banc_dsp, 256); if (r <= 0) break; n += r;
        if (banc_dsp->idle) break;
    }
    b->insn_demarrage = n;
    b->idle = banc_dsp->idle;
    b->version1 = ndb->d_version_number1;
    b->version2 = ndb->d_version_number2;

    dsp_ndb_init();
    /* dsp_db_init() */
    memset(&banc_api[W_PAGE_MOT(0)], 0, sizeof(T_DB_MCU_TO_DSP));
    memset(&banc_api[W_PAGE_MOT(1)], 0, sizeof(T_DB_MCU_TO_DSP));
    memset(&banc_api[R_PAGE_MOT(0)], 0, sizeof(T_DB_DSP_TO_MCU));
    memset(&banc_api[R_PAGE_MOT(1)], 0, sizeof(T_DB_DSP_TO_MCU));
    dsp_api.frame_ctr = 0;
    dsp_api.r_page = dsp_api.w_page = dsp_api.r_page_used = 0;
    /* layer1 init: l1s_reset_hw() */
    l1s_reset_hw();
    g_ul_prec = banc_dsp->data[0x3d9b];
    { static const uint16_t ptrs[6] = { 0x3d91, 0x3d93, 0x3d95, 0x3d97, 0x3d99, 0x3d9b };
      for (int i = 0; i < 6; i++) g_ul_prec_ptr[i] = banc_dsp->data[ptrs[i]]; }
    return (b->bl_status == BL_STATUS_IDLE && b->idle) ? 0 : -1;
}

void l1s_reset_hw(void)
{
    dsp_api.w_page = 0;
    dsp_api.r_page = 0;
    dsp_api.r_page_used = 0;
    pages();
    dsp_api.ndb->d_dsp_page = 0;
}

void dsp_end_scenario(void)
{
    dsp_api.ndb->d_dsp_page = B_GSM_TASK | dsp_api.w_page;
    dsp_api.w_page ^= 1;
    g_irq_armee = 1;            /* tpu_dsp_frameirq_enable(): frame IT at the next frame */
}

void dsp_load_rx_task(uint16_t task, uint8_t burst_id, uint8_t tsc)
{
    dsp_api.db_w->d_task_d = task;
    dsp_api.db_w->d_burst_d = burst_id;
    dsp_api.db_w->d_ctrl_system |= tsc & 0x7;
}

void dsp_load_tx_task(uint16_t task, uint8_t burst_id, uint8_t tsc)
{
    dsp_api.db_w->d_task_u = task;
    dsp_api.db_w->d_burst_u = burst_id;
    dsp_api.db_w->d_ctrl_system |= tsc & 0x7;
}

void dsp_load_tch_param(const struct banc_temps *next_time, uint8_t chan_mode, uint8_t chan_type,
                        uint8_t chan_sub, uint8_t tch_loop, uint8_t sync_tch, uint8_t tn)
{
    uint16_t d_ctrl_tch, fn;
    d_ctrl_tch = (chan_mode << B_CHAN_MODE) | (chan_type << B_CHAN_TYPE) | (chan_sub << B_SUBCHANNEL) |
                 (sync_tch << B_SYNC_TCH_UL) | (sync_tch << B_SYNC_TCH_DL) | (tch_loop << B_TCH_LOOP);
    if (chan_type == TCH_F)
        fn = (uint16_t)(((next_time->fn - (tn * 13) + 104) % 104) | ((next_time->fn % 104) << 8));
    else if (chan_type == TCH_H) {
        uint8_t tn_report = (tn & ~1) | chan_sub;
```

```
        fn = (uint16_t)(((next_time->fn - (tn_report * 13) + 104) % 104) | ((next_time->fn % 104) << 8));
    } else
        fn = 0;
    dsp_api.db_w->d_fn = fn;
    dsp_api.db_w->a_a5fn[0] = (uint16_t)(((uint16_t)next_time->t3 << 5) | next_time->t2);
    dsp_api.db_w->a_a5fn[1] = next_time->t1;
    dsp_api.db_w->d_ctrl_tch = d_ctrl_tch;
}

void dsp_load_ciph_param(int mode, const uint8_t *key)
{
    dsp_api.ndb->d_a5mode = mode;
    if (!mode || !key)
        return;
    dsp_api.ndb->a_kc[⟨1⟩] = (uint16_t)key[⟨2⟩] | ((uint16_t)key[⟨3⟩] << 8);  ×4
    ⟨⟩ = (0,7,6) (1,5,4) (2,3,2) (3,1,0)
}

void dsp_memcpy_to_api(volatile uint16_t *dsp_buf, const uint8_t *mcu_buf, int n, int be)
{
    int odd = n & 1;
    n >>= 1;
    for (int i = 0; i < n; i++) {
        uint16_t w;
        if (be) { w = (uint16_t)(*(mcu_buf++) << 8); w |= *(mcu_buf++); }
        else    { w = *(mcu_buf++); w |= (uint16_t)(*(mcu_buf++) << 8); }
        *(dsp_buf++) = w;
    }
    if (odd)
        *dsp_buf = be ? (uint16_t)(*mcu_buf << 8) : *mcu_buf;
}

void dsp_memcpy_from_api(uint8_t *mcu_buf, const volatile uint16_t *dsp_buf, int n, int be)
{
    int odd = n & 1;
    n >>= 1;
    for (int i = 0; i < n; i++) {
        uint16_t w = *(dsp_buf++);
        if (be) { *(mcu_buf++) = w >> 8; *(mcu_buf++) = (uint8_t)w; }
        else    { *(mcu_buf++) = (uint8_t)w; *(mcu_buf++) = w >> 8; }
    }
    if (odd)
        *mcu_buf = be ? (uint8_t)(*dsp_buf >> 8) : (uint8_t)*dsp_buf;
}

/* ---- layer1/tdma_sched.c ------------------------------------------------- */
#define TDMASCHED_NUM_FRAMES 25
#define TDMASCHED_NUM_CB     8
static struct { struct tdma_sched_item item[TDMASCHED_NUM_CB]; uint8_t num_items; }
 ↪  g_bucket[TDMASCHED_NUM_FRAMES];
static uint8_t g_cur_bucket;

int tdma_end_set(uint8_t p1, uint8_t p2, uint16_t p3) { (void)p1; (void)p2; (void)p3; return 0; }
static uint8_t wrap_bucket(uint8_t off) { return (uint8_t)((g_cur_bucket + off) % TDMASCHED_NUM_FRAMES); }

int tdma_schedule(uint8_t frame_offset, tdma_sched_cb *cb, uint8_t p1, uint8_t p2, uint16_t p3, int16_t prio)
{
    uint8_t nr = wrap_bucket(frame_offset);
    if (g_bucket[nr].num_items >= TDMASCHED_NUM_CB) { printf("tdma_schedule bucket overflow\n"); return -1; }
    struct tdma_sched_item *it = &g_bucket[nr].item[g_bucket[nr].num_items++];
    it->cb = cb; it->p1 = p1; it->p2 = p2; it->p3 = p3; it->prio = prio; it->flags = 0;
    return 0;
}

int tdma_schedule_set(uint8_t frame_offset, const struct tdma_sched_item *set, uint16_t p3)
{
```

```
    uint8_t nr = wrap_bucket(frame_offset);
    int j = 0;
    for (int i = 0; ; i++) {
        const struct tdma_sched_item *si = &set[i];
        if (si->cb == &tdma_end_set) break;
        if (si->cb == NULL) { nr = wrap_bucket(++frame_offset); j++; continue; }
        if (g_bucket[nr].num_items >= TDMASCHED_NUM_CB) { printf("tdma_schedule bucket overflow\n"); return -1; }
        g_bucket[nr].item[g_bucket[nr].num_items] = *si;
        g_bucket[nr].item[g_bucket[nr].num_items].p3 = p3;
        g_bucket[nr].num_items++;
    }
    return j;
}

static uint16_t tdma_sched_flag_scan(void)
{
    uint16_t f = 0;
    for (int i = 0; i < g_bucket[g_cur_bucket].num_items; i++) f |= g_bucket[g_cur_bucket].item[i].flags;
    return f;
}

static void tdma_sched_execute(void)
{
    int seq[TDMASCHED_NUM_CB];
    __typeof__(g_bucket[0]) *b = &g_bucket[g_cur_bucket];
    for (int i = 0; i < TDMASCHED_NUM_CB; i++) seq[i] = i;
    for (int i = 0; i < b->num_items; i++) {          /* _tdma_sched_bucket_sort, identical */
        struct tdma_sched_item *ii = &b->item[seq[i]];
        for (int j = i + 1; j < b->num_items; j++) {
            struct tdma_sched_item *jj = &b->item[seq[j]];
            if (ii->prio > jj->prio) { ii = jj; int k = seq[i]; seq[i] = seq[j]; seq[j] = k; }
        }
    }
    for (int i = 0; i < b->num_items; i++) {
        struct tdma_sched_item *it = &b->item[seq[i]];
        if (it->cb(it->p1, it->p2, it->p3) < 0) break;
    }
    b->num_items = 0;
}

void tdma_sched_reset(void)
{
    for (int i = 0; i < TDMASCHED_NUM_FRAMES; i++)
        if (i != g_cur_bucket) g_bucket[i].num_items = 0;
}

/* ---- layer1/mframe_sched.c (SCHEDULE_AHEAD 2, SCHEDULE_LATENCY 1) -------------- */
static struct banc_mframe_item g_mf[32];
static int g_n_mf;
void banc_mframe_ajouter(const struct tdma_sched_item *set, uint16_t modulo, uint16_t frame_nr, uint16_t p3)
{
    if (g_n_mf < 32) g_mf[g_n_mf++] = (struct banc_mframe_item){ set, modulo, frame_nr, p3 };
}
void banc_mframe_vider(void) { g_n_mf = 0; }
static void mframe_schedule(void)
{
    for (int i = 0; i < g_n_mf; i++) {
        unsigned trigger = g_mf[i].frame_nr % g_mf[i].modulo;
        unsigned current = (banc_courant.fn + 2) % g_mf[i].modulo;
        if (current == trigger) tdma_schedule_set(1, g_mf[i].set, g_mf[i].p3);
    }
}

/* ---- layer1/sync.c l1_sync() ------------------------------------------------ */
static void l1_sync(void)
{
```

```
    banc_courant = banc_suivant;
    banc_fn2temps(&banc_suivant, banc_courant.fn + 1);
    dsp_api.frame_ctr++;
    dsp_api.r_page_used = 0;
    pages();
    memset((void *)dsp_api.db_w, 0, sizeof(*dsp_api.db_w));
    /* afc_load_dsp() */
    dsp_api.db_w->d_afc = (uint16_t)banc_cfg.afc_dac;
    dsp_api.db_w->d_ctrl_abb |= (1 << B_AFC);
    if (dsp_api.ndb->d_error_status) {
        banc_stats.erreurs_dsp++;
        banc_stats.derniere_erreur = dsp_api.ndb->d_error_status;
        if (banc_cfg.verbeux) printf("    [l1s] fn=%u DSP Error Status: %u\n", banc_courant.fn,
         ↪  dsp_api.ndb->d_error_status);
        dsp_api.ndb->d_error_status = 0;
    }
    uint16_t flags = tdma_sched_flag_scan();
    tdma_sched_execute();
    if (banc_crochet_l1s) flags |= (uint16_t)banc_crochet_l1s(banc_courant.fn);
    if (dsp_api.r_page_used) {
        memset((void *)dsp_api.db_r, 0, sizeof(*dsp_api.db_r));
        dsp_api.db_r->a_sch[0] = (1 << B_SCH_CRC);
        dsp_api.r_page ^= 1;
    }
    if (flags & TDMA_IFLG_DSP)
        dsp_end_scenario();
    mframe_schedule();
    g_cur_bucket = wrap_bucket(1);       /* tdma_sched_advance() */
}

/* ---- the radio channel ------------------------------------------------------ */
const uint8_t banc_tsc[8][26] = {      /* 45.002 table 5.2.3a */
    { ⟨1⟩,⟨2⟩,⟨3⟩,⟨4⟩,⟨5⟩,⟨6⟩,⟨7⟩,⟨8⟩,⟨9⟩,⟨10⟩,⟨11⟩,⟨12⟩,⟨13⟩,⟨14⟩,⟨15⟩,0,⟨16⟩,⟨17⟩,⟨18⟩,⟨19⟩,⟨20⟩,⟨21⟩,⟨22⟩,⟨23⟩,⟨24⟩,⟨25⟩ },  ×8
    ⟨⟩ = (0,0,1,0,0,1,0,1,1,1,0,0,0,0,1,0,0,1,0,0,1,0,1,1,1)
        (0,0,1,0,1,1,0,1,1,1,0,1,1,1,0,0,1,0,1,1,0,1,1,1)
        (0,1,0,0,0,0,1,1,1,0,1,1,1,0,1,0,1,0,0,0,0,1,1,1,0)
        (0,1,0,0,0,1,1,1,1,0,1,1,0,1,0,0,1,0,0,0,1,1,1,1,0)
        (0,0,0,1,1,0,1,0,1,1,1,0,0,1,0,0,0,0,1,1,0,1,0,1,1)
        (0,1,0,0,1,1,1,0,1,0,1,1,0,0,0,0,1,0,0,1,1,1,0,1,0)
        (1,0,1,0,0,1,1,1,1,1,0,1,1,0,0,1,0,1,0,0,1,1,1,1,1)
        (1,1,1,0,1,1,1,1,0,0,0,1,0,0,1,1,1,1,0,1,1,1,1,0,0)
};
const uint8_t banc_train_sb[64] = {
    1,0,1,1,1,0,0,1,0,1,1,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,0,0,1,1,1,1,
    0,0,1,0,1,1,0,1,0,1,0,0,0,1,0,1,0,1,1,1,0,1,1,0,0,0,0,1,1,0,1,1,
};
const uint8_t banc_factice[148] = {    /* 45.002 5.2.6 */
    0,0,0,
    ⟨1⟩,⟨2⟩,⟨3⟩,⟨4⟩,⟨5⟩,⟨6⟩,⟨7⟩,⟨8⟩,0,⟨9⟩,⟨10⟩,1,⟨11⟩,⟨12⟩,⟨13⟩,⟨14⟩,⟨15⟩,⟨16⟩,⟨17⟩,⟨18⟩,⟨19⟩,⟨20⟩,⟨21⟩,⟨22⟩,⟨23⟩,⟨24⟩,⟨25⟩,⟨26⟩,⟨
 ↪  27⟩,⟨28⟩,⟨29⟩,⟨30⟩,
 ↪  ×4
    ⟨⟩ = (1,1,1,1,1,0,1,1,1,1,0,1,1,0,0,0,0,0,1,0,1,0,0,1,0,0,1,1,1,1,0)
        (0,0,0,0,1,0,0,1,0,0,0,0,0,0,0,0,0,1,1,1,1,1,1,0,0,0,1,1,1,1,0,0)
        (0,1,0,1,1,1,0,0,1,0,1,1,0,0,0,1,0,1,0,1,1,1,1,0,1,0,0,1,0,1,0,1,0)
        (0,0,1,1,0,0,1,1,0,1,1,0,0,1,1,1,1,1,0,1,0,0,1,1,1,1,1,0,0,0,1)
    0,0,1,0,1,1,1,1,1,0,1,0,1,0,
    0,0,0,
};

void banc_burst_nb(const uint8_t *e116, uint8_t tsc, uint8_t bits[148])
{
    memset(bits, 0, 3);
    memcpy(bits + 3, e116, 58);
    memcpy(bits + 61, banc_tsc[tsc & 7], 26);
    memcpy(bits + 87, e116 + 58, 58);
    memset(bits + 145, 0, 3);
```

```
}

void banc_sb_info(uint32_t fn, uint8_t bsic, uint8_t sb_info[4])
{
    uint32_t t1 = fn / 1326, t2 = fn % 26, t3 = fn % 51;
    uint32_t t3p = t3 ? (t3 - 1) / 10 : 0;
    sb_info[0] = (uint8_t)(((bsic & 0x3f) << 2) | ((t1 & 0x600) >> 9));
    sb_info[1] = (uint8_t)((t1 & 0x1fe) >> 1);
    sb_info[2] = (uint8_t)(((t1 & 0x001) << 7) | ((t2 & 0x1f) << 2) | ((t3p & 0x6) >> 1));
    sb_info[3] = (uint8_t)(t3p & 0x1);
}

void banc_burst_sch(uint32_t fn, uint8_t bsic, uint8_t bits[148])
{
    uint8_t sb_info[4];
    ubit_t code[78];
    banc_sb_info(fn, bsic, sb_info);
    gsm0503_sch_encode(code, sb_info);
    memset(bits, 0, 3);
    memcpy(bits + 3, code, 39);
    memcpy(bits + 42, banc_train_sb, 64);
    memcpy(bits + 106, code + 39, 39);
    memset(bits + 145, 0, 3);
}

char banc_cellule(uint32_t fn, uint8_t bits[148], int *amp)
{
    (void)amp;
    uint32_t p51 = fn % 51;
    if (p51 % 10 == 0 && p51 <= 40) { memset(bits, 0, 148); return 'F'; }
    if (p51 % 10 == 1 && p51 <= 41) { banc_burst_sch(fn, banc_bsic, bits); return 'S'; }
    static const int debuts[] = { 2, 6, 12, 16, 22, 26, 32, 36, 42, 46 };
    for (unsigned i = 0; i < sizeof debuts / sizeof debuts[0]; i++) {
        int d = debuts[i];
        if ((int)p51 < d || (int)p51 >= d + 4) continue;
        uint32_t fn0 = fn - (p51 - (uint32_t)d);
        const uint8_t *l2 = banc_cellule_nb ? banc_cellule_nb(fn0, d) : NULL;
        if (!l2) break;
        static ubit_t e[4 * 116];
        gsm0503_xcch_encode(e, l2);
        banc_burst_nb(e + 116 * (p51 - (uint32_t)d), banc_bsic & 7, bits);
        return 'N';
    }
    memcpy(bits, banc_factice, 148);
    return 'D';
}

/* SCH: the BSP waveform (calypso_bsp.c sb_moduler, default settings:
 * spread 1.0, instant 0.35, Gaussian noise sigma 3000 seeded by the fn,
 * shape A). Copied because it is static in calypso_bsp.c. */
static int g_sb_propre = -1;
static void sb_moduler(const uint8_t *bits, int16_t *dst, uint32_t fn, int amp)
{
    if (g_sb_propre < 0) { const char *e = getenv("BANC_SB_PROPRE"); g_sb_propre = (e && *e == '1'); }
    if (g_sb_propre) {                                  /* bare GMSK, like a normal burst */
        gmsk_moduler(bits, 148, amp, 0.0, 0.5, dst);
        gmsk_elargir(dst, 148, 0.3);
        return;
    }
    { static double dsb = -9; if (dsb < -1) { const char *e = getenv("BANC_SB_DEC"); dsb = e ? atof(e) : 0.35; }
      gmsk_moduler(bits, 148, amp, 0.0, dsb, dst); }
    gmsk_elargir(dst, 148, 1.0);
    uint32_t seed = fn * 2654435761u + 777u;
    for (int k = 0; k < 296; k++) {
        seed = seed * 1103515245u + 12345u; double u1 = ((seed >> 8) & 0xffff) / 65536.0 + 1e-6;
        seed = seed * 1103515245u + 12345u; double u2 = ((seed >> 8) & 0xffff) / 65536.0;
```

```
        double g = sqrt(-2.0 * log(u1)) * cos(2.0 * M_PI * u2);
        double v = dst[k] + 3000.0 * g;
        if (v > 32767) v = 32767;
        if (v < -32768) v = -32768;
        dst[k] = (int16_t)lrint(v);
    }
}

static struct { uint32_t fn, tick; uint8_t bits[148]; int amp; bool valide, livre_sb; } g_sch;

/* bsp_ts0_livrer(): framing of the burst according to the window the ROM armed. */
static void deposer(uint32_t fn)
{
    static int16_t iq[2 * 512], iq2[2 * 148];
    uint8_t bits[148];
    int amp = 0;
    char type = banc_source ? banc_source(fn, bits, &amp) : 0;
    if (amp <= 0) amp = 30000;
    const bool one_shot = calypso_rhea_dma_one_shot();
    const int nwin = one_shot ? calypso_rhea_dma_get_len_words() / 2 : 0;
    int marge = nwin >= 190 ? 21 : nwin >= 150 ? 3 : 0;
    /* BANC_FB_DECAL=k: in FB search (continuous DMA), the FCCH is deposited k
     * samples after the window opens (TOA measurement = f(k)). */
    static int fb_decal = -1;
    if (fb_decal < 0) { const char *e = getenv("BANC_FB_DECAL"); fb_decal = e ? atoi(e) : 0; }
    if (!one_shot && type == 'F') marge = fb_decal;
    /* BANC_SB_DECAL=k: in the SB window (one-shot), the SCH is deposited k samples
     * after the window opens (default 21). */
    static int sb_decal = -2;
    if (sb_decal == -2) { const char *e = getenv("BANC_SB_DECAL"); sb_decal = e ? atoi(e) : -1; }
    if (type == 'S' && one_shot && nwin >= 190 && sb_decal >= 0) marge = sb_decal;
    static int sb_neg = 0; { const char *e = getenv("BANC_SB_NEG"); sb_neg = e ? atoi(e) : 0; }
    if (type == 'S' && one_shot && nwin >= 190 && sb_neg > 0) marge = 0;
    /* BANC_NB_DECAL=k: normal burst (one-shot window of 150..189 samples), default 3. */
    static int nb_decal = -2;
    if (nb_decal == -2) { const char *e = getenv("BANC_NB_DECAL"); nb_decal = e ? atoi(e) : -1; }
    if (type != 'S' && type != 'F' && type && one_shot && nwin >= 150 && nwin < 190 && nb_decal >= 0) marge =
    ↪  nb_decal;
    memset(iq, 0, sizeof iq);
    if (type == 'S') {
        g_sch.fn = fn; g_sch.tick = fn; g_sch.amp = amp;
        memcpy(g_sch.bits, bits, 148);
        g_sch.livre_sb = one_shot && nwin >= 190;
        g_sch.valide = true;
        if (g_sch.livre_sb) banc_stats.depots_fenetre_sb++;
        { static int cel = -1; if (cel < 0) cel = getenv("BANC_SB_CELLULE") ? 1 : 0;
          extern char cellule_burst(uint32_t, uint8_t, int, double, int, int16_t *, int *) __attribute__((weak));
          if (cel && cellule_burst) {   /* same waveform as the bridge: src/cellule.c cellule_burst() */
              static int16_t tmp[2 * 512]; int n_iq = 0;
              const char *ed = getenv("BANC_SB_DEC");
              cellule_burst(fn, banc_bsic, amp, ed ? atof(ed) : 0.5, marge, tmp, &n_iq);
              memcpy(iq, tmp, (size_t)(n_iq < 2 * 256 ? n_iq : 2 * 256) * sizeof(int16_t));
          } else {
              sb_moduler(bits, iq + 2 * marge, fn, amp);
             if (sb_neg > 0) { memmove(iq, iq + 2 * sb_neg, (size_t)(2 * (512 - sb_neg)) * sizeof(int16_t)); marge
              ↪  = 1; }
          } }
    } else if (type) {
        /* BANC_NB_NEG=n: normal burst n samples early (start cut off) */
       static int nb_neg = -1; if (nb_neg < 0) { const char *e = getenv("BANC_NB_NEG"); nb_neg = e ? atoi(e) : 0; }
        const bool nb_win = type != 'F' && one_shot && nwin >= 150 && nwin < 190;
        if (nb_neg > 0 && nb_win) marge = 0;
        gmsk_moduler(bits, 148, amp, 0.0, 0.5, iq + 2 * marge);
        if (type != 'F') gmsk_elargir(iq + 2 * marge, 148, 0.3);
        if (nb_neg > 0 && nb_win) { memmove(iq, iq + 2 * nb_neg, (size_t)(2 * (512 - nb_neg)) * sizeof(int16_t));
         ↪  marge = 1; }
```

```
    }
    if (type == 'S' && getenv("BANC_NWIN_DBG")) { static int n; if (n++ < 3) { FILE *f =
     ↪  fopen(getenv("BANC_NWIN_DBG"), "a"); if (f) { fprintf(f, "[nwin] SB one_shot=%d nwin=%d marge=%d
     ↪  len_words=%d\n", one_shot, nwin, marge, calypso_rhea_dma_get_len_words()); fclose(f); } } }
    int total = marge > 0 ? (nwin > marge + 148 ? nwin : marge + 148) : 148;
    if (total > 256) total = 256;
    calypso_bsp_rx_burst(0, fn, iq, 2 * total);
    banc_stats.depots++;
    if (!one_shot) {            /* continuous DMA (FB search): the whole frame */
        for (unsigned tn = 1; tn < 8; tn++) {
            const uint8_t *b = banc_source_autre ? banc_source_autre(fn, tn) : NULL;
            if (!b) b = banc_factice;
            memset(iq2, 0, sizeof iq2);
            gmsk_moduler(b, 148, 30000, 0.0, 0.5, iq2);
            gmsk_elargir(iq2, 148, 0.3);
            calypso_bsp_rx_burst((uint8_t)tn, fn, iq2, 2 * 148);
        }
    }
}

/* calypso_bsp_sb_retenter(): an SCH deposited before the SB window is armed
 * is delivered again in the following one-shot window, at most one tick later. */
static void sb_retenter(uint32_t tick)
{
    static int16_t iq[2 * 512];
    if (!g_sch.valide || g_sch.livre_sb) return;
    if ((int32_t)(tick - g_sch.tick) > 1) { g_sch.valide = false; return; }
    if (!calypso_rhea_dma_rx_armed() || !calypso_rhea_dma_one_shot()) return;
    int total = calypso_rhea_dma_get_len_words() / 2;
    if (total < 190) return;
    if (total > 512) total = 512;
    int marge = 21;
    { const char *e = getenv("BANC_SB_DECAL"); if (e && atoi(e) >= 0) marge = atoi(e); }
    if (marge > total - 148) marge = total - 148;
    memset(iq, 0, sizeof iq);
    sb_moduler(g_sch.bits, iq + 2 * marge, g_sch.fn, g_sch.amp);
    calypso_rif_flush();
    calypso_bsp_rx_burst(0, g_sch.fn, iq, 2 * total);
    g_sch.livre_sb = true;
    banc_stats.depots_fenetre_sb++;
}

/* ---- one frame (src/pont.c: jouer_trame in two phases + servir) ---------------- */
uint32_t banc_fn(void) { return banc_suivant.fn; }
void banc_set_fn(uint32_t fn) { banc_fn2temps(&banc_suivant, fn); banc_fn2temps(&banc_courant, fn ? fn - 1 : 0); }

void banc_trame(void)
{
    C54xState *d = banc_dsp;
    const uint32_t fn = banc_suivant.fn;    /* this frame's l1_sync will have current_time = fn */
    const long budget = banc_cfg.budget;
    uint32_t i0 = d->insn_count;
    long fait = 0;
    g_c54x_exe_fn = fn;
    banc_stats.trames++;

    /* phase A: frame IT (if the ARM closed a scenario), the ROM reads the W
     * page and arms its RX window */
    calypso_dma_tick(d);
    reveil();
    if (g_irq_armee && (d->imr & (1u << C54X_IT_TPU_FRAME_BIT))) {
        c54x_interrupt_ex(d, C54X_IT_TPU_FRAME_VEC, C54X_IT_TPU_FRAME_BIT);
        banc_stats.irq_trame++;
    }
    g_irq_armee = 0;
    if (d->idle && (d->ifr & d->imr) && !(d->st1 & 0x800)) d->idle = false;
```

```
    if (!d->idle) fait = banc_executer(budget / 8);
    while (!d->idle && d->running && (banc_sur_tch || !calypso_rhea_dma_rx_armed()) && fait < budget / 2)
        fait += banc_executer(256);

    /* the ARM: l1_sync() of frame fn; the DSP will read its page at the next frame */
    l1_sync();
    calypso_twl3025_set_afc_dac(banc_cfg.afc_dac);   /* relay of d_afc (pont.c, TICK) */

    /* phase B: the frame's burst, then the rest of the budget */
    deposer(fn);
    reveil();
    if (!d->idle && budget - fait > 0) banc_executer(budget - fait);
    for (int k = 0; k < 40 && d->running; k++) {
        sb_retenter(fn);
        if (!calypso_rhea_dma_pump(d)) break;
        if (d->idle && (d->ifr & d->imr) && !(d->st1 & 0x800)) d->idle = false;
        if (!d->idle) banc_executer(budget / 4);
    }
    if (!d->idle) banc_stats.pas_idle_fin++;
    long n = (long)(d->insn_count - i0);
    if (n > banc_stats.insn_max) banc_stats.insn_max = n;
    banc_ul_capturer(fn);
    if (banc_crochet_fin) banc_crochet_fin(fn);
}

void banc_courir(uint32_t n)
{
    for (uint32_t i = 0; i < n; i++) banc_trame();
}

/* ---- uplink bursts of the ROM (pont.c tx_rom_publier) ---------------------------- */
static void banc_tsp_capturer(uint32_t fn)
{
    const uint16_t *m = banc_dsp->data;
    for (unsigned a = 0x3cbb; a < 0x3d00 - 18; a++) {
        if (m[a] != 0x1c0a) continue;
        const uint16_t *w = &m[a + 2];
        int ok = 1;
        for (int i = 0; i < 16; i++) if ((w[i] & 0x3e) != 0x06 && (w[i] & 0x3e) != 0x04) { ok = 0; break; }
        if (!ok) continue;
        if (!memcmp(w, g_tsp_prec, sizeof g_tsp_prec)) return;
        memcpy(g_tsp_prec, w, sizeof g_tsp_prec);
        struct banc_tsp_burst *t = &banc_tsp[banc_n_tsp % BANC_UL_MAX];
        t->fn = fn; memcpy(t->mots, w, sizeof t->mots);
        banc_n_tsp++;
        return;
    }
}

void banc_ul_capturer(uint32_t fn)
{
    banc_tsp_capturer(fn);
    const uint16_t *m = banc_dsp->data;
    static const uint16_t ptrs[6] = { 0x3d91, 0x3d93, 0x3d95, 0x3d97, 0x3d99, 0x3d9b };
    int bouge = 0;
    for (int i = 0; i < 6; i++) if (m[ptrs[i]] != g_ul_prec_ptr[i]) { g_ul_prec_ptr[i] = m[ptrs[i]]; bouge = 1; }
    if (!bouge) return;
    uint16_t p = m[0x3d9b];
    g_ul_prec = p;
    struct banc_ul_burst *u = &banc_ul[banc_n_ul % BANC_UL_MAX];
    u->fn = fn;
    u->ptr = p;
    u->mode = m[0x3fac];
    for (int i = 0; i < 8; i++) u->raw[i] = m[0x3f8a + i];
    u->rang = (u->mode == 3 && p >= 0x4280 && p < 0x42a0 && (p - 0x4280) % 8 == 0) ? (int)(((p - 0x4280) / 8 + 3) &
 ↪  3) : -1;
```

```
    for (int i = 0; i < 116; i++) {
        /* 57 data, hl, hu, 57 data; hl/hu = the 2 LOW bits of word 7 (k = 126, 127), like
         * pont.c tx_rom_publier. */
        int k = i < 57 ? i : (i < 59 ? 126 + (i - 57) : i - 2);
        u->bits[i] = (m[0x3f8a + k / 16] >> (15 - k % 16)) & 1;
        u->flux[i] = (m[0x3f9b + k / 16] >> (15 - k % 16)) & 1;
    }
    banc_n_ul++;
}

void banc_trame_vide(void)
{
    banc_source_fn s = banc_source;
    banc_source = NULL;
    banc_trame();
    banc_source = s;
}
```

3.32 /opt/GSM/c54x_exe/tools/dsp_banc_commun.h

9726 octets, 193 lignes → 193 lignes

```
/* SPDX-License-Identifier: GPL-2.0-or-later */
/*
 * dsp_banc_commun.h - run the Calypso DSP (TI mask-ROM) the way the osmocom-bb
 * ARM does, without QEMU and without the ARM. Common module, reusable by the
 * offline benches (tools/dsp_tester.c is its first client).
 *
 * What the module provides:
 *   - the C54x core + the ROM + the DMA + the BSP, compiled from qosmo;
 *   - dsp_power_on() of the firmware (calypso/dsp.c): loader, parameters
 *     (dsp_params.c of the firmware, included as is), NDB, pages;
 *   - the firmware primitives: dsp_end_scenario, dsp_load_rx_task,
 *     dsp_load_tx_task, dsp_load_tch_param, dsp_load_ciph_param,
 *     dsp_memcpy_to/from_api, l1s_reset_hw;
 *   - the TDMA scheduler of the firmware (layer1/tdma_sched.c, copied) and a
 *     minimal multiframe scheduler (layer1/mframe_sched.c: SCHEDULE_AHEAD 2,
 *     SCHEDULE_LATENCY 1);
 *   - the bench frame, in the order of src/pont.c (jouer_trame + servir):
 *     frame IT if the ARM closed a scenario, phase A until the RX window is
 *     armed (until the IDLE on TCH), l1_sync() of the ARM, deposit of the
 *     frame's burst (bits -> GMSK, framed like bsp_ts0_livrer), rest of the
 *     budget, DMA pump;
 *   - the capture of the uplink bursts the ROM sends (data[0x3f8a], pointer
 *     data[0x3d9b], see README "ROM and uplink probes").
 *
 * The API RAM structures are those of the firmware (dsp_api.h of osmocom-bb,
 * DSP == 36), laid over data[0x0800..]: bench code is therefore written like
 * firmware code (dsp_api.ndb->a_cd[0], dsp_api.db_w->d_task_d, ...).
 */
#ifndef DSP_BANC_COMMUN_H
#define DSP_BANC_COMMUN_H

#include <stdint.h>
#include <stdbool.h>
#include "calypso_c54x.h"
#include "dsp_api.h"            /* osmocom-bb: include/calypso/dsp_api.h */

/* ---- ARM-side state, like the firmware's struct dsp_api ------------------ */
struct banc_dsp_api {
    T_NDB_MCU_DSP     *ndb;
    T_DB_DSP_TO_MCU   *db_r;
    T_DB_MCU_TO_DSP   *db_w;
    T_PARAM_MCU_DSP   *param;
```

```
    int r_page, w_page, r_page_used;
    uint32_t frame_ctr;
};
extern struct banc_dsp_api dsp_api;
extern C54xState *banc_dsp;
extern uint16_t  *banc_api;            /* = &banc_dsp->data[0x0800] */

/* GSM time (l1s.current_time / l1s.next_time) */
struct banc_temps { uint32_t fn; uint16_t t1; uint8_t t2, t3; };
void banc_fn2temps(struct banc_temps *t, uint32_t fn);
extern struct banc_temps banc_courant, banc_suivant;

/* ---- configuration ------------------------------------------------------ */
struct banc_config {
    const char *rom_dir;    /* calypso_dsp.*.bin (default /opt/GSM) */
    long budget;            /* instructions per frame (default 300000, ~65 MHz) */
    int verbeux;
    int16_t afc_dac;        /* d_afc written by the ARM (-700 = no frequency error) */
};
extern struct banc_config banc_cfg;

/* Boot: what dsp_power_on() observed. */
struct banc_boot {
    long insn_chargeur;     /* instructions up to BL_STATUS_IDLE */
    long insn_demarrage;    /* instructions from 0x7000 up to the IDLE */
    uint16_t bl_status;
    uint16_t version1, version2;
    bool idle;
};

int  banc_init(void);                          /* core, ROM, reset, DMA, BSP */
int  banc_dsp_power_on(struct banc_boot *b);   /* calypso/dsp.c dsp_power_on() */

/* ---- firmware primitives (calypso/dsp.c, layer1/sync.c) ------------------ */
void dsp_end_scenario(void);
void dsp_load_rx_task(uint16_t task, uint8_t burst_id, uint8_t tsc);
void dsp_load_tx_task(uint16_t task, uint8_t burst_id, uint8_t tsc);
void dsp_load_tch_param(const struct banc_temps *next_time, uint8_t chan_mode,
                        uint8_t chan_type, uint8_t chan_sub, uint8_t tch_loop,
                        uint8_t sync_tch, uint8_t tn);
void dsp_load_ciph_param(int mode, const uint8_t *key);
void dsp_memcpy_to_api(volatile uint16_t *dsp_buf, const uint8_t *mcu_buf, int n, int be);
void dsp_memcpy_from_api(uint8_t *mcu_buf, const volatile uint16_t *dsp_buf, int n, int be);
void l1s_reset_hw(void);

/* ---- TDMA scheduler (layer1/tdma_sched.c) -------------------------------- */
#define TDMA_IFLG_TPU   (1 << 0)
#define TDMA_IFLG_DSP   (1 << 1)
typedef int tdma_sched_cb(uint8_t p1, uint8_t p2, uint16_t p3);
struct tdma_sched_item {
    tdma_sched_cb *cb;
    uint8_t p1, p2;
    uint16_t p3;
    int16_t prio;
    uint16_t flags;
};
int tdma_end_set(uint8_t p1, uint8_t p2, uint16_t p3);
#define SCHED_ITEM(x, p, y, z)      { .cb = x, .p1 = y, .p2 = z, .prio = p, .flags = 0 }
#define SCHED_ITEM_DT(x, p, y, z)   { .cb = x, .p1 = y, .p2 = z, .prio = p, \
                                      .flags = TDMA_IFLG_TPU | TDMA_IFLG_DSP }
#define SCHED_END_FRAME()           { .cb = NULL, .p1 = 0, .p2 = 0 }
#define SCHED_END_SET()             { .cb = &tdma_end_set, .p1 = 0, .p2 = 0 }
int  tdma_schedule(uint8_t frame_offset, tdma_sched_cb *cb, uint8_t p1, uint8_t p2,
                   uint16_t p3, int16_t prio);
int  tdma_schedule_set(uint8_t frame_offset, const struct tdma_sched_item *set, uint16_t p3);
void tdma_sched_reset(void);
```

```
/* ---- multiframe (layer1/mframe_sched.c, reduced) ------------------------- */
#define MF_F_SACCH  (1 << 0)
struct banc_mframe_item { const struct tdma_sched_item *set; uint16_t modulo, frame_nr; uint16_t p3; };
void banc_mframe_ajouter(const struct tdma_sched_item *set, uint16_t modulo,
                         uint16_t frame_nr, uint16_t p3);
void banc_mframe_vider(void);

/* ---- the downlink radio channel ------------------------------------------ */
/* The burst the radio receives at frame fn (the followed timeslot: TS0 outside
 * dedicated mode, the dedicated channel's timeslot on its frames, like
 * bsp_ts0_livrer). Returns the type: 'F' FCCH, 'S' SCH, 'N' normal burst,
 * 'D' dummy, 0 = nothing (silence). amp: amplitude (default 30000 if 0). */
typedef char (*banc_source_fn)(uint32_t fn, uint8_t bits[148], int *amp);
extern banc_source_fn banc_source;
/* Bits of the other timeslot tn (1..7) in continuous DMA (FB search); NULL =
 * dummy burst (C0 carrier). */
extern const uint8_t *(*banc_source_autre)(uint32_t fn, unsigned tn);

/* Synthetic TS0 cell, combined CCCH + SDCCH/4 (45.002 table 3):
 * FCCH 0/10/20/30/40, SCH 1/11/21/31/41 (BSIC banc_bsic), the rest: the
 * content returned by banc_cellule_nb (23-byte xCCH block, NULL = dummy). */
extern uint8_t banc_bsic;
typedef const uint8_t *(*banc_cellule_nb_fn)(uint32_t fn0, int bloc_p51);
extern banc_cellule_nb_fn banc_cellule_nb;
char banc_cellule(uint32_t fn, uint8_t bits[148], int *amp);

/* Burst formats (45.002 5.2) */
extern const uint8_t banc_tsc[8][26];
extern const uint8_t banc_train_sb[64];
extern const uint8_t banc_factice[148];
void banc_burst_nb(const uint8_t *e116, uint8_t tsc, uint8_t bits[148]);   /* 57+hl, TSC, hu+57 */
void banc_burst_sch(uint32_t fn, uint8_t bsic, uint8_t bits[148]);
void banc_sb_info(uint32_t fn, uint8_t bsic, uint8_t sb_info[4]);

/* ---- one TDMA frame ------------------------------------------------------ */
/* Hooks: banc_crochet_l1s is called by l1_sync() of every frame, after the
 * scheduler items have run and before the page switch (ARM-side test logic);
 * if it returns TDMA_IFLG_DSP, the DSP scenario is closed as for a
 * SCHED_ITEM_DT item. banc_crochet_fin: at the end of the frame (outputs). */
extern int (*banc_crochet_l1s)(uint32_t fn);
extern void (*banc_crochet_fin)(uint32_t fn);
/* sur_tch: phase A until the IDLE before the deposit (pont.c, PONT_TCH_DEPOT_IDLE) */
extern bool banc_sur_tch;
void banc_trame(void);                /* one frame: phase A, l1_sync, deposit, phase B, pump */
void banc_courir(uint32_t n_trames);
uint32_t banc_fn(void);               /* frame of the next banc_trame() */
void banc_set_fn(uint32_t fn);        /* reset the clock (before the first frame) */

/* Frame statistics */
struct banc_stats {
    unsigned long trames, irq_trame, depots, depots_fenetre_sb, pas_idle_fin;
    unsigned long erreurs_dsp;          /* non-zero d_error_status */
    uint16_t derniere_erreur;
    long insn_max;
};
extern struct banc_stats banc_stats;

/* ---- uplink bursts sent by the ROM --------------------------------------- */
/* On every transmit frame the ROM copies the current burst (116 bits: 57,
 * hl, hu, 57 -> libosmocoding order) into data[0x3f8a..0x3f91] and advances
 * data[0x3d9b] (pont.c tx_rom_publier). flux = uplink A5 stream data[0x3f9b..]. */
/* One burst serialised by the ROM in data[0x3f8a..0x3f91] (serialiser PROM0 0x8900): xCCH ring
 * (pointer 0x3d9b, mode 0x3fac = 3), TCH (0x3d91, mode 0), mode 1 (0x3d93/0x3d95), RACH
 * (0x3d97/0x3d99, mode 2). rang: rank of the burst in the xCCH block (0..3), -1 otherwise;
 * raw: the 8 words as they are. */
```

```
struct banc_ul_burst { uint32_t fn; int rang; int mode; uint16_t ptr; uint16_t raw[8]; uint8_t bits[116]; uint8_t
 ↪  flux[116]; };
#define BANC_UL_MAX 512
extern struct banc_ul_burst banc_ul[BANC_UL_MAX];
/* The FINAL uplink burst as the ROM hands it to the ABB's BSP: 16 BULDATA words (10 data bits
 * in [15:6], register BULDATA1 = 3 in [5:1]) written by the transmitter PROM0 0x8605 into the TSP
 * script (data[0x3cbb..], right after the word TOGBR2 = 0x1c0a and a power word). 160 bits =
 * guard/extended tail + 148-bit burst + guard. Captured on every frame where these 16 words
 * change. */
struct banc_tsp_burst { uint32_t fn; uint16_t mots[16]; };
extern struct banc_tsp_burst banc_tsp[BANC_UL_MAX];
extern int banc_n_tsp;
extern int banc_n_ul;
void banc_ul_capturer(uint32_t fn);    /* called at the end of the frame by banc_trame() */

/* Helpers */
long banc_executer(long n);            /* c54x_run in a loop (yields every 32768) */
void banc_trame_vide(void);            /* one frame without burst or IT (DSP at rest) */

#endif
```

3.33 /opt/GSM/c54x_exe/tools/dsp_tester.c

39501 octets, 815 lignes → 811 lignes (1 groupes compactés)

```
/* SPDX-License-Identifier: GPL-2.0-or-later */
/*
 * dsp_tester.c - function tester for the Calypso DSP (TI 3606 mask-ROM).
 *
 * Without QEMU or ARM: the tester PLAYS the osmocom-bb ARM (tools/dsp_banc_commun.c:
 * dsp_power_on, l1_sync, TDMA scheduler, items of prim_*.c copied) and the
 * BTS (synthetic bursts coded by libosmocoding, GMSK-modulated like the
 * bench BSP). For each DSP function it posts the task codes in the API RAM
 * exactly like the firmware, runs the frames, reads the outputs (R pages,
 * NDB, uplink bursts sent) and compares them with the libosmocoding
 * reference. One PASS / FAIL / NON-IMPL line per function.
 *
 * A test that reveals a defect of the DSP (or of the emulated core) stays
 * FAIL, with the diagnosis; no workaround is applied to make it pass.
 *
 *   ./dsp_tester --list            the tests
 *   ./dsp_tester --all             everything
 *   ./dsp_tester fb sb nb ...      the named tests
 *   options: -v (per-frame detail), --rom-dir DIR, --insns N (budget/frame),
 *            --bsic N
 * Exit code: number of FAILs.
 *
 * Each test runs in a child process, starting from the already booted DSP
 * (fork after dsp_power_on): identical starting state, and a crash of the
 * core only takes its own test down.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <math.h>
#include "dsp_banc_commun.h"
#include "calypso_rhea_dma.h"
#include "verbosite.h"
#include <osmocom/core/bits.h>
#include <osmocom/coding/gsm0503_coding.h>
```

```
#include <osmocom/gsm/a5.h>
#include <osmocom/codec/codec.h>

/* ---- verdicts ---------------------------------------------------------- */
enum { V_PASS = 0, V_FAIL = 1, V_NONIMPL = 2 };
static const char *NOMV[] = { "PASS", "FAIL", "NON-IMPL" };
static char g_detail[8192];
static size_t g_dl;
static int g_v;                     /* -v */
static int g_trace;                 /* --trace N: trace level of the core (src/verbosite.c) */
static void det(const char *fmt, ...) __attribute__((format(printf, 1, 2)));
static void det(const char *fmt, ...)
{
    va_list ap; va_start(ap, fmt);
    int n = vsnprintf(g_detail + g_dl, sizeof g_detail - g_dl, fmt, ap);
    va_end(ap);
    if (n > 0) { g_dl += (size_t)n; if (g_dl >= sizeof g_detail) g_dl = sizeof g_detail - 1; }
}
static void vlog(const char *fmt, ...) __attribute__((format(printf, 1, 2)));
static void vlog(const char *fmt, ...)
{
    if (!g_v) return;
    va_list ap; va_start(ap, fmt);
    printf("      ");
    vprintf(fmt, ap);
    va_end(ap);
}

static struct banc_boot g_boot;
static int g_boot_rc;

/* ---- helpers ----------------------------------------------------------- */
static uint32_t g_alea = 0x1234567u;
static uint8_t alea8(void) { g_alea = g_alea * 1103515245u + 12345u; return (uint8_t)(g_alea >> 16); }
/* 23-byte L2 block determined by a seed: unique content per block, so that
 * a stale a_cd cannot pass for a good decode. */
static void l2_bloc(uint32_t graine, uint8_t l2[23])
{
    uint32_t s = graine * 2654435761u + 0x9e3779b9u;
    for (int i = 0; i < 23; i++) { s = s * 1103515245u + 12345u; l2[i] = (uint8_t)(s >> 16); }
    l2[0] = (uint8_t)(graine >> 8); l2[1] = (uint8_t)graine;     /* readable in the traces */
}
static void hex23(char *o, const uint8_t *b, int n)
{
    for (int i = 0; i < n; i++) sprintf(o + 3 * i, "%02x ", b[i]);
    if (n) o[3 * n - 1] = 0; else o[0] = 0;
}
static uint32_t gsmtime2fn(unsigned t1, unsigned t2, unsigned t3)
{
    return 51u * 26u * t1 + 51u * (((int)t3 - (int)t2 + 26 * 3) % 26) + t3;
}
#define BITFREQ_DIV_PI    86208
#define ANG2FREQ_SCALING  (2 << 15)
#define ANGLE_TO_FREQ(a)  ((int)(int16_t)(a) * BITFREQ_DIV_PI / ANG2FREQ_SCALING)
#define BITS_PER_TDMA     1250
static unsigned popdiff(const uint8_t *a, const uint8_t *b, int n)
{
    unsigned d = 0; for (int i = 0; i < n; i++) d += (a[i] & 1) != (b[i] & 1); return d;
}

/* current radio source + content of the cell's blocks */
static uint8_t g_l2_cell[64][23];
static const uint8_t *cellule_nb_alea(uint32_t fn0, int p51)
{
    (void)p51;
    uint8_t *l2 = g_l2_cell[(fn0 / 51 * 10 + (unsigned)p51) % 64];
```

```
    l2_bloc(fn0, l2);
    return l2;
}

/* ======================================================================== */
/*  boot / idle / checksum                                                  */
/* ======================================================================== */
static int t_boot(void)
{
    det("chargeur : BL_STATUS=%u apres %ld insn ; demarrage 0x7000 -> IDLE en %ld insn ; version API %04x %04x",
        g_boot.bl_status, g_boot.insn_chargeur, g_boot.insn_demarrage, g_boot.version1, g_boot.version2);
    if (g_boot_rc || g_boot.bl_status != 1 || !g_boot.idle) { det("\nle DSP n'a pas atteint l'IDLE"); return
     ↪  V_FAIL; }
    if (g_boot.version1 != 0x3606) { det("\nversion attendue 3606 (calypso_api.h API_VERSION_VALUE)"); return
     ↪  V_FAIL; }
    return V_PASS;
}

static int scenario_vide(uint32_t fn) { (void)fn; return TDMA_IFLG_DSP; }
static int t_idle(void)
{
    /* The ARM closes an empty scenario on every frame (W page zeroed by
     * l1_sync, no task): the ROM gets the frame IT, reads an empty page and
     * must return to IDLE, without error. */
    banc_source = banc_cellule;
    banc_crochet_l1s = scenario_vide;
    uint32_t i0 = banc_dsp->insn_count;
    banc_courir(100);
    long moy = (long)(banc_dsp->insn_count - i0) / 100;
    det("100 trames, %lu IT trame ; %ld insn/trame en moyenne, max %ld ; trames finies hors IDLE : %lu ;
 ↪  d_error_status : %lu",
        banc_stats.irq_trame, moy, banc_stats.insn_max, banc_stats.pas_idle_fin, banc_stats.erreurs_dsp);
    if (banc_stats.irq_trame < 99) { det("\nIT trame non prise (IMR=%04x)", banc_dsp->imr); return V_FAIL; }
    if (banc_stats.pas_idle_fin || banc_stats.erreurs_dsp) return V_FAIL;
    if (moy == 0) { det("\nle DSP n'execute rien sur l'IT trame"); return V_FAIL; }
    return V_PASS;
}

static int g_chk_vu;
static int chk_cmd(uint8_t p1, uint8_t p2, uint16_t p3)
{
    (void)p1; (void)p2; (void)p3;
    dsp_api.db_w->d_task_md = CHECKSUM_DSP_TASK;
    dsp_api.ndb->d_fb_mode = 1;
    return 0;
}
static const struct tdma_sched_item chk_set[] = { SCHED_ITEM_DT(chk_cmd, 0, 0, 0), SCHED_END_FRAME(),
 ↪  SCHED_END_SET() };
static int t_checksum(void)
{
    /* calypso/dsp.c dsp_checksum_task(): d_task_md = CHECKSUM_DSP_TASK (33),
     * then a_pm[0] = code version, a_pm[1] = sum, API 0x08DB = patch. */
    T_DB_DSP_TO_MCU *r0 = (T_DB_DSP_TO_MCU *)&banc_api[0x28], *r1 = (T_DB_DSP_TO_MCU *)&banc_api[0x3C];
    banc_source = banc_cellule;
    tdma_schedule_set(1, chk_set, 0);
    uint16_t av[2][3]; memcpy(av[0], r0->a_pm, 6); memcpy(av[1], r1->a_pm, 6);
    banc_courir(6);
    (void)g_chk_vu;
    det("R0 a_pm=%04x %04x %04x d_task_md=%u | R1 a_pm=%04x %04x %04x d_task_md=%u | patch (API 0x08DB)=%04x",
       r0->a_pm[0], r0->a_pm[1], r0->a_pm[2], r0->d_task_md, r1->a_pm[0], r1->a_pm[1], r1->a_pm[2], r1->d_task_md,
        banc_api[0xDB]);
    int ok = 0;
    for (int p = 0; p < 2; p++) {
        T_DB_DSP_TO_MCU *r = p ? r1 : r0;
        if (r->a_pm[0] == 0x3606 || (r->a_pm[0] && memcmp(av[p], r->a_pm, 6))) ok = 1;
    }
```

```
    if (!ok) { det("\naucune page R ne porte de version/somme : tache non executee"); return V_FAIL; }
    return V_PASS;
}

/* ======================================================================== */
/*  FB / SB (layer1/prim_fbsb.c)                                            */
/* ======================================================================== */
static struct { int det, attempt, mode; uint32_t fn_resp; int16_t toa, pm, angle, snr; int freq; } g_fb;
static uint32_t g_fb_cmd_fn;
static int fbdet_cmd(uint8_t p1, uint8_t p2, uint16_t fb_mode)
{
    (void)p1; (void)p2;
    g_fb_cmd_fn = banc_courant.fn;
    dsp_api.db_w->d_task_md = FB_DSP_TASK;
    dsp_api.ndb->d_fb_mode = fb_mode;
    return 0;
}
static int fbdet_resp(uint8_t p1, uint8_t attempt, uint16_t fb_mode)
{
    (void)p1;
    if (g_fb.det) return 0;
    if (!dsp_api.ndb->d_fb_det) {
        if (attempt == 12) g_fb.det = -1;
        return 0;
    }
    l1s_reset_hw();
    /* read_fb_result() */
    g_fb.det = 1; g_fb.attempt = attempt; g_fb.mode = fb_mode; g_fb.fn_resp = banc_courant.fn;
    g_fb.toa = (int16_t)dsp_api.ndb->a_sync_demod[D_TOA];
    g_fb.pm = (int16_t)(dsp_api.ndb->a_sync_demod[D_PM] >> 3);
    g_fb.angle = (int16_t)dsp_api.ndb->a_sync_demod[D_ANGLE];
    g_fb.snr = (int16_t)dsp_api.ndb->a_sync_demod[D_SNR];
    g_fb.freq = ANGLE_TO_FREQ(g_fb.angle);
    dsp_api.ndb->d_fb_det = 0;
    dsp_api.ndb->a_sync_demod[D_TOA] = 0;
    tdma_sched_reset();
    return 0;
}
static const struct tdma_sched_item fb_sched_set[] = {
    SCHED_ITEM_DT(fbdet_cmd, 0, 0, 0), SCHED_END_FRAME(),
    SCHED_END_FRAME(),
    SCHED_ITEM(fbdet_resp, -4, 0, 〈1〉), SCHED_END_FRAME(), SCHED_ITEM(fbdet_resp, -4, 0, 〈2〉), SCHED_END_FRAME(),
 ↪  ×6
    〈〉 = (1,2) (3,4) (5,6) (7,8) (9,10) (11,12)
    SCHED_END_SET()
};

/* One FB search in mode `mode`, command posted at the next l1_sync.
 * Returns the FCCH frame the firmware derives from it (fn_offset of
 * prim_fbsb.c, with the rounding synchronize_tdma() does through the qbits),
 * -1 if nothing; *residu = TOA offset from the nominal position
 * (23 + n x 1250), in symbols. */
static long une_fb(int mode, uint32_t *debut, int *residu)
{
    memset(&g_fb, 0, sizeof g_fb);
    tdma_schedule_set(0, fb_sched_set, (uint16_t)mode);   /* outside l1_sync: 0 = the next one */
    banc_trame();                          /* this frame's l1_sync posts the command */
    *debut = g_fb_cmd_fn + 1;              /* next_time of the command: 1st search frame */
    for (int k = 0; k < 14 && !g_fb.det; k++) banc_trame();
    if (g_fb.det <= 0) return -1;
    int toa = g_fb.toa - 23;
    int n = (int)lrint((double)toa / BITS_PER_TDMA);
    *residu = toa - n * BITS_PER_TDMA;
    /* prim_fbsb.c: fn_offset = current - attempt + ntdma, and current - attempt = debut */
    return (long)(g_fb.fn_resp - (uint32_t)g_fb.attempt) + n;
}
```

```
static int est_fcch(uint32_t f) { return (f % 51) % 10 == 0 && (f % 51) <= 40; }

static int t_fb_mode(int mode, int16_t dac, int hz_attendu, const char *quoi)
{
    banc_source = banc_cellule;
    banc_cfg.afc_dac = dac;
    int ok = 0, n = 0, sautees = 0;
    for (int essai = 0; essai < 6; essai++) {
        banc_courir(3 + 7 * (unsigned)essai);           /* different positions relative to the FCCH */
        uint32_t debut; int residu = 0;
        long fcch = une_fb(mode, &debut, &residu);
        n++;
        uint32_t premiere = debut; while (!est_fcch(premiere)) premiere++;
        if (fcch < 0) { det("essai %d : recherche des fn=%u, pas de FB en 12 trames (FCCH en %u)\n", essai, debut,
         ↪  premiere); continue; }
        int fe = g_fb.freq - hz_attendu;
        int juste = fcch >= (long)debut && est_fcch((uint32_t)fcch) && abs(residu) <= 64 &&
                    abs(fe) <= (hz_attendu ? abs(hz_attendu) / 4 : 100);
        if (juste && fcch != (long)premiere) sautees++;
        det("essai %d : recherche des fn=%u -> FB%d att=%d TOA=%d (residu %+d symb.) PM=%d angle=%d (%+d Hz)
↪  SNR=%d ; FCCH deduite %ld%s%s\n",
            essai, debut, mode, g_fb.attempt, g_fb.toa, residu, g_fb.pm, g_fb.angle, g_fb.freq, g_fb.snr, fcch,
           fcch == (long)premiere ? "" : est_fcch((uint32_t)fcch) ? " (une FCCH sautee)" : " (PAS une trame FCCH)",
            juste ? "" : "  <- FAUX");
        ok += juste;
    }
    det("%s : %d/%d detections justes (FCCH reelle, TOA a +-64 symboles, frequence %+d Hz +-%d) ; %d fois la 1re
↪  FCCH sautee",
        quoi, ok, n, hz_attendu, hz_attendu ? abs(hz_attendu) / 4 : 100, sautees);
    return ok == n ? V_PASS : V_FAIL;
}
static int t_fb0(void) { return t_fb_mode(0, -700, 0, "FB mode 0 (acquisition)"); }
static int t_fb1(void) { return t_fb_mode(1, -700, 0, "FB mode 1 (poursuite)"); }
/* calypso_twl3025.c: (dac + 700) x 287 / (2^15/947) Hz, ~8.29 Hz/LSB. The VCXO
 * pulls by +60 LSB: the received carrier appears at -497 Hz, so the firmware
 * loop (afc_correct: dac += norm x err / slope) brings it back towards -700. */
static int t_fb_afc(void)
{
    int hz = (int)lrint(-60.0 * 287.0 / (32768.0 / 947.0));
    return t_fb_mode(0, -700 + 60, hz, "FB mode 0, VCXO +60 LSB (d_afc -640)");
}

/* SB */
static struct { int fait, crc_ok; uint32_t sb; int16_t toa, pm, angle, snr; uint32_t fn_resp; } g_sb[3];
static int sbdet_cmd(uint8_t p1, uint8_t p2, uint16_t p3)
{
    (void)p1; (void)p2; (void)p3;
    dsp_api.db_w->d_task_md = SB_DSP_TASK;
    dsp_api.ndb->d_fb_mode = 0;
    return 0;
}
static int sbdet_resp(uint8_t p1, uint8_t attempt, uint16_t p3)
{
    (void)p1; (void)p3;
    if (attempt > 2) return 0;
    g_sb[attempt].fait = 1;
    g_sb[attempt].fn_resp = banc_courant.fn;
    dsp_api.r_page_used = 1;
    if (dsp_api.db_r->a_sch[0] & (1 << B_SCH_CRC)) return 0;
    g_sb[attempt].crc_ok = 1;
    g_sb[attempt].toa = (int16_t)dsp_api.db_r->a_serv_demod[D_TOA];
    g_sb[attempt].pm = (int16_t)(dsp_api.db_r->a_serv_demod[D_PM] >> 3);
    g_sb[attempt].angle = (int16_t)dsp_api.db_r->a_serv_demod[D_ANGLE];
    g_sb[attempt].snr = (int16_t)dsp_api.db_r->a_serv_demod[D_SNR];
    g_sb[attempt].sb = dsp_api.db_r->a_sch[3] | (uint32_t)dsp_api.db_r->a_sch[4] << 16;
    return 0;
```

```
}
static const struct tdma_sched_item sb_sched_set[] = {
    SCHED_ITEM_DT(sbdet_cmd, 0, 0, 1), SCHED_END_FRAME(),
    SCHED_ITEM_DT(sbdet_cmd, 0, 0, 2), SCHED_END_FRAME(),
    SCHED_END_FRAME(),
    SCHED_ITEM(sbdet_resp, -4, 0, 1), SCHED_END_FRAME(),
    SCHED_ITEM(sbdet_resp, -4, 0, 2), SCHED_END_FRAME(),
    SCHED_END_SET()
};
/* l1s_decode_sb() */
static uint8_t decode_sb(uint32_t sb, uint32_t *fn)
{
    unsigned t1 = ((sb >> 23) & 1) | ((sb >> 7) & 0x1fe) | ((sb << 9) & 0x600);
    unsigned t2 = (sb >> 18) & 0x1f;
    unsigned t3p = ((sb >> 24) & 1) | ((sb >> 15) & 6);
    *fn = gsmtime2fn(t1, t2, t3p * 10 + 1);
    return (sb >> 2) & 0x3f;
}

static int t_sb(void)
{
    /* The SB command targets each SCH frame directly (p51 = 1, 11, .., 41):
     * the 1st command of sb_sched_set has next_time = SCH frame. */
    banc_source = banc_cellule;
    int n = 0, ok = 0, crc_faux = 0, faux = 0, rate = 0, fen = 0;
    char ligne[400];
    for (int k = 0; k < 20; k++) {
        uint32_t c = banc_fn();                 /* current_time of the next l1_sync */
        uint32_t s = c + 2; while ((s % 51) % 10 != 1 || (s % 51) > 41) s++;
        unsigned long fen0 = banc_stats.depots_fenetre_sb;
        memset(g_sb, 0, sizeof g_sb);
        while (banc_fn() < s - 2) banc_trame();
        /* l1_sync(s-2) schedules the set at +1: cmd1 at l1_sync(s-1), next_time s */
        tdma_schedule_set(1, sb_sched_set, 0);
        for (int i = 0; i < 6; i++) banc_trame();
        n++;
        bool dans_fenetre = banc_stats.depots_fenetre_sb > fen0;
        fen += dans_fenetre;
        uint32_t fn_dec = 0; uint8_t bsic = 0;
        if (g_sb[1].crc_ok) {
            bsic = decode_sb(g_sb[1].sb, &fn_dec);
            bool juste = bsic == banc_bsic && fn_dec == s;
            if (juste) ok++; else { faux++; crc_faux++; }
            snprintf(ligne, sizeof ligne, "SCH fn=%u : CRC ok, BSIC=%u fn=%u TOA=%d SNR=%d%s\n", s, bsic, fn_dec,
                     g_sb[1].toa, g_sb[1].snr, juste ? "" : "  <- CONTENU FAUX");
        } else {
            rate++;
            snprintf(ligne, sizeof ligne, "SCH fn=%u : CRC KO%s (2e essai sur trame factice : %s)\n", s,
                    dans_fenetre ? "" : ", jamais livre dans une fenetre SB", g_sb[2].crc_ok ? "CRC ok ?!" : "CRC
↪  KO");
            if (g_sb[2].crc_ok) faux++;
        }
        if (g_v || !g_sb[1].crc_ok || faux) det("%s", ligne);
        vlog("%s", ligne);
    }
    det("%d SCH vises : %d decodes justes, %d CRC KO, %d CRC ok au contenu faux ; %d livres dans une fenetre SB",
        n, ok, rate, faux, fen);
    (void)crc_faux;
    if (faux) return V_FAIL;
    /* a healthy receiver decodes a clean SCH every time; 1 in 20 is tolerated */
    return (ok >= n - 1) ? V_PASS : V_FAIL;
}

/* FB0 -> FB1 -> SB, like l1s_fbsb_req() + the responses of prim_fbsb.c */
static int t_fbsb(void)
{
```

```
    banc_source = banc_cellule;
    banc_cfg.afc_dac = -700;
    banc_courir(7);
    uint32_t debut; int residu;
    long f0 = une_fb(0, &debut, &residu);
    if (f0 < 0) { det("FB0 introuvable"); return V_FAIL; }
    det("FB0 : TOA=%d %+d Hz -> FCCH %ld\n", g_fb.toa, g_fb.freq, f0);
    long f1 = une_fb(1, &debut, &residu);
    if (f1 < 0) { det("FB1 introuvable"); return V_FAIL; }
    det("FB1 : TOA=%d %+d Hz -> FCCH %ld\n", g_fb.toa, g_fb.freq, f1);
    /* delay = fn_offset + 11 - fn - 1, scheduled in the l1_sync of the response */
    int toa = g_fb.toa - 23;
    int ntdma = toa < 0 ? -1 : toa / BITS_PER_TDMA;
    int fn_offset = (int)g_fb.fn_resp - g_fb.attempt + ntdma;
    /* we are past the response frame: the set starts at the next l1_sync */
    int delay = fn_offset + 11 - (int)banc_fn() - 1;
    if (delay < 0) delay = 0;
    uint32_t s_vise = (uint32_t)(fn_offset + 11);
    memset(g_sb, 0, sizeof g_sb);
    tdma_schedule_set((uint8_t)(delay + 1), sb_sched_set, 0);
    for (int i = 0; i < delay + 8; i++) banc_trame();
    for (int a = 1; a <= 2; a++) {
        if (!g_sb[a].crc_ok) { det("SB essai %d : CRC KO\n", a); continue; }
        uint32_t fn_dec; uint8_t bsic = decode_sb(g_sb[a].sb, &fn_dec);
        det("SB essai %d : CRC ok BSIC=%u fn=%u (vise %u) TOA=%d\n", a, bsic, fn_dec, s_vise, g_sb[a].toa);
        if (bsic == banc_bsic && fn_dec == s_vise) { det("acquisition complete FB0/FB1/SB : BSIC %u, horloge %u",
         ↪  bsic, fn_dec); return V_PASS; }
    }
    det("SB non decode apres FB1 (SCH vise %u)", s_vise);
    return V_FAIL;
}

/* ================================================================== */
/*  downlink NB (layer1/prim_rx_nb.c)                                 */
/* ================================================================== */
static struct { int n_blocs, ok, fire, faux, vide, mauvais_id, err_tot; uint16_t tache; uint8_t chan_type;
                uint32_t blocs_fn[64]; int toa[4], snr[4]; char premier_ko[400]; } g_nb;
static uint8_t g_nb_l2_attendu[23];
static const uint8_t *(*g_nb_l2_de)(uint32_t fn0);
static int nb_resp(uint8_t p1, uint8_t burst_id, uint16_t p3)
{
    (void)p1; (void)p3;
    if (dsp_api.db_r->d_task_d == 0) { g_nb.vide++; return 0; }          /* "EMPTY" */
    if (dsp_api.db_r->d_burst_d != burst_id) { g_nb.mauvais_id++; return 0; }
    g_nb.toa[burst_id] = (int16_t)dsp_api.db_r->a_serv_demod[D_TOA];
    g_nb.snr[burst_id] = dsp_api.db_r->a_serv_demod[D_SNR];
    if (burst_id == 3) {
        uint32_t fn0 = banc_courant.fn - 5;       /* rx of burst 0: current - 4 - 1 */
        uint8_t l2[23];
        uint16_t fire = (dsp_api.ndb->a_cd[0] & ((1 << B_FIRE1) | (1 << B_FIRE0))) >> B_FIRE0;
        uint16_t err = dsp_api.ndb->a_cd[2];
        dsp_memcpy_from_api(l2, &dsp_api.ndb->a_cd[3], 23, 0);
        const uint8_t *att = g_nb_l2_de(fn0);
        g_nb.n_blocs++;
        g_nb.err_tot += err;
        bool egal = att && !memcmp(l2, att, 23);
        if (!fire && egal) g_nb.ok++;
        else {
            if (fire) g_nb.fire++; else g_nb.faux++;
            if (!g_nb.premier_ko[0]) {
                char h1[80], h2[80]; hex23(h1, l2, 8); hex23(h2, att ? att : l2, 8);
                snprintf(g_nb.premier_ko, sizeof g_nb.premier_ko, "bloc fn0=%u : a_cd0=%04x FIRE=%u err=%u L2 %s..
 ↪  attendu %s..",
                         fn0, dsp_api.ndb->a_cd[0], fire, err, h1, h2);
            }
        }
```

```
        vlog("bloc fn0=%u a_cd0=%04x FIRE=%u err=%u %s TOA=%d/%d/%d/%d\n", fn0, dsp_api.ndb->a_cd[0], fire, err,
             egal ? "L2 egal" : "L2 DIFFERENT", g_nb.toa[0], g_nb.toa[1], g_nb.toa[2], g_nb.toa[3]);
        dsp_api.db_w->d_task_d = 0;
    }
    dsp_api.r_page_used = 1;
    return 0;
}
static int nb_cmd(uint8_t p1, uint8_t burst_id, uint16_t p3)
{
    (void)p1; (void)p3;
    dsp_load_tch_param(&banc_suivant, SIG_ONLY_MODE, g_nb.chan_type, 0, 0, 0, 0);
    dsp_load_rx_task(g_nb.tache, burst_id, banc_bsic & 7);
    return 0;
}
static const struct tdma_sched_item nb_sched_set[] = {
    SCHED_ITEM_DT(nb_cmd, 0, 0, 0), SCHED_END_FRAME(),
    SCHED_ITEM_DT(nb_cmd, 0, 0, 1), SCHED_END_FRAME(),
    SCHED_ITEM(nb_resp, -4, 0, 0), SCHED_ITEM_DT(nb_cmd, 0, 0, 2), SCHED_END_FRAME(),
    SCHED_ITEM(nb_resp, -4, 0, 1), SCHED_ITEM_DT(nb_cmd, 0, 0, 3), SCHED_END_FRAME(),
    SCHED_ITEM(nb_resp, -4, 0, 2), SCHED_END_FRAME(),
    SCHED_ITEM(nb_resp, -4, 0, 3), SCHED_END_FRAME(),
    SCHED_END_SET()
};
static const uint8_t *nb_l2_cellule(uint32_t fn0) { return cellule_nb_alea(fn0, (int)(fn0 % 51)); }

/* Read n xCCH blocks of the cell at position p51 (2 = BCCH, 6 = CCCH,
 * 22 = SDCCH/4 n0, 42 = SACCH/4 ...) with task `tache`. */
static int nb_lecture(uint16_t tache, uint8_t chan_type, int p51, int modulo, int n_blocs, const char *quoi)
{
    memset(&g_nb, 0, sizeof g_nb);
    g_nb.tache = tache; g_nb.chan_type = chan_type;
    g_nb_l2_de = nb_l2_cellule;
    banc_source = banc_cellule;
    banc_cellule_nb = cellule_nb_alea;
    banc_mframe_ajouter(nb_sched_set, (uint16_t)modulo, (uint16_t)p51, 0);
    uint32_t lim = banc_fn() + (uint32_t)(modulo * (n_blocs + 1) + 10);
    while (g_nb.n_blocs < n_blocs && banc_fn() < lim) banc_trame();
    banc_mframe_vider();
    det("%s (tache %u) : %d/%d blocs decodes justes, %d FIRE KO, %d contenu faux, %d erreurs canal au total ; "
        "R vide %d, d_burst_d faux %d ; TOA dernier bloc %d/%d/%d/%d",
        quoi, tache, g_nb.ok, g_nb.n_blocs, g_nb.fire, g_nb.faux, g_nb.err_tot, g_nb.vide, g_nb.mauvais_id,
        g_nb.toa[0], g_nb.toa[1], g_nb.toa[2], g_nb.toa[3]);
    if (g_nb.premier_ko[0]) det("\n1er echec : %s", g_nb.premier_ko);
    if (g_nb.n_blocs == 0) { det("\naucun bloc rendu"); return V_FAIL; }
    return (g_nb.ok == g_nb.n_blocs && g_nb.n_blocs >= n_blocs) ? V_PASS : V_FAIL;
}
static int t_nb_bcch(void) { return nb_lecture(ALLC_DSP_TASK, SDCCH_4, 2, 51, 8, "BCCH norm. (ALLC, comme
↪  osmocom-bb)"); }

/* ================================================================== */
/*  uplink NB (layer1/prim_tx_nb.c)                                   */
/* ================================================================== */
static struct { uint16_t tache; uint8_t l2[16][23]; uint32_t fn_cmd[16][4]; int n_blocs; int sacch; } g_tx;
static int tx_resp(uint8_t p1, uint8_t burst_id, uint16_t p3)
{
    (void)p1; (void)burst_id; (void)p3;
    dsp_api.r_page_used = 1;
    return 0;
}
static int tx_cmd(uint8_t p1, uint8_t burst_id, uint16_t p3)
{
    (void)p3;
    if (burst_id == 0) {
        uint16_t *info = dsp_api.ndb->a_cu;
        int b = g_tx.n_blocs < 16 ? g_tx.n_blocs : 15;
        l2_bloc(0x7000u + (uint32_t)banc_suivant.fn, g_tx.l2[b]);
```

```
        info[0] = (1 << B_BLUD);
        info[1] = 0;
        info[2] = 0;
        dsp_memcpy_to_api(&info[3], g_tx.l2[b], 23, 0);
        g_tx.n_blocs++;
    }
    if (g_tx.n_blocs >= 1 && g_tx.n_blocs <= 16) g_tx.fn_cmd[g_tx.n_blocs - 1][burst_id & 3] = banc_suivant.fn;
    if (p1 == 2) dsp_load_tch_param(&banc_suivant, SIG_ONLY_MODE, SDCCH_4, 0, 0, 0, 0);
    else         dsp_load_tch_param(&banc_suivant, SIG_ONLY_MODE, INVALID_CHANNEL, 0, 0, 0, 0);
    dsp_load_tx_task(g_tx.tache, burst_id, banc_bsic & 7);
    return 0;
}
static const struct tdma_sched_item nb_sched_set_ul[] = {
    SCHED_ITEM_DT(tx_cmd, 3, 2, 0), SCHED_END_FRAME(),
    SCHED_ITEM_DT(tx_cmd, 3, 2, 1), SCHED_END_FRAME(),
    SCHED_ITEM(tx_resp, -4, 2, 0), SCHED_ITEM_DT(tx_cmd, 3, 2, 2), SCHED_END_FRAME(),
    SCHED_ITEM(tx_resp, -4, 2, 1), SCHED_ITEM_DT(tx_cmd, 3, 2, 3), SCHED_END_FRAME(),
    SCHED_ITEM(tx_resp, -4, 2, 2), SCHED_END_FRAME(),
    SCHED_ITEM(tx_resp, -4, 2, 3), SCHED_END_FRAME(),
    SCHED_END_SET()
};

/* Compare the bursts sent (banc_ul) after each block command with
 * gsm0503_xcch_encode; a5: expected stream XORed (algo, Kc) at the burst's fn. */
static int ul_comparer_xcch(int a5_algo, const uint8_t *kc, const char *quoi)
{
    int blocs_ok = 0, bursts_ok = 0, bursts_n = 0, manquants = 0;
    char premier[600] = "";
    for (int b = 0; b < g_tx.n_blocs && b < 16; b++) {
        ubit_t e[4 * 116];
        gsm0503_xcch_encode(e, g_tx.l2[b]);
        int bon = 0;
        for (int k = 0; k < 4; k++) {
            uint32_t f = g_tx.fn_cmd[b][k];
            /* the burst sent for frame f: captured at the end of frame f */
            const struct banc_ul_burst *u = NULL;
            for (int i = 0; i < banc_n_ul && i < BANC_UL_MAX; i++)
                if (banc_ul[i].fn == f) u = &banc_ul[i];
            bursts_n++;
            if (!u) { manquants++; if (!premier[0]) snprintf(premier, sizeof premier, "bloc %d burst %d (fn %u) :
             ↪  aucun burst emis (0x3d9b immobile)", b, k, f); continue; }
            uint8_t att[116];
            memcpy(att, e + 116 * k, 116);
            if (a5_algo) {
                ubit_t dl[114], ul[114];
                osmo_a5(a5_algo, kc, f, dl, ul);
                for (int j = 0; j < 57; j++) { att[j] ^= ul[j]; att[59 + j] ^= ul[57 + j]; }
            }
            unsigned d = popdiff(u->bits, att, 116);
            if (!d && u->rang == k) { bon++; bursts_ok++; continue; }
            if (!premier[0]) {
                int dec = 99;
                if (a5_algo) {            /* ciphered at another frame? */
                    for (int df = -3; df <= 3; df++) {
                        uint8_t a2[116]; memcpy(a2, e + 116 * k, 116);
                        ubit_t dl[114], ul[114]; osmo_a5(a5_algo, kc, f + df, dl, ul);
                        for (int j = 0; j < 57; j++) { a2[j] ^= ul[j]; a2[59 + j] ^= ul[57 + j]; }
                        if (!popdiff(u->bits, a2, 116)) { dec = df; break; }
                    }
                }
                unsigned dclair = popdiff(u->bits, e + 116 * k, 116);
               snprintf(premier, sizeof premier, "bloc %d burst %d (fn %u, rang ROM %d) : %u bits faux sur 116 (%u
 ↪  contre le burst en clair)%s",
                         b, k, f, u->rang, d, dclair,
                         dec != 99 ? (dec ? " ; chiffre avec le flux d'une autre trame" : "") : "");
```

```
                if (dec != 99 && dec) { size_t l = strlen(premier); snprintf(premier + l, sizeof premier - l, "
                 ↪  (fn%+d)", dec); }
            }
        }
        if (bon == 4) blocs_ok++;
    }
    det("%s : %d/%d blocs identiques a gsm0503_xcch_encode%s (%d/%d bursts, %d non emis)",
        quoi, blocs_ok, g_tx.n_blocs, a5_algo ? " + A5" : "", bursts_ok, bursts_n, manquants);
    if (premier[0]) det("\n1er ecart : %s", premier);
    return (g_tx.n_blocs > 0 && blocs_ok == g_tx.n_blocs) ? V_PASS : V_FAIL;
}

static int ul_bloc(uint16_t tache, int p51, int modulo, int n_blocs, int a5, const uint8_t *kc, const char *quoi)
{
    memset(&g_tx, 0, sizeof g_tx);
    g_tx.tache = tache;
    banc_source = banc_cellule;
    if (a5) dsp_load_ciph_param(a5, kc);
    banc_mframe_ajouter(nb_sched_set_ul, (uint16_t)modulo, (uint16_t)p51, 0);
    uint32_t lim = banc_fn() + (uint32_t)(modulo * (n_blocs + 1) + 10);
    while (banc_fn() < lim && g_tx.n_blocs < n_blocs) banc_trame();
    banc_mframe_vider();
    banc_courir(8);
    return ul_comparer_xcch(a5, kc, quoi);
}
static int t_tx_sdcch(void) { return ul_bloc(DUL_DSP_TASK, 22 + 15, 51, 6, 0, NULL, "SDCCH/4 n0 montant (DUL,
 ↪  comme osmocom-bb)"); }
/* Same block, A5/1-ciphered by the ROM: ul_comparer_xcch compares with the osmo_a5(kc, fn) stream
 * of the command's frame (next_time) and tells whether the ROM ciphered with the stream of another
 * frame (fn+-d): at which time the ROM ciphers the uplink (99-couverture.sh:137). */
static int t_tx_sdcch_a5(void)
{
    static const uint8_t kc[8] = { 0x12, 0x34, 0x56, 0x78, 0x9a, 0xbc, 0xde, 0xf0 };
    return ul_bloc(DUL_DSP_TASK, 22 + 15, 51, 6, 1, kc, "SDCCH/4 n0 montant chiffre A5/1 (DUL)");
}

/* ====================================================================== */
/*  uplink RACH (layer1/prim_rach.c)                                      */
/* ====================================================================== */
/* The firmware writes d_rach = (ra << 8) | (bsic << 2) in the NDB and d_task_ra = RACH_DSP_TASK (10)
 * in the W page (prim_rach.c l1s_tx_rach_cmd). The ROM codes the access burst (36 coded bits) in its
 * mode 2 TX ring (pointer 0x3d97), the serialiser 0x8900 copies it into data[0x3f8a] at the transmit
 * frame, and the transmitter 0x85a2 adds the sync sequence (DROM 0xa0d9, 41 bits) and the tail bits.
 * Here the 36 coded bits found in 0x3f8a are compared with gsm0503_rach_ext_encode(ra, bsic)
 * (bits 49..84 of the 148-bit burst). */
static struct { uint8_t ra, bsic; uint32_t fn_cmd; } g_rach;
static int rach_cmd(uint8_t p1, uint8_t p2, uint16_t p3)
{
    (void)p1; (void)p2; (void)p3;
    dsp_api.ndb->d_rach = (uint16_t)((g_rach.ra << 8) | (g_rach.bsic << 2));
    dsp_api.db_w->d_task_ra = RACH_DSP_TASK;
    g_rach.fn_cmd = banc_suivant.fn;
    return 0;
}
static int rach_resp(uint8_t p1, uint8_t p2, uint16_t p3) { (void)p1; (void)p2; (void)p3; dsp_api.r_page_used = 1;
 ↪  return 0; }
static const struct tdma_sched_item rach_sched_set[] = {
    SCHED_ITEM_DT(rach_cmd, 3, 1, 0), SCHED_END_FRAME(),
    SCHED_END_FRAME(),
    SCHED_ITEM(rach_resp, -4, 1, 0), SCHED_END_FRAME(),
    SCHED_END_SET()
};
static int t_rach(void)
{
    banc_source = banc_cellule;
    static const char *sync = "01001011011111111100110011010101000111100";   /* 05.02 5.2.7, 41 bits */
```

```
    int n = 0, ok36 = 0, ok148 = 0;
    for (int essai = 0; essai < 5; essai++) {
        banc_courir(4);
        g_rach.ra = alea8(); g_rach.bsic = banc_bsic;
        int n0 = banc_n_tsp;
        tdma_schedule_set(0, rach_sched_set, 0);
        banc_courir(6);
        n++;
        ubit_t cod[36];
        gsm0503_rach_ext_encode(cod, g_rach.ra, g_rach.bsic, false);
        /* 1. the 36 coded bits in data[0x3f8a..0x3f8c] (the ROM's coder) */
        const uint16_t *m = banc_dsp->data;
        int d36 = 0;
        for (int k = 0; k < 36; k++) d36 += ((m[0x3f8a + k / 16] >> (15 - k % 16)) & 1) != cod[k];
        /* 2. the final burst (TSP script, transmitter 0x8605): [8 guard][8 extended tail 00111010][41 sync][36][3
         ↪  tail][guard] */
        char att[8 + 41 + 36 + 3 + 1]; int p = 0;
        memcpy(att, "00111010", 8); p = 8;
        memcpy(att + p, sync, 41); p += 41;
        for (int k = 0; k < 36; k++) att[p++] = '0' + cod[k];
        memcpy(att + p, "000", 3); p += 3; att[p] = 0;
        int trouve = -1; char flux[161];
        for (int i = n0; i < banc_n_tsp; i++) {
            const struct banc_tsp_burst *t = &banc_tsp[i % BANC_UL_MAX];
            for (int w = 0; w < 16; w++) for (int b = 0; b < 10; b++) flux[10 * w + b] = '0' + ((t->mots[w] >> (15
             ↪  - b)) & 1);
            flux[160] = 0;
            const char *q = strstr(flux, att);
            if (q) { trouve = (int)(q - flux); break; }
        }
        det("essai %d : ra=%02x bsic=%u : 36 bits codes dans 0x3f8a : %d faux ; burst final (TSP, %d capture%s) :
 ↪  [queue 00111010][sync][36][000] %s\n",
            essai, g_rach.ra, g_rach.bsic, d36, banc_n_tsp - n0, banc_n_tsp - n0 > 1 ? "s" : "",
            trouve >= 0 ? "trouve" : "ABSENT");
        if (trouve >= 0) det("          flux de 160 bits : %s (burst a l'offset %d)\n", flux, trouve);
        ok36 += d36 == 0; ok148 += trouve >= 0;
    }
    det("RACH : %d/%d codages (36 bits) = gsm0503_rach_ext_encode ; %d/%d access-bursts complets dans le script
 ↪  TSP de la ROM", ok36, n, ok148, n);
    return (ok36 == n && ok148 == n) ? V_PASS : V_FAIL;
}

/* ======================================================================== */
/*  test table                                                              */
/* ======================================================================== */
/* TOA of the FB when the FCCH is in the VERY FIRST search frame (command at
 * p51 = 9, search from p51 = 10: FCCH). FBPOS_MODE=0|1 (default 1).
 * With BANC_FB_DECAL=k (offset of the burst from the window opening), gives the
 * TOA of the FB as a function of k, without the 48-page terms of a distant FCCH. */
static int t_fb_pos(void)
{
    banc_source = banc_cellule;
    banc_cfg.afc_dac = -700;
    const char *em = getenv("FBPOS_MODE");
    int mode = em ? atoi(em) : 1;
    int vus = 0;
    for (int essai = 0; essai < 3; essai++) {
        uint32_t cible = (banc_fn() / 51 + 2) * 51 + 9;
        while (banc_fn() < cible) banc_trame();
        uint32_t debut; int residu = 0;
        long fcch = une_fb(mode, &debut, &residu);
        if (fcch < 0 || g_fb.det <= 0) { det("essai %d : debut=%u (p51=%u) : pas de FB\n", essai, debut, debut %
         ↪  51); continue; }
        vus++;
        det("essai %d : debut=%u (p51=%u) FB%d att=%d TOA=%d PM=%d angle=%d\n",
            essai, debut, debut % 51, mode, g_fb.attempt, g_fb.toa, g_fb.pm, g_fb.angle);
```

```
    }
    return vus ? V_PASS : V_FAIL;
}

typedef struct { const char *nom, *tache, *desc; int (*f)(void); } Test;
static const Test TESTS[] = {
    { "boot",      "-",            "chargeur + dsp_power_on() du firmware, IDLE, version API", t_boot },
    { "idle",      "0 (aucune)",   "scenario vide chaque trame : IT trame, retour IDLE, pas d'erreur", t_idle },
    { "checksum",  "33 CHECKSUM",  "dsp_checksum_task() : version/somme du code dans a_pm", t_checksum },
    { "fb0",       "5 FB, mode 0", "detection FCCH (d_fb_det, a_sync_demod : TOA -> trame, angle -> Hz)", t_fb0 },
    { "fb1",       "5 FB, mode 1", "detection FCCH en poursuite", t_fb1 },
    { "fbpos",     "5 FB",         "TOA FB, FCCH dans la 1re trame de recherche (FBPOS_MODE, BANC_FB_DECAL)",
     ↪  t_fb_pos },
    { "fb-afc",    "5 FB, mode 0", "erreur de frequence mesuree avec un VCXO decale (+60 LSB)", t_fb_afc },
    { "sb",        "6 SB",         "SCH : CRC (a_sch[0]), BSIC et T1/T2/T3 (a_sch[3..4]) sur 20 SCH", t_sb },
    { "fbsb",      "5,5,6",        "acquisition complete FB0 -> FB1 -> SB (prim_fbsb.c)", t_fbsb },
    { "nb",        "24 ALLC",      "BCCH : 8 blocs, a_cd FIRE + 23 octets == gsm0503_xcch_encode", t_nb_bcch },
    { "rach",      "10 RACH",      "access-burst montant : codage ROM (0x3f8a) et burst final (script TSP) ==
     ↪  05.03/05.02", t_rach },
    { "tx-sdcch",  "12 DUL",       "SDCCH montant : a_cu -> bursts 0x3f8a == gsm0503_xcch_encode", t_tx_sdcch },
    { "tx-sdcch-a5","12 DUL + A5", "SDCCH montant chiffre : bursts 0x3f8a == xcch_encode XOR osmo_a5(kc, fn de la
     ↪  commande)", t_tx_sdcch_a5 },
};
#define N_TESTS (int)(sizeof TESTS / sizeof TESTS[0])

/* One test in a child, from the booted DSP. */
static int lancer(const Test *t)
{
    int tube[2];
    /* DSP_TESTER_NOFORK=1: the test runs in this process (gdb, gprof); a crash takes everything down. */
    static int nofork = -1;
    if (nofork < 0) { const char *e = getenv("DSP_TESTER_NOFORK"); nofork = (e && *e == '1') ? 1 : 0; }
    if (nofork) {
        g_dl = 0; g_detail[0] = 0;
        int v = t->f();
        printf("[%-8s] %-10s %-14s %s\n", NOMV[v], t->nom, t->tache, g_detail);
        return v;
    }
    if (pipe(tube) < 0) { perror("pipe"); return V_FAIL; }
    fflush(stdout);
    pid_t p = fork();
    if (p == 0) {
        close(tube[0]);
        alarm(900);
        g_dl = 0; g_detail[0] = 0;
        int v = t->f();
        fflush(stdout);
        char buf[sizeof g_detail + 4];
        buf[0] = (char)v;
        memcpy(buf + 1, g_detail, g_dl + 1);
        if (write(tube[1], buf, g_dl + 2) < 0) _exit(3);
        close(tube[1]);
        _exit(0);
    }
    close(tube[1]);
    static char buf[sizeof g_detail + 4];
    ssize_t n = 0, r;
    while ((r = read(tube[0], buf + n, sizeof buf - 1 - (size_t)n)) > 0) n += r;
    close(tube[0]);
    int st = 0;
    waitpid(p, &st, 0);
    int v;
    const char *d;
    char crash[200];
    if (n >= 2) { v = buf[0]; buf[n] = 0; d = buf + 1; }
    else {
```

```
        v = V_FAIL;
        snprintf(crash, sizeof crash, "le test s'est arrete sans verdict (%s %d) : plantage du coeur ou delai
 ↪  depasse",
                 WIFSIGNALED(st) ? "signal" : "code", WIFSIGNALED(st) ? WTERMSIG(st) : WEXITSTATUS(st));
        d = crash;
    }
    printf("[%-8s] %-10s %-14s ", NOMV[v], t->nom, t->tache);
    /* first line of the detail on the verdict line, the rest indented */
    const char *q = d;
    while (*q) {
        const char *e = strchr(q, '\n');
        size_t l = e ? (size_t)(e - q) : strlen(q);
        if (q != d) printf("%38s", "");
        printf("%.*s\n", (int)l, q);
        q += l + (e ? 1 : 0);
    }
    if (!*d) printf("\n");
    return v;
}

static void usage(const char *p)
{
    printf("usage : %s [-v] [--trace 0..6] [--rom-dir DIR] [--insns N] [--bsic N] (--all | --list | TEST...)\n",
 ↪  p);
}

int main(int argc, char **argv)
{
    int tous = 0, liste = 0, n_choix = 0;
    const char *choix[64];
    for (int i = 1; i < argc; i++) {
        const char *a = argv[i];
        if (!strcmp(a, "-v")) g_v = 1;
        else if (!strcmp(a, "--all")) tous = 1;
        else if (!strcmp(a, "--list")) liste = 1;
        else if (!strcmp(a, "--rom-dir") && i + 1 < argc) banc_cfg.rom_dir = argv[++i];
        else if (!strcmp(a, "--insns") && i + 1 < argc) banc_cfg.budget = atol(argv[++i]);
        else if (!strcmp(a, "--bsic") && i + 1 < argc) banc_bsic = (uint8_t)(atoi(argv[++i]) & 0x3f);
        else if (!strcmp(a, "--trace") && i + 1 < argc) g_trace = atoi(argv[++i]);
        else if (!strcmp(a, "-h") || !strcmp(a, "--help")) { usage(argv[0]); return 0; }
        else if (a[0] == '-') { usage(argv[0]); return 255; }
        else if (n_choix < 64) choix[n_choix++] = a;
    }
    if (liste) {
       for (int i = 0; i < N_TESTS; i++) printf("  %-10s %-14s %s\n", TESTS[i].nom, TESTS[i].tache, TESTS[i].desc);
        return 0;
    }
    if (!tous && !n_choix) { usage(argv[0]); return 255; }
    for (int k = 0; k < n_choix; k++) {
        int vu = 0;
        for (int i = 0; i < N_TESTS; i++) if (!strcmp(choix[k], TESTS[i].nom)) vu = 1;
        if (!vu) { fprintf(stderr, "test inconnu : %s (--list)\n", choix[k]); return 255; }
    }
    banc_cfg.verbeux = g_v;
    /* The core probes write to stderr; filtered like in c54x_exe
     * (src/verbosite.c). The children write into the same pipe, the
     * filtering thread stays in the parent. */
    verbosite_installer(g_trace);
    if (banc_init() < 0) return 255;
    g_boot_rc = banc_dsp_power_on(&g_boot);
    printf("dsp_tester : ROM %s (API %04x), %ld insn/trame, BSIC %u ; l'ARM et la BTS sont simules, rien n'est
 ↪  envoye au dehors\n",
           banc_cfg.rom_dir, g_boot.version1, banc_cfg.budget, banc_bsic);
    int n[3] = { 0, 0, 0 };
    for (int i = 0; i < N_TESTS; i++) {
        int pris = tous;
```

```
        for (int k = 0; k < n_choix; k++) if (!strcmp(choix[k], TESTS[i].nom)) pris = 1;
        if (!pris) continue;
        n[lancer(&TESTS[i])]++;
    }
    printf("bilan : %d PASS, %d FAIL, %d NON-IMPL\n", n[0], n[1], n[2]);
    verbosite_retirer();
    if (g_trace) verbosite_bilan(stdout);
    return n[1] > 254 ? 254 : n[1];
}
```

3.34 /opt/GSM/c54x_exe/tools/dsp_txt2bin.py

5630 octets, 156 lignes → 156 lignes

```
#!/usr/bin/env python3
"""
dsp_txt2bin.py — Convert calypso_dsp.txt (OsmocomBB Compal DSP dumper format)
to raw 16-bit little-endian binary.

Input format expected (per `c54x_load_rom`, calypso_c54x.c:9798-9854):

    DSP dump: PROM0
    07000 : XXXX XXXX XXXX ... XXXX
    07010 : XXXX XXXX XXXX ... XXXX
    ...
    DSP dump: PROM1
    18000 : ...

Each data line is "ADDR : WORD WORD WORD ..." where ADDR is hex (no prefix)
and each WORD is 4 hex chars. Sections: regs, DROM, PDROM, PROM0..PROM3.

Usage:
  dsp_txt2bin.py INPUT.txt                      # summary only, no output
  dsp_txt2bin.py INPUT.txt OUT.bin               # emit all sections → OUT.SECTION.bin
  dsp_txt2bin.py INPUT.txt OUT.bin --section S   # emit one section, zero-filled
  dsp_txt2bin.py INPUT.txt OUT.bin --slice A L   # emit L words from addr A (hex/dec OK)

Output is raw LE words. Loadable as -M calypso,dsp-blob=OUT.bin if the slice
falls in DARAM's OVLY-accessible range (0x80..0x27FF) and the PC override is
honored.

Examples:
  # split-by-section dump (OUT.bin base → OUT.PROM0.bin, OUT.PROM1.bin, ...)
  dsp_txt2bin.py calypso_dsp.txt calypso_dsp.bin

  # extract first 0x100 words of PROM0 (0x7000..0x70FF) as a probe blob
  dsp_txt2bin.py calypso_dsp.txt prom0_head.bin --slice 0x7000 0x100

  # full PROM0 dump
  dsp_txt2bin.py calypso_dsp.txt prom0.bin --section PROM0
"""

import argparse
import re
import struct
import sys
from collections import defaultdict

SECTION_RE = re.compile(r"DSP dump:\s+(\w+)")
DATA_RE    = re.compile(r"^\s*([0-9a-fA-F]+)\s*:\s*(.+?)\s*$")
WORD_RE    = re.compile(r"[0-9a-fA-F]{1,4}")


def parse_txt(path):
```

```
    """Return dict {section_name: {addr: word}}."""
    sections = defaultdict(dict)
    cur = None
    with open(path) as fp:
        for line in fp:
            m = SECTION_RE.search(line)
            if m:
                cur = m.group(1)
                continue
            if cur is None:
                continue
            m = DATA_RE.match(line)
            if not m:
                continue
            base = int(m.group(1), 16)
            for i, w in enumerate(WORD_RE.findall(m.group(2))):
                sections[cur][base + i] = int(w, 16) & 0xFFFF
    return sections


def merge(sections):
    """Flatten all sections into one {addr: word} dict (last writer wins)."""
    out = {}
    for s in sections.values():
        out.update(s)
    return out


def emit_slice(words, start_addr, length, out_path):
    with open(out_path, "wb") as fp:
        for a in range(start_addr, start_addr + length):
            fp.write(struct.pack("<H", words.get(a, 0)))
    filled = sum(1 for a in range(start_addr, start_addr + length) if a in words)
    print(f"[txt2bin] slice {length} words at {start_addr:#06x} → {out_path} "
          f"({filled} filled, {length - filled} zero-filled gaps)")


def emit_section(words, name, out_path):
    if not words:
        sys.exit(f"[txt2bin] section '{name}' is empty")
    lo, hi = min(words), max(words)
    n = hi - lo + 1
    with open(out_path, "wb") as fp:
        for a in range(lo, hi + 1):
            fp.write(struct.pack("<H", words.get(a, 0)))
    filled = len(words)
    print(f"[txt2bin] section {name}: {n} words [{lo:#06x}..{hi:#06x}] → "
          f"{out_path} ({filled} filled, {n - filled} zero-filled gaps)")


def main():
    ap = argparse.ArgumentParser(
        description="Convert calypso_dsp.txt to raw 16-bit LE binary.",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=__doc__)
    ap.add_argument("input", help="path to calypso_dsp.txt")
    ap.add_argument("output", nargs="?", help="output .bin path")
    grp = ap.add_mutually_exclusive_group()
    grp.add_argument("--section", help="emit one section (regs/DROM/PDROM/PROM0..3)")
    grp.add_argument("--slice", nargs=2, metavar=("ADDR", "LEN"),
                     help="emit LEN words starting at ADDR (hex or decimal)")
    args = ap.parse_args()

    sections = parse_txt(args.input)

    if not args.output:
```

```
        print(f"[txt2bin] sections found in {args.input}:")
        for name in sorted(sections):
            ws = sections[name]
            if not ws:
                print(f"  {name}: (empty)")
                continue
            lo, hi = min(ws), max(ws)
            print(f"  {name:8s}: {len(ws):6d} words, "
                  f"{lo:#07x}..{hi:#07x}  (span {hi - lo + 1})")
        return

    if args.section:
        if args.section not in sections:
            sys.exit(f"[txt2bin] no section named '{args.section}'. "
                     f"Available: {', '.join(sorted(sections))}")
        emit_section(sections[args.section], args.section, args.output)
    elif args.slice:
        addr = int(args.slice[0], 0)
        length = int(args.slice[1], 0)
        emit_slice(merge(sections), addr, length, args.output)
    else:
        # No flags + output given → split each section into OUTPUT.SECTION.bin.
        # Sections live in different address spaces (data vs program) and may
        # overlap in DSP address, so a single merged flat file would be
        # ambiguous. One file per section keeps things semantically clean.
        base = args.output
        if base.lower().endswith(".bin"):
            base = base[:-4]
        any_emitted = False
        for name in sorted(sections):
            if not sections[name]:
                continue
            emit_section(sections[name], name, f"{base}.{name}.bin")
            any_emitted = True
        if not any_emitted:
            sys.exit("[txt2bin] no non-empty sections found in input")


if __name__ == "__main__":
    main()
```

3.35 /opt/GSM/c54x_exe/tools/injecteur_bruit.py

32362 octets, 710 lignes → 710 lignes

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
# injecteur_bruit.py - UDP proxy that degrades the DOWNLINK direction between the
# bridge (osmo-operator/pont) and the layer 1 of the emulated phone.
#
# Single source file:
#   /opt/GSM/qosmo/tools/injecteur_bruit.py
# (c54x_exe/run.sh takes it from there, with a fallback on
# c54x_exe/tools/injecteur_bruit.py, a link to this file;
# qosmo/run_modules/38-bruit.sh starts it for the qosmo plan).
#
# WHERE IT SITS
#   dsp target  : pont_dsp.py --dsp-port 6702 -> [injector 127.0.0.1:6702]
#                 -> c54x_exe BSP moved to CALYPSO_BSP_PORT (16702 by default).
#                 Format: 8-byte header [tn, fn BE32, att, 0, 0] then 148
#                 bytes of HARD bits 0/1 (pont/trx.py Trx.run_data;
#                 calypso_bsp.c bsp_trxd_readable).
#                 The BSP learns the address of its uplink peer from the sender
#                 of the downlink: once the injector sits in between, what the
```

```
#                 BSP sends back (UL TRXDv0 bursts, if any) arrives here and
#                 leaves UNMODIFIED towards the source (the bridge,
#                 127.0.0.1:5702), from the listening port -- exactly the
#                 address the BSP had before.
#   grgsm target: pont.py -> GSMTAP [injector 127.0.0.1:14730] -> gr-gsm L1 of
#                 QEMU (127.0.0.1:4730, port fixed in calypso_l1_grgsm.c).
#                 The bridge sends there when PONT_GSMTAP_PORT=14730
#                 (pont/config.py; pont/pont.py sets it by itself when
#                 BRUIT_MODE is ber|relais).
#                 Format: 16-byte GSMTAP v2 header then the ALREADY DECODED L2
#                 block (23 bytes) -- NOT bursts, NO soft values.
#                 The SCH (4731, "SCH2") and TCH speech (/dev/shm/calypso_tch_dl)
#                 do not go through here and are not touched.
#
# MODES (downlink only; the uplink is always relayed as is)
#   relais  no alteration: checks that the insertion alone changes nothing.
#   ber     random bit inversion, mean rate --ber. --rafales L: clustered
#           errors (Gilbert-Elliott, bursts of L bits on average, rate
#           --ber-rafale inside a burst), same mean rate.
#           dsp: hard bits inverted (0<->1). grgsm: bits of the L2 bytes
#           inverted AFTER decoding -- not physical (neither the error
#           correcting code nor the CRC protect the block any more); --perte
#           is the realistic model for grgsm.
#   souple  AWGN at --snr-db on the soft values. Neither stream carries any
#           (dsp: hard bits 0/1; grgsm: L2 bytes). On hard bits: antipodal
#           +-1 modulation, AWGN, hard decision -- BER = Q(sqrt(2 snr)),
#           announced at start-up; the output stays 0/1, the only format the
#           BSP can read. If a source sends int8 soft values (TRXD convention:
#           +127 = bit 1), they are noised and kept soft. Refused for grgsm.
#   iq      EXPERIMENTAL. Bits -> GMSK BT=0.3 at 4 samples/symbol (same
#           modulator as calypso_gmsk.c, instant 0.5) + AWGN at --snr-db, sent
#           as interleaved int16 I/Q (BSP IQ_PASSTHROUGH, decimation by 4).
#           BYPASSES the BSP's tuned modulation (sb_moduler for the SCH:
#           instant 0.35, widening 1.0, own noise; gmsk_elargir 0.3 on the
#           NBs -- only the latter is imitated here, --iq-elargir).
#           INCOMPATIBLE with CALYPSO_BSP_STREAM=1 (the default of
#           start-native.sh --dsp): in STREAM the BSP stores the first 148
#           bytes of the packet as bits BEFORE the IQ_PASSTHROUGH branch
#           (calypso_bsp.c bsp_trxd_readable -> bsp_ts0_stocker). Refused in
#           that case, unless --forcer-iq. Requires numpy.
#   --perte P (all modes): one burst/block out of 1/P is erased. dsp: the
#           payload is replaced by random bits (fading: the BSP keeps its pace,
#           a dropped burst would make it wait CALYPSO_BSP_ATTENTE_MS).
#           grgsm: the block is not transmitted (= CRC failure on the L1 side).
#
# DETERMINISM: each packet draws its randomness from a seed derived from
# (--graine, fn, tn): same seed + same frames = same errors, whatever the
# arrival order (except --rafales, whose state follows the arrival order).
# Without --graine, a seed is drawn and PRINTED at start-up so a run can be
# replayed.
#
# Clean stop on SIGTERM/SIGINT (final statistics), SIGUSR1 = statistics right
# away. --verifier: checks the configuration and exits (0 = good).
import argparse
import logging
import math
import os
import random
import selectors
import signal
import socket
import struct
import sys
import time

VERSION = "2026-10-03"
```

```
log = logging.getLogger("bruit")

PROFILS = {
    # target: (listen = where the source sends, to = the layer 1)
    "dsp": ("127.0.0.1:6702", "127.0.0.1:16702"),
    "grgsm": ("127.0.0.1:14730", "127.0.0.1:4730"),
}
MODES = ("relais", "ber", "souple", "iq")
MODES_GRGSM = ("relais", "ber")

DSP_HDR = 8              # [tn, fn BE32, att, 0, 0]
NBITS = 148
IQ_MIN_OCTETS = 4 * 146  # calypso_bsp.c threshold to read the payload as I/Q
GSMTAP_TYPE_UM = 0x01
MASQUE64 = (1 << 64) - 1


def _adresse(txt, option):
    hote, sep, port = txt.rpartition(":")
    if not sep or not hote:
        raise argparse.ArgumentTypeError("%s : HOTE:PORT attendu, pas %r" % (option, txt))
    try:
        p = int(port)
    except ValueError:
        raise argparse.ArgumentTypeError("%s : port invalide %r" % (option, port))
    if not 0 < p < 65536:
        raise argparse.ArgumentTypeError("%s : port hors bornes %d" % (option, p))
    return (hote, p)


def _q_gauss(x):
    """Q(x) = P(N(0,1) > x)."""
    return 0.5 * math.erfc(x / math.sqrt(2.0))


def _geometrique(rng, p):
    """Number of trials up to the first success (>= 1), success probability p."""
    if p >= 1.0:
        return 1
    if p <= 0.0:
        return 1 << 62
    u = rng.random()
    return 1 + int(math.log1p(-u) / math.log1p(-p))


class Rafales:
    """Gilbert-Elliott: good state (no error) / bad state (rate q), geometric
    sojourns. Mean burst length L bits, mean rate p."""

    def __init__(self, p, longueur, q):
        self.q = q
        pi_m = p / q                          # fraction of the time in a burst
        self.p_mb = 1.0 / longueur            # bad -> good
        self.p_bm = self.p_mb * pi_m / (1.0 - pi_m) if pi_m < 1.0 else 1.0
        self.mauvais = False
        self.restant = None

    def positions(self, n, rng):
        if self.restant is None:
            self.restant = _geometrique(rng, self.p_bm)
        out = []
        pos = 0
        while pos < n:
            k = min(self.restant, n - pos)
            if self.mauvais:
                out.extend(i for i in range(pos, pos + k) if rng.random() < self.q)
```

```
            pos += k
            self.restant -= k
            if self.restant == 0:
                self.mauvais = not self.mauvais
                self.restant = _geometrique(rng, self.p_mb if self.mauvais else self.p_bm)
        return out


def positions_iid(n, p, rng):
    """Positions of the inverted bits, independent with probability p (geometric skip)."""
    if p <= 0.0:
        return []
    if p >= 1.0:
        return list(range(n))
    out = []
    lp = math.log1p(-p)
    i = -1
    while True:
        i += 1 + int(math.log1p(-rng.random()) / lp)
        if i >= n:
            return out
        out.append(i)


class ModulateurGmsk:
    """GMSK BT=0.3 (45.004) at `sps` samples per symbol, like calypso_gmsk.c
    gmsk_moduler(): differential coding d_i = b_i ^ b_{i-1} (b_{-1} = 1),
    alpha_i = 1 - 2 d_i, phase(t) = pi/2 * sum alpha_i q(t - i), sample j at
    instant t = j/sps + decalage. With sps=4 and decalage=0.5, the samples 4k
    (the ones the BSP keeps, decimation by 4) are those of
    gmsk_moduler(..., decalage=0.5)."""

    def __init__(self, np, sps=4, decalage=0.5):
        self.np = np
        self.sps = sps
        # The q(t) table is computed EXACTLY like calypso_gmsk.c q_init():
        # left Riemann sum, step T/16, normalised, then linear interpolation
        # (q_de). A finer integration would give another waveform (q(0) =
        # 0.500 instead of 0.477: up to 0.05 rad apart) and the iq mode would
        # no longer be "the same modulator" as the BSP.
        bt, os_, lg = 0.3, 16, 3
        k = 2.0 * math.pi * bt / math.sqrt(math.log(2.0))
        dt = 1.0 / os_
        tab, acc = [], 0.0
        for i in range(lg * os_ + 1):
            t = -1.5 + i * dt
            tab.append(acc)
            acc += (_q_gauss(k * (t - 0.5)) - _q_gauss(k * (t + 0.5))) * dt
        tab = [v / acc for v in tab]

        def q(t):
            if t <= -1.5:
                return 0.0
            if t >= 1.5:
                return 1.0
            x = (t + 1.5) * os_
            i = int(x)
            f = x - i
            if i >= lg * os_:
                return 1.0
            return tab[i] * (1.0 - f) + tab[i + 1] * f

        # K(n) = q(n/sps + decalage): 0 for t <= -1.5, 1 for t >= 1.5.
        n_lo = math.floor((-1.5 - decalage) * sps) + 1
        n_un = math.ceil((1.5 - decalage) * sps)
        self.n_lo = n_lo
```

```
        self.n_un = n_un
        self.k_trans = np.array([q(n / sps + decalage) for n in range(n_lo, n_un)])

    def phase(self, bits):
        np = self.np
        b = np.asarray(bits, dtype=np.int8) & 1
        prec = np.concatenate(([1], b[:-1]))
        alpha = 1.0 - 2.0 * (b ^ prec)
        n = len(b)
        long_ = n * self.sps
        d = np.zeros(long_)
        d[:: self.sps] = alpha
        conv = np.convolve(d, self.k_trans)               # conv[k] <-> j = k + n_lo
        j = np.arange(long_)
        part_trans = conv[j - self.n_lo]
        cumul = np.cumsum(d)
        idx = j - self.n_un
        part_marche = np.where(idx >= 0, cumul[np.clip(idx, 0, None)], 0.0)
        return (math.pi / 2.0) * (part_trans + part_marche)


class Stats:
    def __init__(self):
        self.t0 = time.monotonic()
        self.t_prec = self.t0
        self.dl_prec = 0
        self.dl = 0            # downlink packets received
        self.relayes = 0       # downlink packets sent to the L1
        self.alteres = 0       # packets whose payload was degraded
        self.effaces = 0       # --perte
        self.hors_tn = 0       # outside --tn, relayed as is
        self.autres = 0        # unrecognised format, relayed as is
        self.iq_deja = 0       # payload already in I/Q, relayed as is
        self.iq_produits = 0   # bursts converted to I/Q (iq mode)
        self.souples = 0       # packets with soft values seen
        self.bits = 0          # hard bits examined (ber, souple on hard bits)
        self.inverses = 0      # hard bits changed
        self.ul = 0            # uplink packets relayed to the source
        self.ul_sans_dest = 0  # uplink received before any downlink
        self.err_envoi = 0

    def ligne(self, final=False):
        now = time.monotonic()
        dt = max(now - self.t_prec, 1e-9)
        pps = (self.dl - self.dl_prec) / dt
        self.t_prec, self.dl_prec = now, self.dl
        ber = (self.inverses / self.bits) if self.bits else 0.0
        return ("%s t=%.0fs DL recus=%d relayes=%d alteres=%d effaces=%d hors_tn=%d autres=%d iq_deja=%d"
                " iq_produits=%d souples=%d | bits=%d inverses=%d BER=%.3e | UL relayes=%d sans_dest=%d"
                " | erreurs_envoi=%d | %.0f pq/s"
                % ("BILAN" if final else "STATS", now - self.t0, self.dl, self.relayes, self.alteres,
                   self.effaces, self.hors_tn, self.autres, self.iq_deja, self.iq_produits, self.souples,
                   self.bits, self.inverses, ber, self.ul, self.ul_sans_dest, self.err_envoi, pps))


class Degradeur:
    def __init__(self, a, np=None):
        self.a = a
        self.cible = a.cible
        self.mode = a.mode
        self.stats = Stats()
        self.graine = a.graine
        self.tn = a.tn_set
        self.rafales = Rafales(a.ber, a.rafales, a.ber_rafale) if (a.mode == "ber" and a.rafales) else None
        self.snr_lin = 10.0 ** (a.snr_db / 10.0) if a.snr_db is not None else None
        self.p_souple = _q_gauss(math.sqrt(2.0 * self.snr_lin)) if self.snr_lin is not None else 0.0
```

```
        self.np = np
        self.mod = ModulateurGmsk(np) if a.mode == "iq" else None

    def _rng(self, fn, tn, sel=0):
        x = (self.graine * 0x9E3779B97F4A7C15 + (fn + 1) * 0x100000001B3 + tn * 0x2545F491 + sel * 0x632BE5AB) &
↪   MASQUE64
        return random.Random(x)

    # -- dsp target ----------------------------------------------------------
    def _dsp(self, pkt):
        st = self.stats
        if len(pkt) < DSP_HDR + NBITS:
            st.autres += 1
            return pkt
        charge = len(pkt) - DSP_HDR
        if charge >= IQ_MIN_OCTETS:
            st.iq_deja += 1
            return pkt
        tn = pkt[0] & 0x07
        if self.tn is not None and tn not in self.tn:
            st.hors_tn += 1
            return pkt
        fn = struct.unpack_from(">I", pkt, 1)[0]
        rng = self._rng(fn, tn)
        ba = bytearray(pkt)
        dur = not bytes(ba[DSP_HDR:]).translate(None, b"\x00\x01")   # only 0/1: hard bits
        if not dur:
            st.souples += 1
        efface = self.a.perte > 0.0 and rng.random() < self.a.perte
        if efface:
            st.effaces += 1
            if dur:
                for i in range(DSP_HDR, len(ba)):
                    ba[i] = rng.getrandbits(1)
            else:
                for i in range(DSP_HDR, len(ba)):
                    ba[i] = 0                                  # soft: erasure = no information
        mode = self.mode
        if mode == "relais":
            if efface:
                st.alteres += 1
            return bytes(ba)
        if mode == "ber":
            n = len(ba) - DSP_HDR
            pos = self.rafales.positions(n, rng) if self.rafales else positions_iid(n, self.a.ber, rng)
            if dur:
                for i in pos:
                    ba[DSP_HDR + i] ^= 1
            else:
                for i in pos:                                  # soft: sign inverted
                    v = ba[DSP_HDR + i]
                    v = v - 256 if v > 127 else v
                    ba[DSP_HDR + i] = min(127, -v) & 0xFF
            st.bits += n
            st.inverses += len(pos)
            st.alteres += 1
            return bytes(ba)
        if mode == "souple":
            snr = self.snr_lin
            if dur:
                # +-1 + N(0, sigma^2), sigma^2 = 1/(2 snr), then hard decision:
                # each bit changes independently with probability
                # Q(1/sigma) = Q(sqrt(2 snr)). Those positions are therefore
                # drawn directly -- exactly the same law, without 148 gaussians.
                n = len(ba) - DSP_HDR
                pos = positions_iid(n, self.p_souple, rng)
```

```
            for i in pos:
                ba[DSP_HDR + i] ^= 1
            st.bits += n
            st.inverses += len(pos)
        else:
            vals = [(v - 256 if v > 127 else v) for v in ba[DSP_HDR:]]
            amp = (sum(abs(v) for v in vals) / len(vals)) or 127.0
            sigma = amp / math.sqrt(2.0 * snr)
            for k, v in enumerate(vals):
                w = int(round(v + rng.gauss(0.0, sigma)))
                ba[DSP_HDR + k] = max(-127, min(127, w)) & 0xFF
        st.alteres += 1
        return bytes(ba)
    # iq mode
    if dur:
        bits = list(ba[DSP_HDR:DSP_HDR + NBITS])
    else:                                                # soft int8: hard decision (> 0 = 1)
        bits = [1 if 0 < v <= 127 else 0 for v in ba[DSP_HDR:DSP_HDR + NBITS]]
    return bytes(ba[:DSP_HDR]) + self._iq(bits, fn, tn)

def _iq(self, bits, fn, tn):
    np = self.np
    a = self.a
    ph = self.mod.phase(bits)
    i_ = a.amp * np.cos(ph)
    q_ = a.amp * np.sin(ph)
    if a.iq_elargir and any(bits):
        # Imitates the BSP's gmsk_elargir() on the samples it keeps
        # (neighbours at +-1 symbol = +-sps samples); FCCH (all zero)
        # left as is, like the BSP.
        s, e = self.mod.sps, a.iq_elargir
        for v in (i_, q_):
            w = v.copy()
            w[s:] += e * v[:-s]
            w[:-s] += e * v[s:]
            v[:] = w / (1.0 + 2.0 * e)
    if self.snr_lin is not None:
        sigma = math.sqrt(a.amp * a.amp / (2.0 * self.snr_lin))
        g = np.random.Generator(np.random.PCG64(self._rng(fn, tn, 1).getrandbits(64)))
        i_ = i_ + g.normal(0.0, sigma, i_.shape)
        q_ = q_ + g.normal(0.0, sigma, q_.shape)
    out = np.empty(2 * len(i_), dtype="<i2")
    out[0::2] = np.clip(np.rint(i_), -32768, 32767)
    out[1::2] = np.clip(np.rint(q_), -32768, 32767)
    self.stats.iq_produits += 1
    self.stats.alteres += 1
    return out.tobytes()

# -- grgsm target ------------------------------------------------------
def _grgsm(self, pkt):
    st = self.stats
    if len(pkt) < 16 or pkt[0] != 2 or pkt[2] != GSMTAP_TYPE_UM:
        st.autres += 1
        return pkt
    hdr = pkt[1] * 4
    if hdr < 16 or hdr >= len(pkt):
        st.autres += 1
        return pkt
    tn = pkt[3] & 0x07
    if self.tn is not None and tn not in self.tn:
        st.hors_tn += 1
        return pkt
    fn = struct.unpack_from(">I", pkt, 8)[0]
    rng = self._rng(fn, tn, pkt[12])
    if self.a.perte > 0.0 and rng.random() < self.a.perte:
        st.effaces += 1
```

```
            return None                                          # lost block = CRC failure on the L1 side
        if self.mode == "relais":
            return pkt
        ba = bytearray(pkt)
        n = (len(ba) - hdr) * 8
        pos = self.rafales.positions(n, rng) if self.rafales else positions_iid(n, self.a.ber, rng)
        for i in pos:
            ba[hdr + (i >> 3)] ^= 0x80 >> (i & 7)
        st.bits += n
        st.inverses += len(pos)
        st.alteres += 1
        return bytes(ba)

    def descendant(self, pkt):
        self.stats.dl += 1
        return self._dsp(pkt) if self.cible == "dsp" else self._grgsm(pkt)


def _parser():
    p = argparse.ArgumentParser(
        prog="injecteur_bruit.py",
        description="Proxy UDP transparent pont -> couche 1 qui degrade le sens descendant "
                    "(bits inverses, AWGN, I/Q bruite). Le montant est relaye sans modification.")
    p.add_argument("--cible", choices=sorted(PROFILS), default="dsp",
                   help="format du flux : dsp (TRXD 8 o + 148 bits, BSP de c54x_exe) ou grgsm "
                        "(GSMTAP vers la L1 gr-gsm de QEMU) ; defaut dsp")
    p.add_argument("--mode", choices=MODES, default="ber", help="defaut ber")
    p.add_argument("--ecoute", help="HOTE:PORT ou la source envoie (dsp 127.0.0.1:6702, grgsm 127.0.0.1:14730)")
    p.add_argument("--vers", help="HOTE:PORT de la couche 1 (dsp 127.0.0.1:16702, grgsm 127.0.0.1:4730)")
    p.add_argument("--source", help="HOTE:PORT local d'emission vers la L1 (defaut : port libre)")
    p.add_argument("--retour", help="HOTE:PORT ou renvoyer le montant (defaut : l'emetteur du dernier descendant)")
    p.add_argument("--ber", type=float, default=0.0, help="mode ber : taux moyen d'inversion (0..1)")
    p.add_argument("--rafales", type=float, default=0.0, metavar="L",
                   help="mode ber : erreurs en rafales de L bits en moyenne (Gilbert-Elliott)")
    p.add_argument("--ber-rafale", type=float, default=0.5, help="taux d'erreur dans une rafale (defaut 0,5)")
    p.add_argument("--snr-db", type=float, default=None,
                   help="modes souple et iq : SNR en dB, P_signal / (2 sigma^2) par echantillon complexe "
                        "(meme definition que CALYPSO_BSP_SNR_DB)")
    p.add_argument("--perte", type=float, default=0.0,
                   help="probabilite d'effacer un burst (dsp : bits aleatoires) ou un bloc (grgsm : non transmis)")
    p.add_argument("--tn", default="tous", help="intervalles a degrader, ex. 0 ou 1,2 (defaut tous)")
    p.add_argument("--graine", type=lambda s: int(s, 0), default=None, help="graine (deterministe)")
    p.add_argument("--amp", type=int, default=30000, help="mode iq : amplitude (defaut 30000, comme le BSP)")
    p.add_argument("--iq-elargir", type=float, default=0.3,
                   help="mode iq : elargissement des NB comme gmsk_elargir (defaut 0,3 ; 0 = aucun)")
    p.add_argument("--forcer-iq", action="store_true",
                   help="mode iq malgre CALYPSO_BSP_STREAM=1 ou CALYPSO_BSP_IQ_PASSTHROUGH=0")
    p.add_argument("--stats", type=float, default=10.0, help="periode des statistiques en s (0 = seulement le
↪   bilan)")
    p.add_argument("--journal", help="fichier journal (en plus de stderr)")
    p.add_argument("--pidfile", help="ecrit le pid, efface a l'arret")
    p.add_argument("--etiquette", default="", help="libre, pour reconnaitre le processus (pkill -f)")
    p.add_argument("--verifier", action="store_true", help="controle la configuration et sort")
    p.add_argument("--version", action="version", version="%(prog)s " + VERSION)
    return p


def _valider(a, p):
    """Completes and checks the arguments; returns numpy (iq mode) or None."""
    ecoute, vers = PROFILS[a.cible]
    a.ecoute = _adresse(a.ecoute or ecoute, "--ecoute")
    a.vers = _adresse(a.vers or vers, "--vers")
    a.source = _adresse(a.source, "--source") if a.source else None
    a.retour = _adresse(a.retour, "--retour") if a.retour else None
    if a.ecoute == a.vers:
```

```
        p.error("--ecoute et --vers designent la meme adresse %s:%d : l'injecteur s'enverrait ses propres paquets"
↪   % a.ecoute)
    if a.cible == "grgsm" and a.mode not in MODES_GRGSM:
        p.error("cible grgsm : mode %s impossible. Le flux GSMTAP vers QEMU porte des blocs L2 deja decodes "
                "(16 o d'en-tete + 23 o), ni valeurs souples ni bursts a moduler (calypso_l1_grgsm.c "
                "gsmtap_readable). Modes possibles : %s (et --perte)." % (a.mode, ", ".join(MODES_GRGSM)))
    for nom in ("ber", "perte", "ber_rafale"):
        v = getattr(a, nom)
        if not 0.0 <= v <= 1.0:
            p.error("--%s doit etre entre 0 et 1 (%g)" % (nom.replace("_", "-"), v))
    if a.mode == "ber" and a.rafales:
        if a.rafales < 1.0:
            p.error("--rafales : longueur moyenne en bits >= 1 (%g)" % a.rafales)
        if not 0.0 < a.ber_rafale <= 1.0 or a.ber >= a.ber_rafale:
            p.error("--rafales : il faut --ber (%g) < --ber-rafale (%g) <= 1" % (a.ber, a.ber_rafale))
    if a.mode == "souple" and a.snr_db is None:
        p.error("mode souple : --snr-db obligatoire")
    if a.snr_db is not None and not -50.0 <= a.snr_db <= 100.0:
        p.error("--snr-db hors bornes raisonnables (%g)" % a.snr_db)
    if a.tn in ("", "tous", "all"):
        a.tn_set = None
    else:
        try:
            a.tn_set = {int(x) for x in a.tn.split(",") if x.strip() != ""}
        except ValueError:
            p.error("--tn : liste d'intervalles 0..7 attendue (%r)" % a.tn)
        if not a.tn_set or any(not 0 <= x <= 7 for x in a.tn_set):
            p.error("--tn : intervalles 0..7 (%r)" % a.tn)
    if a.graine is None:
        a.graine = int.from_bytes(os.urandom(4), "little")
        a.graine_tiree = True
    else:
        a.graine_tiree = False
    a.graine &= MASQUE64
    np = None
    if a.mode == "iq":
        if a.cible != "dsp":
            p.error("mode iq : cible dsp seulement")
        try:
            import numpy as np  # noqa: F811 - only the iq mode needs it
        except ImportError:
            p.error("mode iq : numpy est necessaire et introuvable pour %s "
                    "(apt-get install python3-numpy, ou pip install numpy). "
                    "Les modes relais, ber et souple n'en ont pas besoin." % sys.executable)
        if not a.forcer_iq:
            if os.environ.get("CALYPSO_BSP_STREAM", "") == "1":
                p.error("mode iq refuse : CALYPSO_BSP_STREAM=1. En STREAM le BSP range les 148 premiers octets "
                        "de chaque paquet comme des BITS avant de regarder s'il s'agit d'I/Q "
                        "(calypso_bsp.c bsp_trxd_readable -> bsp_ts0_stocker) : l'I/Q y serait lu comme du bruit "
                        "de bits. Lancez c54x_exe sans STREAM, ou --forcer-iq (BRUIT_OPTS=--forcer-iq).")
            if os.environ.get("CALYPSO_BSP_IQ_PASSTHROUGH", "") == "0":
                p.error("mode iq refuse : CALYPSO_BSP_IQ_PASSTHROUGH=0, le BSP ne lit pas l'I/Q.")
        if not 1000 <= a.amp <= 32767:
            p.error("--amp : 1000..32767 (%d)" % a.amp)
        if not 0.0 <= a.iq_elargir <= 2.0:
            p.error("--iq-elargir : 0..2 (%g)" % a.iq_elargir)
    return np


def _ouvrir(a):
    amont = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    try:
        amont.bind(a.ecoute)
    except OSError as e:
        amont.close()
        raise SystemExit("injecteur_bruit : impossible d'ecouter sur %s:%d (%s) -- un autre processus "
```

```
                    "tient-il deja ce port ? ss -lunp | grep :%d" % (a.ecoute[0], a.ecoute[1], e.strerror,
                     ↪  a.ecoute[1]))
    aval = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    if a.source:
        loc = a.source
    else:
        loc = ("127.0.0.1", 0) if a.vers[0].startswith("127.") or a.vers[0] == "localhost" else ("0.0.0.0", 0)
    try:
        aval.bind(loc)
    except OSError as e:
        amont.close()
        aval.close()
        raise SystemExit("injecteur_bruit : impossible de lier le port d'emission %s:%d (%s)" % (loc[0], loc[1],
         ↪  e.strerror))
    for s in (amont, aval):
        s.setblocking(False)
        try:
            s.setsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF, 1 << 20)
        except OSError:
            pass
    return amont, aval


def _journal(a):
    fmt = logging.Formatter("%(asctime)s [bruit] %(message)s", "%H:%M:%S")
    log.setLevel(logging.INFO)
    h = logging.StreamHandler(sys.stderr)
    h.setFormatter(fmt)
    log.addHandler(h)
    if a.journal:
        f = logging.FileHandler(a.journal)
        f.setFormatter(fmt)
        log.addHandler(f)


def _annoncer(a, aval, np=None):
    loc = aval.getsockname()
    log.info("injecteur de bruit %s : cible %s, mode %s%s", VERSION, a.cible, a.mode,
             " (EXPERIMENTAL)" if a.mode == "iq" else "")
    if a.mode == "ber":
        if a.rafales:
            log.info("BER moyen %.3e en rafales de %.1f bits (taux %.2f dans une rafale, Gilbert-Elliott)",
                     a.ber, a.rafales, a.ber_rafale)
        else:
            log.info("BER %.3e (bits independants)", a.ber)
        if a.cible == "grgsm" and a.ber > 0:
            log.info("ATTENTION grgsm : bits inverses dans le bloc L2 DEJA DECODE, rien ne les detecte "
                     "(ni FEC ni CRC) ; --perte modelise l'echec CRC")
    if a.mode == "souple":
        snr = 10.0 ** (a.snr_db / 10.0)
        log.info("SNR %.2f dB ; sur bits durs (le cas du pont) : +-1 + AWGN + decision dure, "
                 "BER attendu Q(sqrt(2 snr)) = %.3e", a.snr_db, _q_gauss(math.sqrt(2.0 * snr)))
    if a.mode == "iq":
        log.info("I/Q GMSK 4 ech/symbole, amplitude %d, elargissement NB %.2f, %s ; "
                 "contourne sb_moduler/gmsk_elargir du BSP", a.amp, a.iq_elargir,
                 ("AWGN %.2f dB" % a.snr_db) if a.snr_db is not None else "SANS AWGN")
        if a.snr_db is not None and np is not None:
            # int16 clipping: at 30000 only 2767 of headroom remain. Estimated
            # here to announce it (the C side, CALYPSO_BSP_SNR_DB, clips the same way).
            g = np.random.default_rng(0)
            m = 20000
            s = a.amp * np.exp(1j * g.uniform(0.0, 2.0 * math.pi, m))
            sig = math.sqrt(a.amp * a.amp / (2.0 * 10.0 ** (a.snr_db / 10.0)))
            y = s + sig * (g.standard_normal(m) + 1j * g.standard_normal(m))
            ecr = float(np.mean((np.abs(y.real) > 32767) | (np.abs(y.imag) > 32767)))
            if ecr > 0.01:
```

```
                yc = np.clip(y.real, -32768, 32767) + 1j * np.clip(y.imag, -32768, 32767)
                eff = 10.0 * math.log10(a.amp * a.amp / float(np.mean(np.abs(yc - s) ** 2)))
                log.info("ATTENTION ecretage int16 : ~%.0f %% des echantillons, SNR effectif ~%.1f dB "
                         "(bruit non gaussien) ; --amp plus bas pour l'eviter", 100.0 * ecr, eff)
    if a.perte:
        log.info("effacement : %.3e des %s", a.perte, "bursts (bits aleatoires)" if a.cible == "dsp" else "blocs
↪   (non transmis)")
    if (a.mode == "relais" or (a.mode == "ber" and a.ber == 0.0)) and not a.perte:
        log.info("aucune degradation configuree : relais transparent")
    log.info("intervalles : %s ; graine %d%s", "tous" if a.tn_set is None else ",".join(map(str,
↪   sorted(a.tn_set))),
             a.graine, " (tiree : --graine %d pour rejouer)" % a.graine if a.graine_tiree else "")
    log.info("PRET ecoute=%s:%d vers=%s:%d emission=%s:%d retour=%s", a.ecoute[0], a.ecoute[1], a.vers[0],
↪   a.vers[1],
             loc[0], loc[1], ("%s:%d" % a.retour) if a.retour else "emetteur du descendant")


def main(argv=None):
    p = _parser()
    a = p.parse_args(argv)
    np = _valider(a, p)
    if a.verifier:
        amont, aval = _ouvrir(a)
        amont.close()
        aval.close()
        print("injecteur_bruit : configuration valide (cible %s, mode %s, ecoute %s:%d -> %s:%d)"
              % (a.cible, a.mode, a.ecoute[0], a.ecoute[1], a.vers[0], a.vers[1]))
        return 0
    _journal(a)
    amont, aval = _ouvrir(a)
    deg = Degradeur(a, np)
    st = deg.stats
    if a.pidfile:
        with open(a.pidfile, "w") as f:
            f.write("%d\n" % os.getpid())

    arret = []
    flash = []
    signal.signal(signal.SIGTERM, lambda s, f: arret.append(s))
    signal.signal(signal.SIGINT, lambda s, f: arret.append(s))
    signal.signal(signal.SIGUSR1, lambda s, f: flash.append(s))

    _annoncer(a, aval, np)
    sel = selectors.DefaultSelector()
    sel.register(amont, selectors.EVENT_READ, "dl")
    sel.register(aval, selectors.EVENT_READ, "ul")
    source = None
    prochain = time.monotonic() + a.stats if a.stats > 0 else None
    try:
        while not arret:
            try:
                evts = sel.select(timeout=0.5)
            except InterruptedError:
                evts = []
            for cle, _ in evts:
                s = cle.fileobj
                for _ in range(512):
                    try:
                        pkt, adr = s.recvfrom(65536)
                    except (BlockingIOError, InterruptedError):
                        break
                    except OSError:
                        st.err_envoi += 1            # ICMP reported on the socket: carry on
                        break
                    if cle.data == "dl":
                        source = adr
```

```
                        sortie = deg.descendant(pkt)
                        if sortie is None:
                            continue
                        try:
                            aval.sendto(sortie, a.vers)
                            st.relayes += 1
                        except OSError:
                            st.err_envoi += 1
                    else:
                        dest = a.retour or source
                        if dest is None:
                            st.ul_sans_dest += 1
                            continue
                        try:
                            amont.sendto(pkt, dest)  # uplink: as is
                            st.ul += 1
                        except OSError:
                            st.err_envoi += 1
            now = time.monotonic()
            if flash or (prochain is not None and now >= prochain):
                del flash[:]
                log.info("%s", st.ligne())
                if prochain is not None:
                    prochain = now + a.stats
    finally:
        log.info("%s", st.ligne(final=True))
        log.info("arret (%s)", "signal %d" % arret[0] if arret else "fin")
        sel.close()
        amont.close()
        aval.close()
        if a.pidfile:
            try:
                os.unlink(a.pidfile)
            except OSError:
                pass
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

3.36 /opt/GSM/c54x_exe/tools/isa_examples.py

19204 octets, 448 lignes → 448 lignes

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""isa_examples.py - turn the SPRU172C worked examples into ISA test vectors.

WHY. The C54x core (qosmo l1-dsp) has been fixed one instruction at a time,
each time a probe on the ROM's FB/SB path pointed at one. The manual carries
about 240 "Before Instruction / After Instruction" examples, one oracle per
instruction. This script reads them out of hw/arm/calypso/doc/spru172c.md,
assembles each example with the binutils opcode table (the same table the
disassembler uses, so encoding and decoding agree) and writes a flat test file
that tools/isa_test.c replays against the core.

    tools/isa_examples.py > tools/isa_tests.txt
    make isa_test && ./isa_test tools/isa_tests.txt

Test file format (one record per example):
    T <n> <section> | <instruction as printed>
    W <word> [<word> [<word>]]         assembled instruction
    B <REG> <hex>       B M <addr> <hex>    state before (registers, data memory)
```

```
    A <REG> <hex>       A M <addr> <hex>    state after
    E
Examples that cannot be assembled (labels, far calls, ports) are written as
    S <n> <section> | <instruction> | <reason>
so the count of what is NOT covered stays visible.
"""
import io, os, re, sys

DOC   = "/opt/GSM/qosmo-dsp/hw/arm/calypso/doc/spru172c.md"
TABLE = "/opt/GSM/qosmo-dsp/hw/arm/calypso/doc/opcodes/tic54x-opc.c"

# ---------------------------------------------------------------- the table --
ENTREE = re.compile(
    r'\{\s*"([^"]+)"\s*,\s*(\d+)\s*,[^,]*,[^,]*,\s*(0x[0-9A-Fa-f]+)\s*,\s*(0x[0-9A-Fa-f]+)\s*,\s*\{([^}]*)\}')

def charger_table(chemin):
    src = io.open(chemin, encoding="utf-8", errors="replace").read()
    fin = src.find("tic54x_paroptab")
    if fin > 0:
        src = src[:fin]
    tab = []
    for m in ENTREE.finditer(src):
        nom, mots, op, masque, ops = m.group(1), int(m.group(2)), \
            int(m.group(3), 16), int(m.group(4), 16), m.group(5).strip()
        if nom == "???":
            continue
        ops = [o.strip() for o in ops.split(",") if o.strip()]
        tab.append((nom, mots, op, masque, ops))
    return tab

# ---------------------------------------------------------------- operands ---
MMR = {"IMR": 0, "IFR": 1, "ST0": 6, "ST1": 7, "AL": 8, "AH": 9, "AG": 10, "BL": 11,
       "BH": 12, "BG": 13, "T": 14, "TREG": 14, "TRN": 15, "SP": 0x18, "BK": 0x19,
       "BRC": 0x1A, "RSA": 0x1B, "REA": 0x1C, "PMST": 0x1D, "XPC": 0x1E}
for _i in range(8):
    MMR["AR%d" % _i] = 0x10 + _i

COND = {"UNC": 0x00, "AEQ": 0x45, "ANEQ": 0x44, "ALT": 0x43, "ALEQ": 0x47, "AGT": 0x46,
        "AGEQ": 0x42, "BEQ": 0x4D, "BNEQ": 0x4C, "BLT": 0x4B, "BLEQ": 0x4F, "BGT": 0x4E,
        "BGEQ": 0x4A, "AOV": 0x70, "ANOV": 0x60, "BOV": 0x78, "BNOV": 0x68,
        "TC": 0x30, "NTC": 0x20, "C": 0x0C, "NC": 0x08, "BIO": 0x03, "NBIO": 0x02}

SBITS = {"BRAF": (1, 15), "CPL": (1, 14), "XF": (1, 13), "HM": (1, 12), "INTM": (1, 11),
         "OVM": (1, 9), "SXM": (1, 8), "C16": (1, 7), "FRCT": (1, 6), "CMPT": (1, 5),
         "TC": (0, 12), "C": (0, 11), "OVA": (0, 10), "OVB": (0, 9)}

# indirect Smem modifiers -> MOD nibble (SPRU172C table 3-?; binutils tic54x-dis.c)
SMOD = {"": 0x0, "-": 0x1, "+": 0x2, "-0B": 0x4, "-0": 0x5, "+0": 0x6, "+0B": 0x7,
        "-%": 0x8, "-0%": 0x9, "+%": 0xA, "+0%": 0xB}
XMOD = {"": 0, "-": 1, "+": 2, "+0%": 3}

def norm(t):
    return t.replace("—", "-").replace("−", "-").replace("–", "-").strip()

def nombre(t):
    """TI numbers: 0FFFEh, 1234h, 10h, 248, -8, +3, #..., 15-12 (bit expression)."""
    t = norm(t).lstrip("#").strip()
    if re.fullmatch(r'[+-]?[0-9A-Fa-f]+[hH]', t):
        return int(t[:-1], 16)
    if re.fullmatch(r'[+-]?\d+', t):
        return int(t, 10)
    if re.fullmatch(r'0[xX][0-9A-Fa-f]+', t):
        return int(t, 16)
    m = re.fullmatch(r'(\d+)\s*-\s*(\d+)', t)
    if m:
        return int(m.group(1)) - int(m.group(2))
```

```
    raise ValueError("nombre: %r" % t)

class Smem:
    """Parsed single data-memory operand."""
    def __init__(self, txt):
        t = norm(txt)
        self.lk = None
        if t.startswith("*"):
            m = re.fullmatch(r'\*\(\s*([^)]+)\)', t)
            if m:                                   # *(lk) absolute
                self.code = 0x80 | (0xF << 3); self.lk = nombre(m.group(1)); return
            m = re.fullmatch(r'\*\+AR(\d)\(([^)]+)\)(%?)', t)
            if m:                                   # *+ARn(lk) [%]
                self.code = 0x80 | ((0xE if m.group(3) else 0xD) << 3) | int(m.group(1))
                self.lk = nombre(m.group(2)); return
            m = re.fullmatch(r'\*AR(\d)\(([^)]+)\)', t)
            if m:                                   # *ARn(lk)
                self.code = 0x80 | (0xC << 3) | int(m.group(1)); self.lk = nombre(m.group(2)); return
            m = re.fullmatch(r'\*\+AR(\d)', t)
            if m:
                self.code = 0x80 | (0x3 << 3) | int(m.group(1)); return
            m = re.fullmatch(r'\*AR(\d)(.*)', t)
            if m and m.group(2) in SMOD:
                self.code = 0x80 | (SMOD[m.group(2)] << 3) | int(m.group(1)); return
            raise ValueError("Smem: %r" % txt)
        if t.startswith("@"):
            t = t[1:]
        if t.upper() in MMR:                        # MMR by name = direct address
            self.code = MMR[t.upper()]; return
        v = nombre(t)
        if not 0 <= v <= 0x7F:
            raise ValueError("Smem direct hors 0..7F: %r" % txt)
        self.code = v

def xmem(txt):
    t = norm(txt)
    m = re.fullmatch(r'\*AR(\d)(.*)', t)
    if not m or m.group(2) not in XMOD:
        raise ValueError("Xmem: %r" % txt)
    n = int(m.group(1))
    if not 2 <= n <= 5:
        raise ValueError("Xmem AR%d hors AR2..AR5" % n)
    return (XMOD[m.group(2)] << 2) | (n - 2)

def conds(tokens):
    v = 0
    for c in tokens:
        c = norm(c).upper()
        if c not in COND:
            raise ValueError("cond: %r" % c)
        v |= COND[c]
    return v

# ------------------------------------------------------------------ assembler ---
def split_operands(s):
    out, cur, depth = [], "", 0
    for ch in s:
        if ch == "(":
            depth += 1
        elif ch == ")":
            depth -= 1
        if ch == "," and depth == 0:
            out.append(cur.strip()); cur = ""
        else:
            cur += ch
    if cur.strip():
```

```
        out.append(cur.strip())
    return out

def est_acc(t):
    return norm(t).upper() in ("A", "B")

def assembler(tab, texte):
    """Returns list of words or raises ValueError."""
    texte = norm(texte.split(";")[0])
    m = re.match(r'([A-Za-z][A-Za-z0-9]*)\s*(.*)$', texte)
    if not m:
        raise ValueError("syntaxe")
    nom, reste = m.group(1).lower(), m.group(2)
    ops = split_operands(reste) if reste else []
    erreurs = []
    for (tnom, mots, opc, masque, types) in tab:
        if tnom != nom:
            continue
        try:
            return encoder(tnom, mots, opc, masque, types, list(ops))
        except ValueError as e:
            erreurs.append("%s%s: %s" % (tnom, types, e))
    raise ValueError("; ".join(erreurs) if erreurs else "mnemonique inconnu %s" % nom)

def encoder(nom, mots, opc, masque, types, ops):
    word = opc
    extra = []
    src = dst = None
    types = list(types)
    # optional operands are flagged OPT| in binutils; strip the flag but remember
    optional = [t.startswith("OPT|") for t in types]
    types = [t.replace("OPT|", "") for t in types]
    # implicit-operand types consume nothing from the text
    IMPLICIT = {"OP_None"}
    ti = 0
    for k, ty in enumerate(types):
        if ty in IMPLICIT:
            continue
        if ti >= len(ops):
            if optional[k]:
                continue
            raise ValueError("operande manquant pour %s" % ty)
        o = ops[ti]
        if ty in ("OP_Smem", "OP_Sind", "OP_Lmem", "OP_MMR"):
            if ty == "OP_MMR" and norm(o).upper() in MMR:
                word |= MMR[norm(o).upper()]
            else:
                sm = Smem(o)
                word |= sm.code
                if sm.lk is not None:
                    extra.append(sm.lk & 0xFFFF)
        elif ty == "OP_Xmem":
            word |= xmem(o) << 4
        elif ty == "OP_Ymem":
            word |= xmem(o)
        elif ty == "OP_SRC":
            if not est_acc(o): raise ValueError("src attendu, %r" % o)
            src = 1 if norm(o).upper() == "B" else 0
            word |= src << 9
        elif ty == "OP_SRC1":
            if not est_acc(o): raise ValueError("src attendu, %r" % o)
            word |= (1 if norm(o).upper() == "B" else 0) << 8
        elif ty == "OP_DST":
            if not est_acc(o): raise ValueError("dst attendu, %r" % o)
            dst = 1 if norm(o).upper() == "B" else 0
            word |= dst << 8
```

```
elif ty in ("OP_lk", "OP_lku", "OP_pmad", "OP_dmad", "OP_PA"):
    extra.append(nombre(o) & 0xFFFF)
elif ty == "OP_xpmad":
    v = nombre(o)
    word |= (v >> 16) & 0x7F
    extra.append(v & 0xFFFF)
elif ty in ("OP_k8u", "OP_k8"):
    word |= nombre(o) & 0xFF
elif ty == "OP_k9":
    word |= nombre(o) & 0x1FF
elif ty == "OP_k5":
    word |= nombre(o) & 0x1F
elif ty == "OP_k3":
    word |= nombre(o) & 0x7
elif ty == "OP_SHIFT":
    v = nombre(o)
    if not -16 <= v <= 15: raise ValueError("SHIFT hors -16..15")
    word |= v & 0x1F
elif ty == "OP_SHFT":
    v = nombre(o)
    if not 0 <= v <= 15: raise ValueError("SHFT hors 0..15")
    word |= v & 0xF
elif ty == "OP_031":
    v = nombre(o)
    if not 0 <= v <= 31: raise ValueError("0..31")
    word |= v & 0x1F
elif ty == "OP_16":
    if norm(o) != "16": raise ValueError("16 attendu")
elif ty in ("OP_T", "OP_TS", "OP_ASM", "OP_DP", "OP_ARP", "OP_TRN", "OP_A", "OP_B"):
    attendu = {"OP_T": "T", "OP_TS": "TS", "OP_ASM": "ASM", "OP_DP": "DP", "OP_ARP": "ARP",
               "OP_TRN": "TRN", "OP_A": "A", "OP_B": "B"}[ty]
    if norm(o).upper() != attendu: raise ValueError("%s attendu" % attendu)
elif ty == "OP_RND":
    raise ValueError("OP_RND non gere")
elif ty in ("OP_CC", "OP_CC3"):
    # remaining operands are all conditions
    word |= conds(ops[ti:])
    ti = len(ops)
    continue
elif ty == "OP_CC2":
    u = norm(o).upper()
    if nom in ("saccd", "srccd", "strcd"):  # 4-bit accumulator condition, bits 3-0
        if u not in COND or not (COND[u] & 0x40): raise ValueError("cond acc attendue")
        word |= COND[u] & 0x0F
    else:                                   # CMPR CC, ARx : cc in bits 9-8
        v = nombre(o) if not u.isalpha() else {"EQ": 0, "LT": 1, "GT": 2, "NEQ": 3}[u]
        word |= (v & 3) << 8
elif ty == "OP_ARX":
    m = re.fullmatch(r'AR(\d)', norm(o).upper())
    if not m: raise ValueError("ARx attendu")
    word |= int(m.group(1)) & 7
elif ty in ("OP_MMRX", "OP_MMRY"):       # AR0..AR7 = 0..7, SP = 8 ; X bits 7-4, Y bits 3-0
    u = norm(o).upper()
    m = re.fullmatch(r'AR(\d)', u)
    v = int(m.group(1)) if m else (8 if u == "SP" else None)
    if v is None: raise ValueError("ARx/SP attendu")
    word |= v << (4 if ty == "OP_MMRX" else 0)
elif ty == "OP_N":
    u = norm(o).upper()
    if u in SBITS:                          # RSBX SXM : N and SBIT from the name
        n, b = SBITS[u]; word |= (n << 9) | b
        ti += 1
        if ti < len(ops): raise ValueError("SBIT apres nom")
        return [word] + extra
    word |= (nombre(o) & 1) << 9
elif ty == "OP_SBIT":
```

```
            u = norm(o).upper()
            word |= (SBITS[u][1] if u in SBITS else nombre(o)) & 0xF
        elif ty == "OP_BITC":
            word |= nombre(o) & 0xF
        elif ty == "OP_123":
            word |= ((nombre(o) - 1) & 3) << 8
        elif ty == "OP_12":
            word |= ((nombre(o) - 1) & 1) << 9
        else:
            raise ValueError("type %s non gere" % ty)
        ti += 1
    if ti != len(ops):
        raise ValueError("operandes en trop: %r" % ops[ti:])
    # optional dst omitted: dst = src
    if "OP_DST" in types and dst is None and src is not None:
        word |= src << 8
    if len(extra) + 1 != mots and not (len(extra) + 1 == mots + 1):
        # the table's word count does not include the Smem lk word
        raise ValueError("longueur %d != %d" % (len(extra) + 1, mots))
    return [word & 0xFFFF] + extra

# ------------------------------------------------------------ example parsing ----
REG_OK = {"A", "B", "T", "TRN", "ASM", "C", "TC", "OVA", "OVB", "OVM", "SXM", "FRCT", "C16",
          "CMPT", "DP", "SP", "ARP", "BK", "BRC", "RSA", "REA", "PC", "XPC", "PMST", "ST0",
          "ST1", "IMR", "IFR", "INTM", "BRAF", "CPL", "XF", "HM", "AR0", "AR1", "AR2", "AR3",
          "AR4", "AR5", "AR6", "AR7", "TS"}
HEX = re.compile(r'^[0-9A-Fa-f]+$')

def valeur(tokens):
    """Value from the tokens following a register name. Returns (int, nbits)."""
    toks = [norm(t) for t in tokens]
    if len(toks) >= 3 and all(HEX.match(t) for t in toks[:3]) and len(toks[0]) == 2 \
            and len(toks[1]) == 4 and len(toks[2]) == 4:
        return int("".join(toks[:3]), 16), 40
    if toks and HEX.match(toks[0]) and len(toks[0]) == 4:
        return int(toks[0], 16), 16
    if toks and toks[0] in ("0", "1"):
        return int(toks[0]), 1
    if toks and toks[0].lower() == "x":
        return None, 1
    if toks and HEX.match(toks[0]) and len(toks[0]) in (1, 2, 3):
        return int(toks[0], 16), 16
    raise ValueError("valeur: %r" % toks)

def nettoyer_nom(t):
    return re.sub(r'[†‡‡*]+$', '', norm(t)).upper()

def parser(doc):
    L = io.open(doc, encoding="utf-8", errors="replace").read().split("\n")
    tests, section = [], "?"
    i = 0
    while i < len(L):
        l = L[i]
        m = re.match(r'^Syntax\s+(?:\d+:\s*)?(\S.*)$', l)
        if m:
            section = norm(m.group(1)); i += 1; continue
        m = re.match(r'^\s*(?:\d+:\s*)?Syntax\s+(\S.*)$', l)
        if m:
            section = norm(m.group(1)); i += 1; continue
        m = re.match(r'^Example(?:\s+\d+)?\s+([A-Z][A-Za-z0-9]*(?:\s.*)?)$', l)
        if not m:
            i += 1; continue
        instr = m.group(1).strip()
        t = {"section": section, "instr": instr, "before": {}, "after": {},
             "mem_b": {}, "mem_a": {}, "pmem_b": {}, "pmem_a": {}, "warn": []}
        zone = "reg"
```

```
        j = i + 1
        while j < len(L):
            lj = L[j]
            if re.match(r'^Example', lj) or re.match(r'^\s*(?:\d+:\s*)?Syntax\s', lj) \
                   or re.match(r'^Syntax', lj) or re.match(r'^[A-Z][a-z]+\s{2,}', lj) and "Instruction" not in lj:
                break
            if "Data Memory" in lj:
                zone = "mem"; j += 1; continue
            if "Program Memory" in lj:
                zone = "pmem"; j += 1; continue
            if "Before Instruction" in lj:
                j += 1; continue
            toks = lj.split()
            if not toks:
                j += 1; continue
            if zone in ("mem", "pmem"):
                mm = re.findall(r'([0-9A-Fa-f]+)h\s+([0-9A-Fa-f]{4})', lj)
                if len(mm) >= 1:
                    tgt_b = t["mem_b"] if zone == "mem" else t["pmem_b"]
                    tgt_a = t["mem_a"] if zone == "mem" else t["pmem_a"]
                    tgt_b[int(mm[0][0], 16)] = int(mm[0][1], 16)
                    if len(mm) >= 2:
                        tgt_a[int(mm[1][0], 16)] = int(mm[1][1], 16)
                    j += 1; continue
                # a register row can follow memory rows on some pages
            name = nettoyer_nom(toks[0])
            if name in REG_OK:
                # split at the second occurrence of the name
                idx = [k for k, tk in enumerate(toks) if nettoyer_nom(tk) == name]
                try:
                    if len(idx) >= 2:
                        vb, _ = valeur(toks[idx[0] + 1:idx[1]])
                        va, _ = valeur(toks[idx[1] + 1:])
                        if vb is not None: t["before"][name] = vb
                        if va is not None: t["after"][name] = va
                    else:
                        t["warn"].append("ligne registre incomplete: %s" % lj.strip())
                except ValueError as e:
                    t["warn"].append(str(e))
            j += 1
        tests.append(t)
        i = j
    return tests

def main():
    tab = charger_table(TABLE)
    tests = parser(DOC)
    n_ok = n_skip = 0
    for k, t in enumerate(tests):
        instr = t["instr"]
        if not t["before"] and not t["after"] and not t["mem_a"]:
            continue                                # prose caught by the regex
        # symbolic pmad/dmad (COEFFS, DAT127...) : take the first Program Memory address
        if t["pmem_b"]:
            instr = re.sub(r'\b(COEFFS|DAT\d+)\b', "%04Xh" % min(t["pmem_b"]), instr)
        try:
            words = assembler(tab, instr)
        except ValueError as e:
            # dual-operand examples written with AR6/AR7: only AR2..AR5 can be encoded.
            # Rename AR6->AR4 and AR7->AR5 in the instruction AND the register rows.
            ren = {}
            if "AR6" in instr and "AR4" not in instr and "AR4" not in t["before"]: ren["AR6"] = "AR4"
            if "AR7" in instr and "AR5" not in instr and "AR5" not in t["before"]: ren["AR7"] = "AR5"
            if ren:
                instr2 = instr
                for a, b in ren.items(): instr2 = instr2.replace(a, b)
```

```
                try:
                    words = assembler(tab, instr2)
                    for a, b in ren.items():
                        for d in (t["before"], t["after"]):
                            if a in d: d[b] = d.pop(a)
                    instr = instr2 + " ;(" + ",".join("%s->%s" % kv for kv in ren.items()) + ")"
                    e = None
                except ValueError as e2:
                    e = e2
            if e is not None:
                print("S %d %s | %s | %s" % (k, t["section"], instr, str(e)[:120]))
                n_skip += 1
                continue
        print("T %d %s | %s" % (k, t["section"], instr))
        print("W " + " ".join("%04x" % w for w in words))
        for r, v in sorted(t["before"].items()):
            print("B %s %x" % (r, v))
        for a, v in sorted(t["mem_b"].items()):
            print("B M %04x %04x" % (a, v))
        for a, v in sorted(t["pmem_b"].items()):
            print("B P %04x %04x" % (a, v))
        for r, v in sorted(t["after"].items()):
            print("A %s %x" % (r, v))
        for a, v in sorted(t["mem_a"].items()):
            print("A M %04x %04x" % (a, v))
        for w in t["warn"]:
            print("# %s" % w)
        print("E")
        n_ok += 1
    sys.stderr.write("%d exemples assembles, %d non assembles\n" % (n_ok, n_skip))

if __name__ == "__main__":
    main()
```

3.37 /opt/GSM/c54x_exe/tools/isa_test.c

11637 octets, 223 lignes → 223 lignes

```
/* SPDX-License-Identifier: GPL-2.0-or-later */
/*
 * isa_test.c - replay the SPRU172C worked examples against the C54x core.
 *
 * Reads tools/isa_tests.txt (written by tools/isa_examples.py): for every
 * example the assembled words, the state before and the state after. Each
 * test runs on a fresh C54xState: registers and memory are set, the words are
 * placed at PC, ONE c54x_run(s, 1) executes the instruction, and every
 * register/memory word the manual lists in "After Instruction" is compared.
 *
 *     make isa_test && ./isa_test tools/isa_tests.txt 2>/dev/null
 *     ./isa_test tools/isa_tests.txt -v 2>/dev/null      # print every test
 *
 * The verdict is a scorecard per instruction, not a debugging aid: a FAIL
 * names the register that differs and both values, then the core's handler is
 * read against the manual page. Registers the manual does not list are not
 * compared, so a handler that corrupts an unlisted register can still pass.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <stdbool.h>
#include "qemu/thread.h"
#include "calypso_c54x.h"
```

```
/* what main.c normally provides */
uint32_t g_c54x_exe_fn;
uint32_t calypso_trx_get_fn(void) { return g_c54x_exe_fn; }
void calypso_inth_arm_ack(void) { }

/* Program word at a 16-bit address as the core will fetch it: 0x8000-0xDFFF is
 * banked by XPC (c54x_prog_xlate), so write the word in every page; an example
 * that starts with XPC != 0 (FBACCD B) or changes XPC before its delay slots
 * otherwise fetches 0x0000. */
static void poser_prog(C54xState *s, uint16_t a, uint16_t w)
{
    s->prog[a] = w;
    if (a >= 0x8000 && a < 0xE000)
        for (uint32_t pg = 0; pg < (C54X_PROG_SIZE >> 16); pg++)
            s->prog[((pg << 16) | a) & (C54X_PROG_SIZE - 1)] = w;
}
QemuMutex calypso_pcb_daram_lock;
void cpu_physical_memory_rw(uint64_t addr, void *buf, uint64_t len, bool wr)
{ (void)addr; if (!wr) memset(buf, 0, len); }

extern uint16_t data_read(C54xState *s, uint16_t addr);
extern void     data_write(C54xState *s, uint16_t addr, uint16_t val);

#define ST0_TC_B  (1u << 12)
#define ST0_C_B   (1u << 11)
#define ST0_OVA_B (1u << 10)
#define ST0_OVB_B (1u << 9)

static struct { const char *nom; int reg; uint16_t bit; } BITS[] = {
    { "TC", 0, ST0_TC_B }, { "C", 0, ST0_C_B }, { "OVA", 0, ST0_OVA_B }, { "OVB", 0, ST0_OVB_B },
    { "BRAF", 1, 1u << 15 }, { "CPL", 1, 1u << 14 }, { "XF", 1, 1u << 13 }, { "HM", 1, 1u << 12 },
    { "INTM", 1, 1u << 11 }, { "OVM", 1, 1u << 9 }, { "SXM", 1, 1u << 8 }, { "C16", 1, 1u << 7 },
    { "FRCT", 1, 1u << 6 }, { "CMPT", 1, 1u << 5 },
};

static bool set_reg(C54xState *s, const char *r, uint64_t v)
{
    if (!strcmp(r, "A")) { s->a = (int64_t)(v & 0xFFFFFFFFFFULL); if (s->a & 0x8000000000LL) s->a |=
     ↪  ~0xFFFFFFFFFFLL; return true; }
    if (!strcmp(r, "B")) { s->b = (int64_t)(v & 0xFFFFFFFFFFULL); if (s->b & 0x8000000000LL) s->b |=
     ↪  ~0xFFFFFFFFFFLL; return true; }
    if (!strcmp(r, "T"))   { s->t = v; return true; }
    if (!strcmp(r, "TS"))  { s->t = v; return true; }
    if (!strcmp(r, "TRN")) { s->trn = v; return true; }
    if (!strcmp(r, "SP"))  { s->sp = v; return true; }
    if (!strcmp(r, "BK"))  { s->bk = v; return true; }
    if (!strcmp(r, "BRC")) { s->brc = v; return true; }
    if (!strcmp(r, "RSA")) { s->rsa = v; return true; }
    if (!strcmp(r, "REA")) { s->rea = v; return true; }
    if (!strcmp(r, "PC"))  { s->pc = v; return true; }
    if (!strcmp(r, "XPC")) { s->xpc = v; return true; }
    if (!strcmp(r, "PMST")) { s->pmst = v; return true; }
    if (!strcmp(r, "IPTR")) { s->pmst = (s->pmst & 0x7F) | ((v & 0x1FF) << 7); return true; }
    if (!strcmp(r, "ST0")) { s->st0 = v; return true; }
    if (!strcmp(r, "ST1")) { s->st1 = v; return true; }
    if (!strcmp(r, "IMR")) { s->imr = v; return true; }
    if (!strcmp(r, "IFR")) { s->ifr = v; return true; }
    if (!strcmp(r, "ASM")) { s->st1 = (s->st1 & ~0x1F) | (v & 0x1F); return true; }
    if (!strcmp(r, "DP"))  { s->st0 = (s->st0 & ~0x1FF) | (v & 0x1FF); return true; }
    if (!strcmp(r, "ARP")) { s->st0 = (s->st0 & ~0xE000) | ((v & 7) << 13); return true; }
    if (r[0] == 'A' && r[1] == 'R' && r[2] >= '0' && r[2] <= '7' && !r[3]) { s->ar[r[2] - '0'] = v; return true; }
    for (unsigned i = 0; i < sizeof BITS / sizeof BITS[0]; i++)
        if (!strcmp(r, BITS[i].nom)) {
            uint16_t *st = BITS[i].reg ? &s->st1 : &s->st0;
            if (v) *st |= BITS[i].bit; else *st &= ~BITS[i].bit;
            return true;
```

```
        }
    return false;
}

static bool get_reg(C54xState *s, const char *r, uint64_t *v)
{
    if (!strcmp(r, "A")) { *v = (uint64_t)s->a & 0xFFFFFFFFFFULL; return true; }
    if (!strcmp(r, "B")) { *v = (uint64_t)s->b & 0xFFFFFFFFFFULL; return true; }
    if (!strcmp(r, "T") || !strcmp(r, "TS")) { *v = s->t; return true; }
    if (!strcmp(r, "TRN")) { *v = s->trn; return true; }
    if (!strcmp(r, "SP"))  { *v = s->sp; return true; }
    if (!strcmp(r, "BK"))  { *v = s->bk; return true; }
    if (!strcmp(r, "BRC")) { *v = s->brc; return true; }
    if (!strcmp(r, "RSA")) { *v = s->rsa; return true; }
    if (!strcmp(r, "REA")) { *v = s->rea; return true; }
    if (!strcmp(r, "PC"))  { *v = s->pc & 0xFFFF; return true; }
    if (!strcmp(r, "XPC")) { *v = s->xpc; return true; }
    if (!strcmp(r, "PMST")) { *v = s->pmst; return true; }
    if (!strcmp(r, "IPTR")) { *v = (s->pmst >> 7) & 0x1FF; return true; }
    if (!strcmp(r, "ST0")) { *v = s->st0; return true; }
    if (!strcmp(r, "ST1")) { *v = s->st1; return true; }
    if (!strcmp(r, "IMR")) { *v = s->imr; return true; }
    if (!strcmp(r, "IFR")) { *v = s->ifr; return true; }
    if (!strcmp(r, "ASM")) { *v = s->st1 & 0x1F; return true; }
    if (!strcmp(r, "DP"))  { *v = s->st0 & 0x1FF; return true; }
    if (!strcmp(r, "ARP")) { *v = (s->st0 >> 13) & 7; return true; }
    if (r[0] == 'A' && r[1] == 'R' && r[2] >= '0' && r[2] <= '7' && !r[3]) { *v = s->ar[r[2] - '0']; return true; }
    for (unsigned i = 0; i < sizeof BITS / sizeof BITS[0]; i++)
        if (!strcmp(r, BITS[i].nom)) {
            uint16_t st = BITS[i].reg ? s->st1 : s->st0;
            *v = (st & BITS[i].bit) ? 1 : 0;
            return true;
        }
    return false;
}

typedef struct { char reg[8]; uint64_t val; int mem; uint16_t addr; } Item;

int main(int argc, char **argv)
{
    if (argc < 2) { fprintf(stderr, "usage: %s tools/isa_tests.txt [-v] [-k MOT]\n", argv[0]); return 2; }
    bool verbeux = false; const char *filtre = NULL;
    for (int i = 2; i < argc; i++) {
        if (!strcmp(argv[i], "-v")) verbeux = true;
        else if (!strcmp(argv[i], "-k") && i + 1 < argc) filtre = argv[++i];
    }
    FILE *f = fopen(argv[1], "r");
    if (!f) { perror(argv[1]); return 2; }
    qemu_mutex_init(&calypso_pcb_daram_lock);

    char ligne[1024], titre[1024] = "";
    uint16_t mots[8]; int nmots = 0, npas = 1;
    Item avant[64], apres[64]; int na = 0, np = 0;
    int total = 0, ok = 0, ko = 0, ignores = 0;
    char bilan_ko[65536] = "";

    while (fgets(ligne, sizeof ligne, f)) {
        ligne[strcspn(ligne, "\n")] = 0;
        if (ligne[0] == 'T') {
            snprintf(titre, sizeof titre, "%s", ligne + 2);
            nmots = na = np = 0; npas = 1;
        } else if (ligne[0] == 'W') {
            char *p = ligne + 2; nmots = 0;
            while (*p && nmots < 8) { mots[nmots++] = (uint16_t)strtoul(p, &p, 16); while (*p == ' ') p++; }
        } else if (ligne[0] == 'N') {
            /* N <n>: run n instructions (multi-instruction examples, delay slots) */
```

```
            npas = atoi(ligne + 2); if (npas < 1) npas = 1;
        } else if (ligne[0] == 'B' || ligne[0] == 'A') {
            Item *it = ligne[0] == 'B' ? &avant[na] : &apres[np];
            int *n = ligne[0] == 'B' ? &na : &np;
            if (*n >= 64) continue;
            char k[8], r[8]; unsigned long long a, v;
            if (sscanf(ligne + 2, "%7s %llx %llx", k, &a, &v) == 3 && (k[0] == 'M' || k[0] == 'P') && !k[1]) {
                it->mem = k[0] == 'M' ? 1 : 2; it->addr = (uint16_t)a; it->val = v; it->reg[0] = 0; (*n)++;
            } else if (sscanf(ligne + 2, "%7s %llx", r, &v) == 2) {
                it->mem = 0; snprintf(it->reg, sizeof it->reg, "%s", r); it->val = v; (*n)++;
            }
        } else if (ligne[0] == 'E') {
            if (filtre && !strstr(titre, filtre)) continue;
            total++;
            C54xState *s = c54x_init();
            s->running = true;
            s->pc = 0x9000;
            s->pmst = 0x1FFu << 7;   /* IPTR = 1FFh, its reset value, assumed by the manual's vector examples */
            bool prob = false;
            for (int i = 0; i < na; i++) {
                if (avant[i].mem == 1) { if (avant[i].addr < 0x60) data_write(s, avant[i].addr,
                 ↪  (uint16_t)avant[i].val); else s->data[avant[i].addr] = (uint16_t)avant[i].val; }
                else if (avant[i].mem == 2) s->prog[avant[i].addr] = (uint16_t)avant[i].val;
                else if (!set_reg(s, avant[i].reg, avant[i].val)) prob = true;
            }
            uint16_t pc0 = (uint16_t)s->pc;
            for (int i = 0; i < nmots; i++) poser_prog(s, (uint16_t)(pc0 + i), mots[i]);
            /* the instruction after, so a delayed branch has something to execute */
            poser_prog(s, (uint16_t)(pc0 + nmots), 0xF495); poser_prog(s, (uint16_t)(pc0 + nmots + 1), 0xF495);
            for (int k = 0; k < npas; k++) c54x_run(s, 1);
            /* RPT arms a counter and returns 0 executed words: run the repeated
             * instruction as well so the example's "After" is observable. */
            { int _g = 70000; while (s->rpt_active && _g--) c54x_run(s, 1); }
            for (int k = 0; k < 4 && s->delay_slots; k++) c54x_run(s, 1);
            char detail[2048] = ""; int d = 0; bool echec = false;
            for (int i = 0; i < np; i++) {
                uint64_t got;
                if (apres[i].mem == 1) got = apres[i].addr < 0x60 ? data_read(s, apres[i].addr) :
                 ↪  s->data[apres[i].addr];
                else if (apres[i].mem == 2) got = s->prog[apres[i].addr];
                else if (!get_reg(s, apres[i].reg, &got)) { prob = true; continue; }
                uint64_t att = apres[i].val;
                if (!apres[i].mem && !strcmp(apres[i].reg, "XPC")) { att &= 3; got &= 3; }
                if (got != att) {
                    echec = true;
                    if (apres[i].mem) d += snprintf(detail + d, sizeof detail - d, "  %s[%04x]: attendu %04llx,
                     ↪  obtenu %04llx",
                                                    apres[i].mem == 1 ? "data" : "prog", apres[i].addr,
                                                    (unsigned long long)apres[i].val, (unsigned long long)got);
                    else d += snprintf(detail + d, sizeof detail - d, "  %s: attendu %llx, obtenu %llx",
                                       apres[i].reg, (unsigned long long)apres[i].val, (unsigned long long)got);
                }
            }
            if (echec) {
                ko++;
                printf("FAIL  %-60.60s  [%04x%s%04x%s]%s\n", titre, mots[0], nmots > 1 ? " " : "", nmots > 1 ?
↪  mots[1] : 0, prob ? " ?" : "", detail);
                size_t l = strlen(bilan_ko);
                snprintf(bilan_ko + l, sizeof bilan_ko - l, "  %s\n", titre);
            } else {
                ok++;
                if (verbeux) printf("ok    %-60.60s  [%04x]\n", titre, mots[0]);
            }
            free(s);
        } else if (ligne[0] == 'S') {
            ignores++;
```

```
        }
    }
    printf("\n%d exemples : %d ok, %d FAIL, %d non assembles (S)\n", total, ok, ko, ignores);
    return ko ? 1 : 0;
}
```

3.38 /opt/GSM/c54x_exe/tools/layer1_tester.c

28335 octets, 599 lignes → 599 lignes

```
/* layer1_tester.c - who reads, who writes each word of the DSP data memory
 * (measurement part of tools/layer1_tester.py; "api_carte" is its alias).
 *
 * Replays a dedicated-channel recording (/dev/shm/calypso_rejeu_tch.bin,
 * format of tools/rejeu_banc.c: 'S' 'D' 'T' 'A' 'B') on the qosmo C54x core
 * and the TI ROM, in the order of pont.c, and COUNTS for each of the 65536
 * data words:
 *   - the reads and writes done by a ROM instruction (first PC, first tick,
 *     up to 6 distinct PCs, typical values);
 *   - the ARM writes ('A' records: words modified on the QEMU side between
 *     two frames, hence MODIFICATIONS, not every write) and the value of the
 *     starting state ('S' for the API RAM, 'D' for the rest);
 *   - the writes that do not go through an instruction: DMA, I/Q delivery by
 *     the BSP, interrupt stacking, core crutches writing s->data[] directly
 *     (difference of snapshots before/after each step).
 *
 * THE HOOK. qosmo is not modified: data_read()/data_write() of c54x_mem.c
 * call calypso_mbx() (mailbox monitor) on every access when c54x_rapide == 0,
 * with the address, the value and the PC. The core is linked WITHOUT
 * calypso_mailbox.c and calypso_mbx_actif/_init/_evt are provided here: every
 * DSP access to data memory lands in compter(). Accesses made outside an
 * instruction (DMA, BSP...) are separated by g_dans_dsp. A direct s->data[]
 * write (crutch, DMA) escapes the hook: the snapshot difference catches it
 * (hc_* column). A direct s->data[] READ by the core escapes both: known
 * limit.
 *
 * I/O PORTS. PORTR/PORTW (c54x_exec.c) first call calypso_a5_portr/_portw for
 * EVERY port: wrappers (--wrap) to count port, direction, PC, written value
 * ("io:xxxx" lines of the output). Only the 0x74xx/0x75xx instructions go
 * through there; the other PORTW encoding (0x9Fxx) is only logged by the core
 * and is not counted.
 *
 * ACTIVE PCs (-p). Number of memory accesses per PC (xpc:pc): tells whether a
 * known ROM routine ran during the replay (a routine without memory access
 * does not show up there).
 *
 * NO SIDE EFFECT. The core sources open /dev/shm files and a UDP socket
 * (calypso_bsp.c): the link wraps fopen/open/socket (-Wl,--wrap=...); every
 * open for writing goes to /dev/null, socket() fails. Only the output file
 * (-o) is written. Nothing is launched, no bench process is touched.
 *
 * BUDGET LOOP. c54x_run yields by itself every CALYPSO_DSP_YIELD instructions
 * (32768): courir() loops up to the budget, like pont.c and rejeu_banc.c,
 * otherwise the DSP does not finish its frame.
 *
 * CONSUMPTION. For each word of the API RAM, an ARM modification ('A') is
 * "pending" until a ROM instruction reads the word (consumed, latency in
 * ticks) or the ARM modifies it again (overwritten without a read).
 *
 *   make layer1_tester          (or make api_carte, alias)
 *   tools/layer1_tester [-o sortie.tsv] [-j journal.tsv] [-n ticks_max] [fichier]
 *   python3 tools/layer1_tester.py --rejeu fichier   (runs this binary, judges, builds the map)
 *
 * Output (-o): TSV, one line per touched word (see the header), '#' lines = summary.
```

```
 * Journal (-j): per tick, each API RAM word read or written by the ROM:
 *   tick, word (DSP address), reads, last value read, writes,
 *   last value written, PC of the last write.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <stdbool.h>
#include <stdarg.h>
#include <fcntl.h>
#include <errno.h>
#include <time.h>
#include <sys/socket.h>
#include "qemu/thread.h"
#include "calypso_c54x.h"
#include "calypso_dma.h"
#include "calypso_bsp.h"
#include "calypso_rhea_dma.h"
#include "calypso_mailbox.h"
#include "hw/arm/calypso/calypso_api.h"

/* what main.c usually provides (same set as tools/rejeu_banc.c) */
uint32_t g_c54x_exe_fn;
uint32_t calypso_trx_get_fn(void) { return g_c54x_exe_fn; }
void calypso_inth_arm_ack(void) { }
QemuMutex calypso_pcb_daram_lock;
void cpu_physical_memory_rw(uint64_t addr, void *buf, uint64_t len, bool wr)
{ (void)addr; if (!wr) memset(buf, 0, len); }

/* ---- guards: no file write, no socket ----------------------------------- */
FILE *__real_fopen(const char *path, const char *mode);
int __real_open(const char *path, int flags, ...);
static int g_fichiers_detournes;
FILE *__wrap_fopen(const char *path, const char *mode)
{
    if (strpbrk(mode, "wa+")) { g_fichiers_detournes++; return __real_fopen("/dev/null", "w"); }
    return __real_fopen(path, mode);
}
int __wrap_open(const char *path, int flags, ...)
{
    if (flags & (O_WRONLY | O_RDWR | O_CREAT | O_TRUNC | O_APPEND)) {
        g_fichiers_detournes++;
        return __real_open("/dev/null", O_WRONLY);
    }
    return __real_open(path, flags);
}
int __wrap_socket(int domain, int type, int protocol)
{ (void)domain; (void)type; (void)protocol; errno = EACCES; return -1; }
/* I/O ports: see io_noter() below */
static void io_noter(int ecrit, uint16_t pa, uint16_t val);
bool __real_calypso_a5_portw(C54xState *s, uint16_t pa, uint16_t val);
bool __real_calypso_a5_portr(C54xState *s, uint16_t pa, uint16_t *val);
bool __wrap_calypso_a5_portw(C54xState *s, uint16_t pa, uint16_t val)
{ io_noter(1, pa, val); return __real_calypso_a5_portw(s, pa, val); }
bool __wrap_calypso_a5_portr(C54xState *s, uint16_t pa, uint16_t *val)
{
    bool r = __real_calypso_a5_portr(s, pa, val);
    io_noter(0, pa, r ? *val : 0);
    return r;
}
ssize_t __wrap_sendto(int fd, const void *b, size_t n, int fl, const struct sockaddr *a, socklen_t l)
{ (void)fd; (void)b; (void)fl; (void)a; (void)l; return (ssize_t)n; }

/* ---- state -------------------------------------------------------------- */
#define API_WORDS 0x2000u                 /* API window as seen by the DSP: data 0x0800..0x27ff */
```

```
#define ENREG_API_WORDS CALYPSO_API_WORDS  /* size of the API RAM recorded by pont.c ('S') */
#define NDB 0xD4u
#define BL_ADDR_HI_W 0x7FCu
#define BL_SIZE_W 0x7FDu
#define BL_ADDR_LO_W 0x7FEu
#define BL_STATUS_W 0x7FFu
#define NPC 6                              /* distinct PCs kept per word and per direction */
#define NVAL 4                             /* Misra-Gries: exact if <= 4 distinct values */

typedef struct {
    int64_t a, b; uint16_t ar[8], t, trn, sp, bk, brc, rsa, rea, st0, st1, pmst, imr, ifr, xpc;
    uint32_t pc; uint8_t idle, running;
} EnregRegs;                               /* = pont.c enreg_base() */

typedef struct { uint16_t v[NVAL]; uint32_t n[NVAL]; uint8_t plus; } Valeurs;
typedef struct {
    uint64_t n;                            /* accesses */
    uint32_t pc1, tick1;                   /* first access */
    uint32_t pcs[NPC]; uint8_t npc, pcplus;
    Valeurs val;
    uint16_t dernier;
} Sens;

enum { SEG_BOOT = 0, SEG_DSP, SEG_IRQ, SEG_DMA, SEG_BSP, SEG_POMPE, SEG_N };
static const char *SEG_NOM[SEG_N] = { "boot", "dsp", "irq", "dma", "bsp", "pompe" };

static C54xState *dsp;
static uint16_t *api;
static uint8_t *fichier;
static size_t taille;
static int g_dans_dsp;                     /* 1 during c54x_run */
static int g_seg = SEG_BOOT;
static int g_phase_rejeu;                  /* 0 = local boot, 1 = replay */

static Sens *s_rd, *s_wr, *s_arm;          /* [65536] */
static uint64_t *boot_rd, *boot_wr;        /* accesses during the local boot */
static uint64_t *hors_wr;                  /* writes through the hook outside an instruction */
static uint8_t *hors_src;
static uint64_t *hc_n;                     /* writes outside the hook (snapshots) */
static uint32_t *hc_tick1; static uint8_t *hc_src; static uint16_t *hc_der;
static uint16_t *val_s, *val_d; static uint8_t *a_s, *a_d;
static uint16_t *snap; static uint32_t *gen; static uint32_t g_gen = 1;
static uint64_t g_n_evt, g_n_rd, g_n_wr;
static Sens *io_rd, *io_wr;                /* [65536] ports */
static uint32_t *pc_acc;                   /* [0x40000] memory accesses per xpc:pc */
/* consumption of the ARM modifications (API RAM words) */
static uint8_t *arm_pend; static uint32_t *arm_pend_tick;
static uint64_t *conso_n, *conso_lat, *ecrase_n;
/* per-tick journal (API RAM words) */
static FILE *g_journal;
static uint32_t *j_r, *j_w; static uint16_t *j_vr, *j_vw; static uint32_t *j_pcw;
static uint16_t j_liste[0x2000]; static unsigned j_n;
static inline void j_toucher(uint16_t mot)
{
    unsigned i = mot - 0x800u;
    if (!j_r[i] && !j_w[i]) j_liste[j_n++] = (uint16_t)i;
}

static inline uint32_t pc_complet(void)
{
    uint32_t pc = dsp->pc & 0xffff;
    if (pc >= 0x8000 && dsp->xpc) pc |= (uint32_t)dsp->xpc << 16;
    return pc;
}

static inline void valeur(Valeurs *v, uint16_t x)
```

```
{
    for (int i = 0; i < NVAL; i++) if (v->n[i] && v->v[i] == x) { v->n[i]++; return; }
    for (int i = 0; i < NVAL; i++) if (!v->n[i]) { v->v[i] = x; v->n[i] = 1; return; }
    v->plus = 1;
    for (int i = 0; i < NVAL; i++) v->n[i]--;
}

static inline void noter(Sens *s, uint32_t pc, uint16_t v)
{
    if (!s->n) { s->pc1 = pc; s->tick1 = g_c54x_exe_fn; }
    s->n++;
    s->dernier = v;
    valeur(&s->val, v);
    if (pc != 0xffffffffu) {
        for (int i = 0; i < s->npc; i++) if (s->pcs[i] == pc) goto vu;
        if (s->npc < NPC) s->pcs[s->npc++] = pc; else s->pcplus = 1;
    vu:;
    }
}

static void io_noter(int ecrit, uint16_t pa, uint16_t val)
{
    if (!g_phase_rejeu || !io_rd) return;
    noter(ecrit ? &io_wr[pa] : &io_rd[pa], pc_complet(), val);
}

/* The hook: replaces calypso_mailbox.c (only exported symbols). */
int calypso_mbx_actif = 1;
void calypso_mbx_init(void) { }
void calypso_mbx_evt(CalypsoMbxSens sens, uint16_t mot, uint16_t val, uint16_t avant,
                     uint32_t ctx, uint32_t fn, uint32_t insn)
{
    (void)avant; (void)ctx; (void)fn; (void)insn;
    g_n_evt++;
    if (g_phase_rejeu && g_dans_dsp) pc_acc[pc_complet() & 0x3ffff]++;
    if (sens == MBX_DSP_RD) {
        g_n_rd++;
        if (!g_phase_rejeu) { boot_rd[mot]++; return; }
        if (g_dans_dsp) {
            noter(&s_rd[mot], pc_complet(), val);
            if (mot >= 0x800 && mot < 0x2800) {
                if (arm_pend[mot]) { arm_pend[mot] = 0; conso_n[mot]++; conso_lat[mot] += g_c54x_exe_fn -
                 ↪  arm_pend_tick[mot]; }
                if (g_journal) { j_toucher(mot); j_r[mot - 0x800]++; j_vr[mot - 0x800] = val; }
            }
        }
    } else if (sens == MBX_DSP_WR) {
        g_n_wr++;
        gen[mot] = g_gen;
        if (!g_phase_rejeu) { boot_wr[mot]++; return; }
        if (g_dans_dsp) {
            noter(&s_wr[mot], pc_complet(), val);
            if (g_journal && mot >= 0x800 && mot < 0x2800) {
                j_toucher(mot); j_w[mot - 0x800]++; j_vw[mot - 0x800] = val; j_pcw[mot - 0x800] = pc_complet(); }
        }
        else { hors_wr[mot]++; hors_src[mot] |= (uint8_t)(1u << g_seg); }
    }
}

/* Writes that did not go through the hook since the last snapshot. */
static void instantane(int seg)
{
    const uint64_t *a = (const uint64_t *)dsp->data, *b = (const uint64_t *)snap;
    for (unsigned q = 0; q < C54X_DATA_SIZE / 4; q++) {
        if (a[q] == b[q]) continue;
        for (unsigned k = 0; k < 4; k++) {
```

```
            unsigned m = q * 4 + k;
            if (dsp->data[m] == snap[m]) continue;
            if (m >= 0x60 && gen[m] != g_gen && g_phase_rejeu) {   /* MMR/peripherals excluded */
                if (!hc_n[m]) hc_tick1[m] = g_c54x_exe_fn;
                hc_n[m]++; hc_src[m] |= (uint8_t)(1u << seg); hc_der[m] = dsp->data[m];
            }
            snap[m] = dsp->data[m];
        }
    }
    g_gen++;
}

static void synchro_instantane(void) { memcpy(snap, dsp->data, C54X_DATA_SIZE * 2); g_gen++; }

static uint32_t be32(const uint8_t *p) { return (uint32_t)p[0] << 24 | p[1] << 16 | p[2] << 8 | p[3]; }
static uint16_t be16(const uint8_t *p) { return (uint16_t)(p[0] << 8 | p[1]); }

static long courir(long n)
{
    long k = 0;
    g_dans_dsp = 1;
    while (k < n && dsp->running && !dsp->idle) {
        int r = c54x_run(dsp, (int)(n - k));
        if (r <= 0) break;
        k += r;
    }
    g_dans_dsp = 0;
    instantane(SEG_DSP);
    return k;
}

static void reveil(void)
{
    g_seg = SEG_IRQ;
    if (dsp->idle && calypso_rhea_dma_irq_level() && (dsp->imr & (1u << 14)) && !(dsp->ifr & (1u << 14)))
        c54x_interrupt_ex(dsp, 30, 14);
    if (dsp->idle && (dsp->ifr & dsp->imr) && !(dsp->st1 & 0x800)) dsp->idle = false;
    instantane(SEG_IRQ);
}

static void boot(void)
{
    memset(api, 0, API_WORDS * 2);
    long b = 0;
    g_dans_dsp = 1;
    while (api[BL_STATUS_W] != 1 && b < 8000000) { int ex = c54x_run(dsp, 256); if (ex <= 0) break; b += ex; }
    api[BL_ADDR_HI_W] = 0; api[BL_ADDR_LO_W] = 0x7000; api[BL_SIZE_W] = 0; api[BL_STATUS_W] = 2;
    while (b < 12000000 && dsp->running) { int ex = c54x_run(dsp, 256); if (ex <= 0) break; b += ex; if
     ↪  (dsp->idle) break; }
    g_dans_dsp = 0;
    printf("# boot local : %ld insn, idle=%d\n", b, dsp->idle);
}

static void ecrire_arm(unsigned a, uint16_t v)
{
    if (a >= API_WORDS) return;
    api[a] = v;
    snap[0x800 + a] = v;
    noter(&s_arm[0x800 + a], 0xffffffffu, v);
    if (arm_pend[0x800 + a]) ecrase_n[0x800 + a]++;
    arm_pend[0x800 + a] = 1; arm_pend_tick[0x800 + a] = g_c54x_exe_fn;
}

static void pc_txt(char *o, uint32_t pc)
{
    if (pc > 0xffff) sprintf(o, "%x:%04x", pc >> 16, pc & 0xffff); else sprintf(o, "%04x", pc);
```

```
}

static void sens_txt(FILE *f, const Sens *s)
{
    char b[16];
    fprintf(f, "\t%llu", (unsigned long long)s->n);
    if (!s->n) { fprintf(f, "\t\t\t\t\t"); return; }
    if (s->pc1 != 0xffffffffu) { pc_txt(b, s->pc1); fprintf(f, "\t%s", b); } else fprintf(f, "\t");
    fprintf(f, "\t%u\t", s->tick1);
    for (int i = 0; i < s->npc; i++) { pc_txt(b, s->pcs[i]); fprintf(f, "%s%s", i ? "," : "", b); }
    if (s->pcplus) fprintf(f, ",+");
    fprintf(f, "\t");
    /* values, from the most frequent to the least frequent */
    int ord[NVAL]; for (int i = 0; i < NVAL; i++) ord[i] = i;
    for (int i = 0; i < NVAL; i++) for (int j = i + 1; j < NVAL; j++)
        if (s->val.n[ord[j]] > s->val.n[ord[i]]) { int t = ord[i]; ord[i] = ord[j]; ord[j] = t; }
    int prem = 1;
    for (int i = 0; i < NVAL; i++) if (s->val.n[ord[i]]) {
        fprintf(f, "%s%04x:%u", prem ? "" : ",", s->val.v[ord[i]], s->val.n[ord[i]]); prem = 0; }
    if (s->val.plus) fprintf(f, ",+");
    fprintf(f, "\t%04x", s->dernier);
}

static void src_txt(FILE *f, uint8_t m)
{
    int prem = 1;
    for (int i = 0; i < SEG_N; i++) if (m & (1u << i)) { fprintf(f, "%s%s", prem ? "" : ",", SEG_NOM[i]); prem =
    ↪  0; }
}

typedef struct { size_t debut, fin; uint32_t tick; uint8_t drap; long budget; } Tick;

static void *alloue(size_t n) { void *p = calloc(1, n); if (!p) { perror("calloc"); exit(1); } return p; }

int main(int argc, char **argv)
{
    const char *nom = "/dev/shm/calypso_rejeu_tch.bin", *sortie = NULL, *f_pcs = NULL;
    long max_ticks = 1000000;
    for (int i = 1; i < argc; i++) {
        if (!strcmp(argv[i], "-o") && i + 1 < argc) sortie = argv[++i];
        else if (!strcmp(argv[i], "-p") && i + 1 < argc) f_pcs = argv[++i];
        else if (!strcmp(argv[i], "-j") && i + 1 < argc) {
            g_journal = __real_fopen(argv[++i], "w");
            if (!g_journal) { perror(argv[i]); return 1; }
        }
        else if (!strcmp(argv[i], "-n") && i + 1 < argc) max_ticks = atol(argv[++i]);
        else if (argv[i][0] == '-') { fprintf(stderr, "usage : %s [-o sortie.tsv] [-j journal.tsv] [-p pcs.tsv]
        ↪  [-n ticks] [fichier]\n", argv[0]); return 2; }
        else nom = argv[i];
    }
    FILE *f = __real_fopen(nom, "rb");
    if (!f) { perror(nom); return 1; }
    fseek(f, 0, SEEK_END); taille = (size_t)ftell(f); fseek(f, 0, SEEK_SET);
    fichier = alloue(taille);
    if (fread(fichier, 1, taille, f) != taille) { fprintf(stderr, "lecture courte\n"); return 1; }
    fclose(f);
    FILE *o = sortie ? __real_fopen(sortie, "w") : stdout;
    if (!o) { perror(sortie); return 1; }

    s_rd = alloue(65536 * sizeof(Sens)); s_wr = alloue(65536 * sizeof(Sens)); s_arm = alloue(65536 * sizeof(Sens));
    boot_rd = alloue(65536 * 8); boot_wr = alloue(65536 * 8); hors_wr = alloue(65536 * 8); hors_src =
↪  alloue(65536);
    hc_n = alloue(65536 * 8); hc_tick1 = alloue(65536 * 4); hc_src = alloue(65536); hc_der = alloue(65536 * 2);
    val_s = alloue(65536 * 2); val_d = alloue(65536 * 2); a_s = alloue(65536); a_d = alloue(65536);
    snap = alloue(65536 * 2); gen = alloue(65536 * 4);
    io_rd = alloue(65536 * sizeof(Sens)); io_wr = alloue(65536 * sizeof(Sens)); pc_acc = alloue(0x40000 * 4);
```

```
arm_pend = alloue(65536); arm_pend_tick = alloue(65536 * 4);
conso_n = alloue(65536 * 8); conso_lat = alloue(65536 * 8); ecrase_n = alloue(65536 * 8);
j_r = alloue(0x2000 * 4); j_w = alloue(0x2000 * 4); j_vr = alloue(0x2000 * 2); j_vw = alloue(0x2000 * 2);
j_pcw = alloue(0x2000 * 4);
if (g_journal) fprintf(g_journal, "tick\tmot\tr\tval_r\tw\tval_w\tpc_w\n");

/* same environment as tools/rejeu_banc.c; the /dev/shm writes and the
 * socket are neutralised anyway by the wrappers above */
setenv("CALYPSO_BSP_PORT", "16703", 1);
setenv("CALYPSO_BSP_BIND_ADDR", "127.0.0.1", 1);
setenv("CALYPSO_BSP_HORLOGE", "0", 1);
setenv("CALYPSO_C54X_IRQ_LEVEL", "1", 0);
setenv("CALYPSO_BSP_STREAM", "1", 0);
setenv("CALYPSO_RHEA_DMA_XFER", "1", 0);
setenv("CALYPSO_REJEU_ENREG", "0", 1);
qemu_mutex_init(&calypso_pcb_daram_lock);
dsp = c54x_init();
api = &dsp->data[0x800];
c54x_set_api_ram(dsp, api);
static const struct { const char *s; uint32_t a; bool p; } R[] = {
    { "PROM0", 0x07000, true }, { "PROM1", 0x18000, true }, { "PROM2", 0x28000, true },
    { "PROM3", 0x38000, true }, { "DROM", 0x09000, false }, { "PDROM", 0x0E000, false }, { "PDROM", 0x0E000,
    ↪  true } };
const char *rom = getenv("API_CARTE_ROM") ? getenv("API_CARTE_ROM") : "/opt/GSM";
for (unsigned i = 0; i < 7; i++) {
    char c[512]; snprintf(c, sizeof c, "%s/calypso_dsp.%s.bin", rom, R[i].s);
    if (c54x_load_section(dsp, c, R[i].a, R[i].p) < 0) { fprintf(stderr, "ROM %s\n", c); return 1; }
}
{ char c[512]; snprintf(c, sizeof c, "%s/calypso_dsp.Registers.bin", rom); c54x_load_registers(dsp, c); }
c54x_reset(dsp);
calypso_dma_init();
calypso_bsp_init(dsp);
struct timespec t0; clock_gettime(CLOCK_MONOTONIC, &t0);
boot();
g_phase_rejeu = 1;

/* First pass: starting state and split into ticks. */
Tick *ticks = alloue(sizeof(Tick) * 200000); int nt = 0;
size_t p = 0;
while (p < taille) {
    uint8_t k = fichier[p];
    size_t l;
    switch (k) {
    case 'S': l = 5 + (size_t)ENREG_API_WORDS * 2; break;
    case 'D': l = 5 + sizeof(EnregRegs) + C54X_DATA_SIZE * 2; break;
    case 'T': l = 10; break;
    case 'A': l = 8 + 4 * (size_t)be16(fichier + p + 6); break;
    case 'B': l = 15 + 2 * (size_t)be16(fichier + p + 13); break;
    default: fprintf(stderr, "enregistrement inconnu 0x%02x a %zu\n", k, p); goto fini;
    }
    if (p + l > taille) break;
    if (k == 'S') {
        memcpy(api, fichier + p + 5, API_WORDS * 2);
        for (unsigned i = 0; i < API_WORDS; i++) { val_s[0x800 + i] = api[i]; a_s[0x800 + i] = 1; }
        printf("# etat S pose (tick %u)\n", be32(fichier + p + 1));
    } else if (k == 'D') {
        EnregRegs r; memcpy(&r, fichier + p + 5, sizeof r);
        uint16_t *sauve = alloue(API_WORDS * 2); memcpy(sauve, api, API_WORDS * 2);
        memcpy(dsp->data, fichier + p + 5 + sizeof r, C54X_DATA_SIZE * 2);
        for (unsigned i = 0; i < C54X_DATA_SIZE; i++) { val_d[i] = dsp->data[i]; a_d[i] = 1; }
        memcpy(api, sauve, API_WORDS * 2); free(sauve);   /* 'S' is authoritative for the API RAM */
        dsp->a = r.a; dsp->b = r.b; memcpy(dsp->ar, r.ar, sizeof r.ar); dsp->t = r.t; dsp->trn = r.trn;
        dsp->sp = r.sp; dsp->bk = r.bk; dsp->brc = r.brc; dsp->rsa = r.rsa; dsp->rea = r.rea;
        dsp->st0 = r.st0; dsp->st1 = r.st1; dsp->pmst = r.pmst; dsp->imr = r.imr; dsp->ifr = r.ifr;
        dsp->xpc = r.xpc; dsp->pc = r.pc; dsp->idle = r.idle; dsp->running = r.running;
        printf("# etat D pose : pc=%04x sp=%04x idle=%d imr=%04x\n", r.pc & 0xffff, r.sp, r.idle, r.imr);
```

```
        } else if (k == 'T') {
            if (nt > 0) ticks[nt - 1].fin = p;
            if (nt < 200000) {
                ticks[nt].debut = p; ticks[nt].tick = be32(fichier + p + 1);
                ticks[nt].drap = fichier[p + 5]; ticks[nt].budget = (long)be32(fichier + p + 6);
                nt++;
            }
        }
        p += l;
    }
fini:
    if (nt > 0) ticks[nt - 1].fin = p;
    printf("# %d ticks enregistres (%u..%u)\n", nt, nt ? ticks[0].tick : 0, nt ? ticks[nt - 1].tick : 0);
    synchro_instantane();

    uint16_t prec_cd = 0xffff, prec_fd = 0xffff, prec_dd = 0;
    int sacch_ok = 0, sacch_ko = 0, facch_ok = 0, facch_ko = 0;
    unsigned long parole_n = 0, parole_bfi = 0, arm_hors_fenetre = 0;
    uint64_t insns = 0;
    int joues = 0;
    for (int it = 0; it < nt && it < max_ticks; it++, joues++) {
        Tick *t = &ticks[it];
        g_c54x_exe_fn = t->tick;
        long budget = t->budget;
        /* ARM writes, window 0 (before the TICK) */
        for (size_t q = t->debut; q < t->fin;) {
            uint8_t k = fichier[q];
            size_t l = k == 'T' ? 10 : k == 'A' ? 8 + 4 * (size_t)be16(fichier + q + 6)
                     : k == 'B' ? 15 + 2 * (size_t)be16(fichier + q + 13) : 0;
            if (!l) break;
            if (k == 'A' && fichier[q + 5] == 0)
                for (unsigned i = 0, n = be16(fichier + q + 6); i < n; i++) {
                    unsigned a = be16(fichier + q + 8 + 4 * i);
                    if (a >= API_WORDS) arm_hors_fenetre++;
                    ecrire_arm(a, be16(fichier + q + 10 + 4 * i));
                }
            q += l;
        }
        /* phase A */
        g_seg = SEG_DMA; calypso_dma_tick(dsp); instantane(SEG_DMA);
        reveil();
        g_seg = SEG_IRQ;
        if ((dsp->imr & (1u << C54X_IT_TPU_FRAME_BIT)) && (t->drap & 1))
            c54x_interrupt_ex(dsp, C54X_IT_TPU_FRAME_VEC, C54X_IT_TPU_FRAME_BIT);
        instantane(SEG_IRQ);
        g_seg = SEG_DSP;
        long fait = 0;
        if (!dsp->idle) fait = courir(budget / 8);
        while (!dsp->idle && fait < budget / 2) { long r = courir(256); if (r <= 0) break; fait += r; }
        /* ARM writes, window 1, then I/Q deliveries in order */
        for (size_t q = t->debut; q < t->fin;) {
            uint8_t k = fichier[q];
            size_t l = k == 'T' ? 10 : k == 'A' ? 8 + 4 * (size_t)be16(fichier + q + 6)
                     : k == 'B' ? 15 + 2 * (size_t)be16(fichier + q + 13) : 0;
            if (!l) break;
            if (k == 'A' && fichier[q + 5] == 1)
                for (unsigned i = 0, n = be16(fichier + q + 6); i < n; i++) {
                    unsigned a = be16(fichier + q + 8 + 4 * i);
                    if (a >= API_WORDS) arm_hors_fenetre++;
                    ecrire_arm(a, be16(fichier + q + 10 + 4 * i));
                }
            if (k == 'B') {
                int n = be16(fichier + q + 13);
                static int16_t iq[1024];
                memcpy(iq, fichier + q + 15, 2 * (size_t)(n > 1024 ? 1024 : n));
                g_seg = SEG_BSP;
```

```
            calypso_bsp_rx_burst(fichier[q + 5], be32(fichier + q + 6), iq, n);
            instantane(SEG_BSP);
        }
        q += l;
    }
    /* phase B: rest of the budget, then the DMA pump of pont.c */
    reveil();
    g_seg = SEG_DSP;
    if (!dsp->idle && budget - fait > 0) fait += courir(budget - fait);
    for (int k = 0; k < 40 && dsp->running; k++) {
        g_seg = SEG_POMPE;
        int pompe = calypso_rhea_dma_pump(dsp);
        instantane(SEG_POMPE);
        if (!pompe) break;
        if (dsp->idle && (dsp->ifr & dsp->imr) && !(dsp->st1 & 0x800)) dsp->idle = false;
        g_seg = SEG_DSP;
        if (!dsp->idle) fait += courir(budget / 4);
    }
    insns += (uint64_t)fait;
    /* fidelity check: same summaries as tools/rejeu_banc.c */
    uint16_t *cd = &api[NDB + 0x1FC / 2], *fd = &api[NDB + 0x21A / 2], *dd = &api[NDB + 0x238 / 2];
    if (cd[0] != prec_cd && (cd[0] & 0x8000)) { if (cd[0] & (1u << 6)) sacch_ko++; else sacch_ok++; }
    if (fd[0] != prec_fd && (fd[0] & 0x8000)) { if (fd[0] & (1u << 6)) facch_ko++; else facch_ok++; }
    if (dd[0] != prec_dd && (dd[0] & 0x8000)) { parole_n++; if (dd[0] & 0x4) parole_bfi++; }
    prec_cd = cd[0]; prec_fd = fd[0]; prec_dd = dd[0];
    if (g_journal) {
        for (unsigned q = 0; q < j_n; q++) {
            unsigned i = j_liste[q]; char b[16];
            pc_txt(b, j_pcw[i]);
            fprintf(g_journal, "%u\t%04x\t%u\t%04x\t%u\t%04x\t%s\n", t->tick, 0x800 + i, j_r[i], j_vr[i],
↪ j_w[i], j_vw[i],
                    j_w[i] ? b : "");
            j_r[i] = j_w[i] = 0;
        }
        j_n = 0;
    }
}
if (g_journal) fclose(g_journal);
struct timespec t1; clock_gettime(CLOCK_MONOTONIC, &t1);
double sec = (double)(t1.tv_sec - t0.tv_sec) + (t1.tv_nsec - t0.tv_nsec) / 1e9;

fprintf(o, "# layer1_tester : fichier=%s ticks_joues=%d/%d tick=%u..%u insn=%llu duree=%.1fs\n", nom, joues,
↪ nt,
        nt ? ticks[0].tick : 0, joues ? ticks[joues - 1].tick : 0, (unsigned long long)insns, sec);
fprintf(o, "# controle : SACCH %d bonnes / %d Fire KO ; FACCH %d / %d KO ; PAROLE %lu trames, %lu BFI\n",
        sacch_ok, sacch_ko, facch_ok, facch_ko, parole_n, parole_bfi);
fprintf(o, "# crochet : %llu evenements (%llu lectures, %llu ecritures DSP) ; ouvertures detournees vers
↪ /dev/null : %d ;"
        " mots ARM hors fenetre DSP : %lu\n", (unsigned long long)g_n_evt, (unsigned long long)g_n_rd,
        (unsigned long long)g_n_wr, g_fichiers_detournes, arm_hors_fenetre);
fprintf(o, "adr\tdsp_r\tr_pc1\tr_tick1\tr_pcs\tr_vals\tr_der"
           "\tdsp_w\tw_pc1\tw_tick1\tw_pcs\tw_vals\tw_der"
           "\tarm_w\tarm_pc1\tarm_tick1\tarm_pcs\tarm_vals\tarm_der"
           "\thors_w\thors_src\thc_w\thc_src\thc_tick1\thc_der\tval_s\tval_d\tval_fin\tboot_r\tboot_w"
           "\tarm_conso\tarm_lat_moy\tarm_ecrase\n");
for (unsigned a = 0; a < 65536; a++) {
    bool api_mot = a >= 0x800 && a < 0x800 + API_WORDS;
    if (!api_mot && !s_rd[a].n && !s_wr[a].n && !hors_wr[a] && !hc_n[a] && !boot_rd[a] && !boot_wr[a])
    ↪ continue;
    fprintf(o, "%04x", a);
    sens_txt(o, &s_rd[a]); sens_txt(o, &s_wr[a]); sens_txt(o, &s_arm[a]);
    fprintf(o, "\t%llu\t", (unsigned long long)hors_wr[a]); src_txt(o, hors_src[a]);
    fprintf(o, "\t%llu\t", (unsigned long long)hc_n[a]); src_txt(o, hc_src[a]);
    if (hc_n[a]) fprintf(o, "\t%u\t%04x", hc_tick1[a], hc_der[a]); else fprintf(o, "\t\t");
    if (a_s[a]) fprintf(o, "\t%04x", val_s[a]); else fprintf(o, "\t");
    if (a_d[a]) fprintf(o, "\t%04x", val_d[a]); else fprintf(o, "\t");
```

```
        fprintf(o, "\t%04x\t%llu\t%llu", dsp->data[a], (unsigned long long)boot_rd[a], (unsigned long
↪   long)boot_wr[a]);
        if (conso_n[a]) fprintf(o, "\t%llu\t%.2f", (unsigned long long)conso_n[a], (double)conso_lat[a] /
         ↪  conso_n[a]);
        else fprintf(o, "\t0\t");
        fprintf(o, "\t%llu\n", (unsigned long long)ecrase_n[a]);
    }
    for (unsigned a = 0; a < 65536; a++) {
        if (!io_rd[a].n && !io_wr[a].n) continue;
        fprintf(o, "io:%04x", a);
        sens_txt(o, &io_rd[a]); sens_txt(o, &io_wr[a]);
        static const Sens zero;
        sens_txt(o, &zero);                 /* no ARM columns for a port */
        for (int k = 0; k < 15; k++) fputc('\t', o);
        fputc('\n', o);
    }
    if (o != stdout) fclose(o);
    if (f_pcs) {
        FILE *fp = __real_fopen(f_pcs, "w");
        if (!fp) { perror(f_pcs); return 1; }
        fprintf(fp, "pc\tacces\n");
        for (unsigned a = 0; a < 0x40000; a++) if (pc_acc[a]) {
            char b[16]; pc_txt(b, a); fprintf(fp, "%s\t%u\n", b, pc_acc[a]); }
        fclose(fp);
    }
    printf("# fini : %d ticks, %llu insn, %.1f s ; SACCH %d/%d KO ; PAROLE %lu (%lu BFI)\n", joues,
           (unsigned long long)insns, sec, sacch_ok, sacch_ko, parole_n, parole_bfi);
    return 0;
}
```

3.39 /opt/GSM/c54x_exe/tools/layer1_tester.py

20697 octets, 435 lignes → 435 lignes

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""layer1_tester.py - the layer1 (ARM) <-> Calypso DSP ROM interface, field by field.

What the tool does (outputs PASS / FAIL / ECART / NON-EXERCE / INFO):

 1. EXTRACT. The API RAM structures as COMPILED for the firmware:
    - DWARF of layer1.highram.elf (gdb-multiarch "ptype /o"): offset and size of
      each field of T_DB_MCU_TO_DSP, T_DB_DSP_TO_MCU, T_NDB_MCU_DSP, T_PARAM_MCU_DSP;
    - offsetof() on dsp_api.h + l1_environment.h (same #define: DSP=36, CHIPSET=12,
      ANLG_FAM=2) compiled on the host: both must agree field by field;
    - BASE_API_* (header) against the dsp_api initialiser of the ELF, dsp_params (PARAM),
      B_*, *_DSP_TASK, BL_* macros (calypso/dsp.c).
    Each field -> ARM address, byte offset in the API RAM, DSP word address
    (0x0800 + bytes/2).
 2. CROSS-CHECK with the constants of qosmo (calypso_api.h, calypso_fbsb.h,
    calypso_arm2dsp.c, calypso_c54x.h, calypso_bsp.c...) and of c54x_exe (pont.c,
    montant.c, rejouer.c, tools/rejeu_banc.c...): named macros (unit table below,
    values never copied) + scan of the lines that cite a field next to a hard-coded
    address (SUSPECT if the address lands on another field).
 3. FUNCTIONS. Static scan of the layer1 functions (layer1/*.c, calypso/dsp.c):
    which fields each one writes / reads. With a replay (--rejeu or --mesure), for
    each field and each function: does the ARM write it ('A' records), with coherent
    values (task codes, burst_d/u, d_ctrl_system, d_ctrl_tch, d_fn, a_a5fn,
    d_dsp_page, B_BLUD...), does the ROM read it (hook of tools/layer1_tester.c),
    does the ROM produce what the layer1 reads; W page written = page announced by
    d_dsp_page; echo W page -> R page; PARAM and NDB initialisations of the 'S'
    state = firmware values.
 4. MAP (--md, alias "api_carte"): docs/carte-memoire.md, generated tables +
    meanings from tools/layer1_tester.notes.tsv and internal addresses from
```

```
    tools/layer1_tester.internes.tsv (hand-editable, one source per line).

    make layer1_tester
    python3 tools/layer1_tester.py                          # 1-3 (static)
    python3 tools/layer1_tester.py --rejeu FICHIER          # + measurement (≈ 1 min)
    python3 tools/layer1_tester.py --rejeu FICHIER --md docs/carte-memoire.md
    python3 tools/layer1_tester.py --mesure m.tsv --journal j.tsv --enreg FICHIER --md ...
    python3 tools/layer1_tester.py api_carte --rejeu FICHIER   # = --md docs/carte-memoire.md

Exit code 1 if there is at least one FAIL. Read-only everywhere (firmware, qosmo,
recording); only writes --md, --cache and the temporary files.
"""
import argparse
import collections
import datetime
import os
import re
import shutil
import struct
import subprocess
import sys
import tempfile

ICI = os.path.dirname(os.path.abspath(__file__))
RACINE = os.path.dirname(ICI)                       # c54x_exe
GSM = os.path.dirname(RACINE)                       # /opt/GSM
API_ARM = 0xFFD00000
API_DSP = 0x0800
API_MOTS = 0x2000

# ---------------------------------------------------------------------- results
class Resultats:
    ORDRE = ['FAIL', 'SUSPECT', 'ECART', 'NON-EXERCE', 'PASS', 'INFO']

    def __init__(self, bavard=False):
        self.lignes = []
        self.bavard = bavard

    def __call__(self, statut, cat, sujet, detail='', source=''):
        self.lignes.append((statut, cat, sujet, detail, source))
        if self.bavard or statut in ('FAIL', 'SUSPECT', 'ECART'):
            print('%-10s %-10s %-34s %s%s' % (statut, cat, sujet, detail, ('  [%s]' % source) if source else ''))

    def compte(self, cat=None):
        c = collections.Counter(l[0] for l in self.lignes if cat is None or l[1] == cat)
        return c

    def de(self, cat):
        return [l for l in self.lignes if l[1] == cat]


def rel(p):
    """Path relative to /opt/GSM for the cited sources."""
    p = os.path.abspath(p)
    return os.path.relpath(p, GSM) if p.startswith(GSM + os.sep) else p


def lancer(cmd, **kw):
    r = subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE, universal_newlines=True, **kw)
    if r.returncode != 0:
        raise RuntimeError('%s : %s' % (' '.join(cmd[:3]), r.stderr[-2000:]))
    return r.stdout


# ------------------------------------------------------- minimal C evaluation
_SUFFIXE = re.compile(r'\b(0[xX][0-9a-fA-F]+|\d+)[uUlL]+\b')
```

```
_CAST = re.compile(r'\(\s*(?:const\s+)?(?:volatile\s+)?(?:unsigned\s+|signed\s+)?'
                   r'(?:u?int(?:8|16|32|64)_t|int|long|short|char|API|API_SIGNED)\s*\*?\s*\)')


def c_vers_py(expr):
    e = _CAST.sub('', expr)
    e = _SUFFIXE.sub(r'\1', e)
    e = e.replace('&&', ' and ').replace('||', ' or ').replace('!', ' not ').replace(' not =', '!=')
    # ternary a ? b : c (a single level)
    m = re.match(r'^\s*\(?\s*(.+?)\s*\?\s*(.+?)\s*:\s*(.+?)\s*\)?\s*$', e)
    if m and '?' in e:
        e = '((%s) if (%s) else (%s))' % (m.group(2), m.group(1), m.group(3))
    e = re.sub(r'(?<![\w])/(?![/=])', '//', e)
    return e


def evaluer(expr, env, prof=0):
    """Evaluates a C constant expression; env: name -> (text, params) or integer."""
    if prof > 20:
        raise ValueError('recursion')
    expr = re.sub(r'/\*.*?\*/', '', expr)
    expr = re.sub(r'//.*', '', expr).strip()
    if not expr:
        raise ValueError('vide')

    def sub(m):
        n = m.group(0)
        if n in env:
            v = env[n]
            if isinstance(v, int):
                return str(v)
            texte, params = v
            if params is not None:
                raise ValueError('macro a parametres ' + n)
            return '(%d)' % evaluer(texte, env, prof + 1)
        if re.match(r'^(0[xX][0-9a-fA-F]+|\d+)$', n):
            return n
        if n in ('and', 'or', 'not', 'if', 'else'):
            return n
        raise ValueError('inconnu ' + n)
    e = c_vers_py(expr)
    e = re.sub(r'\b[A-Za-z_]\w*\b|\b0[xX][0-9a-fA-F]+\b|\b\d+\b', sub, e)
    if re.search(r'[^\s\d()+\-*/%<>&|^~xXa-fA-F,.]|\b(?!(if|else|and|or|not)\b)[g-wyzG-WYZ_]\w*', e):
        pass
    v = eval(e, {'__builtins__': {}}, {})
    if isinstance(v, bool):
        v = int(v)
    if not isinstance(v, int):
        raise ValueError('pas entier')
    return v


_DEFINE = re.compile(r'^\s*#\s*define\s+([A-Za-z_]\w*)(\(([^)]*)\))?(?:[ \t]+(.*?))?\s*$')


def lire_defines(chemin):
    """name -> (text, params|None, line); joins the lines continued by \\."""
    res = {}
    try:
        lignes = open(chemin, errors='replace').read().split('\n')
    except OSError:
        return res
    i = 0
    while i < len(lignes):
        l = lignes[i]
        n0 = i + 1
```

```
        while l.endswith('\\') and i + 1 < len(lignes):
            i += 1
            l = l[:-1] + ' ' + lignes[i]
        m = _DEFINE.match(l)
        if m:
            texte = (m.group(4) or '').strip()
            texte = re.sub(r'/\*.*?\*/', '', texte)
            texte = re.sub(r'//.*', '', texte).strip()
            params = [p.strip() for p in m.group(3).split(',')] if m.group(2) else None
            res.setdefault(m.group(1), (texte, params, n0))
        i += 1
    return res


def valeur_macro(defs, nom, args=None):
    texte, params, _ = defs[nom]
    env = {k: (v[0], v[1]) for k, v in defs.items()}
    if params is not None:
        for p, a in zip(params, args or []):
            texte = re.sub(r'\b%s\b' % re.escape(p), '(%d)' % a, texte)
    return evaluer(texte, env)


# ---------------------------------------------------------------- extraction
class Interface:
    """The layer1 API structures, as compiled."""
    TYPES = ['T_DB_MCU_TO_DSP', 'T_DB_DSP_TO_MCU', 'T_NDB_MCU_DSP', 'T_PARAM_MCU_DSP']
    ZONES = [('W0', 'T_DB_MCU_TO_DSP', 'BASE_API_W_PAGE_0'), ('W1', 'T_DB_MCU_TO_DSP', 'BASE_API_W_PAGE_1'),
             ('R0', 'T_DB_DSP_TO_MCU', 'BASE_API_R_PAGE_0'), ('R1', 'T_DB_DSP_TO_MCU', 'BASE_API_R_PAGE_1'),
             ('NDB', 'T_NDB_MCU_DSP', 'BASE_API_NDB'), ('PARAM', 'T_PARAM_MCU_DSP', 'BASE_API_PARAM')]

    def __init__(self, fw, elf, res, tmp):
        self.fw, self.elf, self.res, self.tmp = fw, elf, res, tmp
        self.inc = os.path.join(fw, 'include')
        self.hdr = os.path.join(self.inc, 'calypso', 'dsp_api.h')
        self.env_h = os.path.join(self.inc, 'calypso', 'l1_environment.h')
        self.dsp_c = os.path.join(fw, 'calypso', 'dsp.c')
        self.disp = {}          # type -> [(name, off, size, n, sign)]
        self.taille_elf = {}
        self.taille_hote = {}
        self.l1 = {}            # layer1 macro -> value (compiled)
        self.l1_src = {}        # macro -> file:line
        self.dspc = {}          # macros of calypso/dsp.c
        self.params = {}        # dsp_params (ELF)
        self.dsp_api_init = {}
        self.commentaires = {}  # (type, field) -> text
        self.champs = []        # dicts
        self.par_mot = collections.defaultdict(list)

    # -- DWARF
    def _gdb(self):
        cmds = []
        for t in self.TYPES:
            cmds += ['-ex', 'echo @@%s\\n' % t, '-ex', 'ptype /o %s' % t, '-ex', 'print sizeof(%s)' % t]
        cmds += ['-ex', 'echo @@params\\n', '-ex', 'print/x dsp_params', '-ex', 'echo @@api\\n', '-ex', 'print/x
↪  dsp_api']
        gdb = shutil.which('gdb-multiarch') or shutil.which('gdb')
        out = lancer([gdb, '-batch', '-nx'] + cmds + [self.elf])
        blocs = re.split(r'^@@(\w+)$', out, flags=re.M)
        for i in range(1, len(blocs) - 1, 2):
            nom, txt = blocs[i], blocs[i + 1]
            if nom in self.TYPES:
                ch = []
                for m in re.finditer(r'/\*\s*(\d+)\s*\|\s*(\d+)\s*\*/\s*(API_SIGNED|API)\s+(\w+)(?:\[(\d+)\])?;',
                 ↪  txt):
```

```
                ch.append((m.group(4), int(m.group(1)), int(m.group(2)), int(m.group(5) or 1), m.group(3) ==
↪ 'API_SIGNED'))
            self.disp[nom] = ch
            m = re.search(r'^\$\d+ = (\d+)', txt, re.M)
            self.taille_elf[nom] = int(m.group(1)) if m else None
        elif nom == 'params':
            self.params = self._gdb_struct(txt)
        elif nom == 'api':
            self.dsp_api_init = {k: v[0] for k, v in self._gdb_struct(txt).items()}

    @staticmethod
    def _gdb_struct(txt):
        txt = txt[txt.find('{') + 1: txt.rfind('}')]
        res = {}
        for m in re.finditer(r'(\w+) = (\{[^}]*\}|-?0x[0-9a-fA-F]+|-?\d+)', txt):
            v = m.group(2)
            if v.startswith('{'):
                vals = []
                for tok in v[1:-1].split(','):
                    tok = tok.strip()
                    r = re.match(r'(-?0x[0-9a-fA-F]+|-?\d+)(?:\s*<repeats (\d+) times>)?', tok)
                    if r:
                        vals += [int(r.group(1), 0)] * int(r.group(2) or 1)
                res[m.group(1)] = vals
            else:
                res[m.group(1)] = [int(v, 0)]
        return res

    # -- host offsetof + macros
    def _hote(self):
        defs = {}
        for f in (self.env_h, self.hdr):
            for n, (t, p, l) in lire_defines(f).items():
                if p is None and t and not n.startswith('_'):
                    defs.setdefault(n, '%s:%d' % (rel(f), l))
        src = ['#include <stdio.h>', '#include <stddef.h>', '#include "dsp_api.h"', 'int main(void) {']
        for t in self.TYPES:
            for (nom, off, taille, n, sg) in self.disp[t]:
                src.append('printf("F %s %s %%zu %%zu\\n", offsetof(%s, %s), sizeof(((%s *)0)->%s));' % (t, nom,
↪ t, nom, t, nom))
            src.append('printf("S %s %%zu\\n", sizeof(%s));' % (t, t))
        for n in sorted(defs):
            src.append('#ifdef %s\nprintf("M %s %%lld\\n", (long long)(%s));\n#endif' % (n, n, n))
        src.append('return 0; }')
        c = os.path.join(self.tmp, 'offsets.c')
        open(c, 'w').write('\n'.join(src) + '\n')
        exe = os.path.join(self.tmp, 'offsets')
        r = subprocess.run(['gcc', '-w', '-I', os.path.join(self.inc, 'calypso'), '-I', self.inc, '-o', exe, c],
                           stdout=subprocess.PIPE, stderr=subprocess.PIPE, universal_newlines=True)
        if r.returncode != 0:
            # a non-integer macro breaks the compilation: drop the offending M lines
            mauvaises = set(re.findall(r"error: [^\n]*?'(\w+)'", r.stderr))
            src = [s for s in src if not any(('M %s ' % m) in s for m in mauvaises)]
            open(c, 'w').write('\n'.join(src) + '\n')
            lancer(['gcc', '-w', '-I', os.path.join(self.inc, 'calypso'), '-I', self.inc, '-o', exe, c])
        hote = {}
        for l in lancer([exe]).split('\n'):
            p = l.split()
            if not p:
                continue
            if p[0] == 'F':
                hote[(p[1], p[2])] = (int(p[3]), int(p[4]))
            elif p[0] == 'S':
                self.taille_hote[p[1]] = int(p[2])
            elif p[0] == 'M':
                self.l1[p[1]] = int(p[2])
```

```
                self.l1_src[p[1]] = defs[p[1]]
        return hote

    def _commentaires(self):
        """Comment of each field, from the PREPROCESSED header (active branch)."""
        c = os.path.join(self.tmp, 'pp.c')
        open(c, 'w').write('#include "dsp_api.h"\n')
        pp = lancer(['gcc', '-E', '-C', '-P', '-I', os.path.join(self.inc, 'calypso'), '-I', self.inc, c])
        for m in re.finditer(r'typedef\s+struct\s*\{(.*?)\}\s*(\w+)\s*;', pp, re.S):
            corps, t = m.group(1), m.group(2)
            dernier = None
            for l in corps.split('\n'):
                f = re.search(r'\b(API_SIGNED|API)\s+(\w+)\s*(\[[^\]]*\])?\s*;(.*)$', l)
                if f:
                    dernier = f.group(2)
                    com = f.group(4).strip()
                    com = re.sub(r'^//\s*|^/\*\s*|\s*\*/$', '', com)
                    self.commentaires[(t, dernier)] = com
                elif dernier and re.match(r'^\s*//', l):
                    self.commentaires[(t, dernier)] += ' ' + re.sub(r'^\s*//\s*', '', l).strip()
                elif re.match(r'^\s*$', l) or re.match(r'^\s*//', l):
                    continue
                else:
                    dernier = None

    def _dsp_c(self):
        d = lire_defines(self.dsp_c)
        for n in d:
            try:
                self.dspc[n] = valeur_macro(d, n)
            except Exception:
                pass
        self.dspc_src = {n: '%s:%d' % (rel(self.dsp_c), d[n][2]) for n in d}

    def extraire(self):
        r = self.res
        self._gdb()
        hote = self._hote()
        self._commentaires()
        self._dsp_c()
        # 1. ELF offsets == host offsetof
        nb = collections.Counter()
        for t in self.TYPES:
            for (nom, off, taille, n, sg) in self.disp[t]:
                h = hote.get((t, nom))
                if h == (off, taille):
                    nb[t] += 1
                    if r.bavard:
                        r('PASS', 'extraction', '%s.%s' % (t, nom), 'offset %d taille %d (ELF = offsetof)' % (off,
↪   taille))
                else:
                   r('FAIL', 'extraction', '%s.%s' % (t, nom), 'ELF offset %d taille %d, offsetof hote %s' % (off,
↪   taille, h))
            r('PASS', 'extraction', t, '%d champs : offsets DWARF de l\'ELF = offsetof(dsp_api.h, DSP=%s)' %
↪   (nb[t], self.l1.get('DSP')),
              rel(self.elf))
            te, th = self.taille_elf.get(t), self.taille_hote.get(t)
            if te != th:
                dernier = self.disp[t][-1]
                r('ECART', 'extraction', 'sizeof(%s)' % t,
                  'ELF %s octets, hote %s : bourrage de fin (dernier champ %s finit a %d) ; offsets inchanges' %
                  (te, th, dernier[0], dernier[1] + dernier[2]), rel(self.elf))
        # 2. bases: header == dsp_api initialiser of the ELF
        for z, t, b in self.ZONES:
            if b not in self.l1:
                r('FAIL', 'extraction', b, 'absent de dsp_api.h')
```

```
        for cle, base in (('ndb', 'BASE_API_NDB'), ('db_r', 'BASE_API_R_PAGE_0'), ('db_w', 'BASE_API_W_PAGE_0'),
         ↪  ('param', 'BASE_API_PARAM')):
            v = self.dsp_api_init.get(cle)
            st = 'PASS' if v == self.l1.get(base) else 'FAIL'
            r(st, 'extraction', 'dsp_api.%s' % cle, 'ELF %s, %s = 0x%08x' % (hex(v) if v is not None else None,
↪  base, self.l1.get(base, 0)),
              '%s ; %s' % (rel(self.dsp_c), self.l1_src.get(base, '')))
        # 3. field table
        for z, t, b in self.ZONES:
            base = self.l1[b]
            for (nom, off, taille, n, sg) in self.disp[t]:
                arm = base + off
                octet = arm - API_ARM
                mot = API_DSP + octet // 2
                c = dict(zone=z, type=t, nom=nom, off=off, taille=taille, n=n, signe=sg, arm=arm, octet=octet,
                         mot=mot, mots=list(range(mot, mot + taille // 2)),
                         commentaire=self.commentaires.get((t, nom), ''), trou='hole' in nom)
                self.champs.append(c)
                for i, w in enumerate(c['mots']):
                    self.par_mot[w].append((c, i))
        # 4. geometry: disjoint pages, PARAM in a hole of the NDB, everything in the DSP window
        zones = {z: (self.l1[b] - API_ARM, self.l1[b] - API_ARM + self.taille_hote[t]) for z, t, b in self.ZONES}
        for a, b in (('W0', 'W1'), ('W1', 'R0'), ('R0', 'R1'), ('R1', 'NDB')):
            ok = zones[a][1] <= zones[b][0]
            r('PASS' if ok else 'FAIL', 'extraction', 'pages %s/%s' % (a, b),
              '%s 0x%03x..0x%03x, %s commence en 0x%03x' % (a, zones[a][0], zones[a][1] - 1, b, zones[b][0]))
        ndb0, ndb1 = zones['NDB']
        p0, p1 = zones['PARAM']
        trou = [c for c in self.champs if c['zone'] == 'NDB' and c['octet'] <= p0 < c['octet'] + c['taille']]
        if trou:
            tc = trou[0]
            utile = self.disp['T_PARAM_MCU_DSP'][-1]
            fin_utile = p0 + utile[1] + utile[2]
            ok = tc['trou'] and fin_utile <= tc['octet'] + tc['taille']
            r('PASS' if ok else 'FAIL', 'extraction', 'PARAM dans le NDB',
              'PARAM (0x%03x..0x%03x utiles) recouvre NDB.%s (0x%03x..0x%03x)%s' %
              (p0, fin_utile - 1, tc['nom'], tc['octet'], tc['octet'] + tc['taille'] - 1,
               '' if ok else ' : le recouvrement deborde sur un champ utile'))
        fin = max(v[1] for v in zones.values())
        r('PASS' if fin <= API_MOTS * 2 else 'FAIL', 'extraction', 'fenetre DSP',
          'structures 0x000..0x%03x octets = mots DSP 0x0800..0x%04x (fenetre 0x0800..0x%04x)' %
          (fin - 1, API_DSP + (fin - 1) // 2, API_DSP + API_MOTS - 1))
        # 5. loader (calypso/dsp.c)
        base_ram = self.dspc.get('BASE_API_RAM')
        r('PASS' if base_ram == API_ARM else 'FAIL', 'extraction', 'BASE_API_RAM', '0x%08x' % (base_ram or 0),
↪  self.dspc_src.get('BASE_API_RAM', ''))
        api_size = self.dspc.get('API_SIZE')
        r('PASS' if api_size == API_MOTS else 'FAIL', 'extraction', 'API_SIZE', '0x%x mots' % (api_size or 0),
↪  self.dspc_src.get('API_SIZE', ''))
        self.bl = {}
        for n in ('BL_CMD_STATUS', 'BL_ADDR_LO', 'BL_ADDR_HI', 'BL_SIZE'):
            if n in self.dspc:
                a = self.dspc[n]
                self.bl[n] = API_DSP + (a - API_ARM) // 2
                dessous = [c['nom'] for c, i in self.par_mot.get(self.bl[n], [])]
                r('INFO', 'extraction', n, 'ARM 0x%08x = mot DSP 0x%04x (recouvre NDB %s)' % (a, self.bl[n],
↪  ','.join(dessous) or '-'),
                  self.dspc_src[n])
        # 6. hard-coded address of the layer1 itself (dsp.c SC_CHKSUM_VER)
        m = re.search(r'#define\s+SC_CHKSUM_VER\s+\(BASE_API_W_PAGE_0\s*\+\s*\(2\s*\*\s*\((0x[0-9A-Fa-f]+)\s*-
↪  \s*0x800\)\)\)',
                      open(self.dsp_c).read())
        if m:
            w = int(m.group(1), 16)
            noms = [c['nom'] for c, i in self.par_mot.get(w, []) if c['zone'] == 'NDB']
            r('PASS' if noms == ['d_version_number2'] else 'FAIL', 'extraction', 'SC_CHKSUM_VER (dsp.c)',
```

```
            'mot DSP 0x%04x = NDB.%s (« dsp patch version »)' % (w, ','.join(noms)), rel(self.dsp_c))
        return self

    # -- lookup
    def champ(self, zone, nom):
        for c in self.champs:
            if c['nom'] == nom and (c['zone'] == zone or (zone in ('W', 'R') and c['zone'].startswith(zone))):
                return c
        return None

    def zone_base(self, z):
        for zz, t, b in self.ZONES:
            if zz == z:
                return self.l1[b]
        raise KeyError(z)

    def noms(self):
        return set(c['nom'] for c in self.champs)

    def au_mot(self, w):
        """Field(s) at DSP word w, PARAM before the NDB holes."""
        l = self.par_mot.get(w, [])
        return sorted(l, key=lambda ci: (ci[0]['trou'], ci[0]['zone'] == 'NDB'))
```

3.40 /opt/GSM/c54x_exe/tools/rejeu_banc.c

42277 octets, 735 lignes → 735 lignes

```
/* rejeu_banc.c - replays outside the bench a TCH recorded by c54x_exe.
 *
 * Input: /dev/shm/calypso_rejeu_tch.bin (calypso_bsp.c + pont.c,
 * CALYPSO_REJEU_ENREG). Records:
 *   'S' tick, whole API RAM           'D' tick, registers, data memory
 *   'T' tick, flags, budget           'A' tick, window, n x (word, value)
 *   'B' tick, tn, fn, one_shot, nwin, n, I/Q
 * The DSP is booted like tools/tch_rejeu, the 'S'/'D' state is loaded, then
 * each tick in the order of pont.c: ARM writes before the TICK, frame
 * interrupt, phase A until the IDLE (TCH), ARM writes between A and the GO,
 * I/Q deliveries, rest of the budget, DMA pump. Every SACCH (a_cd) and
 * FACCH (a_fd) result is printed.
 *
 * If the replay reproduces the bench's Fire KO, one can iterate here without
 * restarting the bench; REJEU_SANS_D=1 keeps the local boot state instead of 'D'.
 *
 * Diagnostics (with REJEU_TRACE=1): REJEU_STOP_W=adr (writes of one word, offending PC),
 * REJEU_STOP_NW=n (number of writes before stopping, 6), REJEU_ANNEAU_W=n (n instructions before
 * the 1st write), REJEU_STOP_PC=pc, REJEU_STOP_TMIN=tick (ignore before this tick),
 * REJEU_PEEK=adr:n,... (memory at the breakpoint). The API RAM is dsp->data[0x0800..]:
 * a_dd_0 = 0x09f0.
 *
 * Uplink and downlink of the TCH:
 *   REJEU_TX=1        each final uplink burst (TSP script, src/tsp_tx.c): fn of the W page, type,
 *                     TSC, hl/hu, 148 bits -- to decode offline (libosmocoding, A5 at the ROM's fn)
 *   REJEU_TX_PHASE=1  at which point of the tick the ROM finishes this script (phase A, B, DMA pump)
 *   REJEU_BLUD=1      who sets / clears B_BLUD of a_cu, a_fu, a_du (ARM at T, ROM at T+1)
 *   REJEU_DL=1        each downlink burst delivered (window 0): fn of the bridge, fn of the W page
 *                     (a_a5fn, both must be equal), demodulated bits (roughly: TSC at 14/26)
 *
 *   ./rejeu_banc [fichier] [ticks max]
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
```

```
#include <stdbool.h>
#include "qemu/thread.h"
#include "calypso_c54x.h"
#include "calypso_dma.h"
#include "calypso_bsp.h"
#include "calypso_rhea_dma.h"
#include "tsp_tx.h"
#include "calypso_vsp.h"
#include "hw/arm/calypso/calypso_api.h"
#include <math.h>
#include <osmocom/core/bits.h>
#include <osmocom/gsm/a5.h>

uint32_t g_c54x_exe_fn;
uint32_t calypso_trx_get_fn(void) { return g_c54x_exe_fn; }
void calypso_inth_arm_ack(void) { }
QemuMutex calypso_pcb_daram_lock;
void cpu_physical_memory_rw(uint64_t addr, void *buf, uint64_t len, bool wr)
{ (void)addr; if (!wr) memset(buf, 0, len); }

#define API_WORDS 0x2000u          /* API window as seen by the DSP: data 0x0800..0x27ff */
#define ENREG_API_WORDS CALYPSO_API_WORDS   /* size of the API RAM recorded by pont.c */
#define NDB 0xD4u
#define W_PAGE(p) ((p) ? 0x14u : 0x00u)   /* T_DB_MCU_TO_DSP, 17 words (like src/rejouer.c) */
static uint32_t g_page_it_fn; static unsigned g_page_it_pg; static uint16_t g_page_it_td, g_page_it_tu,
 ↪  g_page_it_tra, g_page_it_ctrl, g_it_tch_mode, g_it_du1;
#define PARAM 0x431u
#define BL_ADDR_HI_W 0x7FCu
#define BL_SIZE_W 0x7FDu
#define BL_ADDR_LO_W 0x7FEu
#define BL_STATUS_W 0x7FFu

typedef struct {
    int64_t a, b; uint16_t ar[8], t, trn, sp, bk, brc, rsa, rea, st0, st1, pmst, imr, ifr, xpc;
    uint32_t pc; uint8_t idle, running;
} EnregRegs;

static C54xState *dsp;
static uint16_t *api;
static uint8_t *fichier;
static size_t taille;

static uint32_t be32(const uint8_t *p) { return (uint32_t)p[0] << 24 | p[1] << 16 | p[2] << 8 | p[3]; }
static uint16_t be16(const uint8_t *p) { return (uint16_t)(p[0] << 8 | p[1]); }

static long pump(long max, int stop_idle)
{
    long b = 0;
    while (b < max && dsp->running) {
        int ex = c54x_run(dsp, 256); if (ex <= 0) break; b += ex;
        if (stop_idle && dsp->idle) break;
    }
    return b;
}

static void reveil(void)
{
    if (dsp->idle && calypso_rhea_dma_irq_level() && (dsp->imr & (1u << 14)) && !(dsp->ifr & (1u << 14)))
        c54x_interrupt_ex(dsp, 30, 14);
    if (dsp->idle && (dsp->ifr & dsp->imr) && !(dsp->st1 & 0x800)) dsp->idle = false;
}

static void boot(void)
{
    memset(api, 0, API_WORDS * 2);
    long b = 0;
```

```
    while (api[BL_STATUS_W] != 1 && b < 8000000) { int ex = c54x_run(dsp, 256); if (ex <= 0) break; b += ex; }
    api[BL_ADDR_HI_W] = 0; api[BL_ADDR_LO_W] = 0x7000; api[BL_SIZE_W] = 0; api[BL_STATUS_W] = 2;
    b = pump(4000000, 1);
    printf("boot local : %ld insn, idle=%d\n", b, dsp->idle);
}

/* PROBE (REJEU_SONDE=1): where does the ROM store the SACCH bursts?
 * For each burst of the channel (slot 0 delivered by the BSP), the bits are
 * recovered by differential GMSK demodulation (polarity locked on the TSC7),
 * deciphered (Kc and d_a5mode taken from the API RAM), then at the tick where
 * a_cd changes the DSP memory is searched for the 116 bits of each of the
 * four bursts of the block (soft bits: negative or positive sign = 1). */
static const uint8_t TSC7[26] = {1,1,1,0,1,1,1,1,0,0,0,1,0,0,1,0,1,1,1,0,1,1,1,1,0,0};
static struct { uint32_t fn; uint8_t b[148]; } memo[4096];
static int nmemo;
/* Differential GMSK demodulation of a delivered burst (148 bits, polarity locked on the TSC7);
 * returns the TSC score (0..26) or -1 if the window is too short. */
static int demod_burst(const int16_t *iq, int n, int marge, uint8_t *out, int *polarite)
{
    if (n / 2 < marge + 148) return -1;
    uint8_t d[148], b[2][148];
    for (int k = 0; k < 148; k++) {
        int i0 = 2 * (marge + k), i1 = i0 - 2;
        double ph1 = atan2(iq[i0 + 1], iq[i0]);
        double ph0 = k ? atan2(iq[i1 + 1], iq[i1]) : 0;
        double dp = ph1 - ph0;
        while (dp > M_PI) dp -= 2 * M_PI;
        while (dp < -M_PI) dp += 2 * M_PI;
        d[k] = dp > 0;
    }
    int best = -1, bs = -1;
    for (int pol = 0; pol < 2; pol++) {
        uint8_t prev = 0;
        for (int k = 0; k < 148; k++) { uint8_t dd = d[k] ^ pol; b[pol][k] = dd ^ prev; prev = b[pol][k]; }
        int sc = 0; for (int k = 0; k < 26; k++) sc += b[pol][61 + k] == TSC7[k];
        if (sc > bs) { bs = sc; best = pol; }
    }
    memcpy(out, b[best], 148); *polarite = best;
    return bs;
}
static void memo_burst(uint32_t fn, const int16_t *iq, int n, int marge)
{
    uint8_t b[148]; int pol;
    int bs = demod_burst(iq, n, marge, b, &pol);
    if (bs < 0) return;
    int slot = nmemo % 4096; nmemo++;
    memo[slot].fn = fn; memcpy(memo[slot].b, b, 148);
    static int nlog; if (nlog++ < 3) printf("  [sonde] burst fn=%u : TSC %d/26 (differentielle, pol=%d)\n", fn,
     ↪  bs, pol);
}
static const uint8_t *memo_trouver(uint32_t fn)
{
    /* Exact bits of the BSP probe (calypso_sacch_tf.bin), if any. */
    static uint8_t (*vrai)[156]; static int nvrai = -1;
    if (nvrai < 0) {
        nvrai = 0;
        const char *nom = getenv("REJEU_BITS");
        FILE *f = nom ? fopen(nom, "rb") : NULL;
        if (f) { vrai = malloc(4096 * 156); while (nvrai < 4096 && fread(vrai[nvrai], 1, 156, f) == 156) nvrai++;
         ↪  fclose(f); }
        if (nom) printf("  [sonde] %d bursts exacts lus dans %s\n", nvrai, nom);
    }
    for (int i = 0; i < nvrai; i++) if (be32(vrai[i] + 4) == fn) return vrai[i] + 8;
    if (getenv("REJEU_BITS")) return NULL;
    for (int i = 0; i < 4096 && i < nmemo; i++) if (memo[i].fn == fn) return memo[i].b;
    return NULL;
```

```
}
static void sonde_bloc(uint32_t fn_dernier)
{
    uint8_t kc[8]; uint16_t *akc = &api[NDB + 0x2CE / 2]; int algo = api[NDB + 0x1CE / 2];
    for (int i = 0; i < 4; i++) { kc[7 - 2 * i] = akc[i] & 0xff; kc[6 - 2 * i] = akc[i] >> 8; }
    for (int k = 0; k < 4; k++) {
        uint32_t fn = fn_dernier - 26 * (3 - k);
        const uint8_t *b0 = memo_trouver(fn);
        if (!b0) { printf("    burst %d fn=%u : non enregistre\n", k, fn); continue; }
        uint8_t b[148]; memcpy(b, b0, 148);
        if (algo) { ubit_t dl[114], ul[114]; osmo_a5(algo, kc, fn, dl, ul);
            for (int j = 0; j < 57; j++) { b[3 + j] ^= dl[j]; b[88 + j] ^= dl[57 + j]; } }
        uint8_t t[116]; memcpy(t, b + 3, 58); memcpy(t + 58, b + 87, 58);
        int best[2] = {0, 0}; unsigned ad[2] = {0, 0};
        for (unsigned a = 0x60; a + 116 < C54X_DATA_SIZE; a++) {
            int m0 = 0, m1 = 0;
            for (int i = 0; i < 116; i++) { int16_t v = (int16_t)dsp->data[a + i];
                if (v != 0) { if ((v < 0) == (t[i] != 0)) m0++; if ((v > 0) == (t[i] != 0)) m1++; } }
            if (m0 > best[0]) { best[0] = m0; ad[0] = a; }
            if (m1 > best[1]) { best[1] = m1; ad[1] = a; }
        }
        /* also as 57+57 without the stealing bits (e[] only) */
        uint8_t e[114]; memcpy(e, b + 3, 57); memcpy(e + 57, b + 88, 57);
        int be = 0; unsigned ae = 0;
        for (unsigned a = 0x60; a + 114 < C54X_DATA_SIZE; a++) {
            int m = 0; for (int i = 0; i < 114; i++) { int16_t v = (int16_t)dsp->data[a + i]; if (v != 0 && (v < 0)
             ↪  == (e[i] != 0)) m++; }
            if (m > be) { be = m; ae = a; }
        }
        printf("    burst %d fn=%u (fn%%104=%u) : 116b neg %d@%04x pos %d@%04x | 114b neg %d@%04x\n",
               k, fn, fn % 104, best[0], ad[0], best[1], ad[1], be, ae);
    }
}

/* REJEU_TRACE=1: step by step, ring of the last instructions, stop as soon as
 * SP leaves the stack or PC falls into low DARAM (< 0x0800, outside OVLY). */
uint16_t prog_fetch(C54xState *s, uint16_t pc);
uint16_t data_read(C54xState *s, uint16_t addr);
#define ANNEAU 4096
static struct { uint16_t pc, op, op2, sp, st0, st1, ar[8]; uint32_t tick; int64_t a, b; } anneau[ANNEAU];
static unsigned apos;
static uint16_t *g_pcs; static unsigned long *g_pcs_cpt; static int g_npcs;
static unsigned long *g_pcplage; static unsigned g_pclo, g_pchi;
static void pcplage_resume(void)
{
    for (unsigned a = g_pclo; a <= g_pchi; a++) if (g_pcplage[a - g_pclo]) printf("PCH %04x %lu\n", a, g_pcplage[a
     ↪  - g_pclo]);
}
static void pcs_resume(void)
{
    for (int q = 0; q < g_npcs; q++) printf("PCS %04x : %lu\n", g_pcs[q], g_pcs_cpt[q]);
}
static int trace_on = -1, plante;
static long courir(long n);
static int vsp_courir(C54xState *d, int n) { (void)d; return (int)courir(n); }
static long courir(long n)
{
    if (trace_on < 0) trace_on = getenv("REJEU_TRACE") != NULL;
    /* Loop up to the budget, like the bench (pont.c c54x_run_profile): c54x_run yields by itself at
     * an interruptible point after CALYPSO_DSP_YIELD instructions (32768 by default). A single call
     * would lose the rest of each phase's budget and the DSP would not finish its work. */
    if (!trace_on) {
        long k = 0;
        while (k < n && dsp->running && !dsp->idle) {
            int r = c54x_run(dsp, (int)(n - k));
            if (r <= 0) break;
```

```
        k += r;
    }
    return k;
}
long k = 0;
for (; k < n && dsp->running && !dsp->idle && !plante; k++) {
    unsigned i = apos++ & (ANNEAU - 1);
    uint16_t pc = dsp->pc & 0xffff;
    anneau[i].pc = pc; anneau[i].op = prog_fetch(dsp, pc); anneau[i].op2 = prog_fetch(dsp, pc + 1);
   anneau[i].sp = dsp->sp; anneau[i].st0 = dsp->st0; anneau[i].st1 = dsp->st1; anneau[i].tick = g_c54x_exe_fn;
    memcpy(anneau[i].ar, dsp->ar, sizeof dsp->ar); anneau[i].a = dsp->a; anneau[i].b = dsp->b;
    {   /* REJEU_PCPLAGE=lo-hi: histogram of the PCs executed in the range (end of replay) */
        static int init; static unsigned lo, hi; static unsigned long *h;
        if (!init) { init = 1; const char *e = getenv("REJEU_PCPLAGE");
            if (e && sscanf(e, "%x-%x", &lo, &hi) == 2 && hi >= lo) { h = calloc(hi - lo + 1, sizeof *h);
             ↪  g_pcplage = h; g_pclo = lo; g_pchi = hi; atexit(pcplage_resume); } }
        if (h && pc >= lo && pc <= hi && dsp->xpc == 0) h[pc - lo]++;
    }
    {   /* REJEU_PCS=pc1,pc2,...: number of executions of each PC (summary at end of replay) */
        static int npcs = -1; static uint16_t pcs[32]; static unsigned long cpt[32];
        if (npcs < 0) { npcs = 0; const char *e = getenv("REJEU_PCS");
            while (e && *e && npcs < 32) { pcs[npcs++] = (uint16_t)strtoul(e, (char **)&e, 16); if (*e == ','
             ↪  e++; else break; }
            if (npcs) atexit(pcs_resume); }
        for (int q = 0; q < npcs; q++) if (pc == pcs[q]) cpt[q]++;
        g_pcs = pcs; g_pcs_cpt = cpt; g_npcs = npcs;
    }
    {   /* REJEU_CODEC=file (with REJEU_TRACE=1): the ROM's FR coder, frame by frame.
         * SO = the 160 samples it reads (PDROM 0xe6af, already (s>>3)<<2 like Gsm_Preprocess),
         * ENC = the frame it returns (0x0a24, TI format: c000 52b0 0000 then 260 bits). With
         * CALYPSO_VSP_UL_FICHIER (test microphone), tools/cmp_codeur.py compares them with libgsm. */
        static int vt = -1; static FILE *fv;
       if (vt < 0) { const char *e = getenv("REJEU_CODEC"); vt = e && *e; if (vt) fv = fopen(e, "w"); if (!fv)
         ↪  vt = 0; }
        if (vt && dsp->xpc == 0) {
            if (pc == 0xe6af) { fprintf(fv, "SO"); for (int j = 0; j < 160; j++) fprintf(fv, " %d",
             ↪  (int16_t)dsp->data[0x2bfd + j]); fprintf(fv, "\n"); }
            if (pc == 0xe3fe) { fprintf(fv, "ENC fn=%u A", g_page_it_fn); for (int j = 0; j < 20; j++)
             ↪  fprintf(fv, " %04x", dsp->data[0x0a24 + j]); fprintf(fv, "\n"); }
        }
    }
    {   /* REJEU_MOTS=1: the burst buffer 0x3f8a..0x3f91 as the packing routine receives it,
         * BEFORE the A5 XOR (PC 0x85cb) -- to correlate with the transmitted bits (verif_codage). */
        static int mots = -1; if (mots < 0) mots = getenv("REJEU_MOTS") != NULL;
        if (mots && pc == 0x85cb) {
            printf("MOTS tick=%u fn=%u :", g_c54x_exe_fn, g_page_it_fn);
            for (int j = 0; j < 8; j++) printf(" %04x", dsp->data[0x3f8a + j]);
            printf("\n");
        }
    }
    static int wa = -2; static uint16_t wv;
    if (wa == -2) { const char *e = getenv("REJEU_STOP_W"); wa = e ? (int)strtoul(e, NULL, 16) : -1; }
    if (wa >= 0) wv = dsp->data[wa];
    c54x_run(dsp, 1);
    uint16_t npc = dsp->pc & 0xffff;
    static long tmin = -2; if (tmin == -2) { const char *e = getenv("REJEU_STOP_TMIN"); tmin = e ? atol(e) :
     ↪  0; }
    if (wa >= 0 && dsp->data[wa] != wv && (long)g_c54x_exe_fn >= tmin) {
        printf("ECRITURE data[%04x] %04x -> %04x par pc=%04x tick=%u : AR0=%04x AR2=%04x AR3=%04x AR4=%04x
↪  BK=%04x "
               "[4bcc]=%04x d_task_md(W0/W1)=%04x/%04x d_task_d=%04x/%04x\n",
               wa, wv, dsp->data[wa], pc, g_c54x_exe_fn, dsp->ar[0], dsp->ar[2], dsp->ar[3], dsp->ar[4],
↪  dsp->bk,
               dsp->data[0x4bcc], api[4], api[0x14 + 4], api[0], api[0x14]);
        if (getenv("REJEU_ANNEAU_W")) {
            int m = atoi(getenv("REJEU_ANNEAU_W"));
```

```
                for (int j = m; j > 0; j--) {
                    unsigned q = (apos - j) & (ANNEAU - 1);
                   printf("  %4d pc=%04x op=%04x %04x ar0=%04x ar2=%04x ar3=%04x ar4=%04x ar5=%04x sp=%04x\n", -j,
↪  anneau[q].pc,
                           anneau[q].op, anneau[q].op2, anneau[q].ar[0], anneau[q].ar[2], anneau[q].ar[3],
↪  anneau[q].ar[4], anneau[q].ar[5], anneau[q].sp);
                }
                exit(4);
            }
            static int nw; if (++nw >= (getenv("REJEU_STOP_NW") ? atoi(getenv("REJEU_STOP_NW")) : 6)) exit(4);
        }
        if (dsp->sp < 0x5900 || dsp->sp > 0x5c00 || (getenv("REJEU_STOP_PC") && (long)g_c54x_exe_fn >=
         ↪  (getenv("REJEU_STOP_TMIN") ? atol(getenv("REJEU_STOP_TMIN")) : 0) && npc ==
         ↪  (uint16_t)strtoul(getenv("REJEU_STOP_PC"), NULL, 16)) || (getenv("REJEU_STOP_DEBUG") &&
         ↪  dsp->data[0x08dc] != 0x0074 && g_c54x_exe_fn > 6200)) {
            plante = 1;
            printf("PLANTAGE tick=%u pc=%04x sp=%04x xpc=%d\n", g_c54x_exe_fn, npc, dsp->sp, dsp->xpc);
            {   /* REJEU_PEEK=start:n[,start:n...]: memory read through data_read at the breakpoint */
                const char *pk = getenv("REJEU_PEEK");
                while (pk && *pk) {
                    unsigned a0 = strtoul(pk, (char **)&pk, 16), m = 1;
                    if (*pk == ':') m = strtoul(pk + 1, (char **)&pk, 10);
                    printf("PEEK %04x:", a0);
                    for (unsigned i = 0; i < m; i++) printf(" %04x", data_read(dsp, (uint16_t)(a0 + i)));
                    printf("\n");
                    if (*pk == ',') pk++; else break;
                }
            }
            int m = getenv("REJEU_ANNEAU") ? atoi(getenv("REJEU_ANNEAU")) : 120;
            for (int j = m; j > 0; j--) {
                unsigned q = (apos - j) & (ANNEAU - 1);
               printf("  %4d t=%u pc=%04x op=%04x %04x sp=%04x st0=%04x st1=%04x ar=%04x %04x %04x %04x %04x %04x
↪  %04x %04x a=%010llx b=%010llx\n",
                      -j, anneau[q].tick, anneau[q].pc, anneau[q].op, anneau[q].op2, anneau[q].sp, anneau[q].st0,
                       ↪  anneau[q].st1,
                       anneau[q].ar[0], anneau[q].ar[1], anneau[q].ar[2], anneau[q].ar[3], anneau[q].ar[4],
↪  anneau[q].ar[5],
                       anneau[q].ar[6], anneau[q].ar[7], (unsigned long long)(anneau[q].a & 0xFFFFFFFFFFULL),
                       (unsigned long long)(anneau[q].b & 0xFFFFFFFFFFULL));
            }
            exit(3);
        }
    }
    return k;
}

/* REJEU_TX_PHASE=1: at which point of the tick does the ROM finish the uplink burst in the TSP
 * script (after phase A, after phase B, at which iteration of the DMA pump)? Compares the 16
 * BULDATA words with the last read, without touching the `nouveau` of tsp_tx_lire (local copy). */
static void tsp_phase(uint32_t tick, const char *etape, int k)
{
    static uint16_t vu[16]; static int actif = -1;
    if (actif < 0) actif = getenv("REJEU_TX_PHASE") != NULL;
    if (!actif) return;
    struct tsp_tx_burst tb; bool nouveau;
    const uint16_t *d = dsp->data;
    uint16_t mots[16] = {0}; bool trouve = false;
    for (unsigned a = 0x3cbb; a + 18 <= 0x3d00 && !trouve; a++) {
        if (d[a] != 0x1c0a) continue;
        int ok = 1; for (int i = 0; i < 16; i++) if ((d[a + 2 + i] & 0x3e) != 0x06 && (d[a + 2 + i] & 0x3e) !=
         ↪  0x04) { ok = 0; break; }
        if (ok) { memcpy(mots, &d[a + 2], sizeof mots); trouve = true; }
    }
    if (!trouve || memcmp(mots, vu, sizeof vu) == 0) return;
    memcpy(vu, mots, sizeof vu);
    (void)tb; (void)nouveau;
```

```
    printf("tick=%u SCRIPT TSP nouveau : %s%s%d\n", tick, etape, k >= 0 ? " iteration " : "", k >= 0 ? k : 0);
}

/* One tick = the records between two 'T'. */
typedef struct { size_t debut, fin; uint32_t tick; uint8_t drap; long budget; } Tick;

int main(int argc, char **argv)
{
    const char *nom = argc > 1 ? argv[1] : "/dev/shm/calypso_rejeu_tch.bin";
    long max_ticks = argc > 2 ? atol(argv[2]) : 100000;
    FILE *f = fopen(nom, "rb");
    if (!f) { perror(nom); return 1; }
    fseek(f, 0, SEEK_END); taille = (size_t)ftell(f); fseek(f, 0, SEEK_SET);
    fichier = malloc(taille);
    if (fread(fichier, 1, taille, f) != taille) { printf("lecture courte\n"); return 1; }
    fclose(f);

    setenv("CALYPSO_BSP_PORT", "16703", 1);
    setenv("CALYPSO_BSP_BIND_ADDR", "127.0.0.1", 1);
    setenv("CALYPSO_BSP_HORLOGE", "0", 1);
    setenv("CALYPSO_C54X_IRQ_LEVEL", "1", 0);
    setenv("CALYPSO_BSP_STREAM", "1", 0);
    setenv("CALYPSO_RHEA_DMA_XFER", "1", 0);
    setenv("CALYPSO_REJEU_ENREG", "0", 1);
    qemu_mutex_init(&calypso_pcb_daram_lock);
    dsp = c54x_init();
    api = &dsp->data[0x800];
    c54x_set_api_ram(dsp, api);
    static const struct { const char *s; uint32_t a; bool p; } R[] = {
        { "PROM0", 0x07000, true }, { "PROM1", 0x18000, true }, { "PROM2", 0x28000, true },
        { "PROM3", 0x38000, true }, { "DROM", 0x09000, false }, { "PDROM", 0x0E000, false }, { "PDROM", 0x0E000,
         ↪  true } };
    for (unsigned i = 0; i < 7; i++) {
        char c[256]; snprintf(c, sizeof c, "/opt/GSM/calypso_dsp.%s.bin", R[i].s);
        if (c54x_load_section(dsp, c, R[i].a, R[i].p) < 0) { printf("ROM %s\n", c); return 1; }
    }
    c54x_load_registers(dsp, "/opt/GSM/calypso_dsp.Registers.bin");
    c54x_reset(dsp);
    calypso_dma_init();
    calypso_bsp_init(dsp);
    boot();

    /* First pass: the starting state and the split into ticks. */
    static Tick ticks[40000]; int nt = 0;
    size_t p = 0;
    while (p < taille) {
        uint8_t k = fichier[p];
        size_t l;
        switch (k) {
        case 'S': l = 5 + (size_t)ENREG_API_WORDS * 2; break;
        case 'D': l = 5 + sizeof(EnregRegs) + C54X_DATA_SIZE * 2; break;
        case 'P': l = 5 + 0x8000 * 2; break;   /* program memory 0x0000-0x7FFF */
        case 'T': l = 10; break;
        case 'A': l = 8 + 4 * (size_t)be16(fichier + p + 6); break;
        case 'B': l = 15 + 2 * (size_t)be16(fichier + p + 13); break;
        default: printf("enregistrement inconnu 0x%02x a %zu\n", k, p); goto fini;
        }
        if (p + l > taille) break;
        if (k == 'S') {
            memcpy(api, fichier + p + 5, API_WORDS * 2);
            printf("etat 'S' pose (tick %u)\n", be32(fichier + p + 1));
        } else if (k == 'P' && !getenv("REJEU_SANS_D")) {
            memcpy(dsp->prog, fichier + p + 5, 0x8000 * 2);
            printf("etat 'P' pose : memoire programme 0x0000-0x7FFF\n");
        } else if (k == 'D' && !getenv("REJEU_SANS_D")) {
            EnregRegs r; memcpy(&r, fichier + p + 5, sizeof r);
```

```
            uint16_t sauve_api[API_WORDS]; memcpy(sauve_api, api, sizeof sauve_api);
            memcpy(dsp->data, fichier + p + 5 + sizeof r, C54X_DATA_SIZE * 2);
            memcpy(api, sauve_api, sizeof sauve_api);    /* 'S' is authoritative for the API RAM */
            dsp->a = r.a; dsp->b = r.b; memcpy(dsp->ar, r.ar, sizeof r.ar); dsp->t = r.t; dsp->trn = r.trn;
            dsp->sp = r.sp; dsp->bk = r.bk; dsp->brc = r.brc; dsp->rsa = r.rsa; dsp->rea = r.rea;
            dsp->st0 = r.st0; dsp->st1 = r.st1; dsp->pmst = r.pmst; dsp->imr = r.imr; dsp->ifr = r.ifr;
            dsp->xpc = r.xpc; dsp->pc = r.pc; dsp->idle = r.idle; dsp->running = r.running;
            printf("etat 'D' pose : pc=%04x sp=%04x idle=%d imr=%04x\n", r.pc & 0xffff, r.sp, r.idle, r.imr);
            {   /* REJEU_POKE=adr=val[,adr=val...]: patch the starting state */
                const char *e = getenv("REJEU_POKE");
                while (e && *e) {
                    unsigned a, v; if (sscanf(e, "%x=%x", &a, &v) != 2) break;
                    printf("poke data[%04x] = %04x (etait %04x)\n", a, v, dsp->data[a & 0xffff]);
                    dsp->data[a & 0xffff] = (uint16_t)v;
                    e = strchr(e, ','); if (e) e++;
                }
            }
        } else if (k == 'T') {
            if (nt > 0) ticks[nt - 1].fin = p;
            if (nt < 40000) {
                ticks[nt].debut = p; ticks[nt].tick = be32(fichier + p + 1);
                ticks[nt].drap = fichier[p + 5]; ticks[nt].budget = (long)be32(fichier + p + 6);
                nt++;
            }
        }
        p += l;
    }
fini:
    if (nt > 0) ticks[nt - 1].fin = p;
    printf("%d ticks enregistres (%u..%u)\n", nt, nt ? ticks[0].tick : 0, nt ? ticks[nt - 1].tick : 0);

    uint16_t prec_cd = 0xffff, prec_fd = 0xffff;
    int sacch_ok = 0, sacch_ko = 0;
    unsigned long tx_n = 0, tx_rach = 0, tx_nb = 0, tx_inc = 0;   /* final uplink bursts (TSP script) */
    unsigned long tx_hist26[26] = {0};
    static char tx_dernier[16] = "-";
    uint16_t prec_dd = 0; unsigned long parole_n = 0, parole_bfi = 0, parole_err = 0;
    for (int it = 0; it < nt && it < max_ticks; it++) {
        Tick *t = &ticks[it];
        g_c54x_exe_fn = t->tick;
        long budget = t->budget;
        /* REJEU_BUDGET=n: forced budget per tick (the bench records INSNS) */
        if (getenv("REJEU_BUDGET")) budget = atol(getenv("REJEU_BUDGET"));
        uint32_t dernier_livre = 0xffffffffu;
        /* ARM writes, window 0 */
        for (size_t q = t->debut; q < t->fin;) {
            uint8_t k = fichier[q];
            size_t l = k == 'T' ? 10 : k == 'A' ? 8 + 4 * (size_t)be16(fichier + q + 6)
                     : k == 'B' ? 15 + 2 * (size_t)be16(fichier + q + 13) : 0;
            if (!l) break;
            if (k == 'A' && fichier[q + 5] == 0)
                for (unsigned i = 0, n = be16(fichier + q + 6); i < n; i++) {
                    unsigned a = be16(fichier + q + 8 + 4 * i);
                    if (a < API_WORDS) api[a] = be16(fichier + q + 10 + 4 * i);
                }
            q += l;
        }
        {   /* REJEU_TCH_MODE_AND / _OR (hex): force bits of d_tch_mode (NDB word 3) on every
             * tick, after the ARM writes -- e.g. AND=7fff clears B_MUTE_VOCODEC_UL. */
            static int lu; static unsigned m_and = 0xffff, m_or = 0;
            if (!lu) { lu = 1; const char *e;
                       if ((e = getenv("REJEU_TCH_MODE_AND"))) m_and = strtoul(e, NULL, 16);
                       if ((e = getenv("REJEU_TCH_MODE_OR")))  m_or  = strtoul(e, NULL, 16);
                       if (m_and != 0xffff || m_or) printf("d_tch_mode force a chaque tick : AND %04x OR %04x\n",
                        ↪  m_and, m_or); }
            if (m_and != 0xffff || m_or) api[NDB + 3] = (uint16_t)((api[NDB + 3] & m_and) | m_or);
```

```
}
{   /* REJEU_W16_BIT14=1: when the ARM sets B_TASK_ABORT (bit 15 of d_ctrl_system, word 16
     * of the W page), add bit 14 -- the ROM (PROM0 0xa569) then only starts task 43 (speech
     * decoder) if this bit comes with it. */
    static int on = -1; static unsigned long n;
    if (on < 0) on = getenv("REJEU_W16_BIT14") != NULL;
    for (int pg = 0; on && pg < 2; pg++) {
        uint16_t *w16 = &api[W_PAGE(pg) + 16];
        if (!(*w16 & 0x4000)) { *w16 |= 0x4000; if (n++ < 3) printf("tick=%u : d_ctrl_system page %d |=
         ↪  0x4000\n", t->tick, pg); }
    }
}
/* REJEU_MISC_TASK=1: B_MISC_TASK (bit 3) in d_dsp_page, like the TI stack -- the ROM
 * only starts task 34 (audio commands -> task 64 -> speech decoder 0xc4f7) if this bit
 * is set (PROM0 0xb0f9, 0x3fb0 = copy of d_dsp_page). */
if (getenv("REJEU_MISC_TASK") && (api[NDB + 0] & 0x0002)) api[NDB + 0] |= 0x0008;
/* REJEU_SYNC_DL=1: the TI L1 sets B_SYNC_TCH_UL (bit 8 of d_ctrl_tch, W word 9) at the first
 * TCH but never bit 9 (B_SYNC_AMR at TI, B_SYNC_TCH_DL for the ROM and osmocom dsp.c): set it. */
/* REJEU_SYNC_DL=2: like montant.c, also on the first page with a TCH task (task_d 13):
 * the Magnetite TI L1 sets d_ctrl_tch=0x0091, without bit 8. */
if (getenv("REJEU_SYNC_DL")) {
    static bool fait;
    int mode2 = atoi(getenv("REJEU_SYNC_DL")) == 2;
    for (int pg = 0; pg < 2; pg++) {
        uint16_t *c = &api[W_PAGE(pg) + 9];
        bool tch = (api[W_PAGE(pg) + 0] & 0x7fff) == 13;
        if (!(*c & 0x0200) && ((*c & 0x0100) || (mode2 && tch && !fait))) {
            *c |= 0x0200; fait = true;
            printf("tick=%u : d_ctrl_tch page %d |= B_SYNC_TCH_DL (%04x)\n", t->tick, pg, *c);
        }
    }
}
{   /* REJEU_DU1_VERS_DU0=1: the speech frame the firmware posts in a_du_1
     * (prim_tch.c: tch_sub ? a_du_0 : a_du_1) is copied into a_du_0 -- the ROM re-reads
     * a_du_0 at PC 0x909c once per block, never a_du_1 beyond the dispatch. */
    static int on = -1; static unsigned long n;
    if (on < 0) on = getenv("REJEU_DU1_VERS_DU0") != NULL;
    uint16_t *du1 = &api[NDB + 0x134 / 2], *du0 = &api[NDB + 0x2A0 / 2];
    if (on && (du1[0] & 0x8000)) {
        memcpy(du0, du1, 22 * sizeof(uint16_t));
        du1[0] &= 0x7fff;
        if (n++ < 3) printf("tick=%u : a_du_1 -> a_du_0 (en-tete %04x)\n", t->tick, du0[0]);
    }
}
{   /* REJEU_DD0=1: each downlink speech frame (a_dd_0, B_BLUD, words 3..19, TI format)
     * with its tick -- libgsm reference to judge the output of the ROM vocoder. */
    static int on = -1; static uint16_t prec[22];
    if (on < 0) on = getenv("REJEU_DD0") != NULL;
    uint16_t *dd0 = &api[NDB + 0x238 / 2];
    if (on && (dd0[0] & 0x8000) && memcmp(dd0 + 3, prec + 3, 17 * 2)) {
        printf("DD0 tick=%u hdr=%04x :", t->tick, dd0[0]);
        for (int i = 0; i < 17; i++) printf(" %04x", dd0[3 + i]);
        printf("\n");
        memcpy(prec, dd0, sizeof prec);
    }
}
{   /* REJEU_DU1=1: each speech frame posted by the ARM (a_du_1 with B_BLUD, TI format,
     * words 3.. high byte first), with the fn of the W page -- reference for offline
     * comparison with the ROM's coding (verif_codage.py). */
    static int on = -1; static uint16_t prec[22];
    if (on < 0) on = getenv("REJEU_DU1") != NULL;
    uint16_t *du1 = &api[NDB + 0x134 / 2];
    if (on && (du1[0] & 0x8000) && memcmp(du1 + 3, prec + 3, 17 * 2)) {
        uint16_t v = api[NDB + 0]; unsigned pg = (v & 1u) ? 1u : 0u; const uint16_t *w = &api[W_PAGE(pg)];
        unsigned t2 = w[12] & 0x1f, t3 = (w[12] >> 5) & 0x3f, t1 = w[13] & 0x7ff;
        uint32_t f = (1326u * t1 + 51u * ((t3 + 26u - t2) % 26u) + t3) % 2715648u;
```

```
        printf("DU1 tick=%u fn=%u :", t->tick, f);
        for (int i = 0; i < 17; i++) printf(" %04x", du1[3 + i]);
        printf("\n");
        memcpy(prec, du1, sizeof prec);
    }
}
/* phase A: like jouer_trame(phase 1) */
calypso_dma_tick(dsp);
reveil();
if ((dsp->imr & (1u << C54X_IT_TPU_FRAME_BIT)) && (t->drap & 1))
    {   /* GSM fn of the W page the ROM is about to read at this IT (like pont.c tx_tsp_dater) */
        uint16_t v = api[NDB + 0]; unsigned pg = (v & 1u) ? 1u : 0u; const uint16_t *w = &api[W_PAGE(pg)];
        unsigned t2 = w[12] & 0x1f, t3 = (w[12] >> 5) & 0x3f, t1 = w[13] & 0x7ff;
        g_page_it_fn = (1326u * t1 + 51u * ((t3 + 26u - t2) % 26u) + t3) % 2715648u;
        g_page_it_pg = pg; g_page_it_td = w[0]; g_page_it_tu = w[2]; g_page_it_tra = w[7];
        g_page_it_ctrl = w[9];   /* d_ctrl_tch (W 0x0809 / 0x081d) */
        g_it_tch_mode = api[NDB + 3];          /* d_tch_mode (NDB word 3): B_PLAY_UL, mutes */
        g_it_du1 = api[NDB + 0x134 / 2];       /* a_du_1[0]: B_BLUD = speech frame from the ARM */
    }
    c54x_interrupt_ex(dsp, C54X_IT_TPU_FRAME_VEC, C54X_IT_TPU_FRAME_BIT);
long fait = 0;
if (!dsp->idle) fait = courir(budget / 8);
while (!dsp->idle && fait < budget / 2) fait += courir(256);
tsp_phase(t->tick, "fin de phase A", -1);
/* ARM writes, window 1, then I/Q deliveries in order */
for (size_t q = t->debut; q < t->fin;) {
    uint8_t k = fichier[q];
    size_t l = k == 'T' ? 10 : k == 'A' ? 8 + 4 * (size_t)be16(fichier + q + 6)
             : k == 'B' ? 15 + 2 * (size_t)be16(fichier + q + 13) : 0;
    if (!l) break;
    if (k == 'A' && fichier[q + 5] == 1) {
        for (unsigned i = 0, n = be16(fichier + q + 6); i < n; i++) {
            unsigned a = be16(fichier + q + 8 + 4 * i);
            if (a < API_WORDS) api[a] = be16(fichier + q + 10 + 4 * i);
        }
        if (getenv("REJEU_W16_BIT14")) { api[W_PAGE(0) + 16] |= 0x4000; api[W_PAGE(1) + 16] |= 0x4000; }
      if (getenv("REJEU_MISC_TASK") && (api[NDB + 0] & 0x0002)) api[NDB + 0] |= 0x0008;   /* B_MISC_TASK */
    }
    if (k == 'B') {
        int n = be16(fichier + q + 13);
        static int16_t iq[1024];
        size_t qb = q;
        /* REJEU_IQ_DECAL=n: on the TCH task ticks (td=0x000d), deliver the I/Q recorded at
         * tick +n (window 0) -- which alignment makes the ROM read the UA of the FACCH? */
        {   static int dec = 9999;
            if (dec == 9999) { const char *e = getenv("REJEU_IQ_DECAL"); dec = e ? atoi(e) : 0; }
            if (dec && fichier[q + 5] == 0 && g_page_it_td == 0x000d && it + dec >= 0 && it + dec < nt) {
                Tick *u = &ticks[it + dec];
                for (size_t r = u->debut; r < u->fin;) {
                    uint8_t kk = fichier[r];
                    size_t ll = kk == 'T' ? 10 : kk == 'A' ? 8 + 4 * (size_t)be16(fichier + r + 6)
                              : kk == 'B' ? 15 + 2 * (size_t)be16(fichier + r + 13) : 0;
                    if (!ll) break;
                    if (kk == 'B' && fichier[r + 5] == 0) { qb = r; break; }
                    r += ll;
                }
            }
        }
        n = be16(fichier + qb + 13);
        memcpy(iq, fichier + qb + 15, 2 * (size_t)(n > 1024 ? 1024 : n));
        calypso_bsp_rx_burst(fichier[q + 5], be32(fichier + qb + 6), iq, n);
        if (fichier[q + 5] == 0) dernier_livre = be32(fichier + q + 6);
        if (fichier[q + 5] == 0 && getenv("REJEU_SONDE")) {
            int nwin = be16(fichier + q + 11) / 2, marge = nwin >= 190 ? 21 : nwin >= 150 ? 3 : 0;
            memo_burst(be32(fichier + q + 6), iq, n, marge);
        }
```

```
            /* REJEU_DL=1: each downlink burst delivered (window 0), with the fn the bridge gave
             * it and the fn of the W page (a_a5fn) read at this tick's IT: both must be equal
             * for the ROM to decipher at the right fn. Demodulated bits to decipher/decode
             * offline (libosmocoding). */
            if (fichier[q + 5] == 0 && getenv("REJEU_DL")) {
                int nwin = be16(fichier + q + 11) / 2, marge = nwin >= 190 ? 21 : nwin >= 150 ? 3 : 0;
                uint8_t b[148]; int pol; int sc = demod_burst(iq, n, marge, b, &pol);
                if (sc >= 0) {
                    char s[149]; for (int i = 0; i < 148; i++) s[i] = '0' + b[i]; s[148] = 0;
                    printf("DL tick=%u fn=%u fnW=%u td=%04x nwin=%d tsc=%d/26 pol=%d : %s\n", t->tick,
↪ be32(fichier + q + 6),
                           g_page_it_fn, g_page_it_td, nwin, sc, pol, s);
                }
            }
        }
        q += l;
    }
    /* phase B: rest of the budget, then the DMA pump of pont.c */
    reveil();
    tsp_phase(t->tick, "apres les livraisons I/Q (avant phase B)", -1);
    if (!dsp->idle && budget - fait > 0) courir(budget - fait);
    tsp_phase(t->tick, "fin de phase B", -1);
    for (int k = 0; k < 40 && dsp->running; k++) {
        if (!calypso_rhea_dma_pump(dsp)) break;
        if (dsp->idle && (dsp->ifr & dsp->imr) && !(dsp->st1 & 0x800)) dsp->idle = false;
        if (!dsp->idle) courir(budget / 4);
        tsp_phase(t->tick, "pompe DMA", k);
    }
    /* voice port (CALYPSO_VSP=1, with CALYPSO_VSP_SHM=0 outside the bench) */
    if (dsp->running) calypso_vsp_trame(dsp, vsp_courir, g_page_it_fn);
    {   /* the ROM's final uplink burst (TSP script, src/tsp_tx.c), REJEU_TX=1: every burst */
        struct tsp_tx_burst tb; bool nouveau = false;
        if (tsp_tx_lire(dsp->data, &tb, &nouveau) && nouveau) {
            tx_n++; if (tb.type == TSP_TX_RACH) tx_rach++; else if (tb.type == TSP_TX_NB) tx_nb++; else
↪ tx_inc++;
            snprintf(tx_dernier, sizeof tx_dernier, "%s%s", tb.type == TSP_TX_RACH ? "RACH" : tb.type ==
↪ TSP_TX_NB ? "NB" : "?",
                     tb.type == TSP_TX_NB ? (tb.bits[60] || tb.bits[87] ? "/vol" : "/parole") : "");
            /* GSM fn of the W page designated by d_dsp_page (a_a5fn, see pont.c tx_tsp_dater) */
            uint32_t fn_gsm = g_page_it_fn;
            tx_hist26[fn_gsm % 26]++;
            if (getenv("REJEU_TX")) {   /* current Kc/algo, to decipher offline */
                static char kc_prec[40];
                char kcs[40]; uint16_t *akc = &api[NDB + 0x2CE / 2];
                snprintf(kcs, sizeof kcs, "%d %04x%04x%04x%04x", api[NDB + 0x1CE / 2], akc[3], akc[2], akc[1],
↪ akc[0]);
              if (strcmp(kcs, kc_prec)) { printf("KC algo,kc(a_kc[3..0])= %s\n", kcs); strcpy(kc_prec, kcs); }
            }
            if (getenv("REJEU_TX") || tx_n <= 12) {
                char bits[149]; for (int i = 0; i < 148; i++) bits[i] = '0' + tb.bits[i]; bits[148] = 0;
                printf("tick=%u fn=%u (a_a5fn page %u a l'IT, taches d/u/ra %04x/%04x/%04x) TX %s tsc=%d
↪ fn%%26=%u fn%%51=%u hl=%d hu=%d : %s\n", t->tick, fn_gsm, g_page_it_pg,
                       g_page_it_td, g_page_it_tu, g_page_it_tra, tb.type == TSP_TX_RACH ? "RACH" : tb.type ==
↪ TSP_TX_NB ? "NB" : "?", tb.tsc, fn_gsm % 26, fn_gsm % 51,
                       tb.bits[60], tb.bits[87], bits);
            }
        }
    }
    {   /* REJEU_CTRL=1: per tick, what the ROM reads at the IT (d/u tasks, d_ctrl_tch of the
         * W page) and what it sends at the end of the tick -- to tell whether uplink speech is
         * missing for lack of d_ctrl_tch = TCH_F/FS (0x0011) or for another reason. */
        static int ctrl_on = -1; static unsigned long tx_avant;
        if (ctrl_on < 0) ctrl_on = getenv("REJEU_CTRL") != NULL;
        char scr[64] = "-";
        if (ctrl_on && (g_page_it_tu || g_page_it_td)) {   /* any TSP script (0x1c0a + 16 words 0x04/0x06) */
            int k = 0; scr[0] = 0;
```

```
                for (unsigned a = 0x0080; a + 18 < 0x8000 && k < 4; a++) {
                    if (dsp->data[a] != 0x1c0a) continue;
                    int ok = 1; for (int i = 0; i < 16; i++) { uint16_t v = dsp->data[a + 2 + i] & 0x3e; if (v !=
                     ↪  0x06 && v != 0x04) { ok = 0; break; } }
                   if (ok) { k += snprintf(scr + strlen(scr), sizeof scr - strlen(scr), "%s%04x", k ? "," : "", a)
                    ↪  > 0; }
                }
                if (!scr[0]) strcpy(scr, "-");
            }
            if (ctrl_on && (g_page_it_tu || g_page_it_td))
                printf("SCR %s ", scr), printf("CTRL tick=%u fn=%u fn%%26=%u pg=%u td=%04x tu=%04x ctrl_tch=%04x
↪  tch_mode=%04x du1=%04x du1_fin=%04x tx=%s\n", t->tick, g_page_it_fn,
                       g_page_it_fn % 26, g_page_it_pg, g_page_it_td, g_page_it_tu, g_page_it_ctrl, g_it_tch_mode,
↪  g_it_du1,
                       api[NDB + 0x134 / 2],
                       tx_n == tx_avant ? "-" : tx_dernier);
            tx_avant = tx_n;
        }
        uint16_t *cd = &api[NDB + 0x1FC / 2], *fd = &api[NDB + 0x21A / 2];
        uint16_t *dd = &api[NDB + 0x238 / 2];   /* a_dd_0: downlink speech */
        {   /* REJEU_BLUD=1: who clears B_BLUD of the uplink buffers (a_cu, a_fu, a_du)?
             * At the end of the tick (after the ROM); the sets are recorded ARM writes. */
            static int blud_on = -1; static uint16_t blud_prec[3];
            if (blud_on < 0) blud_on = getenv("REJEU_BLUD") != NULL;
            if (blud_on) {
                static const unsigned off[3] = { NDB_A_CU, NDB_A_FU, NDB_A_DU_1 };
                static const char *nom[3] = { "a_cu", "a_fu", "a_du" };
                for (int i = 0; i < 3; i++) {
                    uint16_t v = api[(API_NDB + off[i]) / 2] & B_BLUD;
                    if (v != blud_prec[i]) { printf("tick=%u BLUD %s %s (fin de tick)\n", t->tick, nom[i], v ?
                     ↪  "pose" : "EFFACE"); blud_prec[i] = v; }
                }
            }
        }
        {   static int dit;
            if (!dsp->idle && dit < 5) { dit++; printf("tick=%u : DSP PAS A L'IDLE en fin de tick, pc=%04x
             ↪  sp=%04x\n", t->tick, dsp->pc & 0xffff, dsp->sp); } }
        {   /* REJEU_DUMP=t1-t2,dir: DSP data memory at the end of the tick */
            const char *e = getenv("REJEU_DUMP");
            unsigned t1, t2; char dos[512];
            if (e && sscanf(e, "%u-%u,%511s", &t1, &t2, dos) == 3 && t->tick >= t1 && t->tick <= t2) {
                char nm[600]; snprintf(nm, sizeof nm, "%s/mem_%u.bin", dos, t->tick);
                FILE *fm = fopen(nm, "wb"); if (fm) { fwrite(dsp->data, 2, C54X_DATA_SIZE, fm); fclose(fm); }
            }
        }
        if (getenv("REJEU_SONDE") && dernier_livre != 0xffffffffu && dernier_livre % 104 == 12) {
            printf("tick=%u fn=%u (4e burst SACCH TS2) : a_cd0=%04x %s err=%u\n", t->tick, dernier_livre, cd[0],
                   (cd[0] & 0x8000) ? ((cd[0] & 0x40) ? "FIRE KO" : "ok") : "pas de BLUD", cd[2]);
            sonde_bloc(dernier_livre);
        }
        if (cd[0] != prec_cd && (cd[0] & 0x8000)) {
            bool ko = cd[0] & (1u << 6);
            ko ? sacch_ko++ : sacch_ok++;
            printf("tick=%u SACCH a_cd0=%04x %s err=%u L2=%02x %02x %02x %02x\n", t->tick, cd[0],
                   ko ? "FIRE KO" : "ok", cd[2], cd[3] & 0xff, cd[3] >> 8, cd[4] & 0xff, cd[4] >> 8);

        }
        if (fd[0] != prec_fd && (fd[0] & 0x8000))
            printf("tick=%u FACCH a_fd0=%04x %s L2=%02x %02x %02x\n", t->tick, fd[0],
                   (fd[0] & (1u << 6)) ? "FIRE KO" : "ok", fd[3] & 0xff, fd[3] >> 8, fd[4] & 0xff);
        /* a_dd_0: B_BLUD (bit 15), B_BFI (bit 2), word 2 = errors reported by
         * the ROM. One line per speech frame (REJEU_PAROLE=1) and a summary;
         * used to study the B_BFI outside the bench. */
        if (dd[0] != prec_dd && (dd[0] & 0x8000)) {
            parole_n++; if (dd[0] & 0x4) parole_bfi++; parole_err += dd[2];
            if (getenv("REJEU_PAROLE"))
```

```
                printf("tick=%u PAROLE a_dd0=%04x BFI=%d err=%u\n", t->tick, dd[0], (dd[0] & 0x4) ? 1 : 0, dd[2]);
        }
        prec_cd = cd[0]; prec_fd = fd[0]; prec_dd = dd[0];
    }
    printf("SACCH : %d bonnes, %d Fire KO\n", sacch_ok, sacch_ko);
    printf("TX (script TSP) : %lu bursts, RACH %lu, NB %lu, inconnus %lu\n", tx_n, tx_rach, tx_nb, tx_inc);
    printf("TX par fn%%26 (a_a5fn) :"); for (int i = 0; i < 26; i++) printf(" %d:%lu", i, tx_hist26[i]);
 ↪  printf("\n");
    if (parole_n) printf("PAROLE : %lu trames, %lu BFI, %.1f erreurs/trame\n", parole_n, parole_bfi,
     ↪  (double)parole_err / parole_n);
    return 0;
}
```

3.41 /opt/GSM/c54x_exe/tools/sacch_tf_decode.c

4054 octets, 109 lignes → 109 lignes

```
/* sacch_tf_decode.c - decodes outside the DSP the SACCH/TF played by the BSP.
 *
 * Reads /dev/shm/calypso_sacch_tf.bin (calypso_bsp.c, probe [sacch_tf]:
 * 156-byte records = tick BE32, fn BE32, 148 bits 0/1), groups the bursts
 * into blocks per 45.002 (TCH/F, block of the TN at fn%104 = 12 + 26*TN/2
 * modulo 104, then +26, +52, +78) and feeds them to gsm0503_xcch_decode, in
 * clear and deciphered with the Kc of /dev/shm/calypso_kc_l1.
 *
 * If the SACCH decodes here and not in the ROM, the fault is in the
 * BSP -> ROM path; if it does not decode here either, in what the BSP gets.
 *
 * ./sacch_tf_decode [file] [TN=2] [Kc in hex, A5/1; default: calypso_kc_l1]
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <osmocom/core/bits.h>
#include <osmocom/gsm/a5.h>
#include <osmocom/coding/gsm0503_coding.h>

#define MAXB 1024
static struct { uint32_t tick, fn; uint8_t b[148]; } r[MAXB];

static int kc_lire(uint8_t *algo, uint8_t kc[8])
{
    FILE *f = fopen("/dev/shm/calypso_kc_l1", "rb");
    uint8_t b[32];
    if (!f) return -1;
    size_t n = fread(b, 1, sizeof b, f);
    fclose(f);
    if (n < 14) return -1;
    *algo = b[4];
    memcpy(kc, b + 6, 8);
    return (*algo >= 1 && *algo <= 3) ? 0 : -1;
}

static int trouver(int n, uint32_t fn)
{
    for (int i = 0; i < n; i++) if (r[i].fn == fn) return i;
    return -1;
}

int main(int argc, char **argv)
{
    const char *nom = argc > 1 ? argv[1] : "/dev/shm/calypso_sacch_tf.bin";
    int tn = argc > 2 ? atoi(argv[2]) : 2;
```

```
    FILE *f = fopen(nom, "rb");
    if (!f) { perror(nom); return 1; }
    int n = 0;
    uint8_t rec[156];
    while (n < MAXB && fread(rec, 1, 156, f) == 156) {
        r[n].tick = (uint32_t)rec[0] << 24 | rec[1] << 16 | rec[2] << 8 | rec[3];
        r[n].fn = (uint32_t)rec[4] << 24 | rec[5] << 16 | rec[6] << 8 | rec[7];
        memcpy(r[n].b, rec + 8, 148);
        n++;
    }
    fclose(f);
    uint8_t algo = 0, kc[8] = { 0 };
    int a_kc = kc_lire(&algo, kc) == 0;
    if (argc > 3 && strlen(argv[3]) == 16) {
        for (int i = 0; i < 8; i++) sscanf(argv[3] + 2 * i, "%2hhx", &kc[i]);
        algo = 1; a_kc = 1;
    }
    printf("%d bursts, Kc %s", n, a_kc ? "" : "absent");
    if (a_kc) printf("A5/%u %02x%02x%02x%02x%02x%02x%02x%02x", algo, kc[0], kc[1], kc[2], kc[3], kc[4], kc[5],
     ↪  kc[6], kc[7]);
    printf("\n");

    unsigned debut = (12 + 26 * (tn / 2)) % 104;   /* TN2 : 38 */
    int ok[2] = { 0, 0 }, vus = 0;
    for (int i = 0; i < n; i++) {
        if (r[i].fn % 104 != debut) continue;
        int idx[4];
        int complet = 1;
        for (int k = 0; k < 4; k++) {
            idx[k] = trouver(n, r[i].fn + 26 * k);
            if (idx[k] < 0) complet = 0;
        }
        if (!complet) continue;
        vus++;
        printf("bloc fn=%u tick=%u :", r[i].fn, r[i].tick);
        for (int chiffre = 0; chiffre < 2; chiffre++) {
            if (chiffre && !a_kc) break;
            sbit_t sb[464];
            for (int k = 0; k < 4; k++) {
                uint8_t b[148];
                memcpy(b, r[idx[k]].b, 148);
                if (chiffre) {
                    ubit_t dl[114], ul[114];
                    osmo_a5(algo, kc, r[idx[k]].fn, dl, ul);
                    for (int j = 0; j < 57; j++) { b[3 + j] ^= dl[j]; b[88 + j] ^= dl[57 + j]; }
                }
                for (int j = 0; j < 58; j++) {
                    sb[k * 116 + j] = b[3 + j] ? -127 : 127;
                    sb[k * 116 + 58 + j] = b[87 + j] ? -127 : 127;
                }
            }
            uint8_t l2[23];
            int nerr = 0, nbits = 0;
            int rc = gsm0503_xcch_decode(l2, sb, &nerr, &nbits);
            if (rc == 0) ok[chiffre]++;
            printf("  %s rc=%d err=%d/%d L2=%02x %02x %02x %02x %02x %02x", chiffre ? "dechiffre" : "tel-quel",
                   rc, nerr, nbits, l2[0], l2[1], l2[2], l2[3], l2[4], l2[5]);
        }
        printf("\n");
    }
    printf("blocs complets=%d  bons tel-quel=%d  bons dechiffres=%d\n", vus, ok[0], ok[1]);
    return 0;
}
```

3.42 /opt/GSM/c54x_exe/tools/cch_ref/cch_interleave_ref.py

7736 octets, 171 lignes → 170 lignes (1 groupes compactés)

```
#!/usr/bin/env python3
# ====================================================================
# GSM 05.03 xCCH block interleaving — REFERENCE
# Applies to BCCH / CCCH (PCH/AGCH) / SDCCH / SACCH.
# One 456-bit coded block -> 4 normal bursts of 114 data bits each,
# NO cross-block overlap (block-rectangular, unlike the 8-burst TCH).
#
# Byte-for-byte identical to libosmocore:
#     gsm0503_xcch_interleave() / gsm0503_xcch_deinterleave()
#         B = k & 3
#         j = 2*((49*k) % 57) + ((k % 8) >> 2)
#         iB[B*114 + j] = cB[k]        (deinterleave: cB[k] = iB[B*114+j])
#
# Burst data layout for j:
#     j =  0..56  -> first  57-bit half  (before the 26-bit midamble)
#     j = 57..113 -> second 57-bit half  (after  the midamble)
#   The 2 stealing flags are NOT part of these 114 bits.
# ====================================================================
import csv
import random
from collections import Counter

N = 456

def fwd_map():
    """k -> (k, B, j, iB_index)."""
    rows = []
    for k in range(N):
        B = k & 3                                   # burst 0..3  (k mod 4)
        j = 2 * ((49 * k) % 57) + ((k % 8) >> 2)    # position 0..113
        rows.append((k, B, j, B * 114 + j))
    return rows

# ---- verify the map is a real interleaver (bijection + full coverage) ----
def verify(rows):
    idx = [r[3] for r in rows]
    assert sorted(idx) == list(range(N)), "iB index is NOT a bijection"
    pairs = Counter((r[1], r[2]) for r in rows)
    assert len(pairs) == N and all(v == 1 for v in pairs.values()), "duplicate (B,j)"
    for B in range(4):
        js = sorted(r[2] for r in rows if r[1] == B)
        assert js == list(range(114)), f"burst {B}: j not full 0..113"
    return True

# ---- rate-1/2 K=5 convolutional code (GSM 05.03 4.1.3) --------------------
# G0 = 1 + D^3 + D^4   (0b10011 = 0x13)  -> even coded index  c(2k)
# G1 = 1 + D + D^3 + D^4 (0b11011 = 0x1B) -> odd  coded index  c(2k+1)
def conv_encode(u):
    assert len(u) == 228                # 184 info + 40 Fire parity + 4 tail(=0)
    U = lambda i: u[i] if 0 <= i < 228 else 0
    c = [0] * N
    for k in range(228):
        c[2*k]   = U(k) ^ U(k-3) ^ U(k-4)
        c[2*k+1] = U(k) ^ U(k-1) ^ U(k-3) ^ U(k-4)
    return c

def interleave(c):
    iB = [0] * N
    for (k, B, j, idx) in fwd_map():
        iB[idx] = c[k]
    return iB
```

```
def deinterleave(iB):
    c = [0] * N
    for (k, B, j, idx) in fwd_map():
        c[k] = iB[idx]
    return c

# ============================================================================
if __name__ == "__main__":
    rows = fwd_map()
    verify(rows)

    # round-trip proof: u -> encode -> interleave -> deinterleave == c
    random.seed(0)
    u = [random.randint(0, 1) for _ in range(224)] + [0, 0, 0, 0]
    c = conv_encode(u)
    assert deinterleave(interleave(c)) == c
    print("OK  interleaver is a bijection over 0..455")
    print("OK  per-burst j fully covers 0..113")
    print("OK  conv-encode -> interleave -> deinterleave round-trip exact\n")

    # ---- main table: k -> (burst, pos), plus the two bug variants ----
    with open("cch_interleave_ref.csv", "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["k", "lane_G(0=G0/c0,1=G1/c1)", "burst_B", "pos_j",
                    "iB_index", "j_if_LSB_dropped", "j_if_LSB_inverted",
                    "iB_index_if_LSB_inverted"])
        for (k, B, j, idx) in rows:
            base = 2 * ((49 * k) % 57)
            j_drop = base
            j_inv = base + (1 - ((k % 8) >> 2))
            w.writerow([k, k & 1, B, j, idx, j_drop, j_inv, B * 114 + j_inv])

    # ---- inverse table: what a deinterleaver actually indexes ----
    inv = [0] * N
    for (k, B, j, idx) in rows:
        inv[idx] = k
    with open("cch_deinterleave_inverse_ref.csv", "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["iB_index", "burst_B", "pos_j", "source_coded_k", "lane_G"])
        for idx in range(N):
            k = inv[idx]
            w.writerow([idx, idx // 114, idx % 114, k, k & 1])

    # ---- deinterleave with a deliberate bug, for stage isolation ----
    def deinterleave_mode(iB, mode):
        c = [0] * N
        for k in range(N):
            B = k & 3
            base = 2 * ((49 * k) % 57)
            if mode == "correct":
                j = base + ((k % 8) >> 2)
            elif mode == "lsb_dropped":     # ((k%8)>>2) forced to 0
                j = base
            elif mode == "lsb_inverted":    # 0<->1 on the LSB term
                j = base + (1 - ((k % 8) >> 2))
            c[k] = iB[B * 114 + j]
        return c

    # Known test vector. u is pseudo-random (dense in 1s) on purpose:
    # any interleaver error then shows up massively at the cB level.
    # Fire is NOT needed here - this isolates interleaver + conv-decoder only.
    random.seed(1)
    u_tv = [random.randint(0, 1) for _ in range(224)] + [0, 0, 0, 0]
    c_tv = conv_encode(u_tv)                 # 456 coded bits, correct order
    iB_tv = interleave(c_tv)                 # 4 x 114, from the CORRECT interleaver
    cB_correct = deinterleave_mode(iB_tv, "correct")      # == c_tv (Viterbi(cB)->u)
```

```
    cB_drop    = deinterleave_mode(iB_tv, "lsb_dropped")
    cB_inv     = deinterleave_mode(iB_tv, "lsb_inverted")
    assert cB_correct == c_tv

    s = lambda bits: "".join(map(str, bits))
    with open("cch_testvectors.txt", "w") as f:
        f.write("# GSM 05.03 xCCH stage-isolation test vector\n")
        f.write("# Fixed pseudo-random 228-bit u (u[224:228]=tail=0).\n")
        f.write("# TEST A (interleaver): feed burst0..3 into YOUR deinterleaver,\n")
        f.write("#   compare the 456-bit result against the three candidates below:\n")
        f.write("#     == cB_correct       -> interleaver OK, bug is downstream\n")
        f.write("#     == cB_lsb_dropped   -> you never apply the ((k%8)>>2) term\n")
        f.write("#     == cB_lsb_inverted  -> that term is inverted / off-by-one\n")
        f.write("#     == none of them     -> different bug (burst order? half swap?)\n")
        f.write("# TEST B (Viterbi/lane): feed cB_correct into YOUR Viterbi,\n")
        f.write("#   compare the 228-bit result against u. Mismatch with cB correct\n")
        f.write("#   => c0/c1 swap, wrong polynomials, or tail handling.\n\n")
        f.write("u_228           = " + s(u_tv) + "\n\n")
        f.write("burst⟨1⟩_114      = " + s(iB_tv[⟨2⟩:⟨3⟩]) + "\n")  ×3
    ⟨⟩ = (0,0,114) (1,114,228) (2,228,342)
        f.write("burst3_114      = " + s(iB_tv[342:456]) + "\n\n")
        f.write("cB_correct      = " + s(cB_correct) + "\n\n")
        f.write("cB_lsb_dropped  = " + s(cB_drop) + "\n\n")
        f.write("cB_lsb_inverted = " + s(cB_inv) + "\n")

    dd = sum(a != b for a, b in zip(cB_correct, cB_drop))
    di = sum(a != b for a, b in zip(cB_correct, cB_inv))
    print(f"test vector written: cB differs from correct in "
          f"{dd}/456 (dropped) and {di}/456 (inverted) positions\n")

    # ---- how big / what shape is the LSB bug ----
    diff = [k for (k, B, j, idx) in rows
            if (B * 114 + j) != (B * 114 + 2 * ((49 * k) % 57) + (1 - ((k % 8) >> 2)))]
    print(f"LSB-inverted bug moves {len(diff)}/456 coded bits to a wrong burst slot")
    print("  -> every k is displaced by exactly +/-1 in j (even<->odd swap),")
    print("     within its own burst, so the burst assignment (k&3) stays right.\n")

    print("spot check (k : burst, pos):")
    for k in [0, 1, 2, 3, 4, 5, 6, 7, 8, 57, 114, 228, 455]:
        B = k & 3
        j = 2 * ((49 * k) % 57) + ((k % 8) >> 2)
        print(f"  k={k:3d} -> burst {B}, pos {j:3d}   (lane {'G1/c1' if k & 1 else 'G0/c0'})")
```

4 Logs

4.1 /opt/GSM/c54x_exe/toa.log

7679 octets, 198 lignes → 191 lignes (4 groupes compactés)

```
```
magic 23 reproduction — 2026-10-04T16:50Z — Linux 6.8.0-146-generic x86_64

1. ROM dump -> PROM0.bin
$ sha256sum dsp-rom-3606-dump.txt
e35798f50fd0a29aa61cd53e417c010d7f172582c7bc055211daab8af1e05470 dsp-rom-3606-dump.txt

PROM0.bin: 28672 words (0x7000-0xdfff)
  7cbd: f010 2be4
  84a4: 7713 2a27
  b21d: 7713 00bf
  b2b1: 7713 0097
  b32d: 7713 0030
$ sha256sum PROM0.bin
```
```

```
75ce5375a07eac85ee42ba00af7ca82089d15b5df517710a756e6c26497021d7  PROM0.bin


## 2. objdump tic54x (binutils 2.21.1)
$ build-tic54x/binutils/objdump --version
GNU objdump (GNU Binutils) 2.21.1
Copyright 2011 Free Software Foundation, Inc.
This program is free software; you may redistribute it under the terms of
the GNU General Public License version 3 or (at your option) any later version.
This program has absolutely no warranty.

full disassembly: PROM0.dis (28665 lines)

### expected: 7cbd sub #11236,a | 84a4 stm #10791,ar3 | b21d stm #191,ar3 | b2b1 stm #151,ar3 | b32d stm #48,ar3
    7cbd:   f010    sub    #11236,a
    84a4:   7713    stm    #10791,ar3
    b21d:   7713    stm    #191,ar3
    b2b1:   7713    stm    #151,ar3
    b32d:   7713    stm    #48,ar3

### SB TOA chain
    7c3d:   7712    stm    #3278,ar2
    81c8:   f272    rptbd  0x81db
    7c49:   7712    stm    #11498,ar2
    7a1c:   fe00    retd
    84a1:   f7b6    ssbx   st1,frct
    84ca:   f6b6    rsbx   st1,frct
    84dd:   e900    ld     #0,b
    7c5f:   7210    mvdm   0x2f06,ar0
    7ca6:   f4bc    rsbx   st0,tc
    7cbc:   4817    ldm    ar7,a
    7e8a:   711f    mvdk   DP+0x1f,0x3fa4
    b1e7:   10f8    ld     *(0x3fa4),a

### FB TOA: 0x793e-0x795a
    793e:   1194    ld     *ar4+,b
    793f:   f762    sfta   b,2
    7940:   44f8    ld     *(0x3fb4),16,a
    7941:   3fb4
    7942:   f061    sub    #3,16,a
    7943:   0003
    7944:   3c84    add    *ar4,16,a
    7945:   f061    sub    #2,16,a
    7946:   0002
    7947:   f48c    mpya   a
    7948:   f500    add    a,b
    7949:   f310    sub    #5,b
    794a:   0005
    794b:   6ff8    sub    *(0xc3d),b,a
    794c:   0c3d
    794d:   0e20
    794e:   fc47    rc     aleq
    794f:   44f8    ld     *(0x3fb3),16,a
    7950:   3fb3
    7951:   f061    sub    #3,16,a
    7952:   0003
    7953:   770e    stm    #48,t
    7954:   0030
    7955:   f48c    mpya   a
    7956:   08f8    sub    *(0xc3e),a
    7957:   0c3e
    7958:   f620    sub    b,a
    7959:   fc47    rc     aleq
    795a:   81f8    stl    b,*(0x8fa)

### DMA programming routine: 0xa5cd-0xa5f6
```

```
    a5cd:   ea86    ld     #134,dp
    a5ce:   76f8    st     #-3980,*(0x158)
    a5cf:   0158
    a5d0:   f074
    a5d1:   76f8    st     #29250,*(0x159)
    a5d2:   0159
    a5d3:   7242
    a5d4:   76f8    st     #-3981,*(0x15a)
    a5d5:   015a
    a5d6:   f073
    a5d7:   4811    ldm    ar1,a
    a5d8:   80f8    stl    a,*(0x15b)
    a5d9:   015b
    a5da:   69f8    orm    #4,*(0x3fdc)
    a5db:   3fdc
    a5dc:   0004
    a5dd:   75f8    portw  *(0x3fdc),pa63744
    a5de:   3fdc
    a5df:   f900
    a5e0:   47f8    rpt    *(0xc32)
    a5e1:   0c32
    a5e2:   f495    nop
    a5e3:   4812    ldm    ar2,a
    a5e4:   f010    sub    #2048,a
    a5e5:   0800
    a5e6:   6f56    stl    a,1,DP+0x56
    a5e7:   0c81
    a5e8:   7556    portw  DP+0x56,pa64548
    a5e9:   fc24
    a5ea:   4813    ldm    ar3,a
    a5eb:   f110    sub    #48,a,b
    a5ec:   0030
    a5ed:   6f56    stl    a,2,DP+0x56
    a5ee:   0c82
    a5ef:   7556    portw  DP+0x56,pa64550
    a5f0:   fc26
    a5f1:   7656    st     #1197,DP+0x56
    a5f2:   04ad
    a5f3:   ff4d    xc     2,beq
    a5f4:   7656    st     #1193,DP+0x56
    a5f5:   04a9
    a5f6:   7556    portw  DP+0x56,pa64552
```

```
## 3. layer1 git history
$ git clone -q https://gitea.osmocom.org/phone-side/osmocom-bb

$ git -C osmocom-bb log --format=%h %ad %an %s --date=short -Stoa -= 23 -- src/target/firmware/layer1
8978ec57 2010-05-19 Harald Welte layer1/l1ctl: Split L1CTL_NEW_CCCH_REQ in FBSB_REQ nad SYNC_REQ
5f751d3d 2010-04-09 Harald Welte layer1: move FB and SB detection into prim_fbsb.c
fbe7b94c 2010-02-18 Harald Welte Initial import of OsmocomBB into git repository

$ git -C osmocom-bb log --format=%h %ad %an %s --date=short -Smagic 23 -- src/target/firmware/layer1
8978ec57 2010-05-19 Harald Welte layer1/l1ctl: Split L1CTL_NEW_CCCH_REQ in FBSB_REQ nad SYNC_REQ
5f751d3d 2010-04-09 Harald Welte layer1: move FB and SB detection into prim_fbsb.c
48dfd41e 2010-04-07 Harald Welte layer1: add 'struct mon_state' argument to read_{fb,sb}_result()

### cb71b972: toa -= 23 / fbinfo2cellinfo lines (expect added fbinfo2cellinfo, no removed toa -= 23)
20:@@ -87,7 +87,7 @@ static void dump_mon_state(struct mon_state *fb)
29:@@ -98,14 +98,16 @@ static int l1ctl_fbsb_resp(uint8_t res)
⑴:@@ -⑵,⑶ +⑷,⑸ @@ static int l1s_sbdet_resp(__unused uint8_t p1, uint8_t attempt,  ×3
    ⟨⟩ = (48,179,25,181,24) (80,225,25,226,14) (108,259,18,249,13)
134:@@ -279,6 +264,9 @@ static int l1s_sbdet_cmd(__unused uint8_t p1, __unused uint8_t p2,
144:@@ -330,6 +318,41 @@ static int read_fb_result(struct mon_state *st, int attempt)
148:+static void fbinfo2cellinfo(struct l1_cell_info *cinfo,
154:+   last_fb->toa -= 23;
186:@@ -354,6 +377,8 @@ static int l1s_fbdet_cmd(__unused uint8_t p1, __unused uint8_t p2,
```

```
⟨1⟩:@@ -⟨2⟩,⟨3⟩ +⟨4⟩,⟨5⟩ @@ static int l1s_fbdet_resp(__unused uint8_t p1, uint8_t attempt,  ×5
    ⟨⟩ = (195,363,16,388,7) (213,385,8,401,22) (236,398,7,428,8) (246,408,6,439,7) (254,423,8,455,15)
267:+               fbinfo2cellinfo(&l1s.serving_cell, last_fb);
271:@@ -458,8 +497,6 @@ static const struct tdma_sched_item fb_sched_set[] = {
280:@@ -467,35 +504,10 @@ static void l1a_fb_compl(__unused enum l1_compl c)
285:-   last_fb->toa -= 23;
313:+   fbinfo2cellinfo(&l1s.serving_cell, last_fb);
319:@@ -513,6 +525,9 @@ void l1s_fbsb_req(uint8_t base_fn, struct l1ctl_fbsb_req *req)

### current constants
38:#define L1_NB_MARGIN_Q       (3 * 4)
39:#define L1_SB_MARGIN_Q       (23 * 4)
40:#define L1_TAIL_DURATION_Q   (3 * 4)
43:#define L1_NB_DURATION_Q (L1_BURST_LENGTH_Q + 2 * L1_NB_MARGIN_Q - L1_TAIL_DURATION_Q)
44:#define L1_SB_DURATION_Q (L1_BURST_LENGTH_Q + 2 * L1_SB_MARGIN_Q - L1_TAIL_DURATION_Q)
54: [L1_RXWIN_SB]   = L1_SB_DURATION_Q,
55: [L1_RXWIN_NB]   = L1_NB_DURATION_Q,
206:    last_fb->toa -= 23;
333:    /* FIXME: where did this magic 23 come from? */
334:    last_fb->toa -= 23;
469:    /* FIXME: where did this magic 23 come from? */
470:    last_fb->toa -= 23;
119:     * is within 0...8. We have to add 75 to get an SB TOA of 4. */
120:    tpu_shift += 75;

## 4. hardware logs: qbits check ((TOA-46) mod 1250)*4
sources: http://lists.osmocom.org/pipermail/baseband-devel/2011-September/002526.html
         https://www.mail-archive.com/baseband-devel@lists.osmocom.org/msg01046.html
  TOA= ⟨1⟩ printed=⟨2⟩  -46:⟨3⟩ OK   -23:⟨4⟩  ×3
    ⟨⟩ = (9651,3420,3420,3512) (8751,4820,4820,4912) (8755,4836,4836,4928)
  TOA=10003 printed=4828  -46:4828 OK   -23:4920
  TOA=10007 printed=4844  -46:4844 OK   -23:4936
  TOA= 8755 printed=4836  -46:4836 OK   -23:4928
  TOA=   ⟨1⟩ printed=⟨2⟩  -46:⟨3⟩ OK   -23:  ⟨4⟩  ×4
    ⟨⟩ = (29,4932,4932,24) (29,4932,4932,24) (27,4924,4924,16) (27,4924,4924,16)
  TOA=   24 printed=4912  -46:4912 OK   -23:   4
  TOA=   26 printed=4920  -46:4920 OK   -23:  12
  12/12 lines match the -46 rule
  FB mode 0 multiples of 48: 7/7
  FB mode 1 == 3 mod 4:      10/10
  first SB after FB sync: [29, 29, 27, 27, 24, 26]  mean delta -0.25
  148 + 2*23 - 3 = 191 ; 148 + 2*3 - 3 = 151 ; (191-148+3)/2 = 23

# done — 195 lines in toa.log
```